Proprietary Notice

This document is protected by copyright and other related rights and the use or implementation of the information contained in this document may be protected by one or more patents or pending patent applications. No part of this document may be reproduced in any form by any means without the express prior written permission of Arm Limited (“Arm”). **No license, express or implied, by estoppel or otherwise to any intellectual property rights is granted by this document unless specifically stated.**

Your access to the information in this document is conditional upon your acceptance that you will not use or permit others to use the information for the purposes of determining whether the subject matter of this document infringes any third party patents.

The content of this document is informational only. Any solutions presented herein are subject to changing conditions, information, scope, and data. This document was produced using reasonable efforts based on information available as of the date of issue of this document. The scope of information in this document may exceed that which Arm is required to provide, and such additional information is merely intended to further assist the recipient and does not represent Arm's view of the scope of its obligations. You acknowledge and agree that you possess the necessary expertise in system security and functional safety and that you shall be solely responsible for compliance with all legal, regulatory, safety and security related requirements concerning your products, notwithstanding any information or support that may be provided by Arm herein. In addition, you are responsible for any applications which are used in conjunction with any Arm technology described in this document, and to minimize risks, adequate design and operating safeguards should be provided for by you.

This document may include technical inaccuracies or typographical errors. THIS DOCUMENT IS PROVIDED “AS IS”. ARM PROVIDES NO REPRESENTATIONS AND NO WARRANTIES, EXPRESS, IMPLIED OR STATUTORY, INCLUDING, WITHOUT LIMITATION, THE IMPLIED WARRANTIES OF MERCHANTABILITY, SATISFACTORY QUALITY, NON-INFRINGEMENT OR FITNESS FOR A PARTICULAR PURPOSE WITH RESPECT TO THE DOCUMENT. For the avoidance of doubt, Arm makes no representation with respect to, and has undertaken no analysis to identify or understand the scope and content of, any patents, copyrights, trade secrets, trademarks, or other rights.

TO THE EXTENT NOT PROHIBITED BY LAW, IN NO EVENT WILL ARM BE LIABLE FOR ANY DAMAGES, INCLUDING WITHOUT LIMITATION ANY DIRECT, INDIRECT, SPECIAL, INCIDENTAL, PUNITIVE, OR CONSEQUENTIAL DAMAGES, HOWEVER CAUSED AND REGARDLESS OF THE THEORY OF LIABILITY, ARISING OUT OF ANY USE OF THIS DOCUMENT, EVEN IF ARM HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

Reference by Arm to any third party’s products or services within this document is not an express or implied approval or endorsement of the use thereof.

This document consists solely of commercial items. You shall be responsible for ensuring that any permitted use, duplication, or disclosure of this document complies fully with any relevant export laws and regulations to assure that this document or any portion thereof is not exported, directly or indirectly, in violation of such export laws. Use of the word “partner” in reference to Arm’s customers is not intended to create or refer to any partnership relationship with any other company. Arm may make changes to this document at any time and without notice.

This document may be translated into other languages for convenience, and you agree that if there is any conflict between the English version of this document and any translation, the terms of the English version of the Agreement shall prevail.

The validity, construction and performance of this notice shall be governed by English Law.

The Arm corporate logo and words marked with ® or ™ are registered trademarks or trademarks of Arm Limited (or its affiliates) in the US and/or elsewhere. Please follow Arm’s trademark usage guidelines at http://www.arm.com/company/policies/trademarks. All rights reserved. Other brands and names mentioned in this document may be the trademarks of their respective owners.

Copyright © 2010-2026 Arm Limited (or its affiliates). All rights reserved.

Arm Limited. Company 02557590 registered in England.

110 Fulbourn Road, Cambridge, England CB1 9NJ.

LES-PRE-20349

8 March 2024

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

A64 -- Base Instructions (alphabetic order)

ABS: Absolute value.

ADC: Add with carry.

ADCS: Add with carry, setting flags.

ADD (extended register): Add extended and scaled register.

ADD (immediate): Add immediate value.

ADD (shifted register): Add optionally-shifted register.

ADDG: Add with tag.

ADDPT: Add checked pointer.

ADDS (extended register): Add extended and scaled register, setting flags.

ADDS (immediate): Add immediate value, setting flags.

ADDS (shifted register): Add optionally-shifted register, setting flags.

ADR: Form PC-relative address.

ADRP: Form PC-relative address to 4KB page.

AND (immediate): Bitwise AND (immediate).

AND (shifted register): Bitwise AND (shifted register).

ANDS (immediate): Bitwise AND (immediate), setting flags.

ANDS (shifted register): Bitwise AND (shifted register), setting flags.

APAS: Associate physical address space: an alias of SYS.

ASR (immediate): Arithmetic shift right (immediate): an alias of SBFM.

ASR (register): Arithmetic shift right (register): an alias of ASRV.

ASRV: Arithmetic shift right variable.

AT: Address translate: an alias of SYS.

AUTDA, AUTDZA: Authenticate data address, using key A.

AUTDB, AUTDZB: Authenticate data address, using key B.

AUTIA, AUTIA1716, AUTIASP, AUTIAZ, AUTIZA: Authenticate instruction address, using key A.

AUTIA171615: Authenticate instruction address, using key A.

AUTIASPPC: Authenticate return address using key A, using an immediate offset.

AUTIASPPCR: Authenticate return address using key A, using a register.

AUTIB, AUTIB1716, AUTIBSP, AUTIBZ, AUTIZB: Authenticate instruction address, using key B.

AUTIB171615: Authenticate instruction address, using key B.

AUTIBSPPC: Authenticate return address using key B, using an immediate offset.

AUTIBSPPCR: Authenticate return address using key B, using a register.

AXFLAG: Convert floating-point condition flags from Arm to external format.

B: Branch.

B.cond: Branch conditionally.

BC.cond: Branch consistent conditionally.

BFC: Bitfield clear: an alias of BFM.

BFI: Bitfield insert: an alias of BFM.

BFM: Bitfield move.

BFXIL: Bitfield extract and insert at low end: an alias of BFM.

BIC (shifted register): Bitwise bit clear (shifted register).

BICS (shifted register): Bitwise bit clear (shifted register), setting flags.

BL: Branch with link.

BLR: Branch with link to register.

BLRAA, BLRAAZ, BLRAB, BLRABZ: Branch with link to register, with pointer authentication.

BR: Branch to register.

BRAA, BRAAZ, BRAB, BRABZ: Branch to register, with pointer authentication.

BRB: Branch record buffer: an alias of SYS.

BRK: Breakpoint instruction.

BTI: Branch target identification.

CAS, CASA, CASAL, CASL: Compare and swap word or doubleword in memory.

CASB, CASAB, CASALB, CASLB: Compare and swap byte in memory.

CASH, CASAH, CASALH, CASLH: Compare and swap halfword in memory.

CASP, CASPA, CASPAL, CASPL: Compare and swap pair of words or doublewords in memory.

CASPT, CASPAT, CASPALT, CASPLT: Compare and swap pair unprivileged.

CAST, CASAT, CASALT, CASLT: Compare and swap unprivileged.

CB<cc> (immediate): Compare register with immediate and branch.

CB<cc> (register): Compare registers and branch.

CBB<cc>: Compare bytes and branch.

CBBLE: Compare signed less than or equal to bytes and branch: an alias of CBB<cc>.

CBBLO: Compare unsigned lower than bytes and branch: an alias of CBB<cc>.

CBBLS: Compare unsigned lower than or same as bytes and branch: an alias of CBB<cc>.

CBBLT: Compare signed less than bytes and branch: an alias of CBB<cc>.

CBGE (immediate): Compare signed greater than or equal to immediate and branch: an alias of CB<cc> (immediate).

CBH<cc>: Compare halfwords and branch.

CBHLE: Compare signed less than or equal to halfwords and branch: an alias of CBH<cc>.

CBHLO: Compare unsigned lower than halfwords and branch: an alias of CBH<cc>.

CBHLS: Compare unsigned lower than or same as halfwords and branch: an alias of CBH<cc>.

CBHLT: Compare signed less than halfwords and branch: an alias of CBH<cc>.

CBHS (immediate): Compare unsigned higher than or same as immediate and branch: an alias of CB<cc> (immediate).

CBLE (immediate): Compare signed less than or equal to immediate and branch: an alias of CB<cc> (immediate).

CBLE (register): Compare signed less than or equal to register and branch: an alias of CB<cc> (register).

CBLO (register): Compare unsigned lower than register and branch: an alias of CB<cc> (register).

CBLS (immediate): Compare unsigned lower than or same as immediate and branch: an alias of CB<cc> (immediate).

CBLS (register): Compare unsigned lower than or same as register and branch: an alias of CB<cc> (register).

CBLT (register): Compare signed less than register and branch: an alias of CB<cc> (register).

CBNZ: Compare and branch on nonzero.

CBZ: Compare and branch on zero.

CCMN (immediate): Conditional compare negative (immediate).

CCMN (register): Conditional compare negative (register).

CCMP (immediate): Conditional compare (immediate).

CCMP (register): Conditional compare (register).

CFINV: Invert carry flag.

CFLT<cc> (immediate): Compare register with immediate and fault.

CFLT<cc> (register): Compare registers and fault.

CFLTGE (immediate): Compare signed greater than or equal to immediate and fault: an alias of CFLT<cc> (immediate).

CFLTHS (immediate): Compare unsigned higher than or same as immediate and fault: an alias of CFLT<cc> (immediate).

CFLTLE (immediate): Compare signed less than or equal to immediate and fault: an alias of CFLT<cc> (immediate).

CFLTLE (register): Compare registers and fault if less than or equal: an alias of CFLT<cc> (register).

CFLTLO (register): Compare registers and fault if lower: an alias of CFLT<cc> (register).

CFLTLS (immediate): Compare unsigned lower than or same as immediate and fault: an alias of CFLT<cc> (immediate).

CFLTLS (register): Compare registers and fault if lower than or same: an alias of CFLT<cc> (register).

CFLTLT (register): Compare registers and fault if less than: an alias of CFLT<cc> (register).

CFLTNZ: Compare and fault on nonzero: an alias of CFLT<cc> (immediate).

CFLTZ: Compare and fault on zero: an alias of CFLT<cc> (immediate).

CFP: Control flow prediction restriction by context: an alias of SYS.

CHKFEAT: Check feature status.

CINC: Conditional increment: an alias of CSINC.

CINV: Conditional invert: an alias of CSINV.

CLRBHB: Clear branch history.

CLREX: Clear exclusive.

CLS: Count leading sign bits.

CLZ: Count leading zeros.

CMN (extended register): Compare negative (extended register): an alias of ADDS (extended register).

CMN (immediate): Compare negative (immediate): an alias of ADDS (immediate).

CMN (shifted register): Compare negative (shifted register): an alias of ADDS (shifted register).

CMP (extended register): Compare (extended register): an alias of SUBS (extended register).

CMP (immediate): Compare (immediate): an alias of SUBS (immediate).

CMP (shifted register): Compare (shifted register): an alias of SUBS (shifted register).

CMPP: Compare with tag: an alias of SUBPS.

CNEG: Conditional negate: an alias of CSNEG.

CNT: Count bits.

COSP: Clear other speculative prediction restriction by context: an alias of SYS.

CPP: Cache prefetch prediction restriction by context: an alias of SYS.

CPYFP, CPYFM, CPYFE: Memory copy forward-only.

CPYFPN, CPYFMN, CPYFEN: Memory copy forward-only, reads and writes non-temporal.

CPYFPRN, CPYFMRN, CPYFERN: Memory copy forward-only, reads non-temporal.

CPYFPRT, CPYFMRT, CPYFERT: Memory copy forward-only, reads unprivileged.

CPYFPRTN, CPYFMRTN, CPYFERTN: Memory copy forward-only, reads unprivileged, reads and writes non-temporal.

CPYFPRTRN, CPYFMRTRN, CPYFERTRN: Memory copy forward-only, reads unprivileged and non-temporal.

CPYFPRTWN, CPYFMRTWN, CPYFERTWN: Memory copy forward-only, reads unprivileged, writes non-temporal.

CPYFPT, CPYFMT, CPYFET: Memory copy forward-only, reads and writes unprivileged.

CPYFPTN, CPYFMTN, CPYFETN: Memory copy forward-only, reads and writes unprivileged and non-temporal.

CPYFPTRN, CPYFMTRN, CPYFETRN: Memory copy forward-only, reads and writes unprivileged, reads non-temporal.

CPYFPTWN, CPYFMTWN, CPYFETWN: Memory copy forward-only, reads and writes unprivileged, writes non-temporal.

CPYFPWN, CPYFMWN, CPYFEWN: Memory copy forward-only, writes non-temporal.

CPYFPWT, CPYFMWT, CPYFEWT: Memory copy forward-only, writes unprivileged.

CPYFPWTN, CPYFMWTN, CPYFEWTN: Memory copy forward-only, writes unprivileged, reads and writes non-temporal.

CPYFPWTRN, CPYFMWTRN, CPYFEWTRN: Memory copy forward-only, writes unprivileged, reads non-temporal.

CPYFPWTWN, CPYFMWTWN, CPYFEWTWN: Memory copy forward-only, writes unprivileged and non-temporal.

CPYP, CPYM, CPYE: Memory copy.

CPYPN, CPYMN, CPYEN: Memory copy, reads and writes non-temporal.

CPYPRN, CPYMRN, CPYERN: Memory copy, reads non-temporal.

CPYPRT, CPYMRT, CPYERT: Memory copy, reads unprivileged.

CPYPRTN, CPYMRTN, CPYERTN: Memory copy, reads unprivileged, reads and writes non-temporal.

CPYPRTRN, CPYMRTRN, CPYERTRN: Memory copy, reads unprivileged and non-temporal.

CPYPRTWN, CPYMRTWN, CPYERTWN: Memory copy, reads unprivileged, writes non-temporal.

CPYPT, CPYMT, CPYET: Memory copy, reads and writes unprivileged.

CPYPTN, CPYMTN, CPYETN: Memory copy, reads and writes unprivileged and non-temporal.

CPYPTRN, CPYMTRN, CPYETRN: Memory copy, reads and writes unprivileged, reads non-temporal.

CPYPTWN, CPYMTWN, CPYETWN: Memory copy, reads and writes unprivileged, writes non-temporal.

CPYPWN, CPYMWN, CPYEWN: Memory copy, writes non-temporal.

CPYPWT, CPYMWT, CPYEWT: Memory copy, writes unprivileged.

CPYPWTN, CPYMWTN, CPYEWTN: Memory copy, writes unprivileged, reads and writes non-temporal.

CPYPWTRN, CPYMWTRN, CPYEWTRN: Memory copy, writes unprivileged, reads non-temporal.

CPYPWTWN, CPYMWTWN, CPYEWTWN: Memory copy, writes unprivileged and non-temporal.

CRC32B, CRC32H, CRC32W, CRC32X: CRC32 checksum.

CRC32CB, CRC32CH, CRC32CW, CRC32CX: CRC32C checksum.

CSDB: Consumption of speculative data barrier.

CSEL: Conditional select.

CSET: Conditional set: an alias of CSINC.

CSETM: Conditional set mask: an alias of CSINV.

CSINC: Conditional select increment.

CSINV: Conditional select invert.

CSNEG: Conditional select negation.

CTZ: Count trailing zeros.

DC: Data cache operation: an alias of SYS.

DCPS1: Debug change PE state to EL1.

DCPS2: Debug change PE state to EL2.

DCPS3: Debug change PE state to EL3.

DGH: Data gathering hint.

DMB: Data memory barrier.

DRPS: Debug restore PE state.

DSB: Data synchronization barrier.

DVP: Data value prediction restriction by context: an alias of SYS.

EON (shifted register): Bitwise exclusive-OR NOT (shifted register).

EOR (immediate): Bitwise exclusive-OR (immediate).

EOR (shifted register): Bitwise exclusive-OR (shifted register).

ERET: Exception return.

ERETAA, ERETAB: Exception return, with pointer authentication.

ESB: Error synchronization barrier.

EXTR: Extract register.

FLT.cond: Conditional fault.

GCSB: Guarded Control Stack barrier.

GCSPOPCX: Guarded Control Stack pop and compare exception return record: an alias of SYS.

GCSPOPM: Guarded Control Stack pop: an alias of SYSL.

GCSPOPX: Guarded Control Stack pop exception return record: an alias of SYS.

GCSPUSHM: Guarded Control Stack push: an alias of SYS.

GCSPUSHX: Guarded Control Stack push exception return record: an alias of SYS.

GCSSS1: Guarded Control Stack switch stack 1: an alias of SYS.

GCSSS2: Guarded Control Stack switch stack 2: an alias of SYSL.

GCSSTR: Guarded Control Stack store register.

GCSSTTR: Guarded Control Stack store register (unprivileged).

GIC: Generic Interrupt Controller: an alias of SYS.

GICR: Generic Interrupt Controller with result: an alias of SYSL.

GMI: Tag mask insert.

GSB: Generic Interrupt Controller synchronization barrier: an alias of SYS.

HINT: Hint instruction.

HINTE: Hint instruction (extended).

HLT: Halt instruction.

HVC: Hypervisor call.

IC: Instruction cache operation: an alias of SYS.

IRG: Insert random tag.

ISB: Instruction synchronization barrier.

LD64B: Single-copy atomic 64-byte load.

LDA64B: Single-copy atomic 64-byte load-acquire.

LDADD, LDADDA, LDADDAL, LDADDL: Atomic add on word or doubleword.

LDADDB, LDADDAB, LDADDALB, LDADDLB: Atomic add on byte.

LDADDH, LDADDAH, LDADDALH, LDADDLH: Atomic add on halfword.

LDAP: Load-acquire pair of registers.

LDAPP: Load-acquire RCpc pair of registers.

LDAPR: Load-acquire RCpc register.

LDAPRB: Load-acquire RCpc register byte.

LDAPRH: Load-acquire RCpc register halfword.

LDAPUR: Load-acquire RCpc register (unscaled).

LDAPURB: Load-acquire RCpc register byte (unscaled).

LDAPURH: Load-acquire RCpc register halfword (unscaled).

LDAPURSB: Load-acquire RCpc register signed byte (unscaled).

LDAPURSH: Load-acquire RCpc register signed halfword (unscaled).

LDAPURSW: Load-acquire RCpc register signed word (unscaled).

LDAR: Load-acquire register.

LDARB: Load-acquire register byte.

LDARH: Load-acquire register halfword.

LDATXR: Load-acquire unprivileged exclusive register.

LDAXP: Load-acquire exclusive pair of registers.

LDAXR: Load-acquire exclusive register.

LDAXRB: Load-acquire exclusive register byte.

LDAXRH: Load-acquire exclusive register halfword.

LDCLR, LDCLRA, LDCLRAL, LDCLRL: Atomic bit clear on word or doubleword.

LDCLRB, LDCLRAB, LDCLRALB, LDCLRLB: Atomic bit clear on byte.

LDCLRH, LDCLRAH, LDCLRALH, LDCLRLH: Atomic bit clear on halfword.

LDCLRP, LDCLRPA, LDCLRPAL, LDCLRPL: Atomic bit clear on quadword.

LDEOR, LDEORA, LDEORAL, LDEORL: Atomic exclusive-OR on word or doubleword.

LDEORB, LDEORAB, LDEORALB, LDEORLB: Atomic exclusive-OR on byte.

LDEORH, LDEORAH, LDEORALH, LDEORLH: Atomic exclusive-OR on halfword.

LDG: Load Allocation Tag.

LDGM: Load tag multiple.

LDIAPP: Load-Acquire RCpc ordered pair of registers.

LDLAR: Load LOAcquire register.

LDLARB: Load LOAcquire register byte.

LDLARH: Load LOAcquire register halfword.

LDNP: Load pair of registers, with non-temporal hint.

LDP: Load pair of registers.

LDPSW: Load pair of registers signed word.

LDR (immediate): Load register (immediate).

LDR (literal): Load register (literal).

LDR (register): Load register (register).

LDRAA, LDRAB: Load register, with pointer authentication.

LDRB (immediate): Load register byte (immediate).

LDRB (register): Load register byte (register).

LDRH (immediate): Load register halfword (immediate).

LDRH (register): Load register halfword (register).

LDRSB (immediate): Load register signed byte (immediate).

LDRSB (register): Load register signed byte (register).

LDRSH (immediate): Load register signed halfword (immediate).

LDRSH (register): Load register signed halfword (register).

LDRSW (immediate): Load register signed word (immediate).

LDRSW (literal): Load register signed word (literal).

LDRSW (register): Load register signed word (register).

LDSET, LDSETA, LDSETAL, LDSETL: Atomic bit set on word or doubleword.

LDSETB, LDSETAB, LDSETALB, LDSETLB: Atomic bit set on byte.

LDSETH, LDSETAH, LDSETALH, LDSETLH: Atomic bit set on halfword.

LDSETP, LDSETPA, LDSETPAL, LDSETPL: Atomic bit set on quadword.

LDSMAX, LDSMAXA, LDSMAXAL, LDSMAXL: Atomic signed maximum on word or doubleword.

LDSMAXB, LDSMAXAB, LDSMAXALB, LDSMAXLB: Atomic signed maximum on byte.

LDSMAXH, LDSMAXAH, LDSMAXALH, LDSMAXLH: Atomic signed maximum on halfword.

LDSMIN, LDSMINA, LDSMINAL, LDSMINL: Atomic signed minimum on word or doubleword.

LDSMINB, LDSMINAB, LDSMINALB, LDSMINLB: Atomic signed minimum on byte.

LDSMINH, LDSMINAH, LDSMINALH, LDSMINLH: Atomic signed minimum on halfword.

LDTADD, LDTADDA, LDTADDAL, LDTADDL: Atomic add unprivileged.

LDTCLR, LDTCLRA, LDTCLRAL, LDTCLRL: Atomic bit clear unprivileged.

LDTNP: Load unprivileged pair of registers, with non-temporal hint.

LDTP: Load unprivileged pair of registers.

LDTR: Load register (unprivileged).

LDTRB: Load register byte (unprivileged).

LDTRH: Load register halfword (unprivileged).

LDTRSB: Load register signed byte (unprivileged).

LDTRSH: Load register signed halfword (unprivileged).

LDTRSW: Load register signed word (unprivileged).

LDTSET, LDTSETA, LDTSETAL, LDTSETL: Atomic bit set unprivileged.

LDTXR: Load unprivileged exclusive register.

LDUMAX, LDUMAXA, LDUMAXAL, LDUMAXL: Atomic unsigned maximum on word or doubleword.

LDUMAXB, LDUMAXAB, LDUMAXALB, LDUMAXLB: Atomic unsigned maximum on byte.

LDUMAXH, LDUMAXAH, LDUMAXALH, LDUMAXLH: Atomic unsigned maximum on halfword.

LDUMIN, LDUMINA, LDUMINAL, LDUMINL: Atomic unsigned minimum on word or doubleword.

LDUMINB, LDUMINAB, LDUMINALB, LDUMINLB: Atomic unsigned minimum on byte.

LDUMINH, LDUMINAH, LDUMINALH, LDUMINLH: Atomic unsigned minimum on halfword.

LDUR: Load register (unscaled).

LDURB: Load register byte (unscaled).

LDURH: Load register halfword (unscaled).

LDURSB: Load register signed byte (unscaled).

LDURSH: Load register signed halfword (unscaled).

LDURSW: Load register signed word (unscaled).

LDXP: Load exclusive pair of registers.

LDXR: Load exclusive register.

LDXRB: Load exclusive register byte.

LDXRH: Load exclusive register halfword.

LSL (immediate): Logical shift left (immediate): an alias of UBFM.

LSL (register): Logical shift left (register): an alias of LSLV.

LSLV: Logical shift left variable.

LSR (immediate): Logical shift right (immediate): an alias of UBFM.

LSR (register): Logical shift right (register): an alias of LSRV.

LSRV: Logical shift right variable.

MADD: Multiply-add.

MADDPT: Multiply-add checked pointer.

MNEG: Multiply-negate: an alias of MSUB.

MOV (bitmask immediate): Move bitmask immediate value: an alias of ORR (immediate).

MOV (inverted wide immediate): Move inverted wide immediate value: an alias of MOVN.

MOV (register): Move register value: an alias of ORR (shifted register).

MOV (to/from SP): Move register value to or from SP: an alias of ADD (immediate).

MOV (wide immediate): Move wide immediate value: an alias of MOVZ.

MOVK: Move wide with keep.

MOVN: Move wide with NOT.

MOVZ: Move wide with zero.

MRRS: Move System register to two adjacent general-purpose registers.

MRS: Move System register to general-purpose register.

MSR (immediate): Move immediate value to special register.

MSR (register): Move general-purpose register to System register.

MSRR: Move two adjacent general-purpose registers to System register.

MSUB: Multiply-subtract.

MSUBPT: Multiply-subtract checked pointer.

MUL: Multiply: an alias of MADD.

MVN: Bitwise NOT: an alias of ORN (shifted register).

NEG (shifted register): Negate (shifted register): an alias of SUB (shifted register).

NEGS: Negate, setting flags: an alias of SUBS (shifted register).

NGC: Negate with carry: an alias of SBC.

NGCS: Negate with carry, setting flags: an alias of SBCS.

NOP: No operation.

ORN (shifted register): Bitwise OR NOT (shifted register).

ORR (immediate): Bitwise OR (immediate).

ORR (shifted register): Bitwise OR (shifted register).

PACDA, PACDZA: Pointer Authentication Code for data address, using key A.

PACDB, PACDZB: Pointer Authentication Code for data address, using key B.

PACGA: Pointer Authentication Code, using generic key.

PACIA, PACIA1716, PACIASP, PACIAZ, PACIZA: Pointer Authentication Code for instruction address, using key A.

PACIA171615: Pointer Authentication Code for instruction address, using key A.

PACIASPPC: Pointer Authentication Code for return address, using key A.

PACIB, PACIB1716, PACIBSP, PACIBZ, PACIZB: Pointer Authentication Code for instruction address, using key B.

PACIB171615: Pointer Authentication Code for instruction address, using key B.

PACIBSPPC: Pointer Authentication Code for return address, using key B.

PACM: Pointer authentication modifier.

PACNBIASPPC: Pointer Authentication Code for return address, using key A, not a branch target.

PACNBIBSPPC: Pointer Authentication Code for return address, using key B, not a branch target.

PRFM (immediate): Prefetch memory (immediate).

PRFM (literal): Prefetch memory (literal).

PRFM (register): Prefetch memory (register).

PRFUM: Prefetch memory (unscaled offset).

PSB: Profiling synchronization barrier.

PSSBB: Physical speculative store bypass barrier: an alias of DSB.

RBIT: Reverse bits.

RCWCAS, RCWCASA, RCWCASAL, RCWCASL: Read check write compare and swap doubleword in memory.

RCWCASP, RCWCASPA, RCWCASPAL, RCWCASPL: Read check write compare and swap quadword in memory.

RCWCLR, RCWCLRA, RCWCLRAL, RCWCLRL: Read check write atomic bit clear on doubleword in memory.

RCWCLRP, RCWCLRPA, RCWCLRPAL, RCWCLRPL: Read check write atomic bit clear on quadword in memory.

RCWSCAS, RCWSCASA, RCWSCASAL, RCWSCASL: Read check write software compare and swap doubleword in memory.

RCWSCASP, RCWSCASPA, RCWSCASPAL, RCWSCASPL: Read check write software compare and swap quadword in memory.

RCWSCLR, RCWSCLRA, RCWSCLRAL, RCWSCLRL: Read check write software atomic bit clear on doubleword in memory.

RCWSCLRP, RCWSCLRPA, RCWSCLRPAL, RCWSCLRPL: Read check write software atomic bit clear on quadword in memory.

RCWSET, RCWSETA, RCWSETAL, RCWSETL: Read check write atomic bit set on doubleword in memory.

RCWSETP, RCWSETPA, RCWSETPAL, RCWSETPL: Read check write atomic bit set on quadword in memory.

RCWSSET, RCWSSETA, RCWSSETAL, RCWSSETL: Read check write software atomic bit set on doubleword in memory.

RCWSSETP, RCWSSETPA, RCWSSETPAL, RCWSSETPL: Read check write software atomic bit set on quadword in memory.

RCWSSWP, RCWSSWPA, RCWSSWPAL, RCWSSWPL: Read check write software swap doubleword in memory.

RCWSSWPP, RCWSSWPPA, RCWSSWPPAL, RCWSSWPPL: Read check write software swap quadword in memory.

RCWSWP, RCWSWPA, RCWSWPAL, RCWSWPL: Read check write swap doubleword in memory.

RCWSWPP, RCWSWPPA, RCWSWPPAL, RCWSWPPL: Read check write swap quadword in memory.

RET: Return from subroutine.

RETAA, RETAB: Return from subroutine, with pointer authentication.

RETAASPPC, RETABSPPC: Return from subroutine, with enhanced pointer authentication using an immediate offset.

RETAASPPCR, RETABSPPCR: Return from subroutine, with enhanced pointer authentication using a register.

REV: Reverse bytes.

REV16: Reverse bytes in 16-bit halfwords.

REV32: Reverse bytes in 32-bit words.

REV64: Reverse bytes: an alias of REV.

RMIF: Rotate, mask insert flags.

ROR (immediate): Rotate right (immediate): an alias of EXTR.

ROR (register): Rotate right (register): an alias of RORV.

RORV: Rotate right variable.

RPRFM: Range prefetch memory.

SB: Speculation barrier.

SBC: Subtract with carry.

SBCS: Subtract with carry, setting flags.

SBFIZ: Signed bitfield insert in zeros: an alias of SBFM.

SBFM: Signed bitfield move.

SBFX: Signed bitfield extract: an alias of SBFM.

SDIV (quotient): Signed divide.

SETF8, SETF16: Evaluation of 8-bit or 16-bit flag values.

SETGP, SETGM, SETGE: Memory set with tag setting.

SETGPN, SETGMN, SETGEN: Memory set with tag setting, non-temporal.

SETGPT, SETGMT, SETGET: Memory set with tag setting, unprivileged.

SETGPTN, SETGMTN, SETGETN: Memory set with tag setting, unprivileged and non-temporal.

SETP, SETM, SETE: Memory set.

SETPN, SETMN, SETEN: Memory set, non-temporal.

SETPT, SETMT, SETET: Memory set, unprivileged.

SETPTN, SETMTN, SETETN: Memory set, unprivileged and non-temporal.

SEV: Send event.

SEVL: Send event local.

SHUH: Shared update hint.

SLBND: Scoped release boundary.

SMADDL: Signed multiply-add long.

SMAX (immediate): Signed maximum (immediate).

SMAX (register): Signed maximum (register).

SMC: Secure monitor call.

SMIN (immediate): Signed minimum (immediate).

SMIN (register): Signed minimum (register).

SMNEGL: Signed multiply-negate long: an alias of SMSUBL.

SMSTART: Enable access to Streaming SVE mode and SME architectural state: an alias of MSR (immediate).

SMSTOP: Disable access to Streaming SVE mode and SME architectural state: an alias of MSR (immediate).

SMSUBL: Signed multiply-subtract long.

SMULH: Signed multiply high.

SMULL (32-bit): Signed multiply long: an alias of SMADDL.

SRLS: Scope release.

SSBB: Speculative store bypass barrier: an alias of DSB.

ST2G: Store Allocation Tags.

ST64B: Single-copy atomic 64-byte store without status result.

ST64BV: Single-copy atomic 64-byte store with status result.

ST64BV0: Single-copy atomic 64-byte EL0 store with status result.

STADD, STADDL: Atomic add on word or doubleword, without return: an alias of LDADD, LDADDA, LDADDAL, LDADDL.

STADDB, STADDLB: Atomic add on byte, without return: an alias of LDADDB, LDADDAB, LDADDALB, LDADDLB.

STADDH, STADDLH: Atomic add on halfword, without return: an alias of LDADDH, LDADDAH, LDADDALH, LDADDLH.

STCLR, STCLRL: Atomic bit clear on word or doubleword, without return: an alias of LDCLR, LDCLRA, LDCLRAL, LDCLRL.

STCLRB, STCLRLB: Atomic bit clear on byte, without return: an alias of LDCLRB, LDCLRAB, LDCLRALB, LDCLRLB.

STCLRH, STCLRLH: Atomic bit clear on halfword, without return: an alias of LDCLRH, LDCLRAH, LDCLRALH, LDCLRLH.

STCPH: Store concurrent priority hint.

STEOR, STEORL: Atomic exclusive-OR on word or doubleword, without return: an alias of LDEOR, LDEORA, LDEORAL, LDEORL.

STEORB, STEORLB: Atomic exclusive-OR on byte, without return: an alias of LDEORB, LDEORAB, LDEORALB, LDEORLB.

STEORH, STEORLH: Atomic exclusive-OR on halfword, without return: an alias of LDEORH, LDEORAH, LDEORALH, LDEORLH.

STG: Store Allocation Tag.

STGM: Store Allocation Tag multiple.

STGP: Store Allocation Tag and pair of registers.

STILP: Store-release ordered pair of registers.

STL64B: Single-copy atomic 64-byte store-release without status result.

STL64BV: Single-copy atomic 64-byte store-release with status result.

STL64BV0: Single-copy atomic 64-byte EL0 store-release with status result.

STLLR: Store LORelease register.

STLLRB: Store LORelease register byte.

STLLRH: Store LORelease register halfword.

STLP: Store-release pair of registers.

STLR: Store-release register.

STLRB: Store-release register byte.

STLRH: Store-release register halfword.

STLTXR: Store-release unprivileged exclusive register.

STLUR: Store-release register (unscaled).

STLURB: Store-release register byte (unscaled).

STLURH: Store-release register halfword (unscaled).

STLXP: Store-release exclusive pair of registers.

STLXR: Store-release exclusive register.

STLXRB: Store-release exclusive register byte.

STLXRH: Store-release exclusive register halfword.

STNP: Store pair of registers, with non-temporal hint.

STP: Store pair of registers.

STR (immediate): Store register (immediate).

STR (register): Store register (register).

STRB (immediate): Store register byte (immediate).

STRB (register): Store register byte (register).

STRH (immediate): Store register halfword (immediate).

STRH (register): Store register halfword (register).

STSET, STSETL: Atomic bit set on word or doubleword, without return: an alias of LDSET, LDSETA, LDSETAL, LDSETL.

STSETB, STSETLB: Atomic bit set on byte, without return: an alias of LDSETB, LDSETAB, LDSETALB, LDSETLB.

STSETH, STSETLH: Atomic bit set on halfword, without return: an alias of LDSETH, LDSETAH, LDSETALH, LDSETLH.

STSHH: Store shared hint.

STSMAX, STSMAXL: Atomic signed maximum on word or doubleword, without return: an alias of LDSMAX, LDSMAXA, LDSMAXAL, LDSMAXL.

STSMAXB, STSMAXLB: Atomic signed maximum on byte, without return: an alias of LDSMAXB, LDSMAXAB, LDSMAXALB, LDSMAXLB.

STSMAXH, STSMAXLH: Atomic signed maximum on halfword, without return: an alias of LDSMAXH, LDSMAXAH, LDSMAXALH, LDSMAXLH.

STSMIN, STSMINL: Atomic signed minimum on word or doubleword, without return: an alias of LDSMIN, LDSMINA, LDSMINAL, LDSMINL.

STSMINB, STSMINLB: Atomic signed minimum on byte, without return: an alias of LDSMINB, LDSMINAB, LDSMINALB, LDSMINLB.

STSMINH, STSMINLH: Atomic signed minimum on halfword, without return: an alias of LDSMINH, LDSMINAH, LDSMINALH, LDSMINLH.

STTADD, STTADDL: Atomic add unprivileged, without return: an alias of LDTADD, LDTADDA, LDTADDAL, LDTADDL.

STTCLR, STTCLRL: Atomic bit clear unprivileged, without return: an alias of LDTCLR, LDTCLRA, LDTCLRAL, LDTCLRL.

STTNP: Store unprivileged pair of registers, with non-temporal hint.

STTP: Store unprivileged pair of registers.

STTR: Store register (unprivileged).

STTRB: Store register byte (unprivileged).

STTRH: Store register halfword (unprivileged).

STTSET, STTSETL: Atomic bit set unprivileged, without return: an alias of LDTSET, LDTSETA, LDTSETAL, LDTSETL.

STTXR: Store unprivileged exclusive register.

STUMAX, STUMAXL: Atomic unsigned maximum on word or doubleword, without return: an alias of LDUMAX, LDUMAXA, LDUMAXAL, LDUMAXL.

STUMAXB, STUMAXLB: Atomic unsigned maximum on byte, without return: an alias of LDUMAXB, LDUMAXAB, LDUMAXALB, LDUMAXLB.

STUMAXH, STUMAXLH: Atomic unsigned maximum on halfword, without return: an alias of LDUMAXH, LDUMAXAH, LDUMAXALH, LDUMAXLH.

STUMIN, STUMINL: Atomic unsigned minimum on word or doubleword, without return: an alias of LDUMIN, LDUMINA, LDUMINAL, LDUMINL.

STUMINB, STUMINLB: Atomic unsigned minimum on byte, without return: an alias of LDUMINB, LDUMINAB, LDUMINALB, LDUMINLB.

STUMINH, STUMINLH: Atomic unsigned minimum on halfword, without return: an alias of LDUMINH, LDUMINAH, LDUMINALH, LDUMINLH.

STUR: Store register (unscaled).

STURB: Store register byte (unscaled).

STURH: Store register halfword (unscaled).

STXP: Store exclusive pair of registers.

STXR: Store exclusive register.

STXRB: Store exclusive register byte.

STXRH: Store exclusive register halfword.

STZ2G: Store Allocation Tags, zeroing.

STZG: Store Allocation Tag, zeroing.

STZGM: Store Allocation Tag and zero multiple.

SUB (extended register): Subtract extended and scaled register.

SUB (immediate): Subtract immediate value.

SUB (shifted register): Subtract optionally-shifted register.

SUBG: Subtract with tag.

SUBP: Subtract pointer.

SUBPS: Subtract pointer, setting flags.

SUBPT: Subtract checked pointer.

SUBS (extended register): Subtract extended and scaled register, setting flags.

SUBS (immediate): Subtract immediate value, setting flags.

SUBS (shifted register): Subtract optionally-shifted register, setting flags.

SVC: Supervisor call.

SWP, SWPA, SWPAL, SWPL: Swap word or doubleword in memory.

SWPB, SWPAB, SWPALB, SWPLB: Swap byte in memory.

SWPH, SWPAH, SWPALH, SWPLH: Swap halfword in memory.

SWPP, SWPPA, SWPPAL, SWPPL: Swap quadword in memory.

SWPT, SWPTA, SWPTAL, SWPTL: Swap unprivileged.

SXTB: Signed extend byte: an alias of SBFM.

SXTH: Sign extend halfword: an alias of SBFM.

SXTW: Sign extend word: an alias of SBFM.

SYS: System instruction.

SYSL: System instruction with result.

SYSP: 128-bit system instruction.

TBNZ: Test bit and branch if nonzero.

TBZ: Test bit and branch if zero.

TFLTNZ: Test bit and fault if nonzero.

TFLTZ: Test bit and fault if zero.

TLBI: TLB invalidate operation: an alias of SYS.

TLBIP: TLB invalidate pair operation: an alias of SYSP.

TRCIT: Trace instrumentation: an alias of SYS.

TSB: Trace synchronization barrier.

TST (immediate): Test bits (immediate): an alias of ANDS (immediate).

TST (shifted register): Test (shifted register): an alias of ANDS (shifted register).

UBFIZ: Unsigned bitfield insert in zeros: an alias of UBFM.

UBFM: Unsigned bitfield move.

UBFX: Unsigned bitfield extract: an alias of UBFM.

UDF: Permanently undefined.

UDIV (quotient): Unsigned divide.

UMADDL: Unsigned multiply-add long.

UMAX (immediate): Unsigned maximum (immediate).

UMAX (register): Unsigned maximum (register).

UMIN (immediate): Unsigned minimum (immediate).

UMIN (register): Unsigned minimum (register).

UMNEGL: Unsigned multiply-negate long: an alias of UMSUBL.

UMSUBL: Unsigned multiply-subtract long.

UMULH: Unsigned multiply high.

UMULL (32-bit): Unsigned multiply long: an alias of UMADDL.

UXTB: Unsigned extend byte: an alias of UBFM.

UXTH: Unsigned extend halfword: an alias of UBFM.

WFE: Wait for event.

WFET: Wait for event with timeout.

WFI: Wait for interrupt.

WFIT: Wait for interrupt with timeout.

XAFLAG: Convert floating-point condition flags from external format to Arm format.

XPACD, XPACI, XPACLRI: Strip Pointer Authentication Code.

YIELD: Yield.

2026-09_rel2026-06_diff_tag2026-09-212026-09-08 1409:2244:3801

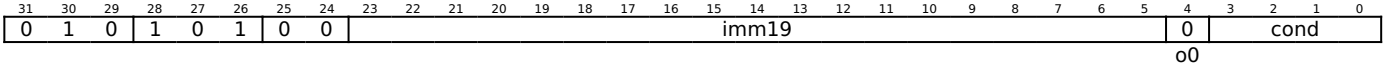
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

B.cond

Branch conditionally

This instruction branches conditionally to a label at a PC-relative offset. This instruction provides a hint that this is not a subroutine call or return.



Encoding

```
B.<cond> <label>
```

Decode for this encoding

```
let offset : bits(64) = SignExtend{}(imm19::'00');
let condition : bits(4) = cond;
```

Assembler Symbols

<cond> Is one of the standard conditions, encoded in the standard way, and encoded in “cond”:

cond	<cond>
0000	EQ
0001	NE
0010	CS|HS
0011	CC|LO
0100	MI
0101	PL
0110	VS
0111	VC
1000	HI
1001	LS
1010	GE
1011	LT
1100	GT
1101	LE
1110	AL
1111	NV

<label> Is the program label to be conditionally branched to. Its offset from the address of this instruction, in the range +/-1MB, is encoded as "imm19" times 4.

Operation

```
let branch_conditional : boolean = TRUE;
if ConditionHolds(condition) then
    BranchTo{64}(PC64() + offset, BranchType_DIR, branch_conditional);
else
    BranchNotTaken(BranchType_DIR, branch_conditional);
end;
```

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

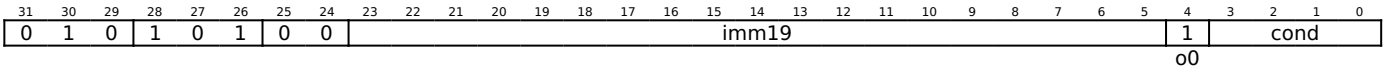
BC.cond

Branch consistent conditionally

This instruction branches conditionally to a label at a PC-relative offset, with a hint that this branch is very unlikely to change direction. This instruction provides a hint that this is not a subroutine call or return.

19-bit signed PC-relative branch offset

(FEAT_HBC)



Encoding

BC.<cond> <label>

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_HBC) then EndOfDecode(Decode_UNDEF); end;
let offset : bits(64) = SignExtend{}(imm19::'00');
let condition : bits(4) = cond;
```

Assembler Symbols

<cond> Is one of the standard conditions, encoded in the standard way, and encoded in “cond”:

cond	<cond>
0000	EQ
0001	NE
0010	CS|HS
0011	CC|LO
0100	MI
0101	PL
0110	VS
0111	VC
1000	HI
1001	LS
1010	GE
1011	LT
1100	GT
1101	LE
1110	AL
1111	NV

<label> Is the program label to be conditionally branched to. Its offset from the address of this instruction, in the range +/-1MB, is encoded as "imm19" times 4.

Operation

```
let branch_conditional : boolean = TRUE;
if ConditionHolds(condition) then
    BranchTo{64}(PC64() + offset, BranchType_DIR, branch_conditional);
else
    BranchNotTaken(BranchType_DIR, branch_conditional);
end;
```

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CAS, CASA, CASAL, CASL

Compare and swap word or doubleword in memory

This instruction reads a 32-bit word or 64-bit doubleword from memory, and compares it against the value held in a first register. If the comparison is equal, the value in a second register is written to memory. If the comparison is not equal, the architecture permits writing the value read from the location to memory. If the write is performed, the read and write occur atomically such that no other modification of the memory location can take place between the read and write.

- If the destination register is not one of WZR or XZR, `CASA` and `CASAL` load from memory with acquire semantics.
- `CASL` and `CASAL` store to memory with release semantics.
- `CAS` has neither acquire nor release semantics.

The architecture permits that the data read clears any exclusive monitors associated with that location, even if the compare subsequently fails.

If the instruction generates a synchronous Data Abort, the register which is compared and loaded, that is <Ws>, or <Xs>, is restored to the value held in the register before the instruction was executed.

For a `CAS` or `CASA` instruction, when <Ws> or <Xs> specifies the same register as <Wt> or <Xt>, this signals to the memory system that an additional subsequent `CAS`, `CASA`, `CASAL`, or `CASL` access to the specified location is expected to occur in the near future in the absence of taking an exception or exceptional software conditions. The memory system can respond by taking actions that are expected to enable the subsequent `CAS`, `CASA`, `CASAL`, or `CASL` access to succeed when it does occur.

A code sequence starting with a `CAS` or `CASA` instruction for which <Ws> or <Xs> specifies the same register as <Wt> or <Xt>, and ending with a subsequent `CAS`, `CASA`, `CASAL`, or `CASL` to the same location, exhibits the following properties for best performance when the location may be accessed concurrently, on one or more other PEs:

- The sequence does not contain any direct System register writes, address translation instructions, cache or TLB maintenance operations, exception producing instructions, exception returns, or `ISB` barriers.
- The execution of the sequence includes 32 or fewer instructions.
- The value provided in <Ws> or <Xs> of the first `CAS` or `CASA` is a value likely to result in the comparison failing. A failing comparison result may lead to better performance due to the hardware not performing a write to memory.

Note

For a `CAS` or `CASA` instruction, when <Ws> or <Xs> specifies the same register as <Wt> or <Xt>, the value in memory is not modified, because the `CAS` or `CASA` either fails its compare or writes the same value back to memory.

For performance reasons, Arm recommends that software not use a `CAS` or `CASA` instruction when <Ws> or <Xs> specifies the same register as <Wt> or <Xt> in a routine that includes a subsequent `CAS`, `CASA`, `CASAL`, or `CASL` for any of the following purposes:

- To load the value that will be compared to conditionally determine whether or not to execute the subsequent `CAS`, `CASA`, `CASAL`, or `CASL`.
- As a form of prefetch executed between the instruction loading the value that will be compared and the subsequent compare instruction that conditionally determines whether or not to execute the subsequent `CAS`, `CASA`, `CASAL`, or `CASL`.

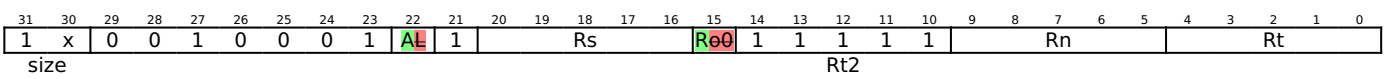
This guidance does not apply when <Ws> or <Xs> specifies the same register as <Wt> or <Xt> for a `CAS` or `CASA` instruction and the compare between the `CAS` or `CASA` and the subsequent `CAS`, `CASA`, `CASAL`, or `CASL` is intended to detect an exceptional software condition such that the result of the compare does not prevent the subsequent `CAS`, `CASA`, `CASAL`, or `CASL` from executing under normal operating conditions.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.

For information about addressing modes, see *Load/Store addressing modes*.

No offset

(FEAT_LSE)



Encoding for the 32-bit CAS variant

Applies when `(size == 10 && AL == 0 && Ro0 == 0)`

```
CAS <Ws>, <Wt>, [<Xn|SP>{, #0}]
```

Encoding for the 32-bit CASA variant

Applies when `(size == 10 && AL == 1 && Ro0 == 0)`

```
CASA <Ws>, <Wt>, [<Xn|SP>{, #0}]
```

Encoding for the 32-bit CASAL variant

Applies when `(size == 10 && AL == 1 && Ro0 == 1)`

```
CASAL <Ws>, <Wt>, [<Xn|SP>{, #0}]
```

Encoding for the 32-bit CASL variant

Applies when (size == 10 && AL == 0 && Ro0 == 1)

```
CASL <Ws>, <Wt>, [<Xn|SP>{, #0}]
```

Encoding for the 64-bit CAS variant

Applies when (size == 11 && AL == 0 && Ro0 == 0)

```
CAS <Xs>, <Xt>, [<Xn|SP>{, #0}]
```

Encoding for the 64-bit CASA variant

Applies when (size == 11 && AL == 1 && Ro0 == 0)

```
CASA <Xs>, <Xt>, [<Xn|SP>{, #0}]
```

Encoding for the 64-bit CASAL variant

Applies when (size == 11 && AL == 1 && Ro0 == 1)

```
CASAL <Xs>, <Xt>, [<Xn|SP>{, #0}]
```

Encoding for the 64-bit CASL variant

Applies when (size == 11 && AL == 0 && Ro0 == 1)

```
CASL <Xs>, <Xt>, [<Xn|SP>{, #0}]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSE) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let datasize : integer{} = 8 << UInt(size);
let regsize : integer{} = if datasize == 64 then 64 else 32;
let acquire : boolean = A == '1' && s != 31;
let release : boolean = R == '1';
let acquire : boolean = L == '1' && s != 31;
let release : boolean = o0 == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register to be compared and loaded, encoded in the "Rs" field.
<Wt>	Is the 32-bit name of the general-purpose register to be conditionally stored, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xs>	Is the 64-bit name of the general-purpose register to be compared and loaded, encoded in the "Rs" field.
<Xt>	Is the 64-bit name of the general-purpose register to be conditionally stored, encoded in the "Rt" field.

Operation

```
var address : bits(64);
var comparevalue : bits(datasize);
var newvalue : bits(datasize);

let privileged : boolean = PSTATE.EL != EL0;

let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_CAS, acquire, release,
                                                       tagchecked, privileged, t, s);

comparevalue = X{datasize}(s);
newvalue = X{datasize}(t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let data : bits(datasize) = MemAtomic{}(address, comparevalue, newvalue, accdesc);
X{regsize}(s) = ZeroExtend{regsize}(data);
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

CASB, CASAB, CASALB, CASLB

Compare and swap byte in memory

This instruction reads an 8-bit byte from memory, and compares it against the value held in a first register. If the comparison is equal, the value in a second register is written to memory. If the comparison is not equal, the architecture permits writing the value read from the location to memory. If the write is performed, the read and write occur atomically such that no other modification of the memory location can take place between the read and write.

- If the destination register is not WZR, `CASAB` and `CASALB` load from memory with acquire semantics.
- `CASLB` and `CASALB` store to memory with release semantics.
- `CASB` has neither acquire nor release semantics.

The architecture permits that the data read clears any exclusive monitors associated with that location, even if the compare subsequently fails.

If the instruction generates a synchronous Data Abort, the register which is compared and loaded, that is <Ws>, is restored to the values held in the register before the instruction was executed.

For a `CASB` or `CASAB` instruction, when <Ws> specifies the same register as <Wt>, this signals to the memory system that an additional subsequent `CASB`, `CASAB`, `CASALB`, or `CASLB` access to the specified location is expected to occur in the near future in the absence of taking an exception or exceptional software conditions. The memory system can respond by taking actions that are expected to enable the subsequent `CASB`, `CASAB`, `CASALB`, or `CASLB` access to succeed when it does occur.

A code sequence starting with a `CASB` or `CASAB` instruction for which <Ws> specifies the same register as <Wt>, and ending with a subsequent `CASB`, `CASAB`, `CASALB`, or `CASLB` to the same location, exhibits the following properties for best performance when the location may be accessed concurrently, on one or more other PEs:

- The sequence does not contain any direct System register writes, address translation instructions, cache or TLB maintenance operations, exception producing instructions, exception returns, or `ISB` barriers.
- The execution of the sequence includes 32 or fewer instructions.
- The value provided in <Ws> of the first `CASB` or `CASAB` is a value likely to result in the comparison failing. A failing comparison result may lead to better performance due to the hardware not performing a write to memory.

Note

For a `CASB` or `CASAB` instruction, when <Ws> specifies the same register as <Wt>, the value in memory is not modified, because the `CASB` or `CASAB` either fails its compare or writes the same value back to memory.

For performance reasons, Arm recommends that software not use a `CASB` or `CASAB` instruction when <Ws> specifies the same register as <Wt> in a routine that includes a subsequent `CASB`, `CASAB`, `CASALB`, or `CASLB` for any of the following purposes:

- To load the value that will be compared to conditionally determine whether or not to execute the subsequent `CASB`, `CASAB`, `CASALB`, or `CASLB`.
- As a form of prefetch executed between the instruction loading the value that will be compared and the subsequent compare instruction that conditionally determines whether or not to execute the subsequent `CASB`, `CASAB`, `CASALB`, or `CASLB`.

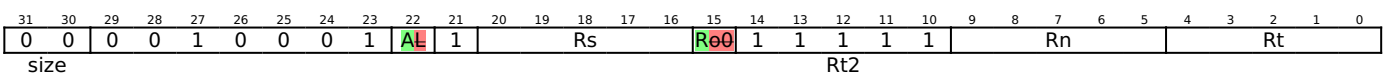
This guidance does not apply when <Ws> specifies the same register as <Wt> for a `CASB` or `CASAB` instruction and the compare between the `CASB` or `CASAB` and the subsequent `CASB`, `CASAB`, `CASALB`, or `CASLB` is intended to detect an exceptional software condition such that the result of the compare does not prevent the subsequent `CASB`, `CASAB`, `CASALB`, or `CASLB` from executing under normal operating conditions.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.

For information about addressing modes, see *Load/Store addressing modes*.

No offset

(FEAT_LSE)



Encoding for the CASB variant

Applies when (A L == 0 && R 0 0 == 0)

```
CASB <Ws>, <Wt>, [<Xn|SP>{, #0}]
```

Encoding for the CASAB variant

Applies when (A L == 1 && R 0 0 == 0)

```
CASAB <Ws>, <Wt>, [<Xn|SP>{, #0}]
```

Encoding for the CASALB variant

Applies when (A L == 1 && R 0 0 == 1)

```
CASALB <Ws>, <Wt>, [<Xn|SP>{, #0}]
```

Encoding for the CASLB variant

Applies when (A~~L~~ == 0 && R~~o0~~ == 1)

```
CASLB <Ws>, <Wt>, [<Xn|SP>{, #0}]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSE) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let acquire : boolean = A == '1' && s != 31;
let release : boolean = R == '1';
let acquire : boolean = L == '1' && s != 31;
let release : boolean = o0 == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register to be compared and loaded, encoded in the "Rs" field.
<Wt>	Is the 32-bit name of the general-purpose register to be conditionally stored, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
var address : bits(64);
var comparevalue : bits(8);
var newvalue : bits(8);

let privileged : boolean = PSTATE.EL != EL0;

let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_CAS, acquire, release,
                                                       tagchecked, privileged, t, s);

comparevalue = X{8}(s);
newvalue = X{8}(t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let data : bits(8) = MemAtomic{}(address, comparevalue, newvalue, accdesc);
X{32}(s) = ZeroExtend{32}(data);
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

CASH, CASAH, CASALH, CASLH

Compare and swap halfword in memory

This instruction reads a 16-bit halfword from memory, and compares it against the value held in a first register. If the comparison is equal, the value in a second register is written to memory. If the comparison is not equal, the architecture permits writing the value read from the location to memory. If the write is performed, the read and write occur atomically such that no other modification of the memory location can take place between the read and write.

- If the destination register is not WZR, `CASAH` and `CASALH` load from memory with acquire semantics.
- `CASLH` and `CASALH` store to memory with release semantics.
- `CASH` has neither acquire nor release semantics.

The architecture permits that the data read clears any exclusive monitors associated with that location, even if the compare subsequently fails.

If the instruction generates a synchronous Data Abort, the register which is compared and loaded, that is <Ws>, is restored to the values held in the register before the instruction was executed.

For a `CASH` or `CASAH` instruction, when <Ws> specifies the same register as <Wt>, this signals to the memory system that an additional subsequent `CASH`, `CASAH`, `CASALH`, or `CASLH` access to the specified location is expected to occur in the near future in the absence of taking an exception or exceptional software conditions. The memory system can respond by taking actions that are expected to enable the subsequent `CASH`, `CASAH`, `CASALH`, or `CASLH` access to succeed when it does occur.

A code sequence starting with a `CASH` or `CASAH` instruction for which <Ws> specifies the same register as <Wt>, and ending with a subsequent `CASH`, `CASAH`, `CASALH`, or `CASLH` to the same location, exhibits the following properties for best performance when the location may be accessed concurrently, on one or more other PEs:

- The sequence does not contain any direct System register writes, address translation instructions, cache or TLB maintenance operations, exception producing instructions, exception returns, or `ISB` barriers.
- The execution of the sequence includes 32 or fewer instructions.
- The value provided in <Ws> of the first `CASH` or `CASAH` is a value likely to result in the comparison failing. A failing comparison result may lead to better performance due to the hardware not performing a write to memory.

Note

For a `CASH` or `CASAH` instruction, when <Ws> specifies the same register as <Wt>, the value in memory is not modified, because the `CASH` or `CASAH` either fails its compare or writes the same value back to memory.

For performance reasons, Arm recommends that software not use a `CASH` or `CASAH` instruction when <Ws> specifies the same register as <Wt> in a routine that includes a subsequent `CASH`, `CASAH`, `CASALH`, or `CASLH` for any of the following purposes:

- To load the value that will be compared to conditionally determine whether or not to execute the subsequent `CASH`, `CASAH`, `CASALH`, or `CASLH`.
- As a form of prefetch executed between the instruction loading the value that will be compared and the subsequent compare instruction that conditionally determines whether or not to execute the subsequent `CASH`, `CASAH`, `CASALH`, or `CASLH`.

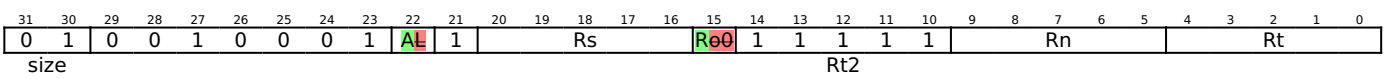
This guidance does not apply when <Ws> specifies the same register as <Wt> for a `CASH` or `CASAH` instruction and the compare between the `CASH` or `CASAH` and the subsequent `CASH`, `CASAH`, `CASALH`, or `CASLH` is intended to detect an exceptional software condition such that the result of the compare does not prevent the subsequent `CASH`, `CASAH`, `CASALH`, or `CASLH` from executing under normal operating conditions.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.

For information about addressing modes, see *Load/Store addressing modes*.

No offset

(FEAT_LSE)



Encoding for the CASH variant

Applies when (AL == 0 && Rs0 == 0)

```
CASH <Ws>, <Wt>, [<Xn|SP>{, #0}]
```

Encoding for the CASAH variant

Applies when (AL == 1 && Rs0 == 0)

```
CASAH <Ws>, <Wt>, [<Xn|SP>{, #0}]
```

Encoding for the CASALH variant

Applies when (AL == 1 && Rs0 == 1)

```
CASALH <Ws>, <Wt>, [<Xn|SP>{, #0}]
```

Encoding for the CASLH variant

Applies when (A~~L~~ == 0 && R~~o~~0 == 1)

```
CASLH <Ws>, <Wt>, [<Xn|SP>{, #0}]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSE) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let acquire : boolean = A == '1' && s != 31;
let release : boolean = R == '1';
let acquire : boolean = L == '1' && s != 31;
let release : boolean = o0 == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register to be compared and loaded, encoded in the "Rs" field.
<Wt>	Is the 32-bit name of the general-purpose register to be conditionally stored, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
var address : bits(64);
var comparevalue : bits(16);
var newvalue : bits(16);

let privileged : boolean = PSTATE.EL != EL0;

let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_CAS, acquire, release,
                                                       tagchecked, privileged, t, s);

comparevalue = X{16}(s);
newvalue = X{16}(t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let data : bits(16) = MemAtomic{}(address, comparevalue, newvalue, accdesc);
X{32}(s) = ZeroExtend{32}(data);
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

CASP, CASPA, CASPAL, CASPL

Compare and swap pair of words or doublewords in memory

This instruction reads a pair of 32-bit words or 64-bit doublewords from memory, and compares them against the values held in the first pair of registers. If the comparison is equal, the values in the second pair of registers are written to memory. If the comparison is not equal, the architecture permits writing the value read from the location to memory. If the writes are performed, the reads and writes occur atomically such that no other modification of the memory location can take place between the reads and writes.

- `CASPA` and `CASPAL` load from memory with acquire semantics.
- `CASPL` and `CASPAL` store to memory with release semantics.
- `CASP` has neither acquire nor release semantics.

The architecture permits that the data read clears any exclusive monitors associated with that location, even if the compare subsequently fails.

If the instruction generates a synchronous Data Abort, the registers which are compared and loaded, that is <Ws> and <W(s+1)>, or <Xs> and <X(s+1)>, are restored to the values held in the registers before the instruction was executed.

For a `CASP` or `CASPA` instruction, when <Ws> or <Xs> specifies the same register as <Wt> or <Xt>, this signals to the memory system that an additional subsequent `CASP`, `CASPA`, `CASPAL`, or `CASPL` access to the specified location is expected to occur in the near future in the absence of taking an exception or exceptional software conditions. The memory system can respond by taking actions that are expected to enable the subsequent `CASP`, `CASPA`, `CASPAL`, or `CASPL` access to succeed when it does occur.

A code sequence starting with a `CASP` or `CASPA` instruction for which <Ws> or <Xs> specifies the same register as <Wt> or <Xt>, and ending with a subsequent `CASP`, `CASPA`, `CASPAL`, or `CASPL` to the same location, exhibits the following properties for best performance when the location may be accessed concurrently, on one or more other PEs:

- The sequence does not contain any direct System register writes, address translation instructions, cache or TLB maintenance operations, exception producing instructions, exception returns, or `ISB` barriers.
- The execution of the sequence includes 32 or fewer instructions.
- The value provided in <Ws> or <Xs> of the first `CASP` or `CASPA` is a value likely to result in the comparison failing. A failing comparison result may lead to better performance due to the hardware not performing a write to memory.

Note

For a `CASP` or `CASPA` instruction, when <Ws> or <Xs> specifies the same register as <Wt> or <Xt>, the value in memory is not modified, because the `CASP` or `CASPA` either fails its compare or writes the same value back to memory.

For performance reasons, Arm recommends that software not use a `CASP` or `CASPA` instruction when <Ws> or <Xs> specifies the same register as <Wt> or <Xt> in a routine that includes a subsequent `CASP`, `CASPA`, `CASPAL`, or `CASPL` for any of the following purposes:

- To load the value that will be compared to conditionally determine whether or not to execute the subsequent `CASP`, `CASPA`, `CASPAL`, or `CASPL`.
- As a form of prefetch executed between the instruction loading the value that will be compared and the subsequent compare instruction that conditionally determines whether or not to execute the subsequent `CASP`, `CASPA`, `CASPAL`, or `CASPL`.

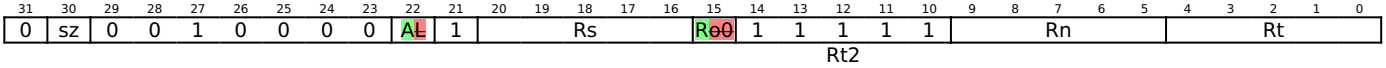
This guidance does not apply when <Ws> or <Xs> specifies the same register as <Wt> or <Xt> for a `CASP` or `CASPA` instruction and the compare between the `CASP` or `CASPA` and the subsequent `CASP`, `CASPA`, `CASPAL`, or `CASPL` is intended to detect an exceptional software condition such that the result of the compare does not prevent the subsequent `CASP`, `CASPA`, `CASPAL`, or `CASPL` from executing under normal operating conditions.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.

For information about addressing modes, see *Load/Store addressing modes*.

No offset

(FEAT_LSE)



Encoding for the 32-bit CASP variant

Applies when `(sz == 0 && A L == 0 && R 0 == 0)`

```
CASP <Ws>, <W(s+1)>, <Wt>, <W(t+1)>, [<Xn|SP>{, #0}]
```

Encoding for the 32-bit CASPA variant

Applies when `(sz == 0 && A L == 1 && R 0 == 0)`

```
CASPA <Ws>, <W(s+1)>, <Wt>, <W(t+1)>, [<Xn|SP>{, #0}]
```

Encoding for the 32-bit CASPAL variant

Applies when `(sz == 0 && A L == 1 && R 0 == 1)`

```
CASPAL <Ws>, <W(s+1)>, <Wt>, <W(t+1)>, [<Xn|SP>{, #0}]
```

Encoding for the 32-bit CASPL variant

Applies when `(sz == 0 && AL == 0 && Ro0 == 1)`

```
CASPL <Ws>, <W(s+1)>, <Wt>, <W(t+1)>, [<Xn|SP>{, #0}]
```

Encoding for the 64-bit CASP variant

Applies when `(sz == 1 && AL == 0 && Ro0 == 0)`

```
CASP <Xs>, <X(s+1)>, <Xt>, <X(t+1)>, [<Xn|SP>{, #0}]
```

Encoding for the 64-bit CASPA variant

Applies when `(sz == 1 && AL == 1 && Ro0 == 0)`

```
CASPA <Xs>, <X(s+1)>, <Xt>, <X(t+1)>, [<Xn|SP>{, #0}]
```

Encoding for the 64-bit CASPAL variant

Applies when `(sz == 1 && AL == 1 && Ro0 == 1)`

```
CASPAL <Xs>, <X(s+1)>, <Xt>, <X(t+1)>, [<Xn|SP>{, #0}]
```

Encoding for the 64-bit CASPL variant

Applies when `(sz == 1 && AL == 0 && Ro0 == 1)`

```
CASPL <Xs>, <X(s+1)>, <Xt>, <X(t+1)>, [<Xn|SP>{, #0}]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSE) then EndOfDecode(Decode_UNDEF); end;
if Rs[0] == '1' || Rt[0] == '1' then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let datasize : integer{} = 32 << UInt(sz);
let acquire : boolean = A == '1';
let release : boolean = R == '1';
let acquire : boolean = L == '1';
let release : boolean = o0 == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Ws>	Is the 32-bit name of the first general-purpose register to be compared and loaded, encoded in the "Rs" field. <Ws> must be an even-numbered register.
<W(s+1)>	Is the 32-bit name of the second general-purpose register to be compared and loaded.
<Wt>	Is the 32-bit name of the first general-purpose register to be conditionally stored, encoded in the "Rt" field. <Wt> must be an even-numbered register.
<W(t+1)>	Is the 32-bit name of the second general-purpose register to be conditionally stored.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xs>	Is the 64-bit name of the first general-purpose register to be compared and loaded, encoded in the "Rs" field. <Xs> must be an even-numbered register.
<X(s+1)>	Is the 64-bit name of the second general-purpose register to be compared and loaded.
<Xt>	Is the 64-bit name of the first general-purpose register to be conditionally stored, encoded in the "Rt" field. <Xt> must be an even-numbered register.
<X(t+1)>	Is the 64-bit name of the second general-purpose register to be conditionally stored.

Operation

```
var address : bits(64);
var comparevalue : bits(2*datasize);
var newvalue : bits(2*datasize);

let s1 : bits(datasize) = X{}(s);
let s2 : bits(datasize) = X{}(s+1);
let t1 : bits(datasize) = X{}(t);
let t2 : bits(datasize) = X{}(t+1);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_CAS, acquire, release,
                                                       tagchecked, privileged, t, t+1, s, s+1);

comparevalue = if BigEndian(accdesc.acctype) then s1::s2 else s2::s1;
newvalue     = if BigEndian(accdesc.acctype) then t1::t2 else t2::t1;

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let data : bits(2*datasize) = MemAtomic{}(address, comparevalue, newvalue, accdesc);

if BigEndian(accdesc.acctype) then
    X{datasize}(s)   = data[2*datasize-1:datasize];
    X{datasize}(s+1) = data[datasize-1:0];
else
    X{datasize}(s)   = data[datasize-1:0];
    X{datasize}(s+1) = data[2*datasize-1:datasize];
end;
```

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

CASPT, CASPAT, CASPALT, CASPLT

Compare and swap pair unprivileged

This instruction reads a pair of 64-bit doublewords from memory, and compares them against the values held in the first pair of registers. If the comparison is equal, the values in the second pair of registers are written to memory. If the comparison is not equal, the architecture permits writing the value read from the location to memory. If the writes are performed, the reads and writes occur atomically such that no other modification of the memory location can take place between the reads and writes.

- `CASPAT` and `CASPALT` load from memory with acquire semantics.
- `CASPLT` and `CASPALT` store to memory with release semantics.
- `CASPT` has neither acquire nor release semantics.

The architecture permits that the data read clears any exclusive monitors associated with that location, even if the compare subsequently fails.

If the instruction generates a synchronous Data Abort, the registers which are compared and loaded, that is <Xs> and <X(s+1)>, are restored to the values held in the registers before the instruction was executed.

Explicit Memory effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory effects operate with the restrictions determined by the Exception level at which the instruction is executed.

For a `CASPT` or `CASPAT` instruction, when <Xs> specifies the same register as <Xt>, this signals to the memory system that an additional subsequent `CASPT`, `CASPAT`, `CASPALT`, or `CASPLT` access to the specified location is expected to occur in the near future in the absence of taking an exception or exceptional software conditions. The memory system can respond by taking actions that are expected to enable the subsequent `CASPT`, `CASPAT`, `CASPALT`, or `CASPLT` access to succeed when it does occur.

A code sequence starting with a `CASPT` or `CASPAT` instruction for which <Xs> specifies the same register as <Xt>, and ending with a subsequent `CASPT`, `CASPAT`, `CASPALT`, or `CASPLT` to the same location, exhibits the following properties for best performance when the location may be accessed concurrently, on one or more other PEs:

- The sequence does not contain any direct System register writes, address translation instructions, cache or TLB maintenance operations, exception producing instructions, exception returns, or `ISB` barriers.
- The execution of the sequence includes 32 or fewer instructions.
- The value provided in <Xs> of the first `CASPT` or `CASPAT` is a value likely to result in the comparison failing. A failing comparison result may lead to better performance due to the hardware not performing a write to memory.

Note

For a `CASPT` or `CASPAT` instruction, when <Xs> specifies the same register as <Xt>, the value in memory is not modified, because the `CASPT` or `CASPAT` either fails its compare or writes the same value back to memory.

For performance reasons, Arm recommends that software not use a `CASPT` or `CASPAT` instruction when <Xs> specifies the same register as <Xt> in a routine that includes a subsequent `CASPT`, `CASPAT`, `CASPALT`, or `CASPLT` for any of the following purposes:

- To load the value that will be compared to conditionally determine whether or not to execute the subsequent `CASPT`, `CASPAT`, `CASPALT`, or `CASPLT`.
- As a form of prefetch executed between the instruction loading the value that will be compared and the subsequent compare instruction that conditionally determines whether or not to execute the subsequent `CASPT`, `CASPAT`, `CASPALT`, or `CASPLT`.

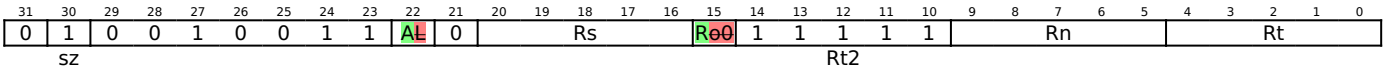
This guidance does not apply when <Xs> specifies the same register as <Xt> for a `CASPT` or `CASPAT` instruction and the compare between the `CASPT` or `CASPAT` and the subsequent `CASPT`, `CASPAT`, `CASPALT`, or `CASPLT` is intended to detect an exceptional software condition such that the result of the compare does not prevent the subsequent `CASPT`, `CASPAT`, `CASPALT`, or `CASPLT` from executing under normal operating conditions.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.

For information about addressing modes, see *Load/Store addressing modes*.

No offset

(FEAT_LSUI)



Encoding for the CASPT variant

Applies when (AL == 0 && Ro0 == 0)

```
CASPT <Xs>, <X(s+1)>, <Xt>, <X(t+1)>, [<Xn|SP>{, #0}]
```

Encoding for the CASPAT variant

Applies when (AL == 1 && Ro0 == 0)

```
CASPAT <Xs>, <X(s+1)>, <Xt>, <X(t+1)>, [<Xn|SP>{, #0}]
```

Encoding for the CASPALT variant

Applies when (A~~L~~ == 1 && R~~o0~~ == 1)

```
CASPALT <Xs>, <X(s+1)>, <Xt>, <X(t+1)>, [<Xn|SP>{, #0}]
```

Encoding for the CASPLT variant

Applies when (A~~L~~ == 0 && R~~o0~~ == 1)

```
CASPLT <Xs>, <X(s+1)>, <Xt>, <X(t+1)>, [<Xn|SP>{, #0}]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSUI) then EndOfDecode(Decode_UNDEF); end;
if Rs[0] == '1' || Rt[0] == '1' then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let datasize : integer{} = 64;
let acquire : boolean = A == '1';
let release : boolean = R == '1';
let acquire : boolean = L == '1';
let release : boolean = o0 == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Xs>	Is the 64-bit name of the first general-purpose register to be compared and loaded, encoded in the "Rs" field. <Xs> must be an even-numbered register.
<X(s+1)>	Is the 64-bit name of the second general-purpose register to be compared and loaded.
<Xt>	Is the 64-bit name of the first general-purpose register to be conditionally stored, encoded in the "Rt" field. <Xt> must be an even-numbered register.
<X(t+1)>	Is the 64-bit name of the second general-purpose register to be conditionally stored.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
var address : bits(64);
var comparevalue : bits(2*datasize);
var newvalue : bits(2*datasize);

let s1 : bits(datasize) = X{}(s);
let s2 : bits(datasize) = X{}(s+1);
let t1 : bits(datasize) = X{}(t);
let t2 : bits(datasize) = X{}(t+1);

let privileged : boolean = AArch64_IsUnprivAccessPriv();
let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_CAS, acquire, release,
                                                       tagchecked, privileged, t, t+1, s, s+1);

comparevalue = if BigEndian(accdesc.acctype) then s1::s2 else s2::s1;
newvalue     = if BigEndian(accdesc.acctype) then t1::t2 else t2::t1;

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let data : bits(2*datasize) = MemAtomic{}(address, comparevalue, newvalue, accdesc);

if BigEndian(accdesc.acctype) then
    X{datasize}(s)   = data[2*datasize-1:datasize];
    X{datasize}(s+1) = data[datasize-1:0];
else
    X{datasize}(s)   = data[datasize-1:0];
    X{datasize}(s+1) = data[2*datasize-1:datasize];
end;
```

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

CAST, CASAT, CASALT, CASLT

Compare and swap unprivileged

This instruction reads a 64-bit doubleword from memory, and compares it against the value held in a first register. If the comparison is equal, the value in a second register is written to memory. If the comparison is not equal, the architecture permits writing the value read from the location to memory. If the write is performed, the read and write occur atomically such that no other modification of the memory location can take place between the read and write.

- If the destination register is not XZR, `CASAT` and `CASALT` load from memory with acquire semantics.
- `CASLT` and `CASALT` store to memory with release semantics.
- `CAST` has neither acquire nor release semantics.

The architecture permits that the data read clears any exclusive monitors associated with that location, even if the compare subsequently fails.

If the instruction generates a synchronous Data Abort, the register which is compared and loaded, that is <Xs>, is restored to the value held in the register before the instruction was executed.

Explicit Memory effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory effects operate with the restrictions determined by the Exception level at which the instruction is executed.

For a `CAST` or `CASAT` instruction, when <Xs> specifies the same register as <Xt>, this signals to the memory system that an additional subsequent `CAST`, `CASAT`, `CASALT`, or `CASLT` access to the specified location is expected to occur in the near future in the absence of taking an exception or exceptional software conditions. The memory system can respond by taking actions that are expected to enable the subsequent `CAST`, `CASAT`, `CASALT`, or `CASLT` access to succeed when it does occur.

A code sequence starting with a `CAST` or `CASAT` instruction for which <Xs> specifies the same register as <Xt>, and ending with a subsequent `CAST`, `CASAT`, `CASALT`, or `CASLT` to the same location, exhibits the following properties for best performance when the location may be accessed concurrently, on one or more other PEs:

- The sequence does not contain any direct System register writes, address translation instructions, cache or TLB maintenance operations, exception producing instructions, exception returns, or `ISB` barriers.
- The execution of the sequence includes 32 or fewer instructions.
- The value provided in <Xs> of the first `CAST` or `CASAT` is a value likely to result in the comparison failing. A failing comparison result may lead to better performance due to the hardware not performing a write to memory.

Note

For a `CAST` or `CASAT` instruction, when <Xs> specifies the same register as <Xt>, the value in memory is not modified, because the `CAST` or `CASAT` either fails its compare or writes the same value back to memory.

For performance reasons, Arm recommends that software not use a `CAST` or `CASAT` instruction when <Xs> specifies the same register as <Xt> in a routine that includes a subsequent `CAST`, `CASAT`, `CASALT`, or `CASLT` for any of the following purposes:

- To load the value that will be compared to conditionally determine whether or not to execute the subsequent `CAST`, `CASAT`, `CASALT`, or `CASLT`.
- As a form of prefetch executed between the instruction loading the value that will be compared and the subsequent compare instruction that conditionally determines whether or not to execute the subsequent `CAST`, `CASAT`, `CASALT`, or `CASLT`.

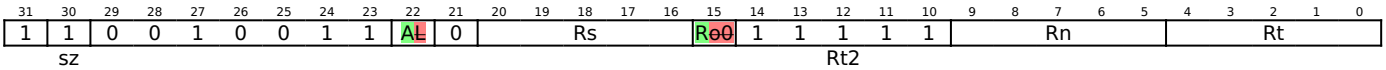
This guidance does not apply when <Xs> specifies the same register as <Xt> for a `CAST` or `CASAT` instruction and the compare between the `CAST` or `CASAT` and the subsequent `CAST`, `CASAT`, `CASALT`, or `CASLT` is intended to detect an exceptional software condition such that the result of the compare does not prevent the subsequent `CAST`, `CASAT`, `CASALT`, or `CASLT` from executing under normal operating conditions.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.

For information about addressing modes, see *Load/Store addressing modes*.

No offset

(FEAT_LSUI)



Encoding for the CAST variant

Applies when (AL == 0 && Ro0 == 0)

```
CAST <Xs>, <Xt>, [<Xn|SP>{, #0}]
```

Encoding for the CASAT variant

Applies when (AL == 1 && Ro0 == 0)

```
CASAT <Xs>, <Xt>, [<Xn|SP>{, #0}]
```

Encoding for the CASALT variant

Applies when (A~~L~~ == 1 && R~~o0~~ == 1)

```
CASALT <Xs>, <Xt>, [<Xn|SP>{, #0}]
```

Encoding for the CASLT variant

Applies when (A~~L~~ == 0 && R~~o0~~ == 1)

```
CASLT <Xs>, <Xt>, [<Xn|SP>{, #0}]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSUI) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let acquire : boolean = A == '1' && s != 31;
let release : boolean = R == '1';
~~let acquire : boolean = L == '1' && s != 31;~~
~~let release : boolean = o0 == '1';~~
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Xs>	Is the 64-bit name of the general-purpose register to be compared and loaded, encoded in the "Rs" field.
<Xt>	Is the 64-bit name of the general-purpose register to be conditionally stored, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
var address : bits(64);
var comparevalue : bits(64);
var newvalue : bits(64);

let privileged : boolean = AArch64_IsUnprivAccessPriv();

let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_CAS, acquire, release,
                                                       tagchecked, privileged, t, s);

comparevalue = X{64}(s);
newvalue = X{64}(t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

X{64}(s) = MemAtomic{64}(address, comparevalue, newvalue, accdesc);
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

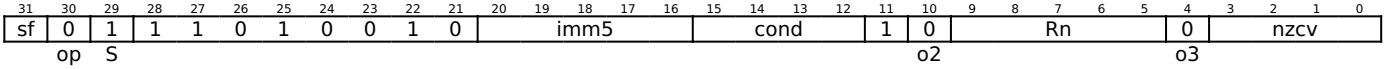
(old) htmldiff from- (new)

(old) htmldiff from- (new)

CCMN (immediate)

Conditional compare negative (immediate)

This instruction sets the value of the condition flags to the result of the comparison of a register value and a negated immediate value if the condition is TRUE, and an immediate value otherwise.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
CCMN <Wn>, #<imm>, #<nzcv>, <cond>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
CCMN <Xn>, #<imm>, #<nzcv>, <cond>
```

Decode for all variants of this encoding

```
let n : integer{} = UInt(Rn);
let datasize : integer{} = 32 << UInt(sf);
let condition : bits(4) = cond;
var flags : bits(4) = nzcv;
let imm : bits(datasize) = ZeroExtend{}(imm5);
```

Assembler Symbols

<Wn> Is the 32-bit name of the general-purpose source register, encoded in the "Rn" field.

<imm> Is a five bit unsigned (positive) immediate encoded in the "imm5" field.

<nzcv> Is the flag bit specifier, an immediate in the range 0 to 15, giving the alternative state for the 4-bit NZCV condition flags, encoded in the "nzcv" field.

<cond> Is one of the standard conditions, encoded in the standard way, and encoded in “cond”:

cond	<cond>
0000	EQ
0001	NE
0010	CS|HS
0011	CC|LO
0100	MI
0101	PL
0110	VS
0111	VC
1000	HI
1001	LS
1010	GE
1011	LT
1100	GT
1101	LE
1110	AL
1111	NV

<Xn> Is the 64-bit name of the general-purpose source register, encoded in the "Rn" field.

Operation

```
if ConditionHolds(condition) then
    let operand1 : bits(datasize) = X{datasize}(n);
    let operand2 : bits(datasize) = imm;

    (-, flags) = AddWithCarry{datasize}(operand1, operand2, '0');
end;
PSTATE.[N,Z,C,V] = flags;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

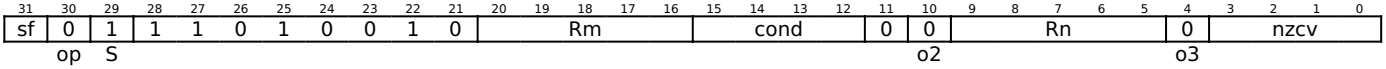
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

CCMN (register)

Conditional compare negative (register)

This instruction sets the value of the condition flags to the result of the comparison of a register value and the inverse of another register value if the condition is TRUE, and an immediate value otherwise.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
CCMN <Wn>, <Wm>, #<nzcv>, <cond>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
CCMN <Xn>, <Xm>, #<nzcv>, <cond>
```

Decode for all variants of this encoding

```
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 32 << UInt(sf);
let condition : bits(4) = cond;
var flags : bits(4) = nzcv;
```

Assembler Symbols

<Wn> Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Wm> Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.

<nzcv> Is the flag bit specifier, an immediate in the range 0 to 15, giving the alternative state for the 4-bit NZCV condition flags, encoded in the "nzcv" field.

<cond> Is one of the standard conditions, encoded in the standard way, and encoded in “cond”:

cond	<cond>
0000	EQ
0001	NE
0010	CS|HS
0011	CC|LO
0100	MI
0101	PL
0110	VS
0111	VC
1000	HI
1001	LS
1010	GE
1011	LT
1100	GT
1101	LE
1110	AL
1111	NV

<Xn> Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.

Operation

```
if ConditionHolds(condition) then
    let operand1 : bits(datasize) = X{datasize}(n);
    let operand2 : bits(datasize) = X{datasize}(m);

    (-, flags) = AddWithCarry{datasize}(operand1, operand2, '0');
end;
PSTATE.[N,Z,C,V] = flags;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel2026-06_diff_tag 2026-09-212026-09-08 1409:2244:3801

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

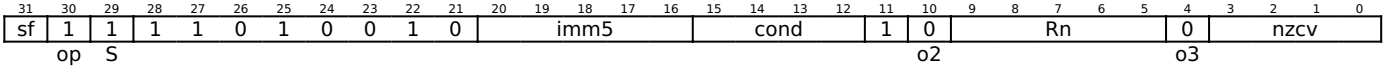
(old) htmldiff from- (new)

(old) htmldiff from- (new)

CCMP (immediate)

Conditional compare (immediate)

This instruction sets the value of the condition flags to the result of the comparison of a register value and an immediate value if the condition is TRUE, and an immediate value otherwise.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
CCMP <Wn>, #<imm>, #<nzcv>, <cond>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
CCMP <Xn>, #<imm>, #<nzcv>, <cond>
```

Decode for all variants of this encoding

```
let n : integer{} = UInt(Rn);
let datasize : integer{} = 32 << UInt(sf);
let condition : bits(4) = cond;
var flags : bits(4) = nzcv;
let imm : bits(datasize) = ZeroExtend{}(imm5);
```

Assembler Symbols

<Wn> Is the 32-bit name of the general-purpose source register, encoded in the "Rn" field.

<imm> Is a five bit unsigned (positive) immediate encoded in the "imm5" field.

<nzcv> Is the flag bit specifier, an immediate in the range 0 to 15, giving the alternative state for the 4-bit NZCV condition flags, encoded in the "nzcv" field.

<cond> Is one of the standard conditions, encoded in the standard way, and encoded in “cond”:

cond	<cond>
0000	EQ
0001	NE
0010	CS|HS
0011	CC|LO
0100	MI
0101	PL
0110	VS
0111	VC
1000	HI
1001	LS
1010	GE
1011	LT
1100	GT
1101	LE
1110	AL
1111	NV

<Xn> Is the 64-bit name of the general-purpose source register, encoded in the "Rn" field.

Operation

```
if ConditionHolds(condition) then
    let operand1 : bits(datasize) = X{datasize}(n);
    let operand2 : bits(datasize) = imm;

    (-, flags) = AddWithCarry{datasize}(operand1, NOT operand2, '1');
end;
PSTATE.[N,Z,C,V] = flags;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

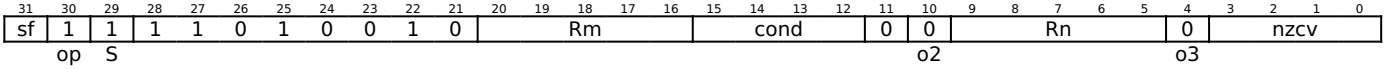
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

CCMP (register)

Conditional compare (register)

This instruction sets the value of the condition flags to the result of the comparison of two registers if the condition is TRUE, and an immediate value otherwise.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
CCMP <Wn>, <Wm>, #<nzcv>, <cond>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
CCMP <Xn>, <Xm>, #<nzcv>, <cond>
```

Decode for all variants of this encoding

```
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 32 << UInt(sf);
let condition : bits(4) = cond;
var flags : bits(4) = nzcv;
```

Assembler Symbols

<Wn> Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Wm> Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.

<nzcv> Is the flag bit specifier, an immediate in the range 0 to 15, giving the alternative state for the 4-bit NZCV condition flags, encoded in the "nzcv" field.

<cond> Is one of the standard conditions, encoded in the standard way, and encoded in “cond”:

cond	<cond>
0000	EQ
0001	NE
0010	CS|HS
0011	CC|LO
0100	MI
0101	PL
0110	VS
0111	VC
1000	HI
1001	LS
1010	GE
1011	LT
1100	GT
1101	LE
1110	AL
1111	NV

<Xn> Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.

Operation

```
if ConditionHolds(condition) then
    let operand1 : bits(datasize) = X{datasize}(n);
    let operand2 : bits(datasize) = X{datasize}(m);

    (-, flags) = AddWithCarry{datasize}(operand1, NOT operand2, '1');
end;
PSTATE.[N,Z,C,V] = flags;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel2026-06_diff_tag 2026-09-212026-09-08 1409:2244:3801

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

CFLT<cc> (immediate)

Compare register with immediate and fault

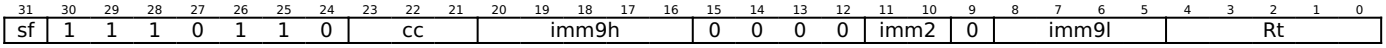
This instruction compares the value in a register with an immediate, and generates a Conditional fault exception if the comparison is true. If an exception is generated, the <reason> operand is copied to *ESR_ELx*.

This instruction does not affect the condition flags.

This instruction is used by the pseudo-instructions CFLTGE (immediate), CFLTHS (immediate), CFLTLE (immediate), CFLTLS (immediate), CFLTNZ, and CFLTZ.

Fault

(FEAT_CFLT)



Encoding for the 32-bit greater than variant

Applies when `(sf == 0 && cc == 000)`

```
CFLTGT #<reason>, <Wt>, #<imm>
```

Encoding for the 32-bit less than variant

Applies when `(sf == 0 && cc == 001)`

```
CFLTLT #<reason>, <Wt>, #<imm>
```

Encoding for the 32-bit higher variant

Applies when `(sf == 0 && cc == 010)`

```
CFLTHI #<reason>, <Wt>, #<imm>
```

Encoding for the 32-bit lower variant

Applies when `(sf == 0 && cc == 011)`

```
CFLTLO #<reason>, <Wt>, #<imm>
```

Encoding for the 32-bit equal variant

Applies when `(sf == 0 && cc == 100)`

```
CFLTEQ #<reason>, <Wt>, #<imm>
```

Encoding for the 32-bit not equal variant

Applies when `(sf == 0 && cc == 101)`

```
CFLTNE #<reason>, <Wt>, #<imm>
```

Encoding for the 64-bit greater than variant

Applies when `(sf == 1 && cc == 000)`

```
CFLTGT #<reason>, <Xt>, #<imm>
```

Encoding for the 64-bit less than variant

Applies when `(sf == 1 && cc == 001)`

```
CFLTLT #<reason>, <Xt>, #<imm>
```

Encoding for the 64-bit higher variant

Applies when `(sf == 1 && cc == 010)`

```
CFLTHI #<reason>, <Xt>, #<imm>
```

Encoding for the 64-bit lower variant

Applies when `(sf == 1 && cc == 011)`

```
CFLTLO #<reason>, <Xt>, #<imm>
```

Encoding for the 64-bit equal variant

Applies when `(sf == 1 && cc == 100)`

```
CFLTEQ #<reason>, <Xt>, #<imm>
```

Encoding for the 64-bit not equal variant

Applies when `(sf == 1 && cc == 101)`

```
CFLTNE #<reason>, <Xt>, #<imm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_CFLT) then EndOfDecode(Decode_UNDEF); end;
let datasize : integer{} = 32 << UInt(sf);
let t : integer = UInt(Rt);
let imm : bits(9) = imm9h::imm9l;
let reason : bits(16) = ZeroExtend{}(imm2);

var cmp : CmpOp;
var unsigned : boolean;

case cc of
    when '000' => cmp = Cmp_GT; unsigned = FALSE;
    when '001' => cmp = Cmp_LT; unsigned = FALSE;
    when '010' => cmp = Cmp_GT; unsigned = TRUE;
    when '011' => cmp = Cmp_LT; unsigned = TRUE;
    when '100' => cmp = Cmp_EQ; unsigned = FALSE;
    when '101' => cmp = Cmp_NE; unsigned = FALSE;
    otherwise  => EndOfDecode(Decode_UNDEF);
end;
```

Assembler Symbols

<reason>	Is a 2-bit unsigned immediate, in the range 0 to 3, encoded in the "imm2" field.
<Wt>	Is the 32-bit name of the general-purpose register to be tested, encoded in the "Rt" field.
<imm>	For the "32-bit equal", "32-bit greater than", "32-bit less than", "32-bit not equal", "64-bit equal", "64-bit greater than", "64-bit less than", and "64-bit not equal" variants: is a signed immediate, in the range -256 to 255, encoded in "imm9h:imm9l". For the "32-bit higher", "32-bit lower", "64-bit higher", and "64-bit lower" variants: is an unsigned immediate, in the range 0 to 511, encoded in "imm9h:imm9l".
<Xt>	Is the 64-bit name of the general-purpose register to be tested, encoded in the "Rt" field.

Operation

```
CheckCFLTEnabled();

let operand1 : bits(datasize) = X{}(t);

var cond : boolean;
let value1 : integer = if unsigned then UInt(operand1) else SInt(operand1);
let value2 : integer = if unsigned then UInt(imm) else SInt(imm);

case cmp of
    when Cmp_EQ => cond = value1 == value2;
    when Cmp_NE => cond = value1 != value2;
    when Cmp_LT => cond = value1 < value2;
    when Cmp_GT => cond = value1 > value2;
end;
if cond then
    let isflt : boolean = FALSE;
    AArch64_ConditionalFault(isflt, reason);
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

no old file htmldiff from- (new)

CFLT<cc> (register)

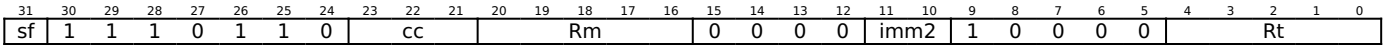
Compare registers and fault

This instruction compares the values in two registers, and generates a Conditional fault exception if the condition is true. This instruction does not affect the condition flags.

This instruction is used by the pseudo-instructions CFLTLE (register), CFLTLO (register), CFLTLS (register), and CFLTLT (register).

Fault

(FEAT_CFLT)



Encoding for the 32-bit greater than variant

Applies when `(sf == 0 && cc == 000)`

```
CFLTGT #<reason>, <Wt|WSP>, <Wm|WSP>
```

Encoding for the 32-bit greater than or equal variant

Applies when `(sf == 0 && cc == 001)`

```
CFLTGE #<reason>, <Wt|WSP>, <Wm|WSP>
```

Encoding for the 32-bit higher variant

Applies when `(sf == 0 && cc == 010)`

```
CFLTHI #<reason>, <Wt|WSP>, <Wm|WSP>
```

Encoding for the 32-bit higher or same variant

Applies when `(sf == 0 && cc == 011)`

```
CFLTHS #<reason>, <Wt|WSP>, <Wm|WSP>
```

Encoding for the 32-bit equal variant

Applies when `(sf == 0 && cc == 110)`

```
CFLTEQ #<reason>, <Wt|WSP>, <Wm|WSP>
```

Encoding for the 32-bit not equal variant

Applies when `(sf == 0 && cc == 111)`

```
CFLTNE #<reason>, <Wt|WSP>, <Wm|WSP>
```

Encoding for the 64-bit greater than variant

Applies when `(sf == 1 && cc == 000)`

```
CFLTGT #<reason>, <Xt|SP>, <Xm|SP>
```

Encoding for the 64-bit greater than or equal variant

Applies when `(sf == 1 && cc == 001)`

```
CFLTGE #<reason>, <Xt|SP>, <Xm|SP>
```

Encoding for the 64-bit higher variant

Applies when `(sf == 1 && cc == 010)`

```
CFLTHI #<reason>, <Xt|SP>, <Xm|SP>
```

Encoding for the 64-bit higher or same variant

Applies when `(sf == 1 && cc == 011)`

```
CFLTHS #<reason>, <Xt|SP>, <Xm|SP>
```

Encoding for the 64-bit equal variant

Applies when `(sf == 1 && cc == 110)`

```
CFLTEQ #<reason>, <Xt|SP>, <Xm|SP>
```

Encoding for the 64-bit not equal variant

Applies when `(sf == 1 && cc == 111)`

```
CFLTNE #<reason>, <Xt|SP>, <Xm|SP>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_CFLT) then EndOfDecode(Decode_UNDEF); end;

let datasize : integer{} = 32 << UInt(sf);
let t : integer = UInt(Rt);
let m : integer = UInt(Rm);
let reason : bits(16) = ZeroExtend{}(imm2);

var cmp : CmpOp;
var unsigned : boolean;

case cc of
    when '000' => cmp = Cmp_GT; unsigned = FALSE;
    when '001' => cmp = Cmp_GE; unsigned = FALSE;
    when '010' => cmp = Cmp_GT; unsigned = TRUE;
    when '011' => cmp = Cmp_GE; unsigned = TRUE;
    when '110' => cmp = Cmp_EQ; unsigned = TRUE;
    when '111' => cmp = Cmp_NE; unsigned = TRUE;
    otherwise  => EndOfDecode(Decode_UNDEF);
end;
```

Assembler Symbols

<reason>	Is a 2-bit unsigned immediate, in the range 0 to 3, encoded in the "imm2" field.
<Wt|WSP>	Is the 32-bit name of the general-purpose register or stack pointer to be tested, encoded in the "Rt" field.
<Wm|WSP>	Is the 32-bit name of the second general-purpose source register or stack pointer, encoded in the "Rm" field.
<Xt|SP>	Is the 64-bit name of the general-purpose register or stack pointer to be tested, encoded in the "Rt" field.
<Xm|SP>	Is the 64-bit name of the second general-purpose source register or stack pointer, encoded in the "Rm" field.

Operation

```
CheckCFLTEnabled();

let operand1 : bits(datasize) = if t == 31 then SP{datasize}() else X{datasize}(t);
let operand2 : bits(datasize) = if m == 31 then SP{datasize}() else X{datasize}(m);

var cond : boolean;
let value1 : integer = if unsigned then UInt(operand1) else SInt(operand1);
let value2 : integer = if unsigned then UInt(operand2) else SInt(operand2);
case cmp of
    when Cmp_EQ => cond = value1 == value2;
    when Cmp_NE => cond = value1 != value2;
    when Cmp_GT => cond = value1 >  value2;
    when Cmp_GE => cond = value1 >= value2;
end;

if cond then
    let isflt : boolean = FALSE;
    AArch64_ConditionalFault(isflt, reason);
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

no old file htmldiff from- (new)

no old file htmldiff from- (new)

CFLTGE (immediate)

Compare signed greater than or equal to immediate and fault

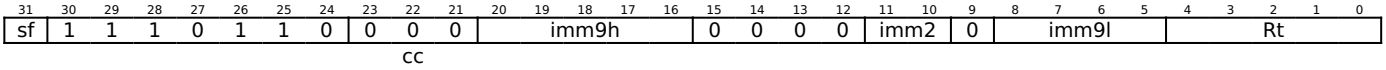
This instruction compares the signed value in a register with an immediate, and generates a Conditional fault exception if the register value is greater than or equal to the immediate. If an exception is generated, the <reason> operand is copied to *ESR_ELx*.
This instruction does not affect the condition flags.

This is a pseudo-instruction of CFLT<cc> (immediate). This means:

- The encodings in this description are named to match the encodings of CFLT<cc> (immediate).
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of CFLT<cc> (immediate) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Fault

(FEAT_CFLT)



Encoding for the 32-bit greater than variant

Applies when `(sf == 0)`

```
CFLTGE #<reason>, <Wt>, #<immp1>
```

is equivalent to

```
CFLTGT #<reason>, <Wt>, #<imm>
```

Encoding for the 64-bit greater than variant

Applies when `(sf == 1)`

```
CFLTGE #<reason>, <Xt>, #<immp1>
```

is equivalent to

```
CFLTGT #<reason>, <Xt>, #<imm>
```

Assembler Symbols

<reason>	Is a 2-bit unsigned immediate, in the range 0 to 3, encoded in the "imm2" field.
<Wt>	Is the 32-bit name of the general-purpose register to be tested, encoded in the "Rt" field.
<immp1>	Is a signed immediate, in the range -255 to 256, encoded as "imm9h:imm9l" plus 1.
<Xt>	Is the 64-bit name of the general-purpose register to be tested, encoded in the "Rt" field.

Operation

The description of CFLT<cc> (immediate) gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

no old file htmldiff from- (new)

no old file htmldiff from- (new)

CFLTHS (immediate)

Compare unsigned higher than or same as immediate and fault

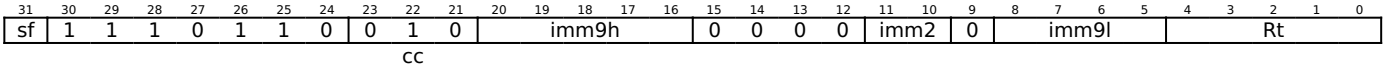
This instruction compares the unsigned value in a register with an immediate, and generates a Conditional fault exception if the register value is higher than or the same as the immediate. If an exception is generated, the <reason> operand is copied to *ESR_ELx*.
This instruction does not affect the condition flags.

This is a pseudo-instruction of CFLT<cc> (immediate). This means:

- The encodings in this description are named to match the encodings of CFLT<cc> (immediate).
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of CFLT<cc> (immediate) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Fault

(FEAT_CFLT)



Encoding for the 32-bit higher variant

Applies when `(sf == 0)`

```
CFLTHS #<reason>, <Wt>, #<immp1>
```

is equivalent to

```
CFLTHI #<reason>, <Wt>, #<imm>
```

Encoding for the 64-bit higher variant

Applies when `(sf == 1)`

```
CFLTHS #<reason>, <Xt>, #<immp1>
```

is equivalent to

```
CFLTHI #<reason>, <Xt>, #<imm>
```

Assembler Symbols

<reason>	Is a 2-bit unsigned immediate, in the range 0 to 3, encoded in the "imm2" field.
<Wt>	Is the 32-bit name of the general-purpose register to be tested, encoded in the "Rt" field.
<immp1>	Is an unsigned immediate, in the range 1 to 512, encoded as "imm9h:imm9l" plus 1.
<Xt>	Is the 64-bit name of the general-purpose register to be tested, encoded in the "Rt" field.

Operation

The description of CFLT<cc> (immediate) gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

no old file htmldiff from- (new)

no old file htmldiff from- (new)

CFLTLE (immediate)

Compare signed less than or equal to immediate and fault

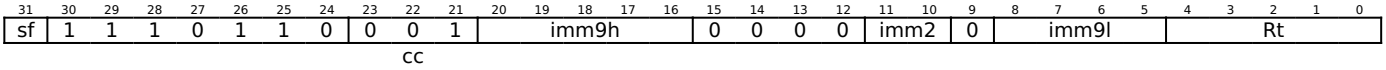
This instruction compares the signed value in a register with an immediate, and generates a Conditional fault exception if the register value is less than or equal to the immediate. If an exception is generated, the <reason> operand is copied to *ESR_ELx*.
This instruction does not affect the condition flags.

This is a pseudo-instruction of CFLT<cc> (immediate). This means:

- The encodings in this description are named to match the encodings of CFLT<cc> (immediate).
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of CFLT<cc> (immediate) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Fault

(FEAT_CFLT)



Encoding for the 32-bit less than variant

Applies when `(sf == 0)`

```
CFLTLE #<reason>, <Wt>, #<imms1>
```

is equivalent to

```
CFLTLT #<reason>, <Wt>, #<imm>
```

Encoding for the 64-bit less than variant

Applies when `(sf == 1)`

```
CFLTLE #<reason>, <Xt>, #<imms1>
```

is equivalent to

```
CFLTLT #<reason>, <Xt>, #<imm>
```

Assembler Symbols

<reason>	Is a 2-bit unsigned immediate, in the range 0 to 3, encoded in the "imm2" field.
<Wt>	Is the 32-bit name of the general-purpose register to be tested, encoded in the "Rt" field.
<imms1>	Is a signed immediate, in the range -257 to 254, encoded as "imm9h:imm9l" minus 1.
<Xt>	Is the 64-bit name of the general-purpose register to be tested, encoded in the "Rt" field.

Operation

The description of CFLT<cc> (immediate) gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

no old file htmldiff from- (new)

CFLTLE (register)

Compare registers and fault if less than or equal

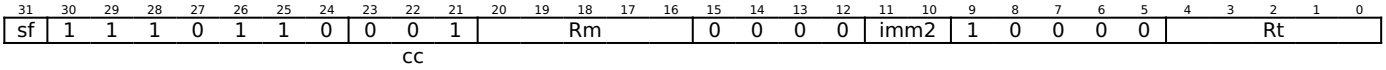
This instruction compares the values in two registers, and generates a Conditional fault exception if the first value is less than or equal to the second. If an exception is generated, the <reason> operand is copied to *ESR_ELx*.
This instruction does not affect the condition flags.

This is a pseudo-instruction of CFLT<cc> (register). This means:

- The encodings in this description are named to match the encodings of CFLT<cc> (register).
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of CFLT<cc> (register) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Fault

(FEAT_CFLT)



Encoding for the 32-bit greater than or equal variant

Applies when `(sf == 0)`

```
CFLTLE #<reason>, <Wm|WSP>, <Wt|WSP>
```

is equivalent to

```
CFLTGE #<reason>, <Wt|WSP>, <Wm|WSP>
```

Encoding for the 64-bit greater than or equal variant

Applies when `(sf == 1)`

```
CFLTLE #<reason>, <Xm|SP>, <Xt|SP>
```

is equivalent to

```
CFLTGE #<reason>, <Xt|SP>, <Xm|SP>
```

Assembler Symbols

<reason>	Is a 2-bit unsigned immediate, in the range 0 to 3, encoded in the "imm2" field.
<Wm|WSP>	Is the 32-bit name of the general-purpose register or stack pointer to be tested, encoded in the "Rm" field.
<Wt|WSP>	Is the 32-bit name of the second general-purpose source register or stack pointer, encoded in the "Rt" field.
<Xm|SP>	Is the 64-bit name of the general-purpose register or stack pointer to be tested, encoded in the "Rm" field.
<Xt|SP>	Is the 64-bit name of the second general-purpose source register or stack pointer, encoded in the "Rt" field.

Operation

The description of CFLT<cc> (register) gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CFLTLO (register)

Compare registers and fault if lower

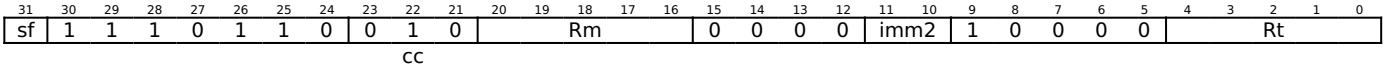
This instruction compares the values in two registers, and generates a Conditional fault exception if the first value is lower than the second. If an exception is generated, the <reason> operand is copied to *ESR_ELx*.
This instruction does not affect the condition flags.

This is a pseudo-instruction of CFLT<cc> (register). This means:

- The encodings in this description are named to match the encodings of CFLT<cc> (register).
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of CFLT<cc> (register) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Fault

(FEAT_CFLT)



Encoding for the 32-bit higher variant

Applies when `(sf == 0)`

```
CFLTLO #<reason>, <Wm|WSP>, <Wt|WSP>
```

is equivalent to

```
CFLTHI #<reason>, <Wt|WSP>, <Wm|WSP>
```

Encoding for the 64-bit higher variant

Applies when `(sf == 1)`

```
CFLTLO #<reason>, <Xm|SP>, <Xt|SP>
```

is equivalent to

```
CFLTHI #<reason>, <Xt|SP>, <Xm|SP>
```

Assembler Symbols

<reason>	Is a 2-bit unsigned immediate, in the range 0 to 3, encoded in the "imm2" field.
<Wm|WSP>	Is the 32-bit name of the general-purpose register or stack pointer to be tested, encoded in the "Rm" field.
<Wt|WSP>	Is the 32-bit name of the second general-purpose source register or stack pointer, encoded in the "Rt" field.
<Xm|SP>	Is the 64-bit name of the general-purpose register or stack pointer to be tested, encoded in the "Rm" field.
<Xt|SP>	Is the 64-bit name of the second general-purpose source register or stack pointer, encoded in the "Rt" field.

Operation

The description of CFLT<cc> (register) gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

no old file htmldiff from- (new)

CFLTLS (immediate)

Compare unsigned lower than or same as immediate and fault

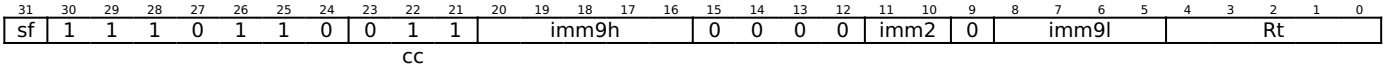
This instruction compares the unsigned value in a register with an immediate, and generates a Conditional fault exception if the register value is lower than or the same as the immediate. If an exception is generated, the <reason> operand is copied to *ESR_ELx*.
This instruction does not affect the condition flags.

This is a pseudo-instruction of CFLT<cc> (immediate). This means:

- The encodings in this description are named to match the encodings of CFLT<cc> (immediate).
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of CFLT<cc> (immediate) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Fault

(FEAT_CFLT)



Encoding for the 32-bit lower variant

Applies when `(sf == 0)`

```
CFLTLS #<reason>, <Wt>, #<imms1>
```

is equivalent to

```
CFLTLO #<reason>, <Wt>, #<imm>
```

Encoding for the 64-bit lower variant

Applies when `(sf == 1)`

```
CFLTLS #<reason>, <Xt>, #<imms1>
```

is equivalent to

```
CFLTLO #<reason>, <Xt>, #<imm>
```

Assembler Symbols

<reason>	Is a 2-bit unsigned immediate, in the range 0 to 3, encoded in the "imm2" field.
<Wt>	Is the 32-bit name of the general-purpose register to be tested, encoded in the "Rt" field.
<imms1>	Is a signed immediate, in the range -1 to 510, encoded as "imm9h:imm9l" minus 1.
<Xt>	Is the 64-bit name of the general-purpose register to be tested, encoded in the "Rt" field.

Operation

The description of CFLT<cc> (immediate) gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

no old file htmldiff from- (new)

no old file htmldiff from- (new)

CFLTLS (register)

Compare registers and fault if lower than or same

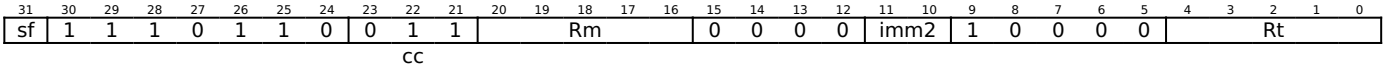
This instruction compares the values in two registers, and generates a Conditional fault exception if the first value is lower than or the same as the second. If an exception is generated, the <reason> operand is copied to *ESR_ELx*.
This instruction does not affect the condition flags.

This is a pseudo-instruction of CFLT<cc> (register). This means:

- The encodings in this description are named to match the encodings of CFLT<cc> (register).
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of CFLT<cc> (register) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Fault

(FEAT_CFLT)



Encoding for the 32-bit higher or same variant

Applies when `(sf == 0)`

```
CFLTLS #<reason>, <Wm|WSP>, <Wt|WSP>
```

is equivalent to

```
CFLTHS #<reason>, <Wt|WSP>, <Wm|WSP>
```

Encoding for the 64-bit higher or same variant

Applies when `(sf == 1)`

```
CFLTLS #<reason>, <Xm|SP>, <Xt|SP>
```

is equivalent to

```
CFLTHS #<reason>, <Xt|SP>, <Xm|SP>
```

Assembler Symbols

<reason>	Is a 2-bit unsigned immediate, in the range 0 to 3, encoded in the "imm2" field.
<Wm|WSP>	Is the 32-bit name of the general-purpose register or stack pointer to be tested, encoded in the "Rm" field.
<Wt|WSP>	Is the 32-bit name of the second general-purpose source register or stack pointer, encoded in the "Rt" field.
<Xm|SP>	Is the 64-bit name of the general-purpose register or stack pointer to be tested, encoded in the "Rm" field.
<Xt|SP>	Is the 64-bit name of the second general-purpose source register or stack pointer, encoded in the "Rt" field.

Operation

The description of CFLT<cc> (register) gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

no old file htmldiff from- (new)

CFLTLT (register)

Compare registers and fault if less than

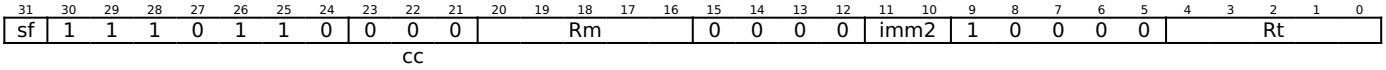
This instruction compares the values in two registers, and generates a Conditional fault exception if the first value is less than the second. If an exception is generated, the <reason> operand is copied to *ESR_ELx*.
This instruction does not affect the condition flags.

This is a pseudo-instruction of CFLT<cc> (register). This means:

- The encodings in this description are named to match the encodings of CFLT<cc> (register).
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of CFLT<cc> (register) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Fault

(FEAT_CFLT)



Encoding for the 32-bit greater than variant

Applies when `(sf == 0)`

```
CFLTLT #<reason>, <Wm|WSP>, <Wt|WSP>
```

is equivalent to

```
CFLTGT #<reason>, <Wt|WSP>, <Wm|WSP>
```

Encoding for the 64-bit greater than variant

Applies when `(sf == 1)`

```
CFLTLT #<reason>, <Xm|SP>, <Xt|SP>
```

is equivalent to

```
CFLTGT #<reason>, <Xt|SP>, <Xm|SP>
```

Assembler Symbols

<reason>	Is a 2-bit unsigned immediate, in the range 0 to 3, encoded in the "imm2" field.
<Wm|WSP>	Is the 32-bit name of the general-purpose register or stack pointer to be tested, encoded in the "Rm" field.
<Wt|WSP>	Is the 32-bit name of the second general-purpose source register or stack pointer, encoded in the "Rt" field.
<Xm|SP>	Is the 64-bit name of the general-purpose register or stack pointer to be tested, encoded in the "Rm" field.
<Xt|SP>	Is the 64-bit name of the second general-purpose source register or stack pointer, encoded in the "Rt" field.

Operation

The description of CFLT<cc> (register) gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CFLTNZ

Compare and fault on nonzero

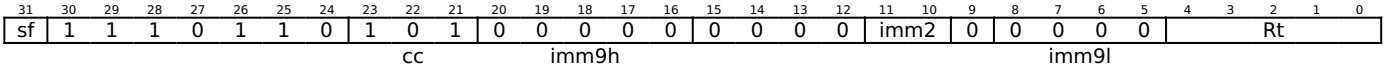
This instruction generates a Conditional fault exception if the register value is nonzero. If an exception is generated, the <reason> operand is copied to *ESR_ELx*.
This instruction does not affect the condition flags.

This is a pseudo-instruction of CFLT<cc> (immediate). This means:

- The encodings in this description are named to match the encodings of CFLT<cc> (immediate).
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of CFLT<cc> (immediate) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Fault

(FEAT_CFLT)



Encoding for the 32-bit not equal variant

Applies when `(sf == 0)`

```
CFLTNZ #<reason>, <Wt>
```

is equivalent to

```
CFLTNE #<reason>, <Wt>, #0
```

Encoding for the 64-bit not equal variant

Applies when `(sf == 1)`

```
CFLTNZ #<reason>, <Xt>
```

is equivalent to

```
CFLTNE #<reason>, <Xt>, #0
```

Assembler Symbols

<reason>	Is a 2-bit unsigned immediate, in the range 0 to 3, encoded in the "imm2" field.
<Wt>	Is the 32-bit name of the general-purpose register to be tested, encoded in the "Rt" field.
<Xt>	Is the 64-bit name of the general-purpose register to be tested, encoded in the "Rt" field.

Operation

The description of CFLT<cc> (immediate) gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

no old file htmldiff from- (new)

CFLTZ

Compare and fault on zero

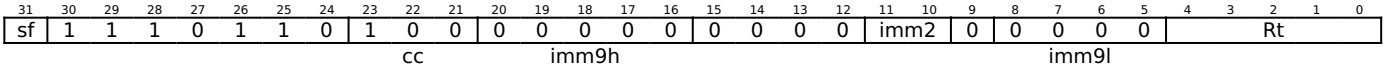
This instruction generates a Conditional fault exception if the register value is zero. If an exception is generated, the <reason> operand is copied to *ESR_ELx*.
This instruction does not affect the condition flags.

This is a pseudo-instruction of CFLT<cc> (immediate). This means:

- The encodings in this description are named to match the encodings of CFLT<cc> (immediate).
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of CFLT<cc> (immediate) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Fault

(FEAT_CFLT)



Encoding for the 32-bit equal variant

Applies when `(sf == 0)`

```
CFLTZ #<reason>, <Wt>
```

is equivalent to

```
CFLTEQ #<reason>, <Wt>, #0
```

Encoding for the 64-bit equal variant

Applies when `(sf == 1)`

```
CFLTZ #<reason>, <Xt>
```

is equivalent to

```
CFLTEQ #<reason>, <Xt>, #0
```

Assembler Symbols

<reason>	Is a 2-bit unsigned immediate, in the range 0 to 3, encoded in the "imm2" field.
<Wt>	Is the 32-bit name of the general-purpose register to be tested, encoded in the "Rt" field.
<Xt>	Is the 64-bit name of the general-purpose register to be tested, encoded in the "Rt" field.

Operation

The description of CFLT<cc> (immediate) gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

no old file htmldiff from- (new)

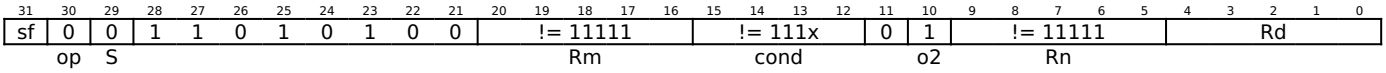
CINC

Conditional increment

This instruction returns, in the destination register, the value of the source register incremented by 1 if the condition is TRUE, and otherwise returns the value of the source register.

This is an alias of CSINC. This means:

- The encodings in this description are named to match the encodings of CSINC.
- The description of CSINC gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
CINC <Wd>, <Wn>, <invcond>
```

is equivalent to

```
CSINC <Wd>, <Wn>, <Wm>, <cond>
```

and is the preferred disassembly when `Rn == Rm`.

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
CINC <Xd>, <Xn>, <invcond>
```

is equivalent to

```
CSINC <Xd>, <Xn>, <Xm>, <cond>
```

and is the preferred disassembly when `Rn == Rm`.

Assembler Symbols

<Wd> Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Wn> Is the 32-bit name of the general-purpose source register, encoded in the "Rn" and "Rm" fields.

<invcond> Is one of the standard conditions, excluding AL and NV, encoded with its least significant bit inverted, and encoded in “cond”:

cond	<invcond>	Description
0000	NE	Maps to <cond> EQ.
0001	EQ	Maps to <cond> NE.
0010	CC|LO	Maps to <cond> CS.
0011	CS|HS	Maps to <cond> CC.
0100	PL	Maps to <cond> MI.
0101	MI	Maps to <cond> PL.
0110	VC	Maps to <cond> VS.
0111	VS	Maps to <cond> VC.
1000	LS	Maps to <cond> HI.
1001	HI	Maps to <cond> LS.
1010	LT	Maps to <cond> GE.
1011	GE	Maps to <cond> LT.
1100	LE	Maps to <cond> GT.
1101	GT	Maps to <cond> LE.

<Xd> Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Xn> Is the 64-bit name of the general-purpose source register, encoded in the "Rn" and "Rm" fields.

Operation

The description of CSINC gives the operational pseudocode for this instruction.

Operational Information

The description of CSINC gives the operational information for this instruction.

2026-09_rel2026-06_diff_tag 2026-09-212026-09-08 1409:2244:3801

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

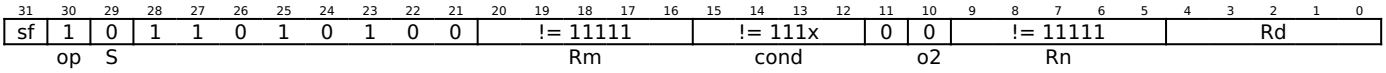
CINV

Conditional invert

This instruction returns, in the destination register, the bitwise inversion of the value of the source register if the condition is TRUE, and otherwise returns the value of the source register.

This is an alias of CSINV. This means:

- The encodings in this description are named to match the encodings of CSINV.
- The description of CSINV gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
CINV <Wd>, <Wn>, <invcond>
```

is equivalent to

```
CSINV <Wd>, <Wn>, <Wm>, <cond>
```

and is the preferred disassembly when `Rn == Rm`.

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
CINV <Xd>, <Xn>, <invcond>
```

is equivalent to

```
CSINV <Xd>, <Xn>, <Xm>, <cond>
```

and is the preferred disassembly when `Rn == Rm`.

Assembler Symbols

<Wd> Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Wn> Is the 32-bit name of the general-purpose source register, encoded in the "Rn" and "Rm" fields.

<invcond> Is one of the standard conditions, excluding AL and NV, encoded with its least significant bit inverted, and encoded in “cond”:

cond	<invcond>	Description
0000	NE	Maps to <cond> EQ.
0001	EQ	Maps to <cond> NE.
0010	CC|LO	Maps to <cond> CS.
0011	CS|HS	Maps to <cond> CC.
0100	PL	Maps to <cond> MI.
0101	MI	Maps to <cond> PL.
0110	VC	Maps to <cond> VS.
0111	VS	Maps to <cond> VC.
1000	LS	Maps to <cond> HI.
1001	HI	Maps to <cond> LS.
1010	LT	Maps to <cond> GE.
1011	GE	Maps to <cond> LT.
1100	LE	Maps to <cond> GT.
1101	GT	Maps to <cond> LE.

<Xd> Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Xn> Is the 64-bit name of the general-purpose source register, encoded in the "Rn" and "Rm" fields.

Operation

The description of CSINV gives the operational pseudocode for this instruction.

Operational Information

The description of CSINV gives the operational information for this instruction.

2026-09_rel2026-06_diff_tag 2026-09-212026-09-08 1409:2244:3801

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

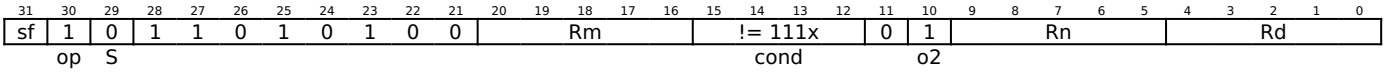
CNEG

Conditional negate

This instruction returns, in the destination register, the negated value of the source register if the condition is TRUE, and otherwise returns the value of the source register.

This is an alias of CSNEG. This means:

- The encodings in this description are named to match the encodings of CSNEG.
- The description of CSNEG gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
CNEG <Wd>, <Wn>, <invcond>
```

is equivalent to

```
CSNEG <Wd>, <Wn>, <Wm>, <cond>
```

and is the preferred disassembly when `Rn == Rm`.

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
CNEG <Xd>, <Xn>, <invcond>
```

is equivalent to

```
CSNEG <Xd>, <Xn>, <Xm>, <cond>
```

and is the preferred disassembly when `Rn == Rm`.

Assembler Symbols

<Wd> Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Wn> Is the 32-bit name of the general-purpose source register, encoded in the "Rn" and "Rm" fields.

<invcond> Is one of the standard conditions, excluding AL and NV, encoded with its least significant bit inverted, and encoded in “cond”:

cond	<invcond>	Description
0000	NE	Maps to <cond> EQ.
0001	EQ	Maps to <cond> NE.
0010	CC|LO	Maps to <cond> CS.
0011	CS|HS	Maps to <cond> CC.
0100	PL	Maps to <cond> MI.
0101	MI	Maps to <cond> PL.
0110	VC	Maps to <cond> VS.
0111	VS	Maps to <cond> VC.
1000	LS	Maps to <cond> HI.
1001	HI	Maps to <cond> LS.
1010	LT	Maps to <cond> GE.
1011	GE	Maps to <cond> LT.
1100	LE	Maps to <cond> GT.
1101	GT	Maps to <cond> LE.

<Xd> Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Xn> Is the 64-bit name of the general-purpose source register, encoded in the "Rn" and "Rm" fields.

Operation

The description of CSNEG gives the operational pseudocode for this instruction.

Operational Information

The description of CSNEG gives the operational information for this instruction.

2026-09_rel2026-06_diff_tag 2026-09-212026-09-08 1409:2244:3801

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

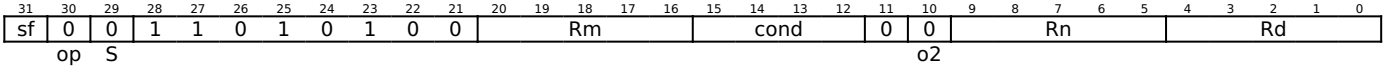
(old) htmldiff from- (new)

(old) htmldiff from- (new)

CSEL

Conditional select

This instruction writes the value of the first source register to the destination register if the condition is TRUE. If the condition is FALSE, it writes the value of the second source register to the destination register.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
CSEL <Wd>, <Wn>, <Wm>, <cond>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
CSEL <Xd>, <Xn>, <Xm>, <cond>
```

Decode for all variants of this encoding

```
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 32 << UInt(sf);
let condition : bits(4) = cond;
```

Assembler Symbols

<Wd> Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Wn> Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Wm> Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.

<cond> Is one of the standard conditions, encoded in the standard way, and encoded in “cond”:

cond	<cond>
0000	EQ
0001	NE
0010	CS|HS
0011	CC|LO
0100	MI
0101	PL
0110	VS
0111	VC
1000	HI
1001	LS
1010	GE
1011	LT
1100	GT
1101	LE
1110	AL
1111	NV

<Xd> Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Xn> Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.

Operation

```
var result : bits(datasize);
if ConditionHolds(condition) then
    result = X{datasize}(n);
else
    result = X{datasize}(m);
end;

X{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel2026-06_diff_tag 2026-09-212026-09-08 1409:2244:3801

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

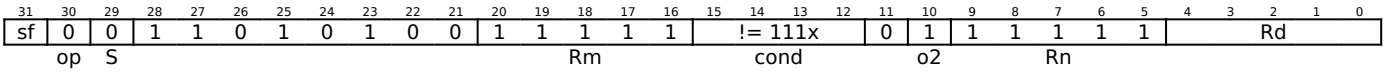
CSET

Conditional set

This instruction sets the destination register to 1 if the condition is TRUE, and otherwise sets it to 0.

This is an alias of CSINC. This means:

- The encodings in this description are named to match the encodings of CSINC.
- The description of CSINC gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
CSET <Wd>, <invcond>
```

is equivalent to

```
CSINC <Wd>, WZR, WZR, <cond>
```

and is always the preferred disassembly.

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
CSET <Xd>, <invcond>
```

is equivalent to

```
CSINC <Xd>, XZR, XZR, <cond>
```

and is always the preferred disassembly.

Assembler Symbols

<Wd> Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.

<invcond> Is one of the standard conditions, excluding AL and NV, encoded with its least significant bit inverted, and encoded in “cond”:

cond	<invcond>	Description
0000	NE	Maps to <cond> EQ.
0001	EQ	Maps to <cond> NE.
0010	CC|LO	Maps to <cond> CS.
0011	CS|HS	Maps to <cond> CC.
0100	PL	Maps to <cond> MI.
0101	MI	Maps to <cond> PL.
0110	VC	Maps to <cond> VS.
0111	VS	Maps to <cond> VC.
1000	LS	Maps to <cond> HI.
1001	HI	Maps to <cond> LS.
1010	LT	Maps to <cond> GE.
1011	GE	Maps to <cond> LT.
1100	LE	Maps to <cond> GT.
1101	GT	Maps to <cond> LE.

<Xd> Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

Operation

The description of CSINC gives the operational pseudocode for this instruction.

Operational Information

The description of CSINC gives the operational information for this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

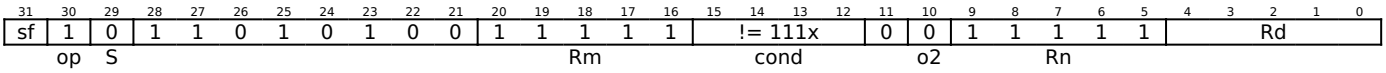
CSETM

Conditional set mask

This instruction sets all bits of the destination register to 1 if the condition is TRUE, and otherwise sets all bits to 0.

This is an alias of CSINV. This means:

- The encodings in this description are named to match the encodings of CSINV.
- The description of CSINV gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
CSETM <Wd>, <invcond>
```

is equivalent to

```
CSINV <Wd>, WZR, WZR, <cond>
```

and is always the preferred disassembly.

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
CSETM <Xd>, <invcond>
```

is equivalent to

```
CSINV <Xd>, XZR, XZR, <cond>
```

and is always the preferred disassembly.

Assembler Symbols

<Wd> Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.

<invcond> Is one of the standard conditions, excluding AL and NV, encoded with its least significant bit inverted, and encoded in “cond”:

cond	<invcond>	Description
0000	NE	Maps to <cond> EQ.
0001	EQ	Maps to <cond> NE.
0010	CC|LO	Maps to <cond> CS.
0011	CS|HS	Maps to <cond> CC.
0100	PL	Maps to <cond> MI.
0101	MI	Maps to <cond> PL.
0110	VC	Maps to <cond> VS.
0111	VS	Maps to <cond> VC.
1000	LS	Maps to <cond> HI.
1001	HI	Maps to <cond> LS.
1010	LT	Maps to <cond> GE.
1011	GE	Maps to <cond> LT.
1100	LE	Maps to <cond> GT.
1101	GT	Maps to <cond> LE.

<Xd> Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

Operation

The description of CSINV gives the operational pseudocode for this instruction.

Operational Information

The description of CSINV gives the operational information for this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

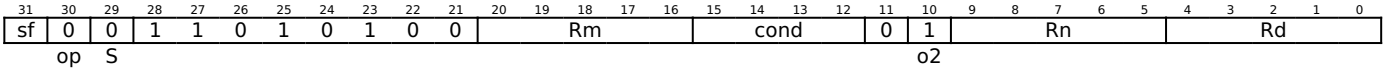
(old) htmldiff from- (new)

CSINC

Conditional select increment

This instruction returns, in the destination register, the value of the first source register if the condition is TRUE, and otherwise returns the value of the second source register incremented by 1.

This instruction is used by the aliases CINC, and CSET.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
CSINC <Wd>, <Wn>, <Wm>, <cond>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
CSINC <Xd>, <Xn>, <Xm>, <cond>
```

Decode for all variants of this encoding

```
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 32 << UInt(sf);
let condition : bits(4) = cond;
```

Assembler Symbols

<Wd> Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Wn> Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Wm> Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.

<cond> Is one of the standard conditions, encoded in the standard way, and encoded in “cond”:

cond	<cond>
0000	EQ
0001	NE
0010	CS|HS
0011	CC|LO
0100	MI
0101	PL
0110	VS
0111	VC
1000	HI
1001	LS
1010	GE
1011	LT
1100	GT
1101	LE
1110	AL
1111	NV

<Xd> Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Xn> Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.

Alias Conditions

Alias	Is preferred when
CINC	`Rm != '11111' && Rn != '11111' && !(cond IN {'111x'}) && Rn == Rm`
CSET	`Rm == '11111' && Rn == '11111' && !(cond IN {'111x'})`

Operation

```
var result : bits(datasize);
if ConditionHolds(condition) then
    result = X{datasize}(n);
else
    result = X{datasize}(m) + 1;
end;

X{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

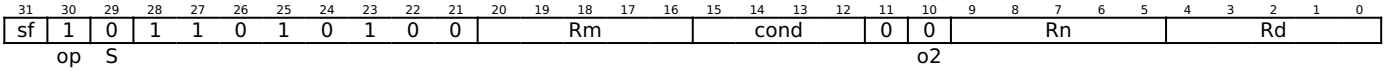
(old) htmldiff from- (new)

CSINV

Conditional select invert

This instruction returns, in the destination register, the value of the first source register if the condition is TRUE, and otherwise returns the bitwise inversion value of the second source register.

This instruction is used by the aliases CINV, and CSETM.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
CSINV <Wd>, <Wn>, <Wm>, <cond>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
CSINV <Xd>, <Xn>, <Xm>, <cond>
```

Decode for all variants of this encoding

```
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 32 << UInt(sf);
let condition : bits(4) = cond;
```

Assembler Symbols

<Wd> Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Wn> Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Wm> Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.

<cond> Is one of the standard conditions, encoded in the standard way, and encoded in “cond”:

cond	<cond>
0000	EQ
0001	NE
0010	CS|HS
0011	CC|LO
0100	MI
0101	PL
0110	VS
0111	VC
1000	HI
1001	LS
1010	GE
1011	LT
1100	GT
1101	LE
1110	AL
1111	NV

<Xd> Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Xn> Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.

Alias Conditions

Alias	Is preferred when
CINV	`Rm != '11111' && Rn != '11111' && !(cond IN {'111x'}) && Rn == Rm`
CSETM	`Rm == '11111' && Rn == '11111' && !(cond IN {'111x'})`

Operation

```
var result : bits(datasize);
if ConditionHolds(condition) then
    result = X{datasize}(n);
else
    result = NOT X{datasize}(m);
end;

X{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel2026-06_diff_tag 2026-09-212026-09-08 1409:2244:3801

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

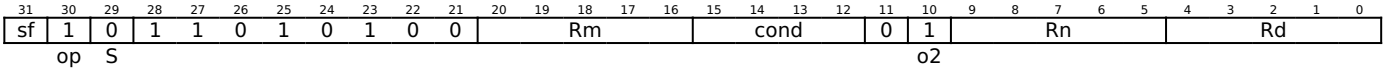
(old) htmldiff from- (new)

CSNEG

Conditional select negation

This instruction returns, in the destination register, the value of the first source register if the condition is TRUE, and otherwise returns the negated value of the second source register.

This instruction is used by the alias CNEG.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
CSNEG <Wd>, <Wn>, <Wm>, <cond>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
CSNEG <Xd>, <Xn>, <Xm>, <cond>
```

Decode for all variants of this encoding

```
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 32 << UInt(sf);
let condition : bits(4) = cond;
```

Assembler Symbols

<Wd> Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Wn> Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Wm> Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.

<cond> Is one of the standard conditions, encoded in the standard way, and encoded in “cond”:

cond	<cond>
0000	EQ
0001	NE
0010	CS|HS
0011	CC|LO
0100	MI
0101	PL
0110	VS
0111	VC
1000	HI
1001	LS
1010	GE
1011	LT
1100	GT
1101	LE
1110	AL
1111	NV

<Xd> Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Xn> Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.

Alias Conditions

Alias	Is preferred when
CNEG	`!(cond IN {'111x'}) && Rn == Rm`

Operation

```
var result : bits(datasize);
if ConditionHolds(condition) then
    result = X{datasize}(n);
else
    result = NOT X{datasize}(m) + 1;
end;

X{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

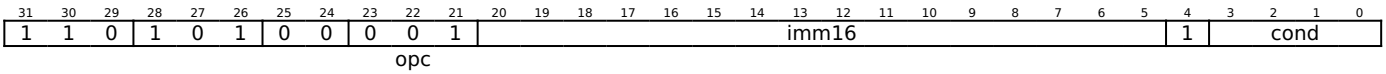
FLT.cond

Conditional fault

This instruction conditionally generates a Conditional fault exception. If an exception is generated, the immediate field is copied to ESR_ELx.

System

(FEAT_CFLT)



Encoding

FLT.<cond> #<imm>

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_CFLT) then EndOfDecode(Decode_UNDEF); end;
let imm : bits(16) = imm16;
let condition : bits(4) = cond;
```

Assembler Symbols

<cond> Is one of the standard conditions, encoded in the standard way, and encoded in “cond”:

cond	<cond>
0000	EQ
0001	NE
0010	CS|HS
0011	CC|LO
0100	MI
0101	PL
0110	VS
0111	VC
1000	HI
1001	LS
1010	GE
1011	LT
1100	GT
1101	LE
1110	AL
1111	NV

<imm> Is a 16-bit unsigned immediate, in the range 0 to 65535, encoded in the "imm16" field.

Operation

```
CheckCFLTEnabled();

if ConditionHolds(condition) then
    let isflt : boolean = TRUE;
    AArch64_ConditionalFault(isflt, imm);
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

no old file htmldiff from- (new)

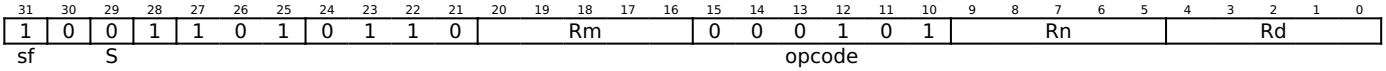
GMI

Tag mask insert

This instruction inserts the tag in the first source register into the excluded set specified in the second source register, writing the new excluded set to the destination register.

Integer

(FEAT_MTE)



Encoding

```
GMI <Xd>, <Xn|SP>, <Xm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_MTE) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
```

Assembler Symbols

<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn|SP>	Is the 64-bit name of the first source general-purpose register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.

Operation

```
let address : bits(64) = if n == 31 then SP{64}() else X{64}(n);
var mask : bits(64) =var mask : bits(64) = X{}(m);
let tag : bits(4) = X{64}(m);
let tag : bits(4) = AArch64_AllocationTagFromAddress(address);

mask[UInt(tag)] = '1';
X{64}(d) = mask;
```

2026-09_rel2026-06_diff_tag 2026-09-212026-09-08 1409:2244:3801

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

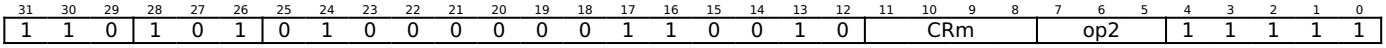
(old) htmldiff from- (new)

HINT

Hint instruction

This instruction is for the instruction set space that is reserved for architectural hint instructions.

Some encodings described here are not allocated in this revision of the architecture, and behave as NOPs. These encodings might be allocated to other hint functionality in future revisions of the architecture and therefore must not be used by software.



Encoding

```
HINT #<imm>
```

Decode for this encoding

```
var op : SystemHintOp;

var stream : boolean;
var priority : boolean;
var stshstrm : boolean;
case CRm::op2 of
    when '0000 000' => op = SystemHintOp_NOP;
    when '0000 001' => op = SystemHintOp_YIELD;
    when '0000 010' => op = SystemHintOp_WFE;
    when '0000 011' => op = SystemHintOp_WFI;
    when '0000 100' => op = SystemHintOp_SEV;
    when '0000 101' => op = SystemHintOp_SEVL;
    when '0000 110' =>
        if !IsFeatureImplemented(FEAT_DGH) then EndOfDecode(Decode_NOP); end;
        op = SystemHintOp_DGH;
    when '0000 111' => See("XPACLRI");
    when '0001 xxx' =>
        case op2 of
            when '000' => See("PACIA1716");
            when '010' => See("PACIB1716");
            when '100' => See("AUTIA1716");
            when '110' => See("AUTIB1716");
            otherwise => EndOfDecode(Decode_NOP);
        end;
    when '0010 000' =>
        if !IsFeatureImplemented(FEAT_RAS) then EndOfDecode(Decode_NOP); end;
        op = SystemHintOp_ESB;
    when '0010 001' =>
        if !IsFeatureImplemented(FEAT_SPE) then EndOfDecode(Decode_NOP); end;
        op = SystemHintOp_PSB;
    when '0010 010' =>
        if !IsFeatureImplemented(FEAT_TRF) then EndOfDecode(Decode_NOP); end;
        op = SystemHintOp_TSB;
    when '0010 011' =>
        if !IsFeatureImplemented(FEAT_GCS) then EndOfDecode(Decode_NOP); end;
        op = SystemHintOp_GCSB;
    when '0010 100' =>
        op = SystemHintOp_CSDB;
    when '0010 110' =>
        if !IsFeatureImplemented(FEAT_CLRBHB) then EndOfDecode(Decode_NOP); end;
        op = SystemHintOp_CLRBHB;
    when '0011 xxx' =>
        case op2 of
            when '000' => See("PACIAZ");
            when '001' => See("PACIASP");
            when '010' => See("PACIBZ");
            when '011' => See("PACIBSP");
            when '100' => See("AUTIAZ");
            when '101' => See("AUTIASP");
            when '110' => See("AUTIBZ");
            when '111' => See("AUTIBSP");
        end;
    when '0100 xx0' =>
        if !IsFeatureImplemented(FEAT_BTI) then EndOfDecode(Decode_NOP); end;

        // Check branch target compatibility between BTI instruction and PSTATE.BTYPE
        SetBTypeCompatible(BTypeCompatible_BTI(op2[2:1]));
        op = SystemHintOp_BTI;
    when '0100 111' => See("PACM");
    when '0101 000' =>
        if !IsFeatureImplemented(FEAT_CHK) then EndOfDecode(Decode_NOP); end;
        op = SystemHintOp_CHKFEAT;
    when '0110 00x' =>
        if !IsFeatureImplemented(FEAT_PCDPHINT) then EndOfDecode(Decode_NOP); end;
        stream = op2[0] == '1';
        op = SystemHintOp_STSHH;
    when '0110 01x' =>
        if !IsFeatureImplemented(FEAT_CMH) then EndOfDecode(Decode_NOP); end;
        priority = op2[0] == '1';

        op = SystemHintOp_SHUH;
    when '0110 100' =>
        if !IsFeatureImplemented(FEAT_CMH) then EndOfDecode(Decode_NOP); end;
        op = SystemHintOp_STCPH;
    when '0110 11x' =>
        if !IsFeatureImplemented(FEAT_RLCS) then    otherwise => EndOfDecode(Decode_NOP); end;
        stshstrm = op2[0] == '1';

        op =};
end; SystemHintOp_SRLS;
    when '0111 000' =>
        if !IsFeatureImplemented(FEAT_RLCS) then EndOfDecode(Decode_NOP); end;
        op = SystemHintOp_SLBND;
    otherwise => EndOfDecode(Decode_NOP);
end;
```

Assembler Symbols

<imm> Is a 7-bit unsigned immediate, in the range 0 to 127, encoded in the "CRm:op2" field. The encodings that are allocated to architectural hint functionality are described in the 'Hints' table in the 'Index by Encoding'.

Note

For allocated encodings of "CRm:op2":

- A disassembler will disassemble the allocated instruction, rather than the `HINT` instruction.
- An assembler may support assembly of allocated encodings using `HINT` with the corresponding <imm> value, but it is not required to do so.

Operation

```
case op of
    when SystemHintOp_YIELD =>
        Hint_Yield();

    when SystemHintOp_DGH =>
        Hint_DGH();

    when SystemHintOp_WFE =>
        Hint_WFE();

    when SystemHintOp_WFI =>
        Hint_WFI();

    when SystemHintOp_SEV =>
        SendEvent();

    when SystemHintOp_SEVL =>
        SendEventLocal();

    when SystemHintOp_ESB =>
        SynchronizeErrors();
        AArch64_ESBOperation();
        if PSTATE.EL IN {EL0, EL1} && EL2Enabled() then
            AArch64_vESBOperation();
        elsif IsFeatureImplemented(FEAT_E3DSE) && PSTATE.EL != EL3 then
            AArch64_dESBOperation();
        end;
        TakeUnmaskedSErrorInterrupts();

    when SystemHintOp_PSB =>
        CheckPSBTrap();

        ProfilingSynchronizationBarrier();

    when SystemHintOp_TSB =>
        CheckTSBTrap();

        TraceSynchronizationBarrier();

    when SystemHintOp_GCSB =>
        GCSSynchronizationBarrier();

    when SystemHintOp_CHKFEAT =>
        X{64}(16) = AArch64_ChkFeat(X{64}(16));

    when SystemHintOp_CSDB =>
        ConsumptionOfSpeculativeDataBarrier();

    when SystemHintOp_CLRBHB =>
        Hint_CLRBHB();

    when SystemHintOp_BTI =>
        SetBTypeNext('00');

    when SystemHintOp_STSHH =>
        Hint_StoreShared(stream);

    when SystemHintOp_SHUH =>
        Hint_SharedUpdate(priority);

    when SystemHintOp_STCPH =>
        Hint_StoreConcurrentPriority();

    when SystemHintOp_SLBND =>
        Hint_ScopedReleaseBoundary();

    when SystemHintOp_SRLS =>
        Hint_ScopeRelease(stshstrm);

    when SystemHintOp_NOP =>
        return; // Do nothing

    otherwise =>
        unreachable;
end;
```

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

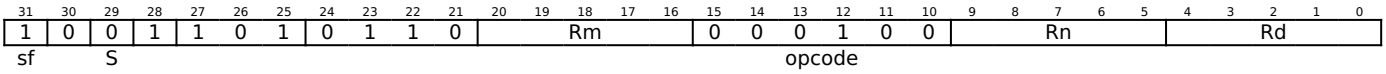
IRG

Insert random tag

This instruction inserts a random Logical Address Tag into the address in the first source register, and writes the result to the destination register. Any tags specified in the optional second source register or in GCR_EL1.Exclude are excluded from the selection of the random Logical Address Tag.

Integer

(FEAT_MTE)



Encoding

```
IRG <Xd|SP>, <Xn|SP>{, <Xm>}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_MTE) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
```

Assembler Symbols

<Xd|SP>	Is the 64-bit name of the destination general-purpose register or stack pointer, encoded in the "Rd" field.
<Xn|SP>	Is the 64-bit name of the first source general-purpose register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field. Defaults to XZR if absent.

Operation

```
let operand : bits(64) = if n == 31 then SP{64}() else X{64}(n);
let exclude_reg : bits(64) =~~let exclude_reg : bits(64) = X{}(m);~~
~~let exclude : bits(16) = exclude_reg[15:0] OR GCR_EL1().Exclude;~~

~~let rtag : bits(4) =~~ X{64}(m);
let exclude : bits(16) = exclude_reg[15:0] OR GCR_EL1().Exclude;

let rtag : bits(4) = AArch64_ChooseTagOrZero(exclude);
let result : bits(64) = AArch64_AddressWithAllocationTag(operand, rtag);

if d == 31 then
    SP{64}() = result;
else
    X{64}(d) = result;
end;
```

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD64B

Single-copy atomic 64-byte load

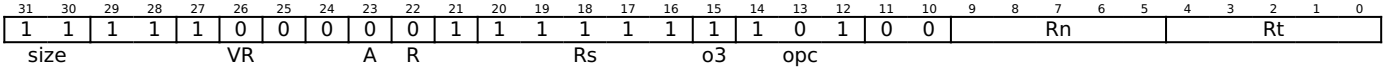
This instruction derives an address from a base register value, loads eight 64-bit doublewords from a memory location, and writes them to consecutive registers. The load starts at register Xt, with the data being read as X(t+7)::X(t+6)::X(t+5)::X(t+4)::X(t+3)::X(t+2)::X(t+1)::Xt = Data[511:0]. The data is loaded atomically and is required to be 64-byte aligned.

It is IMPLEMENTATION DEFINED which memory locations support this instruction. A memory location that supports LD64B also supports LDA64B, ST64B, and STL64B.

For more information, including about the memory types accessible and how the accesses are performed, see *Single-copy atomic 64-byte load/store*.

Integer

(FEAT_LS64)



Encoding

```
LD64B <Xt>, [<Xn|SP> {, #0}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_LS64) then EndOfDecode(Decode_UNDEF); end;
if Rt[4:3] == '11' || Rt[0] == '1' then EndOfDecode(Decode_UNDEF); end;
let withstatus : boolean = FALSE;
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let tagchecked : boolean = n != 31;
let acqrel : boolean = FALSE;let tagchecked : boolean = n != 31;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *LD64B*.

Assembler Symbols

<Xt>	Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
CheckLDST64BEnabled(acqrel);
();

var data : bits(512);
var address : bits(64);
var value : bits(64);

let accdesc : AccessDescriptor = CreateAccDescLS64(MemOp_LOAD, withstatus, acqrel, tagchecked);
, withstatus, tagchecked);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

data = MemLoad64B(address, accdesc);

for i = 0 to 7 do
    value = data[63+64*i : 64*i];
    if BigEndian(accdesc.acctype) then value = BigEndianReverse{64}(value); end;
    X{64}(t+i) = value;
end;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDA64B

Single-copy atomic 64-byte load-acquire

This instruction derives an address from a base register value, loads eight 64-bit doublewords from a memory location, and writes them to consecutive registers. The load starts at register Xt, with the data being read as X(t+7)::X(t+6)::X(t+5)::X(t+4)::X(t+3)::X(t+2)::X(t+1)::Xt = Data[511:0]. The data is loaded atomically and is required to be 64-byte aligned.

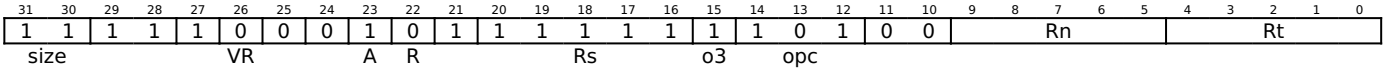
It is IMPLEMENTATION DEFINED which memory locations support this instruction. A memory location that supports LDA64B also supports LD64B, ST64B, and STL64B.

LDA64B loads from memory with acquire semantics.

For more information, including about the memory types accessible and how the accesses are performed, see *Single-copy atomic 64-byte load/store*.

Integer

(FEAT_LS64 && FEAT_LSC64B)



Encoding

```
LDA64B <Xt>, [<Xn|SP> {, #0}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_LSC64B) then EndOfDecode(Decode_UNDEF); end;
if !IsFeatureImplemented(FEAT_LS64) then EndOfDecode(Decode_UNDEF); end;
if Rt[4:3] == '11' || Rt[0] == '1' then EndOfDecode(Decode_UNDEF); end;
let withstatus : boolean = FALSE;
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let tagchecked : boolean = n != 31;
let acqrel : boolean = TRUE;
```

LDA64B has the same CONSTRAINED UNPREDICTABLE behavior as LD64B. For information about this CONSTRAINED UNPREDICTABLE behavior, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *LD64B*.

Assembler Symbols

<Xt>	Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
CheckLDST64BEnabled(acqrel);

var data : bits(512);
var address : bits(64);
var value : bits(64);

let accdesc : AccessDescriptor = CreateAccDescLS64(MemOp_LOAD, withstatus, acqrel, tagchecked);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

data = MemLoad64B(address, accdesc);

for i = 0 to 7 do
    value = data[63+64*i : 64*i];
    if BigEndian(accdesc.acctype) then value = BigEndianReverse{64}(value); end;
    X{64}(t+i) = value;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

no old file htmldiff from- (new)

(old) htmldiff from- (new)

LDAP

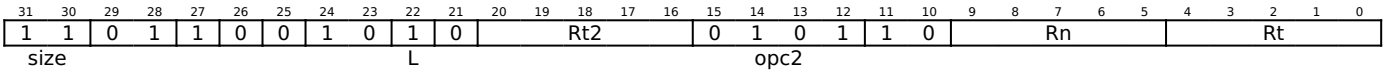
Load-acquire pair of registers

This instruction calculates an address from a base register value, loads two 64-bit doublewords from memory, and writes them to two registers. Explicit Memory effects produced by this instruction have Acquire semantics.~~semantics, unless both destination registers are XZR.~~ For information about memory ordering semantics, see *Load-Acquire, Load-AcquirePC, and Store-Release*.

For information about addressing modes, see *Load/Store addressing modes*.

No offset

(FEAT_LSCP)



Encoding

```
LDAP <Xt1>, <Xt2>, [<Xn|SP>{, #0}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_LSCP) then EndOfDecode(Decode_UNDEF); end;

let t : integer{} = UInt(Rt);
let t2 : integer{} = UInt(Rt2);
let n : integer{} = UInt(Rn);
let acquire : boolean = TRUE;
~~let acquire : boolean = t != 31 && t2 != 31;~~
let tagchecked : boolean = n != 31;
let ispair : boolean = TRUE;
var rt_unknown : boolean = FALSE;

if t == t2 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_LDPOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>     rt_unknown = TRUE;     // Result is UNKNOWN
        when Constraint_UNDEF =>       EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>         EndOfDecode(Decode_NOP);
    end;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *LDP and LDIAPP*, which applies when t==t2.

Assembler Symbols

<Xt1>	Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.
<Xt2>	Is the 64-bit name of the second general-purpose register to be transferred, encoded in the "Rt2" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
var address : bits(64);
var data1 : bits(64);
var data2 : bits(64);
let accdesc : AccessDescriptor = CreateAccDescAcqRel(MemOp_LOAD, tagchecked, ispair,
                                                     acquire, t, t2);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let full_data : bits(128) = Mem{128}(address, accdesc);
if BigEndian(accdesc.acctype) then
    data2 = full_data[63:0];
    data1 = full_data[127:64];
else
    data1 = full_data[63:0];
    data2 = full_data[127:64];
end;

if rt_unknown then
    data1 = ARBITRARY : bits(64);
    data2 = ARBITRARY : bits(64);
end;

X{64}(t) = data1;
X{64}(t2) = data2;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

LDAPP

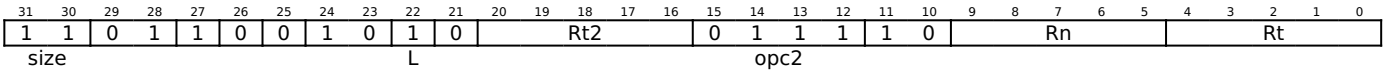
Load-acquire RCpc pair of registers

This instruction calculates an address from a base register value, loads two 64-bit doublewords from memory, and writes them to two registers. Explicit Memory effects produced by this instruction have AcquirePC semantics.~~semantics, unless both destination registers are XZR.~~ For information about memory ordering semantics, see *Load-Acquire, Load-AcquirePC, and Store-Release*.

For information about addressing modes, see *Load/Store addressing modes*.

No offset

(FEAT_LSCP)



Encoding

```
LDAPP <Xt1>, <Xt2>, [<Xn|SP>{, #0}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_LSCP) then EndOfDecode(Decode_UNDEF); end;

let t : integer{} = UInt(Rt);
let t2 : integer{} = UInt(Rt2);
let n : integer{} = UInt(Rn);
let acquire : boolean = TRUE;
~~let acquire : boolean = t != 31 && t2 != 31;~~
let tagchecked : boolean = n != 31;
let ispair : boolean = TRUE;
var rt_unknown : boolean = FALSE;

if t == t2 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_LDPOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>     rt_unknown = TRUE;     // Result is UNKNOWN
        when Constraint_UNDEF =>       EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>         EndOfDecode(Decode_NOP);
    end;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *LDP and LDIAPP*, which applies when t==t2.

Assembler Symbols

<Xt1>	Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.
<Xt2>	Is the 64-bit name of the second general-purpose register to be transferred, encoded in the "Rt2" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
var address : bits(64);
var data1 : bits(64);
var data2 : bits(64);
let accdesc : AccessDescriptor = CreateAccDescLDAcqPC(tagchecked, ispair, acquire, t, t2);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let full_data : bits(128) = Mem{128}(address, accdesc);
if BigEndian(accdesc.acctype) then
    data2 = full_data[63:0];
    data1 = full_data[127:64];
else
    data1 = full_data[63:0];
    data2 = full_data[127:64];
end;

if rt_unknown then
    data1 = ARBITRARY : bits(64);
    data2 = ARBITRARY : bits(64);
end;

X{64}(t) = data1;
X{64}(t2) = data2;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:2244:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

LDGM

Load tag multiple

This instruction reads a naturally aligned block of N Allocation Tags, where the size of N is identified in GMID_EL1.BS, and writes the Allocation Tag read from address A to the destination register at 4*A<7:4>+3:4*A<7:4>. Bits of the destination register not written with an Allocation Tag are set to 0.

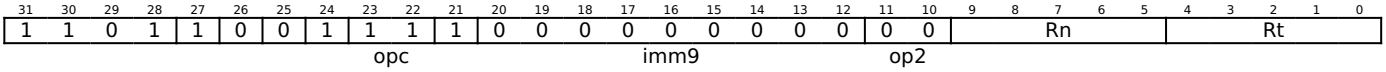
If the value of HCR_EL2.TGE is 0 and the Effective value of SCTLR2_EL1.GM0 is 0, this~~This~~ instruction is UNDEFINED at EL0.

If the value of HCR_EL2.TGE is 1 and the Effective value of SCTLR2_EL2.GM0 is 0, this instruction is UNDEFINED at EL0.

This instruction generates an Unchecked access.

Integer

(FEAT_MTE2)



Encoding

```
LDGM <Xt>, [<Xn|SP>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_MTE2) then EndOfDecode(Decode_UNDEF); end;
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
```

Assembler Symbols

<Xt>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
if PSTATE.EL == EL0 && !~~then Undefined(); end;~~

~~var data : bits(64) = Zeros{};~~
~~var address : bits(64);~~

~~if n == 31 then~~IsGMEnabledAtEL0() then Undefined(); end;

var data : bits(64) = Zeros{};
var address : bits(64);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let size : integer{} = 4 * 2 ^ UInt(GMID_EL1().BS);
~~let size : integer{} = 4 * (2 ^ (UInt(GMID_EL1().BS)));~~
address = AlignDownSize(address, size);
let count : integer = size >> LOG2_TAG_GRANULE;
var index : integer = UInt(address[LOG2_TAG_GRANULE+3:LOG2_TAG_GRANULE]);
let stzgm : boolean = FALSE;
let accdesc : AccessDescriptor = CreateAccDescLDGSTG(MemOp_LOAD, stzgm, t);

for i = 0 to count-1 do
    let tag : bits(4) = AArch64_MemTag(address, accdesc);
    data[index*:4] = tag;
    address = AddressIncrement(address, TAG_GRANULE, accdesc);
    index = index + 1;
end;

X{64}(t) = data;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

LDIAPP

Load-Acquire RCpc ordered pair of registers

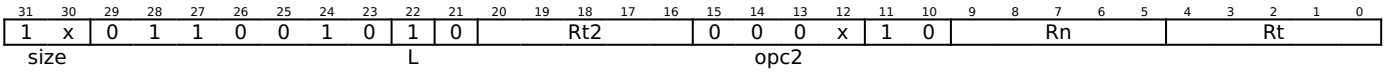
This instruction calculates an address from a base register value and an optional offset, loads two 32-bit words or two 64-bit doublewords from memory, and writes them to two registers. For information on single-copy atomicity and alignment requirements, see *Requirements for single-copy atomicity* and *Alignment of data accesses*. The instruction also has memory ordering semantics, as described in *Load-Acquire, Load-AcquirePC, and Store-Release*, with the additional requirement that:

- The Memory effects associated with Xt1/Wt1 are Ordered-before the Memory effects associated with Xt2/Wt2.
- `LDIAPP` loads from memory with Acquire semantics.
- ~~There is no ordering requirement, separate from the requirements of a Load-AcquirePC or a Store-Release, created by having a Store-Release followed by a Load-AcquirePC instruction.~~
- ~~The reading of a value written by a Store-Release by a Load-AcquirePC instruction by the same observer does not make the write of the Store-Release globally observed.~~

For information about addressing modes, see *Load/Store addressing modes*.

Integer

(FEAT_LRCPC3)



Encoding for the 32-bit post-index variant

Applies when `(size == 10 && opc2 == 0000)`

```
LDIAPP <Wt1>, <Wt2>, [<Xn|SP>], #8
```

Encoding for the 32-bit variant

Applies when `(size == 10 && opc2 == 0001)`

```
LDIAPP <Wt1>, <Wt2>, [<Xn|SP>]
```

Encoding for the 64-bit post-index variant

Applies when `(size == 11 && opc2 == 0000)`

```
LDIAPP <Xt1>, <Xt2>, [<Xn|SP>], #16
```

Encoding for the 64-bit variant

Applies when `(size == 11 && opc2 == 0001)`

```
LDIAPP <Xt1>, <Xt2>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LRCPC3) then EndOfDecode(Decode_UNDEF); end;
let ispair : boolean = TRUE;
let postindex : boolean = opc2[0] == '0';
var wback : boolean = opc2[0] == '0';
```

`LDIAPP` has the same CONSTRAINED UNPREDICTABLE behavior as `LDP`. For information about this CONSTRAINED UNPREDICTABLE behavior, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *LDP and LDIAPP*.

Assembler Symbols

<Wt1>	Is the 32-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.
<Wt2>	Is the 32-bit name of the second general-purpose register to be transferred, encoded in the "Rt2" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xt1>	Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.
<Xt2>	Is the 64-bit name of the second general-purpose register to be transferred, encoded in the "Rt2" field.

Shared Decode

```
let t : integer{} = UInt(Rt);
let t2 : integer{} = UInt(Rt2);
let n : integer{} = UInt(Rn);
let scale : integer{} = 2 + UInt(size[0]);
let datasize : integer{} = 8 << scale;
let offset : integer = if opc2[0] == '0' then 2 << scale else 0;
let acqrel : boolean = TRUE;
let tagchecked : boolean = wback || n != 31;

var rt_unknown : boolean = FALSE;

var wb_unknown : boolean = FALSE;
if wback && (t == n || t2 == n) && n != 31 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_WBOVERLAPLD);
    assert c IN {Constraint_WBSUPPRESS, Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_WBSUPPRESS => wback = FALSE;        // writeback is suppressed
        when Constraint_UNKNOWN =>    wb_unknown = TRUE;    // writeback is UNKNOWN
        when Constraint_UNDEF =>      EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>        EndOfDecode(Decode_NOP);
    end;
end;

if t == t2 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_LDPOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN => rt_unknown = TRUE;    // result is UNKNOWN
        when Constraint_UNDEF =>   EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>     EndOfDecode(Decode_NOP);
    end;
end;
```

Operation

```
var address : bits(64);
var data1 : bits(datasize);
var data2 : bits(datasize);
let accdesc : AccessDescriptor = CreateAccDescLDAcqPC(tagchecked, ispair, acqrel, t, t2);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

if !postindex then
    address = AddressAdd(address, offset, accdesc);
end;

let full_data : bits(2*datasize) = Mem{2*datasize}(address, accdesc);
if BigEndian(accdesc.acctype) then
    data2 = full_data[(datasize-1):0];
    data1 = full_data[(2*datasize-1):datasize];
else
    data1 = full_data[(datasize-1):0];
    data2 = full_data[(2*datasize-1):datasize];
end;

if rt_unknown then
    data1 = ARBITRARY : bits(datasize);
    data2 = ARBITRARY : bits(datasize);
end;

X{datasize}(t) = data1;
X{datasize}(t2) = data2;

if wback then
    if wb_unknown then
        address = ARBITRARY : bits(64);
    elsif postindex then
        address = AddressAdd(address, offset, accdesc);
    end;
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

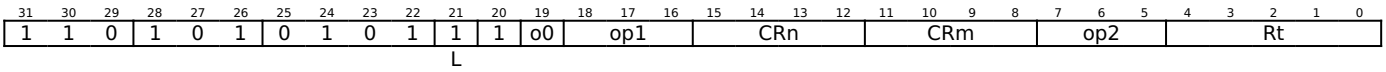
MRRS

Move System register to two adjacent general-purpose registers

This instruction allows the PE to read an AArch64 128-bit System register into two adjacent 64-bit general-purpose registers.

System

(FEAT_SYSREG128)



Encoding

```
MRRS <Xt>, <Xt+1>, (<systemreg>|S<op0>_<op1>_<Cn>_<Cm>_<op2>)
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SYSREG128) then EndOfDecode(Decode_UNDEF); end;
if Rt[0] == '1' then EndOfDecode(Decode_UNDEF); end;

let t : integer       = UInt(Rt);
let t2 : integer      = UInt(Rt+1);
let sys_L : bits(1)   = L;
let sys_op0 : bits(2) = '1' :: o0;
let sys_op1 : bits(3) = op1;
let sys_op2 : bits(3) = op2;
let sys_crn : bits(4) = CRn;
let sys_crm : bits(4) = CRm;
```

Assembler Symbols

<Xt>	Is the 64-bit name of the first general-purpose destination register, encoded in the "Rt" field.
<Xt+1>	Is the 64-bit name of the second general-purpose destination register, encoded as "Rt" +1.
<systemreg>	Is a System register name, encoded in "o0:op1:CRn:CRm:op2".
<op0>	Is an unsigned immediate, encoded in “o0”:

o0	<op0>
0	2
1	3

<op1>	Is a 3-bit unsigned immediate, in the range 0 to 7, encoded in the "op1" field.
<Cn>	Is a name 'Cn', with 'n' in the range 0 to 15, encoded in the "CRn" field.
<Cm>	Is a name 'Cm', with 'm' in the range 0 to 15, encoded in the "CRm" field.
<op2>	Is a 3-bit unsigned immediate, in the range 0 to 7, encoded in the "op2" field.

Operation

```
AArch64_SysRegRead128(sys_op0, sys_op1, sys_crn, sys_crm, sys_op2, t, t2);
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

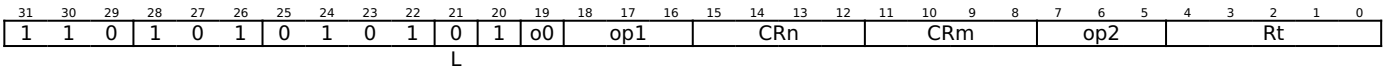
MSRR

Move two adjacent general-purpose registers to System register

This instruction allows the PE to write an AArch64 128-bit System register from two adjacent 64-bit general-purpose registers.

System

(FEAT_SYSREG128)



Encoding

```
MSRR (<systemreg>|S<op0>_<op1>_<Cn>_<Cm>_<op2>), <Xt>, <Xt+1>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SYSREG128) then EndOfDecode(Decode_UNDEF); end;
if Rt[0] == '1' then EndOfDecode(Decode_UNDEF); end;

let t : integer       = UInt(Rt);
let t2 : integer      = UInt(Rt+1);
let sys_L : bits(1)   = L;
let sys_op0 : bits(2) = '1' :: o0;
let sys_op1 : bits(3) = op1;
let sys_op2 : bits(3) = op2;
let sys_crn : bits(4) = CRn;
let sys_crm : bits(4) = CRm;
```

Assembler Symbols

<systemreg>	Is a System register name, encoded in "o0:op1:CRn:CRm:op2".
<op0>	Is an unsigned immediate, encoded in “o0”:

o0	<op0>
0	2
1	3

<op1>	Is a 3-bit unsigned immediate, in the range 0 to 7, encoded in the "op1" field.
<Cn>	Is a name 'Cn', with 'n' in the range 0 to 15, encoded in the "CRn" field.
<Cm>	Is a name 'Cm', with 'm' in the range 0 to 15, encoded in the "CRm" field.
<op2>	Is a 3-bit unsigned immediate, in the range 0 to 7, encoded in the "op2" field.
<Xt>	Is the 64-bit name of the first general-purpose source register, encoded in the "Rt" field.
<Xt+1>	Is the 64-bit name of the second general-purpose source register, encoded as "Rt" +1.

Operation

```
AArch64_SysRegWrite128(sys_op0, sys_op1, sys_crn, sys_crm, sys_op2, t, t2);
```

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

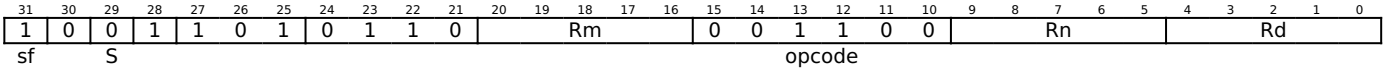
PACGA

Pointer Authentication Code, using generic key

This instruction calculates the Pointer Authentication Code for a 64-bit value in the first source register, using a modifier in the second source register, and the generic key. The calculated Pointer Authentication Code is written to the most significant 32 bits of the destination register, and the least significant 32 bits of the destination register are set to zero.

Integer

(FEAT_PAuth)



Encoding

```
PACGA <Xd>, <Xn>, <Xm|SP>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_PAuth) then EndOfDecode(Decode_UNDEF); end;

var source_is_sp : boolean = FALSE;
let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let source_is_sp : boolean = m == 31;
if m == 31 then source_is_sp = TRUE; end;
```

Assembler Symbols

<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.
<Xm|SP>	Is the 64-bit name of the second general-purpose source register or stack pointer, encoded in the "Rm" field.

Operation

```
if source_is_sp then
    X{64}(d) = AddPACGA(X{64}(n), SP{64}());
else
    X{64}(d) = AddPACGA(X{64}(n), X{64}(m));
end;
```

2026-09_rel2026-06_diff_tag 2026-09-212026-09-08 1409:2244:3801

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

PACIA, PACIA1716, PACIASP, PACIAZ, PACIZA

Pointer Authentication Code for instruction address, using key A

This instruction calculates and inserts a Pointer Authentication Code for an instruction address, using a modifier and key A.
The address is:

- In the general-purpose register that is specified by <Xd> for PACIA and PACIZA.
- In X17, for PACIA1716.
- In X30, for PACIASP and PACIAZ.

The modifier is:

- In the general-purpose register or stack pointer that is specified by <Xn|SP> for PACIA.
- The value zero, for PACIZA and PACIAZ.
- In X16, for PACIA1716.
- In SP, for PACIASP.

If *FEAT_PAuth_LR* is implemented and PSTATE.PACM is 1, then PACIA1716 and PACIASP include a second modifier that is:

- In X15, for PACIA1716.
- The value of PC, for PACIASP.

A PACIASP instruction has an implicit BTI instruction. The implicit BTI instruction of a PACIASP instruction is always compatible with *PSTATE*.BTYPE == 0b01 and *PSTATE*.BTYPE == 0b10. Controls in *SCTLR_ELx* configure whether the implicit BTI instruction of a PACIASP instruction is compatible with *PSTATE*.BTYPE == 0b11. For more information, see *PSTATE.BTYPE*.
It has encodings from 2 classes: Integer and System

Integer

(FEAT_PAuth)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	1	1	0	1	0	1	1	0	0	0	0	0	1	0	0	Z	0	0	0	Rn					Rd				
sf		S									opcode2																				

Encoding for the PACIA variant

Applies when `(Z == 0)`

```
PACIA <Xd>, <Xn|SP>
```

Encoding for the PACIZA variant

Applies when `(Z == 1 && Rn == 11111)`

```
PACIZA <Xd>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_PAuth) then EndOfDecode(Decode_UNDEF); end;

var source_is_sp : boolean = FALSE;
let pacia1716 : boolean = FALSE;
let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
var source_is_sp : boolean = FALSE;
let pacia1716 : boolean = FALSE;

if Z == '0' then // PACIA
    if n == 31 then source_is_sp = TRUE; end;
else // PACIZA
    if n != 31 then EndOfDecode(Decode_UNDEF); end;
end;
```

System

(FEAT_PAuth)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	1	0	1	0	1	0	0	0	0	0	0	1	1	0	0	1	0	0	0	x	1	0	0	x	1	1	1	1	1
																					CRm				op2						

Encoding for the PACIA1716 variant

Applies when `(CRm == 0001 && op2 == 000)`

PACIA1716

Encoding for the PACIASP variant

Applies when (CRm == 0011 && op2 == 001)

PACIASP

Encoding for the PACIAZ variant

Applies when (CRm == 0011 && op2 == 000)

PACIAZ

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_PAuth) then EndOfDecode(Decode_NOP); end;

var d : integer;
var n : integer;
var source_is_sp : boolean = FALSE;
var pacia1716 : boolean = FALSE;

case CRm::op2 of
    when '0011 000' => // PACIAZ
        d = 30;
        n = 31;
    when '0011 001' => // PACIASP
        d = 30;
        source_is_sp = TRUE;
        if IsFeatureImplemented(FEAT_BTI) then
            // Check for branch target compatibility between PSTATE.BTYPE
            // and implicit branch target of PACIASP instruction.
            let pacinst : PACInstType = PACIxSP;
            SetBTypeCompatible(BTypeCompatible_PAC(pacinst));
        end;
    when '0001 000' => // PACIA1716
        d = 17;
        n = 16;
        pacia1716 = TRUE;
end;
```

Assembler Symbols

<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn|SP>	Is the 64-bit name of the general-purpose source register or stack pointer, encoded in the "Rn" field.

Operation

```
if source_is_sp then
    if IsFeatureImplemented(FEAT_PAuth_LR) && PSTATE.PACM == '1' then
        X{64}(d) = AddPACIA2(X{64}(d), SP{64}(), PC64());
    else
        X{64}(d) = AddPACIA(X{64}(d), SP{64}());
    end;
else
    if IsFeatureImplemented(FEAT_PAuth_LR) && PSTATE.PACM == '1' && pacia1716 then
        X{64}(d) = AddPACIA2(X{64}(d), X{64}(n), X{64}(15));
    else
        X{64}(d) = AddPACIA(X{64}(d), X{64}(n));
    end;
end;
```

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

PACIB, PACIB1716, PACIBSP, PACIBZ, PACIZB

Pointer Authentication Code for instruction address, using key B

This instruction calculates and inserts a Pointer Authentication Code for an instruction address, using a modifier and key B. The address is:

- In the general-purpose register that is specified by <Xd> for PACIB and PACIZB.
- In X17, for PACIB1716.
- In X30, for PACIBSP and PACIBZ.

The modifier is:

- In the general-purpose register or stack pointer that is specified by <Xn|SP> for PACIB.
- The value zero, for PACIZB and PACIBZ.
- In X16, for PACIB1716.
- In SP, for PACIBSP.

If *FEAT_PAuth_LR* is implemented and PSTATE.PACM is 1, then PACIB1716 and PACIBSP include a second modifier that is:

- In X15, for PACIB1716.
- The value of PC, for PACIBSP.

A PACIBSP instruction has an implicit BTI instruction. The implicit BTI instruction of a PACIBSP instruction is always compatible with *PSTATE*.BTYPE == 0b01 and *PSTATE*.BTYPE == 0b10. Controls in *SCTLR_ELx* configure whether the implicit BTI instruction of a PACIBSP instruction is compatible with *PSTATE*.BTYPE == 0b11. For more information, see *PSTATE.BTYPE*.

It has encodings from 2 classes: Integer and System

Integer

(FEAT_PAuth)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	1	1	0	1	0	1	1	0	0	0	0	0	1	0	0	Z	0	0	1	Rn					Rd				
sf		S									opcode2																				

Encoding for the PACIB variant

Applies when `(Z == 0)`

```
PACIB <Xd>, <Xn|SP>
```

Encoding for the PACIZB variant

Applies when `(Z == 1 && Rn == 11111)`

```
PACIZB <Xd>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_PAuth) then EndOfDecode(Decode_UNDEF); end;

var source_is_sp : boolean = FALSE;
let pacib1716 : boolean = FALSE;
let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
var source_is_sp : boolean = FALSE;
let pacib1716 : boolean = FALSE;

if Z == '0' then // PACIB
    if n == 31 then source_is_sp = TRUE; end;
else // PACIZB
    if n != 31 then EndOfDecode(Decode_UNDEF); end;
end;
```

System

(FEAT_PAuth)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	1	0	1	0	1	0	0	0	0	0	0	1	1	0	0	1	0	0	0	x	1	0	1	x	1	1	1	1	1
																					CRm				op2						

Encoding for the PACIB1716 variant

Applies when `(CRm == 0001 && op2 == 010)`

PACIB1716

Encoding for the PACIBSP variant

Applies when (CRm == 0011 && op2 == 011)

PACIBSP

Encoding for the PACIBZ variant

Applies when (CRm == 0011 && op2 == 010)

PACIBZ

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_PAuth) then EndOfDecode(Decode_NOP); end;

var d : integer;
var n : integer;
var source_is_sp : boolean = FALSE;
var pacib1716 : boolean = FALSE;

case CRm::op2 of
    when '0011 010' => // PACIBZ
        d = 30;
        n = 31;
    when '0011 011' => // PACIBSP
        d = 30;
        source_is_sp = TRUE;
        if IsFeatureImplemented(FEAT_BTI) then
            // Check for branch target compatibility between PSTATE.BTYPE
            // and implicit branch target of PACIBSP instruction.
            let pacinst : PACInstType = PACIxSP;
            SetBTypeCompatible(BTypeCompatible_PAC(pacinst));
        end;
    when '0001 010' => // PACIB1716
        d = 17;
        n = 16;
        pacib1716 = TRUE;
end;
```

Assembler Symbols

<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn|SP>	Is the 64-bit name of the general-purpose source register or stack pointer, encoded in the "Rn" field.

Operation

```
if source_is_sp then
    if IsFeatureImplemented(FEAT_PAuth_LR) && PSTATE.PACM == '1' then
        X{64}(d) = AddPACIB2(X{64}(d), SP{64}(), PC64());
    else
        X{64}(d) = AddPACIB(X{64}(d), SP{64}());
    end;
else
    if IsFeatureImplemented(FEAT_PAuth_LR) && PSTATE.PACM == '1' && pacib1716 then
        X{64}(d) = AddPACIB2(X{64}(d), X{64}(n), X{64}(15));
    else
        X{64}(d) = AddPACIB(X{64}(d), X{64}(n));
    end;
end;
```

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

no old file htmldiff from- (new)

SLBND

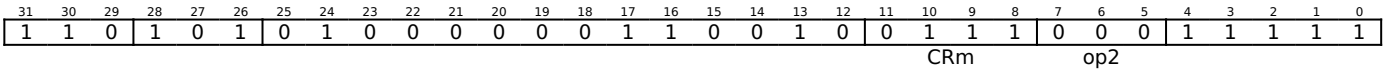
Scoped release boundary

This instruction signals to the memory system that a corresponding instruction with Release semantics or DMB instruction will subsequently occur in program order, and if that corresponding instruction defines a Barrier scope with the SLBND instruction, then these instructions do not create a Barrier-ordered-before relationship between any Effect in program order before the SLBND instruction and any of the following:

- The Effect with Release semantics.
- Any Effect in program order after the DMB Effect.

System

(FEAT_RLCS)



Encoding

SLBND

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_RLCS) then EndOfDecode(Decode_NOP); end;
```

Operation

```
Hint_ScopedReleaseBoundary();
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

no old file htmldiff from- (new)

SRLS

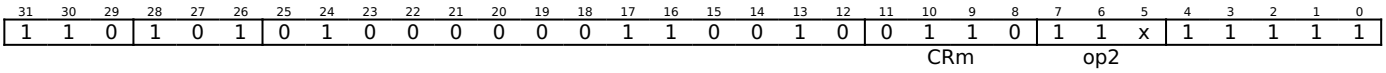
Scope release

This instruction signals to the memory system that if the next instruction in program order has Release semantics or is a DMB instruction that defines a Barrier scope with a preceding SLBND instruction, then these instructions do not create a Barrier-ordered-before relationship between any Effect in program order before the SLBND instruction and any of the following:

- The Effect with Release semantics.
- Any Effect in program order after the DMB Effect.

System

(FEAT_RLCS)



Encoding

```
SRLS {STSHSTRM}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_RLCS) then EndOfDecode(Decode_NOP); end;
let stshstrm : boolean = op2[0] == '1';
```

Assembler Symbols

STSHSTRM — Is a store shared stream hint, and is encoded in "op2[0]". If present, it additionally provides the same hint as the STSHH STRM instruction.

Note

If FEAT_PCDPHINT is not implemented, the optional STSHSTRM hint may not have the performance benefits that are associated with the hint generated by the STSHH STRM instruction.

op2[0]	STSHSTRM
0	[absent]
1	[present]

Operation

```
Hint_ScopeRelease(stshstrm);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

no old file htmldiff from- (new)

ST64B

Single-copy atomic 64-byte store without status result

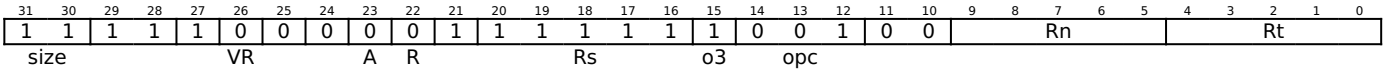
This instruction stores eight 64-bit doublewords from consecutive registers to a memory location. The store starts at register Xt, with the data being formed as Data[511:0] = X(t+7)::X(t+6)::X(t+5)::X(t+4)::X(t+3)::X(t+2)::X(t+1)::Xt. The data is stored atomically and is required to be 64-byte aligned.

It is IMPLEMENTATION DEFINED which memory locations support this instruction. A memory location that supports ST64B also supports LD64B, LDA64B, and STL64B.

For more information, including about the memory types accessible and how the accesses are performed, see *Single-copy atomic 64-byte load/store*.

Integer

(FEAT_LS64)



Encoding

```
ST64B <Xt>, [<Xn|SP> {, #0}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_LS64) then EndOfDecode(Decode_UNDEF); end;
if Rt[4:3] == '11' || Rt[0] == '1' then EndOfDecode(Decode_UNDEF); end;
let withstatus : boolean = FALSE;
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let tagchecked : boolean = n != 31;
let acqrel : boolean = FALSE;let tagchecked : boolean = n != 31;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *ST64B*.

Assembler Symbols

<Xt>	Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
CheckLDST64BEnabled(acqrel);
();

var data : bits(512);
var address : bits(64);
var value : bits(64);

let accdesc : AccessDescriptor = CreateAccDescLS64(MemOp_STORE, withstatus, acqrel, tagchecked);
, withstatus, tagchecked);

for i = 0 to 7 do
    value = X{64}(t+i);
    if BigEndian(accdesc.acctype) then value = BigEndianReverse{64}(value); end;
    data[63+64*i : 64*i] = value;
end;

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

MemStore64B(address, data, accdesc);
```

2026-09_rel2026-06_diff_tag 2026-09-212026-09-08 1409:2244:3801

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

ST64BV

Single-copy atomic 64-byte store with status result

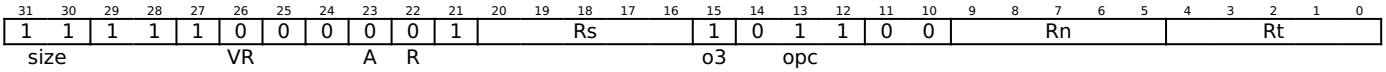
This instruction stores eight 64-bit doublewords from consecutive registers to a memory location, and writes the status result of the store to a register. The store starts at register Xt, with the data being formed as Data[511:0] = X(t+7)::X(t+6)::X(t+5)::X(t+4)::X(t+3)::X(t+2)::X(t+1)::Xt. The data is stored atomically and is required to be 64-byte aligned.

It is IMPLEMENTATION DEFINED which memory locations support this instruction. A memory location that supports ST64BV also supports ST64BV0, STL64BV, and STL64BV0.

For more information, including about the memory types accessible and how the accesses are performed, see *Single-copy atomic 64-byte load/store*.

Integer

(FEAT_LS64_V)



Encoding

```
ST64BV <Xs>, <Xt>, [<Xn|SP>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_LS64_V) then EndOfDecode(Decode_UNDEF); end;
if Rt[4:3] == '11' || Rt[0] == '1' then EndOfDecode(Decode_UNDEF); end;
let withstatus : boolean = TRUE;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let tagchecked : boolean = n != 31;
let acqrel : boolean = FALSE;~~let tagchecked : boolean = n != 31;~~
```

Assembler Symbols

<Xs> Is the 64-bit name of the general-purpose register into which the status result of this instruction is written, encoded in the "Rs" field. The value returned is:

0xFFFFFFFFFFFFFFFF
: If the memory location accessed does not support this instruction. In this case, the value at the memory location is UNKNOWN.

!= 0xFFFFFFFFFFFFFFFF
: If the memory location accessed does support this instruction. In this case, the peripheral that provides the response defines the returned value and provides information on the state of the memory update at the memory location.

If XZR is used, then the return value is ignored.

<Xt> Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
CheckST64BVEnabled(acqrel);
();

var data : bits(512);
var address : bits(64);
var value : bits(64);

let accdesc : AccessDescriptor = CreateAccDescLS64(MemOp_STORE, withstatus, acqrel, tagchecked);
, withstatus, tagchecked);

for i = 0 to 7 do
    value = X{64}(t+i);
    if BigEndian(accdesc.acctype) then value = BigEndianReverse{64}(value); end;
    data[63+64*i : 64*i] = value;
end;

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let status : bits(64) = MemStore64BWithRet(address, data, accdesc);

if s != 31 then X{64}(s) = status; end;
```

2026-09_rel2026-06_diff_tag 2026-09-212026-09-08 1409:2244:3801

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

ST64BV0

Single-copy atomic 64-byte EL0 store with status result

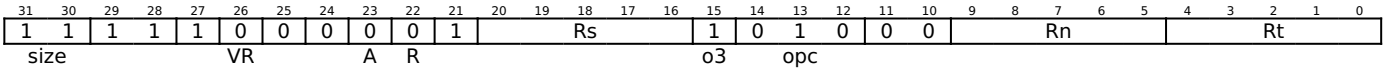
This instruction stores eight 64-bit doublewords from consecutive registers to a memory location, with the bottom 32 bits taken from ACCDATA_EL1, and writes the status result of the store to a register. The store starts at register Xt, with the data being formed as Data[511:0] = X(t+7)::X(t+6)::X(t+5)::X(t+4)::X(t+3)::X(t+2)::X(t+1)::Xt[63:32]::ACCDATA_EL1()[31:0]. The data is stored atomically and is required to be 64-byte aligned.

It is IMPLEMENTATION DEFINED which memory locations support this instruction. A memory location that supports `ST64BV0` also supports `ST64BV`, STL64BV, and STL64BV0.

For more information, including about the memory types accessible and how the accesses are performed, see *Single-copy atomic 64-byte load/store*.

Integer

(FEAT_LS64_ACCDATA)



Encoding

```
ST64BV0 <Xs>, <Xt>, [<Xn|SP>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_LS64_ACCDATA) then EndOfDecode(Decode_UNDEF); end;
if Rt[4:3] == '11' || Rt[0] == '1' then EndOfDecode(Decode_UNDEF); end;
let withstatus : boolean = TRUE;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let tagchecked : boolean = n != 31;
let acqrel : boolean = FALSE;let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Xs> Is the 64-bit name of the general-purpose register into which the status result of this instruction is written, encoded in the "Rs" field. The value returned is:

0xFFFFFFFFFFFFFFFF
: If the memory location accessed does not support this instruction. In this case, the value at the memory location is UNKNOWN.

!= 0xFFFFFFFFFFFFFFFF
: If the memory location accessed does support this instruction. In this case, the peripheral that provides the response defines the returned value and provides information on the state of the memory update at the memory location.

If XZR is used, then the return value is ignored.

<Xt> Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
CheckST64BV0Enabled(acqrel);
();

var data : bits(512);
var address : bits(64);
var value : bits(64);

let accdesc : AccessDescriptor = CreateAccDescLS64(MemOp_STORE, withstatus, acqrel, tagchecked);
, withstatus, tagchecked);

let Xt : bits(64) = X{}(t);
value[31:0] = ACCDATA_EL1()[31:0];
value[63:32] = Xt[63:32];
if BigEndian(accdesc.acctype) then value = BigEndianReverse{64}(value); end;
data[63:0] = value;

for i = 1 to 7 do
    value = X{64}(t+i);
    if BigEndian(accdesc.acctype) then value = BigEndianReverse{64}(value); end;
    data[63+64*i : 64*i] = value;
end;

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let status : bits(64) = MemStore64BWithRet(address, data, accdesc);

if s != 31 then X{64}(s) = status; end;
```

2026-09_rel2026-06_diff_tag 2026-09-212026-09-08 1409:2244:3801

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

STGM

Store Allocation Tag multiple

This instruction writes a naturally aligned block of N Allocation Tags, where the size of N is identified in GMID_EL1.BS, and the Allocation Tag written to address A is taken from the source register at 4*A<7:4>+3:4*A<7:4>.

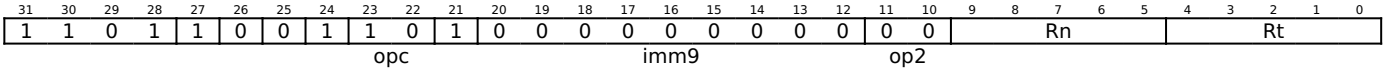
If the value of HCR_EL2.TGE is 0 and the Effective value of SCTLR2_EL1.GM0 is 0, this~~This~~ instruction is UNDEFINED at EL0.

If the value of HCR_EL2.TGE is 1 and the Effective value of SCTLR2_EL2.GM0 is 0, this instruction is UNDEFINED at EL0.

This instruction generates an Unchecked access.

Integer

(FEAT_MTE2)



Encoding

```
STGM <Xt>, [<Xn|SP>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_MTE2) then EndOfDecode(Decode_UNDEF); end;
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
```

Assembler Symbols

<Xt>	Is the 64-bit name of the general-purpose source register, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
if !~~if~~IsGMEnabledAtEL0() && PSTATE.EL == EL0 then Undefined(); end;

let data : bits(64) = X{64}(t);
~~then Undefined(); end;~~

~~let data : bits(64) = X{}(t);~~
var address : bits(64);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let size : integer{} = 4 * 2 ^ UInt(GMID_EL1().BS);
~~let size : integer{} = 4 * (2 ^ (UInt(GMID_EL1().BS)));~~
address = AlignDownSize(address, size);
let count : integer = size >> LOG2_TAG_GRANULE;
var index : integer = UInt(address[LOG2_TAG_GRANULE+3:LOG2_TAG_GRANULE]);
~~let curraddress : bits(64) = address;~~
let stzgm : boolean = FALSE;
let accdesc : AccessDescriptor = CreateAccDescLDGSTG(MemOp_STORE, stzgm, t);

for i = 0 to count-1 do
    let tag : bits(4) = data[index*:4];
    AArch64_MemTag(address, accdesc) = tag;
    address = AddressIncrement(address, TAG_GRANULE, accdesc);
    index = index + 1;
end;
```

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STGP

Store Allocation Tag and pair of registers

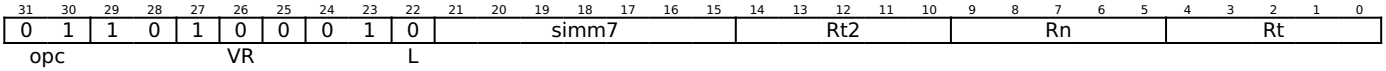
This instruction stores an Allocation Tag and two 64-bit doublewords to memory, from two registers. The address used for the store is calculated from the base register and an immediate signed offset scaled by the Tag Granule. The Allocation Tag is calculated from the Logical Address Tag in the base register.

This instruction generates an Unchecked access.

It has encodings from 3 classes: Post-index , Pre-index and Signed offset

Post-index

(FEAT_MTE)



Encoding

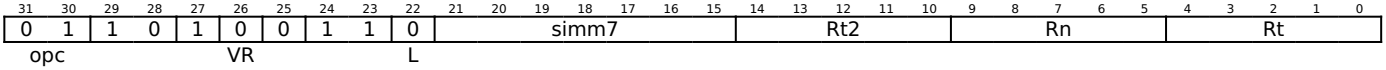
```
STGP <Xt1>, <Xt2>, [<Xn|SP>], #<imm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_MTE) then EndOfDecode(Decode_UNDEF); end;
let t : integer{} = UInt(Rt);
let t2 : integer{} = UInt(Rt2);
let n : integer{} = UInt(Rn);
let offset : bits(64) = LSL(SignExtend{64}(simm7), LOG2_TAG_GRANULE);
let writeback : boolean = TRUE;
let postindex : boolean = TRUE;
```

Pre-index

(FEAT_MTE)



Encoding

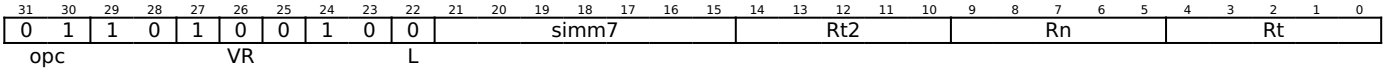
```
STGP <Xt1>, <Xt2>, [<Xn|SP>, #<imm>]!
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_MTE) then EndOfDecode(Decode_UNDEF); end;
let t : integer{} = UInt(Rt);
let t2 : integer{} = UInt(Rt2);
let n : integer{} = UInt(Rn);
let offset : bits(64) = LSL(SignExtend{64}(simm7), LOG2_TAG_GRANULE);
let writeback : boolean = TRUE;
let postindex : boolean = FALSE;
```

Signed offset

(FEAT_MTE)



Encoding

```
STGP <Xt1>, <Xt2>, [<Xn|SP>{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_MTE) then EndOfDecode(Decode_UNDEF); end;
let t : integer{} = UInt(Rt);
let t2 : integer{} = UInt(Rt2);
let n : integer{} = UInt(Rn);
let offset : bits(64) = LSL(SignExtend{64}(simm7), LOG2_TAG_GRANULE);
let writeback : boolean = FALSE;
let postindex : boolean = FALSE;
```

Assembler Symbols

<Xt1>	Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.

<Xt2>	Is the 64-bit name of the second general-purpose register to be transferred, encoded in the "Rt2" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	For the "Post-index" and "Pre-index" variants: is the signed immediate offset, a multiple of 16 in the range -1024 to 1008, encoded in the "simm7" field. For the "Signed offset" variant: is the optional signed immediate offset, a multiple of 16 in the range -1024 to 1008, defaulting to 0 and encoded in the "simm7" field.

Operation

```
var address : bits(64);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let stzgm : boolean = FALSE;
let ispair : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescLDGSTG(MemOp_STORE, stzgm, ispair, t, t2);

if !postindex then
    address = AddressAdd(address, offset, accdesc);
end;

if !IsAlignedSize(address, TAG_GRANULE) then
    let fault : FaultRecord = AlignmentFault(accdesc, address);
    AArch64_Abort(fault);
end;

let data : bits(128) = (if BigEndian(accdesc.acctype) then X{64}(t) :: X{64}(t2)
                        else X{64}(t2) ::{64}(t2):: X{64}(t));
Mem{128}(address, accdesc) = data;

AArch64_MemTag(address, accdesc) = AArch64_AllocationTagFromAddress(address);

if writeback then
    if postindex then
        address = AddressAdd(address, offset, accdesc);
    end;

    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

2026-09_rel2026-06_diff_tag 2026-09-212026-09-08 1409:2244:3801

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

STL64B

Single-copy atomic 64-byte store-release without status result

This instruction stores eight 64-bit doublewords from consecutive registers to a memory location. The store starts at register Xt, with the data being formed as Data[511:0] = X(t+7)::X(t+6)::X(t+5)::X(t+4)::X(t+3)::X(t+2)::X(t+1)::Xt. The data is stored atomically and is required to be 64-byte aligned.

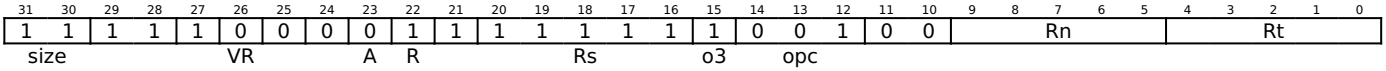
It is IMPLEMENTATION DEFINED which memory locations support this instruction. A memory location that supports `STL64B` also supports `LD64B`, `LDA64B`, and `ST64B`.

`STL64B` stores to memory with release semantics.

For more information, including about the memory types accessible and how the accesses are performed, see *Single-copy atomic 64-byte load/store*.

Integer

(FEAT_LS64 && FEAT_LSC64B)



Encoding

```
STL64B <Xt>, [<Xn|SP> {, #0}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_LSC64B) then EndOfDecode(Decode_UNDEF); end;
if !IsFeatureImplemented(FEAT_LS64) then EndOfDecode(Decode_UNDEF); end;
if Rt[4:3] == '11' || Rt[0] == '1' then EndOfDecode(Decode_UNDEF); end;
let withstatus : boolean = FALSE;
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let tagchecked : boolean = n != 31;
let acqrel : boolean = TRUE;
```

`STL64B` has the same CONSTRAINED UNPREDICTABLE behavior as `ST64B`. For information about this CONSTRAINED UNPREDICTABLE behavior, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *ST64B*.

Assembler Symbols

<Xt>	Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
CheckLDST64BEnabled(acqrel);

var data : bits(512);
var address : bits(64);
var value : bits(64);

let accdesc : AccessDescriptor = CreateAccDescLS64(MemOp_STORE, withstatus, acqrel, tagchecked);

for i = 0 to 7 do
    value = X{64}(t+i);
    if BigEndian(accdesc.acctype) then value = BigEndianReverse{64}(value); end;
    data[63+64*i : 64*i] = value;
end;

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

MemStore64B(address, data, accdesc);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

no old file htmldiff from- (new)

STL64BV

Single-copy atomic 64-byte store-release with status result

This instruction stores eight 64-bit doublewords from consecutive registers to a memory location, and writes the status result of the store to a register. The store starts at register Xt, with the data being formed as Data[511:0] = X(t+7)::X(t+6)::X(t+5)::X(t+4)::X(t+3)::X(t+2)::X(t+1)::Xt. The data is stored atomically and is required to be 64-byte aligned.

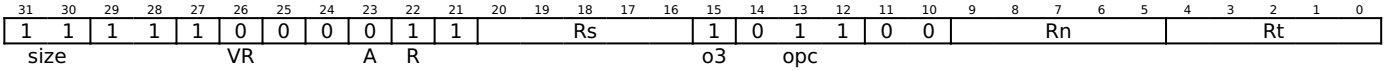
It is IMPLEMENTATION DEFINED which memory locations support this instruction. A memory location that supports `STL64BV` also supports `ST64BV`, `ST64BV0`, and `STL64BV0`.

`STL64BV` stores to memory with release semantics.

For more information, including about the memory types accessible and how the accesses are performed, see *Single-copy atomic 64-byte load/store*.

Integer

(FEAT_LS64_V && FEAT_LSC64B)



Encoding

```
STL64BV <Xs>, <Xt>, [<Xn|SP>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_LSC64B) then EndOfDecode(Decode_UNDEF); end;
if !IsFeatureImplemented(FEAT_LS64_V) then EndOfDecode(Decode_UNDEF); end;
if Rt[4:3] == '11' || Rt[0] == '1' then EndOfDecode(Decode_UNDEF); end;
let withstatus : boolean = TRUE;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let tagchecked : boolean = n != 31;
let acqrel : boolean = TRUE;
```

Assembler Symbols

<Xs> Is the 64-bit name of the general-purpose register into which the status result of this instruction is written, encoded in the "Rs" field. The value returned is:

0xFFFFFFFFFFFFFFFF
: If the memory location accessed does not support this instruction. In this case, the value at the memory location is UNKNOWN.

!= 0xFFFFFFFFFFFFFFFF
: If the memory location accessed does support this instruction. In this case, the peripheral that provides the response defines the returned value and provides information on the state of the memory update at the memory location.

If XZR is used, then the return value is ignored.

<Xt> Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
CheckST64BVEnabled(acqrel);

var data : bits(512);
var address : bits(64);
var value : bits(64);

let accdesc : AccessDescriptor = CreateAccDescLS64(MemOp_STORE, withstatus, acqrel, tagchecked);

for i = 0 to 7 do
    value = X{64}(t+i);
    if BigEndian(accdesc.acctype) then value = BigEndianReverse{64}(value); end;
    data[63+64*i : 64*i] = value;
end;

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let status : bits(64) = MemStore64BWithRet(address, data, accdesc);

if s != 31 then X{64}(s) = status; end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

no old file htmldiff from- (new)

STL64BV0

Single-copy atomic 64-byte EL0 store-release with status result

This instruction stores eight 64-bit doublewords from consecutive registers to a memory location, with the bottom 32 bits taken from ACCDATA_EL1, and writes the status result of the store to a register. The store starts at register Xt, with the data being formed as Data[511:0] = X(t+7)::X(t+6)::X(t+5)::X(t+4)::X(t+3)::X(t+2)::X(t+1)::Xt[63:32]::ACCDATA_EL1()[31:0]. The data is stored atomically and is required to be 64-byte aligned.

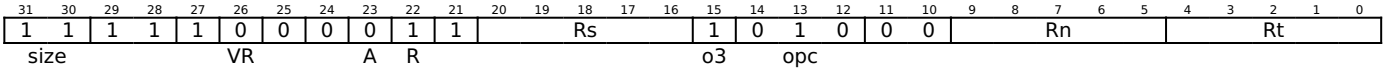
It is IMPLEMENTATION DEFINED which memory locations support this instruction. A memory location that supports `STL64BV0` also supports `ST64BV`, `ST64BV0`, and `STL64BV`.

`STL64BV0` stores to memory with release semantics.

For more information, including about the memory types accessible and how the accesses are performed, see *Single-copy atomic 64-byte load/store*.

Integer

(FEAT_LS64_ACCDATA && FEAT_LSC64B)



Encoding

```
STL64BV0 <Xs>, <Xt>, [<Xn|SP>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_LSC64B) then EndOfDecode(Decode_UNDEF); end;
if !IsFeatureImplemented(FEAT_LS64_ACCDATA) then EndOfDecode(Decode_UNDEF); end;
if Rt[4:3] == '11' || Rt[0] == '1' then EndOfDecode(Decode_UNDEF); end;
let withstatus : boolean = TRUE;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let tagchecked : boolean = n != 31;
let acqrel : boolean = TRUE;
```

Assembler Symbols

<Xs> Is the 64-bit name of the general-purpose register into which the status result of this instruction is written, encoded in the "Rs" field. The value returned is:

0xFFFFFFFFFFFFFFFF
: If the memory location accessed does not support this instruction. In this case, the value at the memory location is UNKNOWN.

!= 0xFFFFFFFFFFFFFFFF
: If the memory location accessed does support this instruction. In this case, the peripheral that provides the response defines the returned value and provides information on the state of the memory update at the memory location.

If XZR is used, then the return value is ignored.

<Xt> Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
CheckST64BV0Enabled(acqrel);

var data : bits(512);
var address : bits(64);
var value : bits(64);

let accdesc : AccessDescriptor = CreateAccDescLS64(MemOp_STORE, withstatus, acqrel, tagchecked);

let Xt : bits(64) = X{}(t);
value[31:0] = ACCDATA_EL1()[31:0];
value[63:32] = Xt[63:32];
if BigEndian(accdesc.acctype) then value = BigEndianReverse{64}(value); end;
data[63:0] = value;

for i = 1 to 7 do
    value = X{64}(t+i);
    if BigEndian(accdesc.acctype) then value = BigEndianReverse{64}(value); end;
    data[63+64*i : 64*i] = value;
end;

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let status : bits(64) = MemStore64BWithRet(address, data, accdesc);

if s != 31 then X{64}(s) = status; end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

no old file htmldiff from- (new)

STLTXR

Store-release unprivileged exclusive register

This instruction stores a 32-bit word or a 64-bit doubleword to memory if the PE has exclusive access to the memory address, and returns a status value of 0 if the store was successful, or of 1 if no store was performed. See *Synchronization and semaphores*. The memory access is atomic. The instruction also has memory ordering semantics as described in *Load-Acquire, Store-Release*.
Explicit Memory effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory effects operate with the restrictions determined by the Exception level at which the instruction is executed.

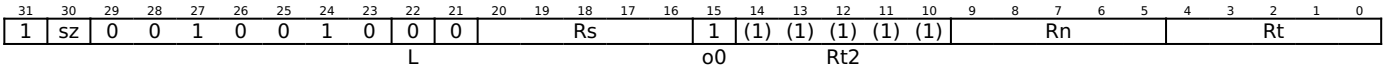
Note

For the purposes of the Exclusives monitors, and the forward progress guarantees for Load-Exclusive and Store-Exclusive loops, STLTXR is equivalent to STXR.

For information about addressing modes, see *Load/Store addressing modes*.

No offset

(FEAT_LSUI)



Encoding for the 32-bit variant

Applies when `(sz == 0)`

```
STLTXR <Ws>, <Wt>, [<Xn|SP>{, #0}]
```

Encoding for the 64-bit variant

Applies when `(sz == 1)`

```
STLTXR <Ws>, <Xt>, [<Xn|SP>{, #0}]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSUI) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let elsize : integer{} = 32 << UInt(sz);
let acqrel : boolean = TRUE;
let tagchecked : boolean = n != 31;

var rt_unknown : boolean = FALSE;
var rn_unknown : boolean = FALSE;
if s == t then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_DATAOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>     rt_unknown = TRUE;     // store UNKNOWN value
        when Constraint_UNDEF =>       EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>         EndOfDecode(Decode_NOP);
    end;
end;
if s == n && n != 31 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_BASEOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>     rn_unknown = TRUE;     // address is UNKNOWN
        when Constraint_UNDEF =>       EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>         EndOfDecode(Decode_NOP);
    end;
end;
```

STLTXR has the same CONSTRAINED UNPREDICTABLE behavior as STLXR. For information about this CONSTRAINED UNPREDICTABLE behavior, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *STLXR*.

Assembler Symbols

<Ws> Is the 32-bit name of the general-purpose register into which the status result of the store exclusive is written, encoded in the "Rs" field. The value returned is:

0
: If the operation updates memory.

1
: If the operation fails to update memory.

<Wt> Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Xt> Is the 64-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

Aborts and alignment

If a synchronous Data Abort or Watchpoint exception is generated by the execution of this instruction:

- Memory is not updated.
- <Ws> is not updated.

Accessing an address that is not aligned to the size of the data being accessed causes an Alignment fault Data Abort exception to be generated, subject to the following rules:

- If AArch64_ExclusiveMonitorsPass() returns TRUE, the exception is generated.
- Otherwise, it is IMPLEMENTATION DEFINED whether the exception is generated.

If AArch64_ExclusiveMonitorsPass() returns FALSE and the memory address, if accessed, would generate a synchronous Data Abort or Watchpoint exception, it is IMPLEMENTATION DEFINED whether the exception is generated.

Operation

```
var address : bits(64);
var data : bits(elsize);

let dbytes : integer{} = elsize DIV 8;
let privileged : boolean = AArch64_IsUnprivAccessPriv();
let accdesc : AccessDescriptor = CreateAccDescExLDST(MemOp_STORE, acqrel, tagchecked,
                                                     privileged, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
elsif rn_unknown then
    address = ARBITRARY : bits(64);
else
    address = X{64}(n);
end;

if rt_unknown then
    data = ARBITRARY : bits(elsize);
else
    data = X{elsize}(t);
end;

var status : bit = '1';
// Check whether the Exclusives monitors are set to include the
// physical memory locations corresponding to virtual address
// range [address, address+dbytes-1].
// If AArch64_ExclusiveMonitorsPass() returns FALSE and the memory address,
// if accessed, would generate a synchronous Data Abort or Watchpoint exception,
// it is IMPLEMENTATION DEFINED whether the exception is generated.
// if accessed, would generate a synchronous Data Abort exception, it is
// IMPLEMENTATION DEFINED whether the exception is generated.
// It is a limitation of Pseudocode that synchronous Data Aborts are never
// generated in this case, as Mem is not called.

// If FEAT_SPE is implemented, it is also IMPLEMENTATION DEFINED whether or not the
// physical address packet is output when permitted and when
// AArch64_ExclusiveMonitorPass() returns FALSE for a Store Exclusive instruction.
// This behavior is not reflected here due to the previously stated limitation.
if AArch64_ExclusiveMonitorsPass(address, dbytes, accdesc) then
    // This atomic write will be rejected if it does not refer
    // to the same physical locations after address translation.
    Mem{elsize}(address, accdesc) = data;
    status = ExclusiveMonitorsStatus();
end;
X{32}(s) = ZeroExtend{32}(status);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

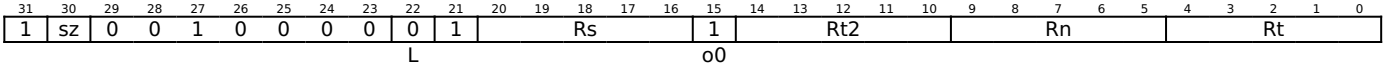
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

STLXP

Store-release exclusive pair of registers

This instruction stores two 32-bit words or two 64-bit doublewords to a memory location if the PE has exclusive access to the memory address, from two registers, and returns a status value of 0 if the store was successful, or of 1 if no store was performed. See *Synchronization and semaphores*. For information on single-copy atomicity and alignment requirements, see *Requirements for single-copy atomicity* and *Alignment of data accesses*. If a 64-bit pair Store-Exclusive succeeds, it causes a single-copy atomic update of the 128-bit memory location being updated. The instruction also has memory ordering semantics, as described in *Load-Acquire, Store-Release*. For information about addressing modes, see *Load/Store addressing modes*.



Encoding for the 32-bit variant

Applies when `(sz == 0)`

```
STLXP <Ws>, <Wt1>, <Wt2>, [<Xn|SP>{, #0}]
```

Encoding for the 64-bit variant

Applies when `(sz == 1)`

```
STLXP <Ws>, <Xt1>, <Xt2>, [<Xn|SP>{, #0}]
```

Decode for all variants of this encoding

```
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let t2 : integer{} = UInt(Rt2);
let n : integer{} = UInt(Rn);

let elsize : integer{} = 32 << UInt(sz);
let datasize : integer{} = elsize * 2;
let acqrel : boolean = TRUE;
let tagchecked : boolean = n != 31;

var rt_unknown : boolean = FALSE;
var rn_unknown : boolean = FALSE;
if s == t || s == t2 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_DATAOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>     rt_unknown = TRUE;     // store UNKNOWN value
        when Constraint_UNDEF =>       EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>         EndOfDecode(Decode_NOP);
    end;
end;
if s == n && n != 31 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_BASEOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>     rn_unknown = TRUE;     // address is UNKNOWN
        when Constraint_UNDEF =>       EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>         EndOfDecode(Decode_NOP);
    end;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *CONSTRAINED UNPREDICTABLE behaviors with Load-Exclusive/Store-Exclusive pairs* and *STLXP*.

Assembler Symbols

<Ws> Is the 32-bit name of the general-purpose register into which the status result of the store exclusive is written, encoded in the "Rs" field. The value returned is:

0
: If the operation updates memory.

1
: If the operation fails to update memory.

<Wt1> Is the 32-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.

<Wt2> Is the 32-bit name of the second general-purpose register to be transferred, encoded in the "Rt2" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Xt1> Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.

<Xt2> Is the 64-bit name of the second general-purpose register to be transferred, encoded in the "Rt2" field.

Aborts and alignment

If a synchronous Data Abort or Watchpoint exception is generated by the execution of this instruction:

- Memory is not updated.
- <Ws> is not updated.

Accessing an address that is not aligned to the size of the data being accessed causes an Alignment fault Data Abort exception to be generated, subject to the following rules:

- If AArch64_ExclusiveMonitorsPass() returns TRUE, the exception is generated.
- Otherwise, it is IMPLEMENTATION DEFINED whether the exception is generated.

If AArch64_ExclusiveMonitorsPass() returns FALSE and the memory address, if accessed, would generate a synchronous Data Abort or Watchpoint exception, it is IMPLEMENTATION DEFINED whether the exception is generated.

Operation

```
var address : bits(64);
var data : bits(datasize);

let dbytes : integer{} = datasize DIV 8;
let privileged : boolean = PSTATE.EL != EL0;
let ispair : boolean = FALSE;
let accdesc : AccessDescriptor = CreateAccDescExLDST(MemOp_STORE, acqrel, tagchecked,
                                                     privileged, ispair, t, t2);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
elsif rn_unknown then
    address = ARBITRARY : bits(64);
else
    address = X{64}(n);
end;

if rt_unknown then
    data = ARBITRARY : bits(datasize);
else
    let el1 : bits(datasize DIV 2) = X{}(t);
    let el2 : bits(datasize DIV 2) = X{}(t2);
    data = if BigEndian(accdesc.acctype) then el1::el2 else el2::el1;
end;
var status : bit = '1';
// Check whether the Exclusives monitors are set to include the
// physical memory locations corresponding to virtual address
// range [address, address+dbytes-1].
// If AArch64_ExclusiveMonitorsPass() returns FALSE and the memory address,
// if accessed, would generate a synchronous Data Abort or Watchpoint exception,
// it is IMPLEMENTATION DEFINED whether the exception is generated.
// if accessed, would generate a synchronous Data Abort exception, it is
// IMPLEMENTATION DEFINED whether the exception is generated.
// It is a limitation of Pseudocode that synchronous Data Aborts are never
// generated in this case, as Mem is not called.

// If FEAT_SPE is implemented, it is also IMPLEMENTATION DEFINED whether or not the
// physical address packet is output when permitted and when
// AArch64_ExclusiveMonitorPass() returns FALSE for a Store Exclusive instruction.
// This behavior is not reflected here due to the previously stated limitation.
if AArch64_ExclusiveMonitorsPass(address, dbytes, accdesc) then
    // This atomic write will be rejected if it does not refer
    // to the same physical locations after address translation.
    Mem{datasize}(address, accdesc) = data;
    status = ExclusiveMonitorsStatus();
end;
X{32}(s) = ZeroExtend{32}(status);
```

Operational information

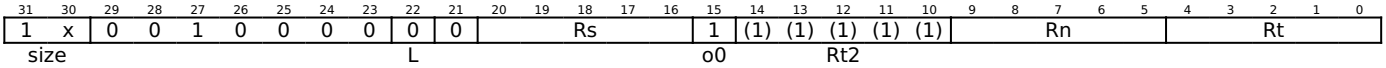
This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STLXR

Store-release exclusive register

This instruction stores a 32-bit word or a 64-bit doubleword to memory if the PE has exclusive access to the memory address, and returns a status value of 0 if the store was successful, or of 1 if no store was performed. See *Synchronization and semaphores*. The memory access is atomic. The instruction also has memory ordering semantics as described in *Load-Acquire, Store-Release*. For information about addressing modes, see *Load/Store addressing modes*.



Encoding for the 32-bit variant

Applies when `(size == 10)`

```
STLXR <Ws>, <Wt>, [<Xn|SP>{, #0}]
```

Encoding for the 64-bit variant

Applies when `(size == 11)`

```
STLXR <Ws>, <Xt>, [<Xn|SP>{, #0}]
```

Decode for all variants of this encoding

```
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let elsize : integer{} = 8 << UInt(size);
let acqrel : boolean = TRUE;
let tagchecked : boolean = n != 31;

var rt_unknown : boolean = FALSE;
var rn_unknown : boolean = FALSE;
if s == t then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_DATAOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>    rt_unknown = TRUE;    // store UNKNOWN value
        when Constraint_UNDEF =>      EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>        EndOfDecode(Decode_NOP);
    end;
end;
if s == n && n != 31 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_BASEOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>    rn_unknown = TRUE;    // address is UNKNOWN
        when Constraint_UNDEF =>      EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>        EndOfDecode(Decode_NOP);
    end;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *STLXR*.

Assembler Symbols

<Ws> Is the 32-bit name of the general-purpose register into which the status result of the store exclusive is written, encoded in the "Rs" field. The value returned is:

0
: If the operation updates memory.

1
: If the operation fails to update memory.

<Wt> Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Xt> Is the 64-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

Aborts and alignment

If a synchronous Data Abort or Watchpoint exception is generated by the execution of this instruction:

- Memory is not updated.
- <Ws> is not updated.

Accessing an address that is not aligned to the size of the data being accessed causes an Alignment fault Data Abort exception to be generated, subject to the following rules:

- If AArch64_ExclusiveMonitorsPass() returns TRUE, the exception is generated.
- Otherwise, it is IMPLEMENTATION DEFINED whether the exception is generated.

If AArch64_ExclusiveMonitorsPass() returns FALSE and the memory address, if accessed, would generate a synchronous Data Abort or Watchpoint exception, it is IMPLEMENTATION DEFINED whether the exception is generated.

Operation

```
var address : bits(64);
var data : bits(elsize);

let dbytes : integer{} = elsize DIV 8;
let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescExLDST(MemOp_STORE, acqrel, tagchecked,
                                                     privileged, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
elsif rn_unknown then
    address = ARBITRARY : bits(64);
else
    address = X{64}(n);
end;

if rt_unknown then
    data = ARBITRARY : bits(elsize);
else
    data = X{elsize}(t);
end;

var status : bit = '1';
// Check whether the Exclusives monitors are set to include the
// physical memory locations corresponding to virtual address
// range [address, address+dbytes-1].
// If AArch64_ExclusiveMonitorsPass() returns FALSE and the memory address,
// if accessed, would generate a synchronous Data Abort or Watchpoint exception,
// it is IMPLEMENTATION DEFINED whether the exception is generated.
// if accessed, would generate a synchronous Data Abort exception, it is
// IMPLEMENTATION DEFINED whether the exception is generated.
// It is a limitation of Pseudocode that synchronous Data Aborts are never
// generated in this case, as Mem is not called.

// If FEAT_SPE is implemented, it is also IMPLEMENTATION DEFINED whether or not the
// physical address packet is output when permitted and when
// AArch64_ExclusiveMonitorPass() returns FALSE for a Store Exclusive instruction.
// This behavior is not reflected here due to the previously stated limitation.
if AArch64_ExclusiveMonitorsPass(address, dbytes, accdesc) then
    // This atomic write will be rejected if it does not refer
    // to the same physical locations after address translation.
    Mem{elsize}(address, accdesc) = data;
    status = ExclusiveMonitorsStatus();
end;
X{32}(s) = ZeroExtend{32}(status);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

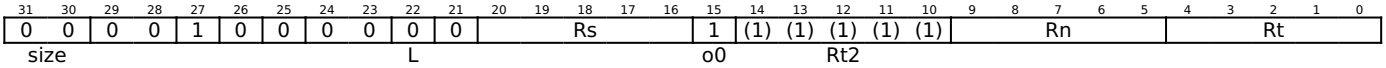
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

STLXRB

Store-release exclusive register byte

This instruction stores a byte from a 32-bit register to memory if the PE has exclusive access to the memory address, and returns a status value of 0 if the store was successful, or of 1 if no store was performed. See *Synchronization and semaphores*. The memory access is atomic. The instruction also has memory ordering semantics as described in *Load-Acquire, Store-Release*. For information about addressing modes, see *Load/Store addressing modes*.



Encoding

```
STLXRB <Ws>, <Wt>, [<Xn|SP>{, #0}]
```

Decode for this encoding

```
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let acqrel : boolean = TRUE;
let tagchecked : boolean = n != 31;

var rt_unknown : boolean = FALSE;
var rn_unknown : boolean = FALSE;
if s == t then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_DATAOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>    rt_unknown = TRUE;    // store UNKNOWN value
        when Constraint_UNDEF =>      EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>        EndOfDecode(Decode_NOP);
    end;
end;
if s == n && n != 31 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_BASEOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>    rn_unknown = TRUE;    // address is UNKNOWN
        when Constraint_UNDEF =>      EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>        EndOfDecode(Decode_NOP);
    end;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *STLXRB*.

Assembler Symbols

<Ws> Is the 32-bit name of the general-purpose register into which the status result of the store exclusive is written, encoded in the "Rs" field. The value returned is:

0
: If the operation updates memory.

1
: If the operation fails to update memory.

<Wt> Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Aborts

If a synchronous Data Abort or Watchpoint exception is generated by the execution of this instruction:

- Memory is not updated.
- <Ws> is not updated.

If AArch64_ExclusiveMonitorsPass() returns FALSE and the memory address, if accessed, would generate a synchronous Data Abort or Watchpoint exception, it is IMPLEMENTATION DEFINED whether the exception is generated.

Operation

```
var address : bits(64);
var data : bits(8);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescExLDST(MemOp_STORE, acqrel, tagchecked,
                                                     privileged, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
elsif rn_unknown then
    address = ARBITRARY : bits(64);
else
    address = X{64}(n);
end;

if rt_unknown then
    data = ARBITRARY : bits(8);
else
    data = X{8}(t);
end;

var status : bit = '1';
// Check whether the Exclusives monitors are set to include the
// physical memory locations corresponding to virtual address
// range [address, address+dbytes-1].
// If AArch64_ExclusiveMonitorsPass() returns FALSE and the memory address,
// if accessed, would generate a synchronous Data Abort or Watchpoint exception,
// it is IMPLEMENTATION DEFINED whether the exception is generated.
// if accessed, would generate a synchronous Data Abort exception, it is
// IMPLEMENTATION DEFINED whether the exception is generated.
// It is a limitation of Pseudocode that synchronous Data Aborts are never
// generated in this case, as Mem is not called.

// If FEAT_SPE is implemented, it is also IMPLEMENTATION DEFINED whether or not the
// physical address packet is output when permitted and when
// AArch64_ExclusiveMonitorPass() returns FALSE for a Store Exclusive instruction.
// This behavior is not reflected here due to the previously stated limitation.
if AArch64_ExclusiveMonitorsPass(address, 1, accdesc) then
    // This atomic write will be rejected if it does not refer
    // to the same physical locations after address translation.
    Mem{8}(address, accdesc) = data;
    status = ExclusiveMonitorsStatus();
end;
X{32}(s) = ZeroExtend{32}(status);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

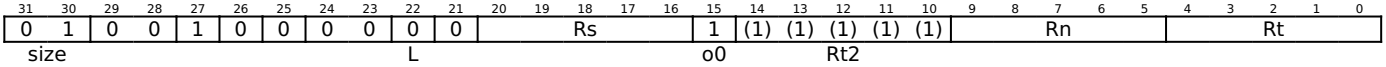
(old) htmldiff from- (new)

(old) htmldiff from- (new)

STLXRH

Store-release exclusive register halfword

This instruction stores a halfword from a 32-bit register to memory if the PE has exclusive access to the memory address, and returns a status value of 0 if the store was successful, or of 1 if no store was performed. See *Synchronization and semaphores*. The memory access is atomic. The instruction also has memory ordering semantics as described in *Load-Acquire, Store-Release*. For information about addressing modes, see *Load/Store addressing modes*.



Encoding

```
STLXRH <Ws>, <Wt>, [<Xn|SP>{, #0}]
```

Decode for this encoding

```
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let acqrel : boolean = TRUE;
let tagchecked : boolean = n != 31;

var rt_unknown : boolean = FALSE;
var rn_unknown : boolean = FALSE;
if s == t then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_DATAOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>    rt_unknown = TRUE;    // store UNKNOWN value
        when Constraint_UNDEF =>      EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>        EndOfDecode(Decode_NOP);
    end;
end;
if s == n && n != 31 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_BASEOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>    rn_unknown = TRUE;    // address is UNKNOWN
        when Constraint_UNDEF =>      EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>        EndOfDecode(Decode_NOP);
    end;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *STLXRH*.

Assembler Symbols

<Ws> Is the 32-bit name of the general-purpose register into which the status result of the store exclusive is written, encoded in the "Rs" field. The value returned is:

0
: If the operation updates memory.

1
: If the operation fails to update memory.

<Wt> Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Aborts and alignment

If a synchronous Data Abort or Watchpoint exception is generated by the execution of this instruction:

- Memory is not updated.
- <Ws> is not updated.

A non halfword-aligned memory address causes an Alignment fault Data Abort exception to be generated, subject to the following rules:

- If AArch64_ExclusiveMonitorsPass() returns TRUE, the exception is generated.
- Otherwise, it is IMPLEMENTATION DEFINED whether the exception is generated.

If AArch64_ExclusiveMonitorsPass() returns FALSE and the memory address, if accessed, would generate a synchronous Data Abort or Watchpoint exception, it is IMPLEMENTATION DEFINED whether the exception is generated.

Operation

```
var address : bits(64);
var data : bits(16);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescExLDST(MemOp_STORE, acqrel, tagchecked,
                                                     privileged, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
elsif rn_unknown then
    address = ARBITRARY : bits(64);
else
    address = X{64}(n);
end;

if rt_unknown then
    data = ARBITRARY : bits(16);
else
    data = X{16}(t);
end;

var status : bit = '1';
// Check whether the Exclusives monitors are set to include the
// physical memory locations corresponding to virtual address
// range [address, address+dbytes-1].
// If AArch64_ExclusiveMonitorsPass() returns FALSE and the memory address,
// if accessed, would generate a synchronous Data Abort or Watchpoint exception,
// it is IMPLEMENTATION DEFINED whether the exception is generated.
// if accessed, would generate a synchronous Data Abort exception, it is
// IMPLEMENTATION DEFINED whether the exception is generated.
// It is a limitation of Pseudocode that synchronous Data Aborts are never
// generated in this case, as Mem is not called.

// If FEAT_SPE is implemented, it is also IMPLEMENTATION DEFINED whether or not the
// physical address packet is output when permitted and when
// AArch64_ExclusiveMonitorPass() returns FALSE for a Store Exclusive instruction.
// This behavior is not reflected here due to the previously stated limitation.
if AArch64_ExclusiveMonitorsPass(address, 2, accdesc) then
    // This atomic write will be rejected if it does not refer
    // to the same physical locations after address translation.
    Mem{16}(address, accdesc) = data;
    status = ExclusiveMonitorsStatus();
end;
X{32}(s) = ZeroExtend{32}(status);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

STTXR

Store unprivileged exclusive register

This instruction stores a 32-bit word or a 64-bit doubleword from a register to memory if the PE has exclusive access to the memory address, and returns a status value of 0 if the store was successful, or of 1 if no store was performed. See *Synchronization and semaphores*.
Explicit Memory effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory effects operate with the restrictions determined by the Exception level at which the instruction is executed.

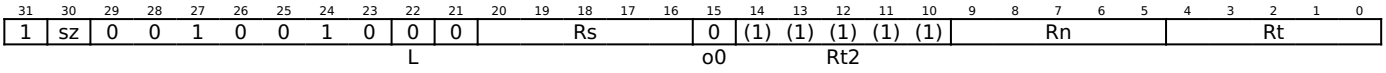
Note

For the purposes of the Exclusives monitors, and the forward progress guarantees for Load-Exclusive and Store-Exclusive loops, `STTXR` is equivalent to `STXR`.

For information about addressing modes, see *Load/Store addressing modes*.

No offset

(FEAT_LSUI)



Encoding for the 32-bit variant

Applies when `(sz == 0)`

```
STTXR <Ws>, <Wt>, [<Xn|SP>{, #0}]
```

Encoding for the 64-bit variant

Applies when `(sz == 1)`

```
STTXR <Ws>, <Xt>, [<Xn|SP>{, #0}]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSUI) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let elsize : integer{} = 32 << UInt(sz);
let acqrel : boolean = FALSE;
let tagchecked : boolean = n != 31;

var rt_unknown : boolean = FALSE;
var rn_unknown : boolean = FALSE;
if s == t then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_DATAOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>     rt_unknown = TRUE;     // store UNKNOWN value
        when Constraint_UNDEF =>       EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>         EndOfDecode(Decode_NOP);
    end;
end;
if s == n && n != 31 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_BASEOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>     rn_unknown = TRUE;     // address is UNKNOWN
        when Constraint_UNDEF =>       EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>         EndOfDecode(Decode_NOP);
    end;
end;
```

`STTXR` has the same CONSTRAINED UNPREDICTABLE behavior as `STXR`. For information about this CONSTRAINED UNPREDICTABLE behavior, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *STXR*.

Assembler Symbols

<Ws> Is the 32-bit name of the general-purpose register into which the status result of the store exclusive is written, encoded in the "Rs" field. The value returned is:

0
: If the operation updates memory.

1
: If the operation fails to update memory.

<Wt> Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Xt> Is the 64-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

Aborts and alignment

If a synchronous Data Abort or Watchpoint exception is generated by the execution of this instruction:

- Memory is not updated.
- <Ws> is not updated.

Accessing an address that is not aligned to the size of the data being accessed causes an Alignment fault Data Abort exception to be generated, subject to the following rules:

- If AArch64_ExclusiveMonitorsPass() returns TRUE, the exception is generated.
- Otherwise, it is IMPLEMENTATION DEFINED whether the exception is generated.

If AArch64_ExclusiveMonitorsPass() returns FALSE and the memory address, if accessed, would generate a synchronous Data Abort or Watchpoint exception, it is IMPLEMENTATION DEFINED whether the exception is generated.

Operation

```
var address : bits(64);
var data : bits(elsize);

let dbytes : integer{} = elsize DIV 8;
let privileged : boolean = AArch64_IsUnprivAccessPriv();
let accdesc : AccessDescriptor = CreateAccDescExLDST(MemOp_STORE, acqrel, tagchecked,
                                                     privileged, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
elsif rn_unknown then
    address = ARBITRARY : bits(64);
else
    address = X{64}(n);
end;

if rt_unknown then
    data = ARBITRARY : bits(elsize);
else
    data = X{elsize}(t);
end;

var status : bit = '1';
// Check whether the Exclusives monitors are set to include the
// physical memory locations corresponding to virtual address
// range [address, address+dbytes-1].
// If AArch64_ExclusiveMonitorsPass() returns FALSE and the memory address,
// if accessed, would generate a synchronous Data Abort or Watchpoint exception,
// it is IMPLEMENTATION DEFINED whether the exception is generated.
// if accessed, would generate a synchronous Data Abort exception, it is
// IMPLEMENTATION DEFINED whether the exception is generated.
// It is a limitation of Pseudocode that synchronous Data Aborts are never
// generated in this case, as Mem is not called.

// If FEAT_SPE is implemented, it is also IMPLEMENTATION DEFINED whether or not the
// physical address packet is output when permitted and when
// AArch64_ExclusiveMonitorPass() returns FALSE for a Store Exclusive instruction.
// This behavior is not reflected here due to the previously stated limitation.
if AArch64_ExclusiveMonitorsPass(address, dbytes, accdesc) then
    // This atomic write will be rejected if it does not refer
    // to the same physical locations after address translation.
    Mem{elsize}(address, accdesc) = data;
    status = ExclusiveMonitorsStatus();
end;
X{32}(s) = ZeroExtend{32}(status);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

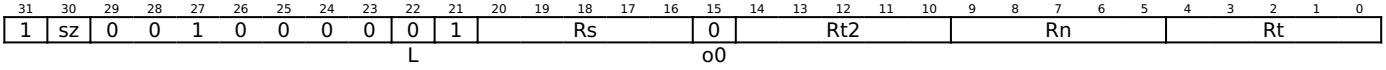
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

STXP

Store exclusive pair of registers

This instruction stores two 32-bit words or two 64-bit doublewords from two registers to a memory location if the PE has exclusive access to the memory address, and returns a status value of 0 if the store was successful, or of 1 if no store was performed. See *Synchronization and semaphores*. For information on single-copy atomicity and alignment requirements, see *Requirements for single-copy atomicity* and *Alignment of data accesses*. If a 64-bit pair Store-Exclusive succeeds, it causes a single-copy atomic update of the 128-bit memory location being updated. For information about addressing modes, see *Load/Store addressing modes*.



Encoding for the 32-bit variant

Applies when `(sz == 0)`

```
STXP <Ws>, <Wt1>, <Wt2>, [<Xn|SP>{, #0}]
```

Encoding for the 64-bit variant

Applies when `(sz == 1)`

```
STXP <Ws>, <Xt1>, <Xt2>, [<Xn|SP>{, #0}]
```

Decode for all variants of this encoding

```
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let t2 : integer{} = UInt(Rt2);
let n : integer{} = UInt(Rn);

let elsize : integer{} = 32 << UInt(sz);
let datasize : integer{} = elsize * 2;
let acqrel : boolean = FALSE;
let tagchecked : boolean = n != 31;

var rt_unknown : boolean = FALSE;
var rn_unknown : boolean = FALSE;
if s == t || s == t2 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_DATAOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>     rt_unknown = TRUE;    // store UNKNOWN value
        when Constraint_UNDEF =>       EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>         EndOfDecode(Decode_NOP);
    end;
end;
if s == n && n != 31 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_BASEOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>     rn_unknown = TRUE;    // address is UNKNOWN
        when Constraint_UNDEF =>       EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>         EndOfDecode(Decode_NOP);
    end;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *CONSTRAINED UNPREDICTABLE behaviors with Load-Exclusive/Store-Exclusive pairs* and *STXP*.

Assembler Symbols

<Ws> Is the 32-bit name of the general-purpose register into which the status result of the store exclusive is written, encoded in the "Rs" field. The value returned is:

0
: If the operation updates memory.

1
: If the operation fails to update memory.

<Wt1> Is the 32-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.

<Wt2> Is the 32-bit name of the second general-purpose register to be transferred, encoded in the "Rt2" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Xt1> Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.

<Xt2> Is the 64-bit name of the second general-purpose register to be transferred, encoded in the "Rt2" field.

Aborts and alignment

If a synchronous Data Abort or Watchpoint exception is generated by the execution of this instruction:

- Memory is not updated.
- <Ws> is not updated.

Accessing an address that is not aligned to the size of the data being accessed causes an Alignment fault Data Abort exception to be generated, subject to the following rules:

- If AArch64_ExclusiveMonitorsPass() returns TRUE, the exception is generated.
- Otherwise, it is IMPLEMENTATION DEFINED whether the exception is generated.

If AArch64_ExclusiveMonitorsPass() returns FALSE and the memory address, if accessed, would generate a synchronous Data Abort or Watchpoint exception, it is IMPLEMENTATION DEFINED whether the exception is generated.

Operation

```
var address : bits(64);
var data : bits(datasize);

let dbytes : integer{} = datasize DIV 8;
let privileged : boolean = PSTATE.EL != EL0;
let ispair : boolean = FALSE;
let accdesc : AccessDescriptor = CreateAccDescExLDST(MemOp_STORE, acqrel, tagchecked,
                                                     privileged, ispair, t, t2);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
elsif rn_unknown then
    address = ARBITRARY : bits(64);
else
    address = X{64}(n);
end;

if rt_unknown then
    data = ARBITRARY : bits(datasize);
else
    let el1 : bits(datasize DIV 2) = X{}(t);
    let el2 : bits(datasize DIV 2) = X{}(t2);
    data = if BigEndian(accdesc.acctype) then el1::el2 else el2::el1;
end;
var status : bit = '1';
// Check whether the Exclusives monitors are set to include the
// physical memory locations corresponding to virtual address
// range [address, address+dbytes-1].
// If AArch64_ExclusiveMonitorsPass() returns FALSE and the memory address,
// if accessed, would generate a synchronous Data Abort or Watchpoint exception,
// it is IMPLEMENTATION DEFINED whether the exception is generated.
// if accessed, would generate a synchronous Data Abort exception, it is
// IMPLEMENTATION DEFINED whether the exception is generated.
// It is a limitation of Pseudocode that synchronous Data Aborts are never
// generated in this case, as Mem is not called.

// If FEAT_SPE is implemented, it is also IMPLEMENTATION DEFINED whether or not the
// physical address packet is output when permitted and when
// AArch64_ExclusiveMonitorPass() returns FALSE for a Store Exclusive instruction.
// This behavior is not reflected here due to the previously stated limitation.
if AArch64_ExclusiveMonitorsPass(address, dbytes, accdesc) then
    // This atomic write will be rejected if it does not refer
    // to the same physical locations after address translation.
    Mem{datasize}(address, accdesc) = data;
    status = ExclusiveMonitorsStatus();
end;
X{32}(s) = ZeroExtend{32}(status);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

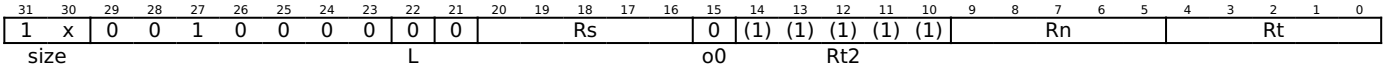
(old) htmldiff from- (new)

STXR

Store exclusive register

This instruction stores a 32-bit word or a 64-bit doubleword from a register to memory if the PE has exclusive access to the memory address, and returns a status value of 0 if the store was successful, or of 1 if no store was performed. See *Synchronization and semaphores*.

For information about addressing modes, see *Load/Store addressing modes*.



Encoding for the 32-bit variant

Applies when `(size == 10)`

```
STXR <Ws>, <Wt>, [<Xn|SP>{, #0}]
```

Encoding for the 64-bit variant

Applies when `(size == 11)`

```
STXR <Ws>, <Xt>, [<Xn|SP>{, #0}]
```

Decode for all variants of this encoding

```
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let elsize : integer{} = 8 << UInt(size);
let acqrel : boolean = FALSE;
let tagchecked : boolean = n != 31;

var rt_unknown : boolean = FALSE;
var rn_unknown : boolean = FALSE;
if s == t then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_DATAOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>     rt_unknown = TRUE;     // store UNKNOWN value
        when Constraint_UNDEF =>       EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>         EndOfDecode(Decode_NOP);
    end;
end;
if s == n && n != 31 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_BASEOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>     rn_unknown = TRUE;     // address is UNKNOWN
        when Constraint_UNDEF =>       EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>         EndOfDecode(Decode_NOP);
    end;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *STXR*.

Assembler Symbols

<Ws> Is the 32-bit name of the general-purpose register into which the status result of the store exclusive is written, encoded in the "Rs" field. The value returned is:

0
: If the operation updates memory.

1
: If the operation fails to update memory.

<Wt> Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Xt> Is the 64-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

Aborts and alignment

If a synchronous Data Abort or Watchpoint exception is generated by the execution of this instruction:

- Memory is not updated.
- <Ws> is not updated.

Accessing an address that is not aligned to the size of the data being accessed causes an Alignment fault Data Abort exception to be generated, subject to the following rules:

- If AArch64_ExclusiveMonitorsPass() returns TRUE, the exception is generated.
- Otherwise, it is IMPLEMENTATION DEFINED whether the exception is generated.

If AArch64_ExclusiveMonitorsPass() returns FALSE and the memory address, if accessed, would generate a synchronous Data Abort or Watchpoint exception, it is IMPLEMENTATION DEFINED whether the exception is generated.

Operation

```
var address : bits(64);
var data : bits(elsize);

let dbytes : integer{} = elsize DIV 8;
let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescExLDST(MemOp_STORE, acqrel, tagchecked,
                                                     privileged, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
elsif rn_unknown then
    address = ARBITRARY : bits(64);
else
    address = X{64}(n);
end;

if rt_unknown then
    data = ARBITRARY : bits(elsize);
else
    data = X{elsize}(t);
end;

var status : bit = '1';
// Check whether the Exclusives monitors are set to include the
// physical memory locations corresponding to virtual address
// range [address, address+dbytes-1].
// If AArch64_ExclusiveMonitorsPass() returns FALSE and the memory address,
// if accessed, would generate a synchronous Data Abort or Watchpoint exception,
// it is IMPLEMENTATION DEFINED whether the exception is generated.
// if accessed, would generate a synchronous Data Abort exception, it is
// IMPLEMENTATION DEFINED whether the exception is generated.
// It is a limitation of Pseudocode that synchronous Data Aborts are never
// generated in this case, as Mem is not called.

// If FEAT_SPE is implemented, it is also IMPLEMENTATION DEFINED whether or not the
// physical address packet is output when permitted and when
// AArch64_ExclusiveMonitorPass() returns FALSE for a Store Exclusive instruction.
// This behavior is not reflected here due to the previously stated limitation.
if AArch64_ExclusiveMonitorsPass(address, dbytes, accdesc) then
    // This atomic write will be rejected if it does not refer
    // to the same physical locations after address translation.
    Mem{elsize}(address, accdesc) = data;
    status = ExclusiveMonitorsStatus();
end;
X{32}(s) = ZeroExtend{32}(status);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

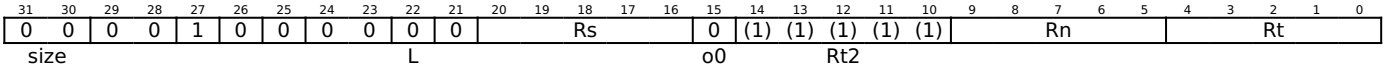
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STXRB

Store exclusive register byte

This instruction stores a byte from a register to memory if the PE has exclusive access to the memory address, and returns a status value of 0 if the store was successful, or of 1 if no store was performed. See *Synchronization and semaphores*. The memory access is atomic.

For information about addressing modes, see *Load/Store addressing modes*.



Encoding

```
STXRB <Ws>, <Wt>, [<Xn|SP>{, #0}]
```

Decode for this encoding

```
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let acqrel : boolean = FALSE;
let tagchecked : boolean = n != 31;

var rt_unknown : boolean = FALSE;
var rn_unknown : boolean = FALSE;
if s == t then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_DATAOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>     rt_unknown = TRUE;     // store UNKNOWN value
        when Constraint_UNDEF =>       EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>         EndOfDecode(Decode_NOP);
    end;
end;
if s == n && n != 31 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_BASEOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>     rn_unknown = TRUE;     // address is UNKNOWN
        when Constraint_UNDEF =>       EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>         EndOfDecode(Decode_NOP);
    end;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *STXRB*.

Assembler Symbols

<Ws> Is the 32-bit name of the general-purpose register into which the status result of the store exclusive is written, encoded in the "Rs" field. The value returned is:

0
: If the operation updates memory.

1
: If the operation fails to update memory.

<Wt> Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Aborts

If a synchronous Data Abort or Watchpoint exception is generated by the execution of this instruction:

- Memory is not updated.
- <Ws> is not updated.

If AArch64_ExclusiveMonitorsPass() returns FALSE and the memory address, if accessed, would generate a synchronous Data Abort or Watchpoint exception, it is IMPLEMENTATION DEFINED whether the exception is generated.

Operation

```
var address : bits(64);
var data : bits(8);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescExLDST(MemOp_STORE, acqrel, tagchecked,
                                                     privileged, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
elsif rn_unknown then
    address = ARBITRARY : bits(64);
else
    address = X{64}(n);
end;

if rt_unknown then
    data = ARBITRARY : bits(8);
else
    data = X{8}(t);
end;

var status : bit = '1';
// Check whether the Exclusives monitors are set to include the
// physical memory locations corresponding to virtual address
// range [address, address+dbytes-1].
// If AArch64_ExclusiveMonitorsPass() returns FALSE and the memory address,
// if accessed, would generate a synchronous Data Abort or Watchpoint exception,
// it is IMPLEMENTATION DEFINED whether the exception is generated.
// if accessed, would generate a synchronous Data Abort exception, it is
// IMPLEMENTATION DEFINED whether the exception is generated.
// It is a limitation of Pseudocode that synchronous Data Aborts are never
// generated in this case, as Mem is not called.

// If FEAT_SPE is implemented, it is also IMPLEMENTATION DEFINED whether or not the
// physical address packet is output when permitted and when
// AArch64_ExclusiveMonitorPass() returns FALSE for a Store Exclusive instruction.
// This behavior is not reflected here due to the previously stated limitation.
if AArch64_ExclusiveMonitorsPass(address, 1, accdesc) then
    // This atomic write will be rejected if it does not refer
    // to the same physical locations after address translation.
    Mem{8}(address, accdesc) = data;
    status = ExclusiveMonitorsStatus();
end;
X{32}(s) = ZeroExtend{32}(status);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel2026-06_diff_tag 2026-09-212026-09-08 1409:2244:3801

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

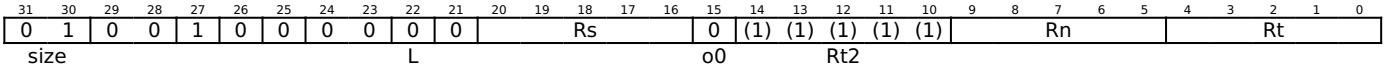
(old) htmldiff from- (new)

STXRH

Store exclusive register halfword

This instruction stores a halfword from a register to memory if the PE has exclusive access to the memory address, and returns a status value of 0 if the store was successful, or of 1 if no store was performed. See *Synchronization and semaphores*. The memory access is atomic.

For information about addressing modes, see *Load/Store addressing modes*.



Encoding

```
STXRH <Ws>, <Wt>, [<Xn|SP>{, #0}]
```

Decode for this encoding

```
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let acqrel : boolean = FALSE;
let tagchecked : boolean = n != 31;

var rt_unknown : boolean = FALSE;
var rn_unknown : boolean = FALSE;
if s == t then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_DATAOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>    rt_unknown = TRUE;    // store UNKNOWN value
        when Constraint_UNDEF =>      EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>        EndOfDecode(Decode_NOP);
    end;
end;
if s == n && n != 31 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_BASEOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>    rn_unknown = TRUE;    // address is UNKNOWN
        when Constraint_UNDEF =>      EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>        EndOfDecode(Decode_NOP);
    end;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *STXRH*.

Assembler Symbols

<Ws> Is the 32-bit name of the general-purpose register into which the status result of the store exclusive is written, encoded in the "Rs" field. The value returned is:

0
: If the operation updates memory.

1
: If the operation fails to update memory.

<Wt> Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Aborts and alignment

If a synchronous Data Abort or Watchpoint exception is generated by the execution of this instruction:

- Memory is not updated.
- <Ws> is not updated.

A non halfword-aligned memory address causes an Alignment fault Data Abort exception to be generated, subject to the following rules:

- If AArch64_ExclusiveMonitorsPass() returns TRUE, the exception is generated.
- Otherwise, it is IMPLEMENTATION DEFINED whether the exception is generated.

If AArch64_ExclusiveMonitorsPass() returns FALSE and the memory address, if accessed, would generate a synchronous Data Abort or Watchpoint exception, it is IMPLEMENTATION DEFINED whether the exception is generated.

Operation

```
var address : bits(64);
var data : bits(16);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescExLDST(MemOp_STORE, acqrel, tagchecked,
                                                     privileged, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
elsif rn_unknown then
    address = ARBITRARY : bits(64);
else
    address = X{64}(n);
end;

if rt_unknown then
    data = ARBITRARY : bits(16);
else
    data = X{16}(t);
end;

var status : bit = '1';
// Check whether the Exclusives monitors are set to include the
// physical memory locations corresponding to virtual address
// range [address, address+dbytes-1].
// If AArch64_ExclusiveMonitorsPass() returns FALSE and the memory address,
// if accessed, would generate a synchronous Data Abort or Watchpoint exception,
// it is IMPLEMENTATION DEFINED whether the exception is generated.
// if accessed, would generate a synchronous Data Abort exception, it is
// IMPLEMENTATION DEFINED whether the exception is generated.
// It is a limitation of Pseudocode that synchronous Data Aborts are never
// generated in this case, as Mem is not called.

// If FEAT_SPE is implemented, it is also IMPLEMENTATION DEFINED whether or not the
// physical address packet is output when permitted and when
// AArch64_ExclusiveMonitorPass() returns FALSE for a Store Exclusive instruction.
// This behavior is not reflected here due to the previously stated limitation.
if AArch64_ExclusiveMonitorsPass(address, 2, accdesc) then
    // This atomic write will be rejected if it does not refer
    // to the same physical locations after address translation.
    Mem{16}(address, accdesc) = data;
    status = ExclusiveMonitorsStatus();
end;
X{32}(s) = ZeroExtend{32}(status);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel2026-06_diff_tag 2026-09-212026-09-08 1409:2244:3801

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

STZGM

Store Allocation Tag and zero multiple

This instruction writes a naturally aligned block of N Allocation Tags and stores zero to the associated data locations, where the size of N is identified in DCZID_EL0.BS, and the Allocation Tag is taken from the source register bits<3:0>.

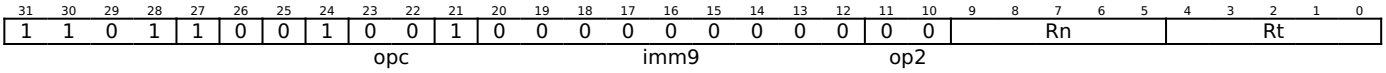
If the value of HCR_EL2.TGE is 0 and the Effective value of SCTLR2_EL1.GM0 is 0, this~~This~~ instruction is UNDEFINED at EL0.~~This instruction generates an Unchecked access.~~

If the value of HCR_EL2.TGE is 1 and the Effective value of SCTLR2_EL2.GM0 is 0, this instruction is UNDEFINED at EL0.

This instruction generates an Unchecked access.

Integer

(FEAT_MTE2)



Encoding

```
STZGM <Xt>, [<Xn|SP>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_MTE2) then EndOfDecode(Decode_UNDEF); end;
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
```

Assembler Symbols

<Xt>	Is the 64-bit name of the general-purpose source register, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
if !~~if~~IsGMEnabledAtEL0() && PSTATE.EL == EL0 then Undefined(); end;

let data : bits(64) = X{64}(t);
~~then Undefined(); end;~~

~~let data : bits(64) = X{}(t);~~
let tag : bits(4) = data[3:0];
var address : bits(64);
if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let size : integer{} = 4 * 2 ^ UInt(DCZID_EL0().BS);
~~let size : integer{} = 4 * (2 ^ (UInt(DCZID_EL0().BS)));~~
address = AlignDownSize(address, size);
let count : integer = size >> LOG2_TAG_GRANULE;
let stzgm : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescLDGSTG(MemOp_STORE, stzgm, t);

for i = 0 to count-1 do
    AArch64_MemTag(address, accdesc) = tag;
    Mem{8*TAG_GRANULE}(address, accdesc) = Zeros{8*TAG_GRANULE};
    address = AddressIncrement(address, TAG_GRANULE, accdesc);
end;
```

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

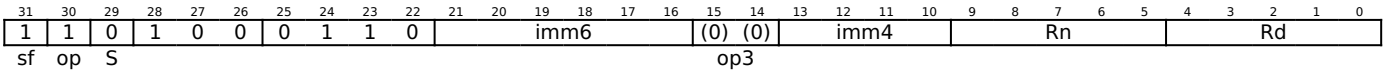
SUBG

Subtract with tag

This instruction subtracts an immediate value scaled by the Tag Granule from the address in the source register, modifies the Logical Address Tag of the address using an immediate value, and writes the result to the destination register. Tags specified in GCR_EL1.Exclude are excluded from the possible outputs when modifying the Logical Address Tag.

Integer

(FEAT_MTE)



Encoding

```
SUBG <Xd|SP>, <Xn|SP>, #<uimm6>, #<uimm4>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_MTE) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let tag_offset : bits(4) = imm4;
let offset : bits(64) = LSL(ZeroExtend{64}(imm6), LOG2_TAG_GRANULE);
```

Assembler Symbols

<Xd|SP>	Is the 64-bit name of the destination general-purpose register or stack pointer, encoded in the "Rd" field.
<Xn|SP>	Is the 64-bit name of the source general-purpose register or stack pointer, encoded in the "Rn" field.
<uimm6>	Is an unsigned immediate, a multiple of 16 in the range 0 to 1008, encoded in the "imm6" field.
<uimm4>	Is an unsigned immediate, in the range 0 to 15, encoded in the "imm4" field.

Operation

```
let operand1 : bits(64) = if n == 31 then SP{64}() else X{64}(n);
let start_tag : bits(4) = AArch64_AllocationTagFromAddress(operand1);
let exclude : bits(16) = GCR_EL1().Exclude;
let rtag : bits(4) = AArch64_ChooseNonExcludedTagOrZero(start_tag, tag_offset, exclude);
var result : bits(64);

(result, -) = AddWithCarry{64}(operand1, NOT offset, '1');
~~{64}(operand1, NOT(offset), '1');~~

result = AArch64_AddressWithAllocationTag(result, rtag);

if d == 31 then
    SP{64}() = result;
else
    X{64}(d) = result;
end;
```

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

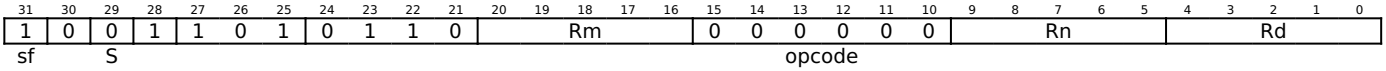
SUBP

Subtract pointer

This instruction subtracts the 56-bit address held in the second source register from the 56-bit address held in the first source register, sign-extends the result to 64 bits, and writes the result to the destination register.

Integer

(FEAT_MTE)



Encoding

```
SUBP <Xd>, <Xn|SP>, <Xm|SP>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_MTE) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
```

Assembler Symbols

<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn|SP>	Is the 64-bit name of the first source general-purpose register or stack pointer, encoded in the "Rn" field.
<Xm|SP>	Is the 64-bit name of the second general-purpose source register or stack pointer, encoded in the "Rm" field.

Operation

```
var operand1 : bits(64) = if n == 31 then SP{64}() else X{64}(n);
var operand2 : bits(64) = if m == 31 then SP{64}() else X{64}(m);
operand1 = SignExtend{64}(operand1[55:0]);
operand2 = NOT SignExtend{64}(operand2[55:0]);
~~operand2 = NOT(SignExtend{64}(operand2[55:0]));~~
var result : bits(64);

(result, -) = AddWithCarry{64}(operand1, operand2, '1');

X{64}(d) = result;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SUBPS

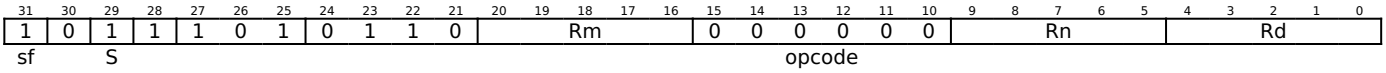
Subtract pointer, setting flags

This instruction subtracts the 56-bit address held in the second source register from the 56-bit address held in the first source register, sign-extends the result to 64 bits, and writes the result to the destination register. It updates the condition flags based on the result of the subtraction.

This instruction is used by the alias CMPP.

Integer

(FEAT_MTE)



Encoding

```
SUBPS <Xd>, <Xn|SP>, <Xm|SP>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_MTE) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
```

Assembler Symbols

<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn|SP>	Is the 64-bit name of the first source general-purpose register or stack pointer, encoded in the "Rn" field.
<Xm|SP>	Is the 64-bit name of the second general-purpose source register or stack pointer, encoded in the "Rm" field.

Alias Conditions

Alias	Is preferred when
CMPP	`S == '1' && Rd == '11111'`

Operation

```
var operand1 : bits(64) = if n == 31 then SP{64}() else X{64}(n);
var operand2 : bits(64) = if m == 31 then SP{64}() else X{64}(m);
operand1 = SignExtend{64}(operand1[55:0]);
operand2 = NOT SignExtend{64}(operand2[55:0]);
operand2 = NOT(SignExtend{64}(operand2[55:0]));
var result : bits(64);

var nzcv : bits(4);

(result, nzcv) = AddWithCarry{64}(operand1, operand2, '1');

X{64}(d) = result;
PSTATE.[N,Z,C,V] = nzcv;
```

2026-09_rel2026-06_diff_tag 2026-09-212026-09-08 1409:2244:3801

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

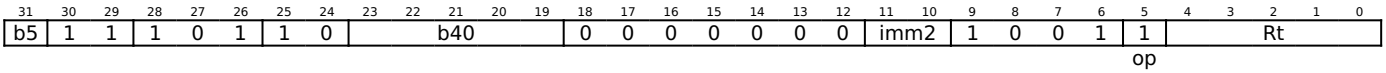
TFLTNZ

Test bit and fault if nonzero

This instruction compares the value of a bit in a general-purpose register with zero, and conditionally generates a Conditional fault exception if the comparison is not equal. This instruction does not affect condition flags.

Fault

(FEAT_CFLT)



Encoding

```
TFLTNZ #<reason>, <R><t>, #<imm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_CFLT) then EndOfDecode(Decode_UNDEF); end;

let datasize : integer{} = 32 << UInt(b5);
let t : integer = UInt(Rt);
let bit_pos : integer = UInt(b5::b40);
let reason : bits(16) = ZeroExtend{}(imm2);
```

Assembler Symbols

<reason> Is a 2-bit unsigned immediate, in the range 0 to 3, encoded in the "imm2" field.

<R> Is a width specifier, encoded in “b5”:

b5	<R>
0	W
1	X

In assembler source code an 'X' specifier is always permitted, but a 'W' specifier is permitted only when the bit number is less than 32.

<t> Is the number [0-30] of the general-purpose register to be tested or the name ZR (31), encoded in the "Rt" field.

<imm> Is the bit number to be tested, in the range 0 to 63, encoded in "b5:b40".

Operation

```
CheckCFLTEnabled();

let operand : bits(datasize) = X{}(t);

if operand[bit_pos] != '0' then
    let isflt : boolean = FALSE;
    AArch64_ConditionalFault(isflt, reason);
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

no old file htmldiff from- (new)

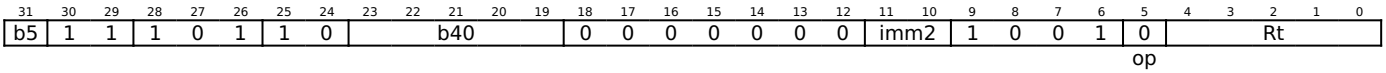
TFLTZ

Test bit and fault if zero

This instruction compares the value of a bit in a general-purpose register with zero, and conditionally generates a Conditional fault exception if the comparison is equal. This instruction does not affect condition flags.

Fault

(FEAT_CFLT)



Encoding

```
TFLTZ #<reason>, <R><t>, #<imm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_CFLT) then EndOfDecode(Decode_UNDEF); end;

let datasize : integer{} = 32 << UInt(b5);
let t : integer = UInt(Rt);
let bit_pos : integer = UInt(b5::b40);
let reason : bits(16) = ZeroExtend{}(imm2);
```

Assembler Symbols

<reason> Is a 2-bit unsigned immediate, in the range 0 to 3, encoded in the "imm2" field.

<R> Is a width specifier, encoded in “b5”:

b5	<R>
0	W
1	X

In assembler source code an 'X' specifier is always permitted, but a 'W' specifier is permitted only when the bit number is less than 32.

<t> Is the number [0-30] of the general-purpose register to be tested or the name ZR (31), encoded in the "Rt" field.

<imm> Is the bit number to be tested, in the range 0 to 63, encoded in "b5:b40".

Operation

```
CheckCFLTEnabled();

let operand : bits(datasize) = X{}(t);

if operand[bit_pos] == '0' then
    let isflt : boolean = FALSE;
    AArch64_ConditionalFault(isflt, reason);
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

no old file htmldiff from- (new)

XPACD, XPACI, XPACLRI

Strip Pointer Authentication Code

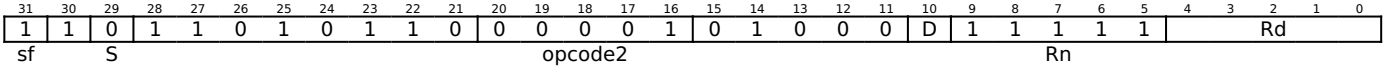
This instruction removes the Pointer Authentication Code from an address. The address is in the specified general-purpose register for `XPACI` and `XPACD`, and is in LR for `XPACLRI`.

The `XPACD` instruction is used for data addresses, and `XPACI` and `XPACLRI` are used for instruction addresses.

It has encodings from 2 classes: Integer and System

Integer

(FEAT_PAuth)



Encoding for the XPACD variant

Applies when `(D == 1)`

```
XPACD <Xd>
```

Encoding for the XPACI variant

Applies when `(D == 0)`

```
XPACI <Xd>
```

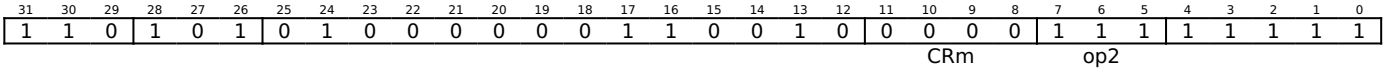
Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_PAuth) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let data : boolean = D == '1';~~let data : boolean = (D == '1');~~
```

System

(FEAT_PAuth)



Encoding

```
XPACLRI
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_PAuth) then EndOfDecode(Decode_NOP); end;

let d : integer{} = 30;
~~let d : integer = 30;~~
let data : boolean = FALSE;
```

Assembler Symbols

<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

Operation

```
X{64}(d) = Strip(X{64}(d), data);
```

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

A64 -- SIMD and Floating-point Instructions (alphabetic order)

ABS: Absolute value (vector).

ADD (vector): Add (vector).

ADDHN, ADDHN2: Add returning high narrow.

ADDP (scalar): Add pair of elements (scalar).

ADDP (vector): Add pairwise (vector).

ADDV: Add across vector.

AESD: AES single round decryption.

AESE: AES single round encryption.

AESIMC: AES inverse mix columns.

AESMC: AES mix columns.

AND (vector): Bitwise AND (vector).

BCAX: Bit clear and exclusive-OR.

BF1CVTL, BF1CVTL2, BF2CVTL, BF2CVTL2: 8-bit floating-point convert to BFloat16 (vector).

BFCVT: Single-precision convert to BFloat16 (scalar).

BFCVTN, BFCVTN2: Single-precision convert to BFloat16 (vector).

BFDOT (by element): BFloat16 dot product to single-precision (vector, by element).

BFDOT (vector): BFloat16 dot product to single-precision (vector).

BFMLALB, BFMLALT (by element): BFloat16 multiply-add to single-precision (by element).

BFMLALB, BFMLALT (vector): BFloat16 multiply-add to single-precision (vector).

BFMMLA (widening): BFloat16 matrix multiply-accumulate to single-precision.

BIC (vector, immediate): Bitwise bit clear (vector, immediate).

BIC (vector, register): Bitwise bit clear (vector, register).

BIF: Bitwise insert if false.

BIT: Bitwise insert if true.

BSL: Bitwise select.

CLS (vector): Count leading sign bits (vector).

CLZ (vector): Count leading zero bits (vector).

CMEQ (register): Compare bitwise equal (vector).

CMEQ (zero): Compare bitwise equal to zero (vector).

CMGE (register): Compare signed greater than or equal (vector).

CMGE (zero): Compare signed greater than or equal to zero (vector).

CMGT (register): Compare signed greater than (vector).

CMGT (zero): Compare signed greater than zero (vector).

CMHI (register): Compare unsigned higher (vector).

CMHS (register): Compare unsigned higher or same (vector).

CMLE (zero): Compare signed less than or equal to zero (vector).

CMLT (zero): Compare signed less than zero (vector).

CMTST: Compare bitwise test bits nonzero (vector).

CNT: Population count per byte.

DUP (element): Duplicate vector element to vector or scalar.

DUP (general): Duplicate general-purpose register to vector.

EOR (vector): Bitwise exclusive-OR (vector).

EOR3: Three-way exclusive-OR.

EXT: Extract vector from pair of vectors.

F1CVTL, F1CVTL2, F2CVTL, F2CVTL2: 8-bit floating-point convert to half-precision (vector).

FABD: Floating-point absolute difference (vector).

FABS (scalar): Floating-point absolute value (scalar).

FABS (vector): Floating-point absolute value (vector).

FACGE: Floating-point absolute compare greater than or equal (vector).

FACGT: Floating-point absolute compare greater than (vector).

FADD (scalar): Floating-point add (scalar).

FADD (vector): Floating-point add (vector).

FADDP (scalar): Floating-point add pair of elements (scalar).

FADDP (vector): Floating-point add pairwise (vector).

FAMAX: Floating-point absolute maximum.

FAMIN: Floating-point absolute minimum.

FCADD: Floating-point complex add.

FCCMP: Floating-point conditional quiet compare (scalar).

FCCMPE: Floating-point conditional signaling compare (scalar).

FCMEQ (register): Floating-point compare equal (vector).

FCMEQ (zero): Floating-point compare equal to zero (vector).

FCMGE (register): Floating-point compare greater than or equal (vector).

FCMGE (zero): Floating-point compare greater than or equal to zero (vector).

FCMGT (register): Floating-point compare greater than (vector).

FCMGT (zero): Floating-point compare greater than zero (vector).

FCMLA: Floating-point complex multiply accumulate.

FCMLA (by element): Floating-point complex multiply accumulate (by element).

FCMLE (zero): Floating-point compare less than or equal to zero (vector).

FCMLT (zero): Floating-point compare less than zero (vector).

FCMP: Floating-point quiet compare (scalar).

FCMPE: Floating-point signaling compare (scalar).

FCSEL: Floating-point conditional select (scalar).

FCVT: Floating-point convert precision (scalar).

FCVTAS (scalar SIMD&FP): Floating-point convert to signed integer, rounding to nearest with ties to away (scalar SIMD&FP).

FCVTAS (scalar): Floating-point convert to signed integer, rounding to nearest with ties to away (scalar).

FCVTAS (vector): Floating-point convert to signed integer, rounding to nearest with ties to away (vector).

FCVTAU (scalar SIMD&FP): Floating-point convert to unsigned integer, rounding to nearest with ties to away (scalar SIMD&FP).

FCVTAU (scalar): Floating-point convert to unsigned integer, rounding to nearest with ties to away (scalar).

FCVTAU (vector): Floating-point convert to unsigned integer, rounding to nearest with ties to away (vector).

FCVTL, FCVTL2: Floating-point convert to higher precision long (vector).

FCVTMS (scalar SIMD&FP): Floating-point convert to signed integer, rounding toward minus infinity (scalar SIMD&FP).

FCVTMS (scalar): Floating-point convert to signed integer, rounding toward minus infinity (scalar).

FCVTMS (vector): Floating-point convert to signed integer, rounding toward minus infinity (vector).

FCVTMU (scalar SIMD&FP): Floating-point convert to unsigned integer, rounding toward minus infinity (scalar SIMD&FP).

FCVTMU (scalar): Floating-point convert to unsigned integer, rounding toward minus infinity (scalar).

FCVTMU (vector): Floating-point convert to unsigned integer, rounding toward minus infinity (vector).

FCVTN (half-precision to 8-bit floating-point): Half-precision convert to 8-bit floating-point (vector).

FCVTN, FCVTN2 (double to single-precision, single to half-precision): Floating-point convert to lower precision narrow (vector).

FCVTN, FCVTN2 (single-precision to 8-bit floating-point): Single-precision convert to 8-bit floating-point (vector).

FCVTNS (scalar SIMD&FP): Floating-point convert to signed integer, rounding to nearest with ties to even (scalar SIMD&FP).

FCVTNS (scalar): Floating-point convert to signed integer, rounding to nearest with ties to even (scalar).

FCVTNS (vector): Floating-point convert to signed integer, rounding to nearest with ties to even (vector).

FCVTNU (scalar SIMD&FP): Floating-point convert to unsigned integer, rounding to nearest with ties to even (scalar SIMD&FP).

FCVTNU (scalar): Floating-point convert to unsigned integer, rounding to nearest with ties to even (scalar).

FCVTNU (vector): Floating-point convert to unsigned integer, rounding to nearest with ties to even (vector).

FCVTPS (scalar SIMD&FP): Floating-point convert to signed integer, rounding toward plus infinity (scalar SIMD&FP).

FCVTPS (scalar): Floating-point convert to signed integer, rounding toward plus infinity (scalar).

FCVTPS (vector): Floating-point convert to signed integer, rounding toward plus infinity (vector).

FCVTPU (scalar SIMD&FP): Floating-point convert to unsigned integer, rounding toward plus infinity (scalar SIMD&FP).

FCVTPU (scalar): Floating-point convert to unsigned integer, rounding toward plus infinity (scalar).

FCVTPU (vector): Floating-point convert to unsigned integer, rounding toward plus infinity (vector).

FCVTXN, FCVTXN2: Floating-point convert to lower precision narrow, rounding to odd (vector).

FCVTZS (scalar SIMD&FP): Floating-point convert to signed integer, rounding toward zero (scalar SIMD&FP).

FCVTZS (scalar, fixed-point): Floating-point convert to signed fixed-point, rounding toward zero (scalar).

FCVTZS (scalar, integer): Floating-point convert to signed integer, rounding toward zero (scalar).

FCVTZS (vector, fixed-point): Floating-point convert to signed fixed-point, rounding toward zero (vector).

FCVTZS (vector, integer): Floating-point convert to signed integer, rounding toward zero (vector).

FCVTZU (scalar SIMD&FP): Floating-point convert to unsigned integer, rounding toward zero (scalar SIMD&FP).

FCVTZU (scalar, fixed-point): Floating-point convert to unsigned fixed-point, rounding toward zero (scalar).

FCVTZU (scalar, integer): Floating-point convert to unsigned integer, rounding toward zero (scalar).

FCVTZU (vector, fixed-point): Floating-point convert to unsigned fixed-point, rounding toward zero (vector).

FCVTZU (vector, integer): Floating-point convert to unsigned integer, rounding toward zero (vector).

FDIV (scalar): Floating-point divide (scalar).

FDIV (vector): Floating-point divide (vector).

FDOT (8-bit floating-point to half-precision, by element): 8-bit floating-point dot product to half-precision (vector, by element).

FDOT (8-bit floating-point to half-precision, vector): 8-bit floating-point dot product to half-precision (vector).

FDOT (8-bit floating-point to single-precision, by element): 8-bit floating-point dot product to single-precision (vector, by element).

FDOT (8-bit floating-point to single-precision, vector): 8-bit floating-point dot product to single-precision (vector).

FDOT (half-precision to single-precision, by element): Half-precision dot product to single-precision (vector, by element).

FDOT (half-precision to single-precision, vector): Half-precision dot product to single-precision (vector).

FJCVTZS: Floating-point JavaScript convert to signed fixed-point, rounding toward zero.

FMADD: Floating-point fused multiply-add (scalar).

FMAX (scalar): Floating-point maximum (scalar).

FMAX (vector): Floating-point maximum (vector).

FMAXNM (scalar): Floating-point maximum number (scalar).

FMAXNM (vector): Floating-point maximum number (vector).

FMAXNMP (scalar): Floating-point maximum number of pair of elements (scalar).

FMAXNMP (vector): Floating-point maximum number pairwise (vector).

FMAXNMV: Floating-point maximum number across vector.

FMAXP (scalar): Floating-point maximum of pair of elements (scalar).

FMAXP (vector): Floating-point maximum pairwise (vector).

FMAXV: Floating-point maximum across vector.

FMIN (scalar): Floating-point minimum (scalar).

FMIN (vector): Floating-point minimum (vector).

FMINNM (scalar): Floating-point minimum number (scalar).

FMINNM (vector): Floating-point minimum number (vector).

FMINNMP (scalar): Floating-point minimum number of pair of elements (scalar).

FMINNMP (vector): Floating-point minimum number pairwise (vector).

FMINNMV: Floating-point minimum number across vector.

FMINP (scalar): Floating-point minimum of pair of elements (scalar).

FMINP (vector): Floating-point minimum pairwise (vector).

FMINV: Floating-point minimum across vector.

FMLA (by element): Floating-point fused multiply-add to accumulator (by element).

FMLA (vector): Floating-point fused multiply-add to accumulator (vector).

FMLAL, FMLAL2 (by element): Floating-point fused multiply-add long to accumulator (by element).

FMLAL, FMLAL2 (vector): Floating-point fused multiply-add long to accumulator (vector).

FMLALB, FMLALT (by element): 8-bit floating-point multiply-add to half-precision (vector, by element).

FMLALB, FMLALT (vector): 8-bit floating-point multiply-add to half-precision (vector).

FMLALLBB, FMLALLBT, FMLALLTB, FMLALLTT (by element): 8-bit floating-point multiply-add to single-precision (vector, by element).

FMLALLBB, FMLALLBT, FMLALLTB, FMLALLTT (vector): 8-bit floating-point multiply-add to single-precision (vector).

FMLS (by element): Floating-point fused multiply-subtract from accumulator (by element).

FMLS (vector): Floating-point fused multiply-subtract from accumulator (vector).

FMLSL, FMLSL2 (by element): Floating-point fused multiply-subtract long from accumulator (by element).

FMLSL, FMLSL2 (vector): Floating-point fused multiply-subtract long from accumulator (vector).

FMMLA (non-widening): Half-precision matrix multiply-accumulate.

FMMLA (widening, 8-bit floating-point to half-precision): 8-bit floating-point matrix multiply-accumulate to half-precision.

FMMLA (widening, 8-bit floating-point to single-precision): 8-bit floating-point matrix multiply-accumulate to single-precision.

FMMLA (widening, half-precision to single-precision): Half-precision matrix multiply-accumulate to single-precision.

FMOV (general): Floating-point move to or from general-purpose register without conversion.

FMOV (register): Floating-point move register without conversion.

FMOV (scalar, immediate): Floating-point move immediate (scalar).

FMOV (vector, immediate): Floating-point move immediate (vector).

FMSUB: Floating-point fused multiply-subtract (scalar).

FMUL (by element): Floating-point multiply (by element).

FMUL (scalar): Floating-point multiply (scalar).

FMUL (vector): Floating-point multiply (vector).

FMULX: Floating-point multiply extended.

FMULX (by element): Floating-point multiply extended (by element).

FNEG (scalar): Floating-point negate (scalar).

FNEG (vector): Floating-point negate (vector).

FNMADD: Floating-point negated fused multiply-add (scalar).

FNMSUB: Floating-point negated fused multiply-subtract (scalar).

FNMUL (scalar): Floating-point multiply-negate (scalar).

FRECPE: Floating-point reciprocal estimate.

FRECPS: Floating-point reciprocal step.

FRECPX: Floating-point reciprocal exponent (scalar).

FRINT32X (scalar): Floating-point round to 32-bit integer, using current rounding mode (scalar).

FRINT32X (vector): Floating-point round to 32-bit integer, using current rounding mode (vector).

FRINT32Z (scalar): Floating-point round to 32-bit integer toward zero (scalar).

FRINT32Z (vector): Floating-point round to 32-bit integer toward zero (vector).

FRINT64X (scalar): Floating-point round to 64-bit integer, using current rounding mode (scalar).

FRINT64X (vector): Floating-point round to 64-bit integer, using current rounding mode (vector).

FRINT64Z (scalar): Floating-point round to 64-bit integer toward zero (scalar).

FRINT64Z (vector): Floating-point round to 64-bit integer toward zero (vector).

FRINTA (scalar): Floating-point round to integral, to nearest with ties to away (scalar).

FRINTA (vector): Floating-point round to integral, to nearest with ties to away (vector).

FRINTI (scalar): Floating-point round to integral, using current rounding mode (scalar).

FRINTI (vector): Floating-point round to integral, using current rounding mode (vector).

FRINTM (scalar): Floating-point round to integral, toward minus infinity (scalar).

FRINTM (vector): Floating-point round to integral, toward minus infinity (vector).

FRINTN (scalar): Floating-point round to integral, to nearest with ties to even (scalar).

FRINTN (vector): Floating-point round to integral, to nearest with ties to even (vector).

FRINTP (scalar): Floating-point round to integral, toward plus infinity (scalar).

FRINTP (vector): Floating-point round to integral, toward plus infinity (vector).

FRINTX (scalar): Floating-point round to integral exact, using current rounding mode (scalar).

FRINTX (vector): Floating-point round to integral exact, using current rounding mode (vector).

FRINTZ (scalar): Floating-point round to integral, toward zero (scalar).

FRINTZ (vector): Floating-point round to integral, toward zero (vector).

FRSQRTE: Floating-point reciprocal square root estimate.

FRSQRTS: Floating-point reciprocal square root step.

FSCALE: Floating-point adjust exponent by vector.

FSQRT (scalar): Floating-point square root (scalar).

FSQRT (vector): Floating-point square root (vector).

FSUB (scalar): Floating-point subtract (scalar).

FSUB (vector): Floating-point subtract (vector).

INS (element): Insert vector element from another vector element.

INS (general): Insert vector element from general-purpose register.

LD1 (multiple structures): Load multiple single-element structures to one, two, three, or four registers.

LD1 (single structure): Load one single-element structure to one lane of one register.

LD1R: Load one single-element structure and replicate to all lanes (of one register).

LD2 (multiple structures): Load multiple 2-element structures to two registers.

LD2 (single structure): Load single 2-element structure to one lane of two registers.

LD2R: Load single 2-element structure and replicate to all lanes of two registers.

LD3 (multiple structures): Load multiple 3-element structures to three registers.

LD3 (single structure): Load single 3-element structure to one lane of three registers.

LD3R: Load single 3-element structure and replicate to all lanes of three registers.

LD4 (multiple structures): Load multiple 4-element structures to four registers.

LD4 (single structure): Load single 4-element structure to one lane of four registers.

LD4R: Load single 4-element structure and replicate to all lanes of four registers.

LDAP1 (SIMD&FP): Load-acquire RCpc one single-element structure to one lane of one register.

LDAPUR (SIMD&FP): Load-acquire RCpc SIMD&FP register (unscaled offset).

LDBFADD, LDBFADDA, LDBFADDAL, LDBFADDL: Atomic BFloat16 add.

LDBFMAX, LDBFMAXA, LDBFMAXAL, LDBFMAXL: Atomic BFloat16 maximum.

LDBFMAXNM, LDBFMAXNMA, LDBFMAXNMAL, LDBFMAXNML: Atomic BFloat16 maximum number.

LDBFMIN, LDBFMINA, LDBFMINAL, LDBFMINL: Atomic BFloat16 minimum.

LDBFMINNM, LDBFMINNMA, LDBFMINNMAL, LDBFMINNML: Atomic BFloat16 minimum number.

LDFADD, LDFADDA, LDFADDAL, LDFADDL: Atomic floating-point add.

LDFMAX, LDFMAXA, LDFMAXAL, LDFMAXL: Atomic floating-point maximum.

LDFMAXNM, LDFMAXNMA, LDFMAXNMAL, LDFMAXNML: Atomic floating-point maximum number.

LDFMIN, LDFMINA, LDFMINAL, LDFMINL: Atomic floating-point minimum.

LDFMINNM, LDFMINNMA, LDFMINNMAL, LDFMINNML: Atomic floating-point minimum number.

LDNP (SIMD&FP): Load pair of SIMD&FP registers, with non-temporal hint.

LDP (SIMD&FP): Load pair of SIMD&FP registers.

LDR (immediate, SIMD&FP): Load SIMD&FP register (immediate offset).

LDR (literal, SIMD&FP): Load SIMD&FP register (PC-relative literal).

LDR (register, SIMD&FP): Load SIMD&FP register (register offset).

LDTNP (SIMD&FP): Load unprivileged pair of SIMD&FP registers, with non-temporal hint.

LDTP (SIMD&FP): Load unprivileged pair of SIMD&FP registers.

LDUR (SIMD&FP): Load SIMD&FP register (unscaled offset).

LUTI2: Lookup table read with 2-bit indices.

LUTI4: Lookup table read with 4-bit indices.

MLA (by element): Multiply-add to accumulator (vector, by element).

MLA (vector): Multiply-add to accumulator (vector).

MLS (by element): Multiply-subtract from accumulator (vector, by element).

MLS (vector): Multiply-subtract from accumulator (vector).

MOV (element): Move vector element to another vector element: an alias of INS (element).

MOV (from general): Move general-purpose register to a vector element: an alias of INS (general).

MOV (scalar): Move vector element to scalar: an alias of DUP (element).

MOV (to general): Move vector element to general-purpose register: an alias of UMOV.

MOV (vector): Move vector: an alias of ORR (vector, register).

MOVI: Move immediate (vector).

MUL (by element): Multiply (vector, by element).

MUL (vector): Multiply (vector).

MVN: Bitwise NOT (vector): an alias of NOT.

MVNI: Move inverted immediate (vector).

NEG (vector): Negate (vector).

NOT: Bitwise NOT (vector).

ORN (vector): Bitwise inclusive OR NOT (vector).

ORR (vector, immediate): Bitwise inclusive OR (vector, immediate).

ORR (vector, register): Bitwise inclusive OR (vector, register).

PMUL: Polynomial multiply.

PMULL, PMULL2: Polynomial multiply long.

RADDHN, RADDHN2: Rounding add returning high narrow.

RAX1: Rotate and exclusive-OR.

RBIT (vector): Reverse bit order (vector).

REV16 (vector): Reverse elements in 16-bit halfwords (vector).

REV32 (vector): Reverse elements in 32-bit words (vector).

REV64: Reverse elements in 64-bit doublewords (vector).

RSHRN, RSHRN2: Rounding shift right narrow (immediate).

RSUBHN, RSUBHN2: Rounding subtract returning high narrow.

SABA: Signed absolute difference and accumulate.

SABAL, SABAL2: Signed absolute difference and accumulate long.

SABD: Signed absolute difference.

SABDL, SABDL2: Signed absolute difference long.

SADALP: Signed add and accumulate long pairwise.

SADDL, SADDL2: Signed add long (vector).

SADDLP: Signed add long pairwise.

SADDLV: Signed add long across vector.

SADDW, SADDW2: Signed add wide.

SCVTF (scalar SIMD&FP): Signed integer convert to floating-point (scalar SIMD&FP).

SCVTF (scalar, fixed-point): Signed fixed-point convert to floating-point (scalar).

SCVTF (scalar, integer): Signed integer convert to floating-point (scalar).

SCVTF (vector, fixed-point): Signed fixed-point convert to floating-point (vector).

SCVTF (vector, integer): Signed integer convert to floating-point (vector).

SDOT (by element): Dot product signed arithmetic (vector, by element).

SDOT (vector): Dot product signed arithmetic (vector).

SHA1C: SHA1 hash update (choose).

SHA1H: SHA1 fixed rotate.

SHA1M: SHA1 hash update (majority).

SHA1P: SHA1 hash update (parity).

SHA1SU0: SHA1 schedule update 0.

SHA1SU1: SHA1 schedule update 1.

SHA256H: SHA-256 hash update (part 1).

SHA256H2: SHA-256 hash update (part 2).

SHA256SU0: SHA-256 schedule update 0.

SHA256SU1: SHA-256 schedule update 1.

SHA512H: SHA-512 hash update part 1.

SHA512H2: SHA-512 hash update part 2.

SHA512SU0: SHA-512 schedule update 0.

SHA512SU1: SHA-512 schedule update 1.

SHADD: Signed halving add.

SHL: Shift left (immediate).

SHLL, SHLL2: Shift left long (by element size).

SHRN, SHRN2: Shift right narrow (immediate).

SHSUB: Signed halving subtract.

SLI: Shift left and insert (immediate).

SM3PARTW1: SM3PARTW1.

SM3PARTW2: SM3PARTW2.

SM3SS1: SM3SS1.

SM3TT1A: SM3TT1A.

SM3TT1B: SM3TT1B.

SM3TT2A: SM3TT2A.

SM3TT2B: SM3TT2B.

SM4E: SM4 encode.

SM4EKEY: SM4 key.

SMAX: Signed maximum (vector).

SMAXP: Signed maximum pairwise.

SMAXV: Signed maximum across vector.

SMIN: Signed minimum (vector).

SMINP: Signed minimum pairwise.

SMINV: Signed minimum across vector.

SMLAL, SMLAL2 (by element): Signed multiply-add long (vector, by element).

SMLAL, SMLAL2 (vector): Signed multiply-add long (vector).

SMLSL, SMLSL2 (by element): Signed multiply-subtract long (vector, by element).

SMLSL, SMLSL2 (vector): Signed multiply-subtract long (vector).

SMMLA (vector): Signed 8-bit integer matrix multiply-accumulate to 32-bit integer (vector).

SMOV: Signed move vector element to general-purpose register.

SMULL, SMULL2 (by element): Signed multiply long (vector, by element).

SMULL, SMULL2 (vector): Signed multiply long (vector).

SQABS: Signed saturating absolute value.

SQADD: Signed saturating add.

SQDMLAL, SQDMLAL2 (by element): Signed saturating doubling multiply-add long (by element).

SQDMLAL, SQDMLAL2 (vector): Signed saturating doubling multiply-add long.

SQDMLSL, SQDMLSL2 (by element): Signed saturating doubling multiply-subtract long (by element).

SQDMLSL, SQDMLSL2 (vector): Signed saturating doubling multiply-subtract long.

SQDMULH (by element): Signed saturating doubling multiply returning high half (by element).

SQDMULH (vector): Signed saturating doubling multiply returning high half.

SQDMULL, SQDMULL2 (by element): Signed saturating doubling multiply long (by element).

SQDMULL, SQDMULL2 (vector): Signed saturating doubling multiply long.

SQNEG: Signed saturating negate.

SQRDMLAH (by element): Signed saturating rounding doubling multiply accumulate returning high half (by element).

SQRDMLAH (vector): Signed saturating rounding doubling multiply accumulate returning high half (vector).

SQRDMLSH (by element): Signed saturating rounding doubling multiply subtract returning high half (by element).

SQRDMLSH (vector): Signed saturating rounding doubling multiply subtract returning high half (vector).

SQRDMULH (by element): Signed saturating rounding doubling multiply returning high half (by element).

SQRDMULH (vector): Signed saturating rounding doubling multiply returning high half.

SQRSHL: Signed saturating rounding shift left (register).

SQRSHRN, SQRSHRN2: Signed saturating rounded shift right narrow (immediate).

SQRSHRUN, SQRSHRUN2: Signed saturating rounded shift right unsigned narrow (immediate).

SQSHL (immediate): Signed saturating shift left (immediate).

SQSHL (register): Signed saturating shift left (register).

SQSHLU: Signed saturating shift left unsigned (immediate).

SQSHRN, SQSHRN2: Signed saturating shift right narrow (immediate).

SQSHRUN, SQSHRUN2: Signed saturating shift right unsigned narrow (immediate).

SQSUB: Signed saturating subtract.

SQXTN, SQXTN2: Signed saturating extract narrow.

SQXTUN, SQXTUN2: Signed saturating extract unsigned narrow.

SRHADD: Signed rounding halving add.

SRI: Shift right and insert (immediate).

SRSHL: Signed rounding shift left (register).

SRSHR: Signed rounding shift right (immediate).

SRSRA: Signed rounding shift right and accumulate (immediate).

SSHL: Signed shift left (register).

SSHLL, SSHLL2: Signed shift left long (immediate).

SSHR: Signed shift right (immediate).

SSRA: Signed shift right and accumulate (immediate).

SSUBL, SSUBL2: Signed subtract long.

SSUBW, SSUBW2: Signed subtract wide.

ST1 (multiple structures): Store multiple single-element structures from one, two, three, or four registers.

ST1 (single structure): Store a single-element structure from one lane of one register.

ST2 (multiple structures): Store multiple 2-element structures from two registers.

ST2 (single structure): Store single 2-element structure from one lane of two registers.

ST3 (multiple structures): Store multiple 3-element structures from three registers.

ST3 (single structure): Store single 3-element structure from one lane of three registers.

ST4 (multiple structures): Store multiple 4-element structures from four registers.

ST4 (single structure): Store single 4-element structure from one lane of four registers.

STBFADD, STBFADDL: Atomic BFloat16 floating-point add, without return.

STBFMAX, STBFMAXL: Atomic BFloat16 floating-point maximum, without return.

STBFMAXNM, STBFMAXNML: Atomic BFloat16 floating-point maximum number, without return.

STBFMIN, STBFMINL: Atomic BFloat16 floating-point minimum, without return.

STBFMINNM, STBFMINNML: Atomic BFloat16 floating-point minimum number, without return.

STFADD, STFADDL: Atomic floating-point add, without return.

STFMAX, STFMAXL: Atomic floating-point maximum, without return.

STFMAXNM, STFMAXNML: Atomic floating-point maximum number, without return.

STFMIN, STFMINL: Atomic floating-point minimum, without return.

STFMINNM, STFMINNML: Atomic floating-point minimum number, without return.

STL1 (SIMD&FP): Store-release a single-element structure from one lane of one register.

STLUR (SIMD&FP): Store-release SIMD&FP register (unscaled offset).

STNP (SIMD&FP): Store pair of SIMD&FP registers, with non-temporal hint.

STP (SIMD&FP): Store pair of SIMD&FP registers.

STR (immediate, SIMD&FP): Store SIMD&FP register (immediate offset).

STR (register, SIMD&FP): Store SIMD&FP register (register offset).

STTNP (SIMD&FP): Store unprivileged pair of SIMD&FP registers, with non-temporal hint.

STTP (SIMD&FP): Store unprivileged pair of SIMD&FP registers.

STUR (SIMD&FP): Store SIMD&FP register (unscaled offset).

SUB (vector): Subtract (vector).

SUBHN, SUBHN2: Subtract returning high narrow.

SUDOT (by element): Dot product with signed and unsigned integers (vector, by element).

SUQADD: Signed saturating accumulate of unsigned value.

SXTL, SXTL2: Signed extend long: an alias of SSHLL, SSHLL2.

TBL: Table vector lookup.

TBX: Table vector lookup extension.

TRN1: Transpose vectors (primary).

TRN2: Transpose vectors (secondary).

UABA: Unsigned absolute difference and accumulate.

UABAL, UABAL2: Unsigned absolute difference and accumulate long.

UABD: Unsigned absolute difference (vector).

UABDL, UABDL2: Unsigned absolute difference long.

UADALP: Unsigned add and accumulate long pairwise.

UADDL, UADDL2: Unsigned add long (vector).

UADDLP: Unsigned add long pairwise.

UADDLV: Unsigned sum long across vector.

UADDW, UADDW2: Unsigned add wide.

UCVTF (scalar SIMD&FP): Unsigned integer convert to floating-point (scalar SIMD&FP).

UCVTF (scalar, fixed-point): Unsigned fixed-point convert to floating-point (scalar).

UCVTF (scalar, integer): Unsigned integer convert to floating-point (scalar).

UCVTF (vector, fixed-point): Unsigned fixed-point convert to floating-point (vector).

UCVTF (vector, integer): Unsigned integer convert to floating-point (vector).

UDOT (by element): Dot product unsigned arithmetic (vector, by element).

UDOT (vector): Dot product unsigned arithmetic (vector).

UHADD: Unsigned halving add.

UHSUB: Unsigned halving subtract.

UMAX: Unsigned maximum (vector).

UMAXP: Unsigned maximum pairwise.

UMAXV: Unsigned maximum across vector.

UMIN: Unsigned minimum (vector).

UMINP: Unsigned minimum pairwise.

UMINV: Unsigned minimum across vector.

UMLAL, UMLAL2 (by element): Unsigned multiply-add long (vector, by element).

UMLAL, UMLAL2 (vector): Unsigned multiply-add long (vector).

UMLSL, UMLSL2 (by element): Unsigned multiply-subtract long (vector, by element).

UMLSL, UMLSL2 (vector): Unsigned multiply-subtract long (vector).

UMMLA (vector): Unsigned 8-bit integer matrix multiply-accumulate to 32-bit integer (vector).

UMOV: Unsigned move vector element to general-purpose register.

UMULL, UMULL2 (by element): Unsigned multiply long (vector, by element).

UMULL, UMULL2 (vector): Unsigned multiply long (vector).

UQADD: Unsigned saturating add.

UQRSHL: Unsigned saturating rounding shift left (register).

UQRSHRN, UQRSHRN2: Unsigned saturating rounded shift right narrow (immediate).

UQSHL (immediate): Unsigned saturating shift left (immediate).

UQSHL (register): Unsigned saturating shift left (register).

UQSHRN, UQSHRN2: Unsigned saturating shift right narrow (immediate).

UQSUB: Unsigned saturating subtract.

UQXTN, UQXTN2: Unsigned saturating extract narrow.

URECPE: Unsigned reciprocal estimate.

URHADD: Unsigned rounding halving add.

URSHL: Unsigned rounding shift left (register).

URSHR: Unsigned rounding shift right (immediate).

URSQRTE: Unsigned reciprocal square root estimate.

URSRA: Unsigned rounding shift right and accumulate (immediate).

USDOT (by element): Dot product with unsigned and signed integers (vector, by element).

USDOT (vector): Dot product with unsigned and signed integers (vector).

USHL: Unsigned shift left (register).

USHLL, USHLL2: Unsigned shift left long (immediate).

USHR: Unsigned shift right (immediate).

USMMLA (vector): Unsigned and signed 8-bit integer matrix multiply-accumulate to 32-bit integer (vector).

USQADD: Unsigned saturating accumulate of signed value.

USRA: Unsigned shift right and accumulate (immediate).

USUBL, USUBL2: Unsigned subtract long.

USUBW, USUBW2: Unsigned subtract wide.

UXTL, UXTL2: Unsigned extend long: an alias of USHLL, USHLL2.

UZP1: Unzip vectors (primary).

UZP2: Unzip vectors (secondary).

XAR: Exclusive-OR and rotate.

XTN, XTN2: Extract narrow.

ZIP1: Zip vectors (primary).

ZIP2: Zip vectors (secondary).

2026-09_rel2026-06_diff_tag2026-09-212026-09-08 1409:2244:3801

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

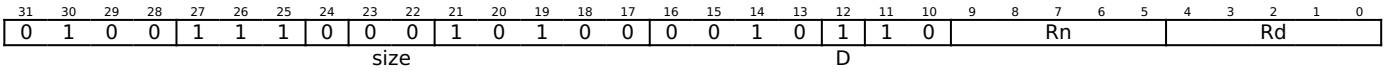
AESD

AES single round decryption

This~~AES~~ instruction reads a 16-byte state array from the first source vector, together with a round key from the second source vector. The state array undergoes a single round of the AddRoundKey(), InvShiftRows(), and InvSubBytes() transformations in accordance with the AES standard. The updated state array is destructively placed in the first source vector.~~decryption.~~

Advanced SIMD

(FEAT_AES)



Encoding

```
AESD <Vd>.16B, <Vn>.16B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AES) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP source and destination register, encoded in the "Rd" field.
<Vn>	Is the name of the second SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

let operand1 : bits(128) = V{}(d);
let operand2 : bits(128) = V{}(n);
var result : bits(128) = operand1 XOR operand2;
result = AESInvShiftRows(result);
V{128}(d) = AESInvSubBytes(result);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

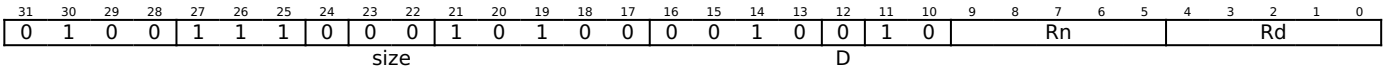
AESE

AES single round encryption

This~~AES~~ instruction reads a 16-byte state array from the first source vector, together with a round key from the second source vector. The state array undergoes a single round of the AddRoundKey(), ShiftRows(), and SubBytes() transformations in accordance with the AES standard. The updated state array is destructively placed in the first source vector.~~encryption.~~

Advanced SIMD

(FEAT_AES)



Encoding

```
AESE <Vd>.16B, <Vn>.16B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AES) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP source and destination register, encoded in the "Rd" field.
<Vn>	Is the name of the second SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

let operand1 : bits(128) = V{}(d);
let operand2 : bits(128) = V{}(n);
var result : bits(128) = operand1 XOR operand2;
result = AESShiftRows(result);
V{128}(d) = AESSubBytes(result);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

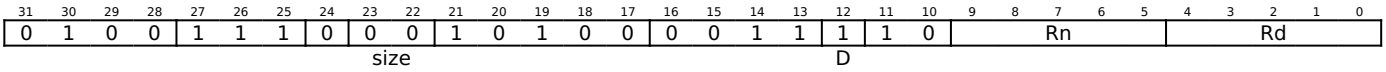
AESIMC

AES inverse mix columns

This instruction reads a 16-byte state array from the source vector and performs a single round of the InvMixColumns() transformation on the state array in accordance with the AES standard. The~~inverse~~ updated~~mix~~ state array is placed in the destination vector.~~columns.~~

Advanced SIMD

(FEAT_AES)



Encoding

```
AESIMC <Vd>.16B, <Vn>.16B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AES) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP destination register, encoded in the "Rd" field.
<Vn>	Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

let operand : bits(128) = V{}(n);
V{128}(d) = AESInvMixColumns(operand);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

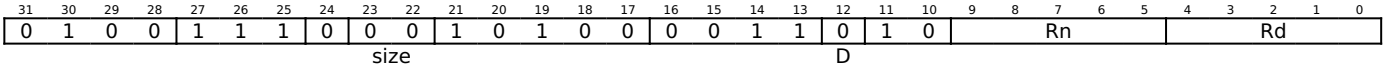
AESMC

AES mix columns

This instruction reads a 16-byte state array from the source vector and performs a single round of the MixColumns() transformation on the state array in accordance with the AES standard. The~~mix~~ updated state array is placed in the destination vector.~~columns.~~

Advanced SIMD

(FEAT_AES)



Encoding

```
AESMC <Vd>.16B, <Vn>.16B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AES) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP destination register, encoded in the "Rd" field.
<Vn>	Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

let operand : bits(128) = V{}(n);
V{128}(d) = AESMixColumns(operand);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

FAMAX

Floating-point absolute maximum

This instruction determines the maximum absolute value from floating-point elements of the first source vector and the corresponding floating-point elements of the second source vector, and places the results in the corresponding elements of the destination vector.

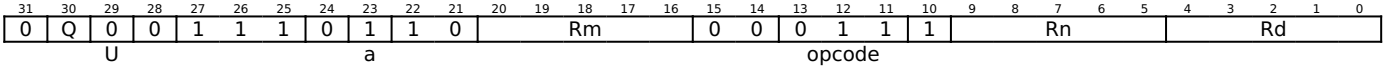
TheRegardless of the value of FPCR.AH, the behavior is as follows:

- AlternateWhen floating-pointFPCR.DN behaviorsis do0, notif apply,either aselement ifis FPCR.AHa NaN, the result is 0.a quiet NaN.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is the default NaN, with the sign bit set to 0.
- Denormalized inputs and results are never flushed to zero, as if FPCR.{FZ, FZ16, FIZ} are all 0.
- Denormalized inputs never generate an Input Denormal floating-point exception.

It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FAMINMAX)



Encoding

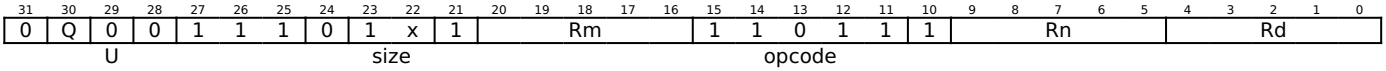
```
FAMAX <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FAMINMAX) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 16;
let datasize : integer{} = if Q == '1' then 128 else 64;
let elements : integer = datasize DIV esize;
```

Single-precision and double-precision

(FEAT_AdvSIMD && FEAT_FAMINMAX)



Encoding

```
FAMAX <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FAMINMAX) then
    EndOfDecode(Decode_UNDEF);
end;
if Q == '0' && size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = if Q == '1' then 128 else 64;
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Single-precision and double-precision" variant: is an arrangement specifier, encoded in “(size[0] :: Q)”:

size[0]	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);

for e = 0 to elements-1 do
    let op1 : bits(esize) = operand1[e*:esize];
    let op2 : bits(esize) = operand2[e*:esize];
    result[e*:esize] = FPAbsMax{esize}(op1, op2, FPCR());
end;
V{datasize}(d) = result;
```

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

FAMIN

Floating-point absolute minimum

This instruction determines the minimum absolute value from floating-point elements of the first source vector and the corresponding floating-point elements of the second source vector, and places the results in the corresponding elements of the destination vector.

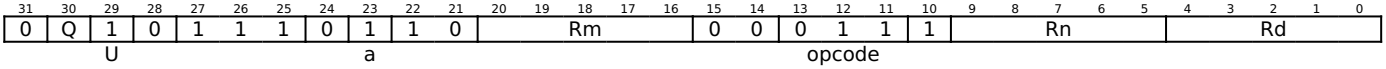
TheRegardless of the value of FPCR.AH, the behavior is as follows:

- AlternateWhen floating-pointFPCR.DN behaviorsis do0, notif apply,either aselement ifis FPCR.AHa NaN, the result is 0.a quiet NaN.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is the default NaN, with the sign bit set to 0.
- Denormalized inputs and results are never flushed to zero, as if FPCR.{FZ, FZ16, FIZ} are all 0.
- Denormalized inputs never generate an Input Denormal floating-point exception.

It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FAMINMAX)



Encoding

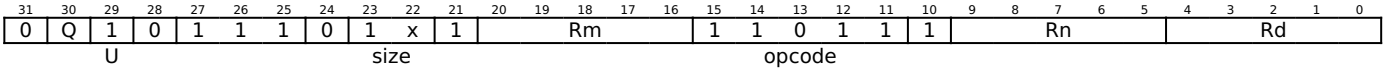
```
FAMIN <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FAMINMAX) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 16;
let datasize : integer{} = if Q == '1' then 128 else 64;
let elements : integer = datasize DIV esize;
```

Single-precision and double-precision

(FEAT_AdvSIMD && FEAT_FAMINMAX)



Encoding

```
FAMIN <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FAMINMAX) then
    EndOfDecode(Decode_UNDEF);
end;
if Q == '0' && size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = if Q == '1' then 128 else 64;
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Single-precision and double-precision" variant: is an arrangement specifier, encoded in “(size[0] :: Q)”:

size[0]	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);

for e = 0 to elements-1 do
    let op1 : bits(esize) = operand1[e*:esize];
    let op2 : bits(esize) = operand2[e*:esize];
    result[e*:esize] = FPAbsMin{esize}(op1, op2, FPCR());
end;
V{datasize}(d) = result;
```

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

FCCMP

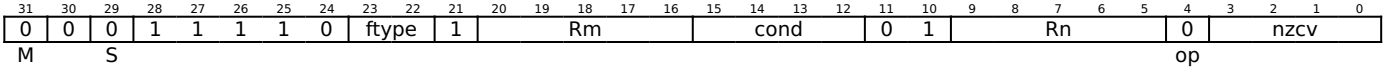
Floating-point conditional quiet compare (scalar)

This instruction, if the condition passes, has the following behavior:

- This instruction compares the two SIMD&FP source register values and writes the result to the *PSTATE*.{N, Z, C, V} flags.
- This instruction raises an Invalid Operation floating-point exception if either or both of the operands is a signaling NaN.
- This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

If the condition does not pass, then the comparison is not performed and the *PSTATE*.{N, Z, C, V} flags are set to the flag bit specifier.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision variant (FEAT_FP16)

Applies when `(ftype == 11)`

```
FCCMP <Hn>, <Hm>, #<nzcv>, <cond>
```

Encoding for the Single-precision variant (FEAT_FP)

Applies when `(ftype == 00)`

```
FCCMP <Sn>, <Sm>, #<nzcv>, <cond>
```

Encoding for the Double-precision variant (FEAT_FP)

Applies when `(ftype == 01)`

```
FCCMP <Dn>, <Dm>, #<nzcv>, <cond>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 8 << UInt(ftype XOR '10');
let condition : bits(4) = cond;
var flags : bits(4) = nzcv;
```

Assembler Symbols

<Hn>	Is the 16-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
<Hm>	Is the 16-bit name of the second SIMD&FP source register, encoded in the "Rm" field.
<nzcv>	Is the flag bit specifier, an immediate in the range 0 to 15, giving the alternative state for the 4-bit NZCV condition flags, encoded in the "nzcv" field.

<cond> Is one of the standard conditions, encoded in the standard way, and encoded in “cond”:

cond	<cond>
0000	EQ
0001	NE
0010	CS|HS
0011	CC|LO
0100	MI
0101	PL
0110	VS
0111	VC
1000	HI
1001	LS
1010	GE
1011	LT
1100	GT
1101	LE
1110	AL
1111	NV

<Sn> Is the 32-bit name of the first SIMD&FP source register, encoded in the "Rn" field.

<Sm> Is the 32-bit name of the second SIMD&FP source register, encoded in the "Rm" field.

<Dn> Is the 64-bit name of the first SIMD&FP source register, encoded in the "Rn" field.

<Dm> Is the 64-bit name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPEnabled();

let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);

if ConditionHolds(condition) then
    let signal_all_nans : boolean = FALSE;
    flags = FPCompare{datasize}(operand1, operand2, signal_all_nans, FPCR());
end;

PSTATE.[N,Z,C,V] = flags;
```

Operational information

The IEEE 754 standard specifies that the result of a comparison is precisely one of <, ==, >, or unordered. If either or both of the operands is a NaN, they are unordered, and all three of (Operand1 < Operand2), (Operand1 == Operand2), and (Operand1 > Operand2) are false. An unordered comparison sets the PSTATE condition flags to N=0, Z=0, C=1, and V=1.

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the NZCV condition flags written by this instruction might be significantly delayed.

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

FCCMPE

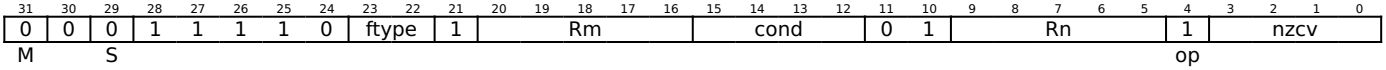
Floating-point conditional signaling compare (scalar)

This instruction, if the condition passes, has the following behavior:

- This instruction compares the two SIMD&FP source register values and writes the result to the *PSTATE*.{N, Z, C, V} flags.
- This instruction raises an Invalid Operation floating-point exception if either or both of the operands is any type of NaN.
- This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

If the condition does not pass, then the comparison is not performed and the *PSTATE*.{N, Z, C, V} flags are set to the flag bit specifier.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision variant (FEAT_FP16)

Applies when `(ftype == 11)`

```
FCCMPE <Hn>, <Hm>, #<nzcv>, <cond>
```

Encoding for the Single-precision variant (FEAT_FP)

Applies when `(ftype == 00)`

```
FCCMPE <Sn>, <Sm>, #<nzcv>, <cond>
```

Encoding for the Double-precision variant (FEAT_FP)

Applies when `(ftype == 01)`

```
FCCMPE <Dn>, <Dm>, #<nzcv>, <cond>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 8 << UInt(ftype XOR '10');
let condition : bits(4) = cond;
var flags : bits(4) = nzcv;
```

Assembler Symbols

<Hn>	Is the 16-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
<Hm>	Is the 16-bit name of the second SIMD&FP source register, encoded in the "Rm" field.
<nzcv>	Is the flag bit specifier, an immediate in the range 0 to 15, giving the alternative state for the 4-bit NZCV condition flags, encoded in the "nzcv" field.

<cond> Is one of the standard conditions, encoded in the standard way, and encoded in “cond”:

cond	<cond>
0000	EQ
0001	NE
0010	CS|HS
0011	CC|LO
0100	MI
0101	PL
0110	VS
0111	VC
1000	HI
1001	LS
1010	GE
1011	LT
1100	GT
1101	LE
1110	AL
1111	NV

<Sn> Is the 32-bit name of the first SIMD&FP source register, encoded in the "Rn" field.

<Sm> Is the 32-bit name of the second SIMD&FP source register, encoded in the "Rm" field.

<Dn> Is the 64-bit name of the first SIMD&FP source register, encoded in the "Rn" field.

<Dm> Is the 64-bit name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPEnabled();

let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);

if ConditionHolds(condition) then
    let signal_all_nans : boolean = TRUE;
    flags = FPCompare{datasize}(operand1, operand2, signal_all_nans, FPCR());
end;

PSTATE.[N,Z,C,V] = flags;
```

Operational information

The IEEE 754 standard specifies that the result of a comparison is precisely one of <, ==, >, or unordered. If either or both of the operands is a NaN, they are unordered, and all three of (Operand1 < Operand2), (Operand1 == Operand2), and (Operand1 > Operand2) are false. An unordered comparison sets the PSTATE condition flags to N=0, Z=0, C=1, and V=1.

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the NZCV condition flags written by this instruction might be significantly delayed.

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

FCMLA (by element)

Floating-point complex multiply accumulate (by element)

This instruction operates on complex numbers that are represented in SIMD&FP registers as pairs of elements, with the more significant element holding the imaginary part of the number and the less significant element holding the real part of the number. Each element holds a floating-point value. It performs the following calculation on complex numbers from the first source register and the destination register with the specified complex number from the second source register:

- Considering the complex number from the second source register on an Argand diagram, the number is rotated counterclockwise by 0, 90, 180, or 270 degrees.
- The two elements of the transformed complex number are multiplied by:
  - The real element of the complex number from the first source register, if the transformation was a rotation by 0 or 180 degrees.
  - The imaginary element of the complex number from the first source register, if the transformation was a rotation by 90 or 270 degrees.
- The complex number resulting from that multiplication is added to the complex number from the destination register.

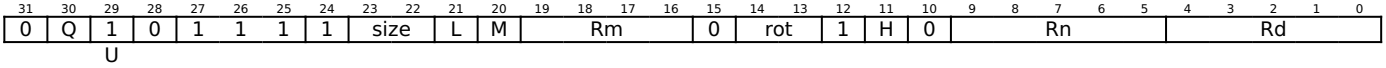
The multiplication and addition operations are performed as a fused multiply-add, without any intermediate rounding.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Vector

(FEAT_FCMA)



Encoding

```
FCMLA <Vd>.<T>, <Vn>.<T>, <Vm>.<Ts>[<index>], #<rotate>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_FCMA) then EndOfDecode(Decode_UNDEF); end;
if size == '00' || size == '11' then EndOfDecode(Decode_UNDEF); end;
if !IsFeatureImplemented(FEAT_FP16) && size == '01' thenif !IsFeatureImplemented(FEAT_FP16) && size == '10' then
EndOfDecode(Decode_UNDEF); end;
if size == '10' && (L == '1' || Q == '0') then EndOfDecode(Decode_UNDEF); end;
if size == '01' && H == '1' && Q == '0' then EndOfDecode(Decode_UNDEF); end;
let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
let m : integer = UInt(M::Rm);
var index : integer;
if size == '01' then index = UInt(H::L); end;
if size == '10' then index = UInt(H); end;
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	x	RESERVED
01	0	4H
01	1	8H
10	0	RESERVED
10	1	4S
11	x	RESERVED

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "M:Rm" fields.

<Ts> Is an element size specifier, encoded in “size”:

size	<Ts>
00	RESERVED
01	H
10	S
11	RESERVED

<index> Is the element index, encoded in “(size :: H :: L)”:

size	<index>
00	RESERVED
01	UInt(H :: L)
10	UInt(H)
11	RESERVED

<rotate> Is the rotation, encoded in “rot”:

rot	<rotate>
00	0
01	90
10	180
11	270

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
let operand3 : bits(datasize) = V{}(d);
var result : bits(datasize);

for e = 0 to (elements DIV 2)-1 do
    var element1 : bits(esize);
    var element2 : bits(esize);
    var element3 : bits(esize);
    var element4 : bits(esize);
    case rot of
        when '00' =>
            element1 = operand2[(index*2)*:esize];
            element2 = operand1[(e*2)*:esize];
            element3 = operand2[(index*2+1)*:esize];
            element4 = operand1[(e*2)*:esize];
        when '01' =>
            element1 = FPNeg{esize}(operand2[(index*2+1)*:esize], FPCR());
            element2 = operand1[(e*2+1)*:esize];
            element3 = operand2[(index*2)*:esize];
            element4 = operand1[(e*2+1)*:esize];
        when '10' =>
            element1 = FPNeg{esize}(operand2[(index*2)*:esize], FPCR());
            element2 = operand1[(e*2)*:esize];
            element3 = FPNeg{esize}(operand2[(index*2+1)*:esize], FPCR());
            element4 = operand1[(e*2)*:esize];
        when '11' =>
            element1 = operand2[(index*2+1)*:esize];
            element2 = operand1[(e*2+1)*:esize];
            element3 = FPNeg{esize}(operand2[(index*2)*:esize], FPCR());
            element4 = operand1[(e*2+1)*:esize];
    end;

    result[(e*2)*:esize] = FPMulAdd{esize}(operand3[(e*2)*:esize], element2, element1, FPCR());
    result[(e*2+1)*:esize] = FPMulAdd{esize}(operand3[(e*2+1)*:esize], element4, element3, FPCR());
end;

V{datasize}(d) = result;
```

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

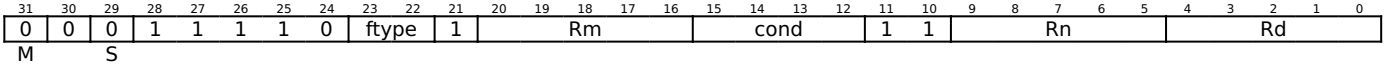
(old) htmldiff from- (new)

FCSEL

Floating-point conditional select (scalar)

This instruction allows the SIMD&FP destination register to take the value from either one or the other of two SIMD&FP source registers. If the condition passes, the first SIMD&FP source register value is taken, otherwise the second SIMD&FP source register value is taken.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision variant (FEAT_FP16)

Applies when `(ftype == 11)`

```
FCSEL <Hd>, <Hn>, <Hm>, <cond>
```

Encoding for the Single-precision variant (FEAT_FP)

Applies when `(ftype == 00)`

```
FCSEL <Sd>, <Sn>, <Sm>, <cond>
```

Encoding for the Double-precision variant (FEAT_FP)

Applies when `(ftype == 01)`

```
FCSEL <Dd>, <Dn>, <Dm>, <cond>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let datasize : integer{} = 8 << UInt(ftype XOR '10');
let condition : bits(4) = cond;
```

Assembler Symbols

<Hd> Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.

<Hn> Is the 16-bit name of the first SIMD&FP source register, encoded in the "Rn" field.

<Hm> Is the 16-bit name of the second SIMD&FP source register, encoded in the "Rm" field.

<cond> Is one of the standard conditions, encoded in the standard way, and encoded in “cond”:

cond	<cond>
0000	EQ
0001	NE
0010	CS|HS
0011	CC|LO
0100	MI
0101	PL
0110	VS
0111	VC
1000	HI
1001	LS
1010	GE
1011	LT
1100	GT
1101	LE
1110	AL
1111	NV

<Sd> Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.

<Sn>	Is the 32-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
<Sm>	Is the 32-bit name of the second SIMD&FP source register, encoded in the "Rm" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Dn>	Is the 64-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
<Dm>	Is the 64-bit name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPEnabled();
var result : bits(datasize);

let condition_holds : boolean = ConditionHolds(condition);
result = if condition_holds then V{datasize}(n) else V{datasize}(m);

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

FMAX (scalar)

Floating-point maximum (scalar)

This instruction compares the two source SIMD&FP registers, and writes the larger of the two floating-point values to the destination SIMD&FP register.

When FPCR.AH is 0, the behavior is as follows:

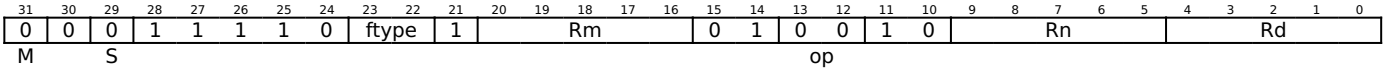
- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either value is a NaN, the result is a quiet NaN derived as follows:~~NaN.~~
  - If at least one value is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN value.
  - Otherwise, the result is a quiet NaN derived from the first NaN value.
- When FPCR.DN is 1, if either value is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows:

- If both values are zeros, regardless of the sign of either zero, the result is the second value.
- If either value is a NaN, regardless of the value of FPCR.DN, the result is the second value.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision variant (FEAT_FP16)

Applies when `(ftype == 11)`

```
FMAX <Hd>, <Hn>, <Hm>
```

Encoding for the Single-precision variant (FEAT_FP)

Applies when `(ftype == 00)`

```
FMAX <Sd>, <Sn>, <Sm>
```

Encoding for the Double-precision variant (FEAT_FP)

Applies when `(ftype == 01)`

```
FMAX <Dd>, <Dn>, <Dm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 8 << UInt(ftype XOR '10');
```

Assembler Symbols

<Hd>	Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
<Hm>	Is the 16-bit name of the second SIMD&FP source register, encoded in the "Rm" field.
<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
<Sm>	Is the 32-bit name of the second SIMD&FP source register, encoded in the "Rm" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Dn>	Is the 64-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
<Dm>	Is the 64-bit name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPEnabled();
let operand1 : bits(esize) = V{}(n);
let operand2 : bits(esize) = V{}(m);

var result : bits(128) = if IsMerging(FPCR()) then V{128}(n) else Zeros{128};

result[0+:esize] = FPMax{esize}(operand1, operand2, FPCR());

V{128}(d) = result;
```

2026-09_rel2026-06_diff_tag 2026-09-212026-09-08 1409:2244:3801

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

FMAX (vector)

Floating-point maximum (vector)

This instruction compares corresponding vector elements in the two source SIMD&FP registers, places the larger of each of the two floating-point values into a vector, and writes the vector to the destination SIMD&FP register.

When FPCR.AH is 0, the behavior is as follows:

- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:~~NaN.~~
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows:

- If both elements are zeros, regardless of the sign of either zero, the result is the second element.
- If either element is a NaN, regardless of the value of FPCR.DN, the result is the second element.

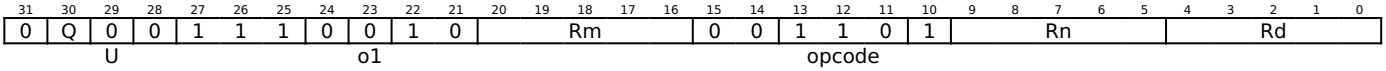
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

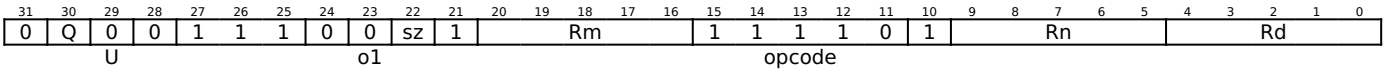
```
FMAX <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FMAX <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
var element1 : bits(esize);
var element2 : bits(esize);

for e = 0 to elements-1 do
    element1 = operand1[e*:esize];
    element2 = operand2[e*:esize];
    result[e*:esize] = FPMax{esize}(element1, element2, FPCR());
end;
V{datasize}(d) = result;
```

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

FMAXNM (scalar)

Floating-point maximum number (scalar)

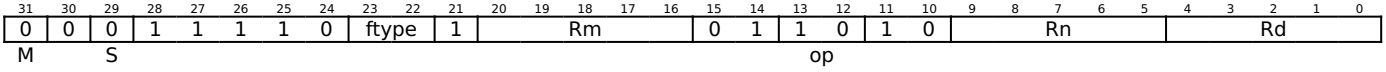
This instruction compares the first and second source SIMD&FP register values, and writes the larger of the two floating-point values to the destination SIMD&FP register.

TheRegardless of the value of FPCR.AH, the behavior is as follows:

- Negative zero compares less than positive zero.
- If one value is numeric and the other is a quiet NaN, the quiet NaN value is ignored and the result is the numeric value.
- When FPCR.DN is 0, if either value is a signaling NaN or if both values are NaNs, the result is a quiet NaN derived as follows:NaN.
  - If FPCR.AH is 0:
    - If at least one value is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN value.
    - Otherwise, the result is a quiet NaN derived from the first NaN value.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN value.
- When FPCR.DN is 1, if either value is a signaling NaN or if both values are NaNs, the result is default NaN.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision variant (FEAT_FP16)

Applies when `(ftype == 11)`

```
FMAXNM <Hd>, <Hn>, <Hm>
```

Encoding for the Single-precision variant (FEAT_FP)

Applies when `(ftype == 00)`

```
FMAXNM <Sd>, <Sn>, <Sm>
```

Encoding for the Double-precision variant (FEAT_FP)

Applies when `(ftype == 01)`

```
FMAXNM <Dd>, <Dn>, <Dm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 8 << UInt(ftype XOR '10');
```

Assembler Symbols

<Hd>	Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
<Hm>	Is the 16-bit name of the second SIMD&FP source register, encoded in the "Rm" field.
<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
<Sm>	Is the 32-bit name of the second SIMD&FP source register, encoded in the "Rm" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Dn>	Is the 64-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
<Dm>	Is the 64-bit name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPEnabled();
let operand1 : bits(esize) = V{}(n);
let operand2 : bits(esize) = V{}(m);

var result : bits(128) = if IsMerging(FPCR()) then V{128}(n) else Zeros{128};

result[0+:esize] = FPMaxNum{esize}(operand1, operand2, FPCR());

V{128}(d) = result;
```

2026-09_rel2026-06_diff_tag 2026-09-212026-09-08 1409:2244:3801

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

FMAXNM (vector)

Floating-point maximum number (vector)

This instruction compares corresponding vector elements in the two source SIMD&FP registers, writes the larger of the two floating-point values into a vector, and writes the vector to the destination SIMD&FP register.

TheRegardless of the value of FPCR.AH, the behavior is as follows:

- Negative zero compares less than positive zero.
- If one element is numeric and the other is a quiet NaN, the quiet NaN element is ignored and the result is the numeric element.value.
- When FPCR.DN is 0, if either element is a signaling NaN or if both elements are NaNs, the result is a quiet NaN derived as follows:NaN.
  - If FPCR.AH is 0:
    - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
    - Otherwise, the result is a quiet NaN derived from the first NaN element.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a signaling NaN or if both elements are NaNs, the result is default NaN.

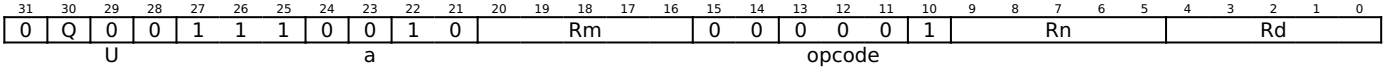
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

```
FMAXNM <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

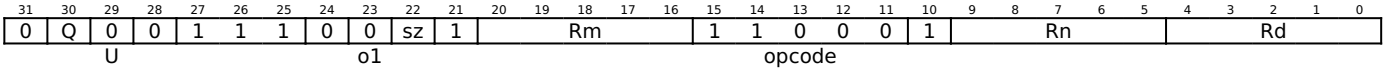
Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FMAXNM <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
var element1 : bits(esize);
var element2 : bits(esize);

for e = 0 to elements-1 do
    element1 = operand1[e*:esize];
    element2 = operand2[e*:esize];
    result[e*:esize] = FPMaxNum{esize}(element1, element2, FPCR());
end;
V{datasize}(d) = result;
```

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

FMAXNMP (scalar)

Floating-point maximum number of pair of elements (scalar)

This instruction compares two vector elements in the source SIMD&FP register and writes the largest of the floating-point values as a scalar to the destination SIMD&FP register.

TheRegardless of the value of FPCR.AH, the behavior is as follows for each pairwise operation:

- Negative zero compares less than positive zero.
- If one element is numeric and the other is a quiet NaN, the quiet NaN element is ignored and the result is the numeric element.value.
- When FPCR.DN is 0, if either element is a signaling NaN or if both elements are NaNs, the result is a quiet NaN derived as follows:NaN.
  - If FPCR.AH is 0:
    - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
    - Otherwise, the result is a quiet NaN derived from the first NaN element.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a signaling NaN or if both elements are NaNs, the result is default NaN.

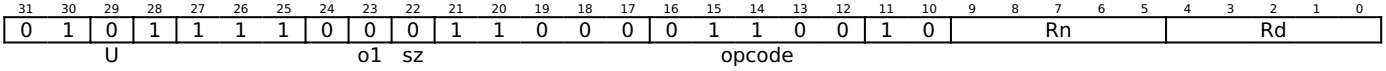
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

```
FMAXNMP H<d>, <Vn>.2H
```

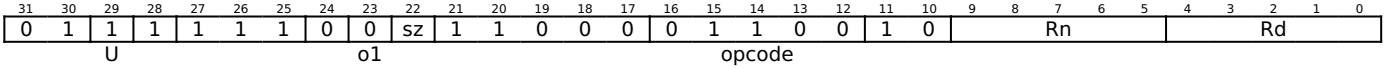
Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
if sz == '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 16;
let datasize : integer{} = 32;
```

Single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FMAXNMP <V><d>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = esize * 2;
```

Assembler Symbols

<d>	Is the number of the SIMD&FP destination register, encoded in the "Rd" field.
<Vn>	Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<V> Is the destination width specifier, encoded in “sz”:

sz	<V>
0	S
1	D

<T> Is the source arrangement specifier, encoded in “sz”:

sz	<T>
0	2S
1	2D

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
let element1 : bits(esize) = operand[0*:esize];
let element2 : bits(esize) = operand[1*:esize];
V{esize}(d) = FPMaxNum{esize}(element1, element2, FPCR());
```

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

FMAXNMP (vector)

Floating-point maximum number pairwise (vector)

This instruction creates a vector by concatenating the vector elements of the first source SIMD&FP register after the vector elements of the second source SIMD&FP register, reads each pair of adjacent vector elements in the two source SIMD&FP registers, writes the largest of each pair of values into a vector, and writes the vector to the destination SIMD&FP register. All the values in this instruction are floating-point values.

TheRegardless of the value of FPCR.AH, the behavior is as follows for each pairwise operation:

- Negative zero compares less than positive zero.
- If one element is numeric and the other is a quiet NaN, the quiet NaN element is ignored and the result is the numeric element.value.
- When FPCR.DN is 0, if either element is a signaling NaN or if both elements are NaNs, the result is a quiet NaN derived as follows:NaN.
  - If FPCR.AH is 0:
    - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
    - Otherwise, the result is a quiet NaN derived from the first NaN element.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a signaling NaN or if both elements are NaNs, the result is default NaN.

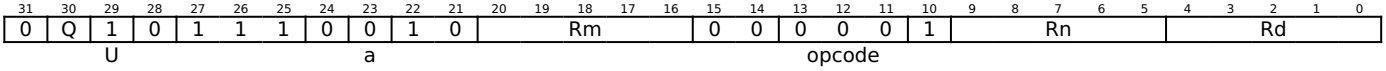
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

```
FMAXNMP <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

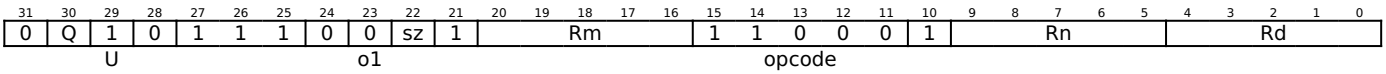
Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FMAXNMP <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
let concat : bits(2*datasize) = operand2::operand1;
var element1 : bits(esize);
var element2 : bits(esize);

for e = 0 to elements-1 do
    element1 = concat[(2*e)*:esize];
    element2 = concat[((2*e)+1)*:esize];
    result[e*:esize] = FPMaxNum{esize}(element1, element2, FPCR());
end;
V{datasize}(d) = result;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

FMAXNMV

Floating-point maximum number across vector

This instruction compares all the vector elements in the source SIMD&FP register, and writes the largest of the values as a scalar to the destination SIMD&FP register. All the values in this instruction are floating-point values.

TheRegardless of the value of FPCR.AH, the behavior is as follows:

- Negative zero compares less than positive zero.
- If one value is numeric and the other is a quiet NaN, the quiet NaN value is ignored and the result is the numeric value.
- When FPCR.DN is 0, if either value is a signaling NaN or if both values are NaNs, the result is a quiet NaN derived as follows:NaN.
  - If FPCR.AH is 0:
    - If at least one value is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN value.
    - Otherwise, the result is a quiet NaN derived from the first NaN value.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN value.
- When FPCR.DN is 1, if either value is a signaling NaN or if both values are NaNs, the result is default NaN.

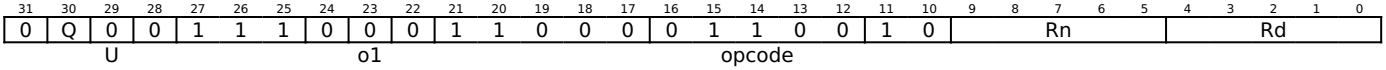
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

```
FMAXNMV <V><d>, <Vn>.<T>
```

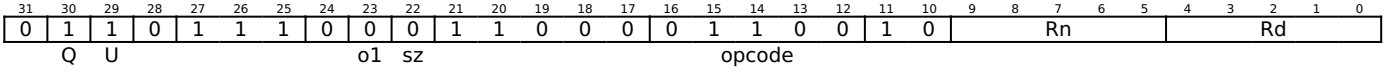
Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
```

Single-precision

(FEAT_AdvSIMD)



Encoding

```
FMAXNMV S<d>, <Vn>.4S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q != '01' then EndOfDecode(Decode_UNDEF); end;    // .4S only

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32;
let datasize : integer{} = 64 << UInt(Q);
```

Assembler Symbols

<V>	Is the destination width specifier, H.
<d>	Is the number of the SIMD&FP destination register, encoded in the "Rd" field.
<Vn>	Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<T> Is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
V{esize}(d) = FPReduce{esize, datasize}(ReduceOp_FMAXNUM, operand, FPCR());
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

FMAXP (scalar)

Floating-point maximum of pair of elements (scalar)

This instruction compares two vector elements in the source SIMD&FP register and writes the largest of the floating-point values as a scalar to the destination SIMD&FP register.

When FPCR.AH is 0, the behavior is as follows for each pairwise operation:

- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:~~NaN.~~
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows for each pairwise operation:

- If both elements are zeros, regardless of the sign of either zero, the result is the second element.
- If either element is a NaN, regardless of the value of FPCR.DN, the result is the second element.

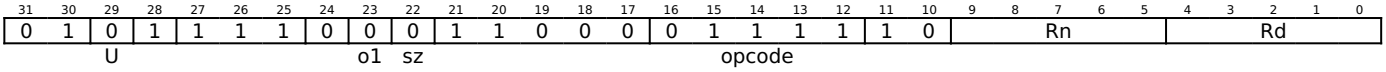
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

```
FMAXP H<d>, <Vn>.2H
```

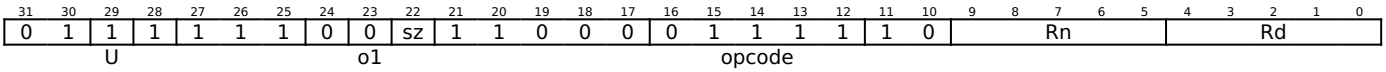
Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
if sz == '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 16;
let datasize : integer{} = 32;
```

Single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FMAXP <V><d>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = esize * 2;
```

Assembler Symbols

<d>	Is the number of the SIMD&FP destination register, encoded in the "Rd" field.
<Vn>	Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<V> Is the destination width specifier, encoded in “sz”:

sz	<V>
0	S
1	D

<T> Is the source arrangement specifier, encoded in “sz”:

sz	<T>
0	2S
1	2D

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand :  bits(datasize) = V{}(n);
let element1 : bits(esize) = operand[0*:esize];
let element2 : bits(esize) = operand[1*:esize];
V{esize}(d) = FPMax{esize}(element1, element2, FPCR());
```

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

FMAXP (vector)

Floating-point maximum pairwise (vector)

This instruction creates a vector by concatenating the vector elements of the first source SIMD&FP register after the vector elements of the second source SIMD&FP register, reads each pair of adjacent vector elements from the concatenated vector, writes the larger of each pair of values into a vector, and writes the vector to the destination SIMD&FP register. All the values in this instruction are floating-point values.

When FPCR.AH is 0, the behavior is as follows for each pairwise operation:

- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:~~NaN.~~
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows for each pairwise operation:

- If both elements are zeros, regardless of the sign of either zero, the result is the second element.
- If either element is a NaN, regardless of the value of FPCR.DN, the result is the second element.

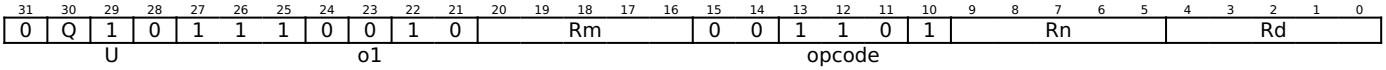
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

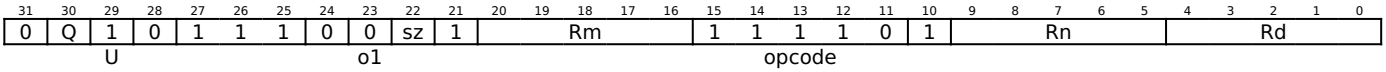
```
FMAXP <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FMAXP <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
let concat : bits(2*datasize) = operand2::operand1;
var element1 : bits(esize);
var element2 : bits(esize);

for e = 0 to elements-1 do
    element1 = concat[(2*e)*:esize];
    element2 = concat[((2*e)+1)*:esize];
    result[e*:esize] = FPMax{esize}(element1, element2, FPCR());
end;
V{datasize}(d) = result;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

FMAXV

Floating-point maximum across vector

This instruction compares all the vector elements in the source SIMD&FP register, and writes the largest of the values as a scalar to the destination SIMD&FP register. All the values in this instruction are floating-point values.

When FPCR.AH is 0, the behavior is as follows:

- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either value is a NaN, the result is a quiet NaN derived as follows:~~NaN.~~
  - If at least one value is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN value.
  - Otherwise, the result is a quiet NaN derived from the first NaN value.
- When FPCR.DN is 1, if either value is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows:

- If both values are zeros, regardless of the sign of either zero, the result is the second value.
- If either value is a NaN, regardless of the value of FPCR.DN, the result is the second value.

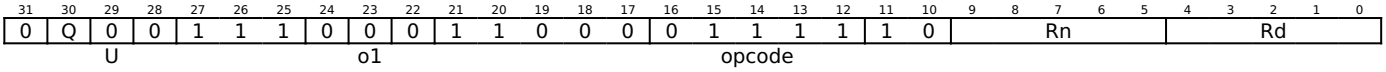
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

```
FMAXV <V><d>, <Vn>.<T>
```

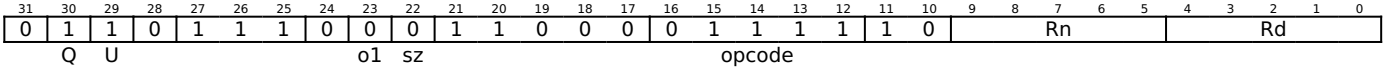
Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
```

Single-precision

(FEAT_AdvSIMD)



Encoding

```
FMAXV S<d>, <Vn>.4S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q != '01' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32;
let datasize : integer{} = 64 << UInt(Q);
```

Assembler Symbols

<V>	Is the destination width specifier, H.
<d>	Is the number of the SIMD&FP destination register, encoded in the "Rd" field.
<Vn>	Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<T> Is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

Operation

```
AArch64.CheckFPAdvSIMDEnabled();
let operand :  bits(datasize) = V{}(n);
V{esize}(d) = FPReduce{esize, datasize}(ReduceOp.FMAX, operand, FPCR());
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

FMIN (scalar)

Floating-point minimum (scalar)

This instruction compares the first and second source SIMD&FP register values, and writes the smaller of the two floating-point values to the destination SIMD&FP register.

When FPCR.AH is 0, the behavior is as follows:

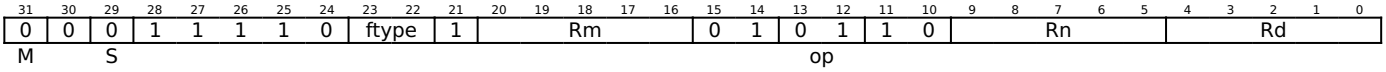
- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either value is a NaN, the result is a quiet NaN derived as follows:~~NaN.~~
  - If at least one value is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN value.
  - Otherwise, the result is a quiet NaN derived from the first NaN value.
- When FPCR.DN is 1, if either value is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows:

- If both values are zeros, regardless of the sign of either zero, the result is the second value.
- If either value is a NaN, regardless of the value of FPCR.DN, the result is the second value.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision variant (FEAT_FP16)

Applies when `(ftype == 11)`

```
FMIN <Hd>, <Hn>, <Hm>
```

Encoding for the Single-precision variant (FEAT_FP)

Applies when `(ftype == 00)`

```
FMIN <Sd>, <Sn>, <Sm>
```

Encoding for the Double-precision variant (FEAT_FP)

Applies when `(ftype == 01)`

```
FMIN <Dd>, <Dn>, <Dm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 8 << UInt(ftype XOR '10');
```

Assembler Symbols

<Hd>	Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
<Hm>	Is the 16-bit name of the second SIMD&FP source register, encoded in the "Rm" field.
<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
<Sm>	Is the 32-bit name of the second SIMD&FP source register, encoded in the "Rm" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Dn>	Is the 64-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
<Dm>	Is the 64-bit name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPEnabled();
let operand1 : bits(esize) = V{}(n);
let operand2 : bits(esize) = V{}(m);

var result : bits(128) = if IsMerging(FPCR()) then V{128}(n) else Zeros{128};

result[0+:esize] = FPMin{esize}(operand1, operand2, FPCR());

V{128}(d) = result;
```

2026-09_rel2026-06_diff_tag 2026-09-212026-09-08 1409:2244:3801

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

FMIN (vector)

Floating-point minimum (vector)

This instruction compares corresponding elements in the vectors in the two source SIMD&FP registers, places the smaller of each of the two floating-point values into a vector, and writes the vector to the destination SIMD&FP register.

When FPCR.AH is 0, the behavior is as follows:

- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:~~NaN.~~
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows:

- If both elements are zeros, regardless of the sign of either zero, the result is the second element.
- If either element is a NaN, regardless of the value of FPCR.DN, the result is the second element.

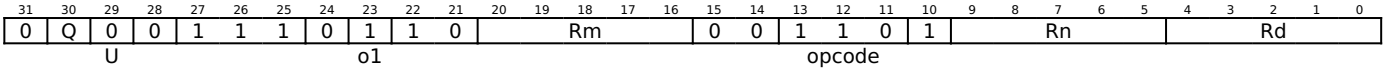
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

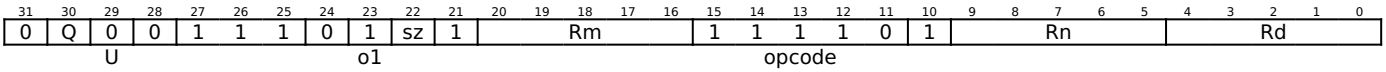
```
FMIN <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FMIN <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
var element1 : bits(esize);
var element2 : bits(esize);

for e = 0 to elements-1 do
    element1 = operand1[e*:esize];
    element2 = operand2[e*:esize];
    result[e*:esize] = FPMin{esize}(element1, element2, FPCR());
end;
V{datasize}(d) = result;
```

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

FMINNM (scalar)

Floating-point minimum number (scalar)

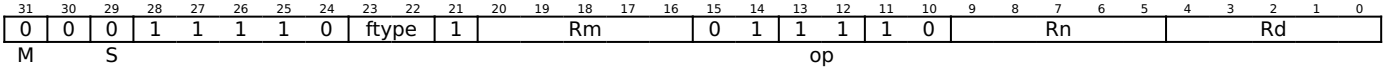
This instruction compares the first and second source SIMD&FP register values, and writes the smaller of the two floating-point values to the destination SIMD&FP register.

TheRegardless of the value of FPCR.AH, the behavior is as follows:

- Negative zero compares less than positive zero.
- If one value is numeric and the other is a quiet NaN, the quiet NaN value is ignored and the result is the numeric value.
- When FPCR.DN is 0, if either value is a signaling NaN or if both values are NaNs, the result is a quiet NaN derived as follows:NaN.
  - If FPCR.AH is 0:
    - If at least one value is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN value.
    - Otherwise, the result is a quiet NaN derived from the first NaN value.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN value.
- When FPCR.DN is 1, if either value is a signaling NaN or if both values are NaNs, the result is default NaN.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision variant (FEAT_FP16)

Applies when `(ftype == 11)`

```
FMINNM <Hd>, <Hn>, <Hm>
```

Encoding for the Single-precision variant (FEAT_FP)

Applies when `(ftype == 00)`

```
FMINNM <Sd>, <Sn>, <Sm>
```

Encoding for the Double-precision variant (FEAT_FP)

Applies when `(ftype == 01)`

```
FMINNM <Dd>, <Dn>, <Dm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 8 << UInt(ftype XOR '10');
```

Assembler Symbols

<Hd>	Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
<Hm>	Is the 16-bit name of the second SIMD&FP source register, encoded in the "Rm" field.
<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
<Sm>	Is the 32-bit name of the second SIMD&FP source register, encoded in the "Rm" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Dn>	Is the 64-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
<Dm>	Is the 64-bit name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPEnabled();
let operand1 : bits(esize) = V{}(n);
let operand2 : bits(esize) = V{}(m);

var result : bits(128) = if IsMerging(FPCR()) then V{128}(n) else Zeros{128};

result[0+:esize] = FPMinNum{esize}(operand1, operand2, FPCR());

V{128}(d) = result;
```

2026-09_rel2026-06_diff_tag 2026-09-212026-09-08 1409:2244:3801

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

FMINNM (vector)

Floating-point minimum number (vector)

This instruction compares corresponding vector elements in the two source SIMD&FP registers, writes the smaller of the two floating-point values into a vector, and writes the vector to the destination SIMD&FP register.

The~~Regardless of the value of FPCR.AH, the~~ behavior is as follows:

- Negative zero compares less than positive zero.
- If one element is numeric and the other is a quiet NaN, the quiet NaN element is ignored and the result is the numeric element.~~value.~~
- When FPCR.DN is 0, if either element is a signaling NaN or if both elements are NaNs, the result is a quiet NaN derived as follows:~~NaN.~~
  - If FPCR.AH is 0:
    - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
    - Otherwise, the result is a quiet NaN derived from the first NaN element.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a signaling NaN or if both elements are NaNs, the result is default NaN.

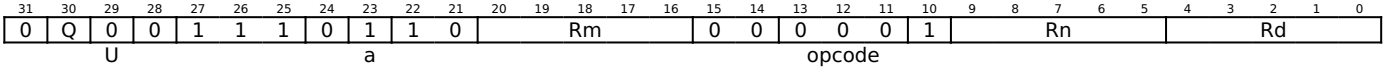
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

```
FMINNM <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

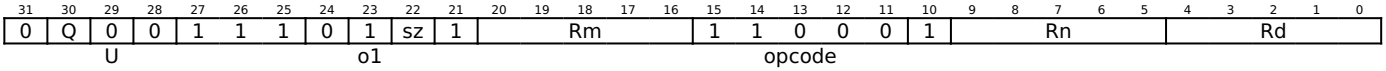
Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FMINNM <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
var element1 : bits(esize);
var element2 : bits(esize);

for e = 0 to elements-1 do
    element1 = operand1[e*:esize];
    element2 = operand2[e*:esize];
    result[e*:esize] = FPMinNum{esize}(element1, element2, FPCR());
end;
V{datasize}(d) = result;
```

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

FMINNMP (scalar)

Floating-point minimum number of pair of elements (scalar)

This instruction compares two vector elements in the source SIMD&FP register and writes the smallest of the floating-point values as a scalar to the destination SIMD&FP register.

TheRegardless of the value of FPCR.AH, the behavior is as follows for each pairwise operation:

- Negative zero compares less than positive zero.
- If one element is numeric and the other is a quiet NaN, the quiet NaN element is ignored and the result is the numeric element.value.
- When FPCR.DN is 0, if either element is a signaling NaN or if both elements are NaNs, the result is a quiet NaN derived as follows:NaN.
  - If FPCR.AH is 0:
    - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
    - Otherwise, the result is a quiet NaN derived from the first NaN element.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a signaling NaN or if both elements are NaNs, the result is default NaN.

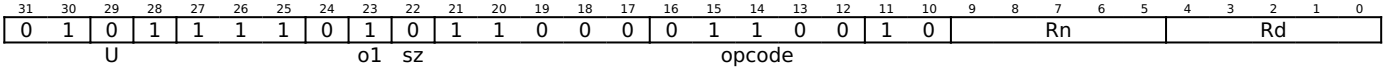
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

```
FMINNMP H<d>, <Vn>.2H
```

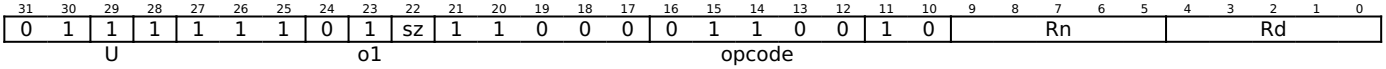
Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
if sz == '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 16;
let datasize : integer{} = 32;
```

Single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FMINNMP <V><d>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = esize * 2;
```

Assembler Symbols

<d>	Is the number of the SIMD&FP destination register, encoded in the "Rd" field.
<Vn>	Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<V> Is the destination width specifier, encoded in “sz”:

sz	<V>
0	S
1	D

<T> Is the source arrangement specifier, encoded in “sz”:

sz	<T>
0	2S
1	2D

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
let element1 : bits(esize) = operand[0*:esize];
let element2 : bits(esize) = operand[1*:esize];
V{esize}(d) = FPMinNum{esize}(element1, element2, FPCR());
```

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

FMINNMP (vector)

Floating-point minimum number pairwise (vector)

This instruction creates a vector by concatenating the vector elements of the first source SIMD&FP register after the vector elements of the second source SIMD&FP register, reads each pair of adjacent vector elements in the two source SIMD&FP registers, writes the smallest of each pair of floating-point values into a vector, and writes the vector to the destination SIMD&FP register. All the values in this instruction are floating-point values.

TheRegardless of the value of FPCR.AH, the behavior is as follows for each pairwise operation:

- Negative zero compares less than positive zero.
- If one element is numeric and the other is a quiet NaN, the quiet NaN element is ignored and the result is the numeric element.value.
- When FPCR.DN is 0, if either element is a signaling NaN or if both elements are NaNs, the result is a quiet NaN derived as follows:NaN.
  - If FPCR.AH is 0:
    - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
    - Otherwise, the result is a quiet NaN derived from the first NaN element.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a signaling NaN or if both elements are NaNs, the result is default NaN.

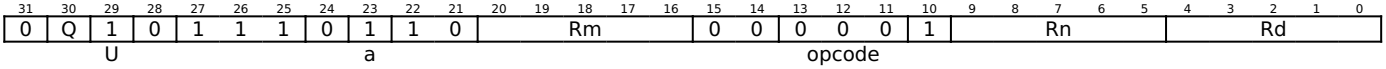
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

```
FMINNMP <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

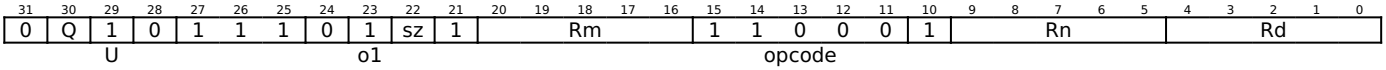
Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FMINNMP <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
let concat : bits(2*datasize) = operand2::operand1;
var element1 : bits(esize);
var element2 : bits(esize);

for e = 0 to elements-1 do
    element1 = concat[(2*e)*:esize];
    element2 = concat[((2*e)+1)*:esize];
    result[e*:esize] = FPMinNum{esize}(element1, element2, FPCR());
end;
V{datasize}(d) = result;
```

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

FMINNMV

Floating-point minimum number across vector

This instruction compares all the vector elements in the source SIMD&FP register, and writes the smallest of the values as a scalar to the destination SIMD&FP register. All the values in this instruction are floating-point values.

TheRegardless of the value of FPCR.AH, the behavior is as follows:

- Negative zero compares less than positive zero.
- If one value is numeric and the other is a quiet NaN, the quiet NaN value is ignored and the result is the numeric value.
- When FPCR.DN is 0, if either value is a signaling NaN or if both values are NaNs, the result is a quiet NaN derived as follows:NaN.
  - If FPCR.AH is 0:
    - If at least one value is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN value.
    - Otherwise, the result is a quiet NaN derived from the first NaN value.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN value.
- When FPCR.DN is 1, if either value is a signaling NaN or if both values are NaNs, the result is default NaN.

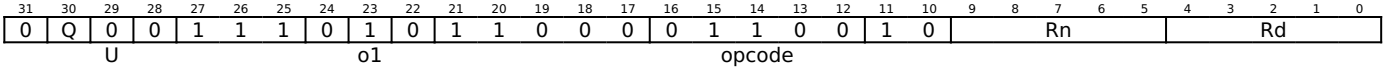
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

```
FMINNMV <V><d>, <Vn>.<T>
```

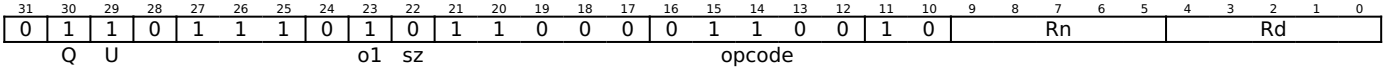
Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
```

Single-precision

(FEAT_AdvSIMD)



Encoding

```
FMINNMV S<d>, <Vn>.4S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q != '01' then EndOfDecode(Decode_UNDEF); end;    // .4S only

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32;
let datasize : integer{} = 64 << UInt(Q);
```

Assembler Symbols

<V>	Is the destination width specifier, H.
<d>	Is the number of the SIMD&FP destination register, encoded in the "Rd" field.
<Vn>	Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<T> Is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
V{esize}(d) = FPReduce{esize, datasize}(ReduceOp_FMINNUM, operand, FPCR());
```

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

FMINP (scalar)

Floating-point minimum of pair of elements (scalar)

This instruction compares two vector elements in the source SIMD&FP register and writes the smallest of the floating-point values as a scalar to the destination SIMD&FP register.

When FPCR.AH is 0, the behavior is as follows for each pairwise operation:

- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:~~NaN.~~
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows for each pairwise operation:

- If both elements are zeros, regardless of the sign of either zero, the result is the second element.
- If either element is a NaN, regardless of the value of FPCR.DN, the result is the second element.

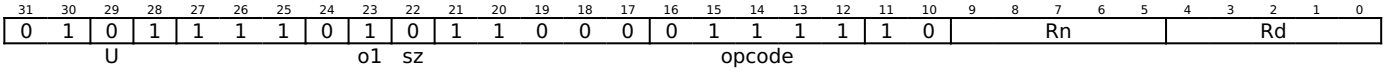
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

```
FMINP H<d>, <Vn>.2H
```

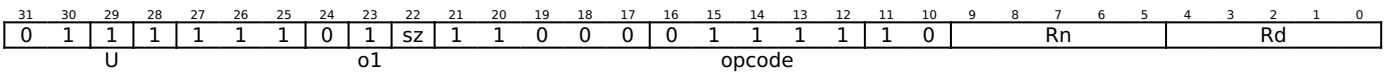
Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
if sz == '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 16;
let datasize : integer{} = 32;
```

Single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FMINP <V><d>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = esize * 2;
```

Assembler Symbols

<d>	Is the number of the SIMD&FP destination register, encoded in the "Rd" field.
<Vn>	Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<V> Is the destination width specifier, encoded in “sz”:

sz	<V>
0	S
1	D

<T> Is the source arrangement specifier, encoded in “sz”:

sz	<T>
0	2S
1	2D

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand :  bits(datasize) = V{}(n);
let element1 : bits(esize) = operand[0*:esize];
let element2 : bits(esize) = operand[1*:esize];
V{esize}(d) = FPMin{esize}(element1, element2, FPCR());
```

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

FMINP (vector)

Floating-point minimum pairwise (vector)

This instruction creates a vector by concatenating the vector elements of the first source SIMD&FP register after the vector elements of the second source SIMD&FP register, reads each pair of adjacent vector elements from the concatenated vector, writes the smaller of each pair of values into a vector, and writes the vector to the destination SIMD&FP register. All the values in this instruction are floating-point values.

When FPCR.AH is 0, the behavior is as follows for each pairwise operation:

- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:~~NaN.~~
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows for each pairwise operation:

- If both elements are zeros, regardless of the sign of either zero, the result is the second element.
- If either element is a NaN, regardless of the value of FPCR.DN, the result is the second element.

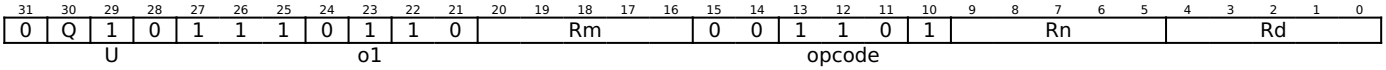
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

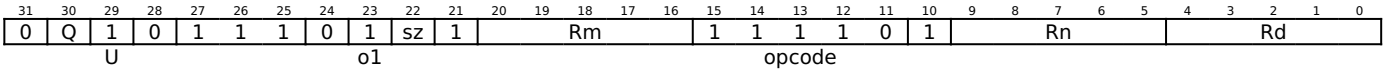
```
FMINP <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FMINP <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
let concat : bits(2*datasize) = operand2::operand1;
var element1 : bits(esize);
var element2 : bits(esize);

for e = 0 to elements-1 do
    element1 = concat[(2*e)*:esize];
    element2 = concat[((2*e)+1)*:esize];
    result[e*:esize] = FPMin{esize}(element1, element2, FPCR());
end;
V{datasize}(d) = result;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

FMINV

Floating-point minimum across vector

This instruction compares all the vector elements in the source SIMD&FP register, and writes the smallest of the values as a scalar to the destination SIMD&FP register. All the values in this instruction are floating-point values.

When FPCR.AH is 0, the behavior is as follows:

- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either value is a NaN, the result is a quiet NaN derived as follows:~~NaN.~~
  - If at least one value is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN value.
  - Otherwise, the result is a quiet NaN derived from the first NaN value.
- When FPCR.DN is 1, if either value is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows:

- If both values are zeros, regardless of the sign of either zero, the result is the second value.
- If either value is a NaN, regardless of the value of FPCR.DN, the result is the second value.

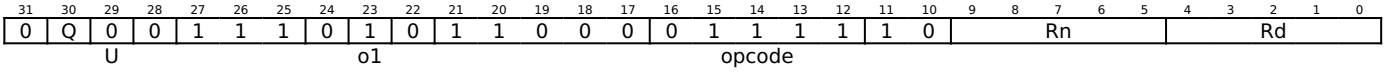
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

```
FMINV <V><d>, <Vn>.<T>
```

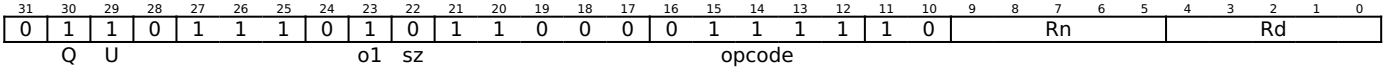
Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
```

Single-precision

(FEAT_AdvSIMD)



Encoding

```
FMINV S<d>, <Vn>.4S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q != '01' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32;
let datasize : integer{} = 64 << UInt(Q);
```

Assembler Symbols

<V>	Is the destination width specifier, H.
<d>	Is the number of the SIMD&FP destination register, encoded in the "Rd" field.
<Vn>	Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<T> Is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand :  bits(datasize) = V{}(n);
V{esize}(d) = FPReduce{esize, datasize}(ReduceOp_FMIN, operand, FPCR());
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

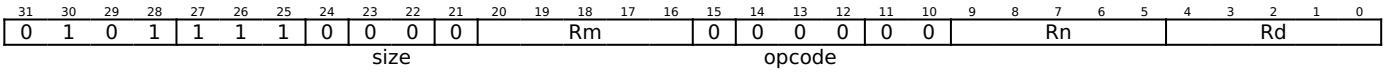
SHA1C

SHA1 hash update (choose)

This~~SHA1~~ instruction reads the five 32-bit words of the SHA-1 working state from the first and second source vectors, together with four 32-bit values from the third source vector, each containing the sum of a message schedule word and the corresponding round constant. The working state undergoes four rounds of the SHA-1 hash computation, applying~~update~~ the ~~(~~choose function and additions defined by the SHA-1 standard. The updated first four words of the working state are destructively placed in the first source vector.~~)~~.

Advanced SIMD

(FEAT_SHA1)



Encoding

```
SHA1C <Qd>, <Sn>, <Vm>.4S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SHA1) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
```

Assembler Symbols

<Qd>	Is the 128-bit name of the SIMD&FP source and destination, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the second SIMD&FP source register, encoded in the "Rn" field.
<Vm>	Is the name of the third SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

var x : bits(128) = V{}(d);
var y : bits(32) = V{}(n);    // Note: 32 not 128 bits wide
let w : bits(128) = V{}(m);

for e = 0 to 3 do
    let t : bits(32) = SHAchoose(x[63:32], x[95:64], x[127:96]);
    y = y + ROL(x[31:0], 5) + t + w[e*:32];
    x[63:32] = ROL(x[63:32], 30);
    let yx : bits(160) = ROL(y::x, 32);
    (y, x) = (yx[159:128], yx[127:0]);
end;
V{128}(d) = x;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

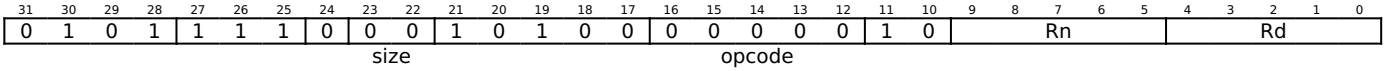
SHA1H

SHA1 fixed rotate

This~~SHA1~~ instruction reads a 32-bit value from the source vector. The value undergoes the fixed 30-bit left rotate operation defined by the SHA-1 standard. The resulting 32-bit value is placed in the destination vector.~~rotate.~~

Advanced SIMD

(FEAT_SHA1)



Encoding

```
SHA1H <Sd>, <Sn>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SHA1) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
```

Assembler Symbols

<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

let operand : bits(32) = V{}(n);        // read element [0] only,  [1-3] zeroed
V{32}(d) = ROL(operand, 30);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

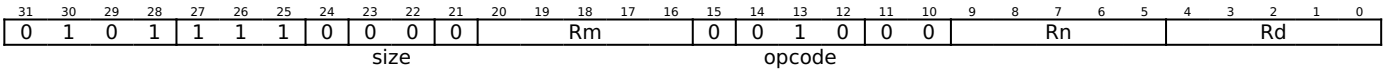
SHA1M

SHA1 hash update (majority)

This~~SHA1~~ instruction reads the five 32-bit words of the SHA-1 working state from the first and second source vectors, together with four 32-bit values from the third source vector, each containing the sum of a message schedule word and the corresponding round constant. The working state undergoes four rounds of the SHA-1 hash computation,~~update~~ applying the ~~(~~majority function and additions defined by the SHA-1 standard. The updated first four words of the working state are destructively placed in the first source vector.~~)~~.

Advanced SIMD

(FEAT_SHA1)



Encoding

```
SHA1M <Qd>, <Sn>, <Vm>.4S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SHA1) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
```

Assembler Symbols

<Qd>	Is the 128-bit name of the SIMD&FP source and destination, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the second SIMD&FP source register, encoded in the "Rn" field.
<Vm>	Is the name of the third SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

var x : bits(128) = V{}(d);
var y : bits(32) = V{}(n);    // Note: 32 not 128 bits wide
let w : bits(128) = V{}(m);

for e = 0 to 3 do
    let t : bits(32) = SHAmajority{32}(x[63:32], x[95:64], x[127:96]);
    y = y + ROL(x[31:0], 5) + t + w[e*:32];
    x[63:32] = ROL(x[63:32], 30);
    let yx : bits(160) = ROL(y::x, 32);
    (y, x) = (yx[159:128], yx[127:0]);
end;
V{128}(d) = x;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

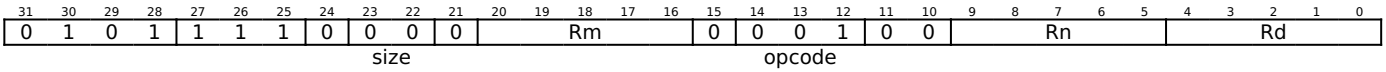
SHA1P

SHA1 hash update (parity)

This~~SHA1~~ instruction reads the five 32-bit words of the SHA-1 working state from the first and second source vectors, together with four 32-bit values from the third source vector, each containing the sum of a message schedule word and the corresponding round constant. The working state undergoes four rounds of the SHA-1 hash computation,~~update~~ applying the ~~(~~parity function and additions defined by the SHA-1 standard. The updated first four words of the working state are destructively placed in the first source vector.~~)~~.

Advanced SIMD

(FEAT_SHA1)



Encoding

```
SHA1P <Qd>, <Sn>, <Vm>.4S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SHA1) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
```

Assembler Symbols

<Qd>	Is the 128-bit name of the SIMD&FP source and destination, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the second SIMD&FP source register, encoded in the "Rn" field.
<Vm>	Is the name of the third SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

var x : bits(128) = V{}(d);
var y : bits(32) = V{}(n);    // Note: 32 not 128 bits wide
let w : bits(128) = V{}(m);

for e = 0 to 3 do
    let t : bits(32) = SHAparity(x[63:32], x[95:64], x[127:96]);
    y = y + ROL(x[31:0], 5) + t + w[e*:32];
    x[63:32] = ROL(x[63:32], 30);
    let yx : bits(160) = ROL(y::x, 32);
    (y, x) = (yx[159:128], yx[127:0]);
end;
V{128}(d) = x;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

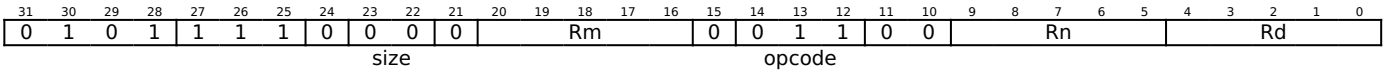
SHA1SU0

SHA1 schedule update 0

This~~SHA1~~ instruction reads previously computed SHA-1 message schedule words~~update~~ from the three source vectors. The values undergo the first part of four successive SHA-1 message schedule updates, applying the exclusive-OR operations defined by the SHA-1 standard. The resulting four 32-bit intermediate values are destructively placed in the first source vector.~~0.~~

Advanced SIMD

(FEAT_SHA1)



Encoding

```
SHA1SU0 <Vd>.4S, <Vn>.4S, <Vm>.4S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SHA1) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP source and destination register, encoded in the "Rd" field.
<Vn>	Is the name of the second SIMD&FP source register, encoded in the "Rn" field.
<Vm>	Is the name of the third SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

let operand1 : bits(128) = V{}(d);
let operand2 : bits(128) = V{}(n);
let operand3 : bits(128) = V{}(m);
var result : bits(128) = operand2[63:0] :: operand1[127:64];
result = result XOR operand1 XOR operand3;
V{128}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

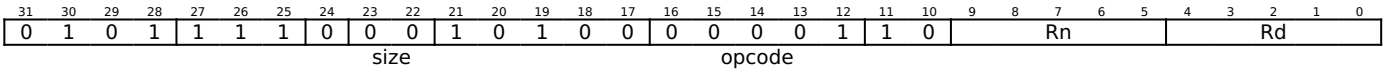
SHA1SU1

SHA1 schedule update 1

This~~SHA1~~ instruction reads four intermediate values for SHA-1 message schedule updates from~~update~~ the first source vector, together with previously computed SHA-1 message schedule words from the second source vector. The values undergo the second part of four successive SHA-1 message schedule updates, applying the exclusive-OR and rotate operations defined by the SHA-1 standard. The resulting four 32-bit message schedule words are destructively placed in the first source vector.~~1~~.

Advanced SIMD

(FEAT_SHA1)



Encoding

```
SHA1SU1 <Vd>.4S, <Vn>.4S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SHA1) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP source and destination register, encoded in the "Rd" field.
<Vn>	Is the name of the second SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

let operand1 : bits(128) = V{}(d);
let operand2 : bits(128) = V{}(n);
let T : bits(128) = operand1 XOR LSR(operand2, 32);

var result : bits(128);
result[31:0]   = ROL(T[31:0],   1);
result[63:32]  = ROL(T[63:32],  1);
result[95:64]  = ROL(T[95:64],  1);
result[127:96] = ROL(T[127:96], 1) XOR ROL(T[31:0], 2);
V{128}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

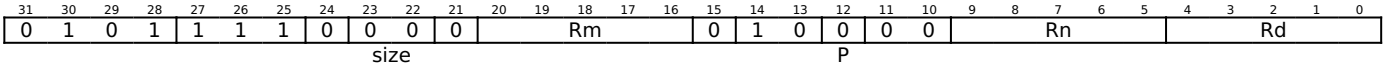
SHA256H

SHA-256 hash update (part 1)

This instruction reads the eight 32-bit words of the SHA-256 working state from the first and second source vectors, together with four 32-bit values from the third source vector, each containing the sum of a message schedule word and the corresponding round constant. The working state undergoes four rounds of the SHA-256 hash computation~~update~~ in~~(part~~ accordance with the SHA-256 standard. The updated first half of the working state is destructively placed in the first source vector.~~1).~~

Advanced SIMD

(FEAT_SHA256)



Encoding

```
SHA256H <Qd>, <Qn>, <Vm>.4S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SHA256) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
```

Assembler Symbols

<Qd>	Is the 128-bit name of the SIMD&FP source and destination, encoded in the "Rd" field.
<Qn>	Is the 128-bit name of the second SIMD&FP source register, encoded in the "Rn" field.
<Vm>	Is the name of the third SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

let part1 : boolean = TRUE;
V{128}(d) = SHA256hash(V{128}(d), V{128}(n), V{128}(m), part1);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

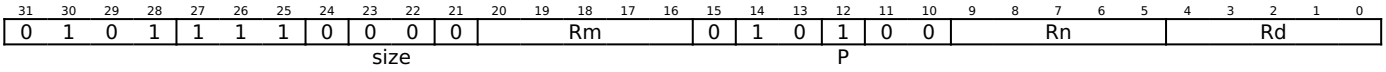
SHA256H2

SHA-256 hash update (part 2)

This instruction reads the eight 32-bit words of the SHA-256 working state from the first and second source vectors, together with four 32-bit values from the third source vector, each containing the sum of a message schedule word and the corresponding round constant. The two halves of the working state are supplied in the opposite order to SHA256H. The working state undergoes four rounds of the SHA-256 hash computation~~update~~ in~~(part~~ accordance with the SHA-256 standard. The updated second half of the working state is destructively placed in the first source vector.~~2).~~

Advanced SIMD

(FEAT_SHA256)



Encoding

```
SHA256H2 <Qd>, <Qn>, <Vm>.4S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SHA256) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
```

Assembler Symbols

<Qd>	Is the 128-bit name of the SIMD&FP source and destination, encoded in the "Rd" field.
<Qn>	Is the 128-bit name of the second SIMD&FP source register, encoded in the "Rn" field.
<Vm>	Is the name of the third SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

let part1 : boolean = FALSE;
V{128}(d) = SHA256hash(V{128}(n), V{128}(d), V{128}(m), part1);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

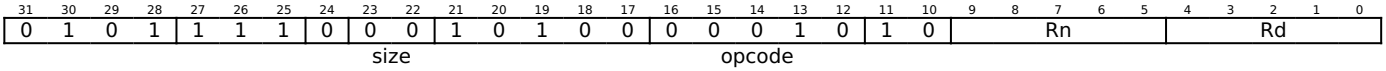
SHA256SU0

SHA-256 schedule update 0

This instruction reads previously computed SHA-256 message schedule words~~update~~ from the first and second source vectors. The values undergo the first part of four successive SHA-256 message schedule updates, applying the sigma0 function and additions defined by the SHA-256 standard. The resulting four 32-bit intermediate values are destructively placed in the first source vector.~~0.~~

Advanced SIMD

(FEAT_SHA256)



Encoding

```
SHA256SU0 <Vd>.4S, <Vn>.4S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SHA256) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP source and destination register, encoded in the "Rd" field.
<Vn>	Is the name of the second SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

let operand1 : bits(128) = V{}(d);
let operand2 : bits(128) = V{}(n);
let T : bits(128) = operand2[31:0] :: operand1[127:32];
var result : bits(128);
var elt : bits(32);

for e = 0 to 3 do
    elt = T[e*:32];
    elt = ROR(elt, 7) XOR ROR(elt, 18) XOR LSR(elt, 3);
    result[e*:32] = elt + operand1[e*:32];
end;
V{128}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

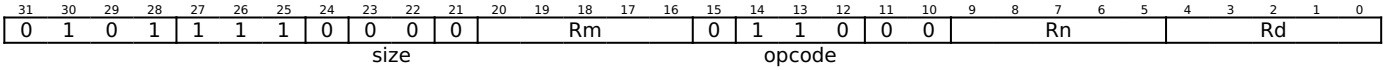
SHA256SU1

SHA-256 schedule update 1

This instruction reads four intermediate values for SHA-256 message schedule updates~~update~~ from the first source vector, together with previously computed SHA-256 message schedule words from the second and third source vectors. The values undergo the second part of four successive SHA-256 message schedule updates, applying the sigma1 function and additions defined by the SHA-256 standard. The resulting four 32-bit message schedule words are destructively placed in the first source vector.~~1.~~

Advanced SIMD

(FEAT_SHA256)



Encoding

```
SHA256SU1 <Vd>.4S, <Vn>.4S, <Vm>.4S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SHA256) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP source and destination register, encoded in the "Rd" field.
<Vn>	Is the name of the second SIMD&FP source register, encoded in the "Rn" field.
<Vm>	Is the name of the third SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

let operand1 : bits(128) = V{}(d);
let operand2 : bits(128) = V{}(n);
let operand3 : bits(128) = V{}(m);
let T0 : bits(128) = operand3[31:0] :: operand2[127:32];

var T1 : bits(64);
var elt : bits(32);
var result : bits(128);

T1 = operand3[127:64];
for e = 0 to 1 do
    elt = T1[e*:32];
    elt = ROR(elt, 17) XOR ROR(elt, 19) XOR LSR(elt, 10);
    elt = elt + operand1[e*:32] + T0[e*:32];
    result[e*:32] = elt;
end;

T1 = result[63:0];
for e = 2 to 3 do
    elt = T1[(e - 2)*:32];
    elt = ROR(elt, 17) XOR ROR(elt, 19) XOR LSR(elt, 10);
    elt = elt + operand1[e*:32] + T0[e*:32];
    result[e*:32] = elt;
end;

V{128}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

A64 -- SVE Instructions (alphabetic order)

ABS: Absolute value (predicated).

ADCLB: Add with carry long (bottom).

ADCLT: Add with carry long (top).

ADD (immediate): Add immediate (unpredicated).

ADD (vectors, predicated): Add (predicated).

ADD (vectors, unpredicated): Add (unpredicated).

ADDHNB: Add narrow high part (bottom).

ADDHNT: Add narrow high part (top).

ADDP: Add pairwise.

ADDPL: Add multiple of predicate register size to scalar register.

ADDPT (predicated): Add checked pointer vectors (predicated).

ADDPT (unpredicated): Add checked pointer vectors (unpredicated).

ADDQP: Add pairwise within quadword vector segments (unpredicated).

ADDQV: Add reduction of quadword vector segments.

ADDSUBP: Add subtract pairwise.

ADDVL: Add multiple of vector register size to scalar register.

ADR: Calculate vector address.

AESD (indexed): Multi-vector AES single round decryption.

AESD (vectors): AES single round decryption.

AESDIMC: Multi-vector AES single round decryption and inverse mix columns.

AESE (indexed): Multi-vector AES single round encryption.

AESE (vectors): AES single round encryption.

AESEMC: Multi-vector AES single round encryption and mix columns.

AESIMC: AES inverse mix columns.

AESMC: AES mix columns.

AND (immediate): Bitwise AND with immediate (unpredicated).

AND (predicates): Bitwise AND predicates.

AND (vectors, predicated): Bitwise AND (predicated).

AND (vectors, unpredicated): Bitwise AND (unpredicated).

ANDQV: Bitwise AND reduction of quadword vector segments.

ANDS: Bitwise AND predicates, setting the condition flags.

ANDV: Bitwise AND reduction to scalar.

ASR (immediate, predicated): Arithmetic shift right by immediate (predicated).

ASR (immediate, unpredicated): Arithmetic shift right by immediate (unpredicated).

ASR (vectors): Arithmetic shift right by vector (predicated).

ASR (wide elements, predicated): Arithmetic shift right by 64-bit wide elements (predicated).

ASR (wide elements, unpredicated): Arithmetic shift right by 64-bit wide elements (unpredicated).

ASRD: Arithmetic shift right for divide by immediate (predicated).

ASRR: Reversed arithmetic shift right by vector (predicated).

BCAX: Bitwise clear and exclusive-OR.

BDEP: Scatter lower bits into positions selected by bitmask.

BEXT: Gather lower bits from positions selected by bitmask.

BF1CVT, BF2CVT: 8-bit floating-point convert to BFloat16.

BF1CVTLT, BF2CVTLT: 8-bit floating-point convert to BFloat16 (top).

BFADD (predicated): BFloat16 add (predicated).

BFADD (unpredicated): BFloat16 add (unpredicated).

BFCLAMP: BFloat16 clamp to minimum/maximum number.

BFCVT: Single-precision convert to BFloat16 (predicated).

BFCVTN: BFloat16 convert to interleaved 8-bit floating-point.

BFCVTNT: Single-precision convert to BFloat16 (top, predicated).

BFDOT (indexed): BFloat16 dot product by indexed element to single-precision.

BFDOT (vectors): BFloat16 dot product to single-precision.

BFMAX: BFloat16 maximum (predicated).

BFMAXNM: BFloat16 maximum number (predicated).

BFMIN: BFloat16 minimum (predicated).

BFMINNM: BFloat16 minimum number (predicated).

BFMLA (indexed): BFloat16 fused multiply-add by indexed element.

BFMLA (vectors): BFloat16 fused multiply-add.

BFMLALB (indexed): BFloat16 multiply-add by indexed element to single-precision (bottom).

BFMLALB (vectors): BFloat16 multiply-add to single-precision (bottom).

BFMLALT (indexed): BFloat16 multiply-add by indexed element to single-precision (top).

BFMLALT (vectors): BFloat16 multiply-add to single-precision (top).

BFMLS (indexed): BFloat16 fused multiply-subtract by indexed element.

BFMLS (vectors): BFloat16 fused multiply-subtract.

BFMLSLB (indexed): BFloat16 multiply-subtract by indexed element from single-precision (bottom).

BFMLSLB (vectors): BFloat16 multiply-subtract from single-precision (bottom).

BFMLSLT (indexed): BFloat16 multiply-subtract by indexed element from single-precision (top).

BFMLSLT (vectors): BFloat16 multiply-subtract from single-precision (top).

BFMMLA (non-widening): BFloat16 matrix multiply-accumulate.

BFMMLA (widening): BFloat16 matrix multiply-accumulate to single-precision.

BFMUL (indexed): BFloat16 multiply by indexed element.

BFMUL (vectors, predicated): BFloat16 multiply (predicated).

BFMUL (vectors, unpredicated): BFloat16 multiply (unpredicated).

BFSCALE: BFloat16 adjust exponent (predicated).

BFSUB (predicated): BFloat16 subtract (predicated).

BFSUB (unpredicated): BFloat16 subtract (unpredicated).

BGRP: Group bits to right or left as selected by bitmask.

BIC (immediate): Bitwise clear bits using immediate (unpredicated): an alias of AND (immediate).

BIC (predicates): Bitwise clear predicates.

BIC (vectors, predicated): Bitwise clear (predicated).

BIC (vectors, unpredicated): Bitwise clear (unpredicated).

BICS: Bitwise clear predicates, setting the condition flags.

BRKA: Break after first true condition.

BRKAS: Break after first true condition, setting the condition flags.

BRKB: Break before first true condition.

BRKBS: Break before first true condition, setting the condition flags.

BRKN: Propagate break to next partition.

BRKNS: Propagate break to next partition, setting the condition flags.

BRKPA: Break after first true condition, propagating from previous partition.

BRKPAS: Break after first true condition, propagating from previous partition and setting the condition flags.

BRKPB: Break before first true condition, propagating from previous partition.

BRKPBS: Break before first true condition, propagating from previous partition and setting the condition flags.

BSL: Bitwise select.

BSL1N: Bitwise select with first input inverted.

BSL2N: Bitwise select with second input inverted.

CADD: Complex integer add.

CDOT (indexed): Complex integer dot product by indexed element.

CDOT (vectors): Complex integer dot product.

CLASTA (scalar): Conditionally extract element after last to general-purpose register.

CLASTA (SIMD&FP scalar): Conditionally extract element after last to SIMD&FP scalar register.

CLASTA (vectors): Conditionally extract element after last to vector register.

CLASTB (scalar): Conditionally extract last element to general-purpose register.

CLASTB (SIMD&FP scalar): Conditionally extract last element to SIMD&FP scalar register.

CLASTB (vectors): Conditionally extract last element to vector register.

CLS: Count leading sign bits (predicated).

CLZ: Count leading zero bits (predicated).

CMLA (indexed): Complex integer multiply-add by indexed element.

CMLA (vectors): Complex integer multiply-add.

CMP<cc> (immediate): Compare vector to immediate.

CMP<cc> (vectors): Compare vectors.

CMP<cc> (wide elements): Compare vector to 64-bit wide elements.

CMPLE (vectors): Compare signed less than or equal to vector, setting the condition flags: an alias of CMP<cc> (vectors).

CMPLO (vectors): Compare unsigned lower than vector, setting the condition flags: an alias of CMP<cc> (vectors).

CMPLS (vectors): Compare unsigned lower than or same as vector, setting the condition flags: an alias of CMP<cc> (vectors).

CMPLT (vectors): Compare signed less than vector, setting the condition flags: an alias of CMP<cc> (vectors).

CNOT: Logically invert boolean condition (predicated).

CNT: Count nonzero bits (predicated).

CNTB, CNTD, CNTH, CNTW: Set scalar to multiple of predicate constraint element count.

CNTP (predicate as counter): Set scalar to count from predicate-as-counter.

CNTP (predicate): Set scalar to count of true predicate elements.

COMPACT: Copy Active vector elements to lower-numbered elements.

CPY (immediate, merging): Copy signed integer immediate to vector elements (merging).

CPY (immediate, zeroing): Copy signed integer immediate to vector elements (zeroing).

CPY (scalar): Copy general-purpose register to vector elements (predicated).

CPY (SIMD&FP scalar): Copy SIMD&FP scalar register to vector elements (predicated).

CTERMEQ, CTERMNE: Compare and terminate loop.

DECB, DECD, DECH, DECW (scalar): Decrement scalar by multiple of predicate constraint element count.

DECD, DECH, DECW (vector): Decrement vector by multiple of predicate constraint element count.

DECP (scalar): Decrement scalar by count of true predicate elements.

DECP (vector): Decrement vector by count of true predicate elements.

DUP (immediate): Broadcast signed immediate to vector elements (unpredicated).

DUP (indexed): Broadcast indexed element to vector (unpredicated).

DUP (scalar): Broadcast general-purpose register to vector elements (unpredicated).

DUPM: Broadcast logical bitmask immediate to vector (unpredicated).

DUPQ: Broadcast indexed element within each quadword vector segment (unpredicated).

EON: Bitwise exclusive-OR with inverted immediate (unpredicated): an alias of EOR (immediate).

EOR (immediate): Bitwise exclusive-OR with immediate (unpredicated).

EOR (predicates): Bitwise exclusive-OR predicates.

EOR (vectors, predicated): Bitwise exclusive-OR (predicated).

EOR (vectors, unpredicated): Bitwise exclusive-OR (unpredicated).

EOR3: Bitwise exclusive-OR between three vectors.

EORBT: Interleaving exclusive-OR (bottom, top).

EORQV: Bitwise exclusive-OR reduction of quadword vector segments.

EORS: Bitwise exclusive-OR predicates, setting the condition flags.

EORTB: Interleaving exclusive-OR (top, bottom).

EORV: Bitwise exclusive-OR reduction to scalar.

EXPAND: Copy lower-numbered vector elements to Active elements.

EXT: Extract vector from pair of vectors.

EXTQ: Extract vector segment from each pair of quadword vector segments.

F1CVT, F2CVT: 8-bit floating-point convert to half-precision.

F1CVTLT, F2CVTLT: 8-bit floating-point convert to half-precision (top).

FABD: Floating-point absolute difference (predicated).

FABS: Floating-point absolute value (predicated).

FAC<cc>: Floating-point absolute compare.

FACLE: Floating-point absolute compare less than or equal: an alias of FAC<cc>.

FACLT: Floating-point absolute compare less than: an alias of FAC<cc>.

FADD (immediate): Floating-point add immediate (predicated).

FADD (vectors, predicated): Floating-point add (predicated).

FADD (vectors, unpredicated): Floating-point add (unpredicated).

FADDA: Floating-point add strictly-ordered reduction, accumulating in scalar.

FADDP: Floating-point add pairwise.

FADDQV: Floating-point add recursive reduction of quadword vector segments.

FADDV: Floating-point add recursive reduction to scalar.

FAMAX: Floating-point absolute maximum (predicated).

FAMIN: Floating-point absolute minimum (predicated).

FCADD: Floating-point complex add (predicated).

FCLAMP: Floating-point clamp to minimum/maximum number.

FCM<cc> (vectors): Floating-point compare.

FCM<cc> (zero): Floating-point compare with zero.

FCMLA (indexed): Floating-point complex multiply-add by indexed element.

FCMLA (vectors): Floating-point complex multiply-add (predicated).

FCMLE (vectors): Floating-point compare less than or equal to vector: an alias of FCM<cc> (vectors).

FCMLT (vectors): Floating-point compare less than vector: an alias of FCM<cc> (vectors).

FCPY: Copy floating-point immediate to vector elements (predicated).

FCVT: Floating-point convert (predicated).

FCVTLT: Floating-point widening convert (top, predicated).

FCVTN: Half-precision convert to interleaved 8-bit floating-point.

FCVTNB: Single-precision convert to interleaved 8-bit floating-point (bottom).

FCVTNT (predicated): Floating-point narrowing convert (top, predicated).

FCVTNT (unpredicated): Single-precision convert to interleaved 8-bit floating-point (top).

FCVTX: Double-precision convert to single-precision, rounding to odd (predicated).

FCVTXNT: Double-precision convert to single-precision, rounding to odd (top, predicated).

FCVTZS: Floating-point convert to signed integer, rounding toward zero (predicated).

FCVTZSN: Floating-point narrowing convert to interleaved signed integer, rounding toward zero.

FCVTZU: Floating-point convert to unsigned integer, rounding toward zero (predicated).

FCVTZUN: Floating-point narrowing convert to interleaved unsigned integer, rounding toward zero.

FDIV: Floating-point divide (predicated).

FDIVR: Floating-point reversed divide (predicated).

FDOT (2-way, indexed, FP16 to FP32): Half-precision dot product by indexed element to single-precision.

FDOT (2-way, indexed, FP8 to FP16): 8-bit floating-point dot product by indexed element to half-precision.

FDOT (2-way, vectors, FP16 to FP32): Half-precision dot product to single-precision.

FDOT (2-way, vectors, FP8 to FP16): 8-bit floating-point dot product to half-precision.

FDOT (4-way, indexed): 8-bit floating-point dot product by indexed element to single-precision.

FDOT (4-way, vectors): 8-bit floating-point dot product to single-precision.

FDUP: Broadcast floating-point immediate to vector elements (unpredicated).

FEXPA: Floating-point exponential accelerator.

FIRSTP: Scalar index of first true predicate element (predicated).

FLOGB: Floating-point base 2 logarithm as integer.

FMAD: Floating-point fused multiply-add to multiplicand (predicated).

FMAX (immediate): Floating-point maximum with immediate (predicated).

FMAX (vectors): Floating-point maximum (predicated).

FMAXNM (immediate): Floating-point maximum number with immediate (predicated).

FMAXNM (vectors): Floating-point maximum number (predicated).

FMAXNMP: Floating-point maximum number pairwise.

FMAXNMQV: Floating-point maximum number recursive reduction of quadword vector segments.

FMAXNMV: Floating-point maximum number recursive reduction to scalar.

FMAXP: Floating-point maximum pairwise.

FMAXQV: Floating-point maximum reduction of quadword vector segments.

FMAXV: Floating-point maximum recursive reduction to scalar.

FMIN (immediate): Floating-point minimum with immediate (predicated).

FMIN (vectors): Floating-point minimum (predicated).

FMINNM (immediate): Floating-point minimum number with immediate (predicated).

FMINNM (vectors): Floating-point minimum number (predicated).

FMINNMP: Floating-point minimum number pairwise.

FMINNMQV: Floating-point minimum number recursive reduction of quadword vector segments.

FMINNMV: Floating-point minimum number recursive reduction to scalar.

FMINP: Floating-point minimum pairwise.

FMINQV: Floating-point minimum recursive reduction of quadword vector segments.

FMINV: Floating-point minimum recursive reduction to scalar.

FMLA (indexed): Floating-point fused multiply-add by indexed element.

FMLA (vectors): Floating-point fused multiply-add (predicated).

FMLALB (indexed, FP16 to FP32): Half-precision multiply-add by indexed element to single-precision (bottom).

FMLALB (indexed, FP8 to FP16): 8-bit floating-point multiply-add by indexed element to half-precision (bottom).

FMLALB (vectors, FP16 to FP32): Half-precision multiply-add to single-precision (bottom).

FMLALB (vectors, FP8 to FP16): 8-bit floating-point multiply-add to half-precision (bottom).

FMLALLBB (indexed): 8-bit floating-point multiply-add by indexed element to single-precision (bottom bottom).

FMLALLBB (vectors): 8-bit floating-point multiply-add to single-precision (bottom bottom).

FMLALLBT (indexed): 8-bit floating-point multiply-add by indexed element to single-precision (bottom top).

FMLALLBT (vectors): 8-bit floating-point multiply-add to single-precision (bottom top).

FMLALLTB (indexed): 8-bit floating-point multiply-add by indexed element to single-precision (top bottom).

FMLALLTB (vectors): 8-bit floating-point multiply-add to single-precision (top bottom).

FMLALLTT (indexed): 8-bit floating-point multiply-add by indexed element to single-precision (top top).

FMLALLTT (vectors): 8-bit floating-point multiply-add to single-precision (top top).

FMLALT (indexed, FP16 to FP32): Half-precision multiply-add by indexed element to single-precision (top).

FMLALT (indexed, FP8 to FP16): 8-bit floating-point multiply-add by indexed element to half-precision (top).

FMLALT (vectors, FP16 to FP32): Half-precision multiply-add to single-precision (top).

FMLALT (vectors, FP8 to FP16): 8-bit floating-point multiply-add to half-precision (top).

FMLS (indexed): Floating-point fused multiply-subtract by indexed element.

FMLS (vectors): Floating-point fused multiply-subtract (predicated).

FMLSLB (indexed): Half-precision multiply-subtract by indexed element from single-precision (bottom).

FMLSLB (vectors): Half-precision multiply-subtract from single-precision (bottom).

FMLSLT (indexed): Half-precision multiply-subtract by indexed element from single-precision (top).

FMLSLT (vectors): Half-precision multiply-subtract from single-precision (top).

FMMLA (non-widening): Floating-point matrix multiply-accumulate.

FMMLA (widening, FP16 to FP32): Half-precision matrix multiply-accumulate to single-precision.

FMMLA (widening, FP8 to FP16): 8-bit floating-point matrix multiply-accumulate to half-precision.

FMMLA (widening, FP8 to FP32): 8-bit floating-point matrix multiply-accumulate to single-precision.

FMOV (immediate, predicated): Move floating-point immediate to vector elements (predicated): an alias of FCPY.

FMOV (immediate, unpredicated): Move floating-point immediate to vector elements (unpredicated): an alias of FDUP.

FMOV (zero, predicated): Move floating-point +0.0 to vector elements (predicated): an alias of CPY (immediate, merging).

FMOV (zero, unpredicated): Move floating-point +0.0 to vector elements (unpredicated): an alias of DUP (immediate).

FMSB: Floating-point fused multiply-subtract to multiplicand (predicated).

FMUL (immediate): Floating-point multiply by immediate (predicated).

FMUL (indexed): Floating-point multiply by indexed element.

FMUL (vectors, predicated): Floating-point multiply (predicated).

FMUL (vectors, unpredicated): Floating-point multiply (unpredicated).

FMULX: Floating-point multiply extended (predicated).

FNEG: Floating-point negate (predicated).

FNMAD: Floating-point negated fused multiply-add to multiplicand (predicated).

FNMLA: Floating-point negated fused multiply-add (predicated).

FNMLS: Floating-point negated fused multiply-subtract (predicated).

FNMSB: Floating-point negated fused multiply-subtract to multiplicand (predicated).

FRECPE: Floating-point reciprocal estimate (unpredicated).

FRECPS: Floating-point reciprocal step (unpredicated).

FRECPX: Floating-point reciprocal exponent (predicated).

FRINT32X: Floating-point round to 32-bit integer (predicated).

FRINT32Z: Floating-point round to 32-bit integer, rounding toward zero (predicated).

FRINT64X: Floating-point round to 64-bit integer (predicated).

FRINT64Z: Floating-point round to 64-bit integer, rounding toward zero (predicated).

FRINT<r>: Floating-point round to integral value (predicated).

FRSQRTE: Floating-point reciprocal square root estimate (unpredicated).

FRSQRTS: Floating-point reciprocal square root step (unpredicated).

FSCALE: Floating-point adjust exponent (predicated).

FSQRT: Floating-point square root (predicated).

FSUB (immediate): Floating-point subtract immediate (predicated).

FSUB (vectors, predicated): Floating-point subtract (predicated).

FSUB (vectors, unpredicated): Floating-point subtract (unpredicated).

FSUBR (immediate): Floating-point reversed subtract from immediate (predicated).

FSUBR (vectors): Floating-point reversed subtract (predicated).

FTMAD: Floating-point trigonometric multiply-add coefficient.

FTSMUL: Floating-point trigonometric starting value.

FTSSEL: Floating-point trigonometric select coefficient.

HISTCNT: Count matching elements in vector.

HISTSEG: Count matching elements in vector segments.

INCB, INCD, INCH, INCW (scalar): Increment scalar by multiple of predicate constraint element count.

INCD, INCH, INCW (vector): Increment vector by multiple of predicate constraint element count.

INCP (scalar): Increment scalar by count of true predicate elements.

INCP (vector): Increment vector by count of true predicate elements.

INDEX (immediate, scalar): Create index starting from immediate and incremented by general-purpose register.

INDEX (immediates): Create index starting from and incremented by immediate.

INDEX (scalar, immediate): Create index starting from general-purpose register and incremented by immediate.

INDEX (scalars): Create index starting from and incremented by general-purpose register.

INSR (scalar): Insert general-purpose register in shifted vector.

INSR (SIMD&FP scalar): Insert SIMD&FP scalar register in shifted vector.

LASTA (scalar): Extract element after last to general-purpose register.

LASTA (SIMD&FP scalar): Extract element after last to SIMD&FP scalar register.

LASTB (scalar): Extract last element to general-purpose register.

LASTB (SIMD&FP scalar): Extract last element to SIMD&FP scalar register.

LASTP: Scalar index of last true predicate element (predicated).

LD1B (scalar plus immediate, consecutive registers): Contiguous load of bytes to multiple consecutive vectors (immediate index).

LD1B (scalar plus immediate, single register): Contiguous load unsigned bytes to vector (immediate index).

LD1B (scalar plus scalar, consecutive registers): Contiguous load of bytes to multiple consecutive vectors (scalar index).

LD1B (scalar plus scalar, single register): Contiguous load unsigned bytes to vector (scalar index).

LD1B (scalar plus vector): Gather load unsigned bytes to vector (vector index).

LD1B (vector plus immediate): Gather load unsigned bytes to vector (immediate index).

LD1D (scalar plus immediate, consecutive registers): Contiguous load of doublewords to multiple consecutive vectors (immediate index).

LD1D (scalar plus immediate, single register): Contiguous load unsigned doublewords to vector (immediate index).

LD1D (scalar plus scalar, consecutive registers): Contiguous load of doublewords to multiple consecutive vectors (scalar index).

LD1D (scalar plus scalar, single register): Contiguous load unsigned doublewords to vector (scalar index).

LD1D (scalar plus vector): Gather load doublewords to vector (vector index).

LD1D (vector plus immediate): Gather load doublewords to vector (immediate index).

LD1H (scalar plus immediate, consecutive registers): Contiguous load of halfwords to multiple consecutive vectors (immediate index).

LD1H (scalar plus immediate, single register): Contiguous load unsigned halfwords to vector (immediate index).

LD1H (scalar plus scalar, consecutive registers): Contiguous load of halfwords to multiple consecutive vectors (scalar index).

LD1H (scalar plus scalar, single register): Contiguous load unsigned halfwords to vector (scalar index).

LD1H (scalar plus vector): Gather load unsigned halfwords to vector (vector index).

LD1H (vector plus immediate): Gather load unsigned halfwords to vector (immediate index).

LD1Q: Gather load quadwords.

LD1RB: Load and broadcast unsigned byte to vector.

LD1RD: Load and broadcast doubleword to vector.

LD1RH: Load and broadcast unsigned halfword to vector.

LD1ROB (scalar plus immediate): Contiguous load and replicate thirty-two bytes (immediate index).

LD1ROB (scalar plus scalar): Contiguous load and replicate thirty-two bytes (scalar index).

LD1ROD (scalar plus immediate): Contiguous load and replicate four doublewords (immediate index).

LD1ROD (scalar plus scalar): Contiguous load and replicate four doublewords (scalar index).

LD1ROH (scalar plus immediate): Contiguous load and replicate sixteen halfwords (immediate index).

LD1ROH (scalar plus scalar): Contiguous load and replicate sixteen halfwords (scalar index).

LD1ROW (scalar plus immediate): Contiguous load and replicate eight words (immediate index).

LD1ROW (scalar plus scalar): Contiguous load and replicate eight words (scalar index).

LD1RQB (scalar plus immediate): Contiguous load and replicate sixteen bytes (immediate index).

LD1RQB (scalar plus scalar): Contiguous load and replicate sixteen bytes (scalar index).

LD1RQD (scalar plus immediate): Contiguous load and replicate two doublewords (immediate index).

LD1RQD (scalar plus scalar): Contiguous load and replicate two doublewords (scalar index).

LD1RQH (scalar plus immediate): Contiguous load and replicate eight halfwords (immediate index).

LD1RQH (scalar plus scalar): Contiguous load and replicate eight halfwords (scalar index).

LD1RQW (scalar plus immediate): Contiguous load and replicate four words (immediate index).

LD1RQW (scalar plus scalar): Contiguous load and replicate four words (scalar index).

LD1RSB: Load and broadcast signed byte to vector.

LD1RSH: Load and broadcast signed halfword to vector.

LD1RSW: Load and broadcast signed word to vector.

LD1RW: Load and broadcast unsigned word to vector.

LD1SB (scalar plus immediate): Contiguous load signed bytes to vector (immediate index).

LD1SB (scalar plus scalar): Contiguous load signed bytes to vector (scalar index).

LD1SB (scalar plus vector): Gather load signed bytes to vector (vector index).

LD1SB (vector plus immediate): Gather load signed bytes to vector (immediate index).

LD1SH (scalar plus immediate): Contiguous load signed halfwords to vector (immediate index).

LD1SH (scalar plus scalar): Contiguous load signed halfwords to vector (scalar index).

LD1SH (scalar plus vector): Gather load signed halfwords to vector (vector index).

LD1SH (vector plus immediate): Gather load signed halfwords to vector (immediate index).

LD1SW (scalar plus immediate): Contiguous load signed words to vector (immediate index).

LD1SW (scalar plus scalar): Contiguous load signed words to vector (scalar index).

LD1SW (scalar plus vector): Gather load signed words to vector (vector index).

LD1SW (vector plus immediate): Gather load signed words to vector (immediate index).

LD1W (scalar plus immediate, consecutive registers): Contiguous load of words to multiple consecutive vectors (immediate index).

LD1W (scalar plus immediate, single register): Contiguous load unsigned words to vector (immediate index).

LD1W (scalar plus scalar, consecutive registers): Contiguous load of words to multiple consecutive vectors (scalar index).

LD1W (scalar plus scalar, single register): Contiguous load unsigned words to vector (scalar index).

LD1W (scalar plus vector): Gather load unsigned words to vector (vector index).

LD1W (vector plus immediate): Gather load unsigned words to vector (immediate index).

LD2B (scalar plus immediate): Contiguous load two-byte structures to two vectors (immediate index).

LD2B (scalar plus scalar): Contiguous load two-byte structures to two vectors (scalar index).

LD2D (scalar plus immediate): Contiguous load two-doubleword structures to two vectors (immediate index).

LD2D (scalar plus scalar): Contiguous load two-doubleword structures to two vectors (scalar index).

LD2H (scalar plus immediate): Contiguous load two-halfword structures to two vectors (immediate index).

LD2H (scalar plus scalar): Contiguous load two-halfword structures to two vectors (scalar index).

LD2Q (scalar plus immediate): Contiguous load two-quadword structures to two vectors (immediate index).

LD2Q (scalar plus scalar): Contiguous load two-quadword structures to two vectors (scalar index).

LD2W (scalar plus immediate): Contiguous load two-word structures to two vectors (immediate index).

LD2W (scalar plus scalar): Contiguous load two-word structures to two vectors (scalar index).

LD3B (scalar plus immediate): Contiguous load three-byte structures to three vectors (immediate index).

LD3B (scalar plus scalar): Contiguous load three-byte structures to three vectors (scalar index).

LD3D (scalar plus immediate): Contiguous load three-doubleword structures to three vectors (immediate index).

LD3D (scalar plus scalar): Contiguous load three-doubleword structures to three vectors (scalar index).

LD3H (scalar plus immediate): Contiguous load three-halfword structures to three vectors (immediate index).

LD3H (scalar plus scalar): Contiguous load three-halfword structures to three vectors (scalar index).

LD3Q (scalar plus immediate): Contiguous load three-quadword structures to three vectors (immediate index).

LD3Q (scalar plus scalar): Contiguous load three-quadword structures to three vectors (scalar index).

LD3W (scalar plus immediate): Contiguous load three-word structures to three vectors (immediate index).

LD3W (scalar plus scalar): Contiguous load three-word structures to three vectors (scalar index).

LD4B (scalar plus immediate): Contiguous load four-byte structures to four vectors (immediate index).

LD4B (scalar plus scalar): Contiguous load four-byte structures to four vectors (scalar index).

LD4D (scalar plus immediate): Contiguous load four-doubleword structures to four vectors (immediate index).

LD4D (scalar plus scalar): Contiguous load four-doubleword structures to four vectors (scalar index).

LD4H (scalar plus immediate): Contiguous load four-halfword structures to four vectors (immediate index).

LD4H (scalar plus scalar): Contiguous load four-halfword structures to four vectors (scalar index).

LD4Q (scalar plus immediate): Contiguous load four-quadword structures to four vectors (immediate index).

LD4Q (scalar plus scalar): Contiguous load four-quadword structures to four vectors (scalar index).

LD4W (scalar plus immediate): Contiguous load four-word structures to four vectors (immediate index).

LD4W (scalar plus scalar): Contiguous load four-word structures to four vectors (scalar index).

LDAPRO: Load-Acquire RCpc octaword.

LDFF1B (scalar plus scalar): Contiguous load first-fault unsigned bytes to vector (scalar index).

LDFF1B (scalar plus vector): Gather load first-fault unsigned bytes to vector (vector index).

LDFF1B (vector plus immediate): Gather load first-fault unsigned bytes to vector (immediate index).

LDFF1D (scalar plus scalar): Contiguous load first-fault doublewords to vector (scalar index).

LDFF1D (scalar plus vector): Gather load first-fault doublewords to vector (vector index).

LDFF1D (vector plus immediate): Gather load first-fault doublewords to vector (immediate index).

LDFF1H (scalar plus scalar): Contiguous load first-fault unsigned halfwords to vector (scalar index).

LDFF1H (scalar plus vector): Gather load first-fault unsigned halfwords to vector (vector index).

LDFF1H (vector plus immediate): Gather load first-fault unsigned halfwords to vector (immediate index).

LDFF1SB (scalar plus scalar): Contiguous load first-fault signed bytes to vector (scalar index).

LDFF1SB (scalar plus vector): Gather load first-fault signed bytes to vector (vector index).

LDFF1SB (vector plus immediate): Gather load first-fault signed bytes to vector (immediate index).

LDFF1SH (scalar plus scalar): Contiguous load first-fault signed halfwords to vector (scalar index).

LDFF1SH (scalar plus vector): Gather load first-fault signed halfwords to vector (vector index).

LDFF1SH (vector plus immediate): Gather load first-fault signed halfwords to vector (immediate index).

LDFF1SW (scalar plus scalar): Contiguous load first-fault signed words to vector (scalar index).

LDFF1SW (scalar plus vector): Gather load first-fault signed words to vector (vector index).

LDFF1SW (vector plus immediate): Gather load first-fault signed words to vector (immediate index).

LDFF1W (scalar plus scalar): Contiguous load first-fault unsigned words to vector (scalar index).

LDFF1W (scalar plus vector): Gather load first-fault unsigned words to vector (vector index).

LDFF1W (vector plus immediate): Gather load first-fault unsigned words to vector (immediate index).

LDNF1B: Contiguous load non-fault unsigned bytes to vector (immediate index).

LDNF1D: Contiguous load non-fault doublewords to vector (immediate index).

LDNF1H: Contiguous load non-fault unsigned halfwords to vector (immediate index).

LDNF1SB: Contiguous load non-fault signed bytes to vector (immediate index).

LDNF1SH: Contiguous load non-fault signed halfwords to vector (immediate index).

LDNF1SW: Contiguous load non-fault signed words to vector (immediate index).

LDNF1W: Contiguous load non-fault unsigned words to vector (immediate index).

LDNT1B (scalar plus immediate, consecutive registers): Contiguous load non-temporal of bytes to multiple consecutive vectors (immediate index).

LDNT1B (scalar plus immediate, single register): Contiguous load non-temporal bytes to vector (immediate index).

LDNT1B (scalar plus scalar, consecutive registers): Contiguous load non-temporal of bytes to multiple consecutive vectors (scalar index).

LDNT1B (scalar plus scalar, single register): Contiguous load non-temporal bytes to vector (scalar index).

LDNT1B (vector plus scalar): Gather load non-temporal unsigned bytes.

LDNT1D (scalar plus immediate, consecutive registers): Contiguous load non-temporal of doublewords to multiple consecutive vectors (immediate index).

LDNT1D (scalar plus immediate, single register): Contiguous load non-temporal doublewords to vector (immediate index).

LDNT1D (scalar plus scalar, consecutive registers): Contiguous load non-temporal of doublewords to multiple consecutive vectors (scalar index).

LDNT1D (scalar plus scalar, single register): Contiguous load non-temporal doublewords to vector (scalar index).

LDNT1D (vector plus scalar): Gather load non-temporal unsigned doublewords.

LDNT1H (scalar plus immediate, consecutive registers): Contiguous load non-temporal of halfwords to multiple consecutive vectors (immediate index).

LDNT1H (scalar plus immediate, single register): Contiguous load non-temporal halfwords to vector (immediate index).

LDNT1H (scalar plus scalar, consecutive registers): Contiguous load non-temporal of halfwords to multiple consecutive vectors (scalar index).

LDNT1H (scalar plus scalar, single register): Contiguous load non-temporal halfwords to vector (scalar index).

LDNT1H (vector plus scalar): Gather load non-temporal unsigned halfwords.

LDNT1SB: Gather load non-temporal signed bytes.

LDNT1SH: Gather load non-temporal signed halfwords.

LDNT1SW: Gather load non-temporal signed words.

LDNT1W (scalar plus immediate, consecutive registers): Contiguous load non-temporal of words to multiple consecutive vectors (immediate index).

LDNT1W (scalar plus immediate, single register): Contiguous load non-temporal words to vector (immediate index).

LDNT1W (scalar plus scalar, consecutive registers): Contiguous load non-temporal of words to multiple consecutive vectors (scalar index).

LDNT1W (scalar plus scalar, single register): Contiguous load non-temporal words to vector (scalar index).

LDNT1W (vector plus scalar): Gather load non-temporal unsigned words.

LDR (predicate): Load predicate register.

LDR (vector): Load vector register.

LSL (immediate, predicated): Logical shift left by immediate (predicated).

LSL (immediate, unpredicated): Logical shift left by immediate (unpredicated).

LSL (vectors): Logical shift left by vector (predicated).

LSL (wide elements, predicated): Logical shift left by 64-bit wide elements (predicated).

LSL (wide elements, unpredicated): Logical shift left by 64-bit wide elements (unpredicated).

LSLR: Reversed logical shift left by vector (predicated).

LSR (immediate, predicated): Logical shift right by immediate (predicated).

LSR (immediate, unpredicated): Logical shift right by immediate (unpredicated).

LSR (vectors): Logical shift right by vector (predicated).

LSR (wide elements, predicated): Logical shift right by 64-bit wide elements (predicated).

LSR (wide elements, unpredicated): Logical shift right by 64-bit wide elements (unpredicated).

LSRR: Reversed logical shift right by vector (predicated).

LUTI2 (8-bit and 16-bit): Lookup table read with 2-bit indices (8-bit and 16-bit).

LUTI4 (8-bit and 16-bit): Lookup table read with 4-bit indices (8-bit and 16-bit).

LUTI6 (16-bit): Lookup table read with 6-bit indices (16-bit).

LUTI6 (8-bit): Lookup table read with 6-bit indices (8-bit).

MAD: Multiply-add to multiplicand (predicated).

MADPT: Multiply-add checked pointer vectors to multiplicand.

MATCH: Detect any matching elements within quadwords, setting the condition flags.

MLA (indexed): Multiply-add by indexed element.

MLA (vectors): Multiply-add (predicated).

MLAPT: Multiply-add checked pointer vectors.

MLS (indexed): Multiply-subtract by indexed element.

MLS (vectors): Multiply-subtract (predicated).

MOV: Move logical bitmask immediate to vector (unpredicated): an alias of DUPM.

MOV: Move predicate (unpredicated): an alias of ORR (predicates).

MOV (immediate, merging): Move signed integer immediate to vector elements (merging): an alias of CPY (immediate, merging).

MOV (immediate, unpredicated): Move signed immediate to vector elements (unpredicated): an alias of DUP (immediate).

MOV (immediate, zeroing): Move signed integer immediate to vector elements (zeroing): an alias of CPY (immediate, zeroing).

MOV (predicate, merging): Move predicates (merging): an alias of SEL (predicates).

MOV (predicate, zeroing): Move predicates (zeroing): an alias of AND (predicates).

MOV (scalar, predicated): Move general-purpose register to vector elements (predicated): an alias of CPY (scalar).

MOV (scalar, unpredicated): Move general-purpose register to vector elements (unpredicated): an alias of DUP (scalar).

MOV (SIMD&FP scalar, predicated): Move SIMD&FP scalar register to vector elements (predicated): an alias of CPY (SIMD&FP scalar).

MOV (SIMD&FP scalar, unpredicated): Move indexed element or SIMD&FP scalar to vector (unpredicated): an alias of DUP (indexed).

MOV (vector, predicated): Move vector elements (predicated): an alias of SEL (vectors).

MOV (vector, unpredicated): Move vector register (unpredicated): an alias of ORR (vectors, unpredicated).

MOVPRFX (predicated): Move prefix (predicated).

MOVPRFX (unpredicated): Move prefix (unpredicated).

MOVS (predicated): Move predicates (zeroing), setting the condition flags: an alias of ANDS.

MOVS (unpredicated): Move predicate (unpredicated), setting the condition flags: an alias of ORRS.

MSB: Multiply-subtract to multiplicand.

MUL (immediate): Multiply by immediate (unpredicated).

MUL (indexed): Multiply by indexed element.

MUL (vectors, predicated): Multiply (predicated).

MUL (vectors, unpredicated): Multiply (unpredicated).

NAND: Bitwise NAND predicates.

NANDS: Bitwise NAND predicates, setting the condition flags.

NBSL: Bitwise inverted select.

NEG: Negate (predicated).

NMATCH: Detect no matching elements within quadwords, setting the condition flags.

NOR: Bitwise NOR predicates.

NORS: Bitwise NOR predicates, setting the condition flags.

NOT (predicate): Bitwise invert predicate: an alias of EOR (predicates).

NOT (vector): Bitwise invert (predicated).

NOTS: Bitwise invert predicate, setting the condition flags: an alias of EORS.

ORN (immediate): Bitwise inclusive OR with inverted immediate (unpredicated): an alias of ORR (immediate).

ORN (predicates): Bitwise inclusive OR inverted predicate.

ORNS: Bitwise inclusive OR inverted predicate, setting the condition flags.

ORQV: Bitwise inclusive OR reduction of quadword vector segments.

ORR (immediate): Bitwise inclusive OR with immediate (unpredicated).

ORR (predicates): Bitwise inclusive OR predicates.

ORR (vectors, predicated): Bitwise inclusive OR (predicated).

ORR (vectors, unpredicated): Bitwise inclusive OR (unpredicated).

ORRS: Bitwise inclusive OR predicates, setting the condition flags.

ORV: Bitwise inclusive OR reduction to scalar.

PEXT (predicate pair): Predicate extract pair from predicate-as-counter.

PEXT (predicate): Predicate extract from predicate-as-counter.

PFALSE: Set all predicate elements to false.

PFIRST: Set the First active predicate element to true.

PMLAL: Multi-vector polynomial multiply long and accumulate.

PMOV (to predicate): Move predicate from vector.

PMOV (to vector): Move predicate to vector.

PMUL: Polynomial multiply (unpredicated).

PMULL: Multi-vector polynomial multiply long.

PMULLB: Polynomial multiply long (bottom).

PMULLT: Polynomial multiply long (top).

PNEXT: Find next active predicate.

PRFB (scalar plus immediate): Contiguous prefetch bytes (immediate index).

PRFB (scalar plus scalar): Contiguous prefetch bytes (scalar index).

PRFB (scalar plus vector): Gather prefetch bytes (scalar plus vector).

PRFB (vector plus immediate): Gather prefetch bytes (vector plus immediate).

PRFD (scalar plus immediate): Contiguous prefetch doublewords (immediate index).

PRFD (scalar plus scalar): Contiguous prefetch doublewords (scalar index).

PRFD (scalar plus vector): Gather prefetch doublewords (scalar plus vector).

PRFD (vector plus immediate): Gather prefetch doublewords (vector plus immediate).

PRFH (scalar plus immediate): Contiguous prefetch halfwords (immediate index).

PRFH (scalar plus scalar): Contiguous prefetch halfwords (scalar index).

PRFH (scalar plus vector): Gather prefetch halfwords (scalar plus vector).

PRFH (vector plus immediate): Gather prefetch halfwords (vector plus immediate).

PRFW (scalar plus immediate): Contiguous prefetch words (immediate index).

PRFW (scalar plus scalar): Contiguous prefetch words (scalar index).

PRFW (scalar plus vector): Gather prefetch words (scalar plus vector).

PRFW (vector plus immediate): Gather prefetch words (vector plus immediate).

PSEL: Predicate select between predicate register or all-false.

PTEST: Set condition flags for predicate.

PTRUE (predicate as counter): Initialize predicate-as-counter to all active.

PTRUE (predicate): Initialize predicate from named constraint.

PTRUES: Initialize predicate from named constraint and set the condition flags.

PUNPKHI, PUNPKLO: Unpack and widen half of predicate.

RADDHNB: Rounding add narrow high part (bottom).

RADDHNT: Rounding add narrow high part (top).

RAX1: Bitwise rotate left by 1 and exclusive-OR.

RBIT: Reverse bits (predicated).

RDFFR (predicated): Return predicate of successfully loaded elements.

RDFFR (unpredicated): Read the first-fault register.

RDFFRS: Return predicate of successfully loaded elements, setting the condition flags.

RDVL: Read multiple of vector register size to scalar register.

REV (predicate): Reverse all elements in a predicate.

REV (vector): Reverse all elements in a vector (unpredicated).

REVB, REVH, REVW: Reverse bytes / halfwords / words within elements (predicated).

REVD: Reverse 64-bit doublewords in elements (predicated).

RSHRNB: Rounding shift right narrow by immediate (bottom).

RSHRNT: Rounding shift right narrow by immediate (top).

RSUBHNB: Rounding subtract narrow high part (bottom).

RSUBHNT: Rounding subtract narrow high part (top).

SABA: Signed absolute difference and accumulate.

SABAL: Two-way signed absolute difference sum and accumulate long.

SABALB: Signed absolute difference and accumulate long (bottom).

SABALT: Signed absolute difference and accumulate long (top).

SABD: Signed absolute difference (predicated).

SABDLB: Signed absolute difference long (bottom).

SABDLT: Signed absolute difference long (top).

SADALP: Signed add and accumulate long pairwise.

SADDLB: Signed add long (bottom).

SADDLBT: Signed add long (bottom + top).

SADDLT: Signed add long (top).

SADDV: Signed add reduction to scalar.

SADDWB: Signed add wide (bottom).

SADDWT: Signed add wide (top).

SBCLB: Subtract with carry long (bottom).

SBCLT: Subtract with carry long (top).

SCLAMP: Signed clamp to minimum/maximum.

SCVTF (bottom, unpredicated): Signed integer convert to floating-point (bottom, unpredicated).

SCVTF (predicated): Signed integer convert to floating-point (predicated).

SCVTFLT: Signed integer convert to floating-point (top, unpredicated).

SDIV: Signed divide (predicated).

SDIVR: Signed reversed divide (predicated).

SDOT (2-way, indexed): Signed integer dot product by indexed element (two-way).

SDOT (2-way, vectors): Signed integer dot product (two-way).

SDOT (4-way, indexed): Signed integer dot product by indexed element (four-way).

SDOT (4-way, vectors): Signed integer dot product (four-way).

SEL (predicates): Conditionally select elements from two predicates.

SEL (vectors): Conditionally select elements from two vectors.

SETFFR: Initialize the first-fault register to all true.

SHADD: Signed halving add.

SHRNB: Shift right narrow by immediate (bottom).

SHRNT: Shift right narrow by immediate (top).

SHSUB: Signed halving subtract.

SHSUBR: Signed halving subtract reversed.

SLI: Shift left and insert (immediate).

SM4E: SM4 encryption and decryption.

SM4EKEY: SM4 key updates.

SMAX (immediate): Signed maximum with immediate (unpredicated).

SMAX (vectors): Signed maximum (predicated).

SMAXP: Signed maximum pairwise.

SMAXQV: Signed maximum reduction of quadword vector segments.

SMAXV: Signed maximum reduction to scalar.

SMIN (immediate): Signed minimum with immediate (unpredicated).

SMIN (vectors): Signed minimum (predicated).

SMINP: Signed minimum pairwise.

SMINQV: Signed minimum reduction of quadword vector segments.

SMINV: Signed minimum reduction to scalar.

SMLALB (indexed): Signed multiply-add long by indexed element (bottom).

SMLALB (vectors): Signed multiply-add long (bottom).

SMLALT (indexed): Signed multiply-add long by indexed element (top).

SMLALT (vectors): Signed multiply-add long (top).

SMLSLB (indexed): Signed multiply-subtract long by indexed element (bottom).

SMLSLB (vectors): Signed multiply-subtract long (bottom).

SMLSLT (indexed): Signed multiply-subtract long by indexed element (top).

SMLSLT (vectors): Signed multiply-subtract long (top).

SMMLA: Signed 8-bit integer matrix multiply-accumulate to 32-bit integer.

SMULH (predicated): Signed multiply returning high half (predicated).

SMULH (unpredicated): Signed multiply returning high half (unpredicated).

SMULLB (indexed): Signed multiply long by indexed element (bottom).

SMULLB (vectors): Signed multiply long (bottom).

SMULLT (indexed): Signed multiply long by indexed element (top).

SMULLT (vectors): Signed multiply long (top).

SPLICE: Splice two vectors under predicate control.

SQABS: Signed saturating absolute value.

SQADD (immediate): Signed saturating add immediate (unpredicated).

SQADD (vectors, predicated): Signed saturating add (predicated).

SQADD (vectors, unpredicated): Signed saturating add (unpredicated).

SQCADD: Saturating complex integer add.

SQCVTN: Signed 32-bit integer saturating extract narrow to interleaved 16-bit integer.

SQCVTUN: Signed 32-bit integer saturating extract narrow to interleaved unsigned 16-bit integer.

SQDECB: Signed saturating decrement scalar by multiple of 8-bit predicate constraint element count.

SQDECD (scalar): Signed saturating decrement scalar by multiple of 64-bit predicate constraint element count.

SQDECD (vector): Signed saturating decrement vector by multiple of 64-bit predicate constraint element count.

SQDECH (scalar): Signed saturating decrement scalar by multiple of 16-bit predicate constraint element count.

SQDECH (vector): Signed saturating decrement vector by multiple of 16-bit predicate constraint element count.

SQDECP (scalar): Signed saturating decrement scalar by count of true predicate elements.

SQDECP (vector): Signed saturating decrement vector by count of true predicate elements.

SQDECW (scalar): Signed saturating decrement scalar by multiple of 32-bit predicate constraint element count.

SQDECW (vector): Signed saturating decrement vector by multiple of 32-bit predicate constraint element count.

SQDMLALB (indexed): Signed saturating doubling multiply-add by indexed element (bottom).

SQDMLALB (vectors): Signed saturating doubling multiply-add (bottom).

SQDMLALBT: Signed saturating doubling multiply-add (bottom × top).

SQDMLALT (indexed): Signed saturating doubling multiply-add by indexed element (top).

SQDMLALT (vectors): Signed saturating doubling multiply-add (top).

SQDMLSLB (indexed): Signed saturating doubling multiply-subtract by indexed element (bottom).

SQDMLSLB (vectors): Signed saturating doubling multiply-subtract (bottom).

SQDMLSLBT: Signed saturating doubling multiply-subtract (bottom × top).

SQDMLSLT (indexed): Signed saturating doubling multiply-subtract by indexed element (top).

SQDMLSLT (vectors): Signed saturating doubling multiply-subtract (top).

SQDMULH (indexed): Signed saturating doubling multiply high by indexed element.

SQDMULH (vectors): Signed saturating doubling multiply high (unpredicated).

SQDMULLB (indexed): Signed saturating doubling multiply by indexed element (bottom).

SQDMULLB (vectors): Signed saturating doubling multiply (bottom).

SQDMULLT (indexed): Signed saturating doubling multiply by indexed element (top).

SQDMULLT (vectors): Signed saturating doubling multiply (top).

SQINCB: Signed saturating increment scalar by multiple of 8-bit predicate constraint element count.

SQINCD (scalar): Signed saturating increment scalar by multiple of 64-bit predicate constraint element count.

SQINCD (vector): Signed saturating increment vector by multiple of 64-bit predicate constraint element count.

SQINCH (scalar): Signed saturating increment scalar by multiple of 16-bit predicate constraint element count.

SQINCH (vector): Signed saturating increment vector by multiple of 16-bit predicate constraint element count.

SQINCP (scalar): Signed saturating increment scalar by count of true predicate elements.

SQINCP (vector): Signed saturating increment vector by count of true predicate elements.

SQINCW (scalar): Signed saturating increment scalar by multiple of 32-bit predicate constraint element count.

SQINCW (vector): Signed saturating increment vector by multiple of 32-bit predicate constraint element count.

SQNEG: Signed saturating negate.

SQRDCMLAH (indexed): Saturating rounding doubling complex integer multiply-add high by indexed element.

SQRDCMLAH (vectors): Saturating rounding doubling complex integer multiply-add high.

SQRDMLAH (indexed): Signed saturating rounding doubling multiply-add high by indexed element.

SQRDMLAH (vectors): Signed saturating rounding doubling multiply-add high (unpredicated).

SQRDMLSH (indexed): Signed saturating rounding doubling multiply-subtract high by indexed element.

SQRDMLSH (vectors): Signed saturating rounding doubling multiply-subtract high (unpredicated).

SQRDMULH (indexed): Signed saturating rounding doubling multiply high by indexed element.

SQRDMULH (vectors): Signed saturating rounding doubling multiply high (unpredicated).

SQRSHL: Signed saturating rounding shift left (predicated).

SQRSHLR: Signed saturating rounding shift left reversed (predicated).

SQRSHRN: Signed saturating rounding shift right narrow by immediate to interleaved integer.

SQRSHRNB: Signed saturating rounding shift right narrow by immediate (bottom).

SQRSHRNT: Signed saturating rounding shift right narrow by immediate (top).

SQRSHRUN: Signed saturating rounding shift right unsigned narrow by immediate and interleave.

SQRSHRUNB: Signed saturating rounding shift right narrow by immediate to unsigned integer (bottom).

SQRSHRUNT: Signed saturating rounding shift right narrow by immediate to unsigned integer (top).

SQSHL (immediate): Signed saturating shift left by immediate.

SQSHL (vectors): Signed saturating shift left (predicated).

SQSHLR: Signed saturating shift left reversed (predicated).

SQSHLU: Signed saturating shift left unsigned by immediate.

SQSHRN: Signed saturating shift right narrow by immediate to interleaved integer.

SQSHRNB: Signed saturating shift right narrow by immediate (bottom).

SQSHRNT: Signed saturating shift right narrow by immediate (top).

SQSHRUN: Signed saturating shift right narrow by immediate to interleaved unsigned integer.

SQSHRUNB: Signed saturating shift right narrow by immediate to unsigned integer (bottom).

SQSHRUNT: Signed saturating shift right narrow by immediate to unsigned integer (top).

SQSUB (immediate): Signed saturating subtract immediate (unpredicated).

SQSUB (vectors, predicated): Signed saturating subtract (predicated).

SQSUB (vectors, unpredicated): Signed saturating subtract (unpredicated).

SQSUBR: Signed saturating subtract reversed (predicated).

SQXTNB: Signed saturating extract narrow (bottom).

SQXTNT: Signed saturating extract narrow (top).

SQXTUNB: Signed saturating extract narrow to unsigned integer (bottom).

SQXTUNT: Signed saturating extract narrow to unsigned integer (top).

SRHADD: Signed rounding halving add.

SRI: Shift right and insert (immediate).

SRSHL: Signed rounding shift left (predicated).

SRSHLR: Signed rounding shift left reversed (predicated).

SRSHR: Signed rounding shift right by immediate.

SRSRA: Signed rounding shift right and accumulate (immediate).

SSHLLB: Signed shift left long by immediate (bottom).

SSHLLT: Signed shift left long by immediate (top).

SSRA: Signed shift right and accumulate (immediate).

SSUBLB: Signed subtract long (bottom).

SSUBLBT: Signed subtract long (bottom - top).

SSUBLT: Signed subtract long (top).

SSUBLTB: Signed subtract long (top - bottom).

SSUBWB: Signed subtract wide (bottom).

SSUBWT: Signed subtract wide (top).

ST1B (scalar plus immediate, consecutive registers): Contiguous store of bytes from multiple consecutive vectors (immediate index).

ST1B (scalar plus immediate, single register): Contiguous store bytes from vector (immediate index).

ST1B (scalar plus scalar, consecutive registers): Contiguous store of bytes from multiple consecutive vectors (scalar index).

ST1B (scalar plus scalar, single register): Contiguous store bytes from vector (scalar index).

ST1B (scalar plus vector): Scatter store bytes from a vector (vector index).

ST1B (vector plus immediate): Scatter store bytes from a vector (immediate index).

ST1D (scalar plus immediate, consecutive registers): Contiguous store of doublewords from multiple consecutive vectors (immediate index).

ST1D (scalar plus immediate, single register): Contiguous store doublewords from vector (immediate index).

ST1D (scalar plus scalar, consecutive registers): Contiguous store of doublewords from multiple consecutive vectors (scalar index).

ST1D (scalar plus scalar, single register): Contiguous store doublewords from vector (scalar index).

ST1D (scalar plus vector): Scatter store doublewords from a vector (vector index).

ST1D (vector plus immediate): Scatter store doublewords from a vector (immediate index).

ST1H (scalar plus immediate, consecutive registers): Contiguous store of halfwords from multiple consecutive vectors (immediate index).

ST1H (scalar plus immediate, single register): Contiguous store halfwords from vector (immediate index).

ST1H (scalar plus scalar, consecutive registers): Contiguous store of halfwords from multiple consecutive vectors (scalar index).

ST1H (scalar plus scalar, single register): Contiguous store halfwords from vector (scalar index).

ST1H (scalar plus vector): Scatter store halfwords from a vector (vector index).

ST1H (vector plus immediate): Scatter store halfwords from a vector (immediate index).

ST1Q: Scatter store quadwords.

ST1W (scalar plus immediate, consecutive registers): Contiguous store of words from multiple consecutive vectors (immediate index).

ST1W (scalar plus immediate, single register): Contiguous store words from vector (immediate index).

ST1W (scalar plus scalar, consecutive registers): Contiguous store of words from multiple consecutive vectors (scalar index).

ST1W (scalar plus scalar, single register): Contiguous store words from vector (scalar index).

ST1W (scalar plus vector): Scatter store words from a vector (vector index).

ST1W (vector plus immediate): Scatter store words from a vector (immediate index).

ST2B (scalar plus immediate): Contiguous store two-byte structures from two vectors (immediate index).

ST2B (scalar plus scalar): Contiguous store two-byte structures from two vectors (scalar index).

ST2D (scalar plus immediate): Contiguous store two-doubleword structures from two vectors (immediate index).

ST2D (scalar plus scalar): Contiguous store two-doubleword structures from two vectors (scalar index).

ST2H (scalar plus immediate): Contiguous store two-halfword structures from two vectors (immediate index).

ST2H (scalar plus scalar): Contiguous store two-halfword structures from two vectors (scalar index).

ST2Q (scalar plus immediate): Contiguous store two-quadword structures from two vectors (immediate index).

ST2Q (scalar plus scalar): Contiguous store two-quadword structures from two vectors (scalar index).

ST2W (scalar plus immediate): Contiguous store two-word structures from two vectors (immediate index).

ST2W (scalar plus scalar): Contiguous store two-word structures from two vectors (scalar index).

ST3B (scalar plus immediate): Contiguous store three-byte structures from three vectors (immediate index).

ST3B (scalar plus scalar): Contiguous store three-byte structures from three vectors (scalar index).

ST3D (scalar plus immediate): Contiguous store three-doubleword structures from three vectors (immediate index).

ST3D (scalar plus scalar): Contiguous store three-doubleword structures from three vectors (scalar index).

ST3H (scalar plus immediate): Contiguous store three-halfword structures from three vectors (immediate index).

ST3H (scalar plus scalar): Contiguous store three-halfword structures from three vectors (scalar index).

ST3Q (scalar plus immediate): Contiguous store three-quadword structures from three vectors (immediate index).

ST3Q (scalar plus scalar): Contiguous store three-quadword structures from three vectors (scalar index).

ST3W (scalar plus immediate): Contiguous store three-word structures from three vectors (immediate index).

ST3W (scalar plus scalar): Contiguous store three-word structures from three vectors (scalar index).

ST4B (scalar plus immediate): Contiguous store four-byte structures from four vectors (immediate index).

ST4B (scalar plus scalar): Contiguous store four-byte structures from four vectors (scalar index).

ST4D (scalar plus immediate): Contiguous store four-doubleword structures from four vectors (immediate index).

ST4D (scalar plus scalar): Contiguous store four-doubleword structures from four vectors (scalar index).

ST4H (scalar plus immediate): Contiguous store four-halfword structures from four vectors (immediate index).

ST4H (scalar plus scalar): Contiguous store four-halfword structures from four vectors (scalar index).

ST4Q (scalar plus immediate): Contiguous store four-quadword structures from four vectors (immediate index).

ST4Q (scalar plus scalar): Contiguous store four-quadword structures from four vectors (scalar index).

ST4W (scalar plus immediate): Contiguous store four-word structures from four vectors (immediate index).

ST4W (scalar plus scalar): Contiguous store four-word structures from four vectors (scalar index).

STLRO: Store-release octaword.

STNT1B (scalar plus immediate, consecutive registers): Contiguous store non-temporal of bytes from multiple consecutive vectors (immediate index).

STNT1B (scalar plus immediate, single register): Contiguous store non-temporal bytes from vector (immediate index).

STNT1B (scalar plus scalar, consecutive registers): Contiguous store non-temporal of bytes from multiple consecutive vectors (scalar index).

STNT1B (scalar plus scalar, single register): Contiguous store non-temporal bytes from vector (scalar index).

STNT1B (vector plus scalar): Scatter store non-temporal bytes.

STNT1D (scalar plus immediate, consecutive registers): Contiguous store non-temporal of doublewords from multiple consecutive vectors (immediate index).

STNT1D (scalar plus immediate, single register): Contiguous store non-temporal doublewords from vector (immediate index).

STNT1D (scalar plus scalar, consecutive registers): Contiguous store non-temporal of doublewords from multiple consecutive vectors (scalar index).

STNT1D (scalar plus scalar, single register): Contiguous store non-temporal doublewords from vector (scalar index).

STNT1D (vector plus scalar): Scatter store non-temporal doublewords.

STNT1H (scalar plus immediate, consecutive registers): Contiguous store non-temporal of halfwords from multiple consecutive vectors (immediate index).

STNT1H (scalar plus immediate, single register): Contiguous store non-temporal halfwords from vector (immediate index).

STNT1H (scalar plus scalar, consecutive registers): Contiguous store non-temporal of halfwords from multiple consecutive vectors (scalar index).

STNT1H (scalar plus scalar, single register): Contiguous store non-temporal halfwords from vector (scalar index).

STNT1H (vector plus scalar): Scatter store non-temporal halfwords.

STNT1W (scalar plus immediate, consecutive registers): Contiguous store non-temporal of words from multiple consecutive vectors (immediate index).

STNT1W (scalar plus immediate, single register): Contiguous store non-temporal words from vector (immediate index).

STNT1W (scalar plus scalar, consecutive registers): Contiguous store non-temporal of words from multiple consecutive vectors (scalar index).

STNT1W (scalar plus scalar, single register): Contiguous store non-temporal words from vector (scalar index).

STNT1W (vector plus scalar): Scatter store non-temporal words.

STR (predicate): Store predicate register.

STR (vector): Store vector register.

SUB (immediate): Subtract immediate (unpredicated).

SUB (vectors, predicated): Subtract (predicated).

SUB (vectors, unpredicated): Subtract (unpredicated).

SUBHNB: Subtract narrow high part (bottom).

SUBHNT: Subtract narrow high part (top).

SUBP: Subtract pairwise.

SUBPT (predicated): Subtract checked pointer vectors (predicated).

SUBPT (unpredicated): Subtract checked pointer vectors (unpredicated).

SUBR (immediate): Reversed subtract from immediate (unpredicated).

SUBR (vectors): Reversed subtract (predicated).

SUDOT: Signed by unsigned 8-bit integer dot product by indexed element to 32-bit integer.

SUNPKHI, SUNPKLO: Signed unpack and extend half of vector.

SUQADD: Signed saturating unsigned add.

SXTB, SXTH, SXTW: Signed byte / halfword / word extend (predicated).

TBL: Programmable table lookup in one or two vector table (zeroing).

TBLQ: Programmable table lookup within each quadword vector segment (zeroing).

TBX: Programmable table lookup in single vector table (merging).

TBXQ: Programmable table lookup within each quadword vector segment (merging).

TRN1, TRN2 (predicates): Interleave even or odd elements from two predicates.

TRN1, TRN2 (vectors): Interleave even or odd elements from two vectors.

UABA: Unsigned absolute difference and accumulate.

UABAL: Two-way unsigned absolute difference sum and accumulate long.

UABALB: Unsigned absolute difference and accumulate long (bottom).

UABALT: Unsigned absolute difference and accumulate long (top).

UABD: Unsigned absolute difference (predicated).

UABDLB: Unsigned absolute difference long (bottom).

UABDLT: Unsigned absolute difference long (top).

UADALP: Unsigned add and accumulate long pairwise.

UADDLB: Unsigned add long (bottom).

UADDLT: Unsigned add long (top).

UADDV: Unsigned add reduction to scalar.

UADDWB: Unsigned add wide (bottom).

UADDWT: Unsigned add wide (top).

UCLAMP: Unsigned clamp to minimum/maximum.

UCVTF (bottom, unpredicated): Unsigned integer convert to floating-point (bottom, unpredicated).

UCVTF (predicated): Unsigned integer convert to floating-point (predicated).

UCVTFLT: Unsigned integer convert to floating-point (top, unpredicated).

UDIV: Unsigned divide (predicated).

UDIVR: Unsigned reversed divide (predicated).

UDOT (2-way, indexed): Unsigned integer dot product by indexed element (two-way).

UDOT (2-way, vectors): Unsigned integer dot product (two-way).

UDOT (4-way, indexed): Unsigned integer dot product by indexed element (four-way).

UDOT (4-way, vectors): Unsigned integer dot product (four-way).

UHADD: Unsigned halving add.

UHSUB: Unsigned halving subtract.

UHSUBR: Unsigned halving subtract reversed.

UMAX (immediate): Unsigned maximum with immediate (unpredicated).

UMAX (vectors): Unsigned maximum (predicated).

UMAXP: Unsigned maximum pairwise.

UMAXQV: Unsigned maximum reduction of quadword vector segments.

UMAXV: Unsigned maximum reduction to scalar.

UMIN (immediate): Unsigned minimum with immediate (unpredicated).

UMIN (vectors): Unsigned minimum (predicated).

UMINP: Unsigned minimum pairwise.

UMINQV: Unsigned minimum reduction of quadword vector segments.

UMINV: Unsigned minimum reduction to scalar.

UMLALB (indexed): Unsigned multiply-add long by indexed element (bottom).

UMLALB (vectors): Unsigned multiply-add long (bottom).

UMLALT (indexed): Unsigned multiply-add long by indexed element (top).

UMLALT (vectors): Unsigned multiply-add long (top).

UMLSLB (indexed): Unsigned multiply-subtract long by indexed element (bottom).

UMLSLB (vectors): Unsigned multiply-subtract long (bottom).

UMLSLT (indexed): Unsigned multiply-subtract long by indexed element (top).

UMLSLT (vectors): Unsigned multiply-subtract long (top).

UMMLA: Unsigned 8-bit integer matrix multiply-accumulate to 32-bit integer.

UMULH (predicated): Unsigned multiply returning high half (predicated).

UMULH (unpredicated): Unsigned multiply returning high half (unpredicated).

UMULLB (indexed): Unsigned multiply long by indexed element (bottom).

UMULLB (vectors): Unsigned multiply long (bottom).

UMULLT (indexed): Unsigned multiply long by indexed element (top).

UMULLT (vectors): Unsigned multiply long (top).

UQADD (immediate): Unsigned saturating add immediate (unpredicated).

UQADD (vectors, predicated): Unsigned saturating add (predicated).

UQADD (vectors, unpredicated): Unsigned saturating add (unpredicated).

UQCVTN: Unsigned 32-bit integer saturating extract narrow to interleaved 16-bit integer.

UQDECB: Unsigned saturating decrement scalar by multiple of 8-bit predicate constraint element count.

UQDECD (scalar): Unsigned saturating decrement scalar by multiple of 64-bit predicate constraint element count.

UQDECD (vector): Unsigned saturating decrement vector by multiple of 64-bit predicate constraint element count.

UQDECH (scalar): Unsigned saturating decrement scalar by multiple of 16-bit predicate constraint element count.

UQDECH (vector): Unsigned saturating decrement vector by multiple of 16-bit predicate constraint element count.

UQDECP (scalar): Unsigned saturating decrement scalar by count of true predicate elements.

UQDECP (vector): Unsigned saturating decrement vector by count of true predicate elements.

UQDECW (scalar): Unsigned saturating decrement scalar by multiple of 32-bit predicate constraint element count.

UQDECW (vector): Unsigned saturating decrement vector by multiple of 32-bit predicate constraint element count.

UQINCB: Unsigned saturating increment scalar by multiple of 8-bit predicate constraint element count.

UQINCD (scalar): Unsigned saturating increment scalar by multiple of 64-bit predicate constraint element count.

UQINCD (vector): Unsigned saturating increment vector by multiple of 64-bit predicate constraint element count.

UQINCH (scalar): Unsigned saturating increment scalar by multiple of 16-bit predicate constraint element count.

UQINCH (vector): Unsigned saturating increment vector by multiple of 16-bit predicate constraint element count.

UQINCP (scalar): Unsigned saturating increment scalar by count of true predicate elements.

UQINCP (vector): Unsigned saturating increment vector by count of true predicate elements.

UQINCW (scalar): Unsigned saturating increment scalar by multiple of 32-bit predicate constraint element count.

UQINCW (vector): Unsigned saturating increment vector by multiple of 32-bit predicate constraint element count.

UQRSHL: Unsigned saturating rounding shift left (predicated).

UQRSHLR: Unsigned saturating rounding shift left reversed (predicated).

UQRSHRN: Unsigned saturating rounding shift right narrow by immediate to interleaved integer.

UQRSHRNB: Unsigned saturating rounding shift right narrow by immediate (bottom).

UQRSHRNT: Unsigned saturating rounding shift right narrow by immediate (top).

UQSHL (immediate): Unsigned saturating shift left by immediate.

UQSHL (vectors): Unsigned saturating shift left (predicated).

UQSHLR: Unsigned saturating shift left reversed (predicated).

UQSHRN: Unsigned saturating shift right narrow by immediate to interleaved integer.

UQSHRNB: Unsigned saturating shift right narrow by immediate (bottom).

UQSHRNT: Unsigned saturating shift right narrow by immediate (top).

UQSUB (immediate): Unsigned saturating subtract immediate (unpredicated).

UQSUB (vectors, predicated): Unsigned saturating subtract (predicated).

UQSUB (vectors, unpredicated): Unsigned saturating subtract (unpredicated).

UQSUBR: Unsigned saturating subtract reversed (predicated).

UQXTNB: Unsigned saturating extract narrow (bottom).

UQXTNT: Unsigned saturating extract narrow (top).

URECPE: Unsigned reciprocal estimate (predicated).

URHADD: Unsigned rounding halving add.

URSHL: Unsigned rounding shift left (predicated).

URSHLR: Unsigned rounding shift left reversed (predicated).

URSHR: Unsigned rounding shift right by immediate.

URSQRTE: Unsigned reciprocal square root estimate (predicated).

URSRA: Unsigned rounding shift right and accumulate (immediate).

USDOT (indexed): Unsigned by signed 8-bit integer dot product by indexed element to 32-bit integer.

USDOT (vectors): Unsigned by signed 8-bit integer dot product to 32-bit integer.

USHLLB: Unsigned shift left long by immediate (bottom).

USHLLT: Unsigned shift left long by immediate (top).

USMMLA: Unsigned by signed 8-bit integer matrix multiply-accumulate to 32-bit integer.

USQADD: Unsigned saturating signed add.

USRA: Unsigned shift right and accumulate (immediate).

USUBLB: Unsigned subtract long (bottom).

USUBLT: Unsigned subtract long (top).

USUBWB: Unsigned subtract wide (bottom).

USUBWT: Unsigned subtract wide (top).

UUNPKHI, UUNPKLO: Unsigned unpack and extend half of vector.

UXTB, UXTH, UXTW: Unsigned byte / halfword / word extend (predicated).

UZP1, UZP2 (predicates): Concatenate even or odd elements from two predicates.

UZP1, UZP2 (vectors): Concatenate even or odd elements from two vectors.

UZPQ1: Concatenate even elements within each pair of quadword vector segments.

UZPQ2: Concatenate odd elements within each pair of quadword vector segments.

WHILEGE (predicate as counter): While decrementing signed scalar greater than or equal to scalar (predicate-as-counter).

WHILEGE (predicate pair): While decrementing signed scalar greater than or equal to scalar (pair of predicates).

WHILEGE (predicate): While decrementing signed scalar greater than or equal to scalar.

WHILEGT (predicate as counter): While decrementing signed scalar greater than scalar (predicate-as-counter).

WHILEGT (predicate pair): While decrementing signed scalar greater than scalar (pair of predicates).

WHILEGT (predicate): While decrementing signed scalar greater than scalar.

WHILEHI (predicate as counter): While decrementing unsigned scalar higher than scalar (predicate-as-counter).

WHILEHI (predicate pair): While decrementing unsigned scalar higher than scalar (pair of predicates).

WHILEHI (predicate): While decrementing unsigned scalar higher than scalar.

WHILEHS (predicate as counter): While decrementing unsigned scalar higher than or same as scalar (predicate-as-counter).

WHILEHS (predicate pair): While decrementing unsigned scalar higher than or same as scalar (pair of predicates).

WHILEHS (predicate): While decrementing unsigned scalar higher than or same as scalar.

WHILELE (predicate as counter): While incrementing signed scalar less than or equal to scalar (predicate-as-counter).

WHILELE (predicate pair): While incrementing signed scalar less than or equal to scalar (pair of predicates).

WHILELE (predicate): While incrementing signed scalar less than or equal to scalar.

WHILELO (predicate as counter): While incrementing unsigned scalar lower than scalar (predicate-as-counter).

WHILELO (predicate pair): While incrementing unsigned scalar lower than scalar (pair of predicates).

WHILELO (predicate): While incrementing unsigned scalar lower than scalar.

WHILELS (predicate as counter): While incrementing unsigned scalar lower than or same as scalar (predicate-as-counter).

WHILELS (predicate pair): While incrementing unsigned scalar lower than or same as scalar (pair of predicates).

WHILELS (predicate): While incrementing unsigned scalar lower than or same as scalar.

WHILELT (predicate as counter): While incrementing signed scalar less than scalar (predicate-as-counter).

WHILELT (predicate pair): While incrementing signed scalar less than scalar (pair of predicates).

WHILELT (predicate): While incrementing signed scalar less than scalar.

WHILERW: While free of read-after-write conflicts.

WHILEWR: While free of write-after-read/write conflicts.

WRFFR: Write the first-fault register.

XAR: Bitwise exclusive-OR and rotate right by immediate.

ZIP1, ZIP2 (predicates): Interleave elements from two half predicates.

ZIP1, ZIP2 (vectors): Interleave elements from two half vectors.

ZIPQ1: Interleave elements from low halves of each pair of quadword vector segments.

ZIPQ2: Interleave elements from high halves of each pair of quadword vector segments.

2026-09_rel2026-06_diff_tag2026-09-212026-09-08 1409:2244:3801

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

BFCLAMP

BFloat16 clamp to minimum/maximum number

This instruction clamps each BFloat16 element in the destination vector to between the BFloat16 minimum value in the corresponding element of the first source vector and the BFloat16 maximum value in the corresponding element of the second source vector and destructively places the clamped results in the corresponding elements of the destination vector.

TheRegardless of the value of FPCR.AH, the behavior is as follows for each minimum number and maximum number operation:

- Negative zero compares less than positive zero.
- If one value is numeric and the other is a quiet NaN, the quiet NaN value is ignored and the result is the numeric value.
- When FPCR.DN is 0, if either value is a signaling NaN or if both values are NaNs, the result is a quiet NaN derived as follows:NaN.
  - If FPCR.AH is 0:
    - If at least one value is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN value.
    - Otherwise, the result is a quiet NaN derived from the first NaN value.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN value.
- When FPCR.DN is 1, if either value is a signaling NaN or if both values are NaNs, the result is default NaN.

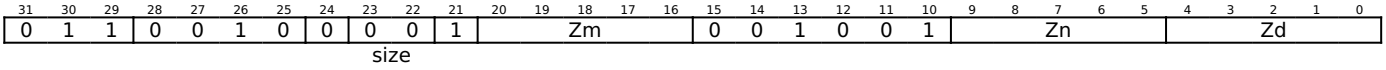
This instruction follows SVE2 non-widening BFloat16 numerical behaviors.

This instruction is unpredicated.

ID_AA64ZFR0_EL1.B16B16 indicates whether this instruction is implemented.

SVE2

(FEAT_SVE_B16B16)



Encoding

```
BFCLAMP <Zd>.H, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_B16B16) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
if IsFeatureImplemented(FEAT_SME2) then CheckSVEEnabled(); else CheckNonStreamingSVEEnabled(); end;
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 16;
var result : bits(VL);
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(d);

for e = 0 to elements-1 do
    let element1 : bits(16) = operand1[e*:16];
    let element2 : bits(16) = operand2[e*:16];
    let element3 : bits(16) = operand3[e*:16];
    let maxelement : bits(16) = BFMaxNum{}(element1, element3, FPCR());
    result[e*:16] = BFMinNum{16}(maxelement, element2, FPCR());
end;
Z{VL}(d) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

BFMAX

BFloat16 maximum (predicated)

This instruction determines the maximum of Active BFloat16 elements of the first source vector and the corresponding BFloat16 elements of the second source vector, and destructively places the results in the corresponding elements of the first source vector.
When FPCR.AH is 0, the behavior is as follows:

- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:~~NaN.~~
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is default NaN.

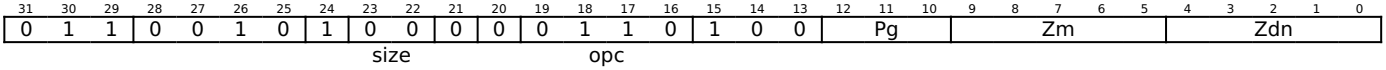
When FPCR.AH is 1, the behavior is as follows:

- If both elements are zeros, regardless of the sign of either zero, the result is the second element.
- If either element is a NaN, regardless of the value of FPCR.DN, the result is the second element.

Inactive elements in the destination vector register remain unmodified.
This instruction follows SVE2 non-widening BFloat16 numerical behaviors.
ID_AA64ZFR0_EL1.B16B16 indicates whether this instruction is implemented.

SVE2

(FEAT_SVE_B16B16)



Encoding

```
BFMAX <Zdn>.H, <Pg>/M, <Zdn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_B16B16) then EndOfDecode(Decode_UNDEF); end;

let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn>	Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
if IsFeatureImplemented(FEAT_SME2) then
    CheckSVEEnabled();
else
    CheckNonStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV 16;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, 16) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(16) = operand1[e*:16];
    if ActivePredicateElement{PL}(mask, e, 16) then
        let element2 : bits(16) = operand2[e*:16];
        result[e*:16] = BFMax{16}(element1, element2, FPCR());
    else
        result[e*:16] = element1;
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

BFMAXNM

BFloat16 maximum number (predicated)

This instruction determines the maximum number value of Active BFloat16 elements of the first source vector and the corresponding BFloat16 elements of the second source vector, and destructively places the results in the corresponding elements of the first source vector.

TheRegardless of the value of FPCR.AH, the behavior is as follows:

- Negative zero compares less than positive zero.
- If one element is numeric and the other is a quiet NaN, the quiet NaN element is ignored and the result is the numeric element. value.
- When FPCR.DN is 0, if either element is a signaling NaN or if both elements are NaNs, the result is a quiet NaN derived as follows:NaN.
  - If FPCR.AH is 0:
    - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
    - Otherwise, the result is a quiet NaN derived from the first NaN element.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a signaling NaN or if both elements are NaNs, the result is default NaN.

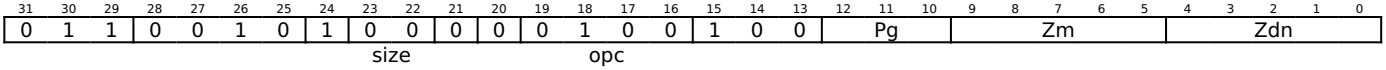
Inactive elements in the destination vector register remain unmodified.

This instruction follows SVE2 non-widening BFloat16 numerical behaviors.

ID_AA64ZFR0_EL1.B16B16 indicates whether this instruction is implemented.

SVE2

(FEAT_SVE_B16B16)



Encoding

```
BFMAXNM <Zdn>.H, <Pg>/M, <Zdn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_B16B16) then EndOfDecode(Decode_UNDEF); end;

let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn>	Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
if IsFeatureImplemented(FEAT_SME2) then
    CheckSVEEnabled();
else
    CheckNonStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV 16;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, 16) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(16) = operand1[e*:16];
    if ActivePredicateElement{PL}(mask, e, 16) then
        let element2 : bits(16) = operand2[e*:16];
        result[e*:16] = BFMaxNum{16}(element1, element2, FPCR());
    else
        result[e*:16] = element1;
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

BFMIN

BFloat16 minimum (predicated)

This instruction determines the minimum of Active BFloat16 elements of the first source vector and the corresponding BFloat16 elements of the second source vector, and destructively places the results in the corresponding elements of the first source vector.
When FPCR.AH is 0, the behavior is as follows:

- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:~~NaN.~~
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is default NaN.

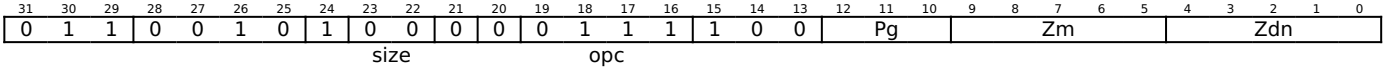
When FPCR.AH is 1, the behavior is as follows:

- If both elements are zeros, regardless of the sign of either zero, the result is the second element.
- If either element is a NaN, regardless of the value of FPCR.DN, the result is the second element.

Inactive elements in the destination vector register remain unmodified.
This instruction follows SVE2 non-widening BFloat16 numerical behaviors.
ID_AA64ZFR0_EL1.B16B16 indicates whether this instruction is implemented.

SVE2

(FEAT_SVE_B16B16)



Encoding

```
BFMIN <Zdn>.H, <Pg>/M, <Zdn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_B16B16) then EndOfDecode(Decode_UNDEF); end;

let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn>	Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
if IsFeatureImplemented(FEAT_SME2) then
    CheckSVEEnabled();
else
    CheckNonStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV 16;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, 16) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(16) = operand1[e*:16];
    if ActivePredicateElement{PL}(mask, e, 16) then
        let element2 : bits(16) = operand2[e*:16];
        result[e*:16] = BFMin{16}(element1, element2, FPCR());
    else
        result[e*:16] = element1;
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

BFMINNM

BFloat16 minimum number (predicated)

This instruction determines the minimum number value of Active BFloat16 elements of the first source vector and the corresponding BFloat16 elements of the second source vector, and destructively places the results in the corresponding elements of the first source vector.

TheRegardless of the value of FPCR.AH, the behavior is as follows:

- Negative zero compares less than positive zero.
- If one element is numeric and the other is a quiet NaN, the quiet NaN element is ignored and the result is the numeric element. value.
- When FPCR.DN is 0, if either element is a signaling NaN or if both elements are NaNs, the result is a quiet NaN derived as follows:NaN.
  - If FPCR.AH is 0:
    - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
    - Otherwise, the result is a quiet NaN derived from the first NaN element.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a signaling NaN or if both elements are NaNs, the result is default NaN.

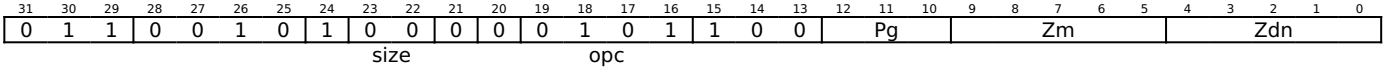
Inactive elements in the destination vector register remain unmodified.

This instruction follows SVE2 non-widening BFloat16 numerical behaviors.

ID_AA64ZFR0_EL1.B16B16 indicates whether this instruction is implemented.

SVE2

(FEAT_SVE_B16B16)



Encoding

```
BFMINNM <Zdn>.H, <Pg>/M, <Zdn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_B16B16) then EndOfDecode(Decode_UNDEF); end;

let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn>	Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
if IsFeatureImplemented(FEAT_SME2) then
    CheckSVEEnabled();
else
    CheckNonStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV 16;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, 16) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(16) = operand1[e*:16];
    if ActivePredicateElement{PL}(mask, e, 16) then
        let element2 : bits(16) = operand2[e*:16];
        result[e*:16] = BFMinNum{16}(element1, element2, FPCR());
    else
        result[e*:16] = element1;
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

CPY (scalar)

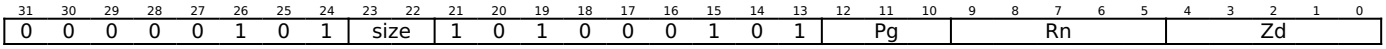
Copy general-purpose register to vector elements (predicated)

This instruction copies the general-purpose scalar source register to each Active element in the destination vector. Inactive elements in the destination vector register remain unmodified.

This instruction is used by the alias MOV (scalar, predicated).

SVE

(FEAT_SVE || FEAT_SME)



Encoding for the 32-bit variant

Applies when (size != 11)

```
CPY <Zd>.<T>, <Pg>/M, <n|SP><Wn|WSP><R>
```

~~Decode for this encoding~~

Encoding for the 64-bit variant

Applies when (size == 11)

```
CPY <Zd>.D, <Pg>/M, <Xn|SP>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Rn);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
~~11~~	~~D~~

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Wn|WSP~~R~~>

~~size~~	~~<R>~~
~~00~~	~~W~~
~~01~~	~~W~~
~~10~~	~~W~~
~~11~~	~~X~~

Is the 32-bit name of the source general-purpose register or stack pointer, encoded in the "Rn" field.~~Is a width specifier, encoded in~~ “~~size~~”~~.~~

<Xn~~n~~|SP> Is the 64-bit~~number~~ name~~[0-30]~~ of the ~~general-purpose~~ source general-purpose register or stack~~the~~ pointer,~~name SP (31),~~ encoded in the "Rn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var result : bits(VL) = Z{}(d);
if AnyActiveElement{PL}(mask, esize) then
    var operand1 : bits(64);
    if n == 31 then
        operand1 = SP{64}();
    else
        operand1 = X{64}(n);
    end;

    for e = 0 to elements-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            result[e*:esize] = operand1[esize-1:0];
        end;
    end;
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

DUP (scalar)

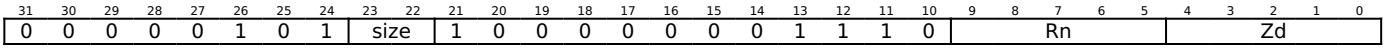
Broadcast general-purpose register to vector elements (unpredicated)

This instruction unconditionally broadcasts the general-purpose scalar source register into each element of the destination vector. This instruction is unpredicated.

This instruction is used by the alias MOV (scalar, unpredicated).

SVE

(FEAT_SVE || FEAT_SME)



Encoding for the 32-bit variant

Applies when (size != 11)

```
DUP <Zd>.<T>, <n|SP><Wn|WSP><R>
```

~~Decode for this encoding~~

Encoding for the 64-bit variant

Applies when (size == 11)

```
DUP <Zd>.D, <Xn|SP>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Rn);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
~~11~~	~~D~~

<Wn|WSP~~R~~>

~~size~~	~~<R>~~
~~00~~	~~W~~
~~01~~	~~W~~
~~10~~	~~W~~
~~11~~	~~X~~

Is the 32-bit name of the source general-purpose register or stack pointer, encoded in the "Rn" field.~~Is a width specifier, encoded in “size”.~~

<Xn~~n~~|SP> Is the 64-bit~~number~~ name~~[0-30]~~ of the ~~general-purpose~~ source general-purpose register or stack~~the~~ pointer,~~name SP (31),~~ encoded in the "Rn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var operand : bits(64);
if n == 31 then
    operand = SP{64}();
else
    operand = X{64}(n);
end;
var result : bits(VL);

for e = 0 to elements-1 do
    result[e*:esize] = operand[esize-1:0];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

FAMAX

Floating-point absolute maximum (predicated)

This instruction determines the maximum absolute value from Active floating-point elements of the first source vector and the corresponding floating-point elements of the second source vector, and destructively places the results in the corresponding elements of the first source vector.

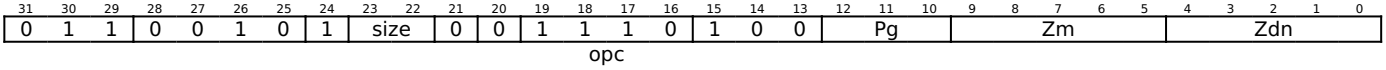
TheRegardless of the value of FPCR.AH, the behavior is as follows:

- AlternateWhen floating-pointFPCR.DN behaviorsis do0, notif apply,either aselement ifis FPCR.AHa NaN, the result is 0.a quiet NaN.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is the default NaN, with the sign bit set to 0.
- Denormalized inputs and results are never flushed to zero, as if FPCR.{FZ, FZ16, FIZ} are all 0.
- Denormalized inputs never generate an Input Denormal floating-point exception.

Inactive elements in the destination vector register remain unmodified.

SVE2

((FEAT_SVE2 || FEAT_SME2) && FEAT_FAMINMAX)



Encoding

```
FAMAX <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if ((!IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME2)) ||
      !IsFeatureImplemented(FEAT_FAMINMAX)) then EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);

let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
if IsFeatureImplemented(FEAT_SME2) then
    CheckSVEEnabled();
else
    CheckNonStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element2 : bits(esize) = operand2[e*:esize];
        result[e*:esize] = FPAbsMax{esize}(element1, element2, FPCR());
    else
        result[e*:esize] = element1;
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

FAMIN

Floating-point absolute minimum (predicated)

This instruction determines the minimum absolute value from Active floating-point elements of the first source vector and the corresponding floating-point elements of the second source vector, and destructively places the results in the corresponding elements of the first source vector.

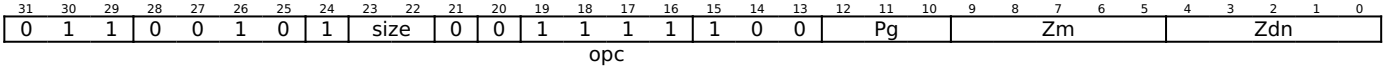
TheRegardless of the value of FPCR.AH, the behavior is as follows:

- AlternateWhen floating-pointFPCR.DN behaviorsis do0, notif apply,either aselement ifis FPCR.AHa NaN, the result is 0.a quiet NaN.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is the default NaN, with the sign bit set to 0.
- Denormalized inputs and results are never flushed to zero, as if FPCR.{FZ, FZ16, FIZ} are all 0.
- Denormalized inputs never generate an Input Denormal floating-point exception.

Inactive elements in the destination vector register remain unmodified.

SVE2

((FEAT_SVE2 || FEAT_SME2) && FEAT_FAMINMAX)



Encoding

```
FAMIN <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if ((!IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME2)) ||
      !IsFeatureImplemented(FEAT_FAMINMAX)) then EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);

let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
if IsFeatureImplemented(FEAT_SME2) then
    CheckSVEEnabled();
else
    CheckNonStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element2 : bits(esize) = operand2[e*:esize];
        result[e*:esize] = FPAbsMin{esize}(element1, element2, FPCR());
    else
        result[e*:esize] = element1;
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

FCLAMP

Floating-point clamp to minimum/maximum number

This instruction clamps each floating-point element in the destination vector to between the floating-point minimum value in the corresponding element of the first source vector and the floating-point maximum value in the corresponding element of the second source vector and destructively places the clamped results in the corresponding elements of the destination vector.

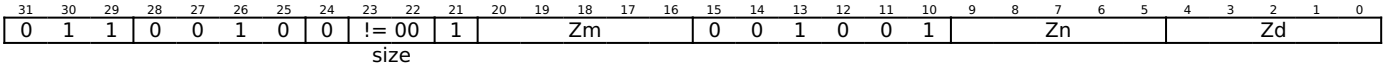
TheRegardless of the value of FPCR.AH, the behavior is as follows for each minimum number and maximum number operation:

- Negative zero compares less than positive zero.
- If one value is numeric and the other is a quiet NaN, the quiet NaN value is ignored and the result is the numeric value.
- When FPCR.DN is 0, if either value is a signaling NaN or if both values are NaNs, the result is a quiet NaN derived as follows:NaN.
  - If FPCR.AH is 0:
    - If at least one value is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN value.
    - Otherwise, the result is a quiet NaN derived from the first NaN value.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN value.
- When FPCR.DN is 1, if either value is a signaling NaN or if both values are NaNs, the result is default NaN.

This instruction is unpredicated.

SVE2

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
FCLAMP <Zd>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var result : bits(VL);
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(d);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    let element2 : bits(esize) = operand2[e*:esize];
    let element3 : bits(esize) = operand3[e*:esize];
    let maxelement : bits(esize) = FPMaxNum{}(element1, element3, FPCR());
    result[e*:esize] = FPMinNum{esize}(maxelement, element2, FPCR());
end;
Z{VL}(d) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

FMAX (immediate)

Floating-point maximum with immediate (predicated)

This instruction determines the maximum of an immediate and each Active floating-point element of the source vector, and destructively places the results in the corresponding elements of the source vector. The immediate may take the value +0.0 or +1.0 only.
When FPCR.AH is 0, the behavior is as follows:

- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if the element is a NaN, the result is a quiet NaN derived from it.~~NaN.~~
- When FPCR.DN is 1, if the element is a NaN, the result is default NaN.

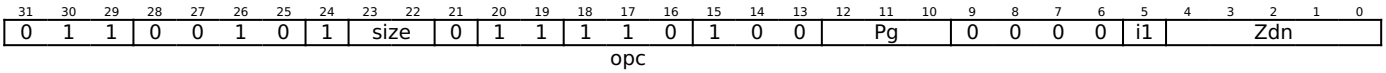
When FPCR.AH is 1, the behavior is as follows:

- If both the element and the immediate are zeros, regardless of the sign of either zero, the result is the immediate.
- If the element is a NaN, regardless of the value of FPCR.DN, the result is the immediate.

Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FMAX <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let imm : bits(esize) = if i1 == '0' then Zeros{esize} else FPOne{esize}('0');
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<const> Is the floating-point immediate value, encoded in “i1”:

i1	<const>
0	#0.0
1	#1.0

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = FPMax{esize}(element1, imm, FPCR());
    else
        result[e*:esize] = element1;
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:2244:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

FMAX (vectors)

Floating-point maximum (predicated)

This instruction determines the maximum of Active floating-point elements of the first source vector and the corresponding floating-point elements of the second source vector, and destructively places the results in the corresponding elements of the first source vector.
When FPCR.AH is 0, the behavior is as follows:

- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:~~NaN.~~
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is default NaN.

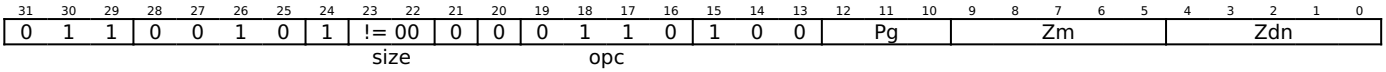
When FPCR.AH is 1, the behavior is as follows:

- If both elements are zeros, regardless of the sign of either zero, the result is the second element.
- If either element is a NaN, regardless of the value of FPCR.DN, the result is the second element.

Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FMAX <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);

let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element2 : bits(esize) = operand2[e*:esize];
        result[e*:esize] = FPMax{esize}(element1, element2, FPCR());
    else
        result[e*:esize] = element1;
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

FMAXNM (immediate)

Floating-point maximum number with immediate (predicated)

This instruction determines the maximum number value of an immediate and each Active floating-point element of the source vector, and destructively places the results in the corresponding elements of the source vector. The immediate may take the value +0.0 or +1.0 only.

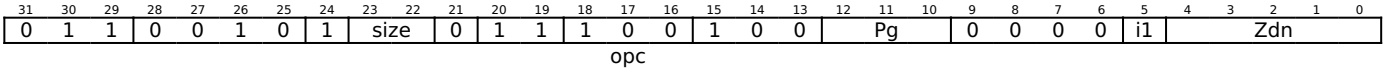
TheRegardless of the value of FPCR.AH, the behavior is as follows:

- Negative zero compares less than positive zero.
- If the element is a quiet NaN, the element is ignored and the result is the immediate value.
- When FPCR.DN is 0, if the element is a signaling NaN, the result is a quiet NaN derived from it.NaN.
- When FPCR.DN is 1, if the element is a signaling NaN, the result is default NaN.

Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FMAXNM <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let imm : bits(esize) = if i1 == '0' then Zeros{esize} else FPOne{esize}('0');
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<const> Is the floating-point immediate value, encoded in “i1”:

i1	<const>
0	#0.0
1	#1.0

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = FPMaxNum{esize}(element1, imm, FPCR());
    else
        result[e*:esize] = element1;
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

FMAXNM (vectors)

Floating-point maximum number (predicated)

This instruction determines the maximum number value of Active floating-point elements of the first source vector and the corresponding floating-point elements of the second source vector, and destructively places the results in the corresponding elements of the first source vector.

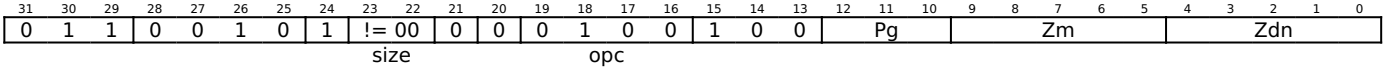
TheRegardless of the value of FPCR.AH, the behavior is as follows:

- Negative zero compares less than positive zero.
- If one element is numeric and the other is a quiet NaN, the quiet NaN element is ignored and the result is the numeric element. value.
- When FPCR.DN is 0, if either element is a signaling NaN or if both elements are NaNs, the result is a quiet NaN derived as follows:NaN.
  - If FPCR.AH is 0:
    - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
    - Otherwise, the result is a quiet NaN derived from the first NaN element.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a signaling NaN or if both elements are NaNs, the result is default NaN.

Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FMAXNM <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);

let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element2 : bits(esize) = operand2[e*:esize];
        result[e*:esize] = FPMaxNum{esize}(element1, element2, FPCR());
    else
        result[e*:esize] = element1;
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

FMAXNMP

Floating-point maximum number pairwise

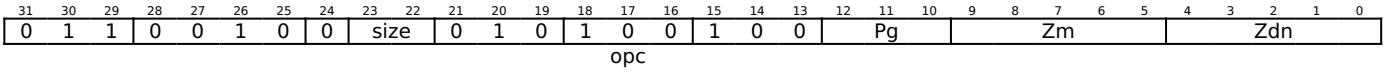
This instruction calculates the maximum value of each pair of adjacent floating-point elements within each source vector, and interleaves the results from corresponding lanes. The interleaved result values are destructively placed in the first source vector.

TheRegardless of the value of FPCR.AH, the behavior is as follows for each pairwise operation:

- Negative zero compares less than positive zero.
- If one element is numeric and the other is a quiet NaN, the quiet NaN element is ignored and the result is the numeric element. value.
- When FPCR.DN is 0, if either element is a signaling NaN or if both elements are NaNs, the result is a quiet NaN derived as follows:NaN.
  - If FPCR.AH is 0:
    - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
    - Otherwise, the result is a quiet NaN derived from the first NaN element.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a signaling NaN or if both elements are NaNs, the result is default NaN.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
FMAXNMP <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);

let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL) = Z{}(dn);
var element1 : bits(esize);
var element2 : bits(esize);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        if IsEven(e) then
            element1 = operand1[(e + 0)*:esize];
            element2 = operand1[(e + 1)*:esize];
        else
            element1 = operand2[(e - 1)*:esize];
            element2 = operand2[(e + 0)*:esize];
        end;
        result[e*:esize] = FPMaxNum{esize}(element1, element2, FPCR());
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

FMAXNMQV

Floating-point maximum number recursive reduction of quadword vector segments

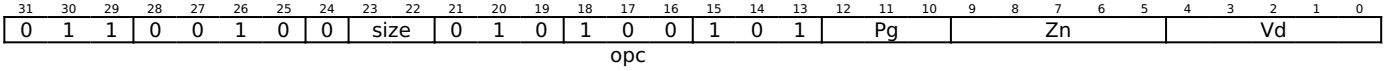
This instruction calculates a floating-point maximum number of the same element numbers from each 128-bit source vector segment using a recursive pairwise reduction, and places each result into the corresponding element number of the 128-bit SIMD&FP destination register. Inactive elements in the source vector are treated as the default NaN.

TheRegardless of the value of FPCR.AH, the behavior is as follows:

- Negative zero compares less than positive zero.
- If one value is numeric and the other is a quiet NaN, the quiet NaN value is ignored and the result is the numeric value.
- When FPCR.DN is 0, if either value is a signaling NaN or if both values are NaNs, the result is a quiet NaN derived as follows:NaN.
  - If FPCR.AH is 0:
    - If at least one value is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN value.
    - Otherwise, the result is a quiet NaN derived from the first NaN value.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN value.
- When FPCR.DN is 1, if either value is a signaling NaN or if both values are NaNs, the result is default NaN.

SVE2

(FEAT_SVE2p1 || FEAT_SME2p1)



Encoding

```
FMAXNMQV <Vd>.<T>, <Pg>, <Zn>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) && !IsFeatureImplemented(FEAT_SME2p1) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Vd);
```

Assembler Symbols

<Vd> Is the name of the destination SIMD&FP register, encoded in the "Vd" field.

<T> Is an arrangement specifier, encoded in “size”:

size	<T>
00	RESERVED
01	8H
10	4S
11	2D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	H
10	S
11	D

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let segments : integer{} = VL DIV 128;
let elempersegment : integer = 128 DIV esize;
let segbits : integer{} = segments*esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
let identity : bits(esize) = FPDefaultNaN{esize}(FPCR());
var result : bits(128) = Zeros{};

for e = 0 to elempersegment-1 do
    var stmp : bits(segbits);
    for s = 0 to segments-1 do
        if ActivePredicateElement{PL}(mask, s * elempersegment + e, esize) then
            stmp[s*:esize] = operand[(s * elempersegment + e)*:esize];
        else
            stmp[s*:esize] = identity;
        end;
    end;
    result[e*:esize] = FPReduce{esize, segbits}(ReduceOp_FMAXNUM, stmp, FPCR());
end;
V{128}(d) = result;
```

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

FMAXNMV

Floating-point maximum number recursive reduction to scalar

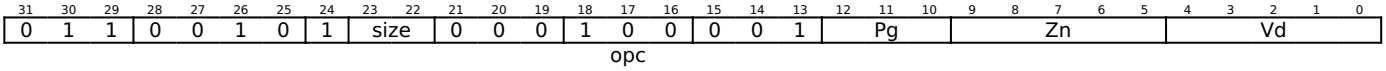
This instruction calculates a floating-point maximum number horizontally over all lanes of a vector using a recursive pairwise reduction, and places the result in the SIMD&FP scalar destination register. Inactive elements in the source vector are treated as the default NaN.

TheRegardless of the value of FPCR.AH, the behavior is as follows:

- Negative zero compares less than positive zero.
- If one value is numeric and the other is a quiet NaN, the quiet NaN value is ignored and the result is the numeric value.
- When FPCR.DN is 0, if either value is a signaling NaN or if both values are NaNs, the result is a quiet NaN derived as follows:NaN.
  - If FPCR.AH is 0:
    - If at least one value is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN value.
    - Otherwise, the result is a quiet NaN derived from the first NaN value.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN value.
- When FPCR.DN is 1, if either value is a signaling NaN or if both values are NaNs, the result is default NaN.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FMAXNMV <V><d>, <Pg>, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Vd);
```

Assembler Symbols

<V> Is a width specifier, encoded in “size”:

size	<V>
00	RESERVED
01	H
10	S
11	D

<d> Is the number [0-31] of the destination SIMD&FP register, encoded in the "Vd" field.

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
let identity : bits(esize) = FPDefaultNaN{esize}(FPCR());

V{esize}(d) = FPReducePredicated{esize, VL, PL}(ReduceOp_FMAXNUM, operand, mask,
                                                identity, FPCR());
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

FMAXP

Floating-point maximum pairwise

This instruction calculates the maximum value of each pair of adjacent floating-point elements within each source vector, and interleaves the results from corresponding lanes. The interleaved result values are destructively placed in the first source vector.
When FPCR.AH is 0, the behavior is as follows for each pairwise operation:

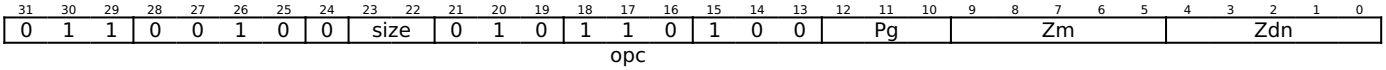
- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:~~NaN.~~
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows for each pairwise operation:

- If both elements are zeros, regardless of the sign of either zero, the result is the second element.
- If either element is a NaN, regardless of the value of FPCR.DN, the result is the second element.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
FMAXP <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);

let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL) = Z{}(dn);
var element1 : bits(esize);
var element2 : bits(esize);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        if IsEven(e) then
            element1 = operand1[(e + 0)*:esize];
            element2 = operand1[(e + 1)*:esize];
        else
            element1 = operand2[(e - 1)*:esize];
            element2 = operand2[(e + 0)*:esize];
        end;
        result[e*:esize] = FPMax{esize}(element1, element2, FPCR());
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

FMAXQV

Floating-point maximum reduction of quadword vector segments

This instruction calculates a floating-point maximum of the same element numbers from each 128-bit source vector segment using a recursive pairwise reduction, and places each result into the corresponding element number of the 128-bit SIMD&FP destination register. Inactive elements in the source vector are treated as -Infinity.
When FPCR.AH is 0, the behavior is as follows:

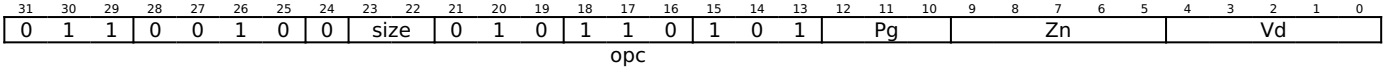
- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either value is a NaN, the result is a quiet NaN derived as follows:~~NaN.~~
  - If at least one value is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN value.
  - Otherwise, the result is a quiet NaN derived from the first NaN value.
- When FPCR.DN is 1, if either value is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows:

- If both values are zeros, regardless of the sign of either zero, the result is the second value.
- If either value is a NaN, regardless of the value of FPCR.DN, the result is the second value.

SVE2

(FEAT_SVE2p1 || FEAT_SME2p1)



Encoding

```
FMAXQV <Vd>.<T>, <Pg>, <Zn>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) && !IsFeatureImplemented(FEAT_SME2p1) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Vd);
```

Assembler Symbols

<Vd> Is the name of the destination SIMD&FP register, encoded in the "Vd" field.

<T> Is an arrangement specifier, encoded in “size”:

size	<T>
00	RESERVED
01	8H
10	4S
11	2D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	H
10	S
11	D

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let segments : integer{} = VL DIV 128;
let elempersegment : integer = 128 DIV esize;
let segbits : integer{} = segments*esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
let identity : bits(esize) = FPInfinity{esize}('1');
var result : bits(128) = Zeros{};

for e = 0 to elempersegment-1 do
    var stmp : bits(segbits);
    for s = 0 to segments-1 do
        if ActivePredicateElement{PL}(mask, s * elempersegment + e, esize) then
            stmp[s*:esize] = operand[(s * elempersegment + e)*:esize];
        else
            stmp[s*:esize] = identity;
        end;
    end;
    result[e*:esize] = FPReduce{esize, segbits}(ReduceOp_FMAX, stmp, FPCR());
end;
V{128}(d) = result;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

FMAXV

Floating-point maximum recursive reduction to scalar

This instruction calculates a floating-point maximum horizontally over all lanes of a vector using a recursive pairwise reduction, and places the result in the SIMD&FP scalar destination register. Inactive elements in the source vector are treated as -Infinity.

When FPCR.AH is 0, the behavior is as follows:

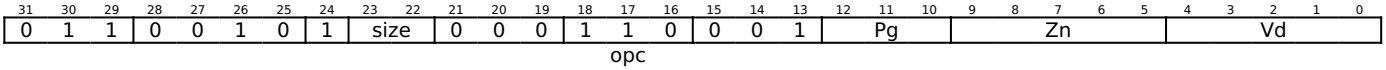
- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either value is a NaN, the result is a quiet NaN derived as follows:~~NaN.~~
  - If at least one value is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN value.
  - Otherwise, the result is a quiet NaN derived from the first NaN value.
- When FPCR.DN is 1, if either value is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows:

- If both values are zeros, regardless of the sign of either zero, the result is the second value.
- If either value is a NaN, regardless of the value of FPCR.DN, the result is the second value.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FMAXV <V><d>, <Pg>, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Vd);
```

Assembler Symbols

<V> Is a width specifier, encoded in “size”:

size	<V>
00	RESERVED
01	H
10	S
11	D

<d> Is the number [0-31] of the destination SIMD&FP register, encoded in the "Vd" field.

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
let identity : bits(esize) = FPInfinity{esize}('1');

V{esize}(d) = FPReducePredicated{esize, VL, PL}(ReduceOp_FMAX, operand, mask,
                                                identity, FPCR());
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

FMIN (immediate)

Floating-point minimum with immediate (predicated)

This instruction determines the minimum of an immediate and each Active floating-point element of the source vector, and destructively places the results in the corresponding elements of the source vector. The immediate may take the value +0.0 or +1.0 only.
When FPCR.AH is 0, the behavior is as follows:

- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if the element is a NaN, the result is a quiet NaN derived from it.~~NaN.~~
- When FPCR.DN is 1, if the element is a NaN, the result is default NaN.

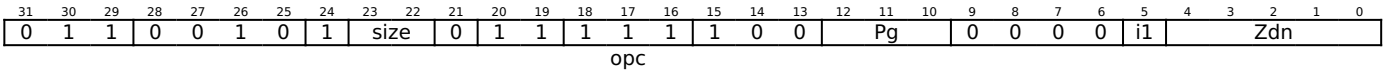
When FPCR.AH is 1, the behavior is as follows:

- If both the element and the immediate are zeros, regardless of the sign of either zero, the result is the immediate.
- If the element is a NaN, regardless of the value of FPCR.DN, the result is the immediate.

Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FMIN <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let imm : bits(esize) = if i1 == '0' then Zeros{esize} else FPOne{esize}('0');
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<const> Is the floating-point immediate value, encoded in “i1”:

i1	<const>
0	#0.0
1	#1.0

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = FPMin{esize}(element1, imm, FPCR());
    else
        result[e*:esize] = element1;
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

FMIN (vectors)

Floating-point minimum (predicated)

This instruction determines the minimum of Active floating-point elements of the first source vector and the corresponding floating-point elements of the second source vector, and destructively places the results in the corresponding elements of the first source vector.
When FPCR.AH is 0, the behavior is as follows:

- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:~~NaN.~~
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is default NaN.

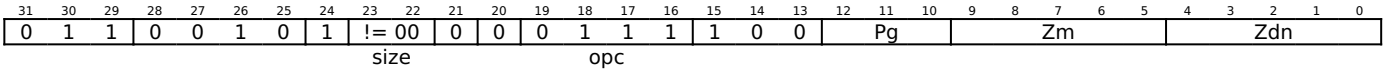
When FPCR.AH is 1, the behavior is as follows:

- If both elements are zeros, regardless of the sign of either zero, the result is the second element.
- If either element is a NaN, regardless of the value of FPCR.DN, the result is the second element.

Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FMIN <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);

let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element2 : bits(esize) = operand2[e*:esize];
        result[e*:esize] = FPMin{esize}(element1, element2, FPCR());
    else
        result[e*:esize] = element1;
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

FMINNM (immediate)

Floating-point minimum number with immediate (predicated)

This instruction determines the minimum number value of an immediate and each Active floating-point element of the source vector, and destructively places the results in the corresponding elements of the source vector. The immediate may take the value +0.0 or +1.0 only.

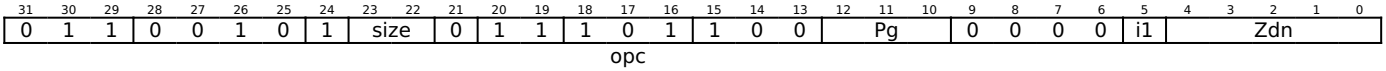
TheRegardless of the value of FPCR.AH, the behavior is as follows:

- Negative zero compares less than positive zero.
- If the element is a quiet NaN, the element is ignored and the result is the immediate value.
- When FPCR.DN is 0, if the element is a signaling NaN, the result is a quiet NaN derived from it.NaN.
- When FPCR.DN is 1, if the element is a signaling NaN, the result is default NaN.

Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FMINNM <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let imm : bits(esize) = if i1 == '0' then Zeros{esize} else FPOne{esize}('0');
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<const> Is the floating-point immediate value, encoded in “i1”:

i1	<const>
0	#0.0
1	#1.0

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = FPMinNum{esize}(element1, imm, FPCR());
    else
        result[e*:esize] = element1;
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

FMINNM (vectors)

Floating-point minimum number (predicated)

This instruction determines the minimum number value of Active floating-point elements of the first source vector and the corresponding floating-point elements of the second source vector, and destructively places the results in the corresponding elements of the first source vector.

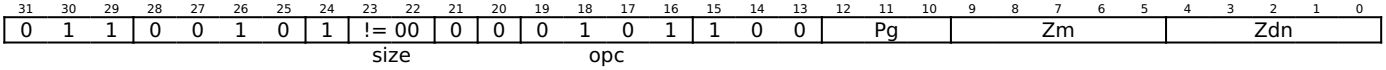
TheRegardless of the value of FPCR.AH, the behavior is as follows:

- Negative zero compares less than positive zero.
- If one element is numeric and the other is a quiet NaN, the quiet NaN element is ignored and the result is the numeric element. value.
- When FPCR.DN is 0, if either element is a signaling NaN or if both elements are NaNs, the result is a quiet NaN derived as follows:NaN.
  - If FPCR.AH is 0:
    - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
    - Otherwise, the result is a quiet NaN derived from the first NaN element.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a signaling NaN or if both elements are NaNs, the result is default NaN.

Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FMINNM <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);

let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element2 : bits(esize) = operand2[e*:esize];
        result[e*:esize] = FPMinNum{esize}(element1, element2, FPCR());
    else
        result[e*:esize] = element1;
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

FMINNMP

Floating-point minimum number pairwise

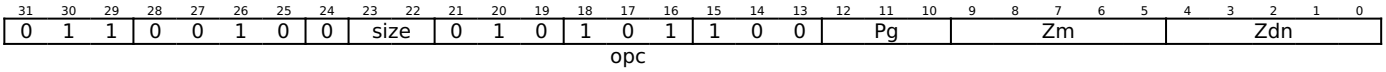
This instruction calculates the minimum value of each pair of adjacent floating-point elements within each source vector, and interleaves the results from corresponding lanes. The interleaved result values are destructively placed in the first source vector.

The~~Regardless of the value of FPCR.AH, the~~ behavior is as follows for each pairwise operation:

- Negative zero compares less than positive zero.
- If one element is numeric and the other is a quiet NaN, the quiet NaN element is ignored and the result is the numeric element.~~ value.~~
- When FPCR.DN is 0, if either element is a signaling NaN or if both elements are NaNs, the result is a quiet NaN derived as follows:~~NaN.~~
  - If FPCR.AH is 0:
    - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
    - Otherwise, the result is a quiet NaN derived from the first NaN element.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a signaling NaN or if both elements are NaNs, the result is default NaN.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
FMINNMP <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);

let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL) = Z{}(dn);
var element1 : bits(esize);
var element2 : bits(esize);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        if IsEven(e) then
            element1 = operand1[(e + 0)*:esize];
            element2 = operand1[(e + 1)*:esize];
        else
            element1 = operand2[(e - 1)*:esize];
            element2 = operand2[(e + 0)*:esize];
        end;
        result[e*:esize] = FPMinNum{esize}(element1, element2, FPCR());
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

FMINNMQV

Floating-point minimum number recursive reduction of quadword vector segments

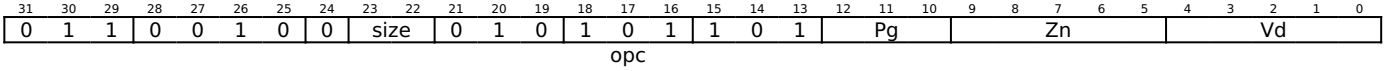
This instruction calculates a floating-point minimum number of the same element numbers from each 128-bit source vector segment using a recursive pairwise reduction, and places each result into the corresponding element number of the 128-bit SIMD&FP destination register. Inactive elements in the source vector are treated as the default NaN.

TheRegardless of the value of FPCR.AH, the behavior is as follows:

- Negative zero compares less than positive zero.
- If one value is numeric and the other is a quiet NaN, the quiet NaN value is ignored and the result is the numeric value.
- When FPCR.DN is 0, if either value is a signaling NaN or if both values are NaNs, the result is a quiet NaN derived as follows:NaN.
  - If FPCR.AH is 0:
    - If at least one value is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN value.
    - Otherwise, the result is a quiet NaN derived from the first NaN value.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN value.
- When FPCR.DN is 1, if either value is a signaling NaN or if both values are NaNs, the result is default NaN.

SVE2

(FEAT_SVE2p1 || FEAT_SME2p1)



Encoding

```
FMINNMQV <Vd>.<T>, <Pg>, <Zn>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) && !IsFeatureImplemented(FEAT_SME2p1) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Vd);
```

Assembler Symbols

<Vd> Is the name of the destination SIMD&FP register, encoded in the "Vd" field.

<T> Is an arrangement specifier, encoded in “size”:

size	<T>
00	RESERVED
01	8H
10	4S
11	2D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	H
10	S
11	D

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let segments : integer{} = VL DIV 128;
let elempersegment : integer = 128 DIV esize;
let segbits : integer{} = segments*esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
let identity : bits(esize) = FPDefaultNaN{esize}(FPCR());
var result : bits(128) = Zeros{};

for e = 0 to elempersegment-1 do
    var stmp : bits(segbits);
    for s = 0 to segments-1 do
        if ActivePredicateElement{PL}(mask, s * elempersegment + e, esize) then
            stmp[s*:esize] = operand[(s * elempersegment + e)*:esize];
        else
            stmp[s*:esize] = identity;
        end;
    end;
    result[e*:esize] = FPReduce{esize, segbits}(ReduceOp_FMINNUM, stmp, FPCR());
end;
V{128}(d) = result;
```

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

FMINNMV

Floating-point minimum number recursive reduction to scalar

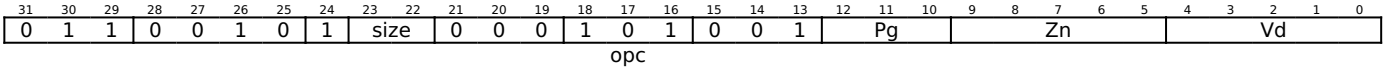
This instruction calculates a floating-point minimum number horizontally over all lanes of a vector using a recursive pairwise reduction, and places the result in the SIMD&FP scalar destination register. Inactive elements in the source vector are treated as the default NaN.

TheRegardless of the value of FPCR.AH, the behavior is as follows:

- Negative zero compares less than positive zero.
- If one value is numeric and the other is a quiet NaN, the quiet NaN value is ignored and the result is the numeric value.
- When FPCR.DN is 0, if either value is a signaling NaN or if both values are NaNs, the result is a quiet NaN derived as follows:NaN.
  - If FPCR.AH is 0:
    - If at least one value is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN value.
    - Otherwise, the result is a quiet NaN derived from the first NaN value.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN value.
- When FPCR.DN is 1, if either value is a signaling NaN or if both values are NaNs, the result is default NaN.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FMINNMV <V><d>, <Pg>, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Vd);
```

Assembler Symbols

<V> Is a width specifier, encoded in “size”:

size	<V>
00	RESERVED
01	H
10	S
11	D

<d> Is the number [0-31] of the destination SIMD&FP register, encoded in the "Vd" field.

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
let identity : bits(esize) = FPDefaultNaN{esize}(FPCR());

V{esize}(d) = FPReducePredicated{esize, VL, PL}(ReduceOp_FMINNUM, operand, mask,
                                                identity, FPCR());
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

FMINP

Floating-point minimum pairwise

This instruction calculates the minimum value of each pair of adjacent floating-point elements within each source vector, and interleaves the results from corresponding lanes. The interleaved result values are destructively placed in the first source vector.

When FPCR.AH is 0, the behavior is as follows for each pairwise operation:

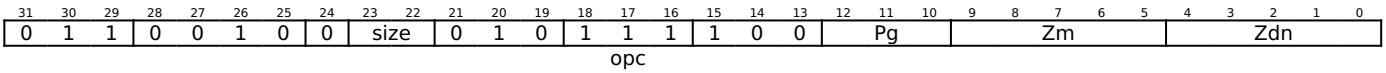
- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:~~NaN.~~
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows for each pairwise operation:

- If both elements are zeros, regardless of the sign of either zero, the result is the second element.
- If either element is a NaN, regardless of the value of FPCR.DN, the result is the second element.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
FMINP <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);

let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL) = Z{}(dn);
var element1 : bits(esize);
var element2 : bits(esize);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        if IsEven(e) then
            element1 = operand1[(e + 0)*:esize];
            element2 = operand1[(e + 1)*:esize];
        else
            element1 = operand2[(e - 1)*:esize];
            element2 = operand2[(e + 0)*:esize];
        end;
        result[e*:esize] = FPMin{esize}(element1, element2, FPCR());
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

FMINQV

Floating-point minimum recursive reduction of quadword vector segments

This instruction calculates a floating-point minimum of the same element numbers from each 128-bit source vector segment using a recursive pairwise reduction, and places each result into the corresponding element number of the 128-bit SIMD&FP destination register. Inactive elements in the source vector are treated as +Infinity.
When FPCR.AH is 0, the behavior is as follows:

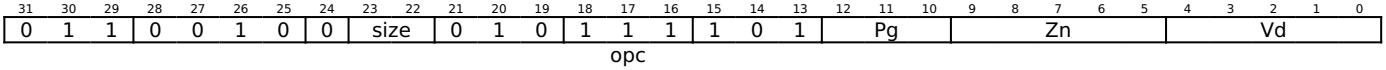
- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either value is a NaN, the result is a quiet NaN derived as follows:~~NaN.~~
  - If at least one value is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN value.
  - Otherwise, the result is a quiet NaN derived from the first NaN value.
- When FPCR.DN is 1, if either value is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows:

- If both values are zeros, regardless of the sign of either zero, the result is the second value.
- If either value is a NaN, regardless of the value of FPCR.DN, the result is the second value.

SVE2

(FEAT_SVE2p1 || FEAT_SME2p1)



Encoding

```
FMINQV <Vd>.<T>, <Pg>, <Zn>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) && !IsFeatureImplemented(FEAT_SME2p1) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Vd);
```

Assembler Symbols

<Vd> Is the name of the destination SIMD&FP register, encoded in the "Vd" field.

<T> Is an arrangement specifier, encoded in “size”:

size	<T>
00	RESERVED
01	8H
10	4S
11	2D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	H
10	S
11	D

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let segments : integer{} = VL DIV 128;
let elempersegment : integer = 128 DIV esize;
let segbits : integer{} = segments*esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
let identity : bits(esize) = FPInfinity{esize}('0');
var result : bits(128) = Zeros{};

for e = 0 to elempersegment-1 do
    var stmp : bits(segbits);
    for s = 0 to segments-1 do
        if ActivePredicateElement{PL}(mask, s * elempersegment + e, esize) then
            stmp[s*:esize] = operand[(s * elempersegment + e)*:esize];
        else
            stmp[s*:esize] = identity;
        end;
    end;
    result[e*:esize] = FPReduce{esize, segbits}(ReduceOp_FMIN, stmp, FPCR());
end;
V{128}(d) = result;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

FMINV

Floating-point minimum recursive reduction to scalar

This instruction calculates a floating-point minimum horizontally over all lanes of a vector using a recursive pairwise reduction, and places the result in the SIMD&FP scalar destination register. Inactive elements in the source vector are treated as +Infinity.

When FPCR.AH is 0, the behavior is as follows:

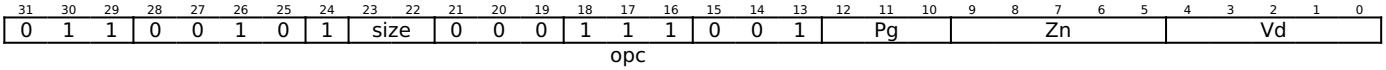
- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either value is a NaN, the result is a quiet NaN derived as follows:~~NaN.~~
  - If at least one value is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN value.
  - Otherwise, the result is a quiet NaN derived from the first NaN value.
- When FPCR.DN is 1, if either value is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows:

- If both values are zeros, regardless of the sign of either zero, the result is the second value.
- If either value is a NaN, regardless of the value of FPCR.DN, the result is the second value.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FMINV <V><d>, <Pg>, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Vd);
```

Assembler Symbols

<V> Is a width specifier, encoded in “size”:

size	<V>
00	RESERVED
01	H
10	S
11	D

<d> Is the number [0-31] of the destination SIMD&FP register, encoded in the "Vd" field.

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
let identity : bits(esize) = FPInfinity{esize}('0');

V{esize}(d) = FPReducePredicated{esize, VL, PL}(ReduceOp_FMIN, operand, mask,
                                                identity, FPCR());
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

FMMLA (non-widening)

Floating-point matrix multiply-accumulate

This instruction supports half-precision, single-precision, and double-precision data types in a 2x2 matrix contained in segments of 64, 128, or 256 bits, respectively. It multiplies the 2x2 matrix in each segment of the first source vector by the 2x2 matrix in the corresponding segment of the second source vector. The intermediate products are rounded before they are summed, and the intermediate sum is rounded before accumulation into the 2x2 matrix held in the corresponding segment of the addend and destination vector. This is equivalent to performing a 2-way dot product per destination element. This instruction is unpredicated. The half-precision and single-precision variants are vector length agnostic. The double-precision variant requires that the Effective SVE vector length is at least 256 bits.

ID_AA64ZFR0_EL1.F32MM indicates whether the single-precision variant is implemented.

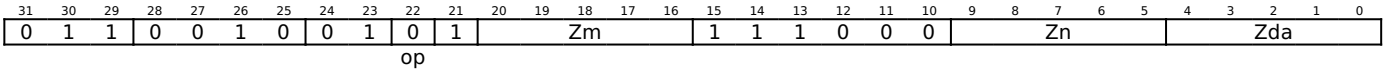
ID_AA64ZFR0_EL1.F64MM indicates whether the double-precision variant is implemented.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 3 classes: 16-bit element , 32-bit element and 64-bit element

16-bit element

(FEAT_SVE2p2 && FEAT_F16MM)



Encoding

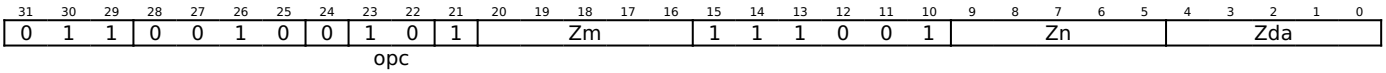
```
FMMLA <Zda>.H, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) || !IsFeatureImplemented(FEAT_F16MM) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

32-bit element

(FEAT_F32MM)



Encoding

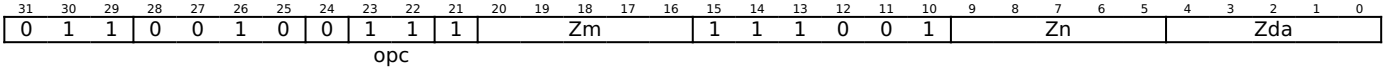
```
FMMLA <Zda>.S, <Zn>.S, <Zm>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_F32MM) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

64-bit element

(FEAT_F64MM)



Encoding

```
FMMLA <Zda>.D, <Zn>.D, <Zm>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_F64MM) then EndOfDecode(Decode_UNDEF); end;
iflet esize : integer{} = 64;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda); MaxImplementedVL() < 256 then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 64;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
if VL < esize * 4 then EndOfDecode(Decode_UNDEF); end;
let segments : integer = VL DIV (4 * esize);
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL) = Zeros{};
var op1, op2 : bits(4*esize);
var res, addend : bits(4*esize);

for s = 0 to segments-1 do
    op1    = operand1[s*:(4*esize)];
    op2    = operand2[s*:(4*esize)];
    addend = operand3[s*:(4*esize)];
    res    = FPMatMulAdd{4*esize}(addend, op1, op2, esize, FPCR());
    result[s*:(4*esize)] = res;
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel2026-06_diff_tag 2026-09-212026-09-08 1409:2244:3801

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

LDAPRO

Load-Acquire RCpc octaword

This instruction loads thirty-two contiguous bytes to elements of a 256-bit (octaword) vector from the memory address generated by a 64-bit scalar base, plus a scaled immediate offset. The base register must be aligned to 32 bytes if alignment is checked.

This instruction is unpredicated.

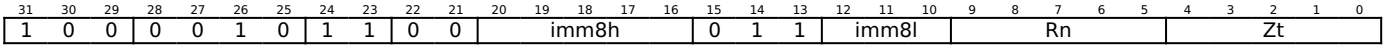
The instruction has memory ordering semantics as described in *Load-Acquire, Load-AcquirePC, and Store-Release*.

This instruction requires that the Effective SVE vector length is at least 256 bits.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE

(FEAT_SVE_LRCPC)



Encoding

```
LDAPRO <Zt>, [<Xn|SP>{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_LRCPC) then EndOfDecode(Decode_UNDEF); end;
if MaxImplementedVL() < 256 then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let imm : integer = UInt(imm8h::imm8l);
let acquirepc : boolean = TRUE;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional positive immediate byte offset, a multiple of 32 in the range 0 to 8160, defaulting to 0, encoded in the "imm8h:imm8l" fields.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
if VL < 256 then EndOfDecode(Decode_UNDEF); end;
var base : bits(64);
let offset : integer = imm * 32;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescLDAcqPC(tagchecked, acquirepc, t);

if n == 31 then
    CheckSPAlignment();
    base = SP{64}();
else
    base = X{64}(n);
end;

let addr : bits(64) = AddressAdd(base, offset, accdesc);

let data : bits(256) = Mem{256}(addr, accdesc);

Z{VL}(t) = ZeroExtend{VL}(data);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MOV (scalar, predicated)

Move general-purpose register to vector elements (predicated)

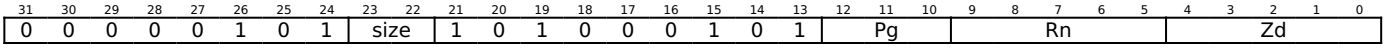
This instruction moves the general-purpose scalar source register to each Active element in the destination vector. Inactive elements in the destination vector register remain unmodified.

This is an alias of CPY (scalar). This means:

- The encodings in this description are named to match the encodings of CPY (scalar).
- The description of CPY (scalar) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

SVE

(FEAT_SVE || FEAT_SME)



Encoding for the 32-bit variant

Applies when (size != 11)

```
MOV <Zd>.<T>, <Pg>/M, <n|SP><Wn|WSP><R>
```

is equivalent to

```
CPY <Zd>.<T>, <Pg>/M, <n|SP><Wn|WSP><R>
```

and is always the preferred disassembly.

Encoding for the 64-bit variant

Applies when (size == 11)

```
MOV <Zd>.D, <Pg>/M, <Xn|SP>
```

is equivalent to

```
CPY <Zd>.D, <Pg>/M, <Xn|SP>
```

and is always the preferred disassembly.

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Wn|WSP>

size	<R>
00	W
01	W
10	W
11	X

Is the 32-bit name of the source general-purpose register or stack pointer, encoded in the "Rn" field.

<Xn|SP> Is the 64-bit name of the general-purpose source register or stack pointer, encoded in the "Rn" field.

Operation

The description of CPY (scalar) gives the operational pseudocode for this instruction.

Operational Information

The description of CPY (scalar) gives the operational information for this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

MOV (scalar, unpredicated)

Move general-purpose register to vector elements (unpredicated)

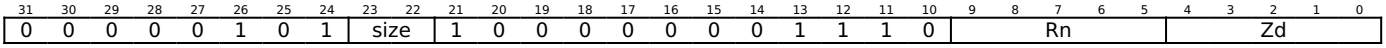
This instruction unconditionally broadcasts the general-purpose scalar source register into each element of the destination vector. This instruction is unpredicated.

This is an alias of DUP (scalar). This means:

- The encodings in this description are named to match the encodings of DUP (scalar).
- The description of DUP (scalar) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

SVE

(FEAT_SVE || FEAT_SME)



Encoding for the 32-bit variant

Applies when (size != 11)

```
MOV <Zd>.<T>, <n|SP><Wn|WSP><R>
```

is equivalent to

```
DUP <Zd>.<T>, <n|SP><Wn|WSP><R>
```

and is always the preferred disassembly.

Encoding for the 64-bit variant

Applies when (size == 11)

```
MOV <Zd>.D, <Xn|SP>
```

is equivalent to

```
DUP <Zd>.D, <Xn|SP>
```

and is always the preferred disassembly.

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Wn|WSP>

size	<R>
00	W
01	W
10	W
11	X

Is the 32-bit name of the source general-purpose register or stack pointer, encoded in the "Rn" field. Is a width specifier, encoded in “size”.

<Xn|SP> Is the 64-bit number name [0-30] of the general-purpose source general-purpose register or stack the pointer, name SP (31), encoded in the "Rn" field.

Operation

The description of DUP (scalar) gives the operational pseudocode for this instruction.

Operational Information

The description of DUP (scalar) gives the operational information for this instruction.

2026-09_rel2026-06_diff_tag 2026-09-212026-09-08 1409:2244:3801

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

PMULLB

Polynomial multiply long (bottom)

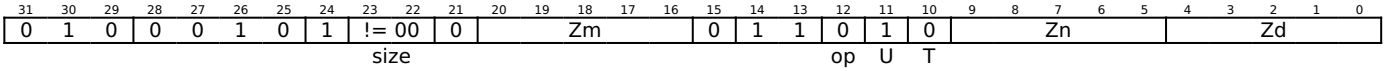
This instruction performs a polynomial multiplication over [0, 1] of the even-numbered elements of the first source vector by the corresponding elements of the second source vector, and places the results in the overlapping double-width elements of the destination vector. This instruction is unpredicated.
ID_AA64ZFR0_EL1.AES indicates whether the 128-bit element variant is implemented. The 128-bit element variant is legal when executed in Streaming SVE mode if one of the following is true:

- Both FEAT_SSVE_AES and FEAT_SVE_PMULL128 are implemented.
- FEAT_SME_FA64 is implemented and enabled.

It has encodings from 2 classes: 16-bit or 64-bit elements and 128-bit element

16-bit or 64-bit elements

(FEAT_SVE2 || FEAT_SME)



Encoding

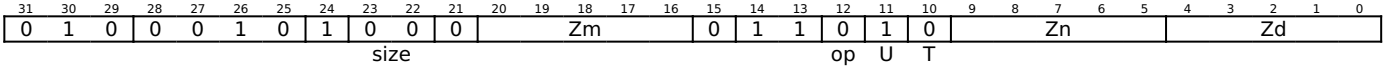
```
PMULLB <Zd>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '10' then~~if size[0] == '0' then~~ EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

128-bit element

(FEAT_SVE_PMULL128)



Encoding

```
PMULLB <Zd>.Q, <Zn>.D, <Zm>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_PMULL128) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 128;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
01	H
10~~1x~~	RESERVED~~D~~
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
01	B
10~~1x~~	RESERVED~~S~~
11	S

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
if esize == 128 then
    if IsFeatureImplemented(FEAT_SSVE_AES) then
        CheckSVEEnabled();
    else
        CheckNonStreamingSVEEnabled();
    end;
else
    CheckSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize DIV 2) = operand1[(2*e + 0)*:(esize DIV 2)];
    let element2 : bits(esize DIV 2) = operand2[(2*e + 0)*:(esize DIV 2)];
    result[e*:esize] = PolynomialMult{esize DIV 2, esize DIV 2}(element1, element2);
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

PMULLT

Polynomial multiply long (top)

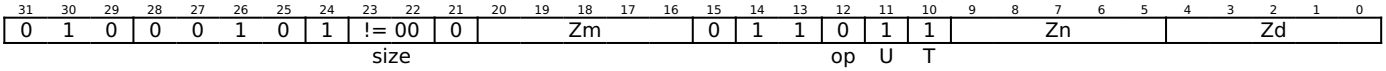
This instruction performs a polynomial multiplication over [0, 1] of the odd-numbered elements of the first source vector by the corresponding elements of the second source vector, and places the results in the overlapping double-width elements of the destination vector. This instruction is unpredicated.
ID_AA64ZFR0_EL1.AES indicates whether the 128-bit element variant is implemented. The 128-bit element variant is legal when executed in Streaming SVE mode if one of the following is true:

- Both FEAT_SSVE_AES and FEAT_SVE_PMULL128 are implemented.
- FEAT_SME_FA64 is implemented and enabled.

It has encodings from 2 classes: 16-bit or 64-bit elements and 128-bit element

16-bit or 64-bit elements

(FEAT_SVE2 || FEAT_SME)



Encoding

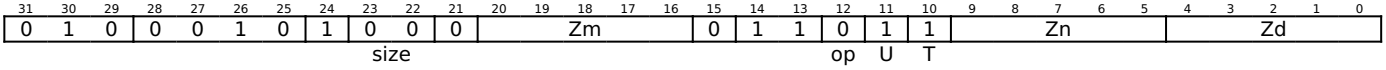
```
PMULLT <Zd>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '10' then~~if size[0] == '0' then~~ EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

128-bit element

(FEAT_SVE_PMULL128)



Encoding

```
PMULLT <Zd>.Q, <Zn>.D, <Zm>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_PMULL128) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 128;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
01	H
10~~1x~~	RESERVED~~D~~
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
01	B
10~~1x~~	RESERVED~~S~~
11	S

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
if esize == 128 then
    if IsFeatureImplemented(FEAT_SSVE_AES) then
        CheckSVEEnabled();
    else
        CheckNonStreamingSVEEnabled();
    end;
else
    CheckSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize DIV 2) = operand1[(2*e + 1)*:(esize DIV 2)];
    let element2 : bits(esize DIV 2) = operand2[(2*e + 1)*:(esize DIV 2)];
    result[e*:esize] = PolynomialMult{esize DIV 2, esize DIV 2}(element1, element2);
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

STLRO

Store-release octaword

This instruction stores a 32-byte (octaword) vector register to a memory address generated by a 64-bit scalar base, plus an immediate offset. The base register must be aligned to 32 bytes if alignment is checked.

This instruction is unpredicated.

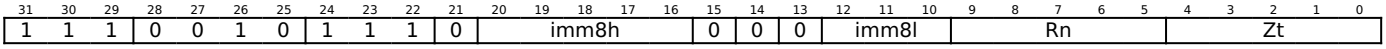
The instruction has memory ordering semantics as described in *Load-Acquire, Store-Release*.

This instruction requires that the Effective SVE vector length is at least 256 bits.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE

(FEAT_SVE_LRCPC)



Encoding

```
STLRO <Zt>, [<Xn|SP>{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_LRCPC) then EndOfDecode(Decode_UNDEF); end;
if MaxImplementedVL() < 256 then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let imm : integer = UInt(imm8h::imm8l);
let acquire : boolean = FALSE;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional positive immediate byte offset, a multiple of 32 in the range 0 to 8160, defaulting to 0, encoded in the "imm8h:imm8l" fields.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
if VL < 256 then EndOfDecode(Decode_UNDEF); end;
var src : bits(VL);
var base : bits(64);
let offset : integer = imm * 32;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescAcqRel(MemOp_STORE, tagchecked, acquire, t);

if n == 31 then
    CheckSPAlignment();
    base = SP{64}();
else
    base = X{64}(n);
end;

src = Z{VL}(t);
let data : bits(256) = src[0 +: 256];

let addr : bits(64) = AddressAdd(base, offset, accdesc);

Mem{256}(addr, accdesc) = data;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

TRN1, TRN2 (vectors)

Interleave even or odd elements from two vectors

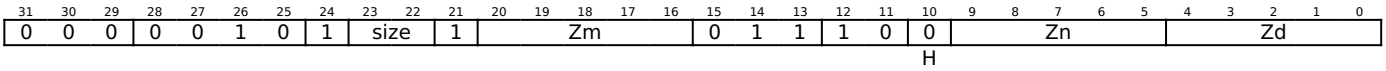
This instruction interleaves alternating even-numbered or odd-numbered elements from the first and second source vectors, and places the values in the destination vector. This instruction is unpredicated.

The Even128-bit (quadwords)element and variant Oddrequires (quadwords)that variantsthe require an Effective SVE vector length ofis at least 256 bits. ID_AA64ZFR0_EL1.F64MM or ID_AA64ZFR0_EL1.EltPerm indicates whether the Even128-bit (quadwords)element and Odd (quadwords) variantsvariant areis implemented. The Even128-bit (quadwords)element and Odd (quadwords) variantsvariant areis legalillegal when executed in Streaming SVE mode, ifunless FEAT_SVE_PERM256 FEAT_SME_FA64 is implemented.implemented and enabled.

It has encodings from 4 classes: Even , Even (quadwords) , Odd and Odd (quadwords)

Even

(FEAT_SVE || FEAT_SME)



Encoding

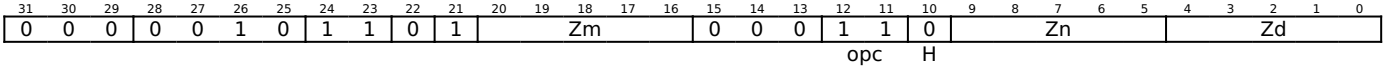
```
TRN1 <Zd>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let part : integer = 0;
```

Even (quadwords)

(FEAT_F64MM || FEAT_SVE_PERM256)



Encoding

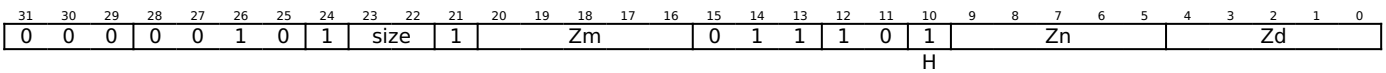
```
TRN1 <Zd>.Q, <Zn>.Q, <Zm>.Q
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_F64MM) && !IsFeatureImplemented(FEAT_SVE_PERM256) thenif !
IsFeatureImplemented(FEAT_F64MM) then
EndOfDecode(Decode_UNDEF);
end;
if); end;
let esize : integer{} = 128;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let part : integer = 0; MaxImplementedVL() < 256 then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 128;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let part : integer = 0;
```

Odd

(FEAT_SVE || FEAT_SME)



Encoding

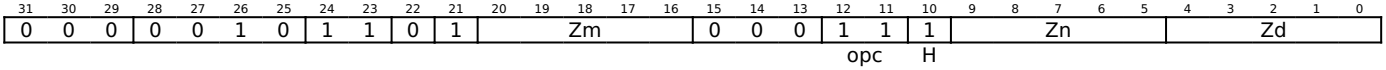
```
TRN2 <Zd>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let part : integer = 1;
```

Odd (quadwords)

(FEAT_F64MM || FEAT_SVE_PERM256)



Encoding

```
TRN2 <Zd>.Q, <Zn>.Q, <Zm>.Q
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_F64MM) && !IsFeatureImplemented(FEAT_SVE_PERM256) thenif !
IsFeatureImplemented(FEAT_F64MM) then
EndOfDecode(Decode_UNDEF);
end;
if); end;
let esize : integer{} = 128;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let part : integer = 1; MaxImplementedVL() < 256 then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 128;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let part : integer = 1;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
if esize < 128 || IsFeatureImplemented(FEAT_SVE_PERM256) thenif esize < 128 then
CheckSVEEnabled();
else
    CheckNonStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
if VL < esize * 2 then EndOfDecode(Decode_UNDEF); end;
let pairs : integer = VL DIV (esize * 2);
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Zeros{};

for p = 0 to pairs-1 do
    result[(2*p+0)*:esize] = operand1[(2*p+part)*:esize];
    result[(2*p+1)*:esize] = operand2[(2*p+part)*:esize];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

UZP1, UZP2 (vectors)

Concatenate even or odd elements from two vectors

This instruction concatenates adjacent even-numbered or odd-numbered elements from the first and second source vectors, and places the values in the destination vector. This instruction is unpredicated.

Note: UZP1 is equivalent to truncating and packing each element from two source vectors into a single destination vector with elements of half the size.

The Even~~128-bit~~ (quadwords)~~element~~ and~~variant~~ Odd~~requires~~ (quadwords)~~that~~ variants~~the~~ require an Effective SVE vector length of~~is~~ at least 256 bits. ID_AA64ZFR0_EL1.F64MM or ID_AA64ZFR0_EL1.EltPerm indicates whether the Even~~128-bit~~ (quadwords)~~element~~ and Odd (quadwords) variants~~variant~~ are~~is~~ implemented. The Even~~128-bit~~ (quadwords)~~element~~ and Odd (quadwords) variants~~variant~~ are~~is~~ legal~~illegal~~ when executed in Streaming SVE mode~~,~~ if~~unless~~ FEAT_SVE_PERM256 ~~FEAT_SME_FA64~~ is implemented.~~implemented and enabled.~~

It has encodings from 4 classes: Even , Even (quadwords) , Odd and Odd (quadwords)

Even

(FEAT_SVE || FEAT_SME)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	1	size		1	Zm					0	1	1	0	1	0	Zn					Zd				
																					H										

Encoding

```
UZP1 <Zd>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let part : integer = 0;
```

Even (quadwords)

(FEAT_F64MM || FEAT_SVE_PERM256)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	1	1	0	1	Zm					0	0	0	0	1	0	Zn					Zd				
																			opc		H										

Encoding

```
UZP1 <Zd>.Q, <Zn>.Q, <Zm>.Q
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_F64MM) && !IsFeatureImplemented(FEAT_SVE_PERM256) then~~if !IsFeatureImplemented(FEAT_F64MM) then~~
EndOfDecode(Decode_UNDEF);
end;
~~if); end;~~
~~let esize : integer{} = 128;~~
~~let n : integer = UInt(Zn);~~
~~let m : integer = UInt(Zm);~~
~~let d : integer = UInt(Zd);~~
~~let part : integer = 0;~~ MaxImplementedVL() < 256 then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 128;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let part : integer = 0;
```

Odd

(FEAT_SVE || FEAT_SME)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	1	size		1	Zm					0	1	1	0	1	1	Zn					Zd				
																					H										

Encoding

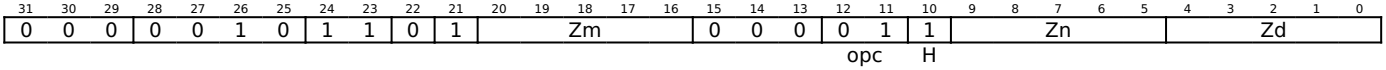
```
UZP2 <Zd>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let part : integer = 1;
```

Odd (quadwords)

(FEAT_F64MM || FEAT_SVE_PERM256)



Encoding

```
UZP2 <Zd>.Q, <Zn>.Q, <Zm>.Q
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_F64MM) && !IsFeatureImplemented(FEAT_SVE_PERM256) thenif !
IsFeatureImplemented(FEAT_F64MM) then
EndOfDecode(Decode_UNDEF);
end;
if); end;
let esize : integer{} = 128;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let part : integer = 1; MaxImplementedVL() < 256 then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 128;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let part : integer = 1;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
if esize < 128 || IsFeatureImplemented(FEAT_SVE_PERM256) thenif esize < 128 then
CheckSVEEnabled();
else
    CheckNonStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
if VL < esize * 2 then EndOfDecode(Decode_UNDEF); end;
let pairs : integer = VL DIV (esize * 2);
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Zeros{};

for p = 0 to pairs - 1 do
    result[p*:esize] = operand1[(2*p+part)*:esize];
end;

for p = 0 to pairs - 1 do
    result[(pairs+p)*:esize] = operand2[(2*p+part)*:esize];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

ZIP1, ZIP2 (vectors)

Interleave elements from two half vectors

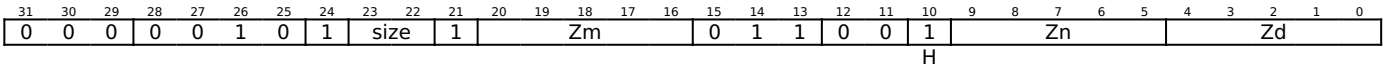
This instruction interleaves alternating elements from the lowest or highest halves of the first and second source vectors, and places the values in the destination vector. This instruction is unpredicated.

The High128-bit halveselement (quadwords)variant andrequires Lowthat halvesthe (quadwords) variants require an Effective SVE vector length ofis at least 256 bits. ID_AA64ZFR0_EL1.F64MM or ID_AA64ZFR0_EL1.EltPerm indicates whether the High128-bit halveselement (quadwords) and Low halves (quadwords) variantsvariant arcis implemented. The High128-bit halveselement (quadwords) and Low halves (quadwords) variantsvariant arcis legalillegal when executed in Streaming SVE mode, ifunless FEAT_SVE_PERM256 FEAT_SME_FA64 is implemented.implemented and enabled.

It has encodings from 4 classes: High halves , High halves (quadwords) , Low halves and Low halves (quadwords)

High halves

(FEAT_SVE || FEAT_SME)



Encoding

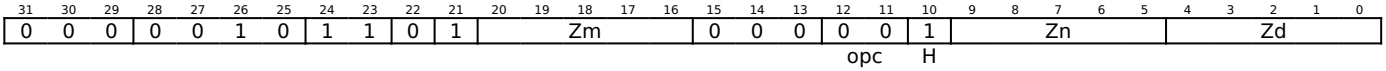
```
ZIP2 <Zd>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let part : integer = 1;
```

High halves (quadwords)

(FEAT_F64MM || FEAT_SVE_PERM256)



Encoding

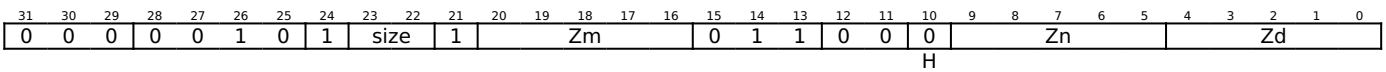
```
ZIP2 <Zd>.Q, <Zn>.Q, <Zm>.Q
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_F64MM) && !IsFeatureImplemented(FEAT_SVE_PERM256) thenif !
IsFeatureImplemented(FEAT_F64MM) then
EndOfDecode(Decode_UNDEF);
end;
if); end;
let esize : integer{} = 128;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let part : integer = 1; MaxImplementedVL() < 256 then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 128;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let part : integer = 1;
```

Low halves

(FEAT_SVE || FEAT_SME)



Encoding

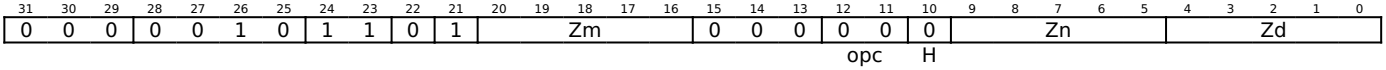
```
ZIP1 <Zd>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let part : integer = 0;
```

Low halves (quadwords)

(FEAT_F64MM || FEAT_SVE_PERM256)



Encoding

```
ZIP1 <Zd>.Q, <Zn>.Q, <Zm>.Q
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_F64MM) && !IsFeatureImplemented(FEAT_SVE_PERM256) thenif !
IsFeatureImplemented(FEAT_F64MM) then
EndOfDecode(Decode_UNDEF);
end;
if); end;
let esize : integer{} = 128;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let part : integer = 0; MaxImplementedVL() < 256 then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 128;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let part : integer = 0;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
if esize < 128 || IsFeatureImplemented(FEAT_SVE_PERM256) thenif esize < 128 then
CheckSVEEnabled();
else
    CheckNonStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
if VL < esize * 2 then EndOfDecode(Decode_UNDEF); end;
let pairs : integer = VL DIV (esize * 2);
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Zeros{};

let base : integer = part * pairs;
for p = 0 to pairs-1 do
    result[(2*p+0)*:esize] = operand1[(base+p)*:esize];
    result[(2*p+1)*:esize] = operand2[(base+p)*:esize];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

A64 -- SME Instructions (alphabetic order)

ADD (to array, array and multiple vectors): Multi-vector accumulate to ZA array vectors.

ADD (to array, multiple and single vector): Multi-vector add by vector to ZA array vectors.

ADD (to array, multiple vectors): Multi-vector add to ZA array vectors.

ADD (to vector, multiple vectors): Multi-vector add by vector to multi-vector.

ADDHA: Add horizontally vector elements to ZA tile.

ADDSPL: Add multiple of Streaming SVE predicate register size to scalar register.

ADDSVL: Add multiple of Streaming SVE vector register size to scalar register.

ADDVA: Add vertically vector elements to ZA tile.

BF1CVT, BF2CVT: Multi-vector 8-bit floating-point convert to BFloat16.

BF1CVTL, BF2CVTL: Multi-vector 8-bit floating-point convert to deinterleaved BFloat16.

BFADD: Multi-vector BFloat16 accumulate to ZA array vectors.

BFCLAMP: Multi-vector BFloat16 clamp to minimum/maximum number.

BFCVT (BFloat16 to 8-bit floating-point): Multi-vector BFloat16 convert to 8-bit floating-point.

BFCVT (single-precision to BFloat16): Multi-vector single-precision convert to BFloat16.

BFCVTN: Multi-vector single-precision convert to interleaved BFloat16.

BFDOT (multiple and indexed vector): Multi-vector BFloat16 dot product by indexed element to single-precision.

BFDOT (multiple and single vector): Multi-vector BFloat16 dot product by vector to single-precision.

BFDOT (multiple vectors): Multi-vector BFloat16 dot product to single-precision.

BFMAX (multiple and single vector): Multi-vector BFloat16 maximum by vector.

BFMAX (multiple vectors): Multi-vector BFloat16 maximum.

BFMAXNM (multiple and single vector): Multi-vector BFloat16 maximum number by vector.

BFMAXNM (multiple vectors): Multi-vector BFloat16 maximum number.

BFMIN (multiple and single vector): Multi-vector BFloat16 minimum by vector.

BFMIN (multiple vectors): Multi-vector BFloat16 minimum.

BFMINNM (multiple and single vector): Multi-vector BFloat16 minimum number by vector.

BFMINNM (multiple vectors): Multi-vector BFloat16 minimum number.

BFMLA (multiple and indexed vector): Multi-vector BFloat16 fused multiply-add by indexed element.

BFMLA (multiple and single vector): Multi-vector BFloat16 fused multiply-add by vector.

BFMLA (multiple vectors): Multi-vector BFloat16 fused multiply-add.

BFMLAL (multiple and indexed vector): Multi-vector BFloat16 multiply-add by indexed element to single-precision.

BFMLAL (multiple and single vector): Multi-vector BFloat16 multiply-add by vector to single-precision.

BFMLAL (multiple vectors): Multi-vector BFloat16 multiply-add to single-precision.

BFMLS (multiple and indexed vector): Multi-vector BFloat16 fused multiply-subtract by indexed element.

BFMLS (multiple and single vector): Multi-vector BFloat16 fused multiply-subtract by vector.

BFMLS (multiple vectors): Multi-vector BFloat16 fused multiply-subtract.

BFMLSL (multiple and indexed vector): Multi-vector BFloat16 multiply-subtract by indexed element from single-precision.

BFMLSL (multiple and single vector): Multi-vector BFloat16 multiply-subtract by vector from single-precision.

BFMLSL (multiple vectors): Multi-vector BFloat16 multiply-subtract from single-precision.

BFMOP4A (non-widening): BFloat16 quarter-tile outer product, accumulating.

BFMOP4A (widening): BFloat16 quarter-tile sum of outer products to single-precision, accumulating.

BFMOP4S (non-widening): BFloat16 quarter-tile outer product, subtracting.

BFMOP4S (widening): BFloat16 quarter-tile sum of outer products to single-precision, subtracting.

BFMOPA (non-widening): BFloat16 outer product, accumulating.

BFMOPA (widening): BFloat16 sum of outer products to single-precision, accumulating.

BFMOPS (non-widening): BFloat16 outer product, subtracting.

BFMOPS (widening): BFloat16 sum of outer products to single-precision, subtracting.

BFMUL (multiple and single vector): Multi-vector BFloat16 multiply by vector.

BFMUL (multiple vectors): Multi-vector BFloat16 multiply.

BFSCALE (multiple and single vector): Multi-vector BFloat16 adjust exponent by vector.

BFSCALE (multiple vectors): Multi-vector BFloat16 adjust exponent.

BFSUB: Multi-vector BFloat16 subtract from ZA array vectors.

BFTMOPA (non-widening): BFloat16 sparse outer product, accumulating.

BFTMOPA (widening): BFloat16 sparse sum of outer products to single-precision, accumulating.

BFVDOT: Multi-vector BFloat16 vertical dot product by indexed element to single-precision.

BMOPA: Bitwise exclusive NOR population count outer product, accumulating.

BMOPS: Bitwise exclusive NOR population count outer product, subtracting.

F1CVT, F2CVT: Multi-vector 8-bit floating-point convert to half-precision.

F1CVTL, F2CVTL: Multi-vector 8-bit floating-point convert to deinterleaved half-precision.

FADD: Multi-vector floating-point accumulate to ZA array vectors.

FAMAX: Multi-vector floating-point absolute maximum.

FAMIN: Multi-vector floating-point absolute minimum.

FCLAMP: Multi-vector floating-point clamp to minimum/maximum number.

FCVT (narrowing, FP16 to FP8): Multi-vector half-precision convert to 8-bit floating-point.

FCVT (narrowing, FP32 to FP16): Multi-vector single-precision convert to half-precision.

FCVT (narrowing, FP32 to FP8): Multi-vector single-precision convert to 8-bit floating-point.

FCVT (widening): Multi-vector half-precision convert to single-precision.

FCVTL: Multi-vector half-precision convert to deinterleaved single-precision.

FCVTN (FP32 to FP16): Multi-vector single-precision convert to interleaved half-precision.

FCVTN (FP32 to FP8): Multi-vector single-precision convert to interleaved 8-bit floating-point.

FCVTZS: Multi-vector single-precision convert to signed 32-bit integer, rounding toward zero.

FCVTZU: Multi-vector single-precision convert to unsigned 32-bit integer, rounding toward zero.

FDOT (2-way, multiple and indexed vector, FP16 to FP32): Multi-vector half-precision dot product by indexed element to single-precision.

FDOT (2-way, multiple and indexed vector, FP8 to FP16): Multi-vector 8-bit floating-point dot product by indexed element to half-precision.

FDOT (2-way, multiple and single vector, FP16 to FP32): Multi-vector half-precision dot product by vector to single-precision.

FDOT (2-way, multiple and single vector, FP8 to FP16): Multi-vector 8-bit floating-point dot product by vector to half-precision.

FDOT (2-way, multiple vectors, FP16 to FP32): Multi-vector half-precision dot product to single-precision.

FDOT (2-way, multiple vectors, FP8 to FP16): Multi-vector 8-bit floating-point dot product to half-precision.

FDOT (4-way, multiple and indexed vector): Multi-vector 8-bit floating-point dot product by indexed element to single-precision.

FDOT (4-way, multiple and single vector): Multi-vector 8-bit floating-point dot product by vector to single-precision.

FDOT (4-way, multiple vectors): Multi-vector 8-bit floating-point dot product to single-precision.

FMAX (multiple and single vector): Multi-vector floating-point maximum by vector.

FMAX (multiple vectors): Multi-vector floating-point maximum.

FMAXNM (multiple and single vector): Multi-vector floating-point maximum number by vector.

FMAXNM (multiple vectors): Multi-vector floating-point maximum number.

FMIN (multiple and single vector): Multi-vector floating-point minimum by vector.

FMIN (multiple vectors): Multi-vector floating-point minimum.

FMINNM (multiple and single vector): Multi-vector floating-point minimum number by vector.

FMINNM (multiple vectors): Multi-vector floating-point minimum number.

FMLA (multiple and indexed vector): Multi-vector floating-point fused multiply-add by indexed element.

FMLA (multiple and single vector): Multi-vector floating-point fused multiply-add by vector.

FMLA (multiple vectors): Multi-vector floating-point fused multiply-add.

FMLAL (multiple and indexed vector, FP16 to FP32): Multi-vector half-precision multiply-add by indexed element to single-precision.

FMLAL (multiple and indexed vector, FP8 to FP16): Multi-vector 8-bit floating-point multiply-add by indexed element to half-precision.

FMLAL (multiple and single vector, FP16 to FP32): Multi-vector half-precision multiply-add by vector to single-precision.

FMLAL (multiple and single vector, FP8 to FP16): Multi-vector 8-bit floating-point multiply-add by vector to half-precision.

FMLAL (multiple vectors, FP16 to FP32): Multi-vector half-precision multiply-add to single-precision.

FMLAL (multiple vectors, FP8 to FP16): Multi-vector 8-bit floating-point multiply-add to half-precision.

FMLALL (multiple and indexed vector): Multi-vector 8-bit floating-point multiply-add by indexed element to single-precision.

FMLALL (multiple and single vector): Multi-vector 8-bit floating-point multiply-add by vector to single-precision.

FMLALL (multiple vectors): Multi-vector 8-bit floating-point multiply-add to single-precision.

FMLS (multiple and indexed vector): Multi-vector floating-point fused multiply-subtract by indexed element.

FMLS (multiple and single vector): Multi-vector floating-point fused multiply-subtract by vector.

FMLS (multiple vectors): Multi-vector floating-point fused multiply-subtract.

FMLSL (multiple and indexed vector): Multi-vector half-precision multiply-subtract by indexed element from single-precision.

FMLSL (multiple and single vector): Multi-vector half-precision multiply-subtract by vector from single-precision.

FMLSL (multiple vectors): Multi-vector half-precision multiply-subtract from single-precision.

FMOP4A (non-widening): Floating-point quarter-tile outer product, accumulating.

FMOP4A (widening, 2-way, FP16 to FP32): Half-precision quarter-tile sum of outer products to single-precision, accumulating.

FMOP4A (widening, 2-way, FP8 to FP16): 8-bit floating-point quarter-tile sum of outer products to half-precision, accumulating.

FMOP4A (widening, 4-way): 8-bit floating-point quarter-tile sum of outer products to single-precision, accumulating.

FMOP4S (non-widening): Floating-point quarter-tile outer product, subtracting.

FMOP4S (widening): Half-precision quarter-tile sum of outer products to single-precision, subtracting.

FMOPA (non-widening): Floating-point outer product, accumulating.

FMOPA (widening, 2-way, FP16 to FP32): Half-precision sum of outer products to single-precision, accumulating.

FMOPA (widening, 2-way, FP8 to FP16): 8-bit floating-point sum of outer products to half-precision, accumulating.

FMOPA (widening, 4-way): 8-bit floating-point sum of outer products to single-precision, accumulating.

FMOPS (non-widening): Floating-point outer product, subtracting.

FMOPS (widening): Half-precision sum of outer products to single-precision, subtracting.

FMUL (multiple and single vector): Multi-vector floating-point multiply by vector.

FMUL (multiple vectors): Multi-vector floating-point multiply.

FRINTA: Multi-vector single-precision round to integral value, to nearest with ties away from zero.

FRINTM: Multi-vector single-precision round to integral value, toward minus Infinity.

FRINTN: Multi-vector single-precision round to integral value, to nearest with ties to even.

FRINTP: Multi-vector single-precision round to integral value, toward plus Infinity.

FSCALE (multiple and single vector): Multi-vector floating-point adjust exponent by vector.

FSCALE (multiple vectors): Multi-vector floating-point adjust exponent.

FSUB: Multi-vector floating-point subtract from ZA array vectors.

FTMOPA (non-widening): Floating-point sparse outer product, accumulating.

FTMOPA (widening, 2-way, FP16 to FP32): Half-precision sparse sum of outer products to single-precision, accumulating.

FTMOPA (widening, 2-way, FP8 to FP16): 8-bit floating-point sparse sum of outer products to half-precision, accumulating.

FTMOPA (widening, 4-way): 8-bit floating-point sparse sum of outer products to single-precision, accumulating.

FVDOT (FP16 to FP32): Multi-vector half-precision vertical dot product by indexed element to single-precision.

FVDOT (FP8 to FP16): Multi-vector 8-bit floating-point vertical dot product by indexed element to half-precision.

FVDOTB: Multi-vector 8-bit floating-point vertical dot product by indexed element to single-precision (bottom).

FVDOTT: Multi-vector 8-bit floating-point vertical dot product by indexed element to single-precision (top).

LD1B (scalar plus immediate, strided registers): Contiguous load of bytes to multiple strided vectors (immediate index).

LD1B (scalar plus scalar, strided registers): Contiguous load of bytes to multiple strided vectors (scalar index).

LD1B (scalar plus scalar, tile slice): Contiguous load of bytes to 8-bit element ZA tile slice.

LD1D (scalar plus immediate, strided registers): Contiguous load of doublewords to multiple strided vectors (immediate index).

LD1D (scalar plus scalar, strided registers): Contiguous load of doublewords to multiple strided vectors (scalar index).

LD1D (scalar plus scalar, tile slice): Contiguous load of doublewords to 64-bit element ZA tile slice.

LD1H (scalar plus immediate, strided registers): Contiguous load of halfwords to multiple strided vectors (immediate index).

LD1H (scalar plus scalar, strided registers): Contiguous load of halfwords to multiple strided vectors (scalar index).

LD1H (scalar plus scalar, tile slice): Contiguous load of halfwords to 16-bit element ZA tile slice.

LD1Q: Contiguous load of quadwords to 128-bit element ZA tile slice.

LD1W (scalar plus immediate, strided registers): Contiguous load of words to multiple strided vectors (immediate index).

LD1W (scalar plus scalar, strided registers): Contiguous load of words to multiple strided vectors (scalar index).

LD1W (scalar plus scalar, tile slice): Contiguous load of words to 32-bit element ZA tile slice.

LDNT1B (scalar plus immediate, strided registers): Contiguous load non-temporal of bytes to multiple strided vectors (immediate index).

LDNT1B (scalar plus scalar, strided registers): Contiguous load non-temporal of bytes to multiple strided vectors (scalar index).

LDNT1D (scalar plus immediate, strided registers): Contiguous load non-temporal of doublewords to multiple strided vectors (immediate index).

LDNT1D (scalar plus scalar, strided registers): Contiguous load non-temporal of doublewords to multiple strided vectors (scalar index).

LDNT1H (scalar plus immediate, strided registers): Contiguous load non-temporal of halfwords to multiple strided vectors (immediate index).

LDNT1H (scalar plus scalar, strided registers): Contiguous load non-temporal of halfwords to multiple strided vectors (scalar index).

LDNT1W (scalar plus immediate, strided registers): Contiguous load non-temporal of words to multiple strided vectors (immediate index).

LDNT1W (scalar plus scalar, strided registers): Contiguous load non-temporal of words to multiple strided vectors (scalar index).

LDR (array vector): Load ZA array vector.

LDR (table): Load ZT0 register.

LUTI2 (four registers): Lookup table read with 2-bit indexes (four registers).

LUTI2 (single): Lookup table read with 2-bit indexes (single).

LUTI2 (two registers): Lookup table read with 2-bit indexes (two registers).

LUTI4 (four registers, 16-bit and 32-bit): Lookup table read with 4-bit indexes (four registers).

LUTI4 (four registers, 8-bit): Lookup table read with 4-bit indexes and 8-bit elements (four registers).

LUTI4 (single): Lookup table read with 4-bit indexes (single).

LUTI4 (two registers): Lookup table read with 4-bit indexes (two registers).

LUTI6 (table, four registers, 8-bit): Lookup table read with 6-bit indices (8-bit).

LUTI6 (table, single, 8-bit): Lookup table read with 6-bit indices (single).

LUTI6 (vector, 16-bit): Lookup table read with 6-bit indices (16-bit).

MOV (array to vector, four registers): Move four ZA single-vector groups to Z four-vector operand: an alias of MOVA (array to vector, four registers).

MOV (array to vector, two registers): Move two ZA single-vector groups to Z two-vector operand: an alias of MOVA (array to vector, two registers).

MOV (tile to vector, four registers): Move ZA four-slice operand to Z four-vector operand: an alias of MOVA (tile to vector, four registers).

MOV (tile to vector, single): Move ZA tile slice to Z vector: an alias of MOVA (tile to vector, single).

MOV (tile to vector, two registers): Move ZA two-slice operand to Z two-vector operand: an alias of MOVA (tile to vector, two registers).

MOV (vector to array, four registers): Move Z four-vector operand to four ZA single-vector groups: an alias of MOVA (vector to array, four registers).

MOV (vector to array, two registers): Move Z two-vector operand to two ZA single-vector groups: an alias of MOVA (vector to array, two registers).

MOV (vector to tile, four registers): Move Z four-vector operand to ZA four-slice operand: an alias of MOVA (vector to tile, four registers).

MOV (vector to tile, single): Move Z vector to ZA tile slice: an alias of MOVA (vector to tile, single).

MOV (vector to tile, two registers): Move Z two-vector operand to ZA two-slice operand: an alias of MOVA (vector to tile, two registers).

MOVA (array to vector, four registers): Move four ZA single-vector groups to Z four-vector operand.

MOVA (array to vector, two registers): Move two ZA single-vector groups to Z two-vector operand.

MOVA (tile to vector, four registers): Move ZA four-slice operand to Z four-vector operand.

MOVA (tile to vector, single): Move ZA tile slice to Z vector.

MOVA (tile to vector, two registers): Move ZA two-slice operand to Z two-vector operand.

MOVA (vector to array, four registers): Move Z four-vector operand to four ZA single-vector groups.

MOVA (vector to array, two registers): Move Z two-vector operand to two ZA single-vector groups.

MOVA (vector to tile, four registers): Move Z four-vector operand to ZA four-slice operand.

MOVA (vector to tile, single): Move Z vector to ZA tile slice.

MOVA (vector to tile, two registers): Move Z two-vector operand to ZA two-slice operand.

MOVAZ (array to vector, four registers): Move and zero four ZA single-vector groups to Z four-vector operand.

MOVAZ (array to vector, two registers): Move and zero two ZA single-vector groups to Z two-vector operand.

MOVAZ (tile to vector, four registers): Move and zero ZA four-slice operand to Z four-vector operand.

MOVAZ (tile to vector, single): Move and zero ZA tile slice to Z vector.

MOVAZ (tile to vector, two registers): Move and zero ZA two-slice operand to Z two-vector operand.

MOVT (scalar to table): Move 8 bytes from general-purpose register to ZT0.

MOVT (table to scalar): Move 8 bytes from ZT0 to general-purpose register.

MOVT (vector to table): Move vector register to ZT0.

RDSVL: Read multiple of Streaming SVE vector register size to scalar register.

SCLAMP: Multi-vector signed clamp to minimum/maximum.

SCVTF: Multi-vector signed 32-bit integer convert to single-precision.

SDOT (2-way, multiple and indexed vector): Multi-vector signed 16-bit integer dot product by indexed element to 32-bit integer.

SDOT (2-way, multiple and single vector): Multi-vector signed 16-bit integer dot product by vector to 32-bit integer.

SDOT (2-way, multiple vectors): Multi-vector signed 16-bit integer dot product to 32-bit integer.

SDOT (4-way, multiple and indexed vector): Multi-vector signed integer dot product by indexed element.

SDOT (4-way, multiple and single vector): Multi-vector signed integer dot product by vector.

SDOT (4-way, multiple vectors): Multi-vector signed integer dot product.

SEL: Multi-vector conditional select.

SMAX (multiple and single vector): Multi-vector signed maximum by vector.

SMAX (multiple vectors): Multi-vector signed maximum.

SMIN (multiple and single vector): Multi-vector signed minimum by vector.

SMIN (multiple vectors): Multi-vector signed minimum.

SMLAL (multiple and indexed vector): Multi-vector signed 16-bit integer multiply-add by indexed element to 32-bit integer.

SMLAL (multiple and single vector): Multi-vector signed 16-bit integer multiply-add by vector to 32-bit integer.

SMLAL (multiple vectors): Multi-vector signed 16-bit integer multiply-add to 32-bit integer.

SMLALL (multiple and indexed vector): Multi-vector signed integer multiply-add long long by indexed element.

SMLALL (multiple and single vector): Multi-vector signed integer multiply-add long long by vector.

SMLALL (multiple vectors): Multi-vector signed integer multiply-add long long.

SMLSL (multiple and indexed vector): Multi-vector signed 16-bit integer multiply-subtract by indexed element from 32-bit integer.

SMLSL (multiple and single vector): Multi-vector signed 16-bit integer multiply-subtract by vector from 32-bit integer.

SMLSL (multiple vectors): Multi-vector signed 16-bit integer multiply-subtract from 32-bit integer.

SMLSLL (multiple and indexed vector): Multi-vector signed integer multiply-subtract long long by indexed element.

SMLSLL (multiple and single vector): Multi-vector signed integer multiply-subtract long long by vector.

SMLSLL (multiple vectors): Multi-vector signed integer multiply-subtract long long.

SMOP4A (2-way): Signed 16-bit integer quarter-tile sum of outer products to 32-bit integer, accumulating.

SMOP4A (4-way): Signed integer quarter-tile sum of outer products, accumulating.

SMOP4S (2-way): Signed 16-bit integer quarter-tile sum of outer products to 32-bit integer, subtracting.

SMOP4S (4-way): Signed integer quarter-tile sum of outer products, subtracting.

SMOPA (2-way): Signed 16-bit integer sum of outer products to 32-bit integer, accumulating.

SMOPA (4-way): Signed integer sum of outer products, accumulating.

SMOPS (2-way): Signed 16-bit integer sum of outer products to 32-bit integer, subtracting.

SMOPS (4-way): Signed integer sum of outer products, subtracting.

SQCVT (four registers): Multi-vector signed saturating extract narrow.

SQCVT (two registers): Multi-vector signed 32-bit integer saturating extract narrow to 16-bit integer.

SQCVTN: Multi-vector signed saturating extract narrow to interleaved integer.

SQCVTU (four registers): Multi-vector signed saturating extract narrow to unsigned integer.

SQCVTU (two registers): Multi-vector signed 32-bit integer saturating extract narrow to unsigned 16-bit integer.

SQCVTUN: Multi-vector signed saturating extract narrow to interleaved unsigned integer.

SQDMULH (multiple and single vector): Multi-vector signed saturating doubling multiply high by vector.

SQDMULH (multiple vectors): Multi-vector signed saturating doubling multiply high.

SQRSHR (four registers): Multi-vector signed saturating rounding shift right narrow by immediate.

SQRSHR (two registers): Multi-vector signed 32-bit integer saturating rounding shift right narrow by immediate to 16-bit integer.

SQRSHRN: Multi-vector signed saturating rounding shift right narrow by immediate to interleaved integer.

SQRSHRU (four registers): Multi-vector signed saturating rounding shift right narrow by immediate to unsigned integer.

SQRSHRU (two registers): Multi-vector signed 32-bit integer saturating rounding shift right narrow by immediate to unsigned 16-bit integer.

SQRSHRUN: Multi-vector signed saturating rounding shift right narrow by immediate to interleaved unsigned integer.

SRSHL (multiple and single vector): Multi-vector signed rounding shift left by vector.

SRSHL (multiple vectors): Multi-vector signed rounding shift left.

ST1B (scalar plus immediate, strided registers): Contiguous store of bytes from multiple strided vectors (immediate index).

ST1B (scalar plus scalar, strided registers): Contiguous store of bytes from multiple strided vectors (scalar index).

ST1B (scalar plus scalar, tile slice): Contiguous store of bytes from 8-bit element ZA tile slice.

ST1D (scalar plus immediate, strided registers): Contiguous store of doublewords from multiple strided vectors (immediate index).

ST1D (scalar plus scalar, strided registers): Contiguous store of doublewords from multiple strided vectors (scalar index).

ST1D (scalar plus scalar, tile slice): Contiguous store of doublewords from 64-bit element ZA tile slice.

ST1H (scalar plus immediate, strided registers): Contiguous store of halfwords from multiple strided vectors (immediate index).

ST1H (scalar plus scalar, strided registers): Contiguous store of halfwords from multiple strided vectors (scalar index).

ST1H (scalar plus scalar, tile slice): Contiguous store of halfwords from 16-bit element ZA tile slice.

ST1Q: Contiguous store of quadwords from 128-bit element ZA tile slice.

ST1W (scalar plus immediate, strided registers): Contiguous store of words from multiple strided vectors (immediate index).

ST1W (scalar plus scalar, strided registers): Contiguous store of words from multiple strided vectors (scalar index).

ST1W (scalar plus scalar, tile slice): Contiguous store of words from 32-bit element ZA tile slice.

STMOPA (2-way): Signed 16-bit integer sparse sum of outer products to 32-bit integer, accumulating.

STMOPA (4-way): Signed 8-bit integer sparse sum of outer products to 32-bit integer, accumulating.

STNT1B (scalar plus immediate, strided registers): Contiguous store non-temporal of bytes from multiple strided vectors (immediate index).

STNT1B (scalar plus scalar, strided registers): Contiguous store non-temporal of bytes from multiple strided vectors (scalar index).

STNT1D (scalar plus immediate, strided registers): Contiguous store non-temporal of doublewords from multiple strided vectors (immediate index).

STNT1D (scalar plus scalar, strided registers): Contiguous store non-temporal of doublewords from multiple strided vectors (scalar index).

STNT1H (scalar plus immediate, strided registers): Contiguous store non-temporal of halfwords from multiple strided vectors (immediate index).

STNT1H (scalar plus scalar, strided registers): Contiguous store non-temporal of halfwords from multiple strided vectors (scalar index).

STNT1W (scalar plus immediate, strided registers): Contiguous store non-temporal of words from multiple strided vectors (immediate index).

STNT1W (scalar plus scalar, strided registers): Contiguous store non-temporal of words from multiple strided vectors (scalar index).

STR (array vector): Store ZA array vector.

STR (table): Store ZT0 register.

SUB (to array, array and multiple vectors): Multi-vector subtract from ZA array vectors.

SUB (to array, multiple and single vector): Multi-vector subtract by vector to ZA array vectors.

SUB (to array, multiple vectors): Multi-vector subtract to ZA array vectors.

SUDOT (4-way, multiple and indexed vector): Multi-vector signed by unsigned 8-bit integer dot product by indexed elements to 32-bit integer.

SUDOT (4-way, multiple and single vector): Multi-vector signed by unsigned 8-bit integer dot product by vector to 32-bit integer.

SUMLALL (multiple and indexed vector): Multi-vector signed by unsigned 8-bit integer multiply-add by indexed element to 32-bit integer.

SUMLALL (multiple and single vector): Multi-vector signed by unsigned 8-bit integer multiply-add by vector to 32-bit integer.

SUMOP4A: Signed by unsigned integer quarter-tile sum of outer products, accumulating.

SUMOP4S: Signed by unsigned integer quarter-tile sum of outer products, subtracting.

SUMOPA (4-way): Signed by unsigned integer sum of outer products, accumulating.

SUMOPS: Signed by unsigned integer sum of outer products, subtracting.

SUNPK: Unpack and sign-extend multi-vector elements.

SUTMOPA: Signed by unsigned 8-bit integer sparse sum of outer products to 32-bit integer, accumulating.

SUVDOT: Multi-vector signed by unsigned 8-bit integer vertical dot product by indexed element to 32-bit integer.

SVDOT (2-way): Multi-vector signed 16-bit integer vertical dot product by indexed element to 32-bit integer.

SVDOT (4-way): Multi-vector signed integer vertical dot product by indexed element.

UCLAMP: Multi-vector unsigned clamp to minimum/maximum.

UCVTF: Multi-vector unsigned 32-bit integer convert to single-precision.

UDOT (2-way, multiple and indexed vector): Multi-vector unsigned 16-bit integer dot product by indexed element to 32-bit integer.

UDOT (2-way, multiple and single vector): Multi-vector unsigned 16-bit integer dot product by vector to 32-bit integer.

UDOT (2-way, multiple vectors): Multi-vector unsigned 16-bit integer dot product to 32-bit integer.

UDOT (4-way, multiple and indexed vector): Multi-vector unsigned integer dot product by indexed element.

UDOT (4-way, multiple and single vector): Multi-vector unsigned integer dot product by vector.

UDOT (4-way, multiple vectors): Multi-vector unsigned integer dot product.

UMAX (multiple and single vector): Multi-vector unsigned maximum by vector.

UMAX (multiple vectors): Multi-vector unsigned maximum.

UMIN (multiple and single vector): Multi-vector unsigned minimum by vector.

UMIN (multiple vectors): Multi-vector unsigned minimum.

UMLAL (multiple and indexed vector): Multi-vector unsigned 16-bit integer multiply-add by indexed element to 32-bit integer.

UMLAL (multiple and single vector): Multi-vector unsigned 16-bit integer multiply-add by vector to 32-bit integer.

UMLAL (multiple vectors): Multi-vector unsigned 16-bit integer multiply-add to 32-bit integer.

UMLALL (multiple and indexed vector): Multi-vector unsigned integer multiply-add long long by indexed element.

UMLALL (multiple and single vector): Multi-vector unsigned integer multiply-add long long by vector.

UMLALL (multiple vectors): Multi-vector unsigned integer multiply-add long long.

UMLSL (multiple and indexed vector): Multi-vector unsigned 16-bit integer multiply-subtract by indexed element from 32-bit integer.

UMLSL (multiple and single vector): Multi-vector unsigned 16-bit integer multiply-subtract by vector from 32-bit integer.

UMLSL (multiple vectors): Multi-vector unsigned 16-bit integer multiply-subtract from 32-bit integer.

UMLSLL (multiple and indexed vector): Multi-vector unsigned integer multiply-subtract long long by indexed element.

UMLSLL (multiple and single vector): Multi-vector unsigned integer multiply-subtract long long by vector.

UMLSLL (multiple vectors): Multi-vector unsigned integer multiply-subtract long long.

UMOP4A (2-way): Unsigned 16-bit integer quarter-tile sum of outer products to 32-bit integer, accumulating.

UMOP4A (4-way): Unsigned integer quarter-tile sum of outer products, accumulating.

UMOP4S (2-way): Unsigned 16-bit integer quarter-tile sum of outer products to 32-bit integer, subtracting.

UMOP4S (4-way): Unsigned integer quarter-tile sum of outer products, subtracting.

UMOPA (2-way): Unsigned 16-bit integer sum of outer products to 32-bit integer, accumulating.

UMOPA (4-way): Unsigned integer sum of outer products, accumulating.

UMOPS (2-way): Unsigned 16-bit integer sum of outer products to 32-bit integer, subtracting.

UMOPS (4-way): Unsigned integer sum of outer products, subtracting.

UQCVT (four registers): Multi-vector unsigned saturating extract narrow.

UQCVT (two registers): Multi-vector unsigned 32-bit integer saturating extract narrow to 16-bit integer.

UQCVTN: Multi-vector unsigned saturating extract narrow to interleaved integer.

UQRSHR (four registers): Multi-vector unsigned saturating rounding shift right narrow by immediate.

UQRSHR (two registers): Multi-vector unsigned 32-bit integer saturating rounding shift right narrow by immediate to 16-bit integer.

UQRSHRN: Multi-vector unsigned saturating rounding shift right narrow by immediate to interleaved integer.

URSHL (multiple and single vector): Multi-vector unsigned rounding shift left by vector.

URSHL (multiple vectors): Multi-vector unsigned rounding shift left.

USDOT (4-way, multiple and indexed vector): Multi-vector unsigned by signed 8-bit integer dot product by indexed element to 32-bit integer.

USDOT (4-way, multiple and single vector): Multi-vector unsigned by signed 8-bit integer dot product by vector to 32-bit integer.

USDOT (4-way, multiple vectors): Multi-vector unsigned by signed 8-bit integer dot product to 32-bit integer.

USMLALL (multiple and indexed vector): Multi-vector unsigned by signed 8-bit integer multiply-add by indexed element to 32-bit integer.

USMLALL (multiple and single vector): Multi-vector unsigned by signed 8-bit integer multiply-add by vector to 32-bit integer.

USMLALL (multiple vectors): Multi-vector unsigned by signed 8-bit integer multiply-add to 32-bit integer.

USMOP4A: Unsigned by signed integer quarter-tile sum of outer products, accumulating.

USMOP4S: Unsigned by signed integer quarter-tile sum of outer products, subtracting.

USMOPA (4-way): Unsigned by signed integer sum of outer products, accumulating.

USMOPS: Unsigned by signed integer sum of outer products, subtracting.

USTMOPA: Unsigned by signed 8-bit integer sparse sum of outer products to 32-bit integer, accumulating.

USVDOT: Multi-vector unsigned by signed 8-bit integer vertical dot product by indexed element to 32-bit integer.

UTMOPA (2-way): Unsigned 16-bit integer sparse sum of outer products to 32-bit integer, accumulating.

UTMOPA (4-way): Unsigned 8-bit integer sparse sum of outer products to 32-bit integer, accumulating.

UUNPK: Unpack and zero-extend multi-vector elements.

UVDOT (2-way): Multi-vector unsigned 16-bit integer vertical dot product by indexed element to 32-bit integer.

UVDOT (4-way): Multi-vector unsigned integer vertical dot product by indexed element.

UZP (four registers): Concatenate elements from four vectors.

UZP (two registers): Concatenate elements from two vectors.

ZERO (double-vector): Zero ZA double-vector groups.

ZERO (quad-vector): Zero ZA quad-vector groups.

ZERO (single-vector): Zero ZA single-vector groups.

ZERO (table): Zero ZT0.

ZERO (tiles): Zero a list of 64-bit element ZA tiles.

ZIP (four registers): Interleave elements from four vectors.

ZIP (two registers): Interleave elements from two vectors.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

BFCLAMP

Multi-vector BFloat16 clamp to minimum/maximum number

This instruction clamps each BFloat16 element in the two or four destination vectors to between the BFloat16 minimum value in the corresponding element of the first source vector and the BFloat16 maximum value in the corresponding element of the second source vector and destructively places the clamped results in the corresponding elements of the two or four destination vectors.

The~~Regardless of the value of FPCR.AH, the~~ behavior is as follows for each minimum number and maximum number operation:

- Negative zero compares less than positive zero.
- If one value is numeric and the other is a quiet NaN, the quiet NaN value is ignored and the result is the numeric value.
- When FPCR.DN is 0, if either value is a signaling NaN or if both values are NaNs, the result is a quiet NaN derived as follows:~~NaN.~~
  - If FPCR.AH is 0:
    - If at least one value is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN value.
    - Otherwise, the result is a quiet NaN derived from the first NaN value.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN value.
- When FPCR.DN is 1, if either value is a signaling NaN or if both values are NaNs, the result is default NaN.

This instruction follows SME2 non-widening BFloat16 numerical behaviors corresponding to instructions that place their results in two or four SVE Z vectors.

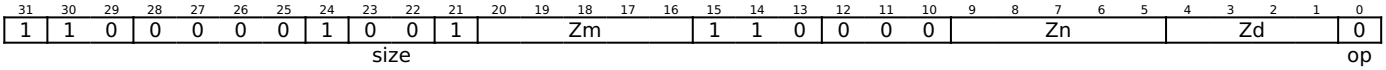
This instruction is unpredicated.

ID_AA64ZFR0_EL1.B16B16 indicates whether this instruction is implemented.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 && FEAT_SVE_B16B16)



Encoding

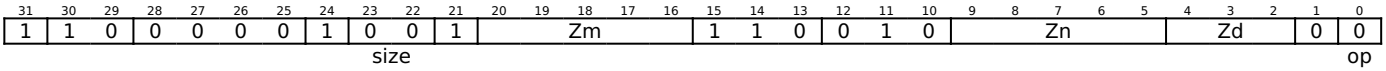
```
BFCLAMP { <Zd1>.H-<Zd2>.H }, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_SVE_B16B16) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd::'0');
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2 && FEAT_SVE_B16B16)



Encoding

```
BFCLAMP { <Zd1>.H-<Zd4>.H }, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_SVE_B16B16) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd::'00');
let nreg : integer{} = 4;
```

Assembler Symbols

<Zd1>	For the "Two registers" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 4.
<Zd2>	Is the name of the second scalable vector register of the destination multi-vector group, encoded as "Zd" times 2 plus 1.

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

<Zd4> Is the name of the fourth scalable vector register of the destination multi-vector group, encoded as "Zd" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 16;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n);
    let operand2 : bits(VL) = Z{VL}(m);
    let operand3 : bits(VL) = Z{VL}(d+r);
    for e = 0 to elements-1 do
        let element1 : bits(16) = operand1[e*:16];
        let element2 : bits(16) = operand2[e*:16];
        let element3 : bits(16) = operand3[e*:16];
        let maxelement : bits(16) = BFMaxNum{16}(element1, element3, FPCR());
        results[[r]][e*:16] = BFMinNum{16}(maxelement, element2, FPCR());
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(d+r) = results[[r]];
end;
```

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

BFMAX (multiple and single vector)

Multi-vector BFloat16 maximum by vector

This instruction determines the maximum of BFloat16 elements of the second source vector and the corresponding BFloat16 elements of the two or four first source vectors and destructively places the results in the corresponding elements of the two or four first source vectors.

When FPCR.AH is 0, the behavior is as follows:

- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:~~NaN.~~
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows:

- If both elements are zeros, regardless of the sign of either zero, the result is the second element.
- If either element is a NaN, regardless of the value of FPCR.DN, the result is the second element.

This instruction follows SME2 non-widening BFloat16 numerical behaviors corresponding to instructions that place their results in two or four SVE Z vectors.

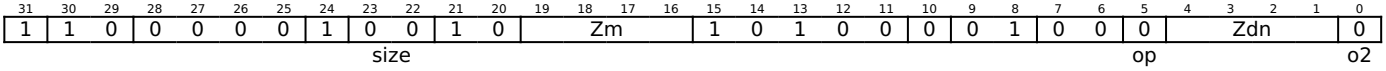
This instruction is unpredicated.

ID_AA64ZFR0_EL1.B16B16 indicates whether this instruction is implemented.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 && FEAT_SVE_B16B16)



Encoding

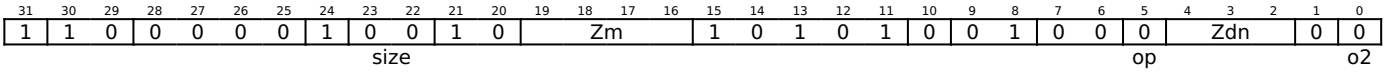
```
BFMAX { <Zdn1>.H-<Zdn2>.H }, { <Zdn1>.H-<Zdn2>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_SVE_B16B16) then
    EndOfDecode(Decode_UNDEF);
end;
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2 && FEAT_SVE_B16B16)



Encoding

```
BFMAX { <Zdn1>.H-<Zdn4>.H }, { <Zdn1>.H-<Zdn4>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_SVE_B16B16) then
    EndOfDecode(Decode_UNDEF);
end;
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1>	For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.
<Zdn2>	Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.

<Zm> Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.

<Zdn4> Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 16;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m);
    for e = 0 to elements-1 do
        let element1 : bits(16) = operand1[e*:16];
        let element2 : bits(16) = operand2[e*:16];
        results[[r]][e*:16] = BFMax{16}(element1, element2, FPCR());
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

BFMAX (multiple vectors)

Multi-vector BFloat16 maximum

This instruction determines the maximum of BFloat16 elements of the two or four second source vectors and the corresponding BFloat16 elements of the two or four first source vectors and destructively places the results in the corresponding elements of the two or four first source vectors.

When FPCR.AH is 0, the behavior is as follows:

- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:~~NaN.~~
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows:

- If both elements are zeros, regardless of the sign of either zero, the result is the second element.
- If either element is a NaN, regardless of the value of FPCR.DN, the result is the second element.

This instruction follows SME2 non-widening BFloat16 numerical behaviors corresponding to instructions that place their results in two or four SVE Z vectors.

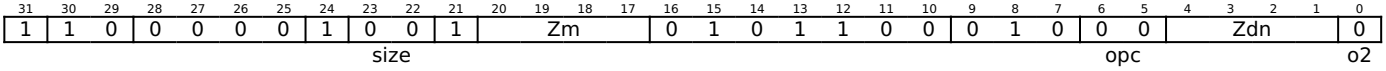
This instruction is unpredicated.

ID_AA64ZFR0_EL1.B16B16 indicates whether this instruction is implemented.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 && FEAT_SVE_B16B16)



Encoding

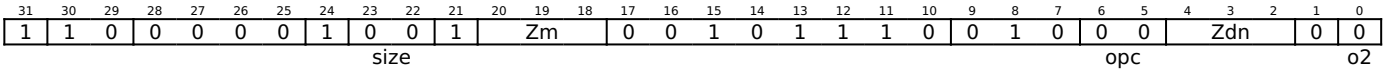
```
BFMAX { <Zdn1>.H-<Zdn2>.H }, { <Zdn1>.H-<Zdn2>.H }, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_SVE_B16B16) then
    EndOfDecode(Decode_UNDEF);
end;
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt(Zm::'0');
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2 && FEAT_SVE_B16B16)



Encoding

```
BFMAX { <Zdn1>.H-<Zdn4>.H }, { <Zdn1>.H-<Zdn4>.H }, { <Zm1>.H-<Zm4>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_SVE_B16B16) then
    EndOfDecode(Decode_UNDEF);
end;
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt(Zm::'00');
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1>	For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.
<Zdn2>	Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.

<Zm1>	For the "Two registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.
<Zdn4>	Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.
<Zm4>	Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 16;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for e = 0 to elements-1 do
        let element1 : bits(16) = operand1[e*:16];
        let element2 : bits(16) = operand2[e*:16];
        results[[r]][e*:16] = BFMax{16}(element1, element2, FPCR());
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

BFMAXNM (multiple and single vector)

Multi-vector BFloat16 maximum number by vector

This instruction determines the maximum number value of BFloat16 elements of the second source vector and the corresponding BFloat16 elements of the two or four first source vectors and destructively places the results in the corresponding elements of the two or four first source vectors.

TheRegardless of the value of FPCR.AH, the behavior is as follows:

- Negative zero compares less than positive zero.
- If one element is numeric and the other is a quiet NaN, the quiet NaN element is ignored and the result is the numeric element. value.
- When FPCR.DN is 0, if either element is a signaling NaN or if both elements are NaNs, the result is a quiet NaN derived as follows:NaN.
  - If FPCR.AH is 0:
    - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
    - Otherwise, the result is a quiet NaN derived from the first NaN element.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a signaling NaN or if both elements are NaNs, the result is default NaN.

This instruction follows SME2 non-widening BFloat16 numerical behaviors corresponding to instructions that place their results in two or four SVE Z vectors.

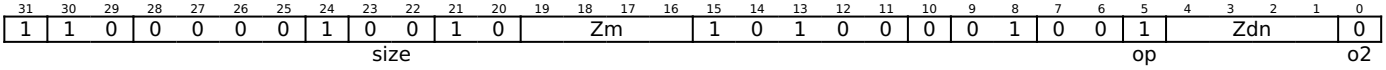
This instruction is unpredicated.

ID_AA64ZFR0_EL1.B16B16 indicates whether this instruction is implemented.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 && FEAT_SVE_B16B16)



Encoding

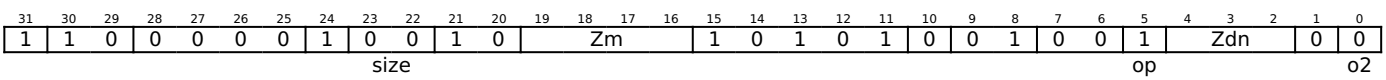
```
BFMAXNM { <Zdn1>.H-<Zdn2>.H }, { <Zdn1>.H-<Zdn2>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_SVE_B16B16) then
    EndOfDecode(Decode_UNDEF);
end;
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2 && FEAT_SVE_B16B16)



Encoding

```
BFMAXNM { <Zdn1>.H-<Zdn4>.H }, { <Zdn1>.H-<Zdn4>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_SVE_B16B16) then
    EndOfDecode(Decode_UNDEF);
end;
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1>	For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.

<Zdn2> Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.

<Zm> Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.

<Zdn4> Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 16;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m);
    for e = 0 to elements-1 do
        let element1 : bits(16) = operand1[e*:16];
        let element2 : bits(16) = operand2[e*:16];
        results[[r]][e*:16] = BFMaxNum{16}(element1, element2, FPCR());
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

BFMAXNM (multiple vectors)

Multi-vector BFloat16 maximum number

This instruction determines the maximum number value of BFloat16 elements of the two or four second source vectors and the corresponding BFloat16 elements of the two or four first source vectors and destructively places the results in the corresponding elements of the two or four first source vectors.

TheRegardless of the value of FPCR.AH, the behavior is as follows:

- Negative zero compares less than positive zero.
- If one element is numeric and the other is a quiet NaN, the quiet NaN element is ignored and the result is the numeric element. value.
- When FPCR.DN is 0, if either element is a signaling NaN or if both elements are NaNs, the result is a quiet NaN derived as follows:NaN.
  - If FPCR.AH is 0:
    - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
    - Otherwise, the result is a quiet NaN derived from the first NaN element.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a signaling NaN or if both elements are NaNs, the result is default NaN.

This instruction follows SME2 non-widening BFloat16 numerical behaviors corresponding to instructions that place their results in two or four SVE Z vectors.

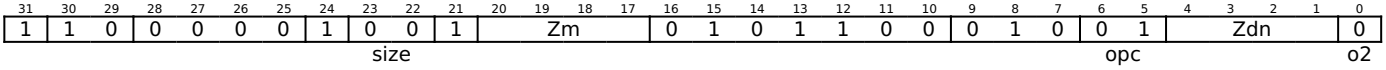
This instruction is unpredicated.

ID_AA64ZFR0_EL1.B16B16 indicates whether this instruction is implemented.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 && FEAT_SVE_B16B16)



Encoding

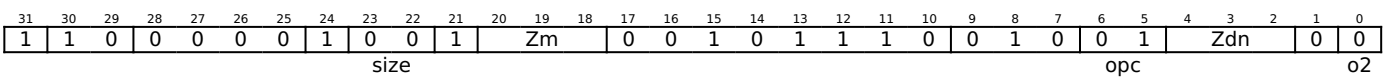
```
BFMAXNM { <Zdn1>.H-<Zdn2>.H }, { <Zdn1>.H-<Zdn2>.H }, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_SVE_B16B16) then
    EndOfDecode(Decode_UNDEF);
end;
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt(Zm::'0');
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2 && FEAT_SVE_B16B16)



Encoding

```
BFMAXNM { <Zdn1>.H-<Zdn4>.H }, { <Zdn1>.H-<Zdn4>.H }, { <Zm1>.H-<Zm4>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_SVE_B16B16) then
    EndOfDecode(Decode_UNDEF);
end;
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt(Zm::'00');
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1> For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.

<Zdn2> Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.

<Zm1> For the "Two registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.

<Zm2> Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zdn4> Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

<Zm4> Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 16;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for e = 0 to elements-1 do
        let element1 : bits(16) = operand1[e*:16];
        let element2 : bits(16) = operand2[e*:16];
        results[[r]][e*:16] = BFMaxNum{16}(element1, element2, FPCR());
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

BFMIN (multiple and single vector)

Multi-vector BFloat16 minimum by vector

This instruction determines the minimum of BFloat16 elements of the second source vector and the corresponding BFloat16 elements of the two or four first source vectors and destructively places the results in the corresponding elements of the two or four first source vectors.

When FPCR.AH is 0, the behavior is as follows:

- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:~~NaN.~~
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows:

- If both elements are zeros, regardless of the sign of either zero, the result is the second element.
- If either element is a NaN, regardless of the value of FPCR.DN, the result is the second element.

This instruction follows SME2 non-widening BFloat16 numerical behaviors corresponding to instructions that place their results in two or four SVE Z vectors.

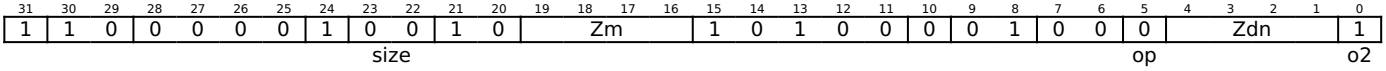
This instruction is unpredicated.

ID_AA64ZFR0_EL1.B16B16 indicates whether this instruction is implemented.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 && FEAT_SVE_B16B16)



Encoding

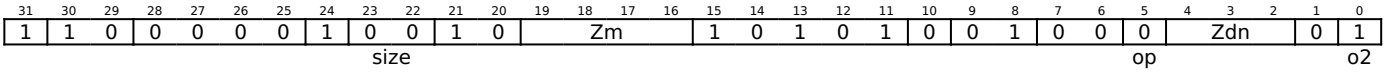
```
BFMIN { <Zdn1>.H-<Zdn2>.H }, { <Zdn1>.H-<Zdn2>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_SVE_B16B16) then
    EndOfDecode(Decode_UNDEF);
end;
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2 && FEAT_SVE_B16B16)



Encoding

```
BFMIN { <Zdn1>.H-<Zdn4>.H }, { <Zdn1>.H-<Zdn4>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_SVE_B16B16) then
    EndOfDecode(Decode_UNDEF);
end;
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1>	For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.
<Zdn2>	Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.

<Zm> Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.

<Zdn4> Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 16;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m);
    for e = 0 to elements-1 do
        let element1 : bits(16) = operand1[e*:16];
        let element2 : bits(16) = operand2[e*:16];
        results[[r]][e*:16] = BFMin{16}(element1, element2, FPCR());
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

BFMIN (multiple vectors)

Multi-vector BFloat16 minimum

This instruction determines the minimum of BFloat16 elements of the two or four second source vectors and the corresponding BFloat16 elements of the two or four first source vectors and destructively places the results in the corresponding elements of the two or four first source vectors.

When FPCR.AH is 0, the behavior is as follows:

- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:~~NaN.~~
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows:

- If both elements are zeros, regardless of the sign of either zero, the result is the second element.
- If either element is a NaN, regardless of the value of FPCR.DN, the result is the second element.

This instruction follows SME2 non-widening BFloat16 numerical behaviors corresponding to instructions that place their results in two or four SVE Z vectors.

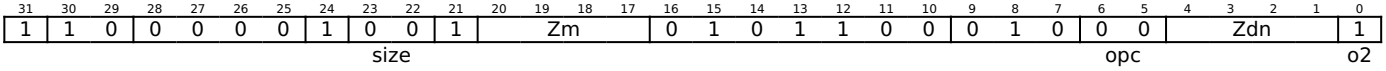
This instruction is unpredicated.

ID_AA64ZFR0_EL1.B16B16 indicates whether this instruction is implemented.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 && FEAT_SVE_B16B16)



Encoding

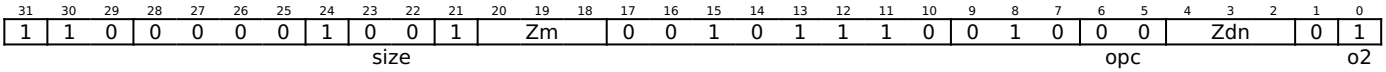
```
BFMIN { <Zdn1>.H-<Zdn2>.H }, { <Zdn1>.H-<Zdn2>.H }, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_SVE_B16B16) then
    EndOfDecode(Decode_UNDEF);
end;
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt(Zm::'0');
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2 && FEAT_SVE_B16B16)



Encoding

```
BFMIN { <Zdn1>.H-<Zdn4>.H }, { <Zdn1>.H-<Zdn4>.H }, { <Zm1>.H-<Zm4>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_SVE_B16B16) then
    EndOfDecode(Decode_UNDEF);
end;
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt(Zm::'00');
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1>	For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.
<Zdn2>	Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.

<Zm1>	For the "Two registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.
<Zdn4>	Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.
<Zm4>	Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 16;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for e = 0 to elements-1 do
        let element1 : bits(16) = operand1[e*:16];
        let element2 : bits(16) = operand2[e*:16];
        results[[r]][e*:16] = BFMin{16}(element1, element2, FPCR());
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

BFMINNM (multiple and single vector)

Multi-vector BFloat16 minimum number by vector

This instruction determines the minimum number value of BFloat16 elements of the second source vector and the corresponding BFloat16 elements of the two or four first source vectors and destructively places the results in the corresponding elements of the two or four first source vectors.

TheRegardless of the value of FPCR.AH, the behavior is as follows:

- Negative zero compares less than positive zero.
- If one element is numeric and the other is a quiet NaN, the quiet NaN element is ignored and the result is the numeric element. value.
- When FPCR.DN is 0, if either element is a signaling NaN or if both elements are NaNs, the result is a quiet NaN derived as follows:NaN.
  - If FPCR.AH is 0:
    - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
    - Otherwise, the result is a quiet NaN derived from the first NaN element.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a signaling NaN or if both elements are NaNs, the result is default NaN.

This instruction follows SME2 non-widening BFloat16 numerical behaviors corresponding to instructions that place their results in two or four SVE Z vectors.

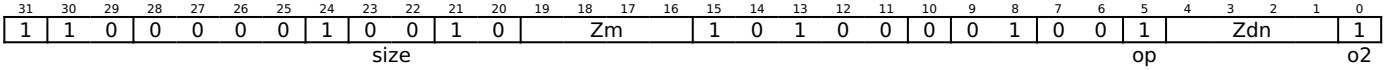
This instruction is unpredicated.

ID_AA64ZFR0_EL1.B16B16 indicates whether this instruction is implemented.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 && FEAT_SVE_B16B16)



Encoding

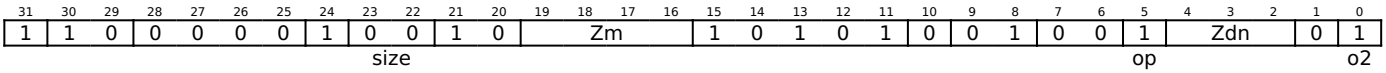
```
BFMINNM { <Zdn1>.H-<Zdn2>.H }, { <Zdn1>.H-<Zdn2>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_SVE_B16B16) then
    EndOfDecode(Decode_UNDEF);
end;
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2 && FEAT_SVE_B16B16)



Encoding

```
BFMINNM { <Zdn1>.H-<Zdn4>.H }, { <Zdn1>.H-<Zdn4>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_SVE_B16B16) then
    EndOfDecode(Decode_UNDEF);
end;
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1>	For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.
<Zdn2>	Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.

<Zm> Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.

<Zdn4> Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 16;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m);
    for e = 0 to elements-1 do
        let element1 : bits(16) = operand1[e*:16];
        let element2 : bits(16) = operand2[e*:16];
        results[[r]][e*:16] = BFMinNum{16}(element1, element2, FPCR());
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

BFMINNM (multiple vectors)

Multi-vector BFloat16 minimum number

This instruction determines the minimum number value of BFloat16 elements of the two or four second source vectors and the corresponding BFloat16 elements of the two or four first source vectors and destructively places the results in the corresponding elements of the two or four first source vectors.

TheRegardless of the value of FPCR.AH, the behavior is as follows:

- Negative zero compares less than positive zero.
- If one element is numeric and the other is a quiet NaN, the quiet NaN element is ignored and the result is the numeric element. value.
- When FPCR.DN is 0, if either element is a signaling NaN or if both elements are NaNs, the result is a quiet NaN derived as follows:NaN.
  - If FPCR.AH is 0:
    - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
    - Otherwise, the result is a quiet NaN derived from the first NaN element.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a signaling NaN or if both elements are NaNs, the result is default NaN.

This instruction follows SME2 non-widening BFloat16 numerical behaviors corresponding to instructions that place their results in two or four SVE Z vectors.

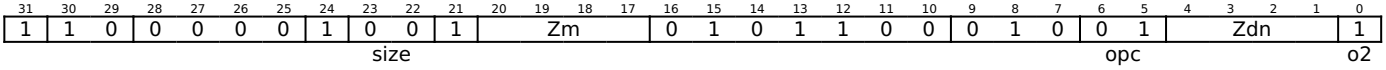
This instruction is unpredicated.

ID_AA64ZFR0_EL1.B16B16 indicates whether this instruction is implemented.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 && FEAT_SVE_B16B16)



Encoding

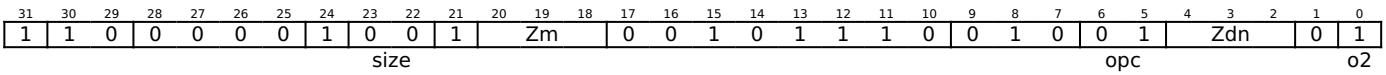
```
BFMINNM { <Zdn1>.H-<Zdn2>.H }, { <Zdn1>.H-<Zdn2>.H }, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_SVE_B16B16) then
    EndOfDecode(Decode_UNDEF);
end;
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt(Zm::'0');
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2 && FEAT_SVE_B16B16)



Encoding

```
BFMINNM { <Zdn1>.H-<Zdn4>.H }, { <Zdn1>.H-<Zdn4>.H }, { <Zm1>.H-<Zm4>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_SVE_B16B16) then
    EndOfDecode(Decode_UNDEF);
end;
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt(Zm::'00');
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1>	For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.

<Zdn2> Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.

<Zm1> For the "Two registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.

<Zm2> Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zdn4> Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

<Zm4> Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 16;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for e = 0 to elements-1 do
        let element1 : bits(16) = operand1[e*:16];
        let element2 : bits(16) = operand2[e*:16];
        results[[r]][e*:16] = BFMinNum{16}(element1, element2, FPCR());
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

FAMAX

Multi-vector floating-point absolute maximum

This instruction determines the maximum absolute value from floating-point elements of the two or four second source vectors and the corresponding floating-point elements of the two or four first source vectors and destructively places the results in the corresponding elements of the two or four first source vectors.

TheRegardless of the value of FPCR.AH, the behavior is as follows:

- AlternateWhen floating-pointFPCR.DN behaviorsis do0, notif apply,either aselement ifis FPCR.AHa NaN, the result is 0.a quiet NaN.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is the default NaN, with the sign bit set to 0.
- Denormalized inputs and results are never flushed to zero, as if FPCR.{FZ, FZ16, FIZ} are all 0.
- Denormalized inputs never generate an Input Denormal floating-point exception.

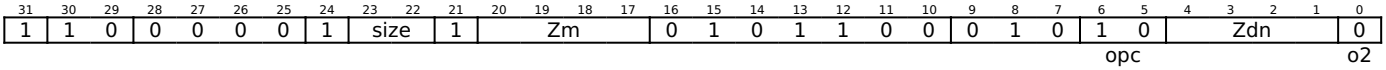
This instruction follows SME2 floating-point numerical behaviors corresponding to instructions that place their results in one or more SVE Z vectors.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 && FEAT_FAMINMAX)



Encoding

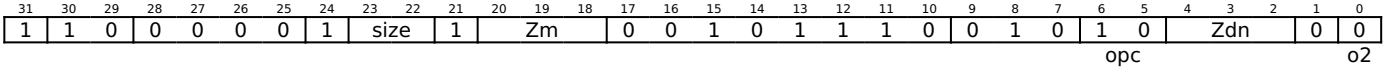
```
FAMAX { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zm1>.<T>-<Zm2>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_FAMINMAX) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt(Zm::'0');
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2 && FEAT_FAMINMAX)



Encoding

```
FAMAX { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zm1>.<T>-<Zm4>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_FAMINMAX) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt(Zm::'00');
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1> For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zdn2> Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.

<Zm1> For the "Two registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.

<Zm2> Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zdn4> Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

<Zm4> Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for e = 0 to elements-1 do
        let element1 : bits(esize) = operand1[e*:esize];
        let element2 : bits(esize) = operand2[e*:esize];
        results[[r]][e*:esize] = FPAbsMax{esize}(element1, element2, FPCR());
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

FAMIN

Multi-vector floating-point absolute minimum

This instruction determines the minimum absolute value from floating-point elements of the two or four second source vectors and the corresponding floating-point elements of the two or four first source vectors and destructively places the results in the corresponding elements of the two or four first source vectors.

TheRegardless of the value of FPCR.AH, the behavior is as follows:

- AlternateWhen floating-pointFPCR.DN behaviorsis do0, notif apply,either aselement ifis FPCR.AHa NaN, the result is 0.a quiet NaN.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is the default NaN, with the sign bit set to 0.
- Denormalized inputs and results are never flushed to zero, as if FPCR.{FZ, FZ16, FIZ} are all 0.
- Denormalized inputs never generate an Input Denormal floating-point exception.

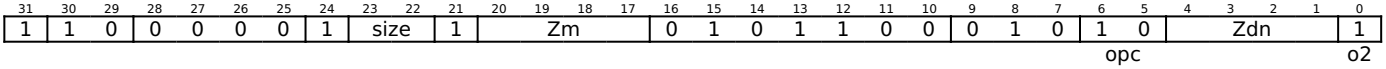
This instruction follows SME2 floating-point numerical behaviors corresponding to instructions that place their results in one or more SVE Z vectors.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 && FEAT_FAMINMAX)



Encoding

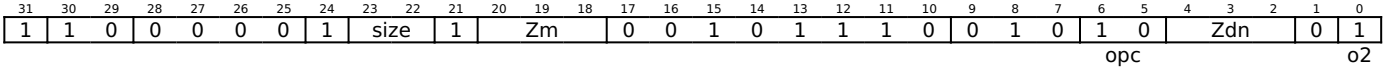
```
FAMIN { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zm1>.<T>-<Zm2>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_FAMINMAX) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt(Zm::'0');
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2 && FEAT_FAMINMAX)



Encoding

```
FAMIN { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zm1>.<T>-<Zm4>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_FAMINMAX) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt(Zm::'00');
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1> For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zdn2> Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.

<Zm1> For the "Two registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.

<Zm2> Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zdn4> Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

<Zm4> Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for e = 0 to elements-1 do
        let element1 : bits(esize) = operand1[e*:esize];
        let element2 : bits(esize) = operand2[e*:esize];
        results[[r]][e*:esize] = FPAbsMin{esize}(element1, element2, FPCR());
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

FCLAMP

Multi-vector floating-point clamp to minimum/maximum number

This instruction clamps each floating-point element in the two or four destination vectors to between the floating-point minimum value in the corresponding element of the first source vector and the floating-point maximum value in the corresponding element of the second source vector and destructively places the clamped results in the corresponding elements of the two or four destination vectors.

TheRegardless of the value of FPCR.AH, the behavior is as follows for each minimum number and maximum number operation:

- Negative zero compares less than positive zero.
- If one value is numeric and the other is a quiet NaN, the quiet NaN value is ignored and the result is the numeric value.
- When FPCR.DN is 0, if either value is a signaling NaN or if both values are NaNs, the result is a quiet NaN derived as follows:NaN.
  - If FPCR.AH is 0:
    - If at least one value is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN value.
    - Otherwise, the result is a quiet NaN derived from the first NaN value.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN value.
- When FPCR.DN is 1, if either value is a signaling NaN or if both values are NaNs, the result is default NaN.

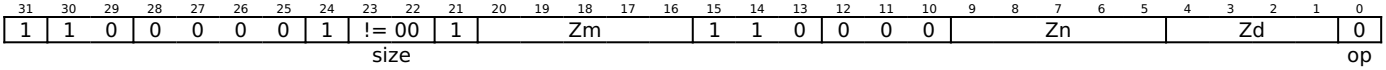
This instruction follows SME2 floating-point numerical behaviors corresponding to instructions that place their results in one or more SVE Z vectors.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

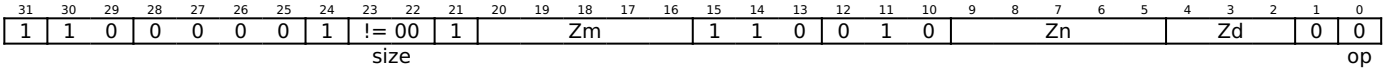
```
FCLAMP { <Zd1>.<T>-<Zd2>.<T> }, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd::'0');
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2)



Encoding

```
FCLAMP { <Zd1>.<T>-<Zd4>.<T> }, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd::'00');
let nreg : integer{} = 4;
```

Assembler Symbols

<Zd1> For the "Two registers" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
01	H
10	S
11	D

<Zd2> Is the name of the second scalable vector register of the destination multi-vector group, encoded as "Zd" times 2 plus 1.

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

<Zd4> Is the name of the fourth scalable vector register of the destination multi-vector group, encoded as "Zd" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n);
    let operand2 : bits(VL) = Z{VL}(m);
    let operand3 : bits(VL) = Z{VL}(d+r);
    for e = 0 to elements-1 do
        let element1 : bits(esize) = operand1[e*:esize];
        let element2 : bits(esize) = operand2[e*:esize];
        let element3 : bits(esize) = operand3[e*:esize];
        let maxelement : bits(esize) = FPMaxNum{esize}(element1, element3, FPCR());
        results[[r]][e*:esize] = FPMinNum{esize}(maxelement, element2, FPCR());
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(d+r) = results[[r]];
end;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

FMAX (multiple and single vector)

Multi-vector floating-point maximum by vector

This instruction determines the maximum of floating-point elements of the second source vector and the corresponding floating-point elements of the two or four first source vectors and destructively places the results in the corresponding elements of the two or four first source vectors.
When FPCR.AH is 0, the behavior is as follows:

- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:~~NaN.~~
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is default NaN.

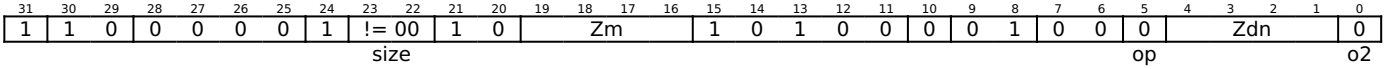
When FPCR.AH is 1, the behavior is as follows:

- If both elements are zeros, regardless of the sign of either zero, the result is the second element.
- If either element is a NaN, regardless of the value of FPCR.DN, the result is the second element.

This instruction follows SME2 floating-point numerical behaviors corresponding to instructions that place their results in one or more SVE Z vectors.
This instruction is unpredicated.
It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

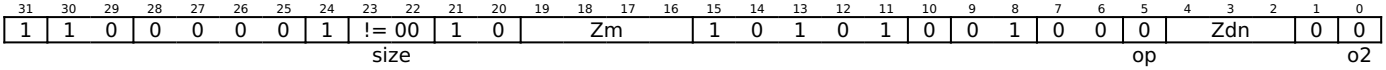
```
FMAX { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zdn1>.<T>-<Zdn2>.<T> }, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2)



Encoding

```
FMAX { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zdn1>.<T>-<Zdn4>.<T> }, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1> For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
01	H
10	S
11	D

<Zdn2> Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.

<Zm> Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.

<Zdn4> Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m);
    for e = 0 to elements-1 do
        let element1 : bits(esize) = operand1[e*:esize];
        let element2 : bits(esize) = operand2[e*:esize];
        results[[r]][e*:esize] = FPMax{esize}(element1, element2, FPCR());
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

FMAX (multiple vectors)

Multi-vector floating-point maximum

This instruction determines the maximum of floating-point elements of the two or four second source vectors and the corresponding floating-point elements of the two or four first source vectors and destructively places the results in the corresponding elements of the two or four first source vectors.

When FPCR.AH is 0, the behavior is as follows:

- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:~~NaN.~~
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows:

- If both elements are zeros, regardless of the sign of either zero, the result is the second element.
- If either element is a NaN, regardless of the value of FPCR.DN, the result is the second element.

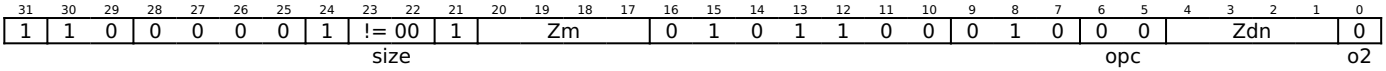
This instruction follows SME2 floating-point numerical behaviors corresponding to instructions that place their results in one or more SVE Z vectors.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

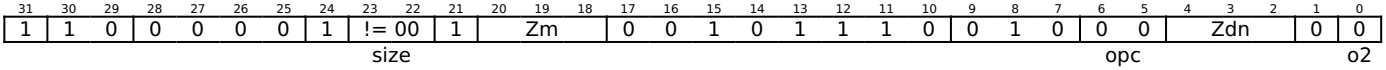
```
FMAX { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zm1>.<T>-<Zm2>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt(Zm::'0');
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2)



Encoding

```
FMAX { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zm1>.<T>-<Zm4>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt(Zm::'00');
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1> For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
01	H
10	S
11	D

<Zdn2> Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.

<Zm1> For the "Two registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.

<Zm2> Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zdn4> Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

<Zm4> Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for e = 0 to elements-1 do
        let element1 : bits(esize) = operand1[e*:esize];
        let element2 : bits(esize) = operand2[e*:esize];
        results[[r]][e*:esize] = FPMax{esize}(element1, element2, FPCR());
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

FMAXNM (multiple and single vector)

Multi-vector floating-point maximum number by vector

This instruction determines the maximum number value of floating-point elements of the second source vector and the corresponding floating-point elements of the two or four first source vectors and destructively places the results in the corresponding elements of the two or four first source vectors.

TheRegardless of the value of FPCR.AH, the behavior is as follows:

- Negative zero compares less than positive zero.
- If one element is numeric and the other is a quiet NaN, the quiet NaN element is ignored and the result is the numeric element. value.
- When FPCR.DN is 0, if either element is a signaling NaN or if both elements are NaNs, the result is a quiet NaN derived as follows:NaN.
  - If FPCR.AH is 0:
    - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
    - Otherwise, the result is a quiet NaN derived from the first NaN element.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a signaling NaN or if both elements are NaNs, the result is default NaN.

This instruction follows SME2 floating-point numerical behaviors corresponding to instructions that place their results in one or more SVE Z vectors.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	!= 00		1	0	Zm				1	0	1	0	0	0	0	1	0	0	1	Zdn				0
size																										op					o2

Encoding

```
FMAXNM { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zdn1>.<T>-<Zdn2>.<T> }, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	!= 00		1	0	Zm				1	0	1	0	1	0	0	1	0	0	1	Zdn			0	0
size																										op					o2

Encoding

```
FMAXNM { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zdn1>.<T>-<Zdn4>.<T> }, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1> For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
01	H
10	S
11	D

<Zdn2> Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.

<Zm> Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.

<Zdn4> Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m);
    for e = 0 to elements-1 do
        let element1 : bits(esize) = operand1[e*:esize];
        let element2 : bits(esize) = operand2[e*:esize];
        results[[r]][e*:esize] = FPMaxNum{esize}(element1, element2, FPCR());
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

FMAXNM (multiple vectors)

Multi-vector floating-point maximum number

This instruction determines the maximum number value of floating-point elements of the two or four second source vectors and the corresponding floating-point elements of the two or four first source vectors and destructively places the results in the corresponding elements of the two or four first source vectors.

TheRegardless of the value of FPCR.AH, the behavior is as follows:

- Negative zero compares less than positive zero.
- If one element is numeric and the other is a quiet NaN, the quiet NaN element is ignored and the result is the numeric element. value.
- When FPCR.DN is 0, if either element is a signaling NaN or if both elements are NaNs, the result is a quiet NaN derived as follows:NaN.
  - If FPCR.AH is 0:
    - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
    - Otherwise, the result is a quiet NaN derived from the first NaN element.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a signaling NaN or if both elements are NaNs, the result is default NaN.

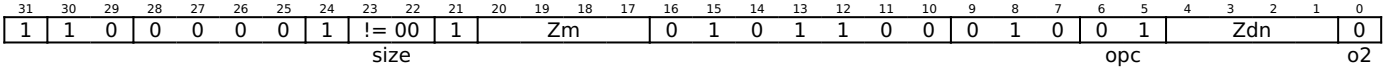
This instruction follows SME2 floating-point numerical behaviors corresponding to instructions that place their results in one or more SVE Z vectors.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

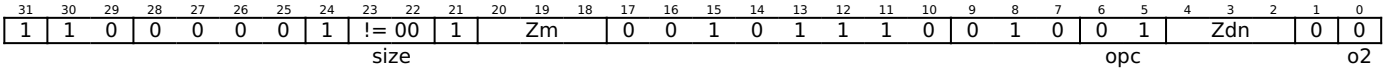
```
FMAXNM { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zm1>.<T>-<Zm2>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt(Zm::'0');
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2)



Encoding

```
FMAXNM { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zm1>.<T>-<Zm4>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt(Zm::'00');
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1> For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
01	H
10	S
11	D

<Zdn2> Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.

<Zm1> For the "Two registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.

<Zm2> Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zdn4> Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

<Zm4> Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for e = 0 to elements-1 do
        let element1 : bits(esize) = operand1[e*:esize];
        let element2 : bits(esize) = operand2[e*:esize];
        results[[r]][e*:esize] = FPMaxNum{esize}(element1, element2, FPCR());
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

FMIN (multiple and single vector)

Multi-vector floating-point minimum by vector

This instruction determines the minimum of floating-point elements of the second source vector and the corresponding floating-point elements of the two or four first source vectors and destructively places the results in the corresponding elements of the two or four first source vectors.
When FPCR.AH is 0, the behavior is as follows:

- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:~~NaN.~~
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is default NaN.

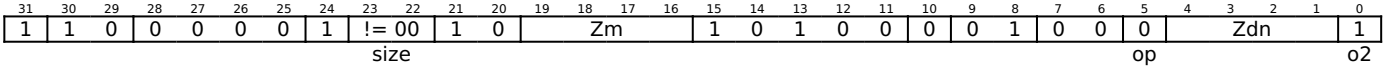
When FPCR.AH is 1, the behavior is as follows:

- If both elements are zeros, regardless of the sign of either zero, the result is the second element.
- If either element is a NaN, regardless of the value of FPCR.DN, the result is the second element.

This instruction follows SME2 floating-point numerical behaviors corresponding to instructions that place their results in one or more SVE Z vectors.
This instruction is unpredicated.
It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

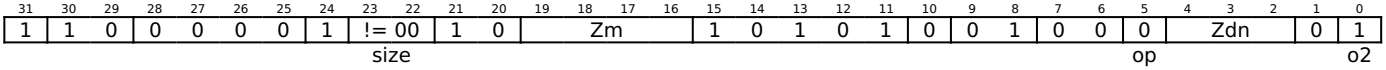
```
FMIN { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zdn1>.<T>-<Zdn2>.<T> }, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2)



Encoding

```
FMIN { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zdn1>.<T>-<Zdn4>.<T> }, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1> For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
01	H
10	S
11	D

<Zdn2> Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.

<Zm> Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.

<Zdn4> Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m);
    for e = 0 to elements-1 do
        let element1 : bits(esize) = operand1[e*:esize];
        let element2 : bits(esize) = operand2[e*:esize];
        results[[r]][e*:esize] = FPMin{esize}(element1, element2, FPCR());
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

FMIN (multiple vectors)

Multi-vector floating-point minimum

This instruction determines the minimum of floating-point elements of the two or four second source vectors and the corresponding floating-point elements of the two or four first source vectors and destructively places the results in the corresponding elements of the two or four first source vectors.

When FPCR.AH is 0, the behavior is as follows:

- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:~~NaN.~~
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows:

- If both elements are zeros, regardless of the sign of either zero, the result is the second element.
- If either element is a NaN, regardless of the value of FPCR.DN, the result is the second element.

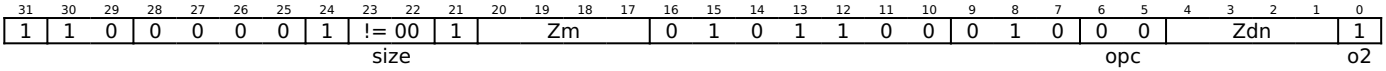
This instruction follows SME2 floating-point numerical behaviors corresponding to instructions that place their results in one or more SVE Z vectors.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

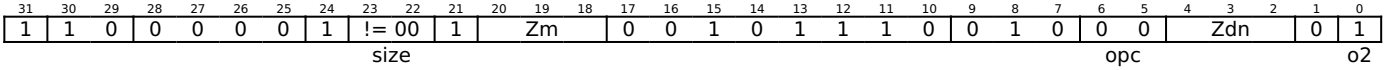
```
FMIN { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zm1>.<T>-<Zm2>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt(Zm::'0');
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2)



Encoding

```
FMIN { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zm1>.<T>-<Zm4>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt(Zm::'00');
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1> For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
01	H
10	S
11	D

<Zdn2> Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.

<Zm1> For the "Two registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.

<Zm2> Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zdn4> Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

<Zm4> Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for e = 0 to elements-1 do
        let element1 : bits(esize) = operand1[e*:esize];
        let element2 : bits(esize) = operand2[e*:esize];
        results[[r]][e*:esize] = FPMin{esize}(element1, element2, FPCR());
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

FMINNM (multiple and single vector)

Multi-vector floating-point minimum number by vector

This instruction determines the minimum number value of floating-point elements of the second source vector and the corresponding floating-point elements of the two or four first source vectors and destructively places the results in the corresponding elements of the two or four first source vectors.

TheRegardless of the value of FPCR.AH, the behavior is as follows:

- Negative zero compares less than positive zero.
- If one element is numeric and the other is a quiet NaN, the quiet NaN element is ignored and the result is the numeric element. value.
- When FPCR.DN is 0, if either element is a signaling NaN or if both elements are NaNs, the result is a quiet NaN derived as follows:NaN.
  - If FPCR.AH is 0:
    - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
    - Otherwise, the result is a quiet NaN derived from the first NaN element.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a signaling NaN or if both elements are NaNs, the result is default NaN.

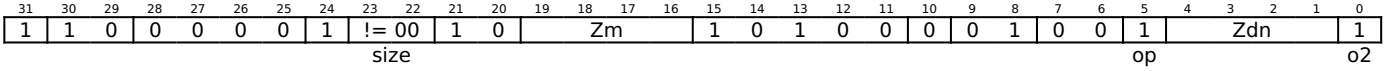
This instruction follows SME2 floating-point numerical behaviors corresponding to instructions that place their results in one or more SVE Z vectors.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

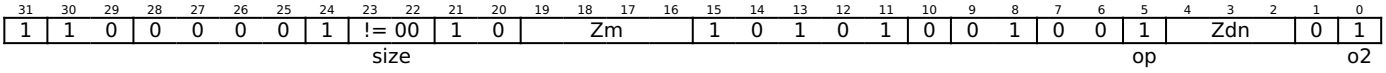
```
FMINNM { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zdn1>.<T>-<Zdn2>.<T> }, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2)



Encoding

```
FMINNM { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zdn1>.<T>-<Zdn4>.<T> }, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1> For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
01	H
10	S
11	D

<Zdn2> Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.

<Zm> Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.

<Zdn4> Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m);
    for e = 0 to elements-1 do
        let element1 : bits(esize) = operand1[e*:esize];
        let element2 : bits(esize) = operand2[e*:esize];
        results[[r]][e*:esize] = FPMinNum{esize}(element1, element2, FPCR());
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

(old) htmldiff from- (new)

FMINNM (multiple vectors)

Multi-vector floating-point minimum number

This instruction determines the minimum number value of floating-point elements of the two or four second source vectors and the corresponding floating-point elements of the two or four first source vectors and destructively places the results in the corresponding elements of the two or four first source vectors.

TheRegardless of the value of FPCR.AH, the behavior is as follows:

- Negative zero compares less than positive zero.
- If one element is numeric and the other is a quiet NaN, the quiet NaN element is ignored and the result is the numeric element. value.
- When FPCR.DN is 0, if either element is a signaling NaN or if both elements are NaNs, the result is a quiet NaN derived as follows:NaN.
  - If FPCR.AH is 0:
    - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
    - Otherwise, the result is a quiet NaN derived from the first NaN element.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a signaling NaN or if both elements are NaNs, the result is default NaN.

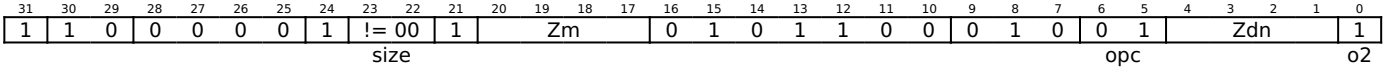
This instruction follows SME2 floating-point numerical behaviors corresponding to instructions that place their results in one or more SVE Z vectors.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

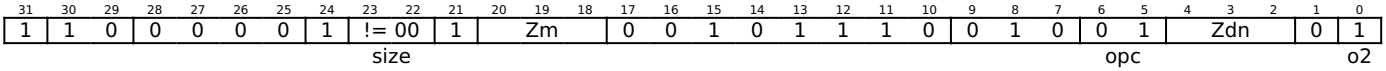
```
FMINNM { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zm1>.<T>-<Zm2>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt(Zm::'0');
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2)



Encoding

```
FMINNM { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zm1>.<T>-<Zm4>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt(Zm::'00');
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1> For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
01	H
10	S
11	D

<Zdn2> Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.

<Zm1> For the "Two registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.

<Zm2> Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zdn4> Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

<Zm4> Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for e = 0 to elements-1 do
        let element1 : bits(esize) = operand1[e*:esize];
        let element2 : bits(esize) = operand2[e*:esize];
        results[[r]][e*:esize] = FPMinNum{esize}(element1, element2, FPCR());
    end;
end;

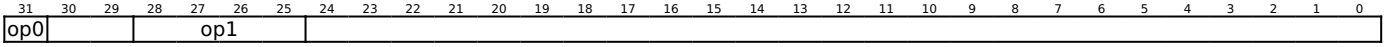
for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

2026-09_rel~~2026-06_diff_tag~~ 2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)

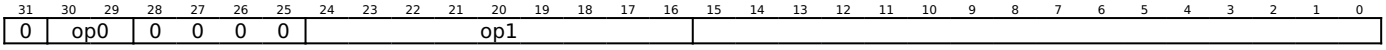
A64 instruction set encoding



Decode fields		Instruction details
op0	op1	
0	0000	Reserved
1	0000	SME encodings
	0010	SVE encodings
	00x1	UNALLOCATED
	100x	Data Processing -- Immediate
	101x	Branches, Exception Generating and System instructions
	x101	Data Processing -- Register
	x111	Data Processing -- Scalar Floating-Point and Advanced SIMD
	x1x0	Loads and Stores

Reserved

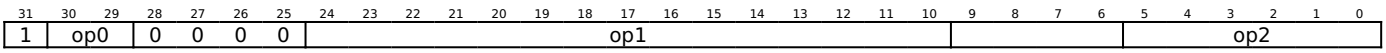
The encodings in this section are decoded from A64 instruction set encoding.



Decode fields		Instruction details
op0	op1	
00	000000000	UDF
00	!= 000000000	UNALLOCATED
!= 00		UNALLOCATED

SME encodings

The encodings in this section are decoded from A64 instruction set encoding.

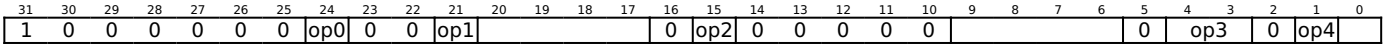


Decode fields			Instruction details
op0	op1	op2	
00	x00xxxxx0x00000	0xx0xx	SME2 Quarter Tile Outer Product - 16-bit and 32-bit
00	x00xxxxx0x00000	1xx0xx	UNALLOCATED
00	x00xxxxx0x00001	xxx0xx	UNALLOCATED
00	x00xxxxx0x0001x	xxx0xx	UNALLOCATED
00	x00xxxxx0x001xx	xxx0xx	UNALLOCATED
00	x00xxxxx0x01xxx	xxx0xx	UNALLOCATED
00	x00xxxxx1x0xxxx	xxx0xx	UNALLOCATED
00	x01xxxxxxx0xxxx	xxx0xx	SME2 Sparse Outer Product
00	x0xxxxxxxx0xxxx	xxx1xx	UNALLOCATED
00	x0xxxxxxxx1xxxx		UNALLOCATED
00	x10xxxxxxxxxxxx	xx00xx	SME FP Outer Product - 32 bit
00	x10xxxxxxxxxxxx	xx10xx	SME2 Outer Product - Misc
01	00xxxxxxxxxxxx		SME2 Multi-vector - Memory (Contiguous)
01	10xxxxxxxxxxxx		SME2 Multi-vector - Memory (Strided)
01	x10xxxxxxxxxxxx	xxx0xx	SME Integer Outer Product - 32 bit
0x	x10xxxxxxxxxxxx	xxx1xx	UNALLOCATED
0x	x11xxxxx0000000	0x1xxx	SME2 Quarter Tile Outer Product - 64-bit
0x	x11xxxxx0000000	1x1xxx	UNALLOCATED
0x	x11xxxxx0000001	xx1xxx	UNALLOCATED
0x	x11xxxxx000001x	xx1xxx	UNALLOCATED
0x	x11xxxxx00001xx	xx1xxx	UNALLOCATED
0x	x11xxxxx0001xxx	xx1xxx	UNALLOCATED
0x	x11xxxxx001xxxx	xx1xxx	UNALLOCATED
0x	x11xxxxx01xxxxx	xx1xxx	UNALLOCATED
0x	x11xxxxx1xxxxxx	xx1xxx	UNALLOCATED
0x	x11xxxxxxxxxxx	xx0xxx	SME Outer Product - 64 bit
10	0000010xxxxxxxx		SME zero array
10	0000011xxxxxxxx		SME2 Multiple Zero
10	0010010xxxxxxxx		SME2 zero lookup table
10	0010011xxxxxxxx		SME2 Move Lookup Table
10	00x011xxxxxxxxx		UNALLOCATED
10	010011xxxxxxxxx		SME2 Expand Lookup Table (Non-contiguous)
10	011011xxxxxxxxx		UNALLOCATED
10	01x001xxxxxxxxx		SME2 Expand Lookup Table (Contiguous)
10	0xx000x0xxxxxxx	x0xxxx	SME Move into Array
10	0xx000x0xxxxxxx	x1xxxx	UNALLOCATED
10	0xx000x1xxxxxxx		SME Move from Array
10	0xx010xxxxxxxxx	xx0xxx	SME Add Vector to Array
10	0xx010xxxxxxxxx	xx1xxx	UNALLOCATED
10	0xx1xxxxxxxxxx		UNALLOCATED
10	10x10xxxx0xxxxx		SME2 Multi-vector - Multiple and Single Array Vectors (Two registers)
10	10x11xxxx0xxxxx		SME2 Multi-vector - Multiple and Single Array Vectors (Four registers)
10	10x1xxxxx1111xx		SME2 Multi-vector - Lookup table with 6-bit indices
10	11x1xxxx00xxxxx		SME2 Multi-vector - Multiple Array Vectors (Two registers)
10	11x1xxxx10xxxxx		SME2 Multi-vector - Multiple Array Vectors (Four registers)

10	11x1xxxxx1111xx		UNALLOCATED
10	1xx00xxxxxxxxx		SME2 Multi-vector - Indexed (One register)
10	1xx01xxxx0xxxxx		SME2 Multi-vector - Indexed (Two registers)
10	1xx01xxxx1xxxxx		SME2 Multi-vector - Indexed (Four registers)
10	1xx10xxxx10100x		SME2 Multi-vector - Multiple and Single SVE Destructive (Two registers)
10	1xx10xxxx10101x	xxxx0x	SME2 Multi-vector - Multiple and Single SVE Destructive (Four registers)
10	1xx10xxxx10101x	xxxx1x	UNALLOCATED
10	1xx11xxxx1010xx		UNALLOCATED
10	1xx1xxx00101110	xxxx0x	SME2 Multi-vector - Multiple Vectors SVE Destructive (Four registers)
10	1xx1xxx00101111	xxxx0x	SME2 Multi-vector - Multiple Vectors SVE Saturating Multiply (Four registers)
10	1xx1xxx1010111x	xxxx0x	UNALLOCATED
10	1xx1xxxx0101100		SME2 Multi-vector - Multiple Vectors SVE Destructive (Two registers)
10	1xx1xxxx0101101		SME2 Multi-vector - Multiple Vectors SVE Saturating Multiply (Two registers)
10	1xx1xxxx010111x	xxxx1x	UNALLOCATED
10	1xx1xxxx11011xx		UNALLOCATED
10	1xx1xxxxx100xxx		SME2 Multi-vector - SVE Select
10	1xx1xxxxx110xxx		SME2 Multi-vector - SVE Constructive Binary
10	1xx1xxxxx111000		SME2 Multi-vector - SVE Constructive Unary
10	1xx1xxxxx111001	0xxxx0	SME2 Multi-vector - FP Multiply
10	1xx1xxxxx111001	!= 0xxxx0	UNALLOCATED
10	1xx1xxxxx111010	0xxxx0	SME2 Multiple and Single Vector - FP multiply
10	1xx1xxxxx111010	!= 0xxxx0	UNALLOCATED
10	1xx1xxxxx111011		UNALLOCATED
11			SME Memory

SME2 Quarter Tile Outer Product - 16-bit and 32-bit

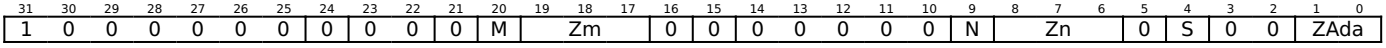
The encodings in this section are decoded from SME encodings.



Decode fields					Instruction details
op0	op1	op2	op3	op4	
0	0	0	x0		SME2 FP32 non-widening quarter tile outer product
0	0	0	x1	0	UNALLOCATED
0	1	0	00		SME2 FP8 to FP32 quarter tile outer product
0	1	0	01	0	SME2 FP8 to FP16 quarter tile outer product
0	1	0	10	1	UNALLOCATED
0	1	0	1x	0	UNALLOCATED
1	0	0	x0		SME2 BFloat16 to FP32 quarter tile outer product
1	0	0	x1	0	SME2 FP16 non-widening quarter tile outer product
1	1	0	x0		SME2 FP16 to FP32 quarter tile outer product
1	1	0	x1	0	SME2 BFloat16 non-widening quarter tile outer product
	0	1	x1		SME2 Int16 to Int32 quarter tile outer product
	1	1	x1		UNALLOCATED
		0	x1	1	UNALLOCATED
		1	x0		SME2 Int8 to Int32 quarter tile outer product

SME2 FP32 non-widening quarter tile outer product

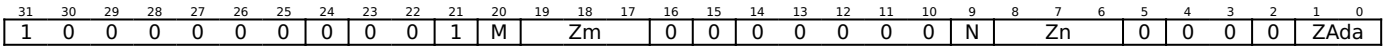
The encodings in this section are decoded from SME2 Quarter Tile Outer Product - 16-bit and 32-bit.



Decode fields M	N	S	Instruction Details	Condition
0	0	0	FMOP4A (non-widening) — Single-precision, single vectors	FEAT_SME_MOP4
0	0	1	FMOP4S (non-widening) — Single-precision, single vectors	FEAT_SME_MOP4
0	1	0	FMOP4A (non-widening) — Single-precision, multiple and single vectors	FEAT_SME_MOP4
0	1	1	FMOP4S (non-widening) — Single-precision, multiple and single vectors	FEAT_SME_MOP4
1	0	0	FMOP4A (non-widening) — Single-precision, single and multiple vectors	FEAT_SME_MOP4
1	0	1	FMOP4S (non-widening) — Single-precision, single and multiple vectors	FEAT_SME_MOP4
1	1	0	FMOP4A (non-widening) — Single-precision, multiple vectors	FEAT_SME_MOP4
1	1	1	FMOP4S (non-widening) — Single-precision, multiple vectors	FEAT_SME_MOP4

SME2 FP8 to FP32 quarter tile outer product

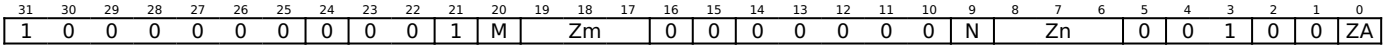
The encodings in this section are decoded from SME2 Quarter Tile Outer Product - 16-bit and 32-bit.



Decode fields M	N	Instruction Details	Condition
0	0	FMOP4A (widening, 4-way) — Single vectors	FEAT_SME_MOP4 && FEAT_SME_F8F32
0	1	FMOP4A (widening, 4-way) — Multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_F8F32
1	0	FMOP4A (widening, 4-way) — Single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_F8F32
1	1	FMOP4A (widening, 4-way) — Multiple vectors	FEAT_SME_MOP4 && FEAT_SME_F8F32

SME2 FP8 to FP16 quarter tile outer product

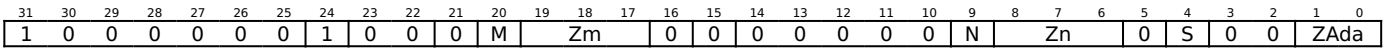
The encodings in this section are decoded from SME2 Quarter Tile Outer Product - 16-bit and 32-bit.



Decode fields M	N	Instruction Details	Condition
0	0	FMOP4A (widening, 2-way, FP8 to FP16) — Single vectors	FEAT_SME_MOP4 && FEAT_SME_F8F16
0	1	FMOP4A (widening, 2-way, FP8 to FP16) — Multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_F8F16
1	0	FMOP4A (widening, 2-way, FP8 to FP16) — Single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_F8F16
1	1	FMOP4A (widening, 2-way, FP8 to FP16) — Multiple vectors	FEAT_SME_MOP4 && FEAT_SME_F8F16

SME2 BFloat16 to FP32 quarter tile outer product

The encodings in this section are decoded from SME2 Quarter Tile Outer Product - 16-bit and 32-bit.



Decode fields			Instruction Details	Condition
M	N	S		
0	0	0	BFMOP4A (widening) — Single vectors	FEAT_SME_MOP4
0	0	1	BFMOP4S (widening) — Single vectors	FEAT_SME_MOP4
0	1	0	BFMOP4A (widening) — Multiple and single vectors	FEAT_SME_MOP4
0	1	1	BFMOP4S (widening) — Multiple and single vectors	FEAT_SME_MOP4
1	0	0	BFMOP4A (widening) — Single and multiple vectors	FEAT_SME_MOP4
1	0	1	BFMOP4S (widening) — Single and multiple vectors	FEAT_SME_MOP4
1	1	0	BFMOP4A (widening) — Multiple vectors	FEAT_SME_MOP4
1	1	1	BFMOP4S (widening) — Multiple vectors	FEAT_SME_MOP4

SME2 FP16 non-widening quarter tile outer product

The encodings in this section are decoded from SME2 Quarter Tile Outer Product - 16-bit and 32-bit.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	1	0	0	0	M	Zm			0	0	0	0	0	0	0	N	Zn			0	S	1	0	0	ZA

Decode fields			Instruction Details	Condition
M	N	S		
0	0	0	FMOP4A (non-widening) — Half-precision, single vectors	FEAT_SME_MOP4 && FEAT_SME_F16F16
0	0	1	FMOP4S (non-widening) — Half-precision, single vectors	FEAT_SME_MOP4 && FEAT_SME_F16F16
0	1	0	FMOP4A (non-widening) — Half-precision, multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_F16F16
0	1	1	FMOP4S (non-widening) — Half-precision, multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_F16F16
1	0	0	FMOP4A (non-widening) — Half-precision, single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_F16F16
1	0	1	FMOP4S (non-widening) — Half-precision, single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_F16F16
1	1	0	FMOP4A (non-widening) — Half-precision, multiple vectors	FEAT_SME_MOP4 && FEAT_SME_F16F16
1	1	1	FMOP4S (non-widening) — Half-precision, multiple vectors	FEAT_SME_MOP4 && FEAT_SME_F16F16

SME2 FP16 to FP32 quarter tile outer product

The encodings in this section are decoded from SME2 Quarter Tile Outer Product - 16-bit and 32-bit.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	1	0	0	1	M	Zm			0	0	0	0	0	0	0	N	Zn			0	S	0	0	ZAda	

Decode fields			Instruction Details	Condition
M	N	S		
0	0	0	FMOP4A (widening, 2-way, FP16 to FP32) — Single vectors	FEAT_SME_MOP4
0	0	1	FMOP4S (widening) — Single vectors	FEAT_SME_MOP4
0	1	0	FMOP4A (widening, 2-way, FP16 to FP32) — Multiple and single vectors	FEAT_SME_MOP4
0	1	1	FMOP4S (widening) — Multiple and single vectors	FEAT_SME_MOP4
1	0	0	FMOP4A (widening, 2-way, FP16 to FP32) — Single and multiple vectors	FEAT_SME_MOP4
1	0	1	FMOP4S (widening) — Single and multiple vectors	FEAT_SME_MOP4
1	1	0	FMOP4A (widening, 2-way, FP16 to FP32) — Multiple vectors	FEAT_SME_MOP4
1	1	1	FMOP4S (widening) — Multiple vectors	FEAT_SME_MOP4

SME2 BFloat16 non-widening quarter tile outer product

The encodings in this section are decoded from SME2 Quarter Tile Outer Product - 16-bit and 32-bit.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	1	0	0	1	M	Zm			0	0	0	0	0	0	0	N	Zn			0	S	1	0	0	ZA

Decode fields			Instruction Details	Condition
M	N	S		
0	0	0	BFMOP4A (non-widening) — Single vectors	FEAT_SME_MOP4 && FEAT_SME_B16B16
0	0	1	BFMOP4S (non-widening) — Single vectors	FEAT_SME_MOP4 && FEAT_SME_B16B16
0	1	0	BFMOP4A (non-widening) — Multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_B16B16
0	1	1	BFMOP4S (non-widening) — Multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_B16B16
1	0	0	BFMOP4A (non-widening) — Single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_B16B16
1	0	1	BFMOP4S (non-widening) — Single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_B16B16
1	1	0	BFMOP4A (non-widening) — Multiple vectors	FEAT_SME_MOP4 && FEAT_SME_B16B16
1	1	1	BFMOP4S (non-widening) — Multiple vectors	FEAT_SME_MOP4 && FEAT_SME_B16B16

SME2 Int16 to Int32 quarter tile outer product

The encodings in this section are decoded from SME2 Quarter Tile Outer Product - 16-bit and 32-bit.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	u0	0	0	0	M	Zm			0	1	0	0	0	0	0	N	Zn			0	S	1	0	ZAda	

Decode fields				Instruction Details	Condition
u0	M	N	S		
0	0	0	0	SMOP4A (2-way) — Single vectors	FEAT_SME_MOP4
0	0	0	1	SMOP4S (2-way) — Single vectors	FEAT_SME_MOP4
0	0	1	0	SMOP4A (2-way) — Multiple and single vectors	FEAT_SME_MOP4
0	0	1	1	SMOP4S (2-way) — Multiple and single vectors	FEAT_SME_MOP4
0	1	0	0	SMOP4A (2-way) — Single and multiple vectors	FEAT_SME_MOP4
0	1	0	1	SMOP4S (2-way) — Single and multiple vectors	FEAT_SME_MOP4
0	1	1	0	SMOP4A (2-way) — Multiple vectors	FEAT_SME_MOP4
0	1	1	1	SMOP4S (2-way) — Multiple vectors	FEAT_SME_MOP4
1	0	0	0	UMOP4A (2-way) — Single vectors	FEAT_SME_MOP4
1	0	0	1	UMOP4S (2-way) — Single vectors	FEAT_SME_MOP4
1	0	1	0	UMOP4A (2-way) — Multiple and single vectors	FEAT_SME_MOP4
1	0	1	1	UMOP4S (2-way) — Multiple and single vectors	FEAT_SME_MOP4
1	1	0	0	UMOP4A (2-way) — Single and multiple vectors	FEAT_SME_MOP4
1	1	0	1	UMOP4S (2-way) — Single and multiple vectors	FEAT_SME_MOP4
1	1	1	0	UMOP4A (2-way) — Multiple vectors	FEAT_SME_MOP4
1	1	1	1	UMOP4S (2-way) — Multiple vectors	FEAT_SME_MOP4

SME2 Int8 to Int32 quarter tile outer product

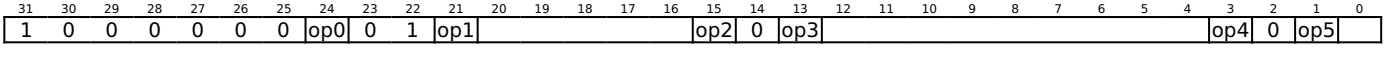
The encodings in this section are decoded from SME2 Quarter Tile Outer Product - 16-bit and 32-bit.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	u0	0	0	u1	M	Zm			0	1	0	0	0	0	0	N	Zn			0	S	0	0	ZAda	

Decode fields					Instruction Details	Condition
u0	u1	M	N	S		
0	0	0	0	0	SMOP4A (4-way) — 32-bit, single vectors	FEAT_SME_MOP4
0	0	0	0	1	SMOP4S (4-way) — 32-bit, single vectors	FEAT_SME_MOP4
0	0	0	1	0	SMOP4A (4-way) — 32-bit, multiple and single vectors	FEAT_SME_MOP4
0	0	0	1	1	SMOP4S (4-way) — 32-bit, multiple and single vectors	FEAT_SME_MOP4
0	0	1	0	0	SMOP4A (4-way) — 32-bit, single and multiple vectors	FEAT_SME_MOP4
0	0	1	0	1	SMOP4S (4-way) — 32-bit, single and multiple vectors	FEAT_SME_MOP4
0	0	1	1	0	SMOP4A (4-way) — 32-bit, multiple vectors	FEAT_SME_MOP4
0	0	1	1	1	SMOP4S (4-way) — 32-bit, multiple vectors	FEAT_SME_MOP4
0	1	0	0	0	SUMOP4A — 32-bit, single vectors	FEAT_SME_MOP4
0	1	0	0	1	SUMOP4S — 32-bit, single vectors	FEAT_SME_MOP4
0	1	0	1	0	SUMOP4A — 32-bit, multiple and single vectors	FEAT_SME_MOP4
0	1	0	1	1	SUMOP4S — 32-bit, multiple and single vectors	FEAT_SME_MOP4
0	1	1	0	0	SUMOP4A — 32-bit, single and multiple vectors	FEAT_SME_MOP4
0	1	1	0	1	SUMOP4S — 32-bit, single and multiple vectors	FEAT_SME_MOP4
0	1	1	1	0	SUMOP4A — 32-bit, multiple vectors	FEAT_SME_MOP4
0	1	1	1	1	SUMOP4S — 32-bit, multiple vectors	FEAT_SME_MOP4
1	0	0	0	0	USMOP4A — 32-bit, single vectors	FEAT_SME_MOP4
1	0	0	0	1	USMOP4S — 32-bit, single vectors	FEAT_SME_MOP4
1	0	0	1	0	USMOP4A — 32-bit, multiple and single vectors	FEAT_SME_MOP4
1	0	0	1	1	USMOP4S — 32-bit, multiple and single vectors	FEAT_SME_MOP4
1	0	1	0	0	USMOP4A — 32-bit, single and multiple vectors	FEAT_SME_MOP4
1	0	1	0	1	USMOP4S — 32-bit, single and multiple vectors	FEAT_SME_MOP4
1	0	1	1	0	USMOP4A — 32-bit, multiple vectors	FEAT_SME_MOP4
1	0	1	1	1	USMOP4S — 32-bit, multiple vectors	FEAT_SME_MOP4
1	1	0	0	0	UMOP4A (4-way) — 32-bit, single vectors	FEAT_SME_MOP4
1	1	0	0	1	UMOP4S (4-way) — 32-bit, single vectors	FEAT_SME_MOP4
1	1	0	1	0	UMOP4A (4-way) — 32-bit, multiple and single vectors	FEAT_SME_MOP4
1	1	0	1	1	UMOP4S (4-way) — 32-bit, multiple and single vectors	FEAT_SME_MOP4
1	1	1	0	0	UMOP4A (4-way) — 32-bit, single and multiple vectors	FEAT_SME_MOP4
1	1	1	0	1	UMOP4S (4-way) — 32-bit, single and multiple vectors	FEAT_SME_MOP4
1	1	1	1	0	UMOP4A (4-way) — 32-bit, multiple vectors	FEAT_SME_MOP4
1	1	1	1	1	UMOP4S (4-way) — 32-bit, multiple vectors	FEAT_SME_MOP4

SME2 Sparse Outer Product

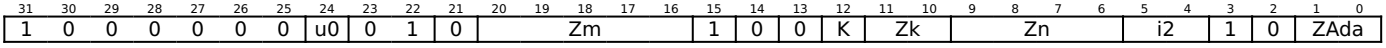
The encodings in this section are decoded from SME encodings.



Decode fields						Instruction details	Condition
op0	op1	op2	op3	op4	op5		
0	0	0	0	0		FTMOPA (non-widening) — Single-precision	FEAT_SME_TMOP
0	0	0	0	1	0	UNALLOCATED	-
0	1	0	0	0		FTMOPA (widening, 4-way)	FEAT_SME_TMOP && FEAT_SME_F8F32
0	1	0	0	1	0	FTMOPA (widening, 2-way, FP8 to FP16)	FEAT_SME_TMOP && FEAT_SME_F8F16
1	0	0	0	0		BFTMOPA (widening)	FEAT_SME_TMOP
1	0	0	0	1	0	FTMOPA (non-widening) — Half-precision	FEAT_SME_TMOP && FEAT_SME_F16F16
1	1	0	0	0		FTMOPA (widening, 2-way, FP16 to FP32)	FEAT_SME_TMOP
1	1	0	0	1	0	BFTMOPA (non-widening)	FEAT_SME_TMOP && FEAT_SME_B16B16
	0	1	0	1		SME2 Int16 to Int32 sparse outer product	-
	1	1	0	1		UNALLOCATED	-
		0	0	1	1	UNALLOCATED	-
		1	0	0		SME Int8 to Int32 sparse outer product	-
			1			UNALLOCATED	-

SME2 Int16 to Int32 sparse outer product

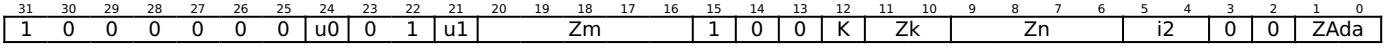
The encodings in this section are decoded from SME2 Sparse Outer Product.



Decode fields	Instruction Details	Condition
u0		
0	STMOPA (2-way)	FEAT_SME_TMOP
1	UTMOPA (2-way)	FEAT_SME_TMOP

SME Int8 to Int32 sparse outer product

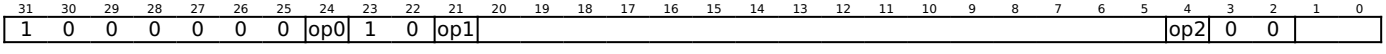
The encodings in this section are decoded from SME2 Sparse Outer Product.



Decode fields		Instruction Details	Condition
u0	u1		
0	0	STMOPA (4-way)	FEAT_SME_TMOP
0	1	SUTMOPA	FEAT_SME_TMOP
1	0	USTMOPA	FEAT_SME_TMOP
1	1	UTMOPA (4-way)	FEAT_SME_TMOP

SME FP Outer Product - 32 bit

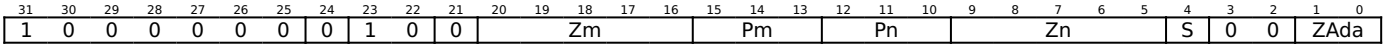
The encodings in this section are decoded from SME encodings.



Decode fields			Instruction details	Condition
op0	op1	op2		
0	0		SME FP32 outer product	-
0	1	0	FMOPA (widening, 4-way)	FEAT_SME_F8F32
0	1	1	UNALLOCATED	-
1	0		SME BFloat16 widening outer product	-
1	1		SME FP16 widening outer product	-

SME FP32 outer product

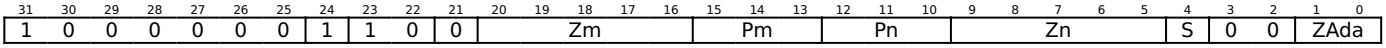
The encodings in this section are decoded from SME FP Outer Product - 32 bit.



Decode fields	Instruction Details	Condition
S		
0	FMOPA (non-widening)	FEAT_SME
1	FMOPS (non-widening)	FEAT_SME

SME BFloat16 widening outer product

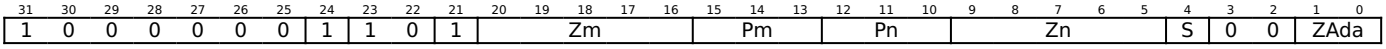
The encodings in this section are decoded from SME FP Outer Product - 32 bit.



Decode fields	Instruction Details	Condition
S		
0	BFMOPA (widening)	FEAT_SME
1	BFMOPS (widening)	FEAT_SME

SME FP16 widening outer product

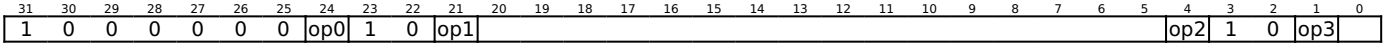
The encodings in this section are decoded from SME FP Outer Product - 32 bit.



Decode fields	Instruction Details	Condition
S		
0	FMOPA (widening, 2-way, FP16 to FP32)	FEAT_SME
1	FMOPS (widening)	FEAT_SME

SME2 Outer Product - Misc

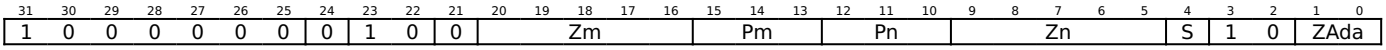
The encodings in this section are decoded from SME encodings.



Decode fields				Instruction details	Condition
op0	op1	op2	op3		
0	0			SME2 32-bit binary outer product	-
0	1	0	0	FMOPA (widening, 2-way, FP8 to FP16)	FEAT_SME_F8F16
0	1	1	0	UNALLOCATED	-
0	1		1	UNALLOCATED	-
1	0		0	SME2 FP16 non-widening outer product	-
1	1		0	SME2 BFloat16 non-widening outer product	-
1			1	UNALLOCATED	-

SME2 32-bit binary outer product

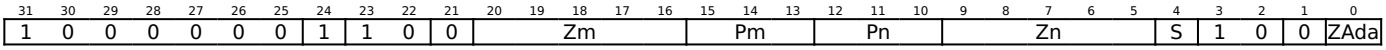
The encodings in this section are decoded from SME2 Outer Product - Misc.



Decode fields	Instruction Details	Condition
S		
0	BMOPA	FEAT_SME2
1	BMOPS	FEAT_SME2

SME2 FP16 non-widening outer product

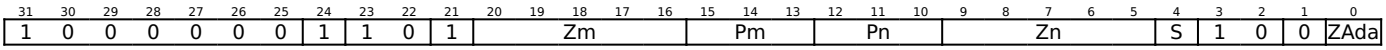
The encodings in this section are decoded from SME2 Outer Product - Misc.



Decode fields	Instruction Details	Condition
S		
0	FMOPA (non-widening)	FEAT_SME_F16F16
1	FMOPS (non-widening)	FEAT_SME_F16F16

SME2 BFloat16 non-widening outer product

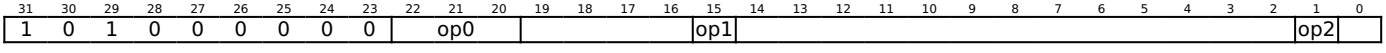
The encodings in this section are decoded from SME2 Outer Product - Misc.



Decode fields	Instruction Details	Condition
S		
0	BFMOPA (non-widening)	FEAT_SME_B16B16
1	BFMOPS (non-widening)	FEAT_SME_B16B16

SME2 Multi-vector - Memory (Contiguous)

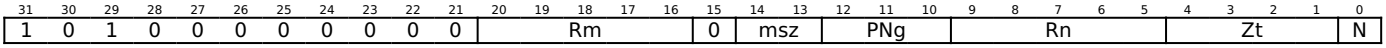
The encodings in this section are decoded from SME encodings.



Decode fields			Instruction details
op0	op1	op2	
00x	0		SME2 multi-vec contiguous load (scalar plus scalar, two registers)
00x	1	0	SME2 multi-vec contiguous load (scalar plus scalar, four registers)
01x	0		SME2 multi-vec contiguous store (scalar plus scalar, two registers)
01x	1	0	SME2 multi-vec contiguous store (scalar plus scalar, four registers)
100	0		SME2 multi-vec contiguous load (scalar plus immediate, two registers)
100	1	0	SME2 multi-vec contiguous load (scalar plus immediate, four registers)
110	0		SME2 multi-vec contiguous store (scalar plus immediate, two registers)
110	1	0	SME2 multi-vec contiguous store (scalar plus immediate, four registers)
1x1			UNALLOCATED
!= 1x1	1	1	UNALLOCATED

SME2 multi-vec contiguous load (scalar plus scalar, two registers)

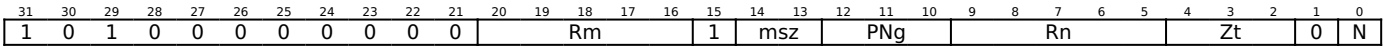
The encodings in this section are decoded from SME2 Multi-vector - Memory (Contiguous).



Decode fields		Instruction Details	Condition
msz	N		
00	0	LD1B (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
00	1	LDNT1B (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
01	0	LD1H (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
01	1	LDNT1H (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
10	0	LD1W (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
10	1	LDNT1W (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
11	0	LD1D (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
11	1	LDNT1D (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1

SME2 multi-vec contiguous load (scalar plus scalar, four registers)

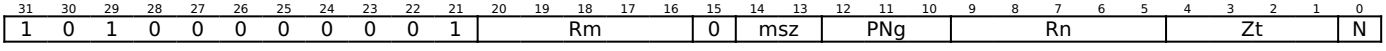
The encodings in this section are decoded from SME2 Multi-vector - Memory (Contiguous).



Decode fields		Instruction Details	Condition
msz	N		
00	0	LD1B (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
00	1	LDNT1B (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
01	0	LD1H (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
01	1	LDNT1H (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
10	0	LD1W (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
10	1	LDNT1W (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
11	0	LD1D (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
11	1	LDNT1D (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1

SME2 multi-vec contiguous store (scalar plus scalar, two registers)

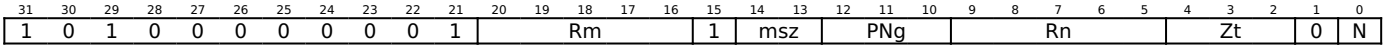
The encodings in this section are decoded from SME2 Multi-vector - Memory (Contiguous).



Decode fields msz	N	Instruction Details	Condition
00	0	ST1B (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
00	1	STNT1B (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
01	0	ST1H (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
01	1	STNT1H (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
10	0	ST1W (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
10	1	STNT1W (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
11	0	ST1D (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
11	1	STNT1D (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1

SME2 multi-vec contiguous store (scalar plus scalar, four registers)

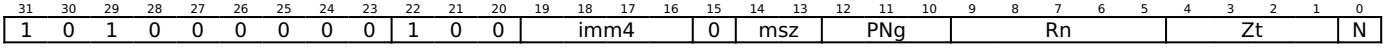
The encodings in this section are decoded from SME2 Multi-vector - Memory (Contiguous).



Decode fields msz	N	Instruction Details	Condition
00	0	ST1B (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
00	1	STNT1B (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
01	0	ST1H (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
01	1	STNT1H (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
10	0	ST1W (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
10	1	STNT1W (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
11	0	ST1D (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
11	1	STNT1D (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1

SME2 multi-vec contiguous load (scalar plus immediate, two registers)

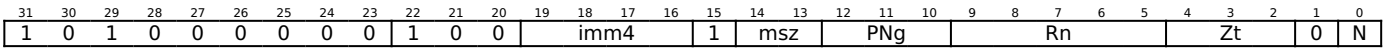
The encodings in this section are decoded from SME2 Multi-vector - Memory (Contiguous).



Decode fields msz	N	Instruction Details	Condition
00	0	LD1B (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
00	1	LDNT1B (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
01	0	LD1H (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
01	1	LDNT1H (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
10	0	LD1W (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
10	1	LDNT1W (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
11	0	LD1D (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
11	1	LDNT1D (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1

SME2 multi-vec contiguous load (scalar plus immediate, four registers)

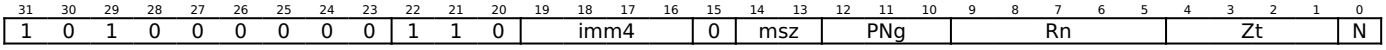
The encodings in this section are decoded from SME2 Multi-vector - Memory (Contiguous).



Decode fields msz	N	Instruction Details	Condition
00	0	LD1B (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
00	1	LDNT1B (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
01	0	LD1H (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
01	1	LDNT1H (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
10	0	LD1W (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
10	1	LDNT1W (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
11	0	LD1D (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
11	1	LDNT1D (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1

SME2 multi-vec contiguous store (scalar plus immediate, two registers)

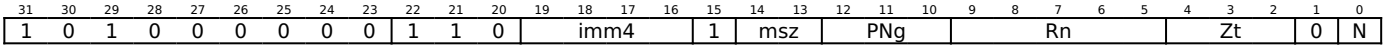
The encodings in this section are decoded from SME2 Multi-vector - Memory (Contiguous).



Decode fields msz	N	Instruction Details	Condition
00	0	ST1B (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
00	1	STNT1B (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
01	0	ST1H (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
01	1	STNT1H (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
10	0	ST1W (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
10	1	STNT1W (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
11	0	ST1D (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
11	1	STNT1D (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1

SME2 multi-vec contiguous store (scalar plus immediate, four registers)

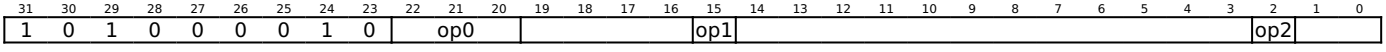
The encodings in this section are decoded from SME2 Multi-vector - Memory (Contiguous).



Decode fields msz	N	Instruction Details	Condition
00	0	ST1B (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
00	1	STNT1B (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
01	0	ST1H (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
01	1	STNT1H (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
10	0	ST1W (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
10	1	STNT1W (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
11	0	ST1D (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
11	1	STNT1D (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1

SME2 Multi-vector - Memory (Strided)

The encodings in this section are decoded from SME encodings.



Decode fields			Instruction details
op0	op1	op2	
00x	0		SME2 multi-vec non-contiguous load (scalar plus scalar, two registers)
00x	1	0	SME2 multi-vec non-contiguous load (scalar plus scalar, four registers)
01x	0		SME2 multi-vec non-contiguous store (scalar plus scalar, two registers)
01x	1	0	SME2 multi-vec non-contiguous store (scalar plus scalar, four registers)
100	0		SME2 multi-vec non-contiguous load (scalar plus immediate, two registers)
100	1	0	SME2 multi-vec non-contiguous load (scalar plus immediate, four registers)
110	0		SME2 multi-vec non-contiguous store (scalar plus immediate, two registers)
110	1	0	SME2 multi-vec non-contiguous store (scalar plus immediate, four registers)
1x1			UNALLOCATED
!= 1x1	1	1	UNALLOCATED

SME2 multi-vec non-contiguous load (scalar plus scalar, two registers)

The encodings in this section are decoded from SME2 Multi-vector - Memory (Strided).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	0	0	1	0	0	0	Rm					0	msz		PNg			Rn					T	N	Zt		

Decode fields		Instruction Details	Condition
msz	N		
00	0	LD1B (scalar plus scalar, strided registers)	FEAT_SME2
00	1	LDNT1B (scalar plus scalar, strided registers)	FEAT_SME2
01	0	LD1H (scalar plus scalar, strided registers)	FEAT_SME2
01	1	LDNT1H (scalar plus scalar, strided registers)	FEAT_SME2
10	0	LD1W (scalar plus scalar, strided registers)	FEAT_SME2
10	1	LDNT1W (scalar plus scalar, strided registers)	FEAT_SME2
11	0	LD1D (scalar plus scalar, strided registers)	FEAT_SME2
11	1	LDNT1D (scalar plus scalar, strided registers)	FEAT_SME2

SME2 multi-vec non-contiguous load (scalar plus scalar, four registers)

The encodings in this section are decoded from SME2 Multi-vector - Memory (Strided).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	0	0	1	0	0	0	Rm					1	msz		PNg			Rn					T	N	0	Zt	

Decode fields		Instruction Details	Condition
msz	N		
00	0	LD1B (scalar plus scalar, strided registers)	FEAT_SME2
00	1	LDNT1B (scalar plus scalar, strided registers)	FEAT_SME2
01	0	LD1H (scalar plus scalar, strided registers)	FEAT_SME2
01	1	LDNT1H (scalar plus scalar, strided registers)	FEAT_SME2
10	0	LD1W (scalar plus scalar, strided registers)	FEAT_SME2
10	1	LDNT1W (scalar plus scalar, strided registers)	FEAT_SME2
11	0	LD1D (scalar plus scalar, strided registers)	FEAT_SME2
11	1	LDNT1D (scalar plus scalar, strided registers)	FEAT_SME2

SME2 multi-vec non-contiguous store (scalar plus scalar, two registers)

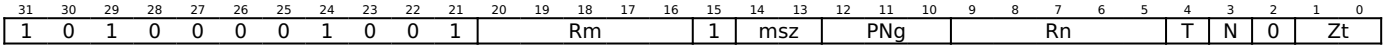
The encodings in this section are decoded from SME2 Multi-vector - Memory (Strided).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	0	0	1	0	0	1	Rm					0	msz		PNg			Rn					T	N	Zt		

Decode fields		Instruction Details	Condition
msz	N		
00	0	ST1B (scalar plus scalar, strided registers)	FEAT_SME2
00	1	STNT1B (scalar plus scalar, strided registers)	FEAT_SME2
01	0	ST1H (scalar plus scalar, strided registers)	FEAT_SME2
01	1	STNT1H (scalar plus scalar, strided registers)	FEAT_SME2
10	0	ST1W (scalar plus scalar, strided registers)	FEAT_SME2
10	1	STNT1W (scalar plus scalar, strided registers)	FEAT_SME2
11	0	ST1D (scalar plus scalar, strided registers)	FEAT_SME2
11	1	STNT1D (scalar plus scalar, strided registers)	FEAT_SME2

SME2 multi-vec non-contiguous store (scalar plus scalar, four registers)

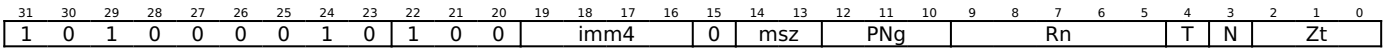
The encodings in this section are decoded from SME2 Multi-vector - Memory (Strided).



Decode fields		Instruction Details	Condition
msz	N		
00	0	ST1B (scalar plus scalar, strided registers)	FEAT_SME2
00	1	STNT1B (scalar plus scalar, strided registers)	FEAT_SME2
01	0	ST1H (scalar plus scalar, strided registers)	FEAT_SME2
01	1	STNT1H (scalar plus scalar, strided registers)	FEAT_SME2
10	0	ST1W (scalar plus scalar, strided registers)	FEAT_SME2
10	1	STNT1W (scalar plus scalar, strided registers)	FEAT_SME2
11	0	ST1D (scalar plus scalar, strided registers)	FEAT_SME2
11	1	STNT1D (scalar plus scalar, strided registers)	FEAT_SME2

SME2 multi-vec non-contiguous load (scalar plus immediate, two registers)

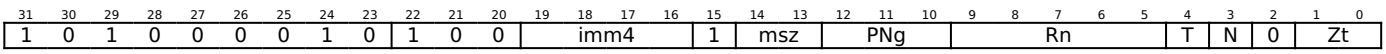
The encodings in this section are decoded from SME2 Multi-vector - Memory (Strided).



Decode fields		Instruction Details	Condition
msz	N		
00	0	LD1B (scalar plus immediate, strided registers)	FEAT_SME2
00	1	LDNT1B (scalar plus immediate, strided registers)	FEAT_SME2
01	0	LD1H (scalar plus immediate, strided registers)	FEAT_SME2
01	1	LDNT1H (scalar plus immediate, strided registers)	FEAT_SME2
10	0	LD1W (scalar plus immediate, strided registers)	FEAT_SME2
10	1	LDNT1W (scalar plus immediate, strided registers)	FEAT_SME2
11	0	LD1D (scalar plus immediate, strided registers)	FEAT_SME2
11	1	LDNT1D (scalar plus immediate, strided registers)	FEAT_SME2

SME2 multi-vec non-contiguous load (scalar plus immediate, four registers)

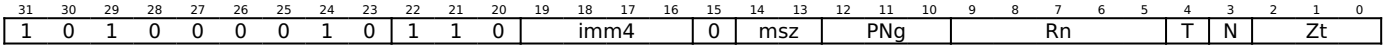
The encodings in this section are decoded from SME2 Multi-vector - Memory (Strided).



Decode fields		Instruction Details	Condition
msz	N		
00	0	LD1B (scalar plus immediate, strided registers)	FEAT_SME2
00	1	LDNT1B (scalar plus immediate, strided registers)	FEAT_SME2
01	0	LD1H (scalar plus immediate, strided registers)	FEAT_SME2
01	1	LDNT1H (scalar plus immediate, strided registers)	FEAT_SME2
10	0	LD1W (scalar plus immediate, strided registers)	FEAT_SME2
10	1	LDNT1W (scalar plus immediate, strided registers)	FEAT_SME2
11	0	LD1D (scalar plus immediate, strided registers)	FEAT_SME2
11	1	LDNT1D (scalar plus immediate, strided registers)	FEAT_SME2

SME2 multi-vec non-contiguous store (scalar plus immediate, two registers)

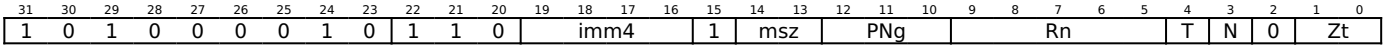
The encodings in this section are decoded from SME2 Multi-vector - Memory (Strided).



Decode fields		Instruction Details	Condition
msz	N		
00	0	ST1B (scalar plus immediate, strided registers)	FEAT_SME2
00	1	STNT1B (scalar plus immediate, strided registers)	FEAT_SME2
01	0	ST1H (scalar plus immediate, strided registers)	FEAT_SME2
01	1	STNT1H (scalar plus immediate, strided registers)	FEAT_SME2
10	0	ST1W (scalar plus immediate, strided registers)	FEAT_SME2
10	1	STNT1W (scalar plus immediate, strided registers)	FEAT_SME2
11	0	ST1D (scalar plus immediate, strided registers)	FEAT_SME2
11	1	STNT1D (scalar plus immediate, strided registers)	FEAT_SME2

SME2 multi-vec non-contiguous store (scalar plus immediate, four registers)

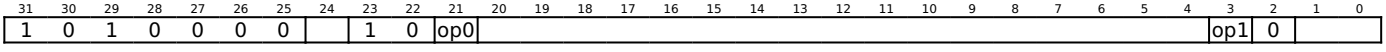
The encodings in this section are decoded from SME2 Multi-vector - Memory (Strided).



Decode fields		Instruction Details	Condition
msz	N		
00	0	ST1B (scalar plus immediate, strided registers)	FEAT_SME2
00	1	STNT1B (scalar plus immediate, strided registers)	FEAT_SME2
01	0	ST1H (scalar plus immediate, strided registers)	FEAT_SME2
01	1	STNT1H (scalar plus immediate, strided registers)	FEAT_SME2
10	0	ST1W (scalar plus immediate, strided registers)	FEAT_SME2
10	1	STNT1W (scalar plus immediate, strided registers)	FEAT_SME2
11	0	ST1D (scalar plus immediate, strided registers)	FEAT_SME2
11	1	STNT1D (scalar plus immediate, strided registers)	FEAT_SME2

SME Integer Outer Product - 32 bit

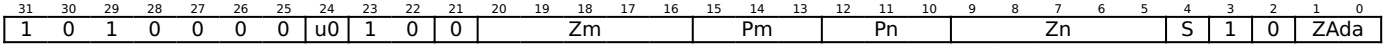
The encodings in this section are decoded from SME encodings.



Decode fields		Instruction details
op0	op1	
0	1	SME2 Int16 two-way outer product
1	1	UNALLOCATED
	0	SME Int8 outer product

SME2 Int16 two-way outer product

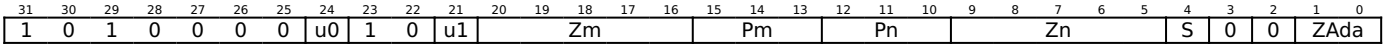
The encodings in this section are decoded from SME Integer Outer Product - 32 bit.



Decode fields		Instruction Details	Condition
u0	S		
0	0	SMOPA (2-way)	FEAT_SME2
0	1	SMOPS (2-way)	FEAT_SME2
1	0	UMOPA (2-way)	FEAT_SME2
1	1	UMOPS (2-way)	FEAT_SME2

SME Int8 outer product

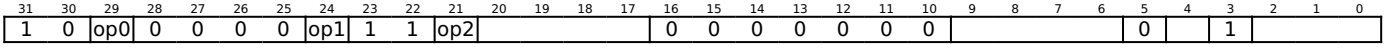
The encodings in this section are decoded from SME Integer Outer Product - 32 bit.



Decode fields			Instruction Details	Condition
u0	u1	S		
0	0	0	SMOPA (4-way)	FEAT_SME
0	0	1	SMOPS (4-way)	FEAT_SME
0	1	0	SUMOPA (4-way)	FEAT_SME
0	1	1	SUMOPS	FEAT_SME
1	0	0	USMOPA (4-way)	FEAT_SME
1	0	1	USMOPS	FEAT_SME
1	1	0	UMOPA (4-way)	FEAT_SME
1	1	1	UMOPS (4-way)	FEAT_SME

SME2 Quarter Tile Outer Product - 64-bit

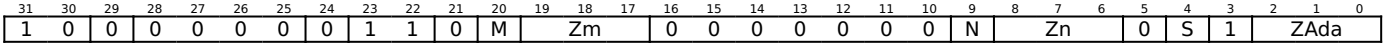
The encodings in this section are decoded from SME encodings.



Decode fields			Instruction details
op0	op1	op2	
0	0	0	SME2 FP64 non-widening quarter tile outer product
0	0	1	UNALLOCATED
0	1		UNALLOCATED
1			SME2 Int16 to Int64 quarter tile outer product

SME2 FP64 non-widening quarter tile outer product

The encodings in this section are decoded from SME2 Quarter Tile Outer Product - 64-bit.



Decode fields			Instruction Details	Condition
M	N	S		
0	0	0	FMOP4A (non-widening) — Double-precision, single vectors	FEAT_SME_MOP4 && FEAT_SME_F64F64
0	0	1	FMOP4S (non-widening) — Double-precision, single vectors	FEAT_SME_MOP4 && FEAT_SME_F64F64
0	1	0	FMOP4A (non-widening) — Double-precision, multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_F64F64
0	1	1	FMOP4S (non-widening) — Double-precision, multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_F64F64
1	0	0	FMOP4A (non-widening) — Double-precision, single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_F64F64
1	0	1	FMOP4S (non-widening) — Double-precision, single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_F64F64
1	1	0	FMOP4A (non-widening) — Double-precision, multiple vectors	FEAT_SME_MOP4 && FEAT_SME_F64F64
1	1	1	FMOP4S (non-widening) — Double-precision, multiple vectors	FEAT_SME_MOP4 && FEAT_SME_F64F64

SME2 Int16 to Int64 quarter tile outer product

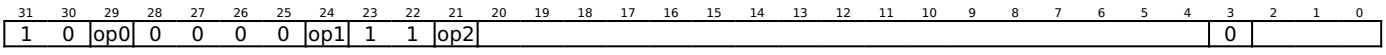
The encodings in this section are decoded from SME2 Quarter Tile Outer Product - 64-bit.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	0	0	u0	1	1	u1	M	Zm			0	0	0	0	0	0	0	N	Zn			0	S	1	ZAda		

Decode fields					Instruction Details	Condition
u0	u1	M	N	S		
0	0	0	0	0	SMOP4A (4-way) — 64-bit, single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
0	0	0	0	1	SMOP4S (4-way) — 64-bit, single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
0	0	0	1	0	SMOP4A (4-way) — 64-bit, multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
0	0	0	1	1	SMOP4S (4-way) — 64-bit, multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
0	0	1	0	0	SMOP4A (4-way) — 64-bit, single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
0	0	1	0	1	SMOP4S (4-way) — 64-bit, single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
0	0	1	1	0	SMOP4A (4-way) — 64-bit, multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
0	0	1	1	1	SMOP4S (4-way) — 64-bit, multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
0	1	0	0	0	SUMOP4A — 64-bit, single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
0	1	0	0	1	SUMOP4S — 64-bit, single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
0	1	0	1	0	SUMOP4A — 64-bit, multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
0	1	0	1	1	SUMOP4S — 64-bit, multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
0	1	1	0	0	SUMOP4A — 64-bit, single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
0	1	1	0	1	SUMOP4S — 64-bit, single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
0	1	1	1	0	SUMOP4A — 64-bit, multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
0	1	1	1	1	SUMOP4S — 64-bit, multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	0	0	0	0	USMOP4A — 64-bit, single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	0	0	0	1	USMOP4S — 64-bit, single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	0	0	1	0	USMOP4A — 64-bit, multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	0	0	1	1	USMOP4S — 64-bit, multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	0	1	0	0	USMOP4A — 64-bit, single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	0	1	0	1	USMOP4S — 64-bit, single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	0	1	1	0	USMOP4A — 64-bit, multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	0	1	1	1	USMOP4S — 64-bit, multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	1	0	0	0	UMOP4A (4-way) — 64-bit, single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	1	0	0	1	UMOP4S (4-way) — 64-bit, single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	1	0	1	0	UMOP4A (4-way) — 64-bit, multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	1	0	1	1	UMOP4S (4-way) — 64-bit, multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	1	1	0	0	UMOP4A (4-way) — 64-bit, single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	1	1	0	1	UMOP4S (4-way) — 64-bit, single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	1	1	1	0	UMOP4A (4-way) — 64-bit, multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	1	1	1	1	UMOP4S (4-way) — 64-bit, multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64

SME Outer Product - 64 bit

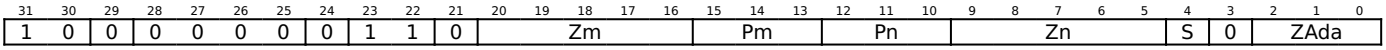
The encodings in this section are decoded from SME encodings.



Decode fields			Instruction details
op0	op1	op2	
0	0	0	SME FP64 outer product
0	0	1	UNALLOCATED
0	1		UNALLOCATED
1			SME Int16 outer product

SME FP64 outer product

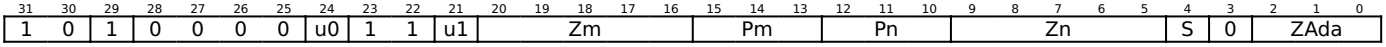
The encodings in this section are decoded from SME Outer Product - 64 bit.



Decode fields S	Instruction Details	Condition
0	FMOPA (non-widening)	FEAT_SME_F64F64
1	FMOPS (non-widening)	FEAT_SME_F64F64

SME Int16 outer product

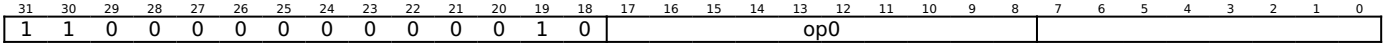
The encodings in this section are decoded from SME Outer Product - 64 bit.



Decode fields u0	u1	S	Instruction Details	Condition
0	0	0	SMOPA (4-way)	FEAT_SME_I16I64
0	0	1	SMOPS (4-way)	FEAT_SME_I16I64
0	1	0	SUMOPA (4-way)	FEAT_SME_I16I64
0	1	1	SUMOPS	FEAT_SME_I16I64
1	0	0	USMOPA (4-way)	FEAT_SME_I16I64
1	0	1	USMOPS	FEAT_SME_I16I64
1	1	0	UMOPA (4-way)	FEAT_SME_I16I64
1	1	1	UMOPS (4-way)	FEAT_SME_I16I64

SME zero array

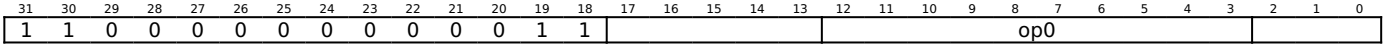
The encodings in this section are decoded from SME encodings.



Decode fields op0	Instruction details	Condition
0000000000	ZERO (tiles)	FEAT_SME
!= 0000000000	UNALLOCATED	-

SME2 Multiple Zero

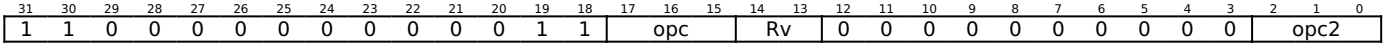
The encodings in this section are decoded from SME encodings.



Decode fields op0	Instruction details
0000000000	SME multiple vectors zero array
!= 0000000000	UNALLOCATED

SME multiple vectors zero array

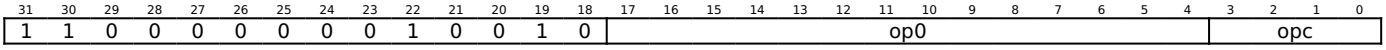
The encodings in this section are decoded from SME2 Multiple Zero.



Decode fields		Instruction Details	Condition
opc	opc2		
x1x	1xx	UNALLOCATED	-
000		ZERO (single-vector) — Two ZA single-vectors	FEAT_SME2p1
001		ZERO (double-vector) — One ZA double-vector	FEAT_SME2p1
010	0xx	ZERO (double-vector) — Two ZA double-vectors	FEAT_SME2p1
011	0xx	ZERO (double-vector) — Four ZA double-vectors	FEAT_SME2p1
100		ZERO (single-vector) — Four ZA single-vectors	FEAT_SME2p1
101	0xx	ZERO (quad-vector) — One ZA quad-vector	FEAT_SME2p1
101	1xx	UNALLOCATED	-
11x	01x	UNALLOCATED	-
110	00x	ZERO (quad-vector) — Two ZA quad-vectors	FEAT_SME2p1
111	00x	ZERO (quad-vector) — Four ZA quad-vectors	FEAT_SME2p1

SME2 zero lookup table

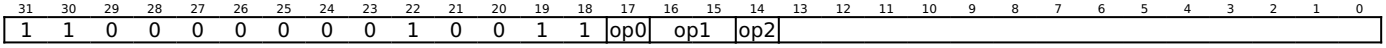
The encodings in this section are decoded from SME encodings.



Decode fields		Instruction Details	Condition
op0	opc		
00000000000000	0001	ZERO (table)	FEAT_SME2
00000000000000	!= 0001	UNALLOCATED	-
!= 00000000000000		UNALLOCATED	-

SME2 Move Lookup Table

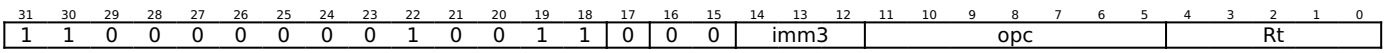
The encodings in this section are decoded from SME encodings.



Decode fields			Instruction details
op0	op1	op2	
0	00		SME2 move from lookup table
0	10		UNALLOCATED
1	00		SME2 move into lookup table
1	10	0	SME2 move vector to lookup table
1	10	1	UNALLOCATED
	x1		UNALLOCATED

SME2 move from lookup table

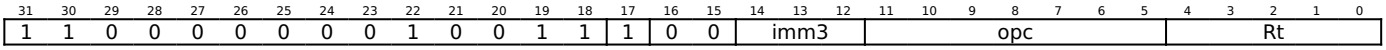
The encodings in this section are decoded from SME2 Move Lookup Table.



Decode fields	Instruction Details	Condition
opc		
0011111	MOVT (table to scalar)	FEAT_SME2
!= 0011111	UNALLOCATED	-

SME2 move into lookup table

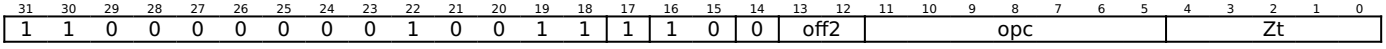
The encodings in this section are decoded from SME2 Move Lookup Table.



Decode fields opc	Instruction Details	Condition
0011111	MOVT (scalar to table)	FEAT_SME2
!= 0011111	UNALLOCATED	-

SME2 move vector to lookup table

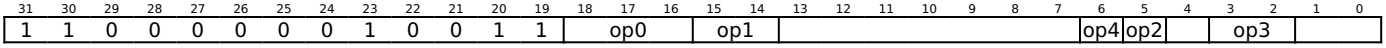
The encodings in this section are decoded from SME2 Move Lookup Table.



Decode fields opc	Instruction Details	Condition
0011111	MOVT (vector to table)	FEAT_SME_LUTv2
!= 0011111	UNALLOCATED	-

SME2 Expand Lookup Table (Non-contiguous)

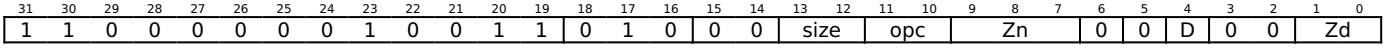
The encodings in this section are decoded from SME encodings.



Decode fields op0	op1	op4	op2	op3	Instruction details
010	00	0	0	00	SME2 lookup table three source registers expand to four non-contiguous destination registers
010	00	1	0	00	UNALLOCATED
011	00		0	00	SME2 lookup table two source registers expand to four non-contiguous destination registers
01x	00		1	00	UNALLOCATED
	10			00	SME2 lookup table expand four non-contiguous registers
	x0			01	UNALLOCATED
	x1			0x	SME2 lookup table expand two non-contiguous registers
				1x	UNALLOCATED
!= 01x	00			00	UNALLOCATED

SME2 lookup table three source registers expand to four non-contiguous destination registers

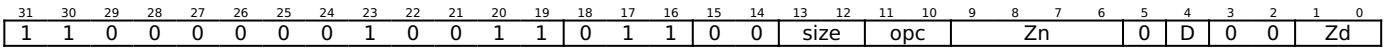
The encodings in this section are decoded from SME2 Expand Lookup Table (Non-contiguous).



Decode fields size	opc	Instruction Details	Condition
00	00	LUTI6 (table, four registers, 8-bit)	FEAT_SME2p3
00	!= 00	UNALLOCATED	-
!= 00		UNALLOCATED	-

SME2 lookup table two source registers expand to four non-contiguous destination registers

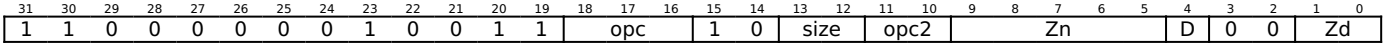
The encodings in this section are decoded from SME2 Expand Lookup Table (Non-contiguous).



Decode fields opc	Instruction Details	Condition
00	LUTI4 (four registers, 8-bit)	FEAT_SME2p1 && FEAT_SME_LUTv2
!= 00	UNALLOCATED	-

SME2 lookup table expand four non-contiguous registers

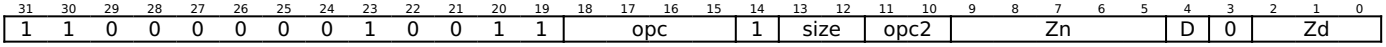
The encodings in this section are decoded from SME2 Expand Lookup Table (Non-contiguous).



Decode fields opc	Decode fields opc2	Instruction Details	Condition
	!= 00	UNALLOCATED	-
00x	00	UNALLOCATED	-
01x	00	LUTI4 (four registers, 16-bit and 32-bit)	FEAT_SME2p1
1xx	00	LUTI2 (four registers)	FEAT_SME2p1

SME2 lookup table expand two non-contiguous registers

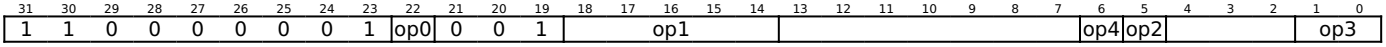
The encodings in this section are decoded from SME2 Expand Lookup Table (Non-contiguous).



Decode fields opc	Decode fields opc2	Instruction Details	Condition
	!= 00	UNALLOCATED	-
00xx	00	UNALLOCATED	-
01xx	00	LUTI4 (two registers)	FEAT_SME2p1
1xxx	00	LUTI2 (two registers)	FEAT_SME2p1

SME2 Expand Lookup Table (Contiguous)

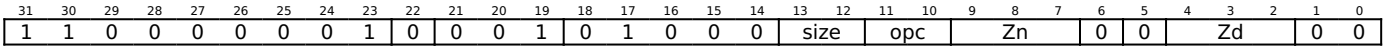
The encodings in this section are decoded from SME encodings.



Decode fields op0	op1	op4	op2	op3	Instruction details
0	00x00			00	UNALLOCATED
0	01000	0	0	00	SME2 lookup table three source registers expand to four contiguous destination registers
0	01000	1	0	00	UNALLOCATED
0	01100		0	00	SME2 lookup table two source registers expand to four contiguous destination registers
0	01x00		1	00	UNALLOCATED
0	1xx00			00	UNALLOCATED
0	xxx10			00	SME2 lookup table expand four contiguous registers
0	xxxx0			10	UNALLOCATED
0	xxxx1			x0	SME2 lookup table expand two contiguous registers
0				x1	UNALLOCATED
1					SME2 lookup table expand one register

SME2 lookup table three source registers expand to four contiguous destination registers

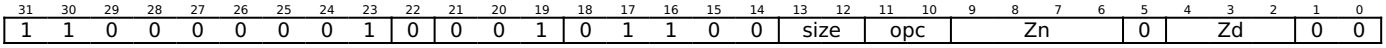
The encodings in this section are decoded from SME2 Expand Lookup Table (Contiguous).



Decode fields		Instruction Details	Condition
size	opc		
00	00	LUTI6 (table, four registers, 8-bit)	FEAT_SME2p3
00	!= 00	UNALLOCATED	-
!= 00		UNALLOCATED	-

SME2 lookup table two source registers expand to four contiguous destination registers

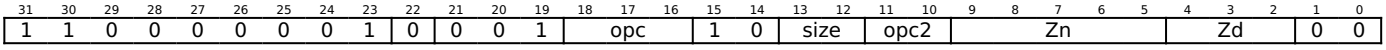
The encodings in this section are decoded from SME2 Expand Lookup Table (Contiguous).



Decode fields	Instruction Details	Condition
opc		
00	LUTI4 (four registers, 8-bit)	FEAT_SME_LUTv2
!= 00	UNALLOCATED	-

SME2 lookup table expand four contiguous registers

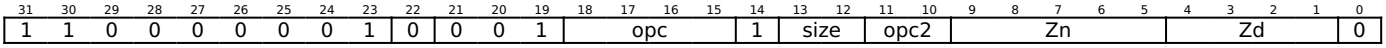
The encodings in this section are decoded from SME2 Expand Lookup Table (Contiguous).



Decode fields		Instruction Details	Condition
opc	opc2		
	!= 00	UNALLOCATED	-
00x	00	UNALLOCATED	-
01x	00	LUTI4 (four registers, 16-bit and 32-bit)	FEAT_SME2
1xx	00	LUTI2 (four registers)	FEAT_SME2

SME2 lookup table expand two contiguous registers

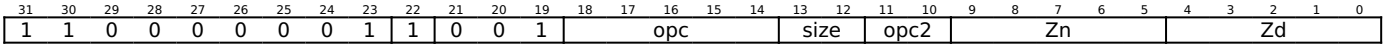
The encodings in this section are decoded from SME2 Expand Lookup Table (Contiguous).



Decode fields		Instruction Details	Condition
opc	opc2		
	!= 00	UNALLOCATED	-
00xx	00	UNALLOCATED	-
01xx	00	LUTI4 (two registers)	FEAT_SME2
1xxx	00	LUTI2 (two registers)	FEAT_SME2

SME2 lookup table expand one register

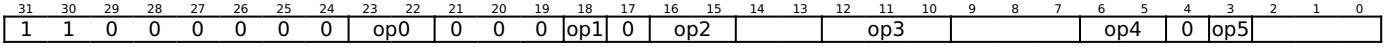
The encodings in this section are decoded from SME2 Expand Lookup Table (Contiguous).



Decode fields			Instruction Details	Condition
opc	size	opc2		
		!= 00	UNALLOCATED	-
00000		00	UNALLOCATED	-
00001	00	00	LUTI6 (table, single, 8-bit)	FEAT_SME2p3
00001	!= 00	00	UNALLOCATED	-
0001x		00	UNALLOCATED	-
001xx		00	UNALLOCATED	-
01xxx		00	LUTI4 (single)	FEAT_SME2
1xxxx		00	LUTI2 (single)	FEAT_SME2

SME Move into Array

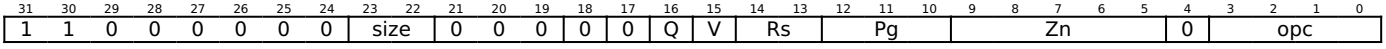
The encodings in this section are decoded from SME encodings.



Decode fields						Instruction details	Condition
op0	op1	op2	op3	op4	op5		
00	1	00	010	x0	0	MOVA (vector to array, two registers)	FEAT_SME2
00	1	00	011	00	0	MOVA (vector to array, four registers)	FEAT_SME2
00	1	00	011	10	0	UNALLOCATED	-
00	1	01	01x	x0	0	UNALLOCATED	-
	0					SME move vector to array	-
	1	0x	000	x0	0	SME2 move vector to tile, two registers	-
	1	0x	001	00	0	SME2 move vector to tile, four registers	-
	1	0x	001	10	0	UNALLOCATED	-
	1	0x	0xx	x0	1	UNALLOCATED	-
	1	0x	0xx	x1		UNALLOCATED	-
	1	0x	1xx			UNALLOCATED	-
	1	1x				UNALLOCATED	-
!= 00	1	0x	01x	x0	0	UNALLOCATED	-

SME move vector to array

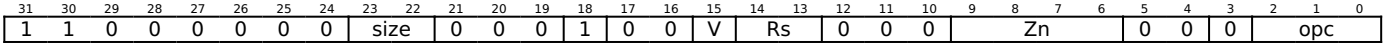
The encodings in this section are decoded from SME Move into Array.



Decode fields		Instruction Details	Condition
size	Q		
00	0	MOVA (vector to tile, single) — 8-bit	FEAT_SME
01	0	MOVA (vector to tile, single) — 16-bit	FEAT_SME
10	0	MOVA (vector to tile, single) — 32-bit	FEAT_SME
11	0	MOVA (vector to tile, single) — 64-bit	FEAT_SME
11	1	MOVA (vector to tile, single) — 128-bit	FEAT_SME
!= 11	1	UNALLOCATED	-

SME2 move vector to tile, two registers

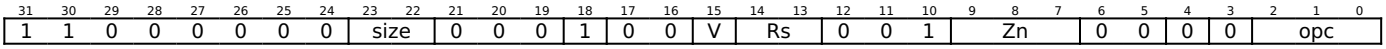
The encodings in this section are decoded from SME Move into Array.



Decode fields size	Instruction Details	Condition
00	MOVA (vector to tile, two registers) — 8-bit	FEAT_SME2
01	MOVA (vector to tile, two registers) — 16-bit	FEAT_SME2
10	MOVA (vector to tile, two registers) — 32-bit	FEAT_SME2
11	MOVA (vector to tile, two registers) — 64-bit	FEAT_SME2

SME2 move vector to tile, four registers

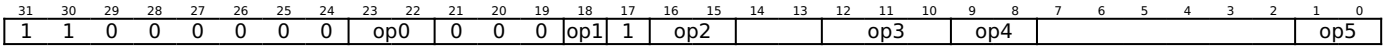
The encodings in this section are decoded from SME Move into Array.



Decode fields size	Decode fields opc	Instruction Details	Condition
00	0xx	MOVA (vector to tile, four registers) — 8-bit	FEAT_SME2
01	0xx	MOVA (vector to tile, four registers) — 16-bit	FEAT_SME2
10	0xx	MOVA (vector to tile, four registers) — 32-bit	FEAT_SME2
11		MOVA (vector to tile, four registers) — 64-bit	FEAT_SME2
!= 11	1xx	UNALLOCATED	-

SME Move from Array

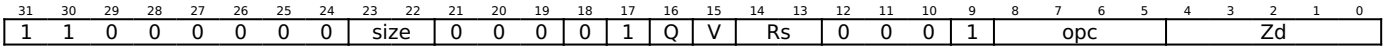
The encodings in this section are decoded from SME encodings.



Decode fields op0	op1	op2	op3	op4	op5	Instruction details	Condition
00	1	00	010	00	x0	MOVA (array to vector, two registers)	FEAT_SME2
00	1	00	010	10	x0	MOVAZ (array to vector, two registers)	FEAT_SME2p1
00	1	00	011	00	00	MOVA (array to vector, four registers)	FEAT_SME2
00	1	00	011	10	00	MOVAZ (array to vector, four registers)	FEAT_SME2p1
00	1	00	011	x0	10	UNALLOCATED	-
00	1	01	01x	x0	x0	UNALLOCATED	-
	0		000	1x		SME zeroing move array to vector	-
	0			0x		SME move array to vector	-
	0		!= 000	1x		UNALLOCATED	-
	1	0x	000	00	x0	SME2 move tile to vector, two registers	-
	1	0x	000	10	x0	SME2 zeroing move tile to vector, two registers	-
	1	0x	001	00	00	SME2 move tile to vector, four registers	-
	1	0x	001	10	00	SME2 zeroing move tile to vector, four registers	-
	1	0x	001	x0	10	UNALLOCATED	-
	1	0x	0xx	x0	x1	UNALLOCATED	-
	1	0x	0xx	x1		UNALLOCATED	-
	1	0x	1xx			UNALLOCATED	-
	1	1x				UNALLOCATED	-
!= 00	1	0x	01x	x0	x0	UNALLOCATED	-

SME zeroing move array to vector

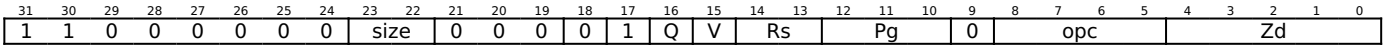
The encodings in this section are decoded from SME Move from Array.



Decode fields size	Q	Instruction Details	Condition
00	0	MOVAZ (tile to vector, single) — 8-bit	FEAT_SME2p1
01	0	MOVAZ (tile to vector, single) — 16-bit	FEAT_SME2p1
10	0	MOVAZ (tile to vector, single) — 32-bit	FEAT_SME2p1
11	0	MOVAZ (tile to vector, single) — 64-bit	FEAT_SME2p1
11	1	MOVAZ (tile to vector, single) — 128-bit	FEAT_SME2p1
!= 11	1	UNALLOCATED	-

SME move array to vector

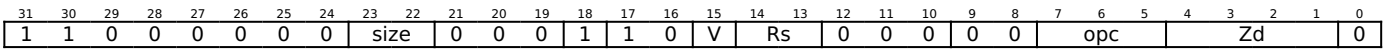
The encodings in this section are decoded from SME Move from Array.



Decode fields size	Q	Instruction Details	Condition
00	0	MOVA (tile to vector, single) — 8-bit	FEAT_SME
01	0	MOVA (tile to vector, single) — 16-bit	FEAT_SME
10	0	MOVA (tile to vector, single) — 32-bit	FEAT_SME
11	0	MOVA (tile to vector, single) — 64-bit	FEAT_SME
11	1	MOVA (tile to vector, single) — 128-bit	FEAT_SME
!= 11	1	UNALLOCATED	-

SME2 move tile to vector, two registers

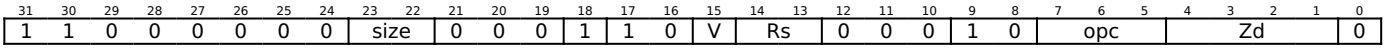
The encodings in this section are decoded from SME Move from Array.



Decode fields size	Instruction Details	Condition
00	MOVA (tile to vector, two registers) — 8-bit	FEAT_SME2
01	MOVA (tile to vector, two registers) — 16-bit	FEAT_SME2
10	MOVA (tile to vector, two registers) — 32-bit	FEAT_SME2
11	MOVA (tile to vector, two registers) — 64-bit	FEAT_SME2

SME2 zeroing move tile to vector, two registers

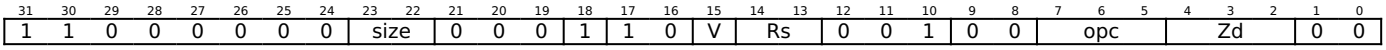
The encodings in this section are decoded from SME Move from Array.



Decode fields size	Instruction Details	Condition
00	MOVAZ (tile to vector, two registers) — 8-bit	FEAT_SME2p1
01	MOVAZ (tile to vector, two registers) — 16-bit	FEAT_SME2p1
10	MOVAZ (tile to vector, two registers) — 32-bit	FEAT_SME2p1
11	MOVAZ (tile to vector, two registers) — 64-bit	FEAT_SME2p1

SME2 move tile to vector, four registers

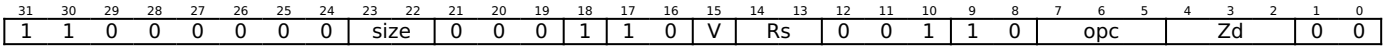
The encodings in this section are decoded from SME Move from Array.



Decode fields		Instruction Details	Condition
size	opc		
00	0xx	MOVA (tile to vector, four registers) — 8-bit	FEAT_SME2
01	0xx	MOVA (tile to vector, four registers) — 16-bit	FEAT_SME2
10	0xx	MOVA (tile to vector, four registers) — 32-bit	FEAT_SME2
11		MOVA (tile to vector, four registers) — 64-bit	FEAT_SME2
!= 11	1xx	UNALLOCATED	-

SME2 zeroing move tile to vector, four registers

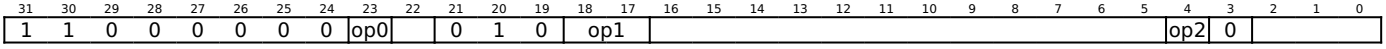
The encodings in this section are decoded from SME Move from Array.



Decode fields		Instruction Details	Condition
size	opc		
00	0xx	MOVAZ (tile to vector, four registers) — 8-bit	FEAT_SME2p1
01	0xx	MOVAZ (tile to vector, four registers) — 16-bit	FEAT_SME2p1
10	0xx	MOVAZ (tile to vector, four registers) — 32-bit	FEAT_SME2p1
11		MOVAZ (tile to vector, four registers) — 64-bit	FEAT_SME2p1
!= 11	1xx	UNALLOCATED	-

SME Add Vector to Array

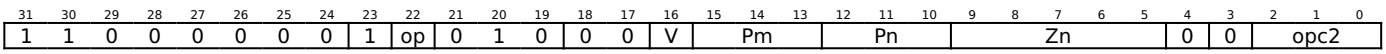
The encodings in this section are decoded from SME encodings.



Decode fields			Instruction details
op0	op1	op2	
0			UNALLOCATED
1	00	0	SME add vector to array
1	00	1	UNALLOCATED
1	!= 00		UNALLOCATED

SME add vector to array

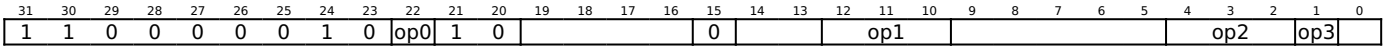
The encodings in this section are decoded from SME Add Vector to Array.



Decode fields			Instruction Details	Condition
op	V	opc2		
0		1xx	UNALLOCATED	-
0	0	0xx	ADDHA — 32-bit	FEAT_SME
0	1	0xx	ADDVA — 32-bit	FEAT_SME
1	0		ADDHA — 64-bit	FEAT_SME_I16I64
1	1		ADDVA — 64-bit	FEAT_SME_I16I64

SME2 Multi-vector - Multiple and Single Array Vectors (Two registers)

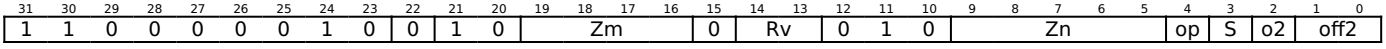
The encodings in this section are decoded from SME encodings.



Decode fields				Instruction details	Condition
op0	op1	op2	op3		
0	000	000	1	FMLALL (multiple and single vector) — Two ZA quad-vectors	FEAT_SME_F8F32
0	000	!= 000	1	UNALLOCATED	-
0	010			SME2 single-multi long FMA two sources	-
0	011			SME2 multiple and single vector long FMA one source	-
0	100			SME2 single-multi FP dot product two registers	-
0	101	x1x		SME2 single-multi mixed dot product two registers	-
1	000		1	UNALLOCATED	-
1	010			SME2 single-multi long MLA two sources	-
1	011			SME2 multiple and single vector long MLA one source	-
1	100			UNALLOCATED	-
1	101	x1x		SME2 single-multi two-way dot product two registers	-
	000		0	SME2 single-multi long long MLA two sources	-
	001			SME2 multiple and single vector long long FMA one source	-
	101	x0x		SME2 single-multi four-way dot product two registers	-
	110	0xx		SME2 single-multi ternary FP two registers	-
	110	1xx		SME2 single-multi ternary int two registers	-
	111	0xx		SME2 single-multi ternary FP16 two registers	-
	111	1xx		UNALLOCATED	-

SME2 single-multi long FMA two sources

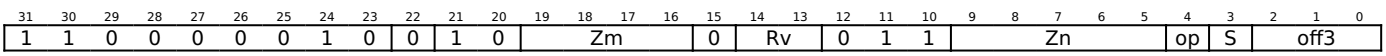
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single Array Vectors (Two registers).



Decode fields			Instruction Details	Condition
op	S	o2		
0	0	0	FMLAL (multiple and single vector, FP16 to FP32)	FEAT_SME2
0	0	1	FMLAL (multiple and single vector, FP8 to FP16)	FEAT_SME_F8F16
0	1	0	FMLSL (multiple and single vector)	FEAT_SME2
0	1	1	UNALLOCATED	-
1		1	UNALLOCATED	-
1	0	0	BFMLAL (multiple and single vector)	FEAT_SME2
1	1	0	BFMLSL (multiple and single vector)	FEAT_SME2

SME2 multiple and single vector long FMA one source

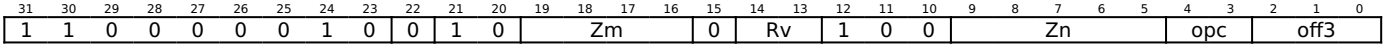
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single Array Vectors (Two registers).



Decode fields		Instruction Details	Condition
op	S		
0	0	FMLAL (multiple and single vector, FP16 to FP32)	FEAT_SME2
0	1	FMLSL (multiple and single vector)	FEAT_SME2
1	0	BFMLAL (multiple and single vector)	FEAT_SME2
1	1	BFMLSL (multiple and single vector)	FEAT_SME2

SME2 single-multi FP dot product two registers

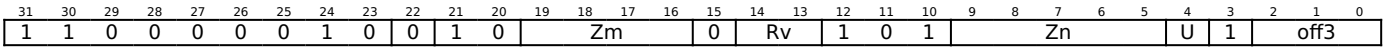
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single Array Vectors (Two registers).



Decode fields	Instruction Details	Condition
opc		
00	FDOT (2-way, multiple and single vector, FP16 to FP32)	FEAT_SME2
01	FDOT (2-way, multiple and single vector, FP8 to FP16)	FEAT_SME_F8F16
10	BFDOT (multiple and single vector)	FEAT_SME2
11	FDOT (4-way, multiple and single vector)	FEAT_SME_F8F32

SME2 single-multi mixed dot product two registers

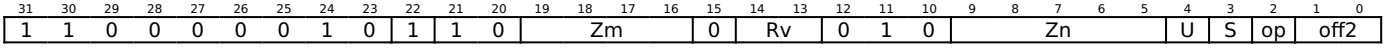
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single Array Vectors (Two registers).



Decode fields	Instruction Details	Condition
U		
0	USDOT (4-way, multiple and single vector)	FEAT_SME2
1	SUDOT (4-way, multiple and single vector)	FEAT_SME2

SME2 single-multi long MLA two sources

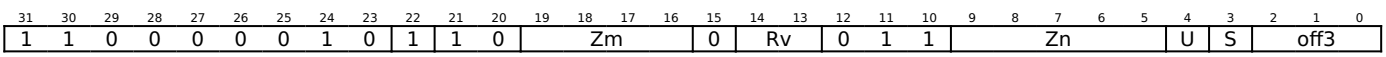
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single Array Vectors (Two registers).



Decode fields			Instruction Details	Condition
U	S	op		
		1	UNALLOCATED	-
0	0	0	SMLAL (multiple and single vector)	FEAT_SME2
0	1	0	SMLSL (multiple and single vector)	FEAT_SME2
1	0	0	UMLAL (multiple and single vector)	FEAT_SME2
1	1	0	UMLSL (multiple and single vector)	FEAT_SME2

SME2 multiple and single vector long MLA one source

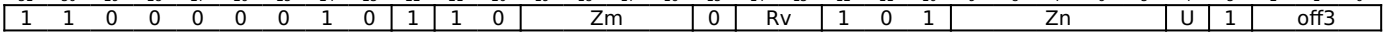
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single Array Vectors (Two registers).



Decode fields U	S	Instruction Details	Condition
0	0	SMLAL (multiple and single vector)	FEAT_SME2
0	1	SMLSL (multiple and single vector)	FEAT_SME2
1	0	UMLAL (multiple and single vector)	FEAT_SME2
1	1	UMLSL (multiple and single vector)	FEAT_SME2

SME2 single-multi two-way dot product two registers

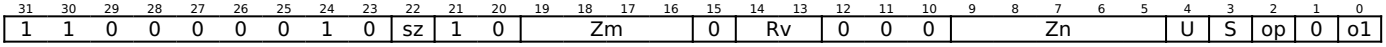
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single Array Vectors (Two registers).



Decode fields U	Instruction Details	Condition
0	SDOT (2-way, multiple and single vector)	FEAT_SME2
1	UDOT (2-way, multiple and single vector)	FEAT_SME2

SME2 single-multi long long MLA two sources

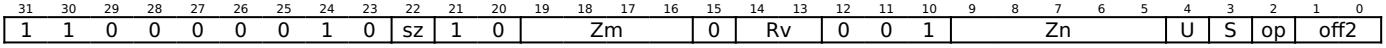
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single Array Vectors (Two registers).



Decode fields sz	U	S	op	Instruction Details	Condition
	0	0	0	SMLALL (multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
	0	1	0	SMLSLL (multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
	1	0	0	UMLALL (multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
	1	1	0	UMLSLL (multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
0		1	1	UNALLOCATED	-
0	0	0	1	USMLALL (multiple and single vector)	FEAT_SME2
0	1	0	1	SUMLALL (multiple and single vector)	FEAT_SME2
1			1	UNALLOCATED	-

SME2 multiple and single vector long long FMA one source

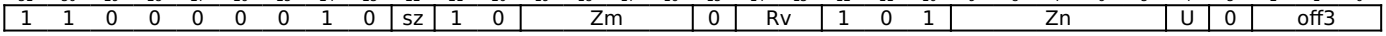
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single Array Vectors (Two registers).



Decode fields sz	U	S	op	Instruction Details	Condition
	0	0	0	SMLALL (multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
	0	1	0	SMLSLL (multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
	1	0	0	UMLALL (multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
	1	1	0	UMLSLL (multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
0	0	0	1	USMLALL (multiple and single vector)	FEAT_SME2
0	0	1	1	UNALLOCATED	-
0	1		1	UNALLOCATED	-
1			1	UNALLOCATED	-

SME2 single-multi four-way dot product two registers

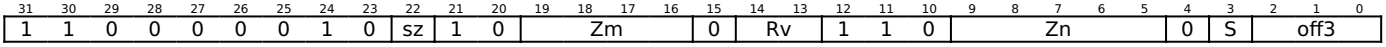
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single Array Vectors (Two registers).



Decode fields U	Instruction Details	Condition
0	SDOT (4-way, multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
1	UDOT (4-way, multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)

SME2 single-multi ternary FP two registers

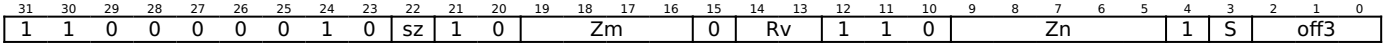
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single Array Vectors (Two registers).



Decode fields S	Instruction Details	Condition
0	FMLA (multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_F64F64)
1	FMLS (multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_F64F64)

SME2 single-multi ternary int two registers

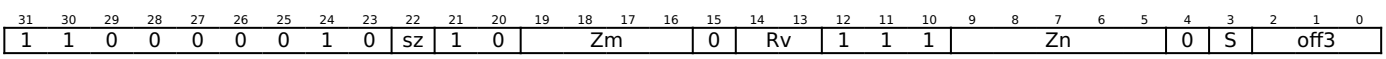
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single Array Vectors (Two registers).



Decode fields S	Instruction Details	Condition
0	ADD (to array, multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
1	SUB (to array, multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)

SME2 single-multi ternary FP16 two registers

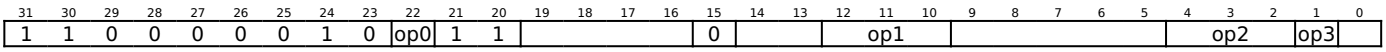
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single Array Vectors (Two registers).



Decode fields sz	Decode fields S	Instruction Details	Condition
0	0	FMLA (multiple and single vector)	FEAT_SME_F16F16
0	1	FMLS (multiple and single vector)	FEAT_SME_F16F16
1	0	BFMLA (multiple and single vector)	FEAT_SME_B16B16
1	1	BFMLS (multiple and single vector)	FEAT_SME_B16B16

SME2 Multi-vector - Multiple and Single Array Vectors (Four registers)

The encodings in this section are decoded from SME encodings.



Decode fields				Instruction details	Condition
op0	op1	op2	op3		
0	000	000	1	FMLALL (multiple and single vector) — Four ZA quad-vectors	FEAT_SME_F8F32
0	000	!= 000	1	UNALLOCATED	-
0	001	000		FMLALL (multiple and single vector) — One ZA quad-vector	FEAT_SME_F8F32
0	001	001		UNALLOCATED	-
0	010			SME2 single-multi long FMA four sources	-
0	011	00x		FMLAL (multiple and single vector, FP8 to FP16) — One ZA double-vector	FEAT_SME_F8F16
0	0x1	!= 00x		UNALLOCATED	-
0	100			SME2 single-multi FP dot product four registers	-
0	101	x1x		SME2 single-multi mixed dot product four registers	-
1	000		1	UNALLOCATED	-
1	010			SME2 single-multi long MLA four sources	-
1	0x1			UNALLOCATED	-
1	100			UNALLOCATED	-
1	101	x1x		SME2 single-multi two-way dot product four registers	-
	000		0	SME2 single-multi long long MLA four sources	-
	101	x0x		SME2 single-multi four-way dot product four registers	-
	110	0xx		SME2 single-multi ternary FP four registers	-
	110	1xx		SME2 single-multi ternary int four registers	-
	111	0xx		SME2 single-multi ternary FP16 four registers	-
	111	1xx		UNALLOCATED	-

SME2 single-multi long FMA four sources

The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single Array Vectors (Four registers).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	0	1	1	Zm				0	Rv		0	1	0	Zn					op	S	o2	off2	

Decode fields			Instruction Details	Condition
op	S	o2		
0	0	0	FMLAL (multiple and single vector, FP16 to FP32)	FEAT_SME2
0	0	1	FMLAL (multiple and single vector, FP8 to FP16)	FEAT_SME_F8F16
0	1	0	FMLSL (multiple and single vector)	FEAT_SME2
0	1	1	UNALLOCATED	-
1		1	UNALLOCATED	-
1	0	0	BFMLAL (multiple and single vector)	FEAT_SME2
1	1	0	BFMLSL (multiple and single vector)	FEAT_SME2

SME2 single-multi FP dot product four registers

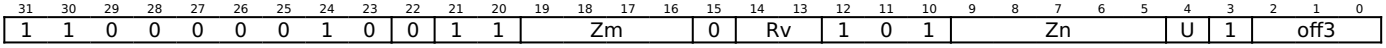
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single Array Vectors (Four registers).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	0	1	1	Zm				0	Rv		1	0	0	Zn					opc		off3		

Decode fields opc	Instruction Details	Condition
00	FDOT (2-way, multiple and single vector, FP16 to FP32)	FEAT_SME2
01	FDOT (2-way, multiple and single vector, FP8 to FP16)	FEAT_SME_F8F16
10	BFDOT (multiple and single vector)	FEAT_SME2
11	FDOT (4-way, multiple and single vector)	FEAT_SME_F8F32

SME2 single-multi mixed dot product four registers

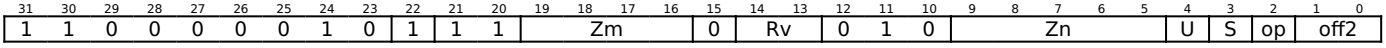
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single Array Vectors (Four registers).



Decode fields U	Instruction Details	Condition
0	USDOT (4-way, multiple and single vector)	FEAT_SME2
1	SUDOT (4-way, multiple and single vector)	FEAT_SME2

SME2 single-multi long MLA four sources

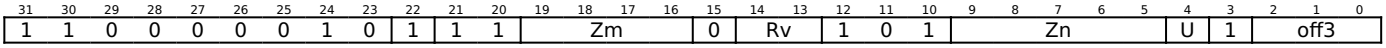
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single Array Vectors (Four registers).



Decode fields U	S	op	Instruction Details	Condition
		1	UNALLOCATED	-
0	0	0	SMLAL (multiple and single vector)	FEAT_SME2
0	1	0	SMLSL (multiple and single vector)	FEAT_SME2
1	0	0	UMLAL (multiple and single vector)	FEAT_SME2
1	1	0	UMLSL (multiple and single vector)	FEAT_SME2

SME2 single-multi two-way dot product four registers

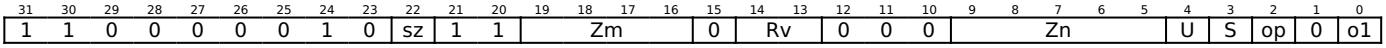
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single Array Vectors (Four registers).



Decode fields U	Instruction Details	Condition
0	SDOT (2-way, multiple and single vector)	FEAT_SME2
1	UDOT (2-way, multiple and single vector)	FEAT_SME2

SME2 single-multi long long MLA four sources

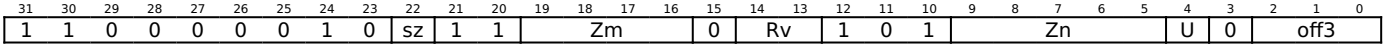
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single Array Vectors (Four registers).



Decode fields				Instruction Details	Condition
sz	U	S	op		
	0	0	0	SMLALL (multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
	0	1	0	SMLSLL (multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
	1	0	0	UMLALL (multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
	1	1	0	UMLSLL (multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
0		1	1	UNALLOCATED	-
0	0	0	1	USMLALL (multiple and single vector)	FEAT_SME2
0	1	0	1	SUMLALL (multiple and single vector)	FEAT_SME2
1			1	UNALLOCATED	-

SME2 single-multi four-way dot product four registers

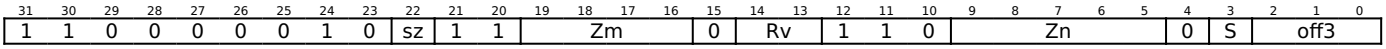
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single Array Vectors (Four registers).



Decode fields	Instruction Details	Condition
U		
0	SDOT (4-way, multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
1	UDOT (4-way, multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)

SME2 single-multi ternary FP four registers

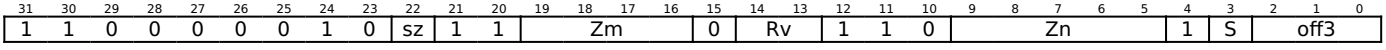
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single Array Vectors (Four registers).



Decode fields	Instruction Details	Condition
S		
0	FMLA (multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_F64F64)
1	FMLS (multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_F64F64)

SME2 single-multi ternary int four registers

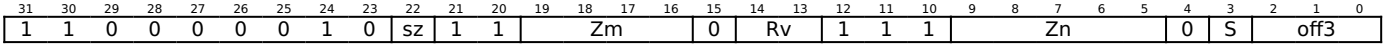
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single Array Vectors (Four registers).



Decode fields	Instruction Details	Condition
S		
0	ADD (to array, multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
1	SUB (to array, multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)

SME2 single-multi ternary FP16 four registers

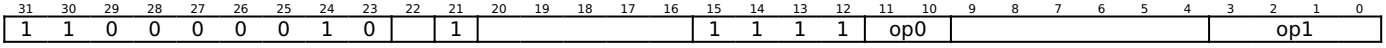
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single Array Vectors (Four registers).



Decode fields		Instruction Details	Condition
sz	S		
0	0	FMLA (multiple and single vector)	FEAT_SME_F16F16
0	1	FMLS (multiple and single vector)	FEAT_SME_F16F16
1	0	BFMLA (multiple and single vector)	FEAT_SME_B16B16
1	1	BFMLS (multiple and single vector)	FEAT_SME_B16B16

SME2 Multi-vector - Lookup table with 6-bit indices

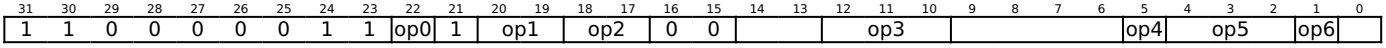
The encodings in this section are decoded from SME encodings.



Decode fields		Instruction details	Condition
op0	op1		
01	xx00	LUTI6 (vector, 16-bit) — Consecutive	FEAT_SME2p3
01	!= xx00	UNALLOCATED	-
11	00xx	LUTI6 (vector, 16-bit) — Strided	FEAT_SME2p3
11	!= 00xx	UNALLOCATED	-
x0		UNALLOCATED	-

SME2 Multi-vector - Multiple Array Vectors (Two registers)

The encodings in this section are decoded from SME encodings.



Decode fields							Instruction details	Condition
op0	op1	op2	op3	op4	op5	op6		
0			000	1	000	0	FMLALL (multiple vectors) — Two ZA quad-vectors	FEAT_SME_F8F32
0			000	1	001	0	UNALLOCATED	-
0			000	1	10x	0	UNALLOCATED	-
0			000	1	x0x	1	UNALLOCATED	-
0			010	0	xx0		SME2 multiple vectors long FMA two sources	-
0			010	1	000		FMLAL (multiple vectors, FP8 to FP16) — Two ZA double-vectors	FEAT_SME_F8F16
0			010	1	100		UNALLOCATED	-
0			010	1	x01		UNALLOCATED	-
0			100		x0x		SME2 multiple vectors FP dot product two registers	-
0			101	0	01x		USDOT (4-way, multiple vectors) — Two ZA single-vectors	FEAT_SME2
0			101	0	11x		UNALLOCATED	-
0			110	1	x0x		UNALLOCATED	-
0			xx0	1	x1x		UNALLOCATED	-
0			!= xx0	1			UNALLOCATED	-
1			010	0	xx0		SME2 multiple vectors long MLA two sources	-
1			100	0	x0x		UNALLOCATED	-
1			101	0	x1x		SME2 multiple vectors two-way dot product two registers	-
1				1			UNALLOCATED	-
	00	00	111	0	0xx		SME2 multiple vectors binary FP two registers	-
	00	00	111	0	1xx		SME2 multiple vectors binary int two registers	-
	00	10	111	0	0xx		SME2 multiple vectors binary FP16 two registers	-
	00	10	111	0	1xx		UNALLOCATED	-
	00	x1	111	0			UNALLOCATED	-
			000	0		0	SME2 multiple vectors long long MLA two sources	-
			000	0		1	UNALLOCATED	-
			010	0	xx1		UNALLOCATED	-
			0x1	0			UNALLOCATED	-
			100	0	x1x		SME2 multiple vectors ternary FP16 two registers	-
			101	0	x0x		SME2 multiple vectors four-way dot product two registers	-
			110	0	0xx		SME2 multiple vectors ternary FP two registers	-
			110	0	1xx		SME2 multiple vectors ternary int two registers	-
	!= 00		111	0			UNALLOCATED	-

SME2 multiple vectors long FMA two sources

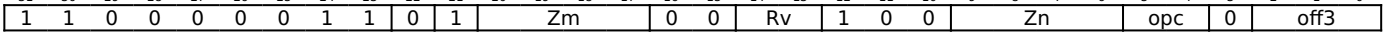
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Two registers).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	0	1	Zm				0	0	Rv		0	1	0	Zn				0	op		S	0	off2	

Decode fields		Instruction Details	Condition
op	S		
0	0	FMLAL (multiple vectors, FP16 to FP32)	FEAT_SME2
0	1	FMLSL (multiple vectors)	FEAT_SME2
1	0	BFMLAL (multiple vectors)	FEAT_SME2
1	1	BFMLSL (multiple vectors)	FEAT_SME2

SME2 multiple vectors FP dot product two registers

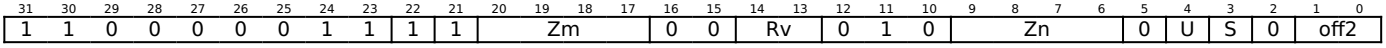
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Two registers).



Decode fields	Instruction Details	Condition
opc		
00	FDOT (2-way, multiple vectors, FP16 to FP32)	FEAT_SME2
01	BFDOT (multiple vectors)	FEAT_SME2
10	FDOT (2-way, multiple vectors, FP8 to FP16)	FEAT_SME_F8F16
11	FDOT (4-way, multiple vectors)	FEAT_SME_F8F32

SME2 multiple vectors long MLA two sources

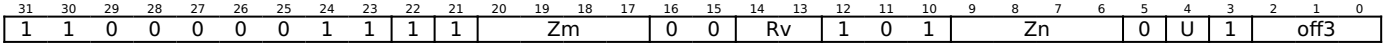
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Two registers).



Decode fields		Instruction Details	Condition
U	S		
0	0	SMLAL (multiple vectors)	FEAT_SME2
0	1	SMLSL (multiple vectors)	FEAT_SME2
1	0	UMLAL (multiple vectors)	FEAT_SME2
1	1	UMLSL (multiple vectors)	FEAT_SME2

SME2 multiple vectors two-way dot product two registers

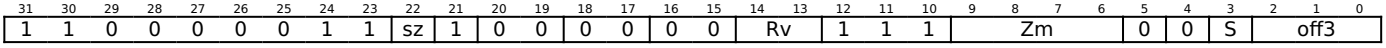
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Two registers).



Decode fields	Instruction Details	Condition
U		
0	SDOT (2-way, multiple vectors)	FEAT_SME2
1	UDOT (2-way, multiple vectors)	FEAT_SME2

SME2 multiple vectors binary FP two registers

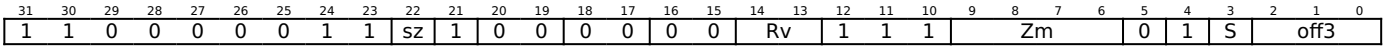
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Two registers).



Decode fields	Instruction Details	Condition
S		
0	FADD	FEAT_SME2 && (sz == '0' || FEAT_SME_F64F64)
1	FSUB	FEAT_SME2 && (sz == '0' || FEAT_SME_F64F64)

SME2 multiple vectors binary int two registers

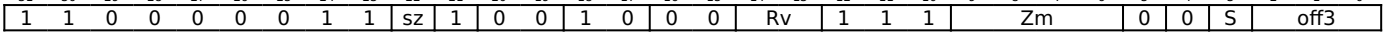
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Two registers).



Decode fields S	Instruction Details	Condition
0	ADD (to array, array and multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
1	SUB (to array, array and multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)

SME2 multiple vectors binary FP16 two registers

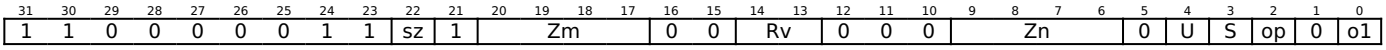
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Two registers).



Decode fields sz	S	Instruction Details	Condition
0	0	FADD	FEAT_SME_F16F16 || FEAT_SME_F8F16
0	1	FSUB	FEAT_SME_F16F16 || FEAT_SME_F8F16
1	0	BFADD	FEAT_SME_B16B16
1	1	BFSUB	FEAT_SME_B16B16

SME2 multiple vectors long long MLA two sources

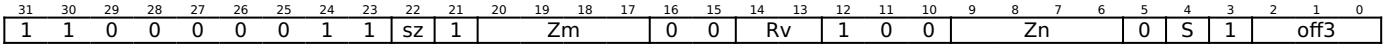
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Two registers).



Decode fields sz	U	S	op	Instruction Details	Condition
	0	0	0	SMLALL (multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
	0	1	0	SMLSLL (multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
	1	0	0	UMLALL (multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
	1	1	0	UMLSLL (multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
0	0	0	1	USMLALL (multiple vectors)	FEAT_SME2
0	0	1	1	UNALLOCATED	-
0	1		1	UNALLOCATED	-
1			1	UNALLOCATED	-

SME2 multiple vectors ternary FP16 two registers

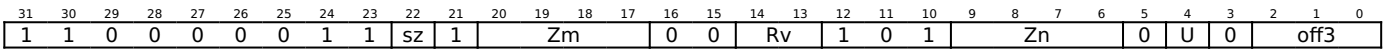
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Two registers).



Decode fields sz	S	Instruction Details	Condition
0	0	FMLA (multiple vectors)	FEAT_SME_F16F16
0	1	FMLS (multiple vectors)	FEAT_SME_F16F16
1	0	BFMLA (multiple vectors)	FEAT_SME_B16B16
1	1	BFMLS (multiple vectors)	FEAT_SME_B16B16

SME2 multiple vectors four-way dot product two registers

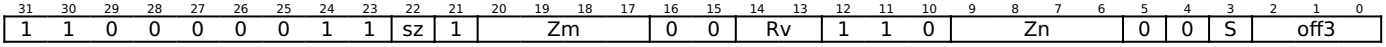
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Two registers).



Decode fields U	Instruction Details	Condition
0	SDOT (4-way, multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
1	UDOT (4-way, multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)

SME2 multiple vectors ternary FP two registers

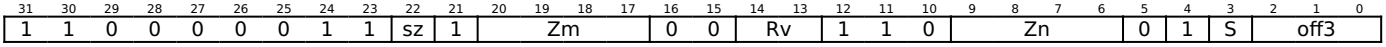
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Two registers).



Decode fields S	Instruction Details	Condition
0	FMLA (multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_F64F64)
1	FMLS (multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_F64F64)

SME2 multiple vectors ternary int two registers

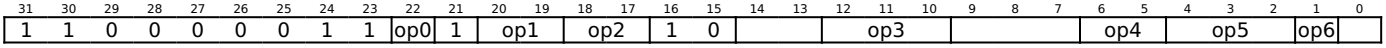
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Two registers).



Decode fields S	Instruction Details	Condition
0	ADD (to array, multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
1	SUB (to array, multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)

SME2 Multi-vector - Multiple Array Vectors (Four registers)

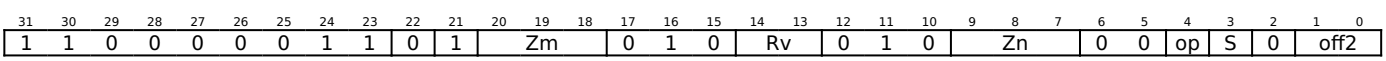
The encodings in this section are decoded from SME encodings.



Decode fields							Instruction details	Condition
op0	op1	op2	op3	op4	op5	op6		
0		x0	000	01	000	0	FMLALL (multiple vectors) — Four ZA quad-vectors	FEAT_SME_F8F32
0		x0	000	01	001	0	UNALLOCATED	-
0		x0	000	01	10x	0	UNALLOCATED	-
0		x0	000	01	x0x	1	UNALLOCATED	-
0		x0	010	00	xx0		SME2 multiple vectors long FMA four sources	-
0		x0	010	01	000		FMLAL (multiple vectors, FP8 to FP16) — Four ZA double-vectors	FEAT_SME_F8F16
0		x0	010	01	100		UNALLOCATED	-
0		x0	010	01	x01		UNALLOCATED	-
0		x0	100	0x	x0x		SME2 multiple vectors FP dot product four registers	-
0		x0	101	00	01x		USDOT (4-way, multiple vectors) — Four ZA single-vectors	FEAT_SME2
0		x0	101	00	11x		UNALLOCATED	-
0		x0	110	01	x0x		UNALLOCATED	-
0		x0	xx0	01	x1x		UNALLOCATED	-
0		x0	!= xx0	01			UNALLOCATED	-
1		x0	010	00	xx0		SME2 multiple vectors long MLA four sources	-
1		x0	100	00	x0x		UNALLOCATED	-
1		x0	101	00	x1x		SME2 multiple vectors two-way dot product four registers	-
1		x0		01			UNALLOCATED	-
	00	00	111	00	0xx		SME2 multiple vectors binary FP four registers	-
	00	00	111	00	1xx		SME2 multiple vectors binary int four registers	-
	00	10	111	00	0xx		SME2 multiple vectors binary FP16 four registers	-
	00	10	111	00	1xx		UNALLOCATED	-
		x0	000	00		0	SME2 multiple vectors long long MLA four sources	-
		x0	000	00		1	UNALLOCATED	-
		x0	010	00	xx1		UNALLOCATED	-
		x0	0x1	00			UNALLOCATED	-
		x0	100	00	x1x		SME2 multiple vectors ternary FP16 four registers	-
		x0	101	00	x0x		SME2 multiple vectors four-way dot product four registers	-
		x0	110	00	0xx		SME2 multiple vectors ternary FP four registers	-
		x0	110	00	1xx		SME2 multiple vectors ternary int four registers	-
		x0		1x			UNALLOCATED	-
		x1					UNALLOCATED	-
	!= 00	x0	111	00			UNALLOCATED	-

SME2 multiple vectors long FMA four sources

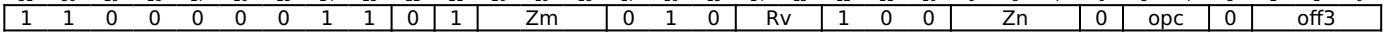
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Four registers).



Decode fields op	S	Instruction Details	Condition
0	0	FMLAL (multiple vectors, FP16 to FP32)	FEAT_SME2
0	1	FMLSL (multiple vectors)	FEAT_SME2
1	0	BFMLAL (multiple vectors)	FEAT_SME2
1	1	BFMLSL (multiple vectors)	FEAT_SME2

SME2 multiple vectors FP dot product four registers

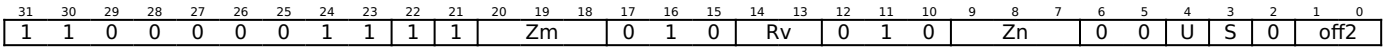
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Four registers).



Decode fields opc	Instruction Details	Condition
00	FDOT (2-way, multiple vectors, FP16 to FP32)	FEAT_SME2
01	BFDOT (multiple vectors)	FEAT_SME2
10	FDOT (2-way, multiple vectors, FP8 to FP16)	FEAT_SME_F8F16
11	FDOT (4-way, multiple vectors)	FEAT_SME_F8F32

SME2 multiple vectors long MLA four sources

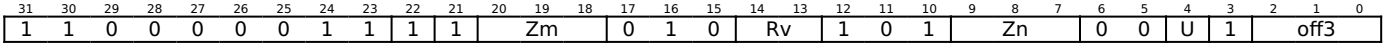
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Four registers).



Decode fields U	S	Instruction Details	Condition
0	0	SMLAL (multiple vectors)	FEAT_SME2
0	1	SMLSL (multiple vectors)	FEAT_SME2
1	0	UMLAL (multiple vectors)	FEAT_SME2
1	1	UMLSL (multiple vectors)	FEAT_SME2

SME2 multiple vectors two-way dot product four registers

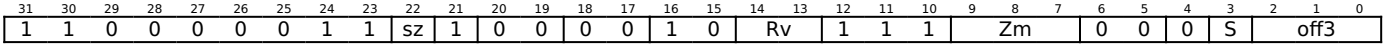
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Four registers).



Decode fields U	Instruction Details	Condition
0	SDOT (2-way, multiple vectors)	FEAT_SME2
1	UDOT (2-way, multiple vectors)	FEAT_SME2

SME2 multiple vectors binary FP four registers

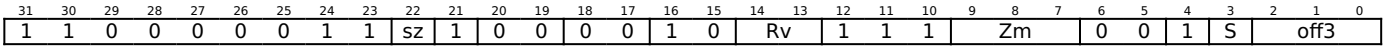
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Four registers).



Decode fields S	Instruction Details	Condition
0	FADD	FEAT_SME2 && (sz == '0' || FEAT_SME_F64F64)
1	FSUB	FEAT_SME2 && (sz == '0' || FEAT_SME_F64F64)

SME2 multiple vectors binary int four registers

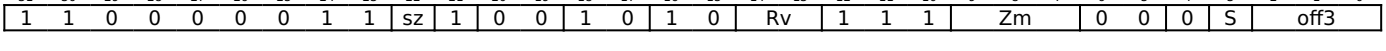
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Four registers).



Decode fields S	Instruction Details	Condition
0	ADD (to array, array and multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
1	SUB (to array, array and multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)

SME2 multiple vectors binary FP16 four registers

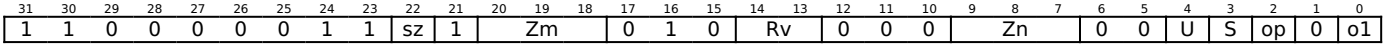
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Four registers).



Decode fields sz	Decode fields S	Instruction Details	Condition
0	0	FADD	FEAT_SME_F16F16 || FEAT_SME_F8F16
0	1	FSUB	FEAT_SME_F16F16 || FEAT_SME_F8F16
1	0	BFADD	FEAT_SME_B16B16
1	1	BFSUB	FEAT_SME_B16B16

SME2 multiple vectors long long MLA four sources

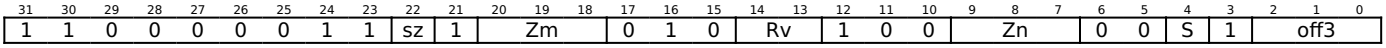
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Four registers).



Decode fields sz	Decode fields U	Decode fields S	Decode fields op	Instruction Details	Condition
	0	0	0	SMLALL (multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
	0	1	0	SMLSLL (multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
	1	0	0	UMLALL (multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
	1	1	0	UMLSLL (multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
0	0	0	1	USMLALL (multiple vectors)	FEAT_SME2
0	0	1	1	UNALLOCATED	-
0	1		1	UNALLOCATED	-
1			1	UNALLOCATED	-

SME2 multiple vectors ternary FP16 four registers

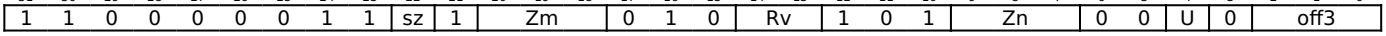
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Four registers).



Decode fields sz	Decode fields S	Instruction Details	Condition
0	0	FMLA (multiple vectors)	FEAT_SME_F16F16
0	1	FMLS (multiple vectors)	FEAT_SME_F16F16
1	0	BFMLA (multiple vectors)	FEAT_SME_B16B16
1	1	BFMLS (multiple vectors)	FEAT_SME_B16B16

SME2 multiple vectors four-way dot product four registers

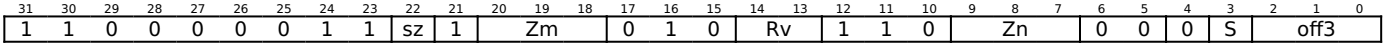
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Four registers).



Decode fields U	Instruction Details	Condition
0	SDOT (4-way, multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
1	UDOT (4-way, multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)

SME2 multiple vectors ternary FP four registers

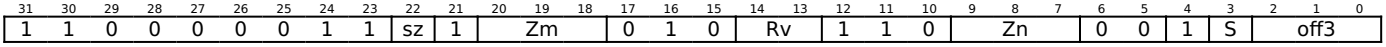
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Four registers).



Decode fields S	Instruction Details	Condition
0	FMLA (multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_F64F64)
1	FMLS (multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_F64F64)

SME2 multiple vectors ternary int four registers

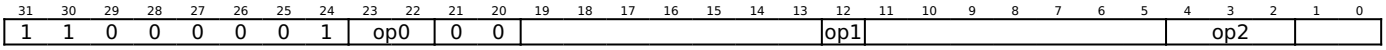
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Four registers).



Decode fields S	Instruction Details	Condition
0	ADD (to array, multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
1	SUB (to array, multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)

SME2 Multi-vector - Indexed (One register)

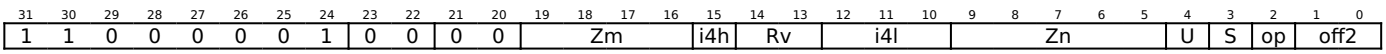
The encodings in this section are decoded from SME encodings.



Decode fields op0	op1	op2	Instruction details	Condition
00			SME2 multi-vec indexed long long MLA one source 32-bit	-
01		000	FMLALL (multiple and indexed vector) — One ZA quad-vector	FEAT_SME_F8F32
01		!= 000	UNALLOCATED	-
10	0	xx0	SME2 multi-vec indexed long long MLA one source 64-bit	-
10	0	xx1	UNALLOCATED	-
10	1		SME2 multi-vec indexed long FMA one source	-
11	0	0xx	FMLAL (multiple and indexed vector, FP8 to FP16) — One ZA double-vector	FEAT_SME_F8F16
11	0	1xx	UNALLOCATED	-
11	1		SME2 multi-vec indexed long MLA one source	-

SME2 multi-vec indexed long long MLA one source 32-bit

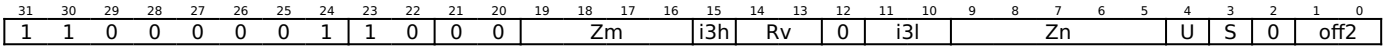
The encodings in this section are decoded from SME2 Multi-vector - Indexed (One register).



Decode fields U	S	op	Instruction Details	Condition
	1	1	UNALLOCATED	-
0	0	0	SMLALL (multiple and indexed vector)	FEAT_SME2
0	0	1	USMLALL (multiple and indexed vector)	FEAT_SME2
0	1	0	SMLSLL (multiple and indexed vector)	FEAT_SME2
1	0	0	UMLALL (multiple and indexed vector)	FEAT_SME2
1	0	1	SUMLALL (multiple and indexed vector)	FEAT_SME2
1	1	0	UMLSLL (multiple and indexed vector)	FEAT_SME2

SME2 multi-vec indexed long long MLA one source 64-bit

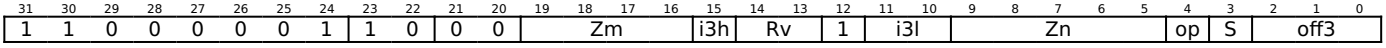
The encodings in this section are decoded from SME2 Multi-vector - Indexed (One register).



Decode fields U	S	Instruction Details	Condition
0	0	SMLALL (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64
0	1	SMLSLL (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64
1	0	UMLALL (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64
1	1	UMLSLL (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64

SME2 multi-vec indexed long FMA one source

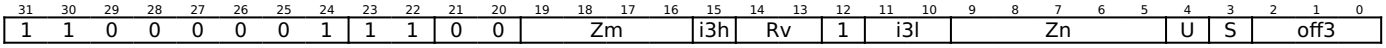
The encodings in this section are decoded from SME2 Multi-vector - Indexed (One register).



Decode fields op	S	Instruction Details	Condition
0	0	FMLAL (multiple and indexed vector, FP16 to FP32)	FEAT_SME2
0	1	FMLSL (multiple and indexed vector)	FEAT_SME2
1	0	BFMLAL (multiple and indexed vector)	FEAT_SME2
1	1	BFMLSL (multiple and indexed vector)	FEAT_SME2

SME2 multi-vec indexed long MLA one source

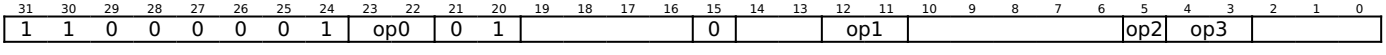
The encodings in this section are decoded from SME2 Multi-vector - Indexed (One register).



Decode fields U	S	Instruction Details	Condition
0	0	SMLAL (multiple and indexed vector)	FEAT_SME2
0	1	SMLSL (multiple and indexed vector)	FEAT_SME2
1	0	UMLAL (multiple and indexed vector)	FEAT_SME2
1	1	UMLSL (multiple and indexed vector)	FEAT_SME2

SME2 Multi-vector - Indexed (Two registers)

The encodings in this section are decoded from SME encodings.



Decode fields				Instruction details	Condition
op0	op1	op2	op3		
00	0x			SME2 multi-vec indexed long long MLA two sources 32-bit	-
00	1x			SME2 multi-vec ternary indexed two registers 16-bit	-
01				SME2 multi-vec ternary indexed two registers 32-bit	-
10	00	0		SME2 multi-vec indexed long long MLA two sources 64-bit	-
10	01	0		UNALLOCATED	-
10	0x	1	00	FMLALL (multiple and indexed vector) — Two ZA quad-vectors	FEAT_SME_F8F32
10	0x	1	!= 00	UNALLOCATED	-
10	1x	0		SME2 multi-vec indexed long FMA two sources	-
10	1x	1	0x	UNALLOCATED	-
10	1x	1	1x	FMLAL (multiple and indexed vector, FP8 to FP16) — Two ZA double-vectors	FEAT_SME_F8F16
11	00	0		SME2 multi-vec ternary indexed two registers 64-bit	-
11	01	0		SME2 multi-vec indexed FP8 two-way vertical dot product to single-precision two registers	-
11	1x	0		SME2 multi-vec indexed long MLA two sources	-
11		1	0x	SME2 multi-vec indexed FP8 two-way dot product to FP16 two registers	-
11		1	1x	UNALLOCATED	-

SME2 multi-vec indexed long long MLA two sources 32-bit

The encodings in this section are decoded from SME2 Multi-vector - Indexed (Two registers).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	0	0	1	Zm				0	Rv		0	i4h		Zn				op	U	S	i4l		o1

Decode fields			Instruction Details	Condition
op	U	S		
0	0	0	SMLALL (multiple and indexed vector)	FEAT_SME2
0	0	1	SMLSLL (multiple and indexed vector)	FEAT_SME2
0	1	0	UMLALL (multiple and indexed vector)	FEAT_SME2
0	1	1	UMLSLL (multiple and indexed vector)	FEAT_SME2
1		1	UNALLOCATED	-
1	0	0	USMLALL (multiple and indexed vector)	FEAT_SME2
1	1	0	SUMLALL (multiple and indexed vector)	FEAT_SME2

SME2 multi-vec ternary indexed two registers 16-bit

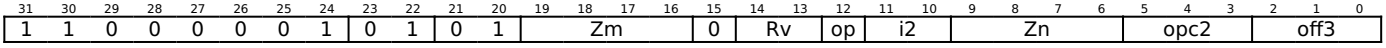
The encodings in this section are decoded from SME2 Multi-vector - Indexed (Two registers).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	0	0	1	Zm				0	Rv		1	i3h		Zn				op	S	i3l		off3	

Decode fields		Instruction Details	Condition
op	S		
0	0	FMLA (multiple and indexed vector)	FEAT_SME_F16F16
0	1	FMLS (multiple and indexed vector)	FEAT_SME_F16F16
1	0	BFMLA (multiple and indexed vector)	FEAT_SME_B16B16
1	1	BFMLS (multiple and indexed vector)	FEAT_SME_B16B16

SME2 multi-vec ternary indexed two registers 32-bit

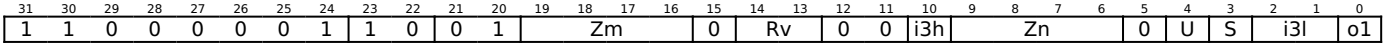
The encodings in this section are decoded from SME2 Multi-vector - Indexed (Two registers).



Decode fields op	Decode fields opc2	Instruction Details	Condition
0	000	FMLA (multiple and indexed vector)	FEAT_SME2
0	001	FVDOT (FP16 to FP32)	FEAT_SME2
0	010	FMLS (multiple and indexed vector)	FEAT_SME2
0	011	BFVDOT	FEAT_SME2
0	100	SVDOT (2-way)	FEAT_SME2
0	101	UNALLOCATED	-
0	110	UVDOT (2-way)	FEAT_SME2
0	111	FDOT (4-way, multiple and indexed vector)	FEAT_SME_F8F32
1	000	SDOT (2-way, multiple and indexed vector)	FEAT_SME2
1	001	FDOT (2-way, multiple and indexed vector, FP16 to FP32)	FEAT_SME2
1	010	UDOT (2-way, multiple and indexed vector)	FEAT_SME2
1	011	BFDOT (multiple and indexed vector)	FEAT_SME2
1	100	SDOT (4-way, multiple and indexed vector)	FEAT_SME2
1	101	USDOT (4-way, multiple and indexed vector)	FEAT_SME2
1	110	UDOT (4-way, multiple and indexed vector)	FEAT_SME2
1	111	SUDOT (4-way, multiple and indexed vector)	FEAT_SME2

SME2 multi-vec indexed long long MLA two sources 64-bit

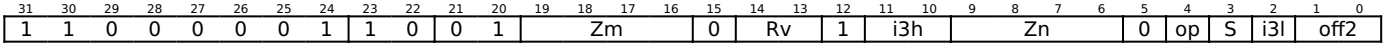
The encodings in this section are decoded from SME2 Multi-vector - Indexed (Two registers).



Decode fields U	Decode fields S	Instruction Details	Condition
0	0	SMLALL (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64
0	1	SMLSLL (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64
1	0	UMLALL (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64
1	1	UMLSLL (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64

SME2 multi-vec indexed long FMA two sources

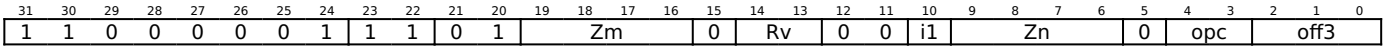
The encodings in this section are decoded from SME2 Multi-vector - Indexed (Two registers).



Decode fields op	Decode fields S	Instruction Details	Condition
0	0	FMLAL (multiple and indexed vector, FP16 to FP32)	FEAT_SME2
0	1	FMLSL (multiple and indexed vector)	FEAT_SME2
1	0	BFMLAL (multiple and indexed vector)	FEAT_SME2
1	1	BFMLSL (multiple and indexed vector)	FEAT_SME2

SME2 multi-vec ternary indexed two registers 64-bit

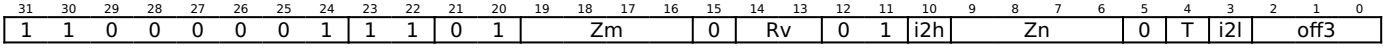
The encodings in this section are decoded from SME2 Multi-vector - Indexed (Two registers).



Decode fields opc	Instruction Details	Condition
00	FMLA (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_F64F64
01	SDOT (4-way, multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64
10	FMLS (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_F64F64
11	UDOT (4-way, multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64

SME2 multi-vec indexed FP8 two-way vertical dot product to single-precision two registers

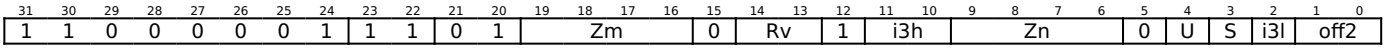
The encodings in this section are decoded from SME2 Multi-vector - Indexed (Two registers).



Decode fields T	Instruction Details	Condition
0	FVDOTB	FEAT_SME_F8F32
1	FVDOTT	FEAT_SME_F8F32

SME2 multi-vec indexed long MLA two sources

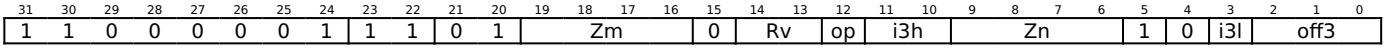
The encodings in this section are decoded from SME2 Multi-vector - Indexed (Two registers).



Decode fields U	Decode fields S	Instruction Details	Condition
0	0	SMLAL (multiple and indexed vector)	FEAT_SME2
0	1	SMLSL (multiple and indexed vector)	FEAT_SME2
1	0	UMLAL (multiple and indexed vector)	FEAT_SME2
1	1	UMLSL (multiple and indexed vector)	FEAT_SME2

SME2 multi-vec indexed FP8 two-way dot product to FP16 two registers

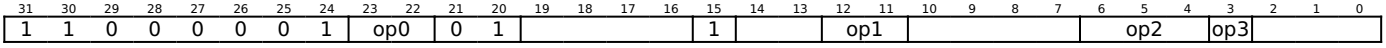
The encodings in this section are decoded from SME2 Multi-vector - Indexed (Two registers).



Decode fields op	Instruction Details	Condition
0	FDOT (2-way, multiple and indexed vector, FP8 to FP16)	FEAT_SME_F8F16
1	FVDOT (FP8 to FP16)	FEAT_SME_F8F16

SME2 Multi-vector - Indexed (Four registers)

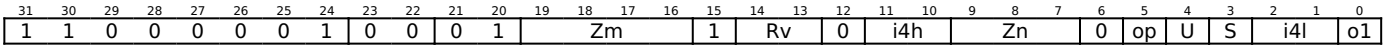
The encodings in this section are decoded from SME encodings.



Decode fields				Instruction details	Condition
op0	op1	op2	op3		
00	0x	0xx		SME2 multi-vec indexed long long MLA four sources 32-bit	-
00	0x	100	0	FMLALL (multiple and indexed vector) — Four ZA quad-vectors	FEAT_SME_F8F32
00	0x	100	1	UNALLOCATED	-
00	1x	0xx		SME2 multi-vec ternary indexed four registers 16-bit	-
00	1x	100		FDOT (2-way, multiple and indexed vector, FP8 to FP16) — Four ZA single-vectors	FEAT_SME_F8F16
00		101		UNALLOCATED	-
00		11x		UNALLOCATED	-
01		0xx		SME2 multi-vec ternary indexed four registers 32-bit	-
10	00	00x		SME2 multi-vec indexed long long MLA four sources 64-bit	-
10	01	00x		UNALLOCATED	-
10	0x	01x		UNALLOCATED	-
10	1x	00x		SME2 multi-vec indexed long FMA four sources	-
10	1x	010		FMLAL (multiple and indexed vector, FP8 to FP16) — Four ZA double-vectors	FEAT_SME_F8F16
10	1x	011		UNALLOCATED	-
11	0x	00x		SME2 multi-vec ternary indexed four registers 64-bit	-
11	1x	00x		SME2 multi-vec indexed long MLA four sources	-
11		01x		UNALLOCATED	-
!= 00		1xx		UNALLOCATED	-

SME2 multi-vec indexed long long MLA four sources 32-bit

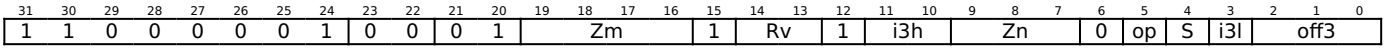
The encodings in this section are decoded from SME2 Multi-vector - Indexed (Four registers).



Decode fields			Instruction Details	Condition
op	U	S		
0	0	0	SMLALL (multiple and indexed vector)	FEAT_SME2
0	0	1	SMLSLL (multiple and indexed vector)	FEAT_SME2
0	1	0	UMLALL (multiple and indexed vector)	FEAT_SME2
0	1	1	UMLSLL (multiple and indexed vector)	FEAT_SME2
1		1	UNALLOCATED	-
1	0	0	USMLALL (multiple and indexed vector)	FEAT_SME2
1	1	0	SUMLALL (multiple and indexed vector)	FEAT_SME2

SME2 multi-vec ternary indexed four registers 16-bit

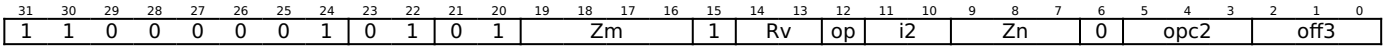
The encodings in this section are decoded from SME2 Multi-vector - Indexed (Four registers).



Decode fields		Instruction Details	Condition
op	S		
0	0	FMLA (multiple and indexed vector)	FEAT_SME_F16F16
0	1	FMLS (multiple and indexed vector)	FEAT_SME_F16F16
1	0	BFMLA (multiple and indexed vector)	FEAT_SME_B16B16
1	1	BFMLS (multiple and indexed vector)	FEAT_SME_B16B16

SME2 multi-vec ternary indexed four registers 32-bit

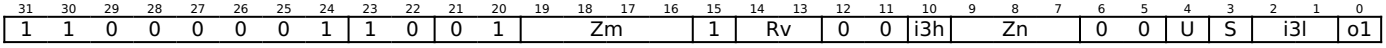
The encodings in this section are decoded from SME2 Multi-vector - Indexed (Four registers).



Decode fields op	Decode fields opc2	Instruction Details	Condition
0	000	FMLA (multiple and indexed vector)	FEAT_SME2
0	001	FDOT (4-way, multiple and indexed vector)	FEAT_SME_F8F32
0	010	FMLS (multiple and indexed vector)	FEAT_SME2
0	011	UNALLOCATED	-
0	100	SVDOT (4-way)	FEAT_SME2
0	101	USVDOT	FEAT_SME2
0	110	UVDOT (4-way)	FEAT_SME2
0	111	SUVDOT	FEAT_SME2
1	000	SDOT (2-way, multiple and indexed vector)	FEAT_SME2
1	001	FDOT (2-way, multiple and indexed vector, FP16 to FP32)	FEAT_SME2
1	010	UDOT (2-way, multiple and indexed vector)	FEAT_SME2
1	011	BFDOT (multiple and indexed vector)	FEAT_SME2
1	100	SDOT (4-way, multiple and indexed vector)	FEAT_SME2
1	101	USDOT (4-way, multiple and indexed vector)	FEAT_SME2
1	110	UDOT (4-way, multiple and indexed vector)	FEAT_SME2
1	111	SUDOT (4-way, multiple and indexed vector)	FEAT_SME2

SME2 multi-vec indexed long long MLA four sources 64-bit

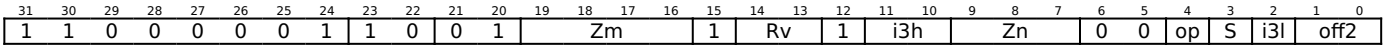
The encodings in this section are decoded from SME2 Multi-vector - Indexed (Four registers).



Decode fields U	Decode fields S	Instruction Details	Condition
0	0	SMLALL (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64
0	1	SMLSLL (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64
1	0	UMLALL (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64
1	1	UMLSLL (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64

SME2 multi-vec indexed long FMA four sources

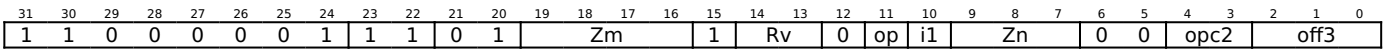
The encodings in this section are decoded from SME2 Multi-vector - Indexed (Four registers).



Decode fields op	Decode fields S	Instruction Details	Condition
0	0	FMLAL (multiple and indexed vector, FP16 to FP32)	FEAT_SME2
0	1	FMLSL (multiple and indexed vector)	FEAT_SME2
1	0	BFMLAL (multiple and indexed vector)	FEAT_SME2
1	1	BFMLSL (multiple and indexed vector)	FEAT_SME2

SME2 multi-vec ternary indexed four registers 64-bit

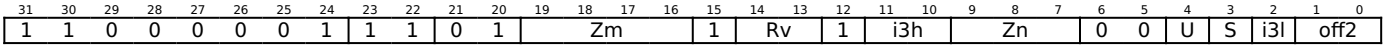
The encodings in this section are decoded from SME2 Multi-vector - Indexed (Four registers).



Decode fields op	Decode fields opc2	Instruction Details	Condition
0	00	FMLA (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_F64F64
0	01	SDOT (4-way, multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64
0	10	FMLS (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_F64F64
0	11	UDOT (4-way, multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64
1	x0	UNALLOCATED	-
1	01	SVDOT (4-way)	FEAT_SME2 && FEAT_SME_I16I64
1	11	UVDOT (4-way)	FEAT_SME2 && FEAT_SME_I16I64

SME2 multi-vec indexed long MLA four sources

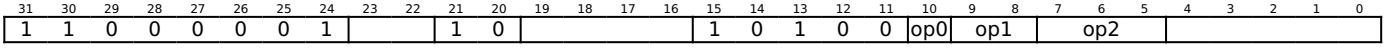
The encodings in this section are decoded from SME2 Multi-vector - Indexed (Four registers).



Decode fields U	Decode fields S	Instruction Details	Condition
0	0	SMLAL (multiple and indexed vector)	FEAT_SME2
0	1	SMLSL (multiple and indexed vector)	FEAT_SME2
1	0	UMLAL (multiple and indexed vector)	FEAT_SME2
1	1	UMLSL (multiple and indexed vector)	FEAT_SME2

SME2 Multi-vector - Multiple and Single SVE Destructive (Two registers)

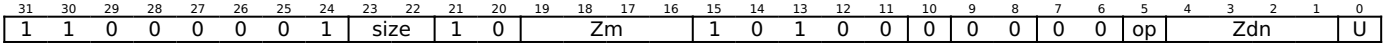
The encodings in this section are decoded from SME encodings.



Decode fields op0	Decode fields op1	Decode fields op2	Instruction details
0	00	00x	SME2 single-multi int min/max two registers
0	01	00x	SME2 single-multi FP min/max two registers
0	01	100	SME2 multiple and single vector FSCALE two registers
0	10		SME2 single-multi shift two registers
0	11	000	SME2 single-multi add two registers
0	11	001	UNALLOCATED
0	11	10x	UNALLOCATED
0	!= 1x	101	UNALLOCATED
0	!= 10	x1x	UNALLOCATED
1	00	000	SME2 single-multi signed saturating doubling multiply high two registers
1	00	!= x00	UNALLOCATED
1	!= 00		UNALLOCATED
	00	100	UNALLOCATED

SME2 single-multi int min/max two registers

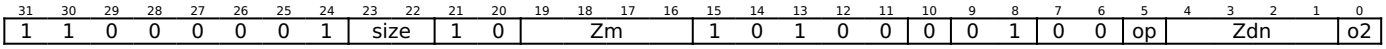
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single SVE Destructive (Two registers).



Decode fields		Instruction Details	Condition
op	U		
0	0	SMAX (multiple and single vector)	FEAT_SME2
0	1	UMAX (multiple and single vector)	FEAT_SME2
1	0	SMIN (multiple and single vector)	FEAT_SME2
1	1	UMIN (multiple and single vector)	FEAT_SME2

SME2 single-multi FP min/max two registers

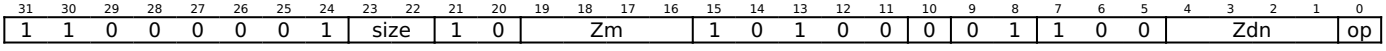
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single SVE Destructive (Two registers).



Decode fields			Instruction Details	Condition
size	op	o2		
00	0	0	BFMAX (multiple and single vector)	FEAT_SME2 && FEAT_SVE_B16B16
00	0	1	BFMIN (multiple and single vector)	FEAT_SME2 && FEAT_SVE_B16B16
00	1	0	BFMAXNM (multiple and single vector)	FEAT_SME2 && FEAT_SVE_B16B16
00	1	1	BFMINNM (multiple and single vector)	FEAT_SME2 && FEAT_SVE_B16B16
!= 00	0	0	FMAX (multiple and single vector)	FEAT_SME2
!= 00	0	1	FMIN (multiple and single vector)	FEAT_SME2
!= 00	1	0	FMAXNM (multiple and single vector)	FEAT_SME2
!= 00	1	1	FMINNM (multiple and single vector)	FEAT_SME2

SME2 multiple and single vector FSCALE two registers

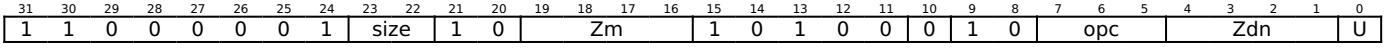
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single SVE Destructive (Two registers).



Decode fields		Instruction Details	Condition
size	op		
	1	UNALLOCATED	-
00	0	BFSCALE (multiple and single vector)	FEAT_SME2 && FEAT_SVE_BFSCALE
!= 00	0	FSCALE (multiple and single vector)	FEAT_SME2 && FEAT_FP8

SME2 single-multi shift two registers

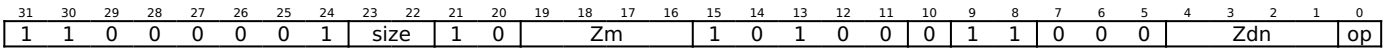
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single SVE Destructive (Two registers).



Decode fields		Instruction Details	Condition
opc	U		
001	0	SRSHL (multiple and single vector)	FEAT_SME2
001	1	URSHL (multiple and single vector)	FEAT_SME2
!= 001		UNALLOCATED	-

SME2 single-multi add two registers

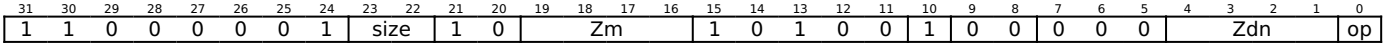
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single SVE Destructive (Two registers).



Decode fields op	Instruction Details	Condition
0	ADD (to vector, multiple vectors)	FEAT_SME2
1	UNALLOCATED	-

SME2 single-multi signed saturating doubling multiply high two registers

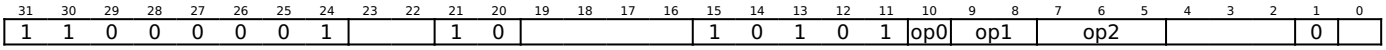
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single SVE Destructive (Two registers).



Decode fields op	Instruction Details	Condition
0	SQDMULH (multiple and single vector)	FEAT_SME2
1	UNALLOCATED	-

SME2 Multi-vector - Multiple and Single SVE Destructive (Four registers)

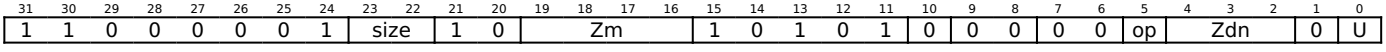
The encodings in this section are decoded from SME encodings.



Decode fields op0	op1	op2	Instruction details
0	00	00x	SME2 single-multi int min/max four registers
0	01	00x	SME2 single-multi FP min/max four registers
0	01	100	SME2 multiple and single vector FSCALE four registers
0	10		SME2 single-multi shift four registers
0	11	000	SME2 single-multi add four registers
0	11	001	UNALLOCATED
0	11	10x	UNALLOCATED
0	!= 1x	101	UNALLOCATED
0	!= 10	x1x	UNALLOCATED
1	00	000	SME2 single-multi signed saturating doubling multiply high four registers
1	00	!= x00	UNALLOCATED
1	!= 00		UNALLOCATED
	00	100	UNALLOCATED

SME2 single-multi int min/max four registers

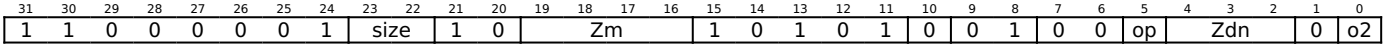
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single SVE Destructive (Four registers).



Decode fields op	U	Instruction Details	Condition
0	0	SMAX (multiple and single vector)	FEAT_SME2
0	1	UMAX (multiple and single vector)	FEAT_SME2
1	0	SMIN (multiple and single vector)	FEAT_SME2
1	1	UMIN (multiple and single vector)	FEAT_SME2

SME2 single-multi FP min/max four registers

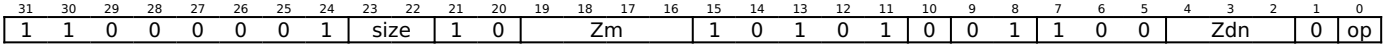
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single SVE Destructive (Four registers).



Decode fields size	op	o2	Instruction Details	Condition
00	0	0	BFMAX (multiple and single vector)	FEAT_SME2 && FEAT_SVE_B16B16
00	0	1	BFMIN (multiple and single vector)	FEAT_SME2 && FEAT_SVE_B16B16
00	1	0	BFMAXNM (multiple and single vector)	FEAT_SME2 && FEAT_SVE_B16B16
00	1	1	BFMINNM (multiple and single vector)	FEAT_SME2 && FEAT_SVE_B16B16
!= 00	0	0	FMAX (multiple and single vector)	FEAT_SME2
!= 00	0	1	FMIN (multiple and single vector)	FEAT_SME2
!= 00	1	0	FMAXNM (multiple and single vector)	FEAT_SME2
!= 00	1	1	FMINNM (multiple and single vector)	FEAT_SME2

SME2 multiple and single vector FSCALE four registers

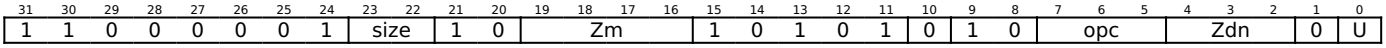
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single SVE Destructive (Four registers).



Decode fields size	op	Instruction Details	Condition
	1	UNALLOCATED	-
00	0	BFSCALE (multiple and single vector)	FEAT_SME2 && FEAT_SVE_BFSCALE
!= 00	0	FSCALE (multiple and single vector)	FEAT_SME2 && FEAT_FP8

SME2 single-multi shift four registers

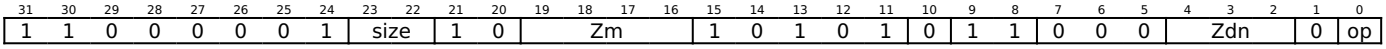
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single SVE Destructive (Four registers).



Decode fields opc	U	Instruction Details	Condition
001	0	SRSHL (multiple and single vector)	FEAT_SME2
001	1	URSHL (multiple and single vector)	FEAT_SME2
!= 001		UNALLOCATED	-

SME2 single-multi add four registers

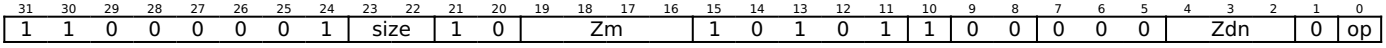
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single SVE Destructive (Four registers).



Decode fields op	Instruction Details	Condition
0	ADD (to vector, multiple vectors)	FEAT_SME2
1	UNALLOCATED	-

SME2 single-multi signed saturating doubling multiply high four registers

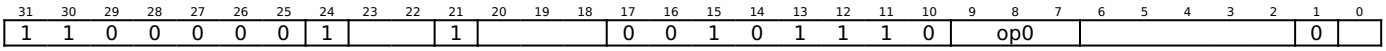
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single SVE Destructive (Four registers).



Decode fields op	Instruction Details	Condition
0	SQDMULH (multiple and single vector)	FEAT_SME2
1	UNALLOCATED	-

SME2 Multi-vector - Multiple Vectors SVE Destructive (Four registers)

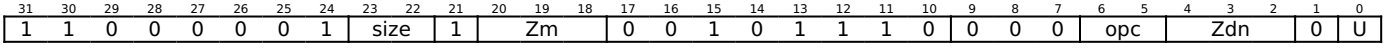
The encodings in this section are decoded from SME encodings.



Decode fields op0	Instruction details
000	SME2 multiple vectors int min/max four registers
001	UNALLOCATED
010	SME2 multiple vectors FP min/max four registers
011	SME2 multiple vectors FSCALE four registers
10x	SME2 multiple vectors shift four registers
11x	UNALLOCATED

SME2 multiple vectors int min/max four registers

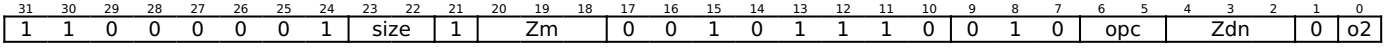
The encodings in this section are decoded from SME2 Multi-vector - Multiple Vectors SVE Destructive (Four registers).



Decode fields opc	U	Instruction Details	Condition
00	0	SMAX (multiple vectors)	FEAT_SME2
00	1	UMAX (multiple vectors)	FEAT_SME2
01	0	SMIN (multiple vectors)	FEAT_SME2
01	1	UMIN (multiple vectors)	FEAT_SME2
1x		UNALLOCATED	-

SME2 multiple vectors FP min/max four registers

The encodings in this section are decoded from SME2 Multi-vector - Multiple Vectors SVE Destructive (Four registers).



Decode fields			Instruction Details	Condition
size	opc	o2		
	10	0	FAMAX	FEAT_SME2 && FEAT_FAMINMAX
	10	1	FAMIN	FEAT_SME2 && FEAT_FAMINMAX
	11		UNALLOCATED	-
00	00	0	BFMAX (multiple vectors)	FEAT_SME2 && FEAT_SVE_B16B16
00	00	1	BFMIN (multiple vectors)	FEAT_SME2 && FEAT_SVE_B16B16
00	01	0	BFMAXNM (multiple vectors)	FEAT_SME2 && FEAT_SVE_B16B16
00	01	1	BFMINNM (multiple vectors)	FEAT_SME2 && FEAT_SVE_B16B16
!= 00	00	0	FMAX (multiple vectors)	FEAT_SME2
!= 00	00	1	FMIN (multiple vectors)	FEAT_SME2
!= 00	01	0	FMAXNM (multiple vectors)	FEAT_SME2
!= 00	01	1	FMINNM (multiple vectors)	FEAT_SME2

SME2 multiple vectors FSCALE four registers

The encodings in this section are decoded from SME2 Multi-vector - Multiple Vectors SVE Destructive (Four registers).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	size		1	Zm			0	0	1	0	1	1	1	0	0	1	1	opc		Zdn			0	o2

Decode fields			Instruction Details	Condition
size	opc	o2		
	00	1	UNALLOCATED	-
	!= 00		UNALLOCATED	-
00	00	0	BFSCALE (multiple vectors)	FEAT_SME2 && FEAT_SVE_BFSCALE
!= 00	00	0	FSCALE (multiple vectors)	FEAT_SME2 && FEAT_FP8

SME2 multiple vectors shift four registers

The encodings in this section are decoded from SME2 Multi-vector - Multiple Vectors SVE Destructive (Four registers).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	size		1	Zm			0	0	1	0	1	1	1	0	1	0	opc			Zdn			0	U

Decode fields		Instruction Details	Condition
opc	U		
001	0	SRSHL (multiple vectors)	FEAT_SME2
001	1	URSHL (multiple vectors)	FEAT_SME2
!= 001		UNALLOCATED	-

SME2 Multi-vector - Multiple Vectors SVE Saturating Multiply (Four registers)

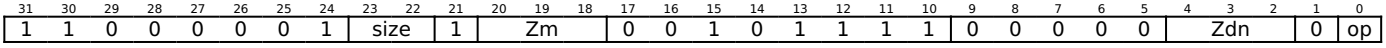
The encodings in this section are decoded from SME encodings.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1			1				0	0	1	0	1	1	1	1	op0								0	

Decode fields	Instruction details
op0	
00000	SME2 multi-vector signed saturating doubling multiply high four registers
!= 00000	UNALLOCATED

SME2 multi-vector signed saturating doubling multiply high four registers

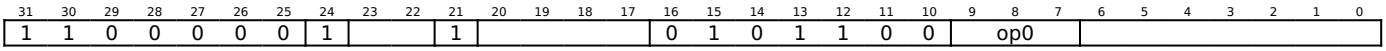
The encodings in this section are decoded from SME2 Multi-vector - Multiple Vectors SVE Saturating Multiply (Four registers).



Decode fields op	Instruction Details	Condition
0	SQDMULH (multiple vectors)	FEAT_SME2
1	UNALLOCATED	-

SME2 Multi-vector - Multiple Vectors SVE Destructive (Two registers)

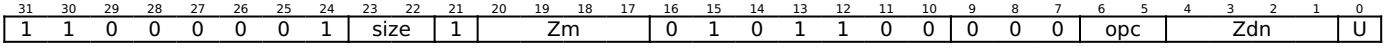
The encodings in this section are decoded from SME encodings.



Decode fields op0	Instruction details
000	SME2 multiple vectors int min/max two registers
001	UNALLOCATED
010	SME2 multiple vectors FP min/max two registers
011	SME2 multiple vectors FSCALE two registers
10x	SME2 multiple vectors shift two registers
11x	UNALLOCATED

SME2 multiple vectors int min/max two registers

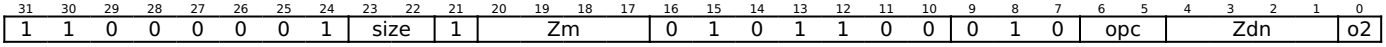
The encodings in this section are decoded from SME2 Multi-vector - Multiple Vectors SVE Destructive (Two registers).



Decode fields opc	U	Instruction Details	Condition
00	0	SMAX (multiple vectors)	FEAT_SME2
00	1	UMAX (multiple vectors)	FEAT_SME2
01	0	SMIN (multiple vectors)	FEAT_SME2
01	1	UMIN (multiple vectors)	FEAT_SME2
1x		UNALLOCATED	-

SME2 multiple vectors FP min/max two registers

The encodings in this section are decoded from SME2 Multi-vector - Multiple Vectors SVE Destructive (Two registers).



Decode fields			Instruction Details	Condition
size	opc	o2		
	10	0	FAMAX	FEAT_SME2 && FEAT_FAMINMAX
	10	1	FAMIN	FEAT_SME2 && FEAT_FAMINMAX
	11		UNALLOCATED	-
00	00	0	BFMAX (multiple vectors)	FEAT_SME2 && FEAT_SVE_B16B16
00	00	1	BFMIN (multiple vectors)	FEAT_SME2 && FEAT_SVE_B16B16
00	01	0	BFMAXNM (multiple vectors)	FEAT_SME2 && FEAT_SVE_B16B16
00	01	1	BFMINNM (multiple vectors)	FEAT_SME2 && FEAT_SVE_B16B16
!= 00	00	0	FMAX (multiple vectors)	FEAT_SME2
!= 00	00	1	FMIN (multiple vectors)	FEAT_SME2
!= 00	01	0	FMAXNM (multiple vectors)	FEAT_SME2
!= 00	01	1	FMINNM (multiple vectors)	FEAT_SME2

SME2 multiple vectors FSCALE two registers

The encodings in this section are decoded from SME2 Multi-vector - Multiple Vectors SVE Destructive (Two registers).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	size		1	Zm				0	1	0	1	1	0	0	0	1	1	opc		Zdn					o2

Decode fields			Instruction Details	Condition
size	opc	o2		
	00	1	UNALLOCATED	-
	!= 00		UNALLOCATED	-
00	00	0	BFSCALE (multiple vectors)	FEAT_SME2 && FEAT_SVE_BFSCALE
!= 00	00	0	FSCALE (multiple vectors)	FEAT_SME2 && FEAT_FP8

SME2 multiple vectors shift two registers

The encodings in this section are decoded from SME2 Multi-vector - Multiple Vectors SVE Destructive (Two registers).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	size		1	Zm				0	1	0	1	1	0	0	1	0	opc			Zdn					U

Decode fields		Instruction Details	Condition
opc	U		
001	0	SRSHL (multiple vectors)	FEAT_SME2
001	1	URSHL (multiple vectors)	FEAT_SME2
!= 001		UNALLOCATED	-

SME2 Multi-vector - Multiple Vectors SVE Saturating Multiply (Two registers)

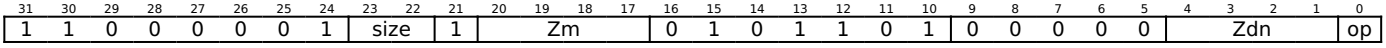
The encodings in this section are decoded from SME encodings.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1			1					0	1	0	1	1	0	1	op0									

Decode fields	Instruction details
op0	
00000	SME2 multi-vector signed saturating doubling multiply high two registers
!= 00000	UNALLOCATED

SME2 multi-vector signed saturating doubling multiply high two registers

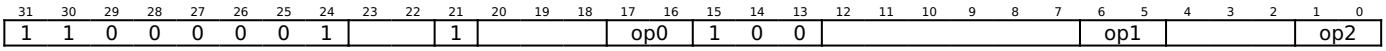
The encodings in this section are decoded from SME2 Multi-vector - Multiple Vectors SVE Saturating Multiply (Two registers).



Decode fields op	Instruction Details	Condition
0	SQDMULH (multiple vectors)	FEAT_SME2
1	UNALLOCATED	-

SME2 Multi-vector - SVE Select

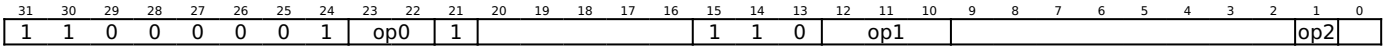
The encodings in this section are decoded from SME encodings.



Decode fields op0	Decode fields op1	Decode fields op2	Instruction details	Condition
01	00	00	SEL — Four registers	FEAT_SME2
01	00	10	UNALLOCATED	-
01	10	x0	UNALLOCATED	-
11	x0	x0	UNALLOCATED	-
x0	x0	x0	SEL — Two registers	FEAT_SME2
	x0	x1	UNALLOCATED	-
	x1		UNALLOCATED	-

SME2 Multi-vector - SVE Constructive Binary

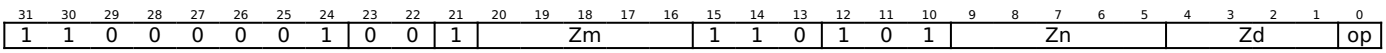
The encodings in this section are decoded from SME encodings.



Decode fields op0	Decode fields op1	Decode fields op2	Instruction details
00	101		SME2 multi-vec quadwords ZIP two registers
01	101		UNALLOCATED
10	101		UNALLOCATED
11	101		SME2 multi-vec saturating shift right narrow two registers
	000		SME2 multi-vec FCLAMP two registers
	001		SME2 multi-vec CLAMP two registers
	010	0	SME2 multi-vec FCLAMP four registers
	011	0	SME2 multi-vec CLAMP four registers
	01x	1	UNALLOCATED
	100		SME2 multi-vec ZIP two registers
	11x		SME2 multi-vec saturating shift right narrow four registers

SME2 multi-vec quadwords ZIP two registers

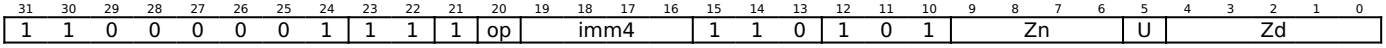
The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Binary.



Decode fields op	Instruction Details	Condition
0	ZIP (two registers)	FEAT_SME2
1	UZP (two registers)	FEAT_SME2

SME2 multi-vec saturating shift right narrow two registers

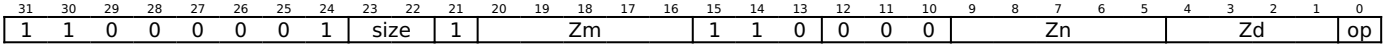
The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Binary.



Decode fields op	U	Instruction Details	Condition
0	0	SQRSHR (two registers)	FEAT_SME2
0	1	UQRSHR (two registers)	FEAT_SME2
1	0	SQRSHRU (two registers)	FEAT_SME2
1	1	UNALLOCATED	-

SME2 multi-vec FCLAMP two registers

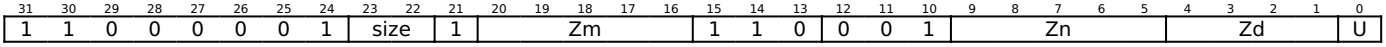
The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Binary.



Decode fields size	op	Instruction Details	Condition
	1	UNALLOCATED	-
00	0	BFCLAMP	FEAT_SME2 && FEAT_SVE_B16B16
!= 00	0	FCLAMP	FEAT_SME2

SME2 multi-vec CLAMP two registers

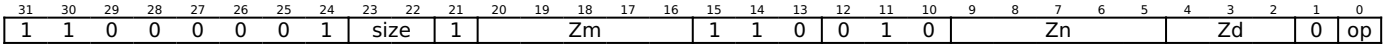
The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Binary.



Decode fields U	Instruction Details	Condition
0	SCLAMP	FEAT_SME2
1	UCLAMP	FEAT_SME2

SME2 multi-vec FCLAMP four registers

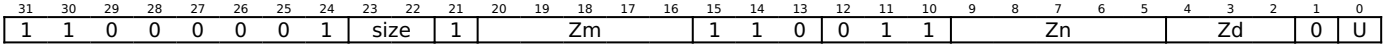
The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Binary.



Decode fields size	op	Instruction Details	Condition
	1	UNALLOCATED	-
00	0	BFCLAMP	FEAT_SME2 && FEAT_SVE_B16B16
!= 00	0	FCLAMP	FEAT_SME2

SME2 multi-vec CLAMP four registers

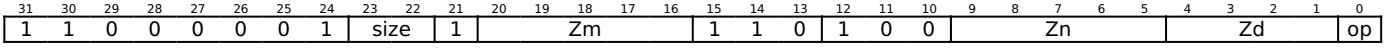
The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Binary.



Decode fields U	Instruction Details	Condition
0	SCLAMP	FEAT_SME2
1	UCLAMP	FEAT_SME2

SME2 multi-vec ZIP two registers

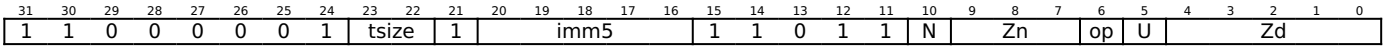
The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Binary.



Decode fields op	Instruction Details	Condition
0	ZIP (two registers)	FEAT_SME2
1	UZP (two registers)	FEAT_SME2

SME2 multi-vec saturating shift right narrow four registers

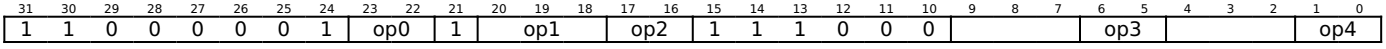
The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Binary.



Decode fields N	op	U	Instruction Details	Condition
	1	1	UNALLOCATED	-
0	0	0	SQRSHR (four registers)	FEAT_SME2
0	0	1	UQRSHR (four registers)	FEAT_SME2
0	1	0	SQRSHRU (four registers)	FEAT_SME2
1	0	0	SQRSHRN	FEAT_SME2
1	0	1	UQRSHRN	FEAT_SME2
1	1	0	SQRSHRUN	FEAT_SME2

SME2 Multi-vector - SVE Constructive Unary

The encodings in this section are decoded from SME encodings.

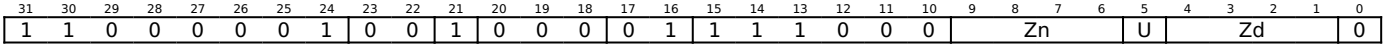


Decode fields					Instruction details
op0	op1	op2	op3	op4	
00	000	01		x0	SME2 multi-vec FP to int convert two registers
00	000	10		x0	SME2 multi-vec int to FP two registers
00	001	11			UNALLOCATED
00	100	01	0x	00	SME2 multi-vec FP to int convert four registers
00	100	10	0x	00	SME2 multi-vec int to FP four registers
00	101	00	0x		SME2 multi-vec FP8 down convert four registers
00	101	11	00	x0	SME2 multi-vec quadwords ZIP four registers
01	000	01		x0	UNALLOCATED
01	001	11			UNALLOCATED
0x	000	00			SME2 multi-vec FP down convert two registers
0x	000	11			SME2 multi-vec int down convert two registers
0x	001	00	x0		SME2 multi-vec FP8 down convert two registers
0x	001	00	x1		UNALLOCATED
10	000	00			SME2 multi-vec convert two registers
10	001	00	1x		UNALLOCATED
10	x01	00	0x		UNALLOCATED
11	00x	00			UNALLOCATED
1x	000	11		x1	UNALLOCATED
1x	000	x1		x0	UNALLOCATED
1x	001	11			UNALLOCATED
x0	100	00	0x	0x	UNALLOCATED
	000	01		x1	UNALLOCATED
	000	10		x1	UNALLOCATED
	001	01			SME2 multi-vec unpack two registers
	001	10			SME2 multi-vec FP8 up convert two registers
	01x		x0	x0	SME2 multi-vec FRINT two registers
	100	01	0x	01	UNALLOCATED
	100	01	1x		UNALLOCATED
	100	10	0x	01	UNALLOCATED
	100	11			SME2 multi-vec int down convert four registers
	100	!= 11	0x	1x	UNALLOCATED
	101	01	x0	0x	SME2 multi-vec unpack four registers
	101	01	x0	1x	UNALLOCATED
	101	01	x1		UNALLOCATED
	101	10	00	x0	SME2 multi-vec ZIP four registers
	101	11	1x		UNALLOCATED
	101	1x	00	x1	UNALLOCATED
	101	1x	01		UNALLOCATED
	10x	x0	1x		UNALLOCATED
	11x		00	00	SME2 multi-vec FRINT four registers
	11x		00	10	UNALLOCATED
	11x		10	x0	UNALLOCATED
	x1x		x0	x1	UNALLOCATED
	x1x		x1		UNALLOCATED
!= x0	101	00	0x	1x	UNALLOCATED

!= x0	10x	00	0x	0x	UNALLOCATED
!= 00	000	10		x0	UNALLOCATED
!= 00	100	01	0x	00	UNALLOCATED
!= 00	100	10	0x	00	UNALLOCATED
!= 00	101	11	00	x0	UNALLOCATED

SME2 multi-vec FP to int convert two registers

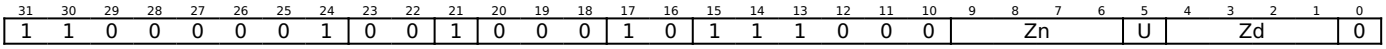
The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Unary.



Decode fields U	Instruction Details	Condition
0	FCVTZS	FEAT_SME2
1	FCVTZU	FEAT_SME2

SME2 multi-vec int to FP two registers

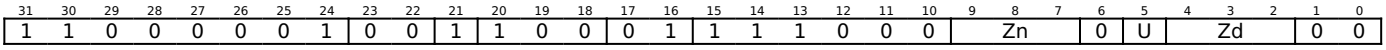
The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Unary.



Decode fields U	Instruction Details	Condition
0	SCVTF	FEAT_SME2
1	UCVTF	FEAT_SME2

SME2 multi-vec FP to int convert four registers

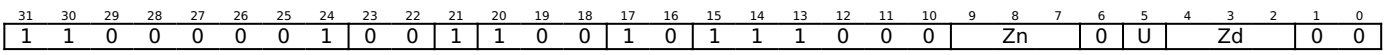
The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Unary.



Decode fields U	Instruction Details	Condition
0	FCVTZS	FEAT_SME2
1	FCVTZU	FEAT_SME2

SME2 multi-vec int to FP four registers

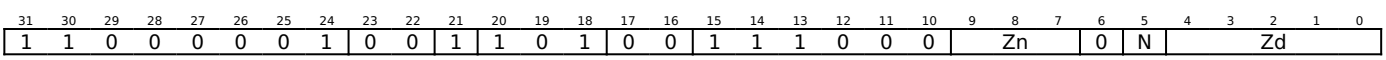
The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Unary.



Decode fields U	Instruction Details	Condition
0	SCVTF	FEAT_SME2
1	UCVTF	FEAT_SME2

SME2 multi-vec FP8 down convert four registers

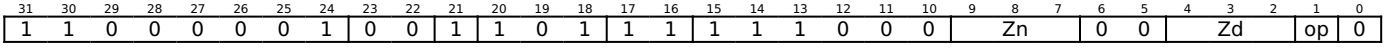
The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Unary.



Decode fields N	Instruction Details	Condition
0	FCVT (narrowing, FP32 to FP8)	FEAT_SME2 && FEAT_FP8
1	FCVTN (FP32 to FP8)	FEAT_SME2 && FEAT_FP8

SME2 multi-vec quadwords ZIP four registers

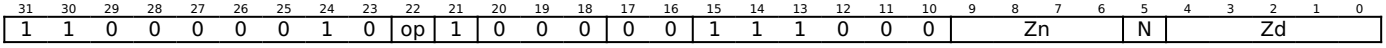
The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Unary.



Decode fields op	Instruction Details	Condition
0	ZIP (four registers)	FEAT_SME2
1	UZP (four registers)	FEAT_SME2

SME2 multi-vec FP down convert two registers

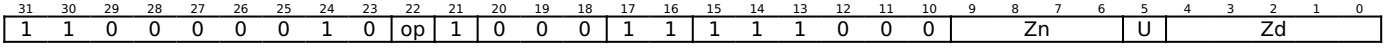
The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Unary.



Decode fields op	N	Instruction Details	Condition
0	0	FCVT (narrowing, FP32 to FP16)	FEAT_SME2
0	1	FCVTN (FP32 to FP16)	FEAT_SME2
1	0	BFCVT (single-precision to BFloat16)	FEAT_SME2
1	1	BFCVTN	FEAT_SME2

SME2 multi-vec int down convert two registers

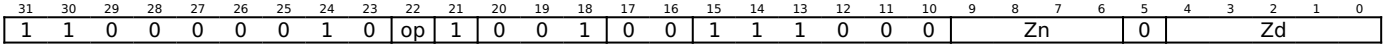
The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Unary.



Decode fields op	U	Instruction Details	Condition
0	0	SQCVT (two registers)	FEAT_SME2
0	1	UQCVT (two registers)	FEAT_SME2
1	0	SQCVTU (two registers)	FEAT_SME2
1	1	UNALLOCATED	-

SME2 multi-vec FP8 down convert two registers

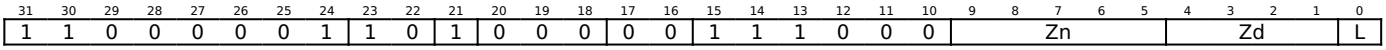
The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Unary.



Decode fields op	Instruction Details	Condition
0	FCVT (narrowing, FP16 to FP8)	FEAT_SME2 && FEAT_FP8
1	BFCVT (BFloat16 to 8-bit floating-point)	FEAT_SME2 && FEAT_FP8

SME2 multi-vec convert two registers

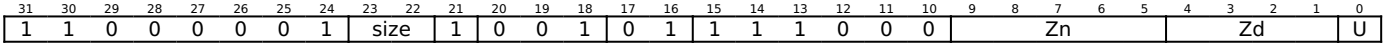
The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Unary.



Decode fields L	Instruction Details	Condition
0	FCVT (widening)	FEAT_SME_F16F16
1	FCVTL	FEAT_SME_F16F16

SME2 multi-vec unpack two registers

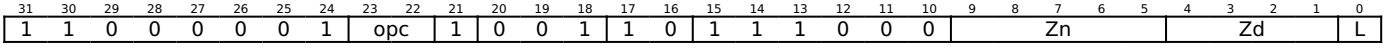
The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Unary.



Decode fields U	Instruction Details	Condition
0	SUNPK	FEAT_SME2
1	UUNPK	FEAT_SME2

SME2 multi-vec FP8 up convert two registers

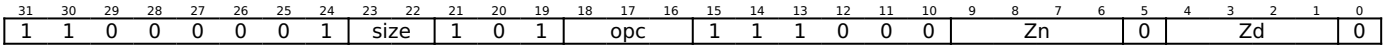
The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Unary.



Decode fields opc	Decode fields L	Instruction Details	Condition
00	0	F1CVT, F2CVT — F1CVT	FEAT_SME2 && FEAT_FP8
00	1	F1CVTL, F2CVTL — F1CVTL	FEAT_SME2 && FEAT_FP8
01	0	BF1CVT, BF2CVT — BF1CVT	FEAT_SME2 && FEAT_FP8
01	1	BF1CVTL, BF2CVTL — BF1CVTL	FEAT_SME2 && FEAT_FP8
10	0	F1CVT, F2CVT — F2CVT	FEAT_SME2 && FEAT_FP8
10	1	F1CVTL, F2CVTL — F2CVTL	FEAT_SME2 && FEAT_FP8
11	0	BF1CVT, BF2CVT — BF2CVT	FEAT_SME2 && FEAT_FP8
11	1	BF1CVTL, BF2CVTL — BF2CVTL	FEAT_SME2 && FEAT_FP8

SME2 multi-vec FRINT two registers

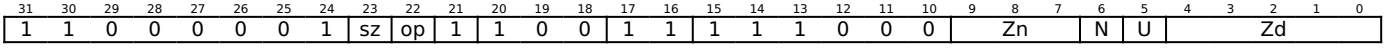
The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Unary.



Decode fields size	Decode fields opc	Instruction Details	Condition
10	000	FRINTN	FEAT_SME2
10	001	FRINTP	FEAT_SME2
10	010	FRINTM	FEAT_SME2
10	011	UNALLOCATED	-
10	100	FRINTA	FEAT_SME2
10	101	UNALLOCATED	-
10	11x	UNALLOCATED	-
!= 10		UNALLOCATED	-

SME2 multi-vec int down convert four registers

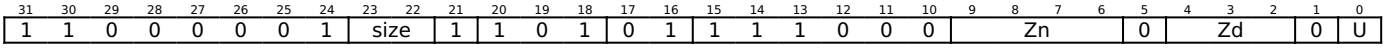
The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Unary.



Decode fields			Instruction Details	Condition
op	N	U		
0	0	0	SQCVT (four registers)	FEAT_SME2
0	0	1	UQCVT (four registers)	FEAT_SME2
0	1	0	SQCVTN	FEAT_SME2
0	1	1	UQCVTN	FEAT_SME2
1		1	UNALLOCATED	-
1	0	0	SQCVTU (four registers)	FEAT_SME2
1	1	0	SQCVTUN	FEAT_SME2

SME2 multi-vec unpack four registers

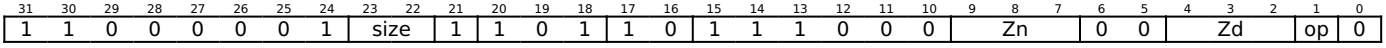
The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Unary.



Decode fields	Instruction Details	Condition
U		
0	SUNPK	FEAT_SME2
1	UUNPK	FEAT_SME2

SME2 multi-vec ZIP four registers

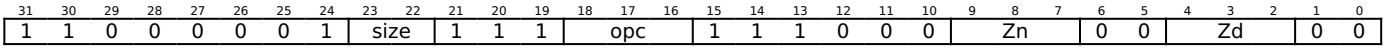
The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Unary.



Decode fields	Instruction Details	Condition
op		
0	ZIP (four registers)	FEAT_SME2
1	UZP (four registers)	FEAT_SME2

SME2 multi-vec FRINT four registers

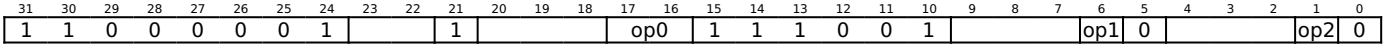
The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Unary.



Decode fields		Instruction Details	Condition
size	opc		
10	000	FRINTN	FEAT_SME2
10	001	FRINTP	FEAT_SME2
10	010	FRINTM	FEAT_SME2
10	011	UNALLOCATED	-
10	100	FRINTA	FEAT_SME2
10	101	UNALLOCATED	-
10	11x	UNALLOCATED	-
!= 10		UNALLOCATED	-

SME2 Multi-vector - FP Multiply

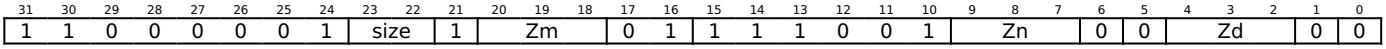
The encodings in this section are decoded from SME encodings.



Decode fields op0	op1	op2	Instruction details
01	0	0	SME2 multi-vec FP multiply (four registers)
01	0	1	UNALLOCATED
01	1		UNALLOCATED
11			UNALLOCATED
x0			SME2 multi-vec FP multiply (two registers)

SME2 multi-vec FP multiply (four registers)

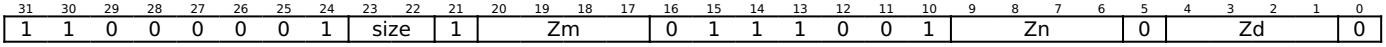
The encodings in this section are decoded from SME2 Multi-vector - FP Multiply.



Decode fields size	Instruction Details	Condition
00	BFMUL (multiple vectors)	FEAT_SME2 && FEAT_SVE_BFSCALE
!= 00	FMUL (multiple vectors)	FEAT_SME2p2

SME2 multi-vec FP multiply (two registers)

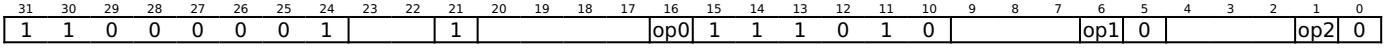
The encodings in this section are decoded from SME2 Multi-vector - FP Multiply.



Decode fields size	Instruction Details	Condition
00	BFMUL (multiple vectors)	FEAT_SME2 && FEAT_SVE_BFSCALE
!= 00	FMUL (multiple vectors)	FEAT_SME2p2

SME2 Multiple and Single Vector - FP multiply

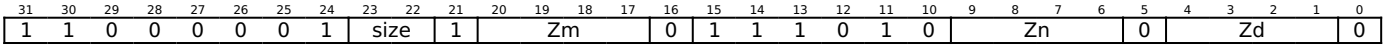
The encodings in this section are decoded from SME encodings.



Decode fields op0	op1	op2	Instruction details
0			SME2 multiple and single vector FP multiply (two registers)
1	0	0	SME2 multiple and single vector FP multiply (four registers)
1	0	1	UNALLOCATED
1	1		UNALLOCATED

SME2 multiple and single vector FP multiply (two registers)

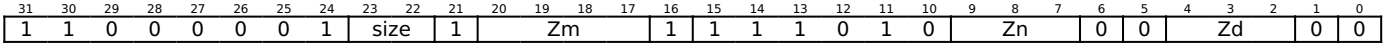
The encodings in this section are decoded from SME2 Multiple and Single Vector - FP multiply.



Decode fields size	Instruction Details	Condition
00	BFMUL (multiple and single vector)	FEAT_SME2 && FEAT_SVE_BFSCALE
!= 00	FMUL (multiple and single vector)	FEAT_SME2p2

SME2 multiple and single vector FP multiply (four registers)

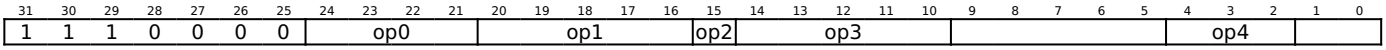
The encodings in this section are decoded from SME2 Multiple and Single Vector - FP multiply.



Decode fields size	Instruction Details	Condition
00	BFMUL (multiple and single vector)	FEAT_SME2 && FEAT_SVE_BFSCALE
!= 00	FMUL (multiple and single vector)	FEAT_SME2p2

SME Memory

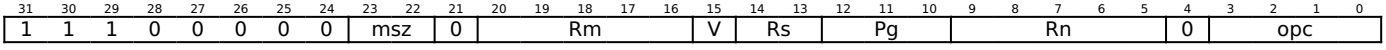
The encodings in this section are decoded from SME encodings.



Decode fields op0	op1	op2	op3	op4	Instruction details	Condition
0xx0				0xx	SME load array vector (elements)	-
0xx1				0xx	SME store array vector (elements)	-
100x	00000	0	xx000	0xx	SME save and restore array	-
100x		1	00000	000	SME2 lookup table load/store	-
100x		1	00000	001	UNALLOCATED	-
100x		1	00000	01x	UNALLOCATED	-
100x		1	01000	0xx	UNALLOCATED	-
100x		1	1x000	0xx	UNALLOCATED	-
100x			!= xx000	0xx	UNALLOCATED	-
100x	!= 00000	0	xx000	0xx	UNALLOCATED	-
101x				0xx	UNALLOCATED	-
110x				0xx	UNALLOCATED	-
1110				0xx	LD1Q	FEAT_SME
1111				0xx	ST1Q	FEAT_SME
				1xx	UNALLOCATED	-

SME load array vector (elements)

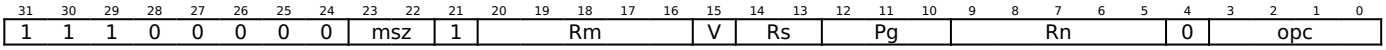
The encodings in this section are decoded from SME Memory.



Decode fields msz	Instruction Details	Condition
00	LD1B (scalar plus scalar, tile slice)	FEAT_SME
01	LD1H (scalar plus scalar, tile slice)	FEAT_SME
10	LD1W (scalar plus scalar, tile slice)	FEAT_SME
11	LD1D (scalar plus scalar, tile slice)	FEAT_SME

SME store array vector (elements)

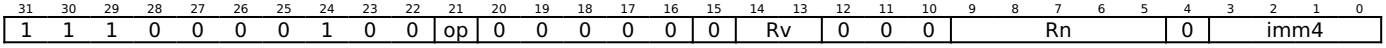
The encodings in this section are decoded from SME Memory.



Decode fields msz	Instruction Details	Condition
00	ST1B (scalar plus scalar, tile slice)	FEAT_SME
01	ST1H (scalar plus scalar, tile slice)	FEAT_SME
10	ST1W (scalar plus scalar, tile slice)	FEAT_SME
11	ST1D (scalar plus scalar, tile slice)	FEAT_SME

SME save and restore array

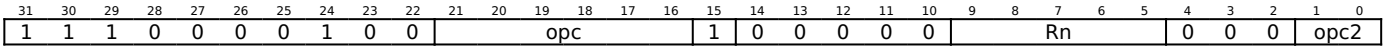
The encodings in this section are decoded from SME Memory.



Decode fields op	Instruction Details	Condition
0	LDR (array vector)	FEAT_SME
1	STR (array vector)	FEAT_SME

SME2 lookup table load/store

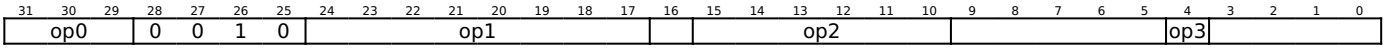
The encodings in this section are decoded from SME Memory.



Decode fields opc	Decode fields opc2	Instruction Details	Condition
x11111	!= 00	UNALLOCATED	-
!= x11111		UNALLOCATED	-
011111	00	LDR (table)	FEAT_SME2
111111	00	STR (table)	FEAT_SME2

SVE encodings

The encodings in this section are decoded from A64 instruction set encoding.



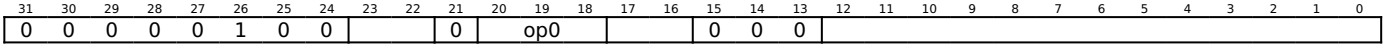
Decode fields				Instruction details	Condition
op0	op1	op2	op3		
000	0xx0xxxx	000xxx		SVE Integer Binary Arithmetic - Predicated	-
000	0xx0xxxx	001xxx		SVE Integer Reduction	-
000	0xx0xxxx	100xxx		SVE Bitwise Shift - Predicated	-
000	0xx0xxxx	101xxx		SVE Integer Unary Arithmetic - Predicated	-
000	0xx0xxxx	x1xxxx		SVE Integer Multiply-Add - Predicated	-
000	0xx1xxxx	001xxx		SVE Bitwise Logical - Unpredicated	-
000	0xx1xxxx	0100xx		SVE Index Generation	-
000	0xx1xxxx	0101xx		SVE Stack Allocation	-
000	0xx1xxxx	011xxx		SVE2 Integer Arithmetic - Unpredicated	-
000	0xx1xxxx	100xxx		SVE Bitwise Shift - Unpredicated	-
000	0xx1xxxx	1011xx		SVE Integer Misc - Unpredicated	-
000	0xx1xxxx	11xxxx		SVE Element Count	-
000	10x1xxxx	000xxx		SVE Permute Vector - Extract	-
000	11x1xxxx	000xxx		SVE Permute Vector - Segments	-
000	1xx00xxx			SVE Bitwise Immediate	-
000	1xx01xxx			SVE Integer Wide Immediate - Predicated	-
000	1xx1xxxx	001001		SVE Permute Vector - One Source Quadwords	-
000	1xx1xxxx	001110		SVE Permute Vector - Unpredicated	-
000	1xx1xxxx	001111		UNALLOCATED	-
000	1xx1xxxx	010xxx		SVE Permute Predicate	-
000	1xx1xxxx	10xxxx		SVE Permute Vector - Predicated	-
001	0xx0xxxx			SVE Integer Compare - Vectors	-
001	1xx00xxx	11xxxx		SVE Propagate Break	-
001	1xx01xxx	01xxxx		SVE Partition Break	-
001	1xx01xxx	11xxxx		SVE Predicate Misc	-
001	1xx100xx	10xxxx		SVE Predicate Count	-
001	1xx101xx	1000xx		SVE Inc/Dec by Predicate Count	-
001	1xx101xx	1001xx		SVE Write FFR	-
001	1xx101xx	101xxx		UNALLOCATED	-
001	1xx11xxx	10xxxx		UNALLOCATED	-
001	1xx1xxxx	00xxxx		SVE Integer Compare - Scalars	-
001	1xx1xxxx	01xxxx	1	SVE Scalar Integer Compare - Predicate-as-counter	-
001	1xx1xxxx	11xxxx		SVE Integer Wide Immediate - Unpredicated	-
010	0x10xxxx	11001x		UNALLOCATED	-
010	0xx0xxxx	0xxxxx		SVE Integer Multiply-Add - Unpredicated	-
010	0xx0xxxx	10xxxx		SVE2 Integer - Predicated	-
010	0xx1xxxx			SVE Multiply - Indexed	-
010	1xx0xxxx	0xxxxx		SVE2 Widening Integer Arithmetic	-
010	1xx0xxxx	10xxxx		SVE Misc	-
010	1xx0xxxx	11xxxx		SVE2 Accumulate	-
010	1xx1xxxx	0xxxxx		SVE2 Narrowing	-
010	1xx1xxxx	101xxx		SVE2 Histogram (Segment) and Lookup Table	-
010	1xx1xxxx	111xxx		SVE2 Crypto Extensions	-
011	0x01xxxx	0111xx		UNALLOCATED	-
011	0x01xxxx	10xx10		SVE2 FP8 widening multiply-add	-

011	0x01xxxx	10xx11		UNALLOCATED	-
011	0x01xxxx	11x1xx		UNALLOCATED	-
011	0x11xxxx	10xx1x		UNALLOCATED	-
011	0x11xxxx	x1x1xx		UNALLOCATED	-
011	0xx00001	100xxx		UNALLOCATED	-
011	0xx0010x	100xxx		UNALLOCATED	-
011	0xx00x0x	11xxxx		UNALLOCATED	-
011	0xx00x1x	1xxxxx		UNALLOCATED	-
011	0xx010xx	11xxxx		UNALLOCATED	-
011	0xx011xx	1xxxxx		SVE2 floating-point unary operations - zeroing predicated	-
011	0xx1xxxx	001011		UNALLOCATED	-
011	0xx1xxxx	0011xx		UNALLOCATED	-
011	0xx1xxxx	01x0xx		SVE floating-point widening multiply-add - indexed	-
011	0xx1xxxx	10x00x		SVE floating-point widening multiply-add	-
011	0xx1xxxx	10x10x		UNALLOCATED	-
011	0xx1xxxx	11101x		UNALLOCATED	-
011	1xx001xx	0010xx		UNALLOCATED	-
011	1xx001xx	0011xx		SVE floating-point unary operations - unpredicated	-
011	1xx010xx	001xxx		SVE floating-point compare - with zero	-
011	1xx011xx	001xxx		SVE floating-point accumulating reduction	-
011	1xx0xxxx	100xxx		SVE floating-point arithmetic - predicated	-
011	1xx0xxxx	101xxx		SVE floating-point unary operations - merging predicated	-
011	1xx1xxxx			SVE floating-point multiply-add	-
100				SVE Memory - 32-bit Gather and Unsized Contiguous	-
101				SVE Memory - Contiguous Load	-
110				SVE Memory - 64-bit Gather	-
111		001xxx		SVE Memory - Non-temporal and Quadword Scatter Store	-
111		011xxx		SVE Memory - Non-temporal and Multi-register Contiguous Store	-
111		0x0xxx		SVE Memory - Contiguous Store and Unsized Contiguous	-
111		101xxx		SVE Memory - Scatter	-
111		111xxx		SVE Memory - Contiguous Store with Immediate Offset	-
111		1x0xxx		SVE Memory - Scatter with Optional Sign Extend	-
000	0xx1xxxx	000xxx		SVE integer add/subtract vectors (unpredicated)	-
000	0xx1xxxx	1010xx		SVE address generation	-
000	1xx1xxxx	001000		DUP (indexed)	FEAT_SVE || FEAT_SME
000	1xx1xxxx	00101x		SVE table lookup (three sources)	-
000	1xx1xxxx	001100		TBL — Single register table	FEAT_SVE || FEAT_SME
000	1xx1xxxx	001101		TBXQ	FEAT_SVE2p1 || FEAT_SME2p1
000	1xx1xxxx	011xxx		SVE permute vector elements	-
000	1xx1xxxx	11xxxx		SEL (vectors)	FEAT_SVE || FEAT_SME
001	0xx1xxxx			SVE integer compare with unsigned immediate	-
001	1xx00xxx	01xxxx		SVE predicate logical operations	-
001	1xx0xxxx	x0xxxx		SVE integer compare with signed immediate	-
001	1xx1xxxx	01xxxx	0	SVE broadcast predicate element	-
010	0000xxxx	11001x		SVE two-way dot product	-

010	0100xxxx	11001x		SVE two-way dot product (indexed)	-
010	0xx0xxxx	11000x		SVE integer clamp	-
010	0xx0xxxx	1101x0		SVE2 multiply-add (checked pointer)	-
010	0xx0xxxx	1101x1		SVE2 absolute difference and accumulate long	-
010	0xx0xxxx	111xxx		SVE permute vector elements (quadwords)	-
010	1xx1xxxx	100xxx		SVE2 character match	-
010	1xx1xxxx	110xxx		HISTCNT	FEAT_SVE2
011	00x1xxxx	111000		SVE2 FP8 matrix multiply-accumulate	-
011	01x1xxxx	111000		SVE2 floating-point non-widening matrix-multiply accumulate	-
011	0x01xxxx	0101xx		SVE2 FP8 multiply-add long (indexed)	-
011	0xx00000	100xxx		FCADD	FEAT_SVE || FEAT_SME
011	0xx0000x	101xxx		SVE floating-point convert (top, predicated)	-
011	0xx0010x	101xxx		SVE floating-point convert precision odd elements	-
011	0xx010xx	100xxx		SVE2 floating-point pairwise operations	-
011	0xx010xx	101xxx		SVE floating-point recursive reduction (quadwords)	-
011	0xx0xxxx	0xxxxx		FCMLA (vectors)	FEAT_SVE || FEAT_SME
011	0xx1xxxx	0000xx		SVE floating-point multiply-add (indexed)	-
011	0xx1xxxx	0001xx		SVE floating-point complex multiply-add (indexed)	-
011	0xx1xxxx	001001		SVE FP clamp	-
011	0xx1xxxx	0010x0		SVE floating-point multiply (indexed)	-
011	0xx1xxxx	1100xx		SVE2 FP8 multiply-add long long (indexed)	-
011	0xx1xxxx	111001		SVE floating-point matrix multiply accumulate	-
011	1xx000xx	001xxx		SVE floating-point recursive reduction	-
011	1xx0xxxx	000xxx		SVE floating-point arithmetic (unpredicated)	-
011	1xx0xxxx	x1xxxx		SVE floating-point compare vectors	-

SVE Integer Binary Arithmetic - Predicated

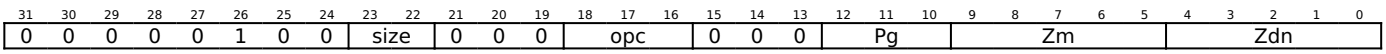
The encodings in this section are decoded from SVE encodings.



Decode fields op0	Instruction details
00x	SVE integer add/subtract vectors (predicated)
01x	SVE integer min/max/difference (predicated)
100	SVE integer multiply vectors (predicated)
101	SVE integer divide vectors (predicated)
11x	SVE bitwise logical operations (predicated)

SVE integer add/subtract vectors (predicated)

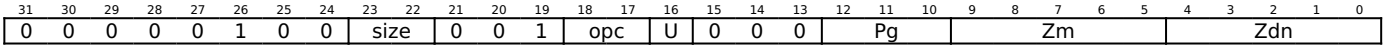
The encodings in this section are decoded from SVE Integer Binary Arithmetic - Predicated.



Decode fields		Instruction Details	Condition
size	opc		
	000	ADD (vectors, predicated)	FEAT_SVE || FEAT_SME
	001	SUB (vectors, predicated)	FEAT_SVE || FEAT_SME
	010	UNALLOCATED	-
	011	SUBR (vectors)	FEAT_SVE || FEAT_SME
11	100	ADDPT (predicated)	FEAT_SVE && FEAT_CPA
11	101	SUBPT (predicated)	FEAT_SVE && FEAT_CPA
11	11x	UNALLOCATED	-
!= 11	1xx	UNALLOCATED	-

SVE integer min/max/difference (predicated)

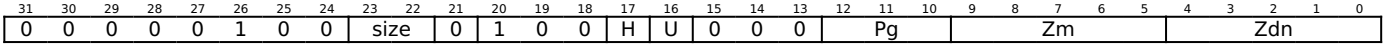
The encodings in this section are decoded from SVE Integer Binary Arithmetic - Predicated.



Decode fields		Instruction Details	Condition
opc	U		
00	0	SMAX (vectors)	FEAT_SVE || FEAT_SME
00	1	UMAX (vectors)	FEAT_SVE || FEAT_SME
01	0	SMIN (vectors)	FEAT_SVE || FEAT_SME
01	1	UMIN (vectors)	FEAT_SVE || FEAT_SME
10	0	SABD	FEAT_SVE || FEAT_SME
10	1	UABD	FEAT_SVE || FEAT_SME
11		UNALLOCATED	-

SVE integer multiply vectors (predicated)

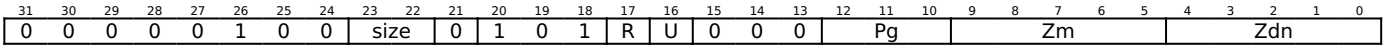
The encodings in this section are decoded from SVE Integer Binary Arithmetic - Predicated.



Decode fields		Instruction Details	Condition
H	U		
0	0	MUL (vectors, predicated)	FEAT_SVE || FEAT_SME
0	1	UNALLOCATED	-
1	0	SMULH (predicated)	FEAT_SVE || FEAT_SME
1	1	UMULH (predicated)	FEAT_SVE || FEAT_SME

SVE integer divide vectors (predicated)

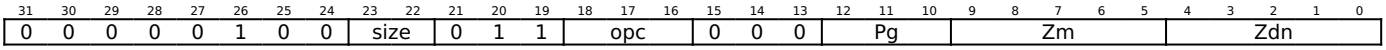
The encodings in this section are decoded from SVE Integer Binary Arithmetic - Predicated.



Decode fields		Instruction Details	Condition
R	U		
0	0	SDIV	FEAT_SVE || FEAT_SME
0	1	UDIV	FEAT_SVE || FEAT_SME
1	0	SDIVR	FEAT_SVE || FEAT_SME
1	1	UDIVR	FEAT_SVE || FEAT_SME

SVE bitwise logical operations (predicated)

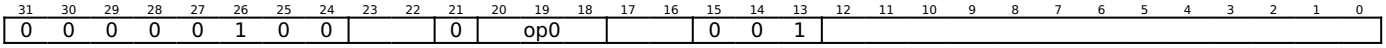
The encodings in this section are decoded from SVE Integer Binary Arithmetic - Predicated.



Decode fields opc	Instruction Details	Condition
000	ORR (vectors, predicated)	FEAT_SVE || FEAT_SME
001	EOR (vectors, predicated)	FEAT_SVE || FEAT_SME
010	AND (vectors, predicated)	FEAT_SVE || FEAT_SME
011	BIC (vectors, predicated)	FEAT_SVE || FEAT_SME
1xx	UNALLOCATED	-

SVE Integer Reduction

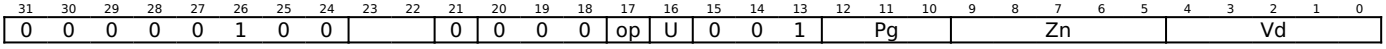
The encodings in this section are decoded from SVE encodings.



Decode fields op0	Instruction details
000	SVE integer add reduction (predicated)
001	SVE integer add reduction (quadwords)
010	SVE integer min/max reduction (predicated)
011	SVE integer min/max reduction (quadwords)
10x	SVE constructive prefix (predicated)
110	SVE bitwise logical reduction (predicated)
111	SVE bitwise logical reduction (quadwords)

SVE integer add reduction (predicated)

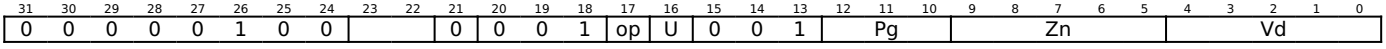
The encodings in this section are decoded from SVE Integer Reduction.



Decode fields op	U	Instruction Details	Condition
0	0	SADDV	FEAT_SVE || FEAT_SME
0	1	UADDV	FEAT_SVE || FEAT_SME
1		UNALLOCATED	-

SVE integer add reduction (quadwords)

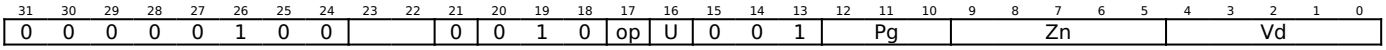
The encodings in this section are decoded from SVE Integer Reduction.



Decode fields op	U	Instruction Details	Condition
0	0	UNALLOCATED	-
0	1	ADDQV	FEAT_SVE2p1 || FEAT_SME2p1
1		UNALLOCATED	-

SVE integer min/max reduction (predicated)

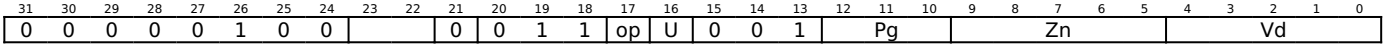
The encodings in this section are decoded from SVE Integer Reduction.



Decode fields		Instruction Details	Condition
op	U		
0	0	SMAXV	FEAT_SVE || FEAT_SME
0	1	UMAXV	FEAT_SVE || FEAT_SME
1	0	SMINV	FEAT_SVE || FEAT_SME
1	1	UMINV	FEAT_SVE || FEAT_SME

SVE integer min/max reduction (quadwords)

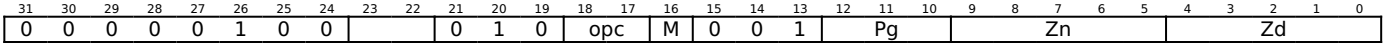
The encodings in this section are decoded from SVE Integer Reduction.



Decode fields		Instruction Details	Condition
op	U		
0	0	SMAXQV	FEAT_SVE2p1 || FEAT_SME2p1
0	1	UMAXQV	FEAT_SVE2p1 || FEAT_SME2p1
1	0	SMINQV	FEAT_SVE2p1 || FEAT_SME2p1
1	1	UMINQV	FEAT_SVE2p1 || FEAT_SME2p1

SVE constructive prefix (predicated)

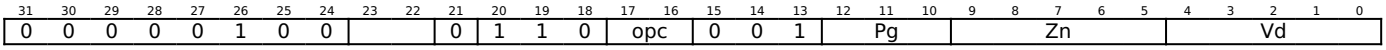
The encodings in this section are decoded from SVE Integer Reduction.



Decode fields	Instruction Details	Condition
opc		
00	MOVPRFX (predicated)	FEAT_SVE || FEAT_SME
!= 00	UNALLOCATED	-

SVE bitwise logical reduction (predicated)

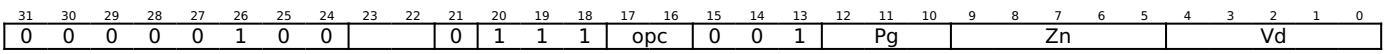
The encodings in this section are decoded from SVE Integer Reduction.



Decode fields	Instruction Details	Condition
opc		
00	ORV	FEAT_SVE || FEAT_SME
01	EORV	FEAT_SVE || FEAT_SME
10	ANDV	FEAT_SVE || FEAT_SME
11	UNALLOCATED	-

SVE bitwise logical reduction (quadwords)

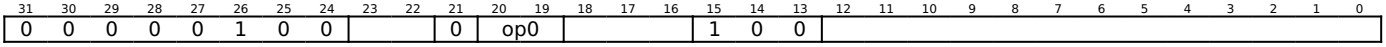
The encodings in this section are decoded from SVE Integer Reduction.



Decode fields opc	Instruction Details	Condition
00	ORQV	FEAT_SVE2p1 || FEAT_SME2p1
01	EORQV	FEAT_SVE2p1 || FEAT_SME2p1
10	ANDQV	FEAT_SVE2p1 || FEAT_SME2p1
11	UNALLOCATED	-

SVE Bitwise Shift - Predicated

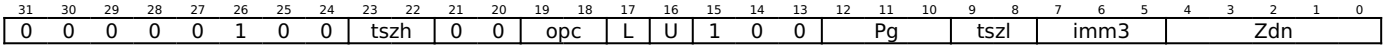
The encodings in this section are decoded from SVE encodings.



Decode fields op0	Instruction details
0x	SVE bitwise shift by immediate (predicated)
10	SVE bitwise shift by vector (predicated)
11	SVE bitwise shift by wide elements (predicated)

SVE bitwise shift by immediate (predicated)

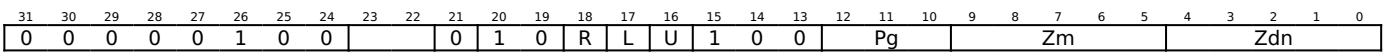
The encodings in this section are decoded from SVE Bitwise Shift - Predicated.



Decode fields opc	L	U	Instruction Details	Condition
00	0	0	ASR (immediate, predicated)	FEAT_SVE || FEAT_SME
00	0	1	LSR (immediate, predicated)	FEAT_SVE || FEAT_SME
00	1	0	UNALLOCATED	-
00	1	1	LSL (immediate, predicated)	FEAT_SVE || FEAT_SME
01	0	0	ASRD	FEAT_SVE || FEAT_SME
01	0	1	UNALLOCATED	-
01	1	0	SQSHL (immediate)	FEAT_SVE2 || FEAT_SME
01	1	1	UQSHL (immediate)	FEAT_SVE2 || FEAT_SME
10			UNALLOCATED	-
11	0	0	SRSHR	FEAT_SVE2 || FEAT_SME
11	0	1	URSHR	FEAT_SVE2 || FEAT_SME
11	1	0	UNALLOCATED	-
11	1	1	SQSHLU	FEAT_SVE2 || FEAT_SME

SVE bitwise shift by vector (predicated)

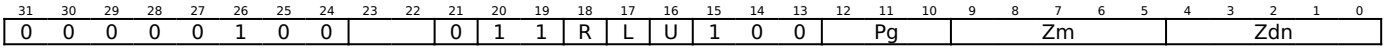
The encodings in this section are decoded from SVE Bitwise Shift - Predicated.



Decode fields R	L	U	Instruction Details	Condition
	1	0	UNALLOCATED	-
0	0	0	ASR (vectors)	FEAT_SVE || FEAT_SME
0	0	1	LSR (vectors)	FEAT_SVE || FEAT_SME
0	1	1	LSL (vectors)	FEAT_SVE || FEAT_SME
1	0	0	ASRR	FEAT_SVE || FEAT_SME
1	0	1	LSRR	FEAT_SVE || FEAT_SME
1	1	1	LSLR	FEAT_SVE || FEAT_SME

SVE bitwise shift by wide elements (predicated)

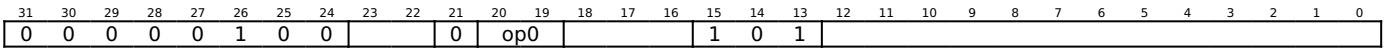
The encodings in this section are decoded from SVE Bitwise Shift - Predicated.



Decode fields R	L	U	Instruction Details	Condition
0	0	0	ASR (wide elements, predicated)	FEAT_SVE || FEAT_SME
0	0	1	LSR (wide elements, predicated)	FEAT_SVE || FEAT_SME
0	1	0	UNALLOCATED	-
0	1	1	LSL (wide elements, predicated)	FEAT_SVE || FEAT_SME
1			UNALLOCATED	-

SVE Integer Unary Arithmetic - Predicated

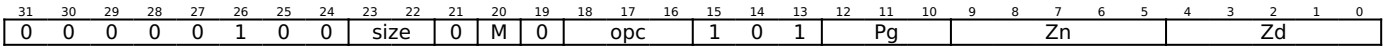
The encodings in this section are decoded from SVE encodings.



Decode fields op0	Instruction details
x0	SVE integer unary operations (predicated)
x1	SVE bitwise unary operations (predicated)

SVE integer unary operations (predicated)

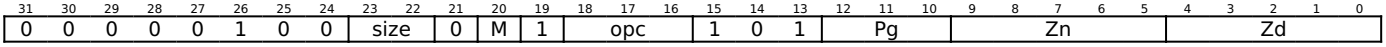
The encodings in this section are decoded from SVE Integer Unary Arithmetic - Predicated.



Decode fields		Instruction Details	Condition
M	opc		
0	000	SXTB, SXTH, SXTW — Byte, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
0	001	UXTB, UXTH, UXTW — Byte, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
0	010	SXTB, SXTH, SXTW — Halfword, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
0	011	UXTB, UXTH, UXTW — Halfword, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
0	100	SXTB, SXTH, SXTW — Word, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
0	101	UXTB, UXTH, UXTW — Word, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
0	110	ABS — Zeroing	FEAT_SVE2p2 || FEAT_SME2p2
0	111	NEG — Zeroing	FEAT_SVE2p2 || FEAT_SME2p2
1	000	SXTB, SXTH, SXTW — Byte, merging	FEAT_SVE || FEAT_SME
1	001	UXTB, UXTH, UXTW — Byte, merging	FEAT_SVE || FEAT_SME
1	010	SXTB, SXTH, SXTW — Halfword, merging	FEAT_SVE || FEAT_SME
1	011	UXTB, UXTH, UXTW — Halfword, merging	FEAT_SVE || FEAT_SME
1	100	SXTB, SXTH, SXTW — Word, merging	FEAT_SVE || FEAT_SME
1	101	UXTB, UXTH, UXTW — Word, merging	FEAT_SVE || FEAT_SME
1	110	ABS — Merging	FEAT_SVE || FEAT_SME
1	111	NEG — Merging	FEAT_SVE || FEAT_SME

SVE bitwise unary operations (predicated)

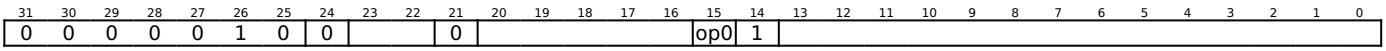
The encodings in this section are decoded from SVE Integer Unary Arithmetic - Predicated.



Decode fields		Instruction Details	Condition
M	opc		
	111	UNALLOCATED	-
0	000	CLS — Zeroing	FEAT_SVE2p2 || FEAT_SME2p2
0	001	CLZ — Zeroing	FEAT_SVE2p2 || FEAT_SME2p2
0	010	CNT — Zeroing	FEAT_SVE2p2 || FEAT_SME2p2
0	011	CNOT — Zeroing	FEAT_SVE2p2 || FEAT_SME2p2
0	100	FABS — Zeroing	FEAT_SVE2p2 || FEAT_SME2p2
0	101	FNEG — Zeroing	FEAT_SVE2p2 || FEAT_SME2p2
0	110	NOT (vector) — Zeroing	FEAT_SVE2p2 || FEAT_SME2p2
1	000	CLS — Merging	FEAT_SVE || FEAT_SME
1	001	CLZ — Merging	FEAT_SVE || FEAT_SME
1	010	CNT — Merging	FEAT_SVE || FEAT_SME
1	011	CNOT — Merging	FEAT_SVE || FEAT_SME
1	100	FABS — Merging	FEAT_SVE || FEAT_SME
1	101	FNEG — Merging	FEAT_SVE || FEAT_SME
1	110	NOT (vector) — Merging	FEAT_SVE || FEAT_SME

SVE Integer Multiply-Add - Predicated

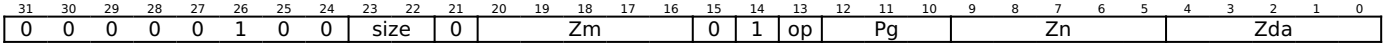
The encodings in this section are decoded from SVE encodings.



Decode fields op0	Instruction details
0	SVE integer multiply-accumulate writing addend (predicated)
1	SVE integer multiply-add writing multiplicand (predicated)

SVE integer multiply-accumulate writing addend (predicated)

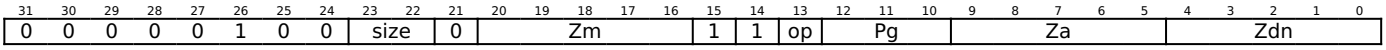
The encodings in this section are decoded from SVE Integer Multiply-Add - Predicated.



Decode fields op	Instruction Details	Condition
0	MLA (vectors)	FEAT_SVE || FEAT_SME
1	MLS (vectors)	FEAT_SVE || FEAT_SME

SVE integer multiply-add writing multiplicand (predicated)

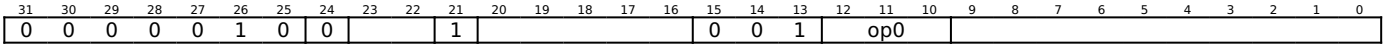
The encodings in this section are decoded from SVE Integer Multiply-Add - Predicated.



Decode fields op	Instruction Details	Condition
0	MAD	FEAT_SVE || FEAT_SME
1	MSB	FEAT_SVE || FEAT_SME

SVE Bitwise Logical - Unpredicated

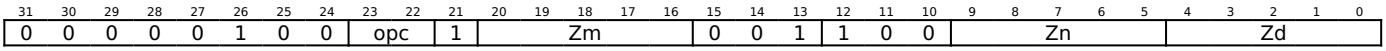
The encodings in this section are decoded from SVE encodings.



Decode fields op0	Instruction details	Condition
0xx	UNALLOCATED	-
100	SVE bitwise logical operations (unpredicated)	-
101	XAR	FEAT_SVE2 || FEAT_SME
11x	SVE2 bitwise ternary operations	-

SVE bitwise logical operations (unpredicated)

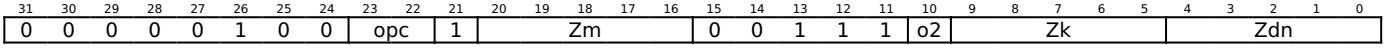
The encodings in this section are decoded from SVE Bitwise Logical - Unpredicated.



Decode fields opc	Instruction Details	Condition
00	AND (vectors, unpredicated)	FEAT_SVE || FEAT_SME
01	ORR (vectors, unpredicated)	FEAT_SVE || FEAT_SME
10	EOR (vectors, unpredicated)	FEAT_SVE || FEAT_SME
11	BIC (vectors, unpredicated)	FEAT_SVE || FEAT_SME

SVE2 bitwise ternary operations

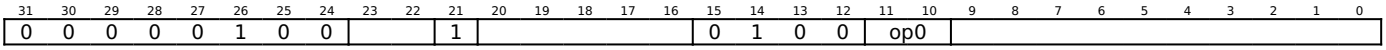
The encodings in this section are decoded from SVE Bitwise Logical - Unpredicated.



Decode fields opc	o2	Instruction Details	Condition
00	0	EOR3	FEAT_SVE2 || FEAT_SME
00	1	BSL	FEAT_SVE2 || FEAT_SME
01	0	BCAX	FEAT_SVE2 || FEAT_SME
01	1	BSL1N	FEAT_SVE2 || FEAT_SME
1x	0	UNALLOCATED	-
10	1	BSL2N	FEAT_SVE2 || FEAT_SME
11	1	NBSL	FEAT_SVE2 || FEAT_SME

SVE Index Generation

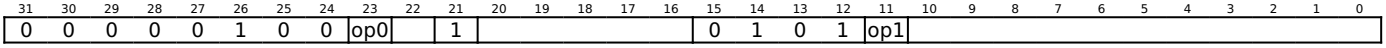
The encodings in this section are decoded from SVE encodings.



Decode fields op0	Instruction details	Condition
00	INDEX (immediates)	FEAT_SVE || FEAT_SME
01	INDEX (scalar, immediate)	FEAT_SVE || FEAT_SME
10	INDEX (immediate, scalar)	FEAT_SVE || FEAT_SME
11	INDEX (scalars)	FEAT_SVE || FEAT_SME

SVE Stack Allocation

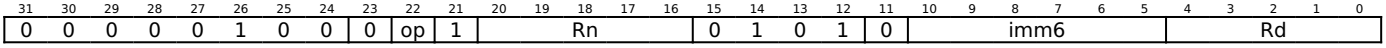
The encodings in this section are decoded from SVE encodings.



Decode fields op0	op1	Instruction details
0	0	SVE stack frame adjustment
0	1	Streaming SVE stack frame adjustment
1	0	SVE stack frame size
1	1	Streaming SVE stack frame size

SVE stack frame adjustment

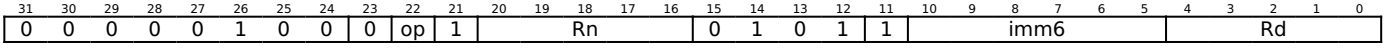
The encodings in this section are decoded from SVE Stack Allocation.



Decode fields op	Instruction Details	Condition
0	ADDVL	FEAT_SVE || FEAT_SME
1	ADDPL	FEAT_SVE || FEAT_SME

Streaming SVE stack frame adjustment

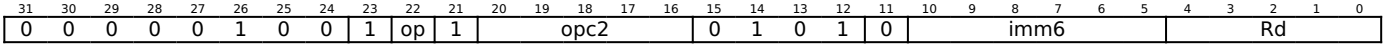
The encodings in this section are decoded from SVE Stack Allocation.



Decode fields op	Instruction Details	Condition
0	ADDSVL	FEAT_SME
1	ADDSPL	FEAT_SME

SVE stack frame size

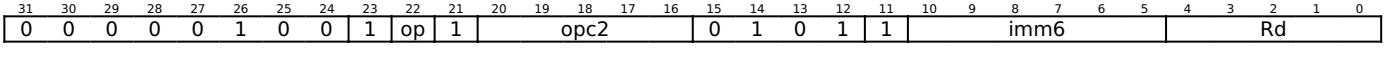
The encodings in this section are decoded from SVE Stack Allocation.



Decode fields op	Decode fields opc2	Instruction Details	Condition
0	11111	RDVL	FEAT_SVE || FEAT_SME
0	!= 11111	UNALLOCATED	-
1		UNALLOCATED	-

Streaming SVE stack frame size

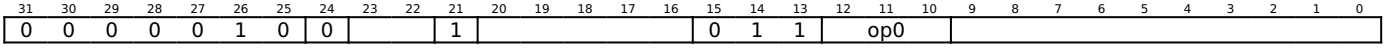
The encodings in this section are decoded from SVE Stack Allocation.



Decode fields op	Decode fields opc2	Instruction Details	Condition
0	11111	RDSVL	FEAT_SME
0	!= 11111	UNALLOCATED	-
1		UNALLOCATED	-

SVE2 Integer Arithmetic - Unpredicated

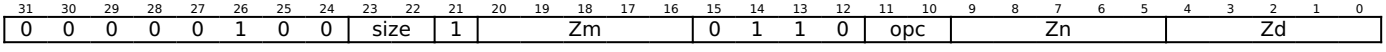
The encodings in this section are decoded from SVE encodings.



Decode fields op0	Instruction details	Condition
0xx	SVE2 integer multiply vectors (unpredicated)	-
10x	SVE2 signed saturating doubling multiply high (unpredicated)	-
110	ADDQP	FEAT_SVE2p3 || FEAT_SME2p3
111	ADDSUBP	FEAT_SVE2p3 || FEAT_SME2p3

SVE2 integer multiply vectors (unpredicated)

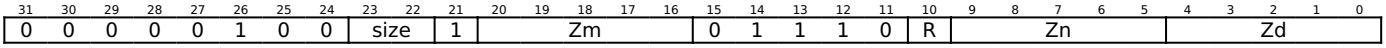
The encodings in this section are decoded from SVE2 Integer Arithmetic - Unpredicated.



Decode fields		Instruction Details	Condition
size	opc		
	00	MUL (vectors, unpredicated)	FEAT_SVE2 || FEAT_SME
	10	SMULH (unpredicated)	FEAT_SVE2 || FEAT_SME
	11	UMULH (unpredicated)	FEAT_SVE2 || FEAT_SME
00	01	PMUL	FEAT_SVE2 || FEAT_SME
!= 00	01	UNALLOCATED	-

SVE2 signed saturating doubling multiply high (unpredicated)

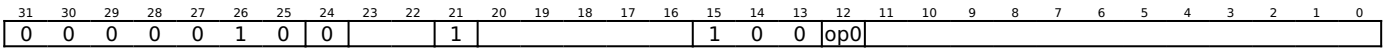
The encodings in this section are decoded from SVE2 Integer Arithmetic - Unpredicated.



Decode fields	Instruction Details	Condition
R		
0	SQDMULH (vectors)	FEAT_SVE2 || FEAT_SME
1	SQRDMULH (vectors)	FEAT_SVE2 || FEAT_SME

SVE Bitwise Shift - Unpredicated

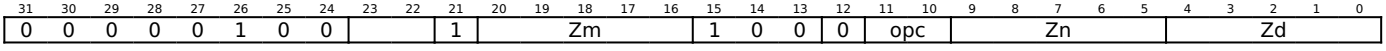
The encodings in this section are decoded from SVE encodings.



Decode fields	Instruction details
op0	
0	SVE bitwise shift by wide elements (unpredicated)
1	SVE bitwise shift by immediate (unpredicated)

SVE bitwise shift by wide elements (unpredicated)

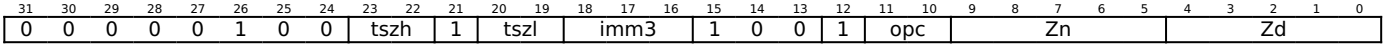
The encodings in this section are decoded from SVE Bitwise Shift - Unpredicated.



Decode fields	Instruction Details	Condition
opc		
00	ASR (wide elements, unpredicated)	FEAT_SVE || FEAT_SME
01	LSR (wide elements, unpredicated)	FEAT_SVE || FEAT_SME
10	UNALLOCATED	-
11	LSL (wide elements, unpredicated)	FEAT_SVE || FEAT_SME

SVE bitwise shift by immediate (unpredicated)

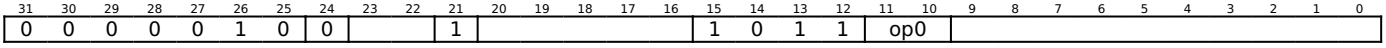
The encodings in this section are decoded from SVE Bitwise Shift - Unpredicated.



Decode fields opc	Instruction Details	Condition
00	ASR (immediate, unpredicated)	FEAT_SVE || FEAT_SME
01	LSR (immediate, unpredicated)	FEAT_SVE || FEAT_SME
10	UNALLOCATED	-
11	LSL (immediate, unpredicated)	FEAT_SVE || FEAT_SME

SVE Integer Misc - Unpredicated

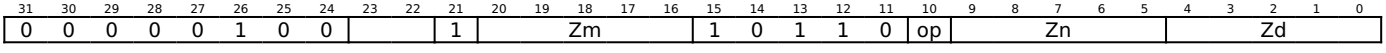
The encodings in this section are decoded from SVE encodings.



Decode fields op0	Instruction details
0x	SVE floating-point trig select coefficient
10	SVE floating-point exponential accelerator
11	SVE constructive prefix (unpredicated)

SVE floating-point trig select coefficient

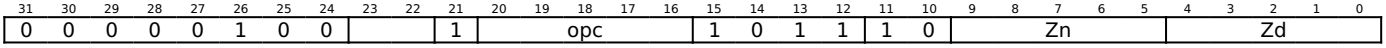
The encodings in this section are decoded from SVE Integer Misc - Unpredicated.



Decode fields op	Instruction Details	Condition
0	FTSSEL	FEAT_SVE
1	UNALLOCATED	-

SVE floating-point exponential accelerator

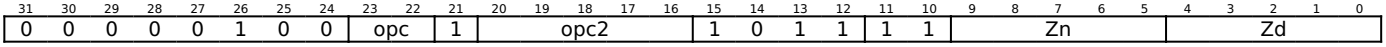
The encodings in this section are decoded from SVE Integer Misc - Unpredicated.



Decode fields opc	Instruction Details	Condition
00000	FEXPA	FEAT_SVE || FEAT_SSVE_FEXPA
!= 00000	UNALLOCATED	-

SVE constructive prefix (unpredicated)

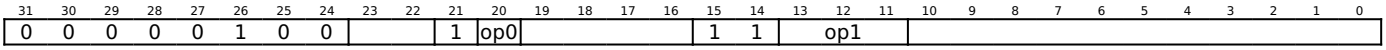
The encodings in this section are decoded from SVE Integer Misc - Unpredicated.



Decode fields opc	Decode fields opc2	Instruction Details	Condition
00	00000	MOVPRFX (unpredicated)	FEAT_SVE || FEAT_SME
00	!= 00000	UNALLOCATED	-
!= 00		UNALLOCATED	-

SVE Element Count

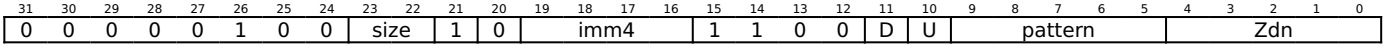
The encodings in this section are decoded from SVE encodings.



Decode fields op0	Decode fields op1	Instruction details
0	00x	SVE saturating inc/dec vector by element count
0	100	SVE element count
0	101	UNALLOCATED
1	000	SVE inc/dec vector by element count
1	100	SVE inc/dec register by element count
1	x01	UNALLOCATED
	01x	UNALLOCATED
	11x	SVE saturating inc/dec register by element count

SVE saturating inc/dec vector by element count

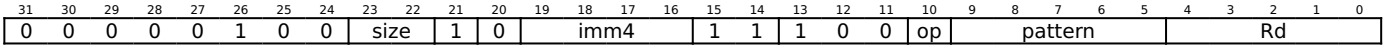
The encodings in this section are decoded from SVE Element Count.



Decode fields size	Decode fields D	Decode fields U	Instruction Details	Condition
00			UNALLOCATED	-
01	0	0	SQINCH (vector)	FEAT_SVE || FEAT_SME
01	0	1	UQINCH (vector)	FEAT_SVE || FEAT_SME
01	1	0	SQDECH (vector)	FEAT_SVE || FEAT_SME
01	1	1	UQDECH (vector)	FEAT_SVE || FEAT_SME
10	0	0	SQINCW (vector)	FEAT_SVE || FEAT_SME
10	0	1	UQINCW (vector)	FEAT_SVE || FEAT_SME
10	1	0	SQDECW (vector)	FEAT_SVE || FEAT_SME
10	1	1	UQDECW (vector)	FEAT_SVE || FEAT_SME
11	0	0	SQINCD (vector)	FEAT_SVE || FEAT_SME
11	0	1	UQINCD (vector)	FEAT_SVE || FEAT_SME
11	1	0	SQDECD (vector)	FEAT_SVE || FEAT_SME
11	1	1	UQDECD (vector)	FEAT_SVE || FEAT_SME

SVE element count

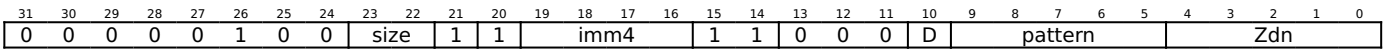
The encodings in this section are decoded from SVE Element Count.



Decode fields size	Decode fields op	Instruction Details	Condition
	1	UNALLOCATED	-
00	0	CNTB, CNTD, CNTH, CNTW — Byte	FEAT_SVE || FEAT_SME
01	0	CNTB, CNTD, CNTH, CNTW — Halfword	FEAT_SVE || FEAT_SME
10	0	CNTB, CNTD, CNTH, CNTW — Word	FEAT_SVE || FEAT_SME
11	0	CNTB, CNTD, CNTH, CNTW — Doubleword	FEAT_SVE || FEAT_SME

SVE inc/dec vector by element count

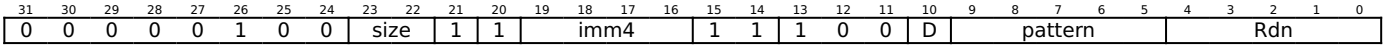
The encodings in this section are decoded from SVE Element Count.



Decode fields		Instruction Details	Condition
size	D		
00		UNALLOCATED	-
01	0	INCD, INCH, INCW (vector) — Halfword	FEAT_SVE || FEAT_SME
01	1	DECD, DECH, DECW (vector) — Halfword	FEAT_SVE || FEAT_SME
10	0	INCD, INCH, INCW (vector) — Word	FEAT_SVE || FEAT_SME
10	1	DECD, DECH, DECW (vector) — Word	FEAT_SVE || FEAT_SME
11	0	INCD, INCH, INCW (vector) — Doubleword	FEAT_SVE || FEAT_SME
11	1	DECD, DECH, DECW (vector) — Doubleword	FEAT_SVE || FEAT_SME

SVE inc/dec register by element count

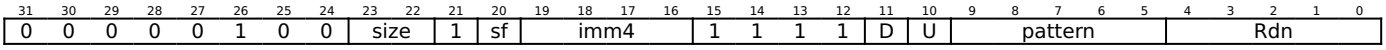
The encodings in this section are decoded from SVE Element Count.



Decode fields		Instruction Details	Condition
size	D		
00	0	INCB, INCD, INCH, INCW (scalar) — Byte	FEAT_SVE || FEAT_SME
00	1	DECB, DECD, DECH, DECW (scalar) — Byte	FEAT_SVE || FEAT_SME
01	0	INCB, INCD, INCH, INCW (scalar) — Halfword	FEAT_SVE || FEAT_SME
01	1	DECB, DECD, DECH, DECW (scalar) — Halfword	FEAT_SVE || FEAT_SME
10	0	INCB, INCD, INCH, INCW (scalar) — Word	FEAT_SVE || FEAT_SME
10	1	DECB, DECD, DECH, DECW (scalar) — Word	FEAT_SVE || FEAT_SME
11	0	INCB, INCD, INCH, INCW (scalar) — Doubleword	FEAT_SVE || FEAT_SME
11	1	DECB, DECD, DECH, DECW (scalar) — Doubleword	FEAT_SVE || FEAT_SME

SVE saturating inc/dec register by element count

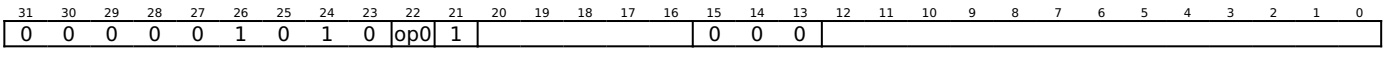
The encodings in this section are decoded from SVE Element Count.



Decode fields				Instruction Details	Condition
size	sf	D	U		
00	0	0	0	SQINCB — 32-bit	FEAT_SVE || FEAT_SME
00	0	0	1	UQINCB — 32-bit	FEAT_SVE || FEAT_SME
00	0	1	0	SQDECB — 32-bit	FEAT_SVE || FEAT_SME
00	0	1	1	UQDECB — 32-bit	FEAT_SVE || FEAT_SME
00	1	0	0	SQINCB — 64-bit	FEAT_SVE || FEAT_SME
00	1	0	1	UQINCB — 64-bit	FEAT_SVE || FEAT_SME
00	1	1	0	SQDECB — 64-bit	FEAT_SVE || FEAT_SME
00	1	1	1	UQDECB — 64-bit	FEAT_SVE || FEAT_SME
01	0	0	0	SQINCH (scalar) — 32-bit	FEAT_SVE || FEAT_SME
01	0	0	1	UQINCH (scalar) — 32-bit	FEAT_SVE || FEAT_SME
01	0	1	0	SQDECH (scalar) — 32-bit	FEAT_SVE || FEAT_SME
01	0	1	1	UQDECH (scalar) — 32-bit	FEAT_SVE || FEAT_SME
01	1	0	0	SQINCH (scalar) — 64-bit	FEAT_SVE || FEAT_SME
01	1	0	1	UQINCH (scalar) — 64-bit	FEAT_SVE || FEAT_SME
01	1	1	0	SQDECH (scalar) — 64-bit	FEAT_SVE || FEAT_SME
01	1	1	1	UQDECH (scalar) — 64-bit	FEAT_SVE || FEAT_SME
10	0	0	0	SQINCW (scalar) — 32-bit	FEAT_SVE || FEAT_SME
10	0	0	1	UQINCW (scalar) — 32-bit	FEAT_SVE || FEAT_SME
10	0	1	0	SQDECW (scalar) — 32-bit	FEAT_SVE || FEAT_SME
10	0	1	1	UQDECW (scalar) — 32-bit	FEAT_SVE || FEAT_SME
10	1	0	0	SQINCW (scalar) — 64-bit	FEAT_SVE || FEAT_SME
10	1	0	1	UQINCW (scalar) — 64-bit	FEAT_SVE || FEAT_SME
10	1	1	0	SQDECW (scalar) — 64-bit	FEAT_SVE || FEAT_SME
10	1	1	1	UQDECW (scalar) — 64-bit	FEAT_SVE || FEAT_SME
11	0	0	0	SQINCD (scalar) — 32-bit	FEAT_SVE || FEAT_SME
11	0	0	1	UQINCD (scalar) — 32-bit	FEAT_SVE || FEAT_SME
11	0	1	0	SQDECD (scalar) — 32-bit	FEAT_SVE || FEAT_SME
11	0	1	1	UQDECD (scalar) — 32-bit	FEAT_SVE || FEAT_SME
11	1	0	0	SQINCD (scalar) — 64-bit	FEAT_SVE || FEAT_SME
11	1	0	1	UQINCD (scalar) — 64-bit	FEAT_SVE || FEAT_SME
11	1	1	0	SQDECD (scalar) — 64-bit	FEAT_SVE || FEAT_SME
11	1	1	1	UQDECD (scalar) — 64-bit	FEAT_SVE || FEAT_SME

SVE Permute Vector - Extract

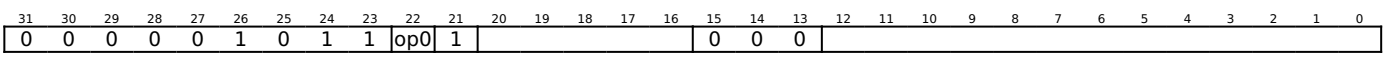
The encodings in this section are decoded from SVE encodings.



Decode fields	Instruction details	Condition
op0		
0	EXT — Destructive	FEAT_SVE || FEAT_SME
1	EXT — Constructive	FEAT_SVE2 || FEAT_SME

SVE Permute Vector - Segments

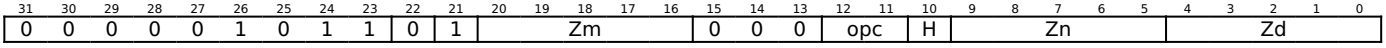
The encodings in this section are decoded from SVE encodings.



Decode fields op0	Instruction details
0	SVE permute vector segments
1	UNALLOCATED

SVE permute vector segments

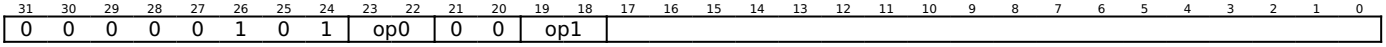
The encodings in this section are decoded from SVE Permute Vector - Segments.



Decode fields opc	H	Instruction Details	Condition
00	0	ZIP1, ZIP2 (vectors) — Low halves (quadwords)	FEAT_F64MM || FEAT_SVE_PERM256
00	1	ZIP1, ZIP2 (vectors) — High halves (quadwords)	FEAT_F64MM || FEAT_SVE_PERM256
01	0	UZP1, UZP2 (vectors) — Even (quadwords)	FEAT_F64MM || FEAT_SVE_PERM256
01	1	UZP1, UZP2 (vectors) — Odd (quadwords)	FEAT_F64MM || FEAT_SVE_PERM256
10		UNALLOCATED	-
11	0	TRN1, TRN2 (vectors) — Even (quadwords)	FEAT_F64MM || FEAT_SVE_PERM256
11	1	TRN1, TRN2 (vectors) — Odd (quadwords)	FEAT_F64MM || FEAT_SVE_PERM256

SVE Bitwise Immediate

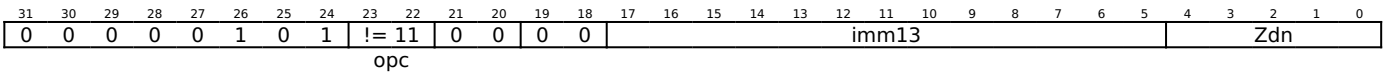
The encodings in this section are decoded from SVE encodings.



Decode fields op0	op1	Instruction details	Condition
11	00	DUPM	FEAT_SVE || FEAT_SME
	!= 00	UNALLOCATED	-
!= 11	00	SVE bitwise logical with immediate (unpredicated)	-

SVE bitwise logical with immediate (unpredicated)

The encodings in this section are decoded from SVE Bitwise Immediate.

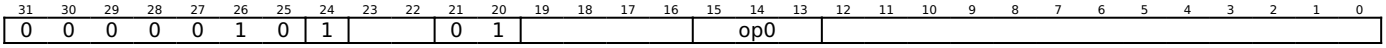


The following constraints also apply to this encoding: opc != '11'

Decode fields opc	Instruction Details	Condition
00	ORR (immediate)	FEAT_SVE || FEAT_SME
01	EOR (immediate)	FEAT_SVE || FEAT_SME
10	AND (immediate)	FEAT_SVE || FEAT_SME

SVE Integer Wide Immediate - Predicated

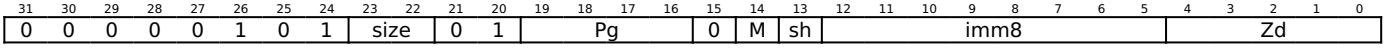
The encodings in this section are decoded from SVE encodings.



Decode fields op0	Instruction details	Condition
0xx	SVE copy integer immediate (predicated)	-
10x	UNALLOCATED	-
110	FCPY	FEAT_SVE || FEAT_SME
111	UNALLOCATED	-

SVE copy integer immediate (predicated)

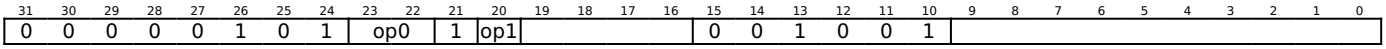
The encodings in this section are decoded from SVE Integer Wide Immediate - Predicated.



Decode fields M	Instruction Details	Condition
0	CPY (immediate, zeroing)	FEAT_SVE || FEAT_SME
1	CPY (immediate, merging)	FEAT_SVE || FEAT_SME

SVE Permute Vector - One Source Quadwords

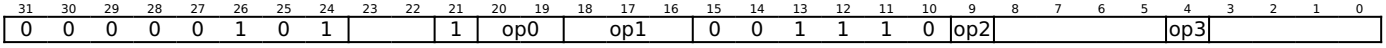
The encodings in this section are decoded from SVE encodings.



Decode fields op0	op1	Instruction details	Condition
00		DUPQ	FEAT_SVE2p1 || FEAT_SME2p1
01	0	EXTQ	FEAT_SVE2p1 || FEAT_SME2p1
01	1	UNALLOCATED	-
1x		UNALLOCATED	-

SVE Permute Vector - Unpredicated

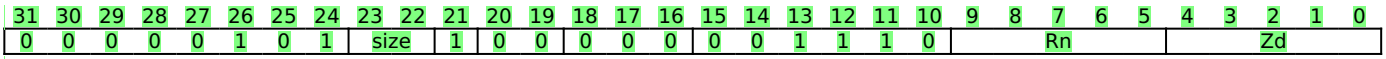
The encodings in this section are decoded from SVE encodings.



Decode fields op0	op1	op2	op3	Instruction details	Condition
00	000			SVE broadcast general register~~DUP (scalar)~~	~~FEAT_SVE || FEAT_SME~~
00	100			INSR (scalar)	FEAT_SVE || FEAT_SME
00	!= x00			UNALLOCATED	-
01	xx0		0	SVE move predicate from vector	-
01	xx0		1	UNALLOCATED	-
01	xx1	0		SVE move predicate into vector	-
01	xx1	1		UNALLOCATED	-
10	0xx			SVE unpack vector elements	-
10	100			INSR (SIMD&FP scalar)	FEAT_SVE || FEAT_SME
10	101			UNALLOCATED	-
10	11x			UNALLOCATED	-
11	000			REV (vector)	FEAT_SVE || FEAT_SME
11	!= 000			UNALLOCATED	-

SVE broadcast general register

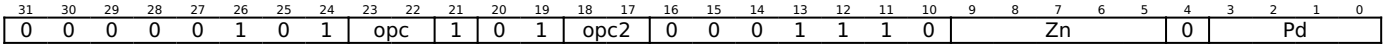
The encodings in this section are decoded from SVE Permute Vector - Unpredicated.



Decode fields size	Instruction Details	Condition
11	DUP (scalar) — 64-bit	FEAT_SVE || FEAT_SME
!= 11	DUP (scalar) — 32-bit	FEAT_SVE || FEAT_SME

SVE move predicate from vector

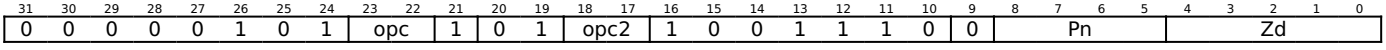
The encodings in this section are decoded from SVE Permute Vector - Unpredicated.



Decode fields opc	Decode fields opc2	Instruction Details	Condition
00	00	UNALLOCATED	-
00	01	PMOV (to predicate) — Byte	FEAT_SVE2p1 || FEAT_SME2p1
00	1x	PMOV (to predicate) — Halfword	FEAT_SVE2p1 || FEAT_SME2p1
01		PMOV (to predicate) — Word	FEAT_SVE2p1 || FEAT_SME2p1
1x		PMOV (to predicate) — Doubleword	FEAT_SVE2p1 || FEAT_SME2p1

SVE move predicate into vector

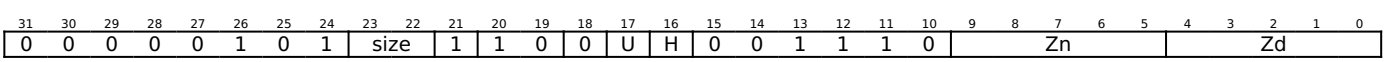
The encodings in this section are decoded from SVE Permute Vector - Unpredicated.



Decode fields opc	Decode fields opc2	Instruction Details	Condition
00	00	UNALLOCATED	-
00	01	PMOV (to vector) — Byte	FEAT_SVE2p1 || FEAT_SME2p1
00	1x	PMOV (to vector) — Halfword	FEAT_SVE2p1 || FEAT_SME2p1
01		PMOV (to vector) — Word	FEAT_SVE2p1 || FEAT_SME2p1
1x		PMOV (to vector) — Doubleword	FEAT_SVE2p1 || FEAT_SME2p1

SVE unpack vector elements

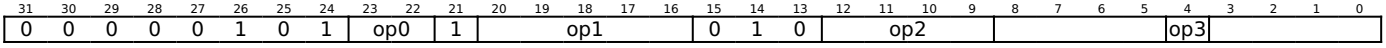
The encodings in this section are decoded from SVE Permute Vector - Unpredicated.



Decode fields U	Decode fields H	Instruction Details	Condition
0	0	SUNPKHI, SUNPKLO — Low half	FEAT_SVE || FEAT_SME
0	1	SUNPKHI, SUNPKLO — High half	FEAT_SVE || FEAT_SME
1	0	UUNPKHI, UUNPKLO — Low half	FEAT_SVE || FEAT_SME
1	1	UUNPKHI, UUNPKLO — High half	FEAT_SVE || FEAT_SME

SVE Permute Predicate

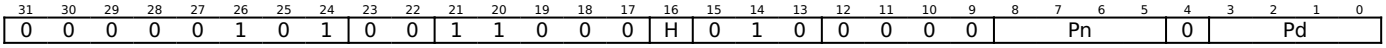
The encodings in this section are decoded from SVE encodings.



Decode fields				Instruction details	Condition
op0	op1	op2	op3		
00	1000x	0000	0	SVE unpack predicate elements	-
	0xxxx	xxx0	0	SVE permute predicate elements	-
	10100	0000	0	REV (predicate)	FEAT_SVE || FEAT_SME
	10101	0000	0	UNALLOCATED	-
	10x0x	0010	0	UNALLOCATED	-
	10x0x	01x0	0	UNALLOCATED	-
	10x0x	1xx0	0	UNALLOCATED	-
	10x1x	xxx0	0	UNALLOCATED	-
	11xxx	xxx0	0	UNALLOCATED	-
		xxx0	1	UNALLOCATED	-
		xxx1		UNALLOCATED	-
!= 00	1000x	0000	0	UNALLOCATED	-

SVE unpack predicate elements

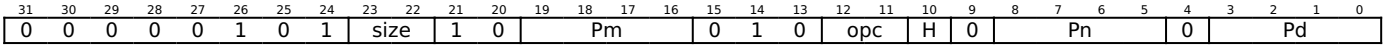
The encodings in this section are decoded from SVE Permute Predicate.



Decode fields	Instruction Details	Condition
H		
0	PUNPKHI, PUNPKLO — Low half	FEAT_SVE || FEAT_SME
1	PUNPKHI, PUNPKLO — High half	FEAT_SVE || FEAT_SME

SVE permute predicate elements

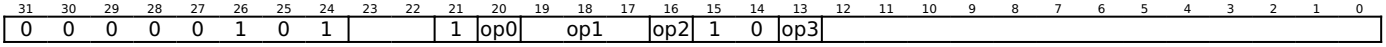
The encodings in this section are decoded from SVE Permute Predicate.



Decode fields		Instruction Details	Condition
opc	H		
00	0	ZIP1, ZIP2 (predicates) — Low halves	FEAT_SVE || FEAT_SME
00	1	ZIP1, ZIP2 (predicates) — High halves	FEAT_SVE || FEAT_SME
01	0	UZP1, UZP2 (predicates) — Even	FEAT_SVE || FEAT_SME
01	1	UZP1, UZP2 (predicates) — Odd	FEAT_SVE || FEAT_SME
10	0	TRN1, TRN2 (predicates) — Even	FEAT_SVE || FEAT_SME
10	1	TRN1, TRN2 (predicates) — Odd	FEAT_SVE || FEAT_SME
11		UNALLOCATED	-

SVE Permute Vector - Predicated

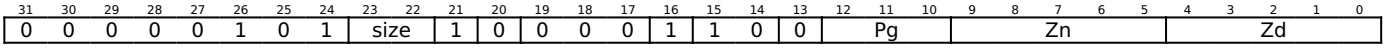
The encodings in this section are decoded from SVE encodings.



Decode fields				Instruction details	Condition
op0	op1	op2	op3		
0	000	0	0	CPY (SIMD&FP scalar)	FEAT_SVE || FEAT_SME
0	000	1	0	SVE compress Active elements	-
0	000		1	SVE extract element to general register	-
0	001	1	1	UNALLOCATED	-
0	001		0	SVE extract element to SIMD&FP scalar register	-
0	01x			SVE reverse within elements	-
0	100	0	1	SVE copy general register to vector (predicated)~~CPY (scalar)~~	~~FEAT_SVE || FEAT_SME~~
0	100		0	SVE conditionally broadcast element to vector	-
0	101		0	SVE conditionally extract element to SIMD&FP scalar	-
0	110	0	0	SPLICE — Destructive	FEAT_SVE || FEAT_SME
0	110	0	1	UNALLOCATED	-
0	110	1	0	SPLICE — Constructive	FEAT_SVE2 || FEAT_SME
0	111	0		SVE reverse doublewords	-
0	111	1	0	UNALLOCATED	-
0	1xx	1	1	UNALLOCATED	-
0	x01	0	1	UNALLOCATED	-
1	000	0	0	UNALLOCATED	-
1	000	1	0	EXPAND	FEAT_SVE2p2 || FEAT_SME2p2
1	000		1	SVE conditionally extract element to general register	-
1	!= 000			UNALLOCATED	-

SVE compress Active elements

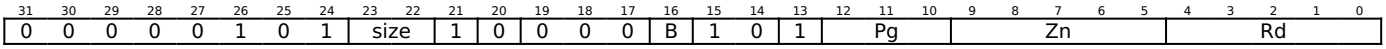
The encodings in this section are decoded from SVE Permute Vector - Predicated.



Decode fields	Instruction Details	Condition
size		
0x	COMPACT — Byte and halfword	FEAT_SVE2p2 || FEAT_SME2p2
1x	COMPACT — Word and doubleword	FEAT_SVE || FEAT_SME2p2

SVE extract element to general register

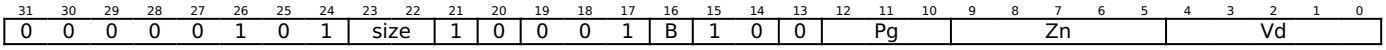
The encodings in this section are decoded from SVE Permute Vector - Predicated.



Decode fields	Instruction Details	Condition
B		
0	LASTA (scalar)	FEAT_SVE || FEAT_SME
1	LASTB (scalar)	FEAT_SVE || FEAT_SME

SVE extract element to SIMD&FP scalar register

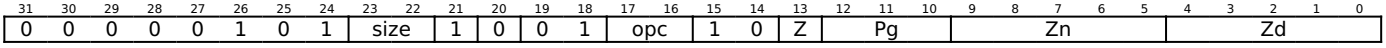
The encodings in this section are decoded from SVE Permute Vector - Predicated.



Decode fields B	Instruction Details	Condition
0	LASTA (SIMD&FP scalar)	FEAT_SVE || FEAT_SME
1	LASTB (SIMD&FP scalar)	FEAT_SVE || FEAT_SME

SVE reverse within elements

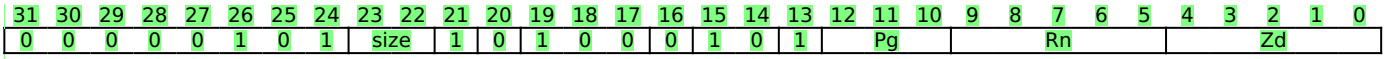
The encodings in this section are decoded from SVE Permute Vector - Predicated.



Decode fields opc	Z	Instruction Details	Condition
00	0	REVB, REVH, REVW — Byte, merging	FEAT_SVE || FEAT_SME
00	1	REVB, REVH, REVW — Byte, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
01	0	REVB, REVH, REVW — Halfword, merging	FEAT_SVE || FEAT_SME
01	1	REVB, REVH, REVW — Halfword, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
10	0	REVB, REVH, REVW — Word, merging	FEAT_SVE || FEAT_SME
10	1	REVB, REVH, REVW — Word, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
11	0	RBIT — Merging	FEAT_SVE || FEAT_SME
11	1	RBIT — Zeroing	FEAT_SVE2p2 || FEAT_SME2p2

SVE copy general register to vector (predicated)

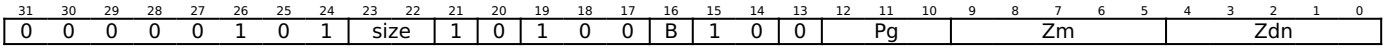
The encodings in this section are decoded from SVE Permute Vector - Predicated.



Decode fields size	Instruction Details	Condition
11	CPY (scalar) — 64-bit	FEAT_SVE || FEAT_SME
!= 11	CPY (scalar) — 32-bit	FEAT_SVE || FEAT_SME

SVE conditionally broadcast element to vector

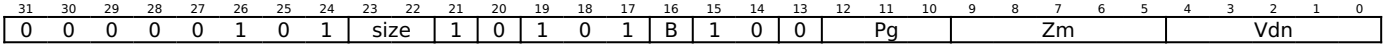
The encodings in this section are decoded from SVE Permute Vector - Predicated.



Decode fields B	Instruction Details	Condition
0	CLASTA (vectors)	FEAT_SVE || FEAT_SME
1	CLASTB (vectors)	FEAT_SVE || FEAT_SME

SVE conditionally extract element to SIMD&FP scalar

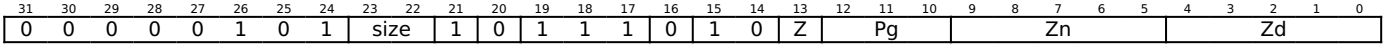
The encodings in this section are decoded from SVE Permute Vector - Predicated.



Decode fields B	Instruction Details	Condition
0	CLASTA (SIMD&FP scalar)	FEAT_SVE || FEAT_SME
1	CLASTB (SIMD&FP scalar)	FEAT_SVE || FEAT_SME

SVE reverse doublewords

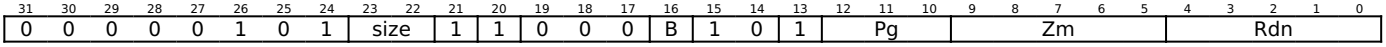
The encodings in this section are decoded from SVE Permute Vector - Predicated.



Decode fields size	Z	Instruction Details	Condition
00	0	REVD — Merging	FEAT_SME || FEAT_SVE2p1
00	1	REVD — Zeroing	FEAT_SVE2p2 || FEAT_SME2p2
!= 00		UNALLOCATED	-

SVE conditionally extract element to general register

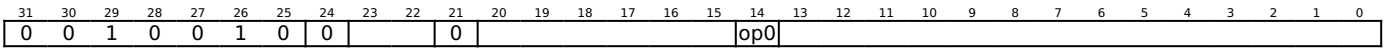
The encodings in this section are decoded from SVE Permute Vector - Predicated.



Decode fields B	Instruction Details	Condition
0	CLASTA (scalar)	FEAT_SVE || FEAT_SME
1	CLASTB (scalar)	FEAT_SVE || FEAT_SME

SVE Integer Compare - Vectors

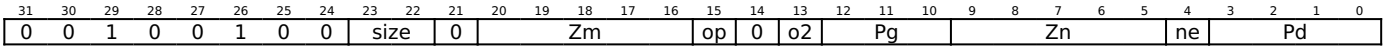
The encodings in this section are decoded from SVE encodings.



Decode fields op0	Instruction details
0	SVE integer compare vectors
1	SVE integer compare with wide elements

SVE integer compare vectors

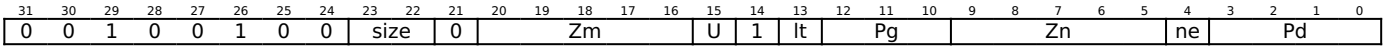
The encodings in this section are decoded from SVE Integer Compare - Vectors.



Decode fields op	o2	ne	Instruction Details	Condition
0	0	0	CMP<cc> (vectors) — Higher or same	FEAT_SVE || FEAT_SME
0	0	1	CMP<cc> (vectors) — Higher	FEAT_SVE || FEAT_SME
0	1	0	CMP<cc> (wide elements) — Equal	FEAT_SVE || FEAT_SME
0	1	1	CMP<cc> (wide elements) — Not equal	FEAT_SVE || FEAT_SME
1	0	0	CMP<cc> (vectors) — Greater than or equal	FEAT_SVE || FEAT_SME
1	0	1	CMP<cc> (vectors) — Greater than	FEAT_SVE || FEAT_SME
1	1	0	CMP<cc> (vectors) — Equal	FEAT_SVE || FEAT_SME
1	1	1	CMP<cc> (vectors) — Not equal	FEAT_SVE || FEAT_SME

SVE integer compare with wide elements

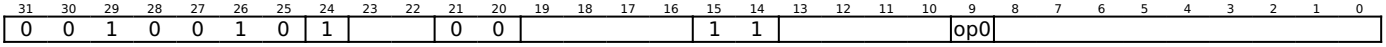
The encodings in this section are decoded from SVE Integer Compare - Vectors.



Decode fields			Instruction Details	Condition
U	lt	ne		
0	0	0	CMP<cc> (wide elements) — Greater than or equal	FEAT_SVE || FEAT_SME
0	0	1	CMP<cc> (wide elements) — Greater than	FEAT_SVE || FEAT_SME
0	1	0	CMP<cc> (wide elements) — Less than	FEAT_SVE || FEAT_SME
0	1	1	CMP<cc> (wide elements) — Less than or equal	FEAT_SVE || FEAT_SME
1	0	0	CMP<cc> (wide elements) — Higher or same	FEAT_SVE || FEAT_SME
1	0	1	CMP<cc> (wide elements) — Higher	FEAT_SVE || FEAT_SME
1	1	0	CMP<cc> (wide elements) — Lower	FEAT_SVE || FEAT_SME
1	1	1	CMP<cc> (wide elements) — Lower or same	FEAT_SVE || FEAT_SME

SVE Propagate Break

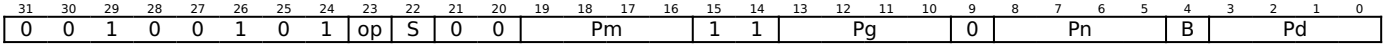
The encodings in this section are decoded from SVE encodings.



Decode fields	Instruction details
op0	
0	SVE propagate break from previous partition
1	UNALLOCATED

SVE propagate break from previous partition

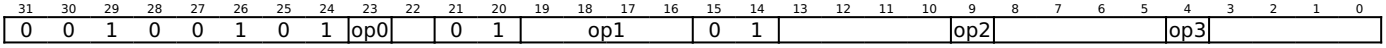
The encodings in this section are decoded from SVE Propagate Break.



Decode fields			Instruction Details	Condition
op	S	B		
0	0	0	BRKPA	FEAT_SVE || FEAT_SME
0	0	1	BRKPB	FEAT_SVE || FEAT_SME
0	1	0	BRKPAS	FEAT_SVE || FEAT_SME
0	1	1	BRKPBS	FEAT_SVE || FEAT_SME
1			UNALLOCATED	-

SVE Partition Break

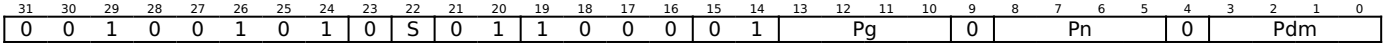
The encodings in this section are decoded from SVE encodings.



Decode fields				Instruction details
op0	op1	op2	op3	
0	1000	0	0	SVE propagate break to next partition
0	1000	0	1	UNALLOCATED
1	1000	0		UNALLOCATED
	0000	0		SVE partition break condition
	x000	1		UNALLOCATED
	!= x000			UNALLOCATED

SVE propagate break to next partition

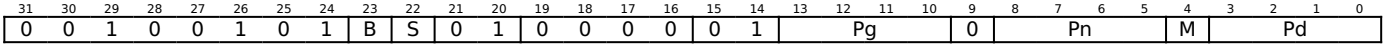
The encodings in this section are decoded from SVE Partition Break.



Decode fields S	Instruction Details	Condition
0	BRKN	FEAT_SVE || FEAT_SME
1	BRKNS	FEAT_SVE || FEAT_SME

SVE partition break condition

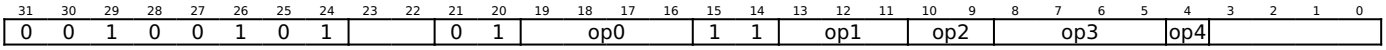
The encodings in this section are decoded from SVE Partition Break.



Decode fields B	Decode fields S	Decode fields M	Instruction Details	Condition
	1	1	UNALLOCATED	-
0	0		BRKA	FEAT_SVE || FEAT_SME
0	1	0	BRKAS	FEAT_SVE || FEAT_SME
1	0		BRKB	FEAT_SVE || FEAT_SME
1	1	0	BRKBS	FEAT_SVE || FEAT_SME

SVE Predicate Misc

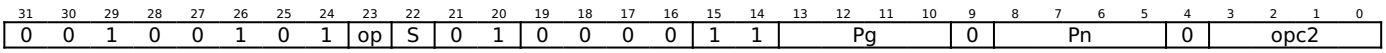
The encodings in this section are decoded from SVE encodings.



Decode fields op0	Decode fields op1	Decode fields op2	Decode fields op3	Decode fields op4	Instruction details	Condition
0000		x0		0	SVE predicate test	-
0000		x1		0	UNALLOCATED	-
0001				0	UNALLOCATED	-
1000	000	00		0	SVE predicate first active	-
1000	000	10		0	UNALLOCATED	-
1000	100	10	0000	0	SVE predicate zero	-
1000	100	10	!= 0000	0	UNALLOCATED	-
1000	110	00		0	SVE predicate read from FFR (predicated)	-
1001	000	00		0	UNALLOCATED	-
1001	000	10		0	PNEXT	FEAT_SVE || FEAT_SME
1001	100	10		0	UNALLOCATED	-
1001	110	00	0000	0	SVE predicate read from FFR (unpredicated)	-
1001	110	00	!= 0000	0	UNALLOCATED	-
100x	010	x0		0	UNALLOCATED	-
100x	0x0	x1		0	UNALLOCATED	-
100x	100	0x		0	SVE predicate initialize	-
100x	100	11		0	UNALLOCATED	-
100x	110	!= 00		0	UNALLOCATED	-
100x	xx1			0	UNALLOCATED	-
x00x				1	UNALLOCATED	-
!= x00x					UNALLOCATED	-

SVE predicate test

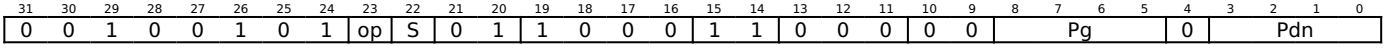
The encodings in this section are decoded from SVE Predicate Misc.



Decode fields			Instruction Details	Condition
op	S	opc2		
0	0		UNALLOCATED	-
0	1	0000	PTEST	FEAT_SVE || FEAT_SME
0	1	!= 0000	UNALLOCATED	-
1			UNALLOCATED	-

SVE predicate first active

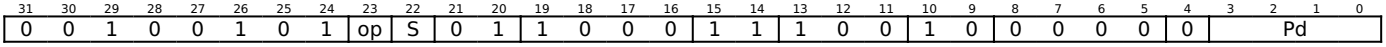
The encodings in this section are decoded from SVE Predicate Misc.



Decode fields		Instruction Details	Condition
op	S		
0	0	UNALLOCATED	-
0	1	PFIRST	FEAT_SVE || FEAT_SME
1		UNALLOCATED	-

SVE predicate zero

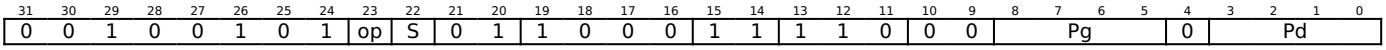
The encodings in this section are decoded from SVE Predicate Misc.



Decode fields		Instruction Details	Condition
op	S		
0	0	PFALSE	FEAT_SVE || FEAT_SME
0	1	UNALLOCATED	-
1		UNALLOCATED	-

SVE predicate read from FFR (predicated)

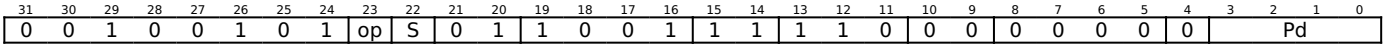
The encodings in this section are decoded from SVE Predicate Misc.



Decode fields		Instruction Details	Condition
op	S		
0	0	RDFFR (predicated)	FEAT_SVE
0	1	RDFFRS	FEAT_SVE
1		UNALLOCATED	-

SVE predicate read from FFR (unpredicated)

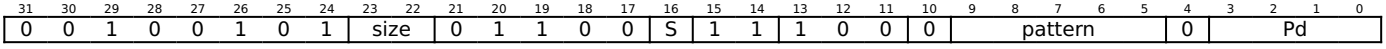
The encodings in this section are decoded from SVE Predicate Misc.



Decode fields		Instruction Details	Condition
op	S		
0	0	RDFFR (unpredicated)	FEAT_SVE
0	1	UNALLOCATED	-
1		UNALLOCATED	-

SVE predicate initialize

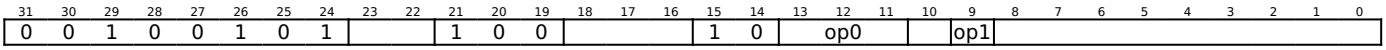
The encodings in this section are decoded from SVE Predicate Misc.



Decode fields	Instruction Details	Condition
S		
0	PTRUE (predicate)	FEAT_SVE || FEAT_SME
1	PTRUES	FEAT_SVE || FEAT_SME

SVE Predicate Count

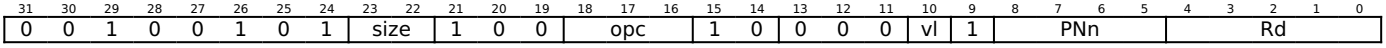
The encodings in this section are decoded from SVE encodings.



Decode fields		Instruction details
op0	op1	
000	1	SVE predicate count (predicate-as-counter)
	0	SVE predicate count
!= 000	1	UNALLOCATED

SVE predicate count (predicate-as-counter)

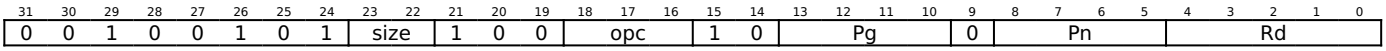
The encodings in this section are decoded from SVE Predicate Count.



Decode fields	Instruction Details	Condition
opc		
000	CNTP (predicate as counter)	FEAT_SME2 || FEAT_SVE2p1
!= 000	UNALLOCATED	-

SVE predicate count

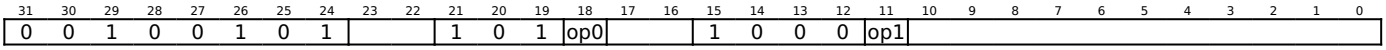
The encodings in this section are decoded from SVE Predicate Count.



Decode fields	Instruction Details	Condition
opc		
000	CNTP (predicate)	FEAT_SVE || FEAT_SME
001	FIRSTP	FEAT_SVE2p2 || FEAT_SME2p2
010	LASTP	FEAT_SVE2p2 || FEAT_SME2p2
011	UNALLOCATED	-
1xx	UNALLOCATED	-

SVE Inc/Dec by Predicate Count

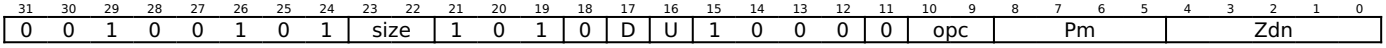
The encodings in this section are decoded from SVE encodings.



Decode fields op0	Decode fields op1	Instruction details
0	0	SVE saturating inc/dec vector by predicate count
0	1	SVE saturating inc/dec register by predicate count
1	0	SVE inc/dec vector by predicate count
1	1	SVE inc/dec register by predicate count

SVE saturating inc/dec vector by predicate count

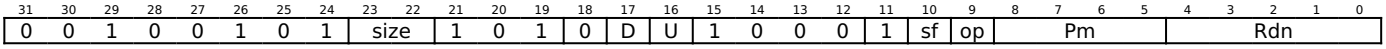
The encodings in this section are decoded from SVE Inc/Dec by Predicate Count.



Decode fields D	Decode fields U	Decode fields opc	Instruction Details	Condition
		!= 00	UNALLOCATED	-
0	0	00	SQINCP (vector)	FEAT_SVE || FEAT_SME
0	1	00	UQINCP (vector)	FEAT_SVE || FEAT_SME
1	0	00	SQDECP (vector)	FEAT_SVE || FEAT_SME
1	1	00	UQDECP (vector)	FEAT_SVE || FEAT_SME

SVE saturating inc/dec register by predicate count

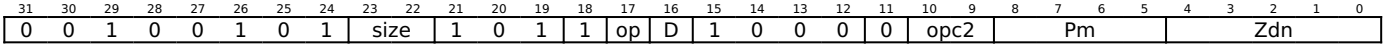
The encodings in this section are decoded from SVE Inc/Dec by Predicate Count.



Decode fields D	Decode fields U	Decode fields sf	Decode fields op	Instruction Details	Condition
			1	UNALLOCATED	-
0	0	0	0	SQINCP (scalar) — 32-bit	FEAT_SVE || FEAT_SME
0	0	1	0	SQINCP (scalar) — 64-bit	FEAT_SVE || FEAT_SME
0	1	0	0	UQINCP (scalar) — 32-bit	FEAT_SVE || FEAT_SME
0	1	1	0	UQINCP (scalar) — 64-bit	FEAT_SVE || FEAT_SME
1	0	0	0	SQDECP (scalar) — 32-bit	FEAT_SVE || FEAT_SME
1	0	1	0	SQDECP (scalar) — 64-bit	FEAT_SVE || FEAT_SME
1	1	0	0	UQDECP (scalar) — 32-bit	FEAT_SVE || FEAT_SME
1	1	1	0	UQDECP (scalar) — 64-bit	FEAT_SVE || FEAT_SME

SVE inc/dec vector by predicate count

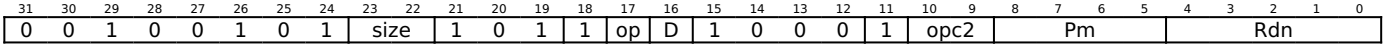
The encodings in this section are decoded from SVE Inc/Dec by Predicate Count.



Decode fields			Instruction Details	Condition
op	D	opc2		
0		!= 00	UNALLOCATED	-
0	0	00	INCP (vector)	FEAT_SVE || FEAT_SME
0	1	00	DECP (vector)	FEAT_SVE || FEAT_SME
1			UNALLOCATED	-

SVE inc/dec register by predicate count

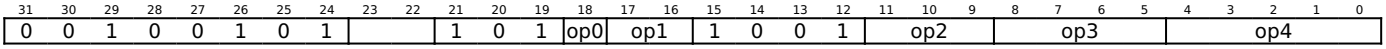
The encodings in this section are decoded from SVE Inc/Dec by Predicate Count.



Decode fields			Instruction Details	Condition
op	D	opc2		
0		!= 00	UNALLOCATED	-
0	0	00	INCP (scalar)	FEAT_SVE || FEAT_SME
0	1	00	DECP (scalar)	FEAT_SVE || FEAT_SME
1			UNALLOCATED	-

SVE Write FFR

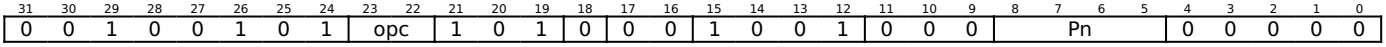
The encodings in this section are decoded from SVE encodings.



Decode fields					Instruction details
op0	op1	op2	op3	op4	
0	00	000		00000	SVE FFR write from predicate
1	00	000	0000	00000	SVE FFR initialize
1	00	000	!= 0000	00000	UNALLOCATED
	00	000		!= 00000	UNALLOCATED
	00	!= 000			UNALLOCATED
	!= 00				UNALLOCATED

SVE FFR write from predicate

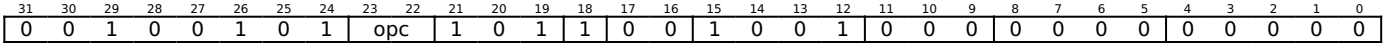
The encodings in this section are decoded from SVE Write FFR.



Decode fields	Instruction Details	Condition
opc		
00	WRFFR	FEAT_SVE
!= 00	UNALLOCATED	-

SVE FFR initialize

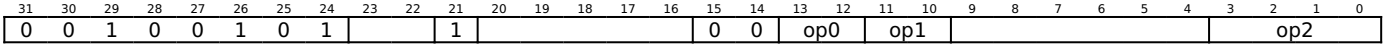
The encodings in this section are decoded from SVE Write FFR.



Decode fields opc	Instruction Details	Condition
00	SETFFR	FEAT_SVE
!= 00	UNALLOCATED	-

SVE Integer Compare - Scalars

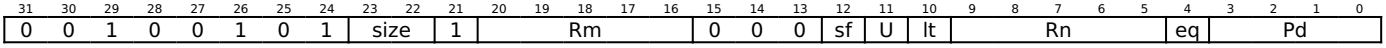
The encodings in this section are decoded from SVE encodings.



Decode fields op0	op1	op2	Instruction details
0x			SVE integer compare scalar count and limit
10	00	0000	SVE conditionally terminate scalars
10	00	!= 0000	UNALLOCATED
11	00		SVE pointer conflict compare
1x	!= 00		UNALLOCATED

SVE integer compare scalar count and limit

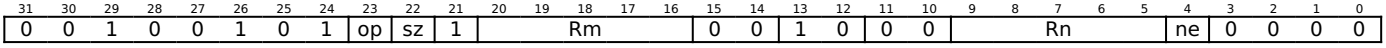
The encodings in this section are decoded from SVE Integer Compare - Scalars.



Decode fields U	lt	eq	Instruction Details	Condition
0	0	0	WHILEGE (predicate)	FEAT_SVE2 || FEAT_SME
0	0	1	WHILEGT (predicate)	FEAT_SVE2 || FEAT_SME
0	1	0	WHILELT (predicate)	FEAT_SVE || FEAT_SME
0	1	1	WHILELE (predicate)	FEAT_SVE || FEAT_SME
1	0	0	WHILEHS (predicate)	FEAT_SVE2 || FEAT_SME
1	0	1	WHILEHI (predicate)	FEAT_SVE2 || FEAT_SME
1	1	0	WHILELO (predicate)	FEAT_SVE || FEAT_SME
1	1	1	WHILELS (predicate)	FEAT_SVE || FEAT_SME

SVE conditionally terminate scalars

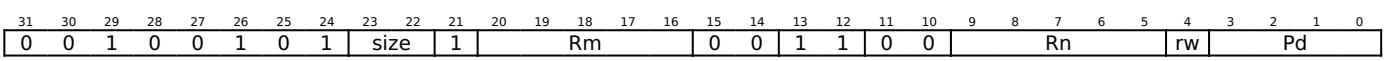
The encodings in this section are decoded from SVE Integer Compare - Scalars.



Decode fields op	ne	Instruction Details	Condition
0		UNALLOCATED	-
1	0	CTERMEQ, CTERMNE — Equal	FEAT_SVE || FEAT_SME
1	1	CTERMEQ, CTERMNE — Not equal	FEAT_SVE || FEAT_SME

SVE pointer conflict compare

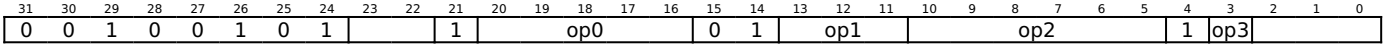
The encodings in this section are decoded from SVE Integer Compare - Scalars.



Decode fields rw	Instruction Details	Condition
0	WHILEWR	FEAT_SVE2 || FEAT_SME
1	WHILERW	FEAT_SVE2 || FEAT_SME

SVE Scalar Integer Compare - Predicate-as-counter

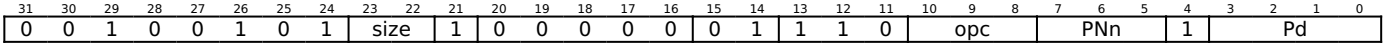
The encodings in this section are decoded from SVE encodings.



Decode fields op0	op1	op2	op3	Instruction details	Condition
00000	110			SVE extract mask predicate from predicate-as-counter	-
00000	111	000000	0	PTRUE (predicate as counter)	FEAT_SME2 || FEAT_SVE2p1
00000	111	000000	1	UNALLOCATED	-
00000	111	!= 000000		UNALLOCATED	-
	01x			SVE integer compare scalar count and limit (predicate pair)	-
	x0x			SVE integer compare scalar count and limit (predicate-as-counter)	-
!= 00000	11x			UNALLOCATED	-

SVE extract mask predicate from predicate-as-counter

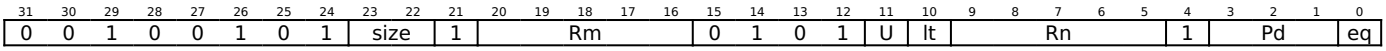
The encodings in this section are decoded from SVE Scalar Integer Compare - Predicate-as-counter.



Decode fields opc	Instruction Details	Condition
0xx	PEXT (predicate)	FEAT_SME2 || FEAT_SVE2p1
10x	PEXT (predicate pair)	FEAT_SME2 || FEAT_SVE2p1
11x	UNALLOCATED	-

SVE integer compare scalar count and limit (predicate pair)

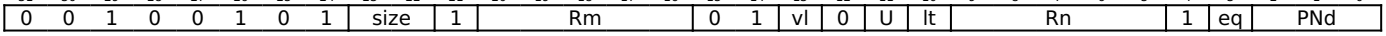
The encodings in this section are decoded from SVE Scalar Integer Compare - Predicate-as-counter.



Decode fields U	lt	eq	Instruction Details	Condition
0	0	0	WHILEGE (predicate pair)	FEAT_SME2 || FEAT_SVE2p1
0	0	1	WHILEGT (predicate pair)	FEAT_SME2 || FEAT_SVE2p1
0	1	0	WHILELT (predicate pair)	FEAT_SME2 || FEAT_SVE2p1
0	1	1	WHILELE (predicate pair)	FEAT_SME2 || FEAT_SVE2p1
1	0	0	WHILEHS (predicate pair)	FEAT_SME2 || FEAT_SVE2p1
1	0	1	WHILEHI (predicate pair)	FEAT_SME2 || FEAT_SVE2p1
1	1	0	WHILELO (predicate pair)	FEAT_SME2 || FEAT_SVE2p1
1	1	1	WHILELS (predicate pair)	FEAT_SME2 || FEAT_SVE2p1

SVE integer compare scalar count and limit (predicate-as-counter)

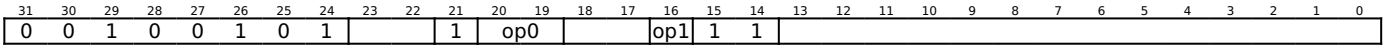
The encodings in this section are decoded from SVE Scalar Integer Compare - Predicate-as-counter.



Decode fields U	lt	eq	Instruction Details	Condition
0	0	0	WHILEGE (predicate as counter)	FEAT_SME2 || FEAT_SVE2p1
0	0	1	WHILEGT (predicate as counter)	FEAT_SME2 || FEAT_SVE2p1
0	1	0	WHILELT (predicate as counter)	FEAT_SME2 || FEAT_SVE2p1
0	1	1	WHILELE (predicate as counter)	FEAT_SME2 || FEAT_SVE2p1
1	0	0	WHILEHS (predicate as counter)	FEAT_SME2 || FEAT_SVE2p1
1	0	1	WHILEHI (predicate as counter)	FEAT_SME2 || FEAT_SVE2p1
1	1	0	WHILELO (predicate as counter)	FEAT_SME2 || FEAT_SVE2p1
1	1	1	WHILELS (predicate as counter)	FEAT_SME2 || FEAT_SVE2p1

SVE Integer Wide Immediate - Unpredicated

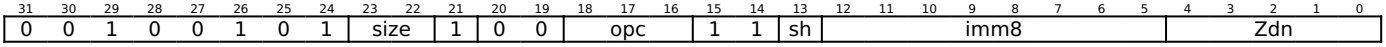
The encodings in this section are decoded from SVE encodings.



Decode fields op0	op1	Instruction details
00		SVE integer add/subtract immediate (unpredicated)
01		SVE integer min/max immediate (unpredicated)
10		SVE integer multiply immediate (unpredicated)
11	0	SVE broadcast integer immediate (unpredicated)
11	1	SVE broadcast floating-point immediate (unpredicated)

SVE integer add/subtract immediate (unpredicated)

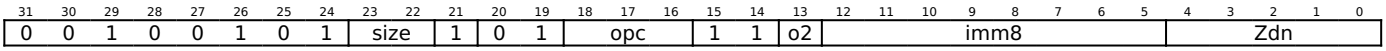
The encodings in this section are decoded from SVE Integer Wide Immediate - Unpredicated.



Decode fields opc	Instruction Details	Condition
000	ADD (immediate)	FEAT_SVE || FEAT_SME
001	SUB (immediate)	FEAT_SVE || FEAT_SME
010	UNALLOCATED	-
011	SUBR (immediate)	FEAT_SVE || FEAT_SME
100	SQADD (immediate)	FEAT_SVE || FEAT_SME
101	UQADD (immediate)	FEAT_SVE || FEAT_SME
110	SQSUB (immediate)	FEAT_SVE || FEAT_SME
111	UQSUB (immediate)	FEAT_SVE || FEAT_SME

SVE integer min/max immediate (unpredicated)

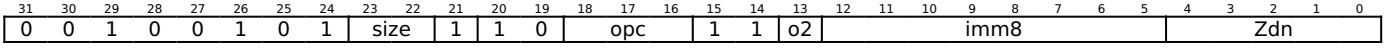
The encodings in this section are decoded from SVE Integer Wide Immediate - Unpredicated.



Decode fields		Instruction Details	Condition
opc	o2		
0xx	1	UNALLOCATED	-
000	0	SMAX (immediate)	FEAT_SVE || FEAT_SME
001	0	UMAX (immediate)	FEAT_SVE || FEAT_SME
010	0	SMIN (immediate)	FEAT_SVE || FEAT_SME
011	0	UMIN (immediate)	FEAT_SVE || FEAT_SME
1xx		UNALLOCATED	-

SVE integer multiply immediate (unpredicated)

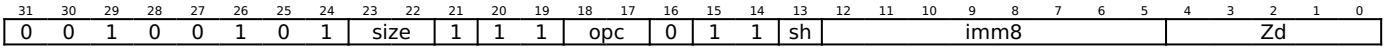
The encodings in this section are decoded from SVE Integer Wide Immediate - Unpredicated.



Decode fields		Instruction Details	Condition
opc	o2		
000	0	MUL (immediate)	FEAT_SVE || FEAT_SME
000	1	UNALLOCATED	-
!= 000		UNALLOCATED	-

SVE broadcast integer immediate (unpredicated)

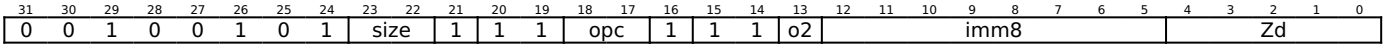
The encodings in this section are decoded from SVE Integer Wide Immediate - Unpredicated.



Decode fields	Instruction Details	Condition
opc		
00	DUP (immediate)	FEAT_SVE || FEAT_SME
!= 00	UNALLOCATED	-

SVE broadcast floating-point immediate (unpredicated)

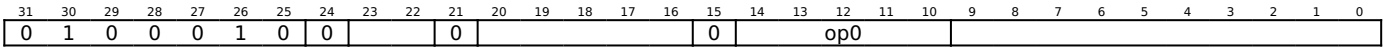
The encodings in this section are decoded from SVE Integer Wide Immediate - Unpredicated.



Decode fields		Instruction Details	Condition
opc	o2		
00	0	FDUP	FEAT_SVE || FEAT_SME
00	1	UNALLOCATED	-
!= 00		UNALLOCATED	-

SVE Integer Multiply-Add - Unpredicated

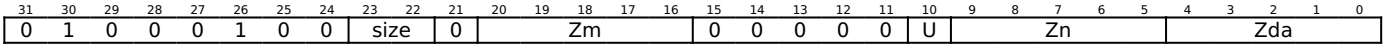
The encodings in this section are decoded from SVE encodings.



Decode fields op0	Instruction details	Condition
0000x	SVE integer dot product (unpredicated)	-
0001x	SVE2 saturating multiply-add interleaved long	-
001xx	CDOT (vectors)	FEAT_SVE2 || FEAT_SME
01xxx	SVE2 complex integer multiply-add	-
10xxx	SVE2 integer multiply-add long	-
110xx	SVE2 saturating multiply-add long	-
1110x	SVE2 saturating multiply-add high	-
11110	SVE mixed sign dot product	-
11111	UNALLOCATED	-

SVE integer dot product (unpredicated)

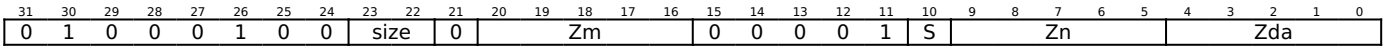
The encodings in this section are decoded from SVE Integer Multiply-Add - Unpredicated.



Decode fields size	U	Instruction Details	Condition
00		UNALLOCATED	-
01	0	SDOT (2-way, vectors)	FEAT_SVE2p3 || FEAT_SME2p3
01	1	UDOT (2-way, vectors)	FEAT_SVE2p3 || FEAT_SME2p3
1x	0	SDOT (4-way, vectors)	FEAT_SVE || FEAT_SME
1x	1	UDOT (4-way, vectors)	FEAT_SVE || FEAT_SME

SVE2 saturating multiply-add interleaved long

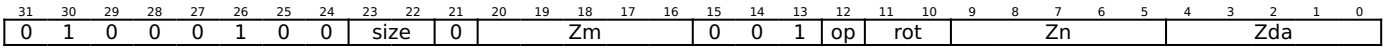
The encodings in this section are decoded from SVE Integer Multiply-Add - Unpredicated.



Decode fields S	Instruction Details	Condition
0	SQDMLALBT	FEAT_SVE2 || FEAT_SME
1	SQDMLSLBT	FEAT_SVE2 || FEAT_SME

SVE2 complex integer multiply-add

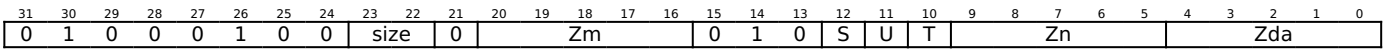
The encodings in this section are decoded from SVE Integer Multiply-Add - Unpredicated.



Decode fields op	Instruction Details	Condition
0	CMLA (vectors)	FEAT_SVE2 || FEAT_SME
1	SQRDCMLAH (vectors)	FEAT_SVE2 || FEAT_SME

SVE2 integer multiply-add long

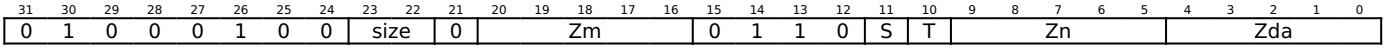
The encodings in this section are decoded from SVE Integer Multiply-Add - Unpredicated.



Decode fields S	U	T	Instruction Details	Condition
0	0	0	SMLALB (vectors)	FEAT_SVE2 || FEAT_SME
0	0	1	SMLALT (vectors)	FEAT_SVE2 || FEAT_SME
0	1	0	UMLALB (vectors)	FEAT_SVE2 || FEAT_SME
0	1	1	UMLALT (vectors)	FEAT_SVE2 || FEAT_SME
1	0	0	SMLSLB (vectors)	FEAT_SVE2 || FEAT_SME
1	0	1	SMLSLT (vectors)	FEAT_SVE2 || FEAT_SME
1	1	0	UMLSLB (vectors)	FEAT_SVE2 || FEAT_SME
1	1	1	UMLSLT (vectors)	FEAT_SVE2 || FEAT_SME

SVE2 saturating multiply-add long

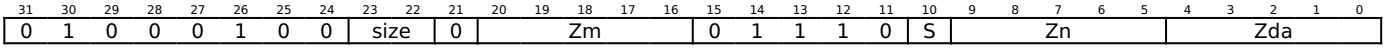
The encodings in this section are decoded from SVE Integer Multiply-Add - Unpredicated.



Decode fields S	T	Instruction Details	Condition
0	0	SQDMLALB (vectors)	FEAT_SVE2 || FEAT_SME
0	1	SQDMLALT (vectors)	FEAT_SVE2 || FEAT_SME
1	0	SQDMLSLB (vectors)	FEAT_SVE2 || FEAT_SME
1	1	SQDMLSLT (vectors)	FEAT_SVE2 || FEAT_SME

SVE2 saturating multiply-add high

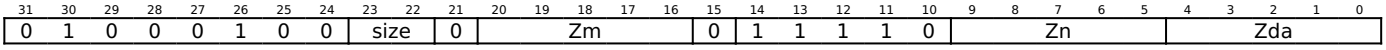
The encodings in this section are decoded from SVE Integer Multiply-Add - Unpredicated.



Decode fields S	Instruction Details	Condition
0	SQRDMLAH (vectors)	FEAT_SVE2 || FEAT_SME
1	SQRDMLSH (vectors)	FEAT_SVE2 || FEAT_SME

SVE mixed sign dot product

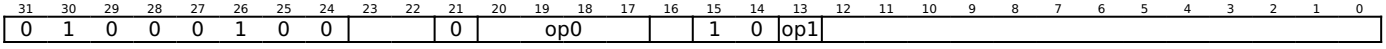
The encodings in this section are decoded from SVE Integer Multiply-Add - Unpredicated.



Decode fields size	Instruction Details	Condition
10	USDOT (vectors)	(FEAT_SVE && FEAT_I8MM) || (FEAT_SME && FEAT_I8MM)
!= 10	UNALLOCATED	-

SVE2 Integer - Predicated

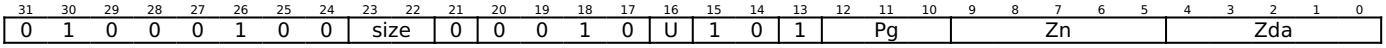
The encodings in this section are decoded from SVE encodings.



Decode fields		Instruction details
op0	op1	
0010	1	SVE2 integer pairwise add and accumulate long
0011	1	UNALLOCATED
011x	1	UNALLOCATED
0x0x	1	SVE2 integer unary operations (predicated)
0xxx	0	SVE2 saturating/rounding bitwise shift left (predicated)
10xx	0	SVE2 integer halving add/subtract (predicated)
10xx	1	SVE2 integer pairwise arithmetic
11xx	0	SVE2 saturating add/subtract
11xx	1	UNALLOCATED

SVE2 integer pairwise add and accumulate long

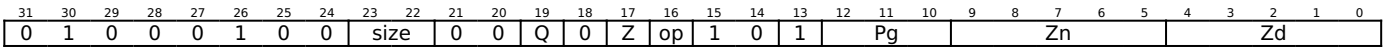
The encodings in this section are decoded from SVE2 Integer - Predicated.



Decode fields	Instruction Details	Condition
U		
0	SADALP	FEAT_SVE2 || FEAT_SME
1	UADALP	FEAT_SVE2 || FEAT_SME

SVE2 integer unary operations (predicated)

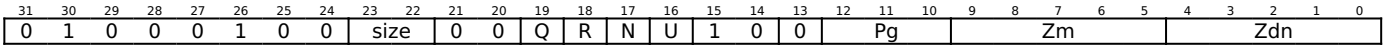
The encodings in this section are decoded from SVE2 Integer - Predicated.



Decode fields			Instruction Details	Condition
Q	Z	op		
0	0	0	URECPE — Merging	FEAT_SVE2 || FEAT_SME
0	0	1	URSQRTE — Merging	FEAT_SVE2 || FEAT_SME
0	1	0	URECPE — Zeroing	FEAT_SVE2p2 || FEAT_SME2p2
0	1	1	URSQRTE — Zeroing	FEAT_SVE2p2 || FEAT_SME2p2
1	0	0	SQABS — Merging	FEAT_SVE2 || FEAT_SME
1	0	1	SQNEG — Merging	FEAT_SVE2 || FEAT_SME
1	1	0	SQABS — Zeroing	FEAT_SVE2p2 || FEAT_SME2p2
1	1	1	SQNEG — Zeroing	FEAT_SVE2p2 || FEAT_SME2p2

SVE2 saturating/rounding bitwise shift left (predicated)

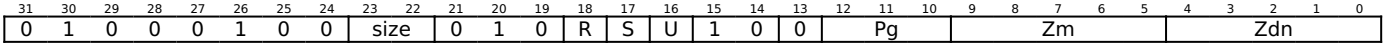
The encodings in this section are decoded from SVE2 Integer - Predicated.



Decode fields				Instruction Details	Condition
Q	R	N	U		
0		0		UNALLOCATED	-
0	0	1	0	SRSHL	FEAT_SVE2 || FEAT_SME
0	0	1	1	URSHL	FEAT_SVE2 || FEAT_SME
0	1	1	0	SRSHLR	FEAT_SVE2 || FEAT_SME
0	1	1	1	URSHLR	FEAT_SVE2 || FEAT_SME
1	0	0	0	SQSHL (vectors)	FEAT_SVE2 || FEAT_SME
1	0	0	1	UQSHL (vectors)	FEAT_SVE2 || FEAT_SME
1	0	1	0	SQRSHL	FEAT_SVE2 || FEAT_SME
1	0	1	1	UQRSHL	FEAT_SVE2 || FEAT_SME
1	1	0	0	SQSHLR	FEAT_SVE2 || FEAT_SME
1	1	0	1	UQSHLR	FEAT_SVE2 || FEAT_SME
1	1	1	0	SQRSHLR	FEAT_SVE2 || FEAT_SME
1	1	1	1	UQRSHLR	FEAT_SVE2 || FEAT_SME

SVE2 integer halving add/subtract (predicated)

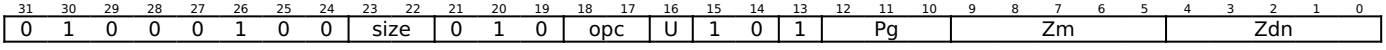
The encodings in this section are decoded from SVE2 Integer - Predicated.



Decode fields			Instruction Details	Condition
R	S	U		
0	0	0	SHADD	FEAT_SVE2 || FEAT_SME
0	0	1	UHADD	FEAT_SVE2 || FEAT_SME
0	1	0	SHSUB	FEAT_SVE2 || FEAT_SME
0	1	1	UHSUB	FEAT_SVE2 || FEAT_SME
1	0	0	SRHADD	FEAT_SVE2 || FEAT_SME
1	0	1	URHADD	FEAT_SVE2 || FEAT_SME
1	1	0	SHSUBR	FEAT_SVE2 || FEAT_SME
1	1	1	UHSUBR	FEAT_SVE2 || FEAT_SME

SVE2 integer pairwise arithmetic

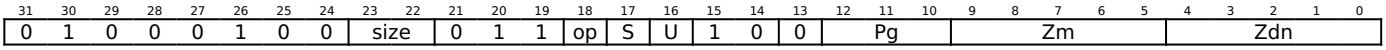
The encodings in this section are decoded from SVE2 Integer - Predicated.



Decode fields		Instruction Details	Condition
opc	U		
00	0	SUBP	FEAT_SVE2p3 || FEAT_SME2p3
00	1	ADDP	FEAT_SVE2 || FEAT_SME
01		UNALLOCATED	-
10	0	SMAXP	FEAT_SVE2 || FEAT_SME
10	1	UMAXP	FEAT_SVE2 || FEAT_SME
11	0	SMINP	FEAT_SVE2 || FEAT_SME
11	1	UMINP	FEAT_SVE2 || FEAT_SME

SVE2 saturating add/subtract

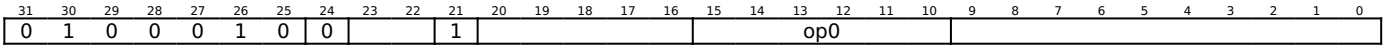
The encodings in this section are decoded from SVE2 Integer - Predicated.



Decode fields			Instruction Details	Condition
op	S	U		
0	0	0	SQADD (vectors, predicated)	FEAT_SVE2 || FEAT_SME
0	0	1	UQADD (vectors, predicated)	FEAT_SVE2 || FEAT_SME
0	1	0	SQSUB (vectors, predicated)	FEAT_SVE2 || FEAT_SME
0	1	1	UQSUB (vectors, predicated)	FEAT_SVE2 || FEAT_SME
1	0	0	SUQADD	FEAT_SVE2 || FEAT_SME
1	0	1	USQADD	FEAT_SVE2 || FEAT_SME
1	1	0	SQSUBR	FEAT_SVE2 || FEAT_SME
1	1	1	UQSUBR	FEAT_SVE2 || FEAT_SME

SVE Multiply - Indexed

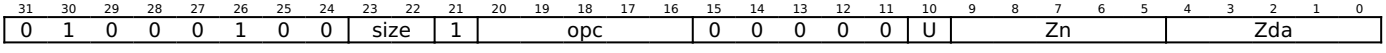
The encodings in this section are decoded from SVE encodings.



Decode fields	Instruction details
op0	
00000x	SVE integer dot product (indexed)
00001x	SVE2 integer multiply-add (indexed)
00010x	SVE2 saturating multiply-add high (indexed)
00011x	SVE mixed sign dot product (indexed)
001xxx	SVE2 saturating multiply-add (indexed)
0100xx	SVE2 complex integer dot product (indexed)
0101xx	UNALLOCATED
0110xx	SVE2 complex integer multiply-add (indexed)
0111xx	SVE2 complex saturating multiply-add (indexed)
10xxxx	SVE2 integer multiply-add long (indexed)
110xxx	SVE2 integer multiply long (indexed)
1110xx	SVE2 saturating multiply (indexed)
11110x	SVE2 saturating multiply high (indexed)
111110	SVE2 integer multiply (indexed)
111111	UNALLOCATED

SVE integer dot product (indexed)

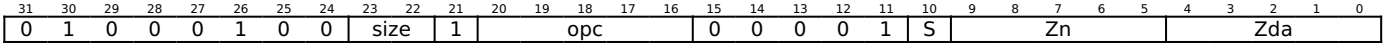
The encodings in this section are decoded from SVE Multiply - Indexed.



Decode fields size	U	Instruction Details	Condition
0x	0	SDOT (2-way, indexed)	FEAT_SVE2p3 || FEAT_SME2p3
0x	1	UDOT (2-way, indexed)	FEAT_SVE2p3 || FEAT_SME2p3
10	0	SDOT (4-way, indexed) — 8-bit to 32-bit	FEAT_SVE || FEAT_SME
10	1	UDOT (4-way, indexed) — 8-bit to 32-bit	FEAT_SVE || FEAT_SME
11	0	SDOT (4-way, indexed) — 16-bit to 64-bit	FEAT_SVE || FEAT_SME
11	1	UDOT (4-way, indexed) — 16-bit to 64-bit	FEAT_SVE || FEAT_SME

SVE2 integer multiply-add (indexed)

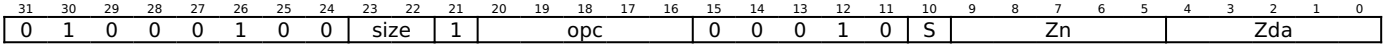
The encodings in this section are decoded from SVE Multiply - Indexed.



Decode fields size	S	Instruction Details	Condition
0x	0	MLA (indexed) — 16-bit	FEAT_SVE2 || FEAT_SME
0x	1	MLS (indexed) — 16-bit	FEAT_SVE2 || FEAT_SME
10	0	MLA (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
10	1	MLS (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
11	0	MLA (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME
11	1	MLS (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME

SVE2 saturating multiply-add high (indexed)

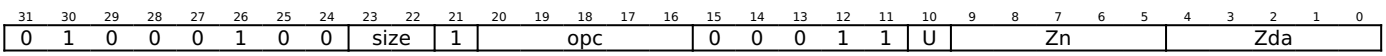
The encodings in this section are decoded from SVE Multiply - Indexed.



Decode fields size	S	Instruction Details	Condition
0x	0	SQRDMLAH (indexed) — 16-bit	FEAT_SVE2 || FEAT_SME
0x	1	SQRDMLSH (indexed) — 16-bit	FEAT_SVE2 || FEAT_SME
10	0	SQRDMLAH (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
10	1	SQRDMLSH (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
11	0	SQRDMLAH (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME
11	1	SQRDMLSH (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME

SVE mixed sign dot product (indexed)

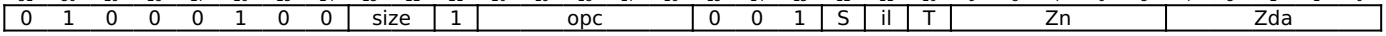
The encodings in this section are decoded from SVE Multiply - Indexed.



Decode fields size	U	Instruction Details	Condition
10	0	USDOT (indexed)	(FEAT_SVE && FEAT_I8MM) || (FEAT_SME && FEAT_I8MM)
10	1	SUDOT	(FEAT_SVE && FEAT_I8MM) || (FEAT_SME && FEAT_I8MM)
!= 10		UNALLOCATED	-

SVE2 saturating multiply-add (indexed)

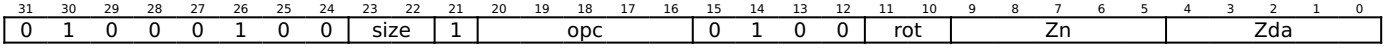
The encodings in this section are decoded from SVE Multiply - Indexed.



Decode fields size	S	T	Instruction Details	Condition
0x			UNALLOCATED	-
10	0	0	SQDMLALB (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
10	0	1	SQDMLALT (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
10	1	0	SQDMLSLB (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
10	1	1	SQDMLSLT (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
11	0	0	SQDMLALB (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME
11	0	1	SQDMLALT (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME
11	1	0	SQDMLSLB (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME
11	1	1	SQDMLSLT (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME

SVE2 complex integer dot product (indexed)

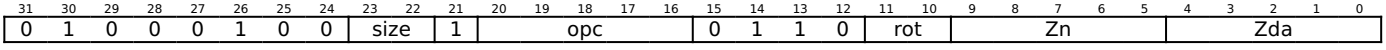
The encodings in this section are decoded from SVE Multiply - Indexed.



Decode fields size	Instruction Details	Condition
0x	UNALLOCATED	-
10	CDOT (indexed) — 8-bit to 32-bit	FEAT_SVE2 || FEAT_SME
11	CDOT (indexed) — 16-bit to 64-bit	FEAT_SVE2 || FEAT_SME

SVE2 complex integer multiply-add (indexed)

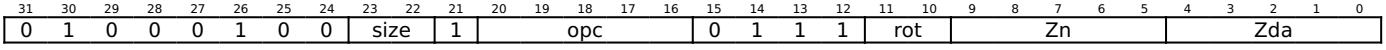
The encodings in this section are decoded from SVE Multiply - Indexed.



Decode fields size	Instruction Details	Condition
0x	UNALLOCATED	-
10	CMLA (indexed) — 16-bit	FEAT_SVE2 || FEAT_SME
11	CMLA (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME

SVE2 complex saturating multiply-add (indexed)

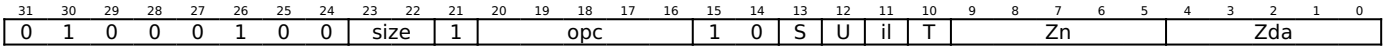
The encodings in this section are decoded from SVE Multiply - Indexed.



Decode fields size	Instruction Details	Condition
0x	UNALLOCATED	-
10	SQRDCMLAH (indexed) — 16-bit	FEAT_SVE2 || FEAT_SME
11	SQRDCMLAH (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME

SVE2 integer multiply-add long (indexed)

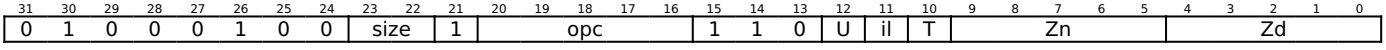
The encodings in this section are decoded from SVE Multiply - Indexed.



Decode fields				Instruction Details	Condition
size	S	U	T		
0x				UNALLOCATED	-
10	0	0	0	SMLALB (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
10	0	0	1	SMLALT (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
10	0	1	0	UMLALB (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
10	0	1	1	UMLALT (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
10	1	0	0	SMLSLB (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
10	1	0	1	SMLSLT (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
10	1	1	0	UMLSLB (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
10	1	1	1	UMLSLT (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
11	0	0	0	SMLALB (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME
11	0	0	1	SMLALT (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME
11	0	1	0	UMLALB (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME
11	0	1	1	UMLALT (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME
11	1	0	0	SMLSLB (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME
11	1	0	1	SMLSLT (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME
11	1	1	0	UMLSLB (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME
11	1	1	1	UMLSLT (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME

SVE2 integer multiply long (indexed)

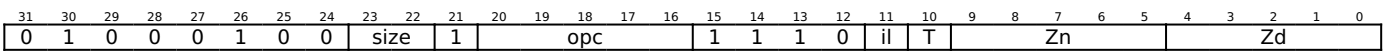
The encodings in this section are decoded from SVE Multiply - Indexed.



Decode fields			Instruction Details	Condition
size	U	T		
0x			UNALLOCATED	-
10	0	0	SMULLB (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
10	0	1	SMULLT (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
10	1	0	UMULLB (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
10	1	1	UMULLT (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
11	0	0	SMULLB (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME
11	0	1	SMULLT (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME
11	1	0	UMULLB (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME
11	1	1	UMULLT (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME

SVE2 saturating multiply (indexed)

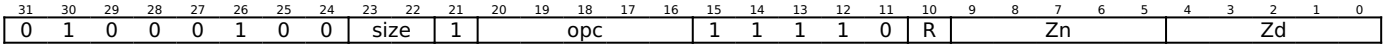
The encodings in this section are decoded from SVE Multiply - Indexed.



Decode fields size	T	Instruction Details	Condition
0x		UNALLOCATED	-
10	0	SQDMULLB (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
10	1	SQDMULLT (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
11	0	SQDMULLB (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME
11	1	SQDMULLT (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME

SVE2 saturating multiply high (indexed)

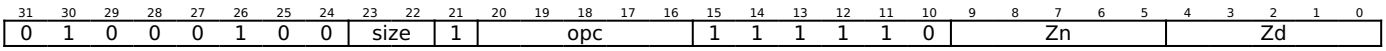
The encodings in this section are decoded from SVE Multiply - Indexed.



Decode fields size	R	Instruction Details	Condition
0x	0	SQDMULH (indexed) — 16-bit	FEAT_SVE2 || FEAT_SME
0x	1	SQRDMULH (indexed) — 16-bit	FEAT_SVE2 || FEAT_SME
10	0	SQDMULH (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
10	1	SQRDMULH (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
11	0	SQDMULH (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME
11	1	SQRDMULH (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME

SVE2 integer multiply (indexed)

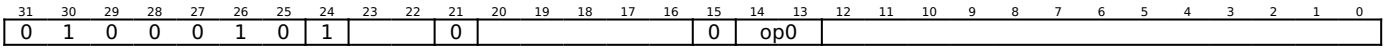
The encodings in this section are decoded from SVE Multiply - Indexed.



Decode fields size	Instruction Details	Condition
0x	MUL (indexed) — 16-bit	FEAT_SVE2 || FEAT_SME
10	MUL (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
11	MUL (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME

SVE2 Widening Integer Arithmetic

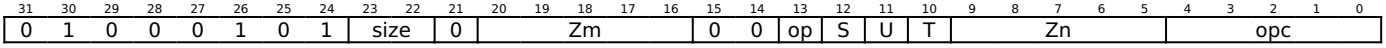
The encodings in this section are decoded from SVE encodings.



Decode fields op0	Instruction details
0x	SVE2 integer add/subtract long
10	SVE2 integer add/subtract wide
11	SVE2 integer multiply long

SVE2 integer add/subtract long

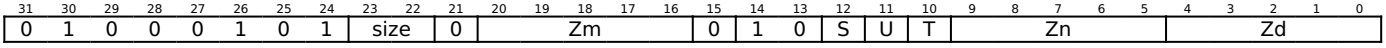
The encodings in this section are decoded from SVE2 Widening Integer Arithmetic.



Decode fields				Instruction Details	Condition
op	S	U	T		
0	0	0	0	SADDLB	FEAT_SVE2 || FEAT_SME
0	0	0	1	SADDLT	FEAT_SVE2 || FEAT_SME
0	0	1	0	UADDLB	FEAT_SVE2 || FEAT_SME
0	0	1	1	UADDLT	FEAT_SVE2 || FEAT_SME
0	1	0	0	SSUBLB	FEAT_SVE2 || FEAT_SME
0	1	0	1	SSUBLT	FEAT_SVE2 || FEAT_SME
0	1	1	0	USUBLB	FEAT_SVE2 || FEAT_SME
0	1	1	1	USUBLT	FEAT_SVE2 || FEAT_SME
1	0			UNALLOCATED	-
1	1	0	0	SABDLB	FEAT_SVE2 || FEAT_SME
1	1	0	1	SABDLT	FEAT_SVE2 || FEAT_SME
1	1	1	0	UABDLB	FEAT_SVE2 || FEAT_SME
1	1	1	1	UABDLT	FEAT_SVE2 || FEAT_SME

SVE2 integer add/subtract wide

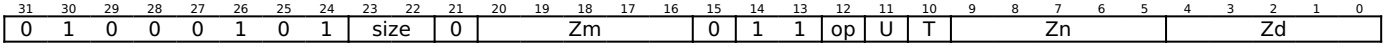
The encodings in this section are decoded from SVE2 Widening Integer Arithmetic.



Decode fields			Instruction Details	Condition
S	U	T		
0	0	0	SADDWB	FEAT_SVE2 || FEAT_SME
0	0	1	SADDWT	FEAT_SVE2 || FEAT_SME
0	1	0	UADDWB	FEAT_SVE2 || FEAT_SME
0	1	1	UADDWT	FEAT_SVE2 || FEAT_SME
1	0	0	SSUBWB	FEAT_SVE2 || FEAT_SME
1	0	1	SSUBWT	FEAT_SVE2 || FEAT_SME
1	1	0	USUBWB	FEAT_SVE2 || FEAT_SME
1	1	1	USUBWT	FEAT_SVE2 || FEAT_SME

SVE2 integer multiply long

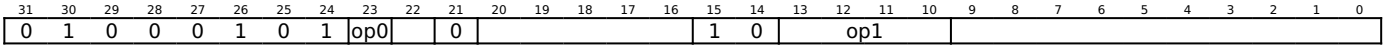
The encodings in this section are decoded from SVE2 Widening Integer Arithmetic.



Decode fields				Instruction Details	Condition
size	op	U	T		
	0	0	0	SQDMULLB (vectors)	FEAT_SVE2 || FEAT_SME
	0	0	1	SQDMULLT (vectors)	FEAT_SVE2 || FEAT_SME
	1	0	0	SMULLB (vectors)	FEAT_SVE2 || FEAT_SME
	1	0	1	SMULLT (vectors)	FEAT_SVE2 || FEAT_SME
	1	1	0	UMULLB (vectors)	FEAT_SVE2 || FEAT_SME
	1	1	1	UMULLT (vectors)	FEAT_SVE2 || FEAT_SME
00	0	1	0	PMULLB — 128-bit element	FEAT_SVE_PMULL128
00	0	1	1	PMULLT — 128-bit element	FEAT_SVE_PMULL128
!= 00	0	1	0	PMULLB — 16-bit or 64-bit elements	FEAT_SVE2 || FEAT_SME
!= 00	0	1	1	PMULLT — 16-bit or 64-bit elements	FEAT_SVE2 || FEAT_SME

SVE Misc

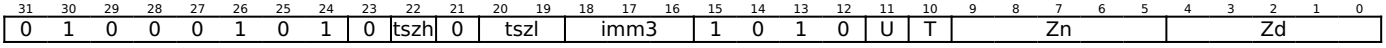
The encodings in this section are decoded from SVE encodings.



Decode fields		Instruction details
op0	op1	
0	10xx	SVE2 bitwise shift left long
1	10xx	UNALLOCATED
	00xx	SVE2 integer add/subtract interleaved long
	010x	SVE2 bitwise exclusive-OR interleaved
	0110	SVE integer matrix multiply accumulate
	0111	UNALLOCATED
	11xx	SVE2 bitwise permute

SVE2 bitwise shift left long

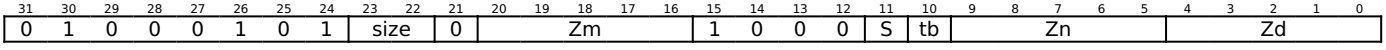
The encodings in this section are decoded from SVE Misc.



Decode fields		Instruction Details	Condition
U	T		
0	0	SSHLLB	FEAT_SVE2 || FEAT_SME
0	1	SSHLLT	FEAT_SVE2 || FEAT_SME
1	0	USHLLB	FEAT_SVE2 || FEAT_SME
1	1	USHLLT	FEAT_SVE2 || FEAT_SME

SVE2 integer add/subtract interleaved long

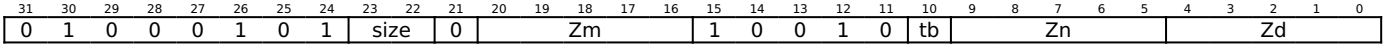
The encodings in this section are decoded from SVE Misc.



Decode fields S	tb	Instruction Details	Condition
0	0	SADDLBT	FEAT_SVE2 || FEAT_SME
0	1	UNALLOCATED	-
1	0	SSUBLBT	FEAT_SVE2 || FEAT_SME
1	1	SSUBLTB	FEAT_SVE2 || FEAT_SME

SVE2 bitwise exclusive-OR interleaved

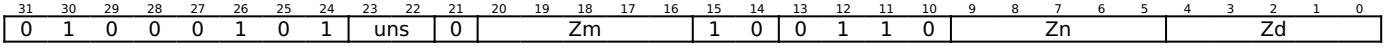
The encodings in this section are decoded from SVE Misc.



Decode fields tb	Instruction Details	Condition
0	EORBT	FEAT_SVE2 || FEAT_SME
1	EORTB	FEAT_SVE2 || FEAT_SME

SVE integer matrix multiply accumulate

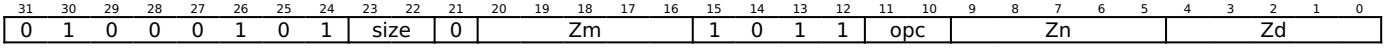
The encodings in this section are decoded from SVE Misc.



Decode fields uns	Instruction Details	Condition
00	SMMLA	FEAT_SVE && FEAT_I8MM
01	UNALLOCATED	-
10	USMMLA	FEAT_SVE && FEAT_I8MM
11	UMMLA	FEAT_SVE && FEAT_I8MM

SVE2 bitwise permute

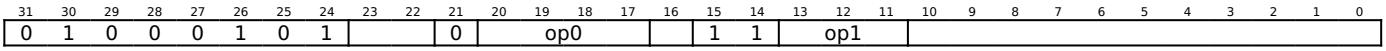
The encodings in this section are decoded from SVE Misc.



Decode fields opc	Instruction Details	Condition
00	BEXT	FEAT_SVE_BitPerm
01	BDEP	FEAT_SVE_BitPerm
10	BGRP	FEAT_SVE_BitPerm
11	UNALLOCATED	-

SVE2 Accumulate

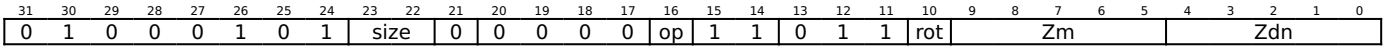
The encodings in this section are decoded from SVE encodings.



Decode fields op0	op1	Instruction details
0000	011	SVE2 complex integer add
	00x	SVE2 integer absolute difference and accumulate long
	010	SVE2 integer add/subtract long with carry
	10x	SVE2 bitwise shift right and accumulate
	110	SVE2 bitwise shift and insert
	111	SVE2 integer absolute difference and accumulate
!= 0000	011	UNALLOCATED

SVE2 complex integer add

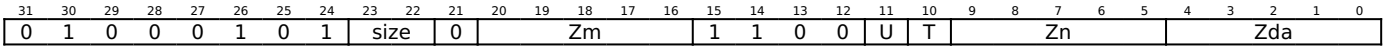
The encodings in this section are decoded from SVE2 Accumulate.



Decode fields op	Instruction Details	Condition
0	CADD	FEAT_SVE2 || FEAT_SME
1	SQCADD	FEAT_SVE2 || FEAT_SME

SVE2 integer absolute difference and accumulate long

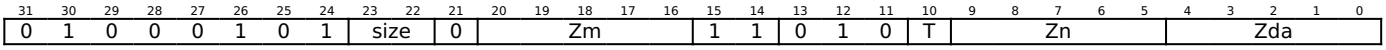
The encodings in this section are decoded from SVE2 Accumulate.



Decode fields U	T	Instruction Details	Condition
0	0	SABALB	FEAT_SVE2 || FEAT_SME
0	1	SABALT	FEAT_SVE2 || FEAT_SME
1	0	UABALB	FEAT_SVE2 || FEAT_SME
1	1	UABALT	FEAT_SVE2 || FEAT_SME

SVE2 integer add/subtract long with carry

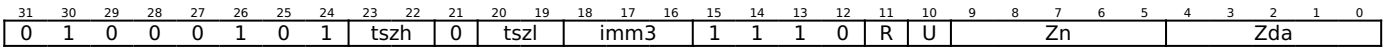
The encodings in this section are decoded from SVE2 Accumulate.



Decode fields size	T	Instruction Details	Condition
0x	0	ADCLB	FEAT_SVE2 || FEAT_SME
0x	1	ADCLT	FEAT_SVE2 || FEAT_SME
1x	0	SBCLB	FEAT_SVE2 || FEAT_SME
1x	1	SBCLT	FEAT_SVE2 || FEAT_SME

SVE2 bitwise shift right and accumulate

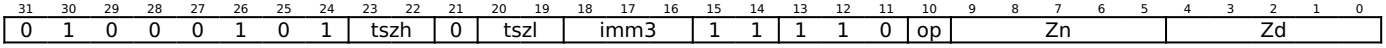
The encodings in this section are decoded from SVE2 Accumulate.



Decode fields R	U	Instruction Details	Condition
0	0	SSRA	FEAT_SVE2 || FEAT_SME
0	1	USRA	FEAT_SVE2 || FEAT_SME
1	0	SRSRA	FEAT_SVE2 || FEAT_SME
1	1	URSRA	FEAT_SVE2 || FEAT_SME

SVE2 bitwise shift and insert

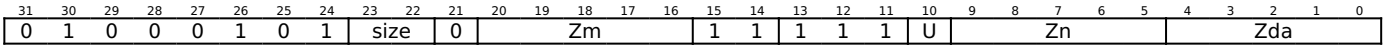
The encodings in this section are decoded from SVE2 Accumulate.



Decode fields op	Instruction Details	Condition
0	SRI	FEAT_SVE2 || FEAT_SME
1	SLI	FEAT_SVE2 || FEAT_SME

SVE2 integer absolute difference and accumulate

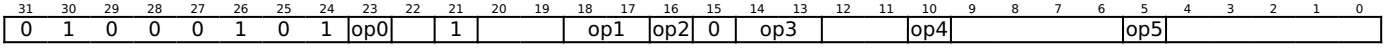
The encodings in this section are decoded from SVE2 Accumulate.



Decode fields U	Instruction Details	Condition
0	SABA	FEAT_SVE2 || FEAT_SME
1	UABA	FEAT_SVE2 || FEAT_SME

SVE2 Narrowing

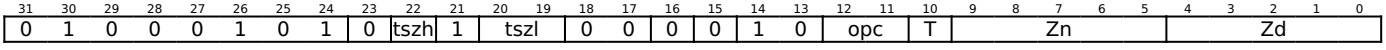
The encodings in this section are decoded from SVE encodings.



Decode fields op0	op1	op2	op3	op4	op5	Instruction details
0	00	0	10			SVE2 saturating extract narrow
0	00	1	10	0	0	SME2 multi-vec extract narrow
0	00	1	10	0	1	UNALLOCATED
0	00	1	10	1		UNALLOCATED
0			0x			SVE2 bitwise shift right narrow
0	!= 00		10			UNALLOCATED
1			0x	0	0	SME2 multi-vec shift narrow
1			0x	0	1	UNALLOCATED
1			0x	1		UNALLOCATED
1			10			UNALLOCATED
			11			SVE2 integer add/subtract narrow high part

SVE2 saturating extract narrow

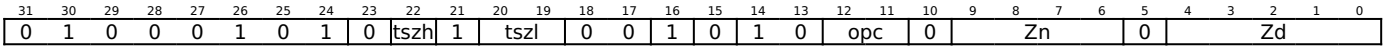
The encodings in this section are decoded from SVE2 Narrowing.



Decode fields		Instruction Details	Condition
opc	T		
00	0	SQXTNB	FEAT_SVE2 || FEAT_SME
00	1	SQXTNT	FEAT_SVE2 || FEAT_SME
01	0	UQXTNB	FEAT_SVE2 || FEAT_SME
01	1	UQXTNT	FEAT_SVE2 || FEAT_SME
10	0	SQXTUNB	FEAT_SVE2 || FEAT_SME
10	1	SQXTUNT	FEAT_SVE2 || FEAT_SME
11		UNALLOCATED	-

SME2 multi-vec extract narrow

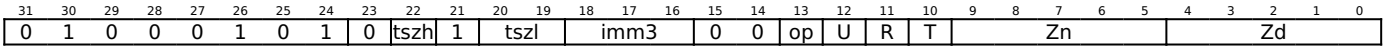
The encodings in this section are decoded from SVE2 Narrowing.



Decode fields			Instruction Details	Condition
tszh	tszl	opc		
0	10	00	SQCVTN	FEAT_SME2 || FEAT_SVE2p1
0	10	01	UQCVTN	FEAT_SME2 || FEAT_SVE2p1
0	10	10	SQCVTUN	FEAT_SME2 || FEAT_SVE2p1
0	10	11	UNALLOCATED	-
0	!= 10		UNALLOCATED	-
1			UNALLOCATED	-

SVE2 bitwise shift right narrow

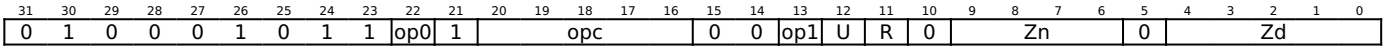
The encodings in this section are decoded from SVE2 Narrowing.



Decode fields				Instruction Details	Condition
op	U	R	T		
0	0	0	0	SQSHRUNB	FEAT_SVE2 || FEAT_SME
0	0	0	1	SQSHRUNT	FEAT_SVE2 || FEAT_SME
0	0	1	0	SQRSHRUNB	FEAT_SVE2 || FEAT_SME
0	0	1	1	SQRSHRUNT	FEAT_SVE2 || FEAT_SME
0	1	0	0	SHRNB	FEAT_SVE2 || FEAT_SME
0	1	0	1	SHRNT	FEAT_SVE2 || FEAT_SME
0	1	1	0	RSHRNB	FEAT_SVE2 || FEAT_SME
0	1	1	1	RSHRNT	FEAT_SVE2 || FEAT_SME
1	0	0	0	SQSHRNB	FEAT_SVE2 || FEAT_SME
1	0	0	1	SQSHRNT	FEAT_SVE2 || FEAT_SME
1	0	1	0	SQRSHRNB	FEAT_SVE2 || FEAT_SME
1	0	1	1	SQRSHRNT	FEAT_SVE2 || FEAT_SME
1	1	0	0	UQSHRNB	FEAT_SVE2 || FEAT_SME
1	1	0	1	UQSHRNT	FEAT_SVE2 || FEAT_SME
1	1	1	0	UQRSHRNB	FEAT_SVE2 || FEAT_SME
1	1	1	1	UQRSHRNT	FEAT_SVE2 || FEAT_SME

SME2 multi-vec shift narrow

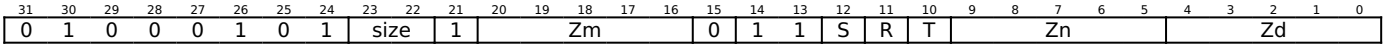
The encodings in this section are decoded from SVE2 Narrowing.



Decode fields					Instruction Details	Condition
op0	opc	op1	U	R		
0		0	0	0	SQSHRN	FEAT_SVE2p3 || FEAT_SME2p3
0		0	1	0	UQSHRN	FEAT_SVE2p3 || FEAT_SME2p3
0		1	0	0	SQSHRUN	FEAT_SVE2p3 || FEAT_SME2p3
0		1	1	0	UNALLOCATED	-
0	00xxx			1	UNALLOCATED	-
0	01xxx	0	0	1	SQRSHRUN — 8-bit	FEAT_SVE2p3 || FEAT_SME2p3
0	01xxx	1	0	1	SQRSHRN — 8-bit	FEAT_SVE2p3 || FEAT_SME2p3
0	01xxx	1	1	1	UQRSHRN — 8-bit	FEAT_SVE2p3 || FEAT_SME2p3
0	1xxxx	0	0	1	SQRSHRUN — 16-bit	FEAT_SME2 || FEAT_SVE2p1
0	1xxxx	1	0	1	SQRSHRN — 16-bit	FEAT_SME2 || FEAT_SVE2p1
0	1xxxx	1	1	1	UQRSHRN — 16-bit	FEAT_SME2 || FEAT_SVE2p1
0	!= 00xxx	0	1	1	UNALLOCATED	-
1					UNALLOCATED	-

SVE2 integer add/subtract narrow high part

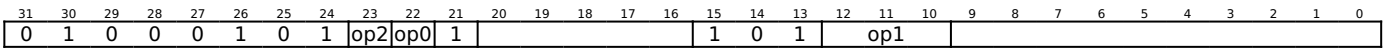
The encodings in this section are decoded from SVE2 Narrowing.



Decode fields			Instruction Details	Condition
S	R	T		
0	0	0	ADDHNB	FEAT_SVE2 || FEAT_SME
0	0	1	ADDHNT	FEAT_SVE2 || FEAT_SME
0	1	0	RADDHNB	FEAT_SVE2 || FEAT_SME
0	1	1	RADDHNT	FEAT_SVE2 || FEAT_SME
1	0	0	SUBHNB	FEAT_SVE2 || FEAT_SME
1	0	1	SUBHNT	FEAT_SVE2 || FEAT_SME
1	1	0	RSUBHNB	FEAT_SVE2 || FEAT_SME
1	1	1	RSUBHNT	FEAT_SVE2 || FEAT_SME

SVE2 Histogram (Segment) and Lookup Table

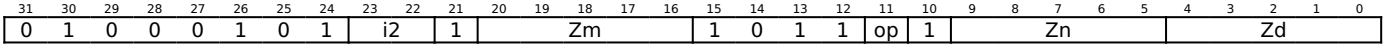
The encodings in this section are decoded from SVE encodings.



Decode fields			Instruction details	Condition
op2	op0	op1		
0	0	001	UNALLOCATED	-
0	0	011	LUTI6 (8-bit)	FEAT_SVE2p3
1	0	0x1	UNALLOCATED	-
	1	001	LUTI4 (8-bit and 16-bit) — Byte, single register table	(FEAT_SVE2 && FEAT_LUT) || (FEAT_SME2 && FEAT_LUT)
	1	011	LUTI6 (16-bit)	FEAT_SVE2p3 || FEAT_SME2p3
		000	HISTSEG	FEAT_SVE2
		100	LUTI2 (8-bit and 16-bit) — Byte	(FEAT_SVE2 && FEAT_LUT) || (FEAT_SME2 && FEAT_LUT)
		1x1	SVE2 lookup table with 4-bit indices and 16-bit element size	-
		x10	LUTI2 (8-bit and 16-bit) — Halfword	(FEAT_SVE2 && FEAT_LUT) || (FEAT_SME2 && FEAT_LUT)

SVE2 lookup table with 4-bit indices and 16-bit element size

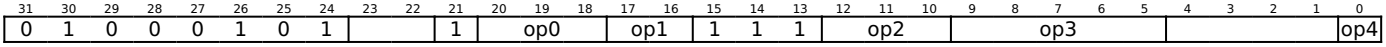
The encodings in this section are decoded from SVE2 Histogram (Segment) and Lookup Table.



Decode fields	Instruction Details	Condition
op		
0	LUTI4 (8-bit and 16-bit) — Halfword, two register table	(FEAT_SVE2 && FEAT_LUT) || (FEAT_SME2 && FEAT_LUT)
1	LUTI4 (8-bit and 16-bit) — Halfword, single register table	(FEAT_SVE2 && FEAT_LUT) || (FEAT_SME2 && FEAT_LUT)

SVE2 Crypto Extensions

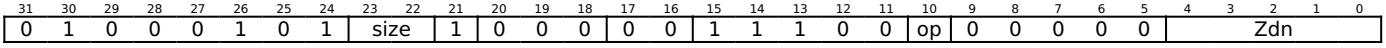
The encodings in this section are decoded from SVE encodings.



Decode fields					Instruction details
op0	op1	op2	op3	op4	
000	00	00x	00000		SVE2 crypto unary operations
000	00	00x	!= 00000		UNALLOCATED
000	01	00x			UNALLOCATED
000	1x	00x			SVE2 crypto destructive binary operations
xx0	1x	01x			SVE2 multi-vector AES single round (two registers)
xx1	1x	01x			SVE2 multi-vector AES single round (four registers)
		10x			SVE2 crypto constructive binary operations
		110		0	SVE2 Multi-vector polynomial multiply long
		111		0	SVE2 Multi-vector polynomial multiply long and accumulate vectors
		11x		1	UNALLOCATED
	!= 1x	01x			UNALLOCATED
!= 000		00x			UNALLOCATED

SVE2 crypto unary operations

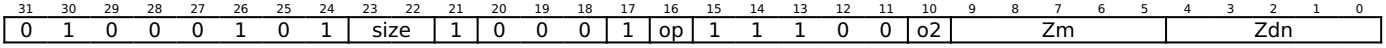
The encodings in this section are decoded from SVE2 Crypto Extensions.



Decode fields		Instruction Details	Condition
size	op		
00	0	AESMC	FEAT_SVE_AES
00	1	AESIMC	FEAT_SVE_AES
!= 00		UNALLOCATED	-

SVE2 crypto destructive binary operations

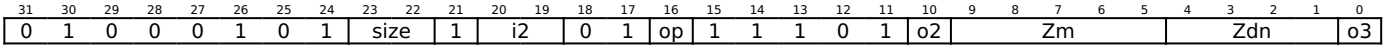
The encodings in this section are decoded from SVE2 Crypto Extensions.



Decode fields			Instruction Details	Condition
size	op	o2		
00	0	0	AESE (vectors)	FEAT_SVE_AES
00	0	1	AESD (vectors)	FEAT_SVE_AES
00	1	0	SM4E	FEAT_SVE_SM4
00	1	1	UNALLOCATED	-
!= 00			UNALLOCATED	-

SVE2 multi-vector AES single round (two registers)

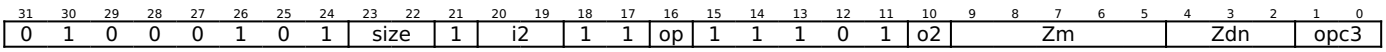
The encodings in this section are decoded from SVE2 Crypto Extensions.



Decode fields				Instruction Details	Condition
size	op	o2	o3		
00			1	UNALLOCATED	-
00	0	0	0	AESE (indexed)	FEAT_SVE_AES2
00	0	1	0	AESD (indexed)	FEAT_SVE_AES2
00	1	0	0	AESEMC	FEAT_SVE_AES2
00	1	1	0	AESDIMC	FEAT_SVE_AES2
!= 00				UNALLOCATED	-

SVE2 multi-vector AES single round (four registers)

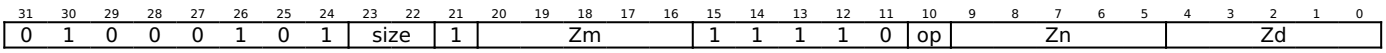
The encodings in this section are decoded from SVE2 Crypto Extensions.



Decode fields				Instruction Details	Condition
size	op	o2	opc3		
00			!= 00	UNALLOCATED	-
00	0	0	00	AESE (indexed)	FEAT_SVE_AES2
00	0	1	00	AESD (indexed)	FEAT_SVE_AES2
00	1	0	00	AESEMC	FEAT_SVE_AES2
00	1	1	00	AESDIMC	FEAT_SVE_AES2
!= 00				UNALLOCATED	-

SVE2 crypto constructive binary operations

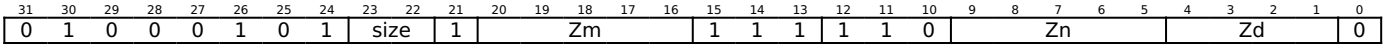
The encodings in this section are decoded from SVE2 Crypto Extensions.



Decode fields size	op	Instruction Details	Condition
00	0	SM4EKEY	FEAT_SVE_SM4
00	1	RAX1	FEAT_SVE_SHA3
!= 00		UNALLOCATED	-

SVE2 Multi-vector polynomial multiply long

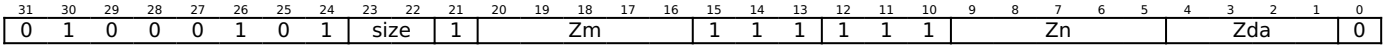
The encodings in this section are decoded from SVE2 Crypto Extensions.



Decode fields size	Instruction Details	Condition
00	PMULL	FEAT_SVE_AES2
!= 00	UNALLOCATED	-

SVE2 Multi-vector polynomial multiply long and accumulate vectors

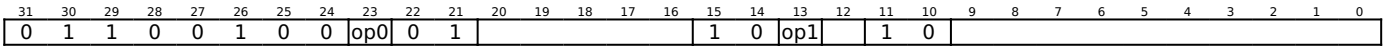
The encodings in this section are decoded from SVE2 Crypto Extensions.



Decode fields size	Instruction Details	Condition
00	PMLAL	FEAT_SVE_AES2
!= 00	UNALLOCATED	-

SVE2 FP8 widening multiply-add

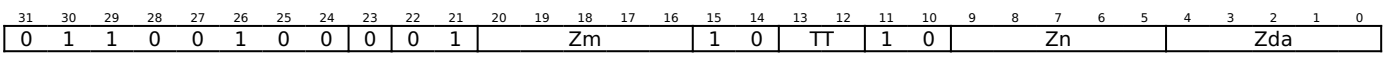
The encodings in this section are decoded from SVE encodings.



Decode fields op0	op1	Instruction details
0		SVE2 FP8 multiply-add long long
1	0	SVE2 FP8 multiply-add long
1	1	UNALLOCATED

SVE2 FP8 multiply-add long long

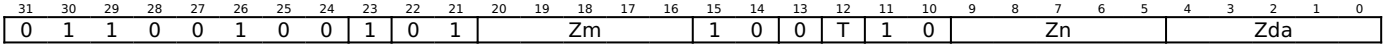
The encodings in this section are decoded from SVE2 FP8 widening multiply-add.



Decode fields TT	Instruction Details	Condition
00	FMLALLBB (vectors)	FEAT_SSVE_FP8FMA || (FEAT_SVE2 && FEAT_FP8FMA)
01	FMLALLBT (vectors)	FEAT_SSVE_FP8FMA || (FEAT_SVE2 && FEAT_FP8FMA)
10	FMLALLTB (vectors)	FEAT_SSVE_FP8FMA || (FEAT_SVE2 && FEAT_FP8FMA)
11	FMLALLTT (vectors)	FEAT_SSVE_FP8FMA || (FEAT_SVE2 && FEAT_FP8FMA)

SVE2 FP8 multiply-add long

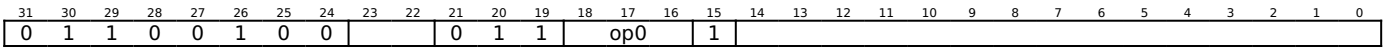
The encodings in this section are decoded from SVE2 FP8 widening multiply-add.



Decode fields T	Instruction Details	Condition
0	FMLALB (vectors, FP8 to FP16)	FEAT_SSVE_FP8FMA || (FEAT_SVE2 && FEAT_FP8FMA)
1	FMLALT (vectors, FP8 to FP16)	FEAT_SSVE_FP8FMA || (FEAT_SVE2 && FEAT_FP8FMA)

SVE2 floating-point unary operations - zeroing predicated

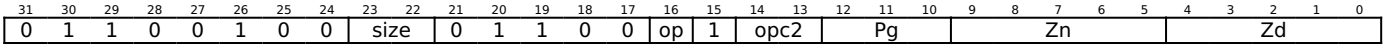
The encodings in this section are decoded from SVE encodings.



Decode fields op0	Instruction details
00x	Floating-point round to integral value (predicated)
010	Floating-point convert (predicated)
011	Floating-point square root (predicated)
10x	Floating-point round and convert from integer (predicated)
11x	Floating-point log and convert to integer

Floating-point round to integral value (predicated)

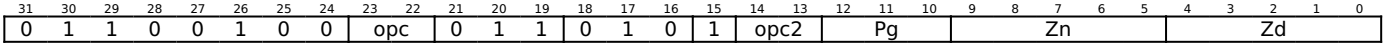
The encodings in this section are decoded from SVE2 floating-point unary operations - zeroing predicated.



Decode fields op	opc2	Instruction Details	Condition
0	00	FRINT<r> — Nearest with ties to even, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
0	01	FRINT<r> — Toward plus infinity, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
0	10	FRINT<r> — Toward minus infinity, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
0	11	FRINT<r> — Toward zero, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
1	00	FRINT<r> — Nearest with ties to away, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
1	01	UNALLOCATED	-
1	10	FRINT<r> — Current mode signaling inexact, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
1	11	FRINT<r> — Current mode, zeroing	FEAT_SVE2p2 || FEAT_SME2p2

Floating-point convert (predicated)

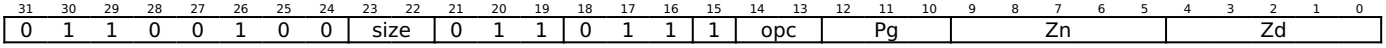
The encodings in this section are decoded from SVE2 floating-point unary operations - zeroing predicated.



Decode fields opc	opc2	Instruction Details	Condition
x0	11	UNALLOCATED	-
00	0x	UNALLOCATED	-
00	10	FCVTX	FEAT_SVE2p2 || FEAT_SME2p2
01		UNALLOCATED	-
10	00	FCVT — Single-precision to half-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
10	01	FCVT — Half-precision to single-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
10	10	BFCVT	FEAT_SVE2p2 || FEAT_SME2p2
11	00	FCVT — Double-precision to half-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
11	01	FCVT — Half-precision to double-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
11	10	FCVT — Double-precision to single-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
11	11	FCVT — Single-precision to double-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2

Floating-point square root (predicated)

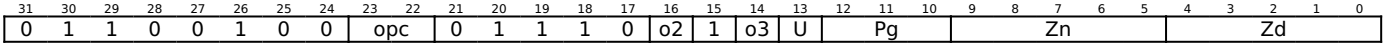
The encodings in this section are decoded from SVE2 floating-point unary operations - zeroing predicated.



Decode fields opc	Instruction Details	Condition
00	FRECPX	FEAT_SVE2p2 || FEAT_SME2p2
01	FSQRT	FEAT_SVE2p2 || FEAT_SME2p2
1x	UNALLOCATED	-

Floating-point round and convert from integer (predicated)

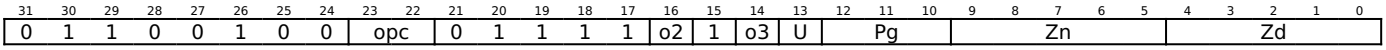
The encodings in this section are decoded from SVE2 floating-point unary operations - zeroing predicated.



Decode fields				Instruction Details	Condition
opc	o2	o3	U		
00	0		0	FRINT32Z	FEAT_SVE2p2 || FEAT_SME2p2
00	0		1	FRINT32X	FEAT_SVE2p2 || FEAT_SME2p2
00	1		0	FRINT64Z	FEAT_SVE2p2 || FEAT_SME2p2
00	1		1	FRINT64X	FEAT_SVE2p2 || FEAT_SME2p2
01	0	0		UNALLOCATED	-
01	0	1	0	SCVTF (predicated) — 16-bit to half-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
01	0	1	1	UCVTF (predicated) — 16-bit to half-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
01	1	0	0	SCVTF (predicated) — 32-bit to half-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
01	1	0	1	UCVTF (predicated) — 32-bit to half-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
01	1	1	0	SCVTF (predicated) — 64-bit to half-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
01	1	1	1	UCVTF (predicated) — 64-bit to half-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
10	0			UNALLOCATED	-
10	1	0	0	SCVTF (predicated) — 32-bit to single-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
10	1	0	1	UCVTF (predicated) — 32-bit to single-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
10	1	1		UNALLOCATED	-
11	0	0	0	SCVTF (predicated) — 32-bit to double-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
11	0	0	1	UCVTF (predicated) — 32-bit to double-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
11	0	1		UNALLOCATED	-
11	1	0	0	SCVTF (predicated) — 64-bit to single-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
11	1	0	1	UCVTF (predicated) — 64-bit to single-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
11	1	1	0	SCVTF (predicated) — 64-bit to double-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
11	1	1	1	UCVTF (predicated) — 64-bit to double-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2

Floating-point log and convert to integer

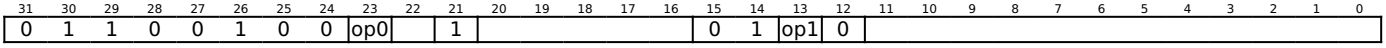
The encodings in this section are decoded from SVE2 floating-point unary operations - zeroing predicated.



Decode fields				Instruction Details	Condition
opc	o2	o3	U		
00	0			FLOGB	FEAT_SVE2p2 || FEAT_SME2p2
00	1			UNALLOCATED	-
01	0	0		UNALLOCATED	-
01	0	1	0	FCVTZS — Half-precision to 16-bit, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
01	0	1	1	FCVTZU — Half-precision to 16-bit, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
01	1	0	0	FCVTZS — Half-precision to 32-bit, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
01	1	0	1	FCVTZU — Half-precision to 32-bit, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
01	1	1	0	FCVTZS — Half-precision to 64-bit, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
01	1	1	1	FCVTZU — Half-precision to 64-bit, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
10	0			UNALLOCATED	-
10	1	0	0	FCVTZS — Single-precision to 32-bit, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
10	1	0	1	FCVTZU — Single-precision to 32-bit, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
10	1	1		UNALLOCATED	-
11	0	0	0	FCVTZS — Double-precision to 32-bit, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
11	0	0	1	FCVTZU — Double-precision to 32-bit, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
11	0	1		UNALLOCATED	-
11	1	0	0	FCVTZS — Single-precision to 64-bit, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
11	1	0	1	FCVTZU — Single-precision to 64-bit, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
11	1	1	0	FCVTZS — Double-precision to 64-bit, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
11	1	1	1	FCVTZU — Double-precision to 64-bit, zeroing	FEAT_SVE2p2 || FEAT_SME2p2

SVE floating-point widening multiply-add - indexed

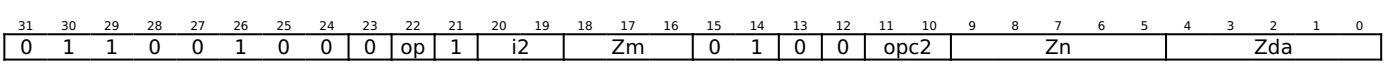
The encodings in this section are decoded from SVE encodings.



Decode fields		Instruction details
op0	op1	
0	0	SVE BFloat16 floating-point dot product (indexed)
0	1	UNALLOCATED
1		SVE floating-point multiply-add long (indexed)

SVE BFloat16 floating-point dot product (indexed)

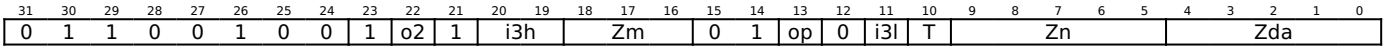
The encodings in this section are decoded from SVE floating-point widening multiply-add - indexed.



Decode fields		Instruction Details	Condition
op	opc2		
0	x1	FDOT (2-way, indexed, FP8 to FP16)	FEAT_SSVE_FP8DOT2 || (FEAT_SVE2 && FEAT_FP8DOT2)
0	00	FDOT (2-way, indexed, FP16 to FP32)	FEAT_SME2 || FEAT_SVE2p1
0	10	UNALLOCATED	-
1	00	BFDOT (indexed)	(FEAT_SVE && FEAT_BF16) || (FEAT_SME && FEAT_BF16)
1	01	FDOT (4-way, indexed)	FEAT_SSVE_FP8DOT4 || (FEAT_SVE2 && FEAT_FP8DOT4)
1	1x	UNALLOCATED	-

SVE floating-point multiply-add long (indexed)

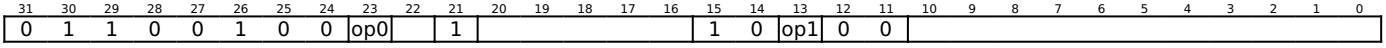
The encodings in this section are decoded from SVE floating-point widening multiply-add - indexed.



Decode fields			Instruction Details	Condition
o2	op	T		
0	0	0	FMLALB (indexed, FP16 to FP32)	FEAT_SVE2 || FEAT_SME
0	0	1	FMLALT (indexed, FP16 to FP32)	FEAT_SVE2 || FEAT_SME
0	1	0	FMLSLB (indexed)	FEAT_SVE2 || FEAT_SME
0	1	1	FMLSLT (indexed)	FEAT_SVE2 || FEAT_SME
1	0	0	BFMLALB (indexed)	(FEAT_SVE && FEAT_BF16) || (FEAT_SME && FEAT_BF16)
1	0	1	BFMLALT (indexed)	(FEAT_SVE && FEAT_BF16) || (FEAT_SME && FEAT_BF16)
1	1	0	BFMLSLB (indexed)	FEAT_SME2 || FEAT_SVE2p1
1	1	1	BFMLSLT (indexed)	FEAT_SME2 || FEAT_SVE2p1

SVE floating-point widening multiply-add

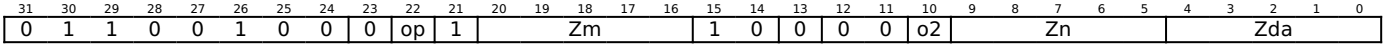
The encodings in this section are decoded from SVE encodings.



Decode fields		Instruction details
op0	op1	
0	0	SVE BFloat16 floating-point dot product
0	1	UNALLOCATED
1		SVE floating-point multiply-add long

SVE BFloat16 floating-point dot product

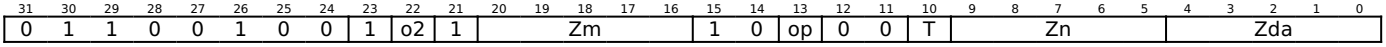
The encodings in this section are decoded from SVE floating-point widening multiply-add.



Decode fields		Instruction Details	Condition
op	o2		
0	0	FDOT (2-way, vectors, FP16 to FP32)	FEAT_SME2 || FEAT_SVE2p1
0	1	FDOT (2-way, vectors, FP8 to FP16)	FEAT_SSVE_FP8DOT2 || (FEAT_SVE2 && FEAT_FP8DOT2)
1	0	BFDOT (vectors)	(FEAT_SVE && FEAT_BF16) || (FEAT_SME && FEAT_BF16)
1	1	FDOT (4-way, vectors)	FEAT_SSVE_FP8DOT4 || (FEAT_SVE2 && FEAT_FP8DOT4)

SVE floating-point multiply-add long

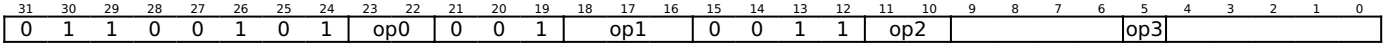
The encodings in this section are decoded from SVE floating-point widening multiply-add.



Decode fields			Instruction Details	Condition
o2	op	T		
0	0	0	FMLALB (vectors, FP16 to FP32)	FEAT_SVE2 || FEAT_SME
0	0	1	FMLALT (vectors, FP16 to FP32)	FEAT_SVE2 || FEAT_SME
0	1	0	FMLSLB (vectors)	FEAT_SVE2 || FEAT_SME
0	1	1	FMLSLT (vectors)	FEAT_SVE2 || FEAT_SME
1	0	0	BFMLALB (vectors)	(FEAT_SVE && FEAT_BF16) || (FEAT_SME && FEAT_BF16)
1	0	1	BFMLALT (vectors)	(FEAT_SVE && FEAT_BF16) || (FEAT_SME && FEAT_BF16)
1	1	0	BFMLSLB (vectors)	FEAT_SME2 || FEAT_SVE2p1
1	1	1	BFMLSLT (vectors)	FEAT_SME2 || FEAT_SVE2p1

SVE floating-point unary operations - unpredicated

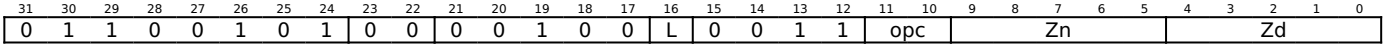
The encodings in this section are decoded from SVE encodings.



Decode fields				Instruction details
op0	op1	op2	op3	
00	00x			SVE2 FP8 upconverts
00	010		0	SVE2 FP8 downconverts
00	010		1	UNALLOCATED
00	011			UNALLOCATED
	100			SVE2 Int to FP upconverts
	101	0x	0	SVE2 FP to Int downconverts
	101	0x	1	UNALLOCATED
	101	1x		UNALLOCATED
	11x	00		SVE floating-point reciprocal estimate (unpredicated)
	11x	!= 00		UNALLOCATED
!= 00	0xx			UNALLOCATED

SVE2 FP8 upconverts

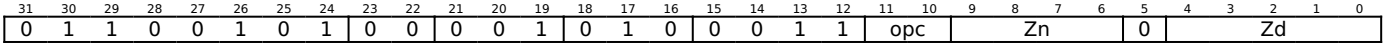
The encodings in this section are decoded from SVE floating-point unary operations - unpredicated.



Decode fields		Instruction Details	Condition
L	opc		
0	00	F1CVT, F2CVT — F1CVT	(FEAT_SVE2 && FEAT_FP8) || (FEAT_SME2 && FEAT_FP8)
0	01	F1CVT, F2CVT — F2CVT	(FEAT_SVE2 && FEAT_FP8) || (FEAT_SME2 && FEAT_FP8)
0	10	BF1CVT, BF2CVT — BF1CVT	(FEAT_SVE2 && FEAT_FP8) || (FEAT_SME2 && FEAT_FP8)
0	11	BF1CVT, BF2CVT — BF2CVT	(FEAT_SVE2 && FEAT_FP8) || (FEAT_SME2 && FEAT_FP8)
1	00	F1CVTLT, F2CVTLT — F1CVTLT	(FEAT_SVE2 && FEAT_FP8) || (FEAT_SME2 && FEAT_FP8)
1	01	F1CVTLT, F2CVTLT — F2CVTLT	(FEAT_SVE2 && FEAT_FP8) || (FEAT_SME2 && FEAT_FP8)
1	10	BF1CVTLT, BF2CVTLT — BF1CVTLT	(FEAT_SVE2 && FEAT_FP8) || (FEAT_SME2 && FEAT_FP8)
1	11	BF1CVTLT, BF2CVTLT — BF2CVTLT	(FEAT_SVE2 && FEAT_FP8) || (FEAT_SME2 && FEAT_FP8)

SVE2 FP8 downconverts

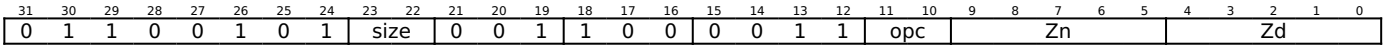
The encodings in this section are decoded from SVE floating-point unary operations - unpredicated.



Decode fields opc	Instruction Details	Condition
00	FCVTN	(FEAT_SVE2 && FEAT_FP8) || (FEAT_SME2 && FEAT_FP8)
01	FCVTNB	(FEAT_SVE2 && FEAT_FP8) || (FEAT_SME2 && FEAT_FP8)
10	BFCVTN	(FEAT_SVE2 && FEAT_FP8) || (FEAT_SME2 && FEAT_FP8)
11	FCVTNT (unpredicated)	(FEAT_SVE2 && FEAT_FP8) || (FEAT_SME2 && FEAT_FP8)

SVE2 Int to FP upconverts

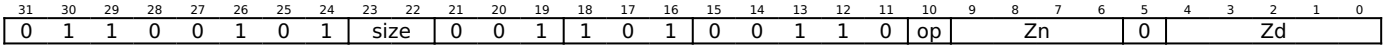
The encodings in this section are decoded from SVE floating-point unary operations - unpredicated.



Decode fields opc	Instruction Details	Condition
00	SCVTF (bottom, unpredicated)	FEAT_SVE2p3 || FEAT_SME2p3
01	UCVTF (bottom, unpredicated)	FEAT_SVE2p3 || FEAT_SME2p3
10	SCVTFLT	FEAT_SVE2p3 || FEAT_SME2p3
11	UCVTFLT	FEAT_SVE2p3 || FEAT_SME2p3

SVE2 FP to Int downconverts

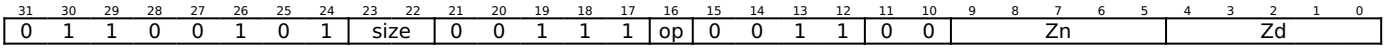
The encodings in this section are decoded from SVE floating-point unary operations - unpredicated.



Decode fields op	Instruction Details	Condition
0	FCVTZSN	FEAT_SVE2p3 || FEAT_SME2p3
1	FCVTZUN	FEAT_SVE2p3 || FEAT_SME2p3

SVE floating-point reciprocal estimate (unpredicated)

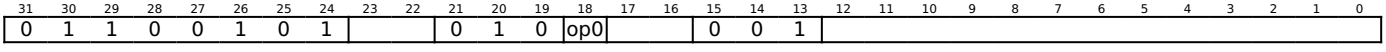
The encodings in this section are decoded from SVE floating-point unary operations - unpredicated.



Decode fields op	Instruction Details	Condition
0	FRECPE	FEAT_SVE || FEAT_SME
1	FRSQRTE	FEAT_SVE || FEAT_SME

SVE floating-point compare - with zero

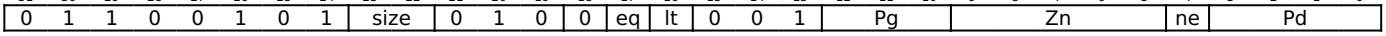
The encodings in this section are decoded from SVE encodings.



Decode fields op0	Instruction details
0	SVE floating-point compare with zero
1	UNALLOCATED

SVE floating-point compare with zero

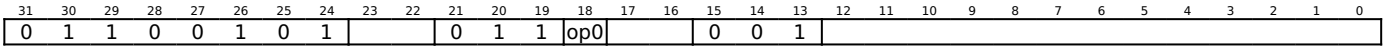
The encodings in this section are decoded from SVE floating-point compare - with zero.



Decode fields			Instruction Details	Condition
eq	lt	ne		
0	0	0	FCM<cc> (zero) — Greater than or equal	FEAT_SVE || FEAT_SME
0	0	1	FCM<cc> (zero) — Greater than	FEAT_SVE || FEAT_SME
0	1	0	FCM<cc> (zero) — Less than	FEAT_SVE || FEAT_SME
0	1	1	FCM<cc> (zero) — Less than or equal	FEAT_SVE || FEAT_SME
1		1	UNALLOCATED	-
1	0	0	FCM<cc> (zero) — Equal	FEAT_SVE || FEAT_SME
1	1	0	FCM<cc> (zero) — Not equal	FEAT_SVE || FEAT_SME

SVE floating-point accumulating reduction

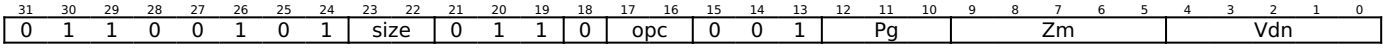
The encodings in this section are decoded from SVE encodings.



Decode fields	Instruction details
op0	
0	SVE floating-point serial reduction (predicated)
1	UNALLOCATED

SVE floating-point serial reduction (predicated)

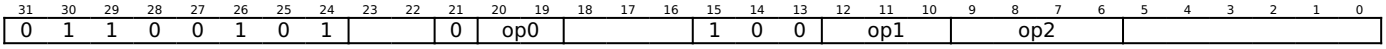
The encodings in this section are decoded from SVE floating-point accumulating reduction.



Decode fields	Instruction Details	Condition
opc		
00	FADDA	FEAT_SVE
!= 00	UNALLOCATED	-

SVE floating-point arithmetic - predicated

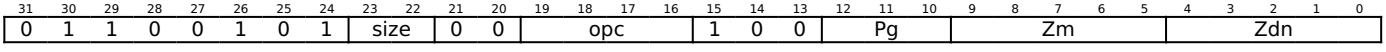
The encodings in this section are decoded from SVE encodings.



Decode fields			Instruction details	Condition
op0	op1	op2		
0x			SVE floating-point arithmetic (predicated)	-
10	000		FTMAD	FEAT_SVE
10	!= 000		UNALLOCATED	-
11		0000	SVE floating-point arithmetic with immediate (predicated)	-
11		!= 0000	UNALLOCATED	-

SVE floating-point arithmetic (predicated)

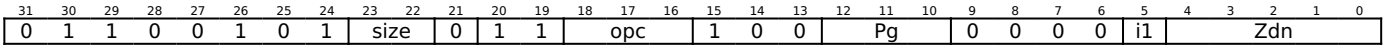
The encodings in this section are decoded from SVE floating-point arithmetic - predicated.



Decode fields size	Decode fields opc	Instruction Details	Condition
	0011	FSUBR (vectors)	FEAT_SVE || FEAT_SME
	1000	FABD	FEAT_SVE || FEAT_SME
	1010	FMULX	FEAT_SVE || FEAT_SME
	1011	UNALLOCATED	-
	1100	FDIVR	FEAT_SVE || FEAT_SME
	1101	FDIV	FEAT_SVE || FEAT_SME
	1110	FAMAX	(FEAT_SVE2 && FEAT_FAMINMAX) || (FEAT_SME2 && FEAT_FAMINMAX)
	1111	FAMIN	(FEAT_SVE2 && FEAT_FAMINMAX) || (FEAT_SME2 && FEAT_FAMINMAX)
00	0000	BFADD (predicated)	FEAT_SVE_B16B16
00	0001	BFSUB (predicated)	FEAT_SVE_B16B16
00	0010	BFMUL (vectors, predicated)	FEAT_SVE_B16B16
00	0100	BFMAXNM	FEAT_SVE_B16B16
00	0101	BFMINNM	FEAT_SVE_B16B16
00	0110	BFMAX	FEAT_SVE_B16B16
00	0111	BFMIN	FEAT_SVE_B16B16
00	1001	BFSCALE	FEAT_SVE_BFSCALE
!= 00	0000	FADD (vectors, predicated)	FEAT_SVE || FEAT_SME
!= 00	0001	FSUB (vectors, predicated)	FEAT_SVE || FEAT_SME
!= 00	0010	FMUL (vectors, predicated)	FEAT_SVE || FEAT_SME
!= 00	0100	FMAXNM (vectors)	FEAT_SVE || FEAT_SME
!= 00	0101	FMINNM (vectors)	FEAT_SVE || FEAT_SME
!= 00	0110	FMAX (vectors)	FEAT_SVE || FEAT_SME
!= 00	0111	FMIN (vectors)	FEAT_SVE || FEAT_SME
!= 00	1001	FSCALE	FEAT_SVE || FEAT_SME

SVE floating-point arithmetic with immediate (predicated)

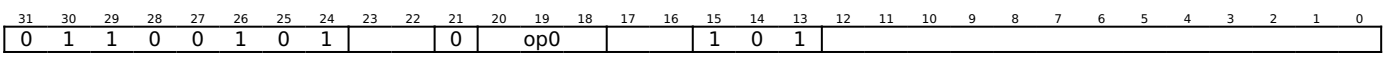
The encodings in this section are decoded from SVE floating-point arithmetic - predicated.



Decode fields opc	Instruction Details	Condition
000	FADD (immediate)	FEAT_SVE || FEAT_SME
001	FSUB (immediate)	FEAT_SVE || FEAT_SME
010	FMUL (immediate)	FEAT_SVE || FEAT_SME
011	FSUBR (immediate)	FEAT_SVE || FEAT_SME
100	FMAXNM (immediate)	FEAT_SVE || FEAT_SME
101	FMINNM (immediate)	FEAT_SVE || FEAT_SME
110	FMAX (immediate)	FEAT_SVE || FEAT_SME
111	FMIN (immediate)	FEAT_SVE || FEAT_SME

SVE floating-point unary operations - merging predicated

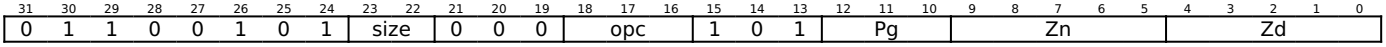
The encodings in this section are decoded from SVE encodings.



Decode fields op0	Instruction details
00x	SVE floating-point round to integral value
010	SVE floating-point convert precision
011	SVE floating-point unary operations
10x	SVE integer convert to floating-point
11x	SVE floating-point convert to integer

SVE floating-point round to integral value

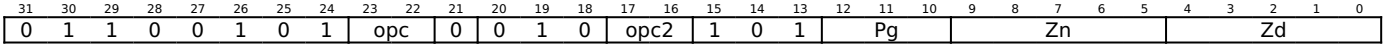
The encodings in this section are decoded from SVE floating-point unary operations - merging predicated.



Decode fields opc	Instruction Details	Condition
000	FRINT<r> — Nearest with ties to even, merging	FEAT_SVE || FEAT_SME
001	FRINT<r> — Toward plus infinity, merging	FEAT_SVE || FEAT_SME
010	FRINT<r> — Toward minus infinity, merging	FEAT_SVE || FEAT_SME
011	FRINT<r> — Toward zero, merging	FEAT_SVE || FEAT_SME
100	FRINT<r> — Nearest with ties to away, merging	FEAT_SVE || FEAT_SME
101	UNALLOCATED	-
110	FRINT<r> — Current mode signaling inexact, merging	FEAT_SVE || FEAT_SME
111	FRINT<r> — Current mode, merging	FEAT_SVE || FEAT_SME

SVE floating-point convert precision

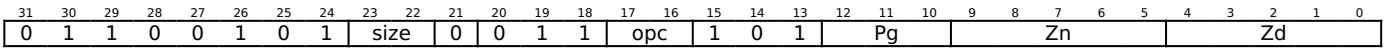
The encodings in this section are decoded from SVE floating-point unary operations - merging predicated.



Decode fields opc	Decode fields opc2	Instruction Details	Condition
x0	11	UNALLOCATED	-
00	0x	UNALLOCATED	-
00	10	FCVTX	FEAT_SVE2 || FEAT_SME
01		UNALLOCATED	-
10	00	FCVT — Single-precision to half-precision, merging	FEAT_SVE || FEAT_SME
10	01	FCVT — Half-precision to single-precision, merging	FEAT_SVE || FEAT_SME
10	10	BFCVT	(FEAT_SVE && FEAT_BF16) || (FEAT_SME && FEAT_BF16)
11	00	FCVT — Double-precision to half-precision, merging	FEAT_SVE || FEAT_SME
11	01	FCVT — Half-precision to double-precision, merging	FEAT_SVE || FEAT_SME
11	10	FCVT — Double-precision to single-precision, merging	FEAT_SVE || FEAT_SME
11	11	FCVT — Single-precision to double-precision, merging	FEAT_SVE || FEAT_SME

SVE floating-point unary operations

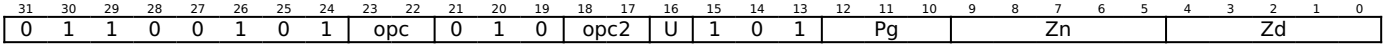
The encodings in this section are decoded from SVE floating-point unary operations - merging predicated.



Decode fields opc	Instruction Details	Condition
00	FRECPX	FEAT_SVE || FEAT_SME
01	FSQRT	FEAT_SVE || FEAT_SME
1x	UNALLOCATED	-

SVE integer convert to floating-point

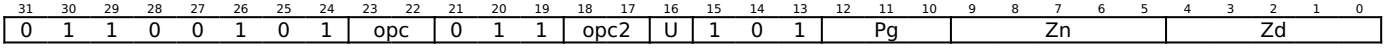
The encodings in this section are decoded from SVE floating-point unary operations - merging predicated.



Decode fields opc	opc2	U	Instruction Details	Condition
00	0x	0	FRINT32Z	FEAT_SVE2p2 || FEAT_SME2p2
00	0x	1	FRINT32X	FEAT_SVE2p2 || FEAT_SME2p2
00	1x	0	FRINT64Z	FEAT_SVE2p2 || FEAT_SME2p2
00	1x	1	FRINT64X	FEAT_SVE2p2 || FEAT_SME2p2
01	00		UNALLOCATED	-
01	01	0	SCVTF (predicated) — 16-bit to half-precision, merging	FEAT_SVE || FEAT_SME
01	01	1	UCVTF (predicated) — 16-bit to half-precision, merging	FEAT_SVE || FEAT_SME
01	10	0	SCVTF (predicated) — 32-bit to half-precision, merging	FEAT_SVE || FEAT_SME
01	10	1	UCVTF (predicated) — 32-bit to half-precision, merging	FEAT_SVE || FEAT_SME
01	11	0	SCVTF (predicated) — 64-bit to half-precision, merging	FEAT_SVE || FEAT_SME
01	11	1	UCVTF (predicated) — 64-bit to half-precision, merging	FEAT_SVE || FEAT_SME
10	10	0	SCVTF (predicated) — 32-bit to single-precision, merging	FEAT_SVE || FEAT_SME
10	10	1	UCVTF (predicated) — 32-bit to single-precision, merging	FEAT_SVE || FEAT_SME
10	!= 10		UNALLOCATED	-
11	00	0	SCVTF (predicated) — 32-bit to double-precision, merging	FEAT_SVE || FEAT_SME
11	00	1	UCVTF (predicated) — 32-bit to double-precision, merging	FEAT_SVE || FEAT_SME
11	01		UNALLOCATED	-
11	10	0	SCVTF (predicated) — 64-bit to single-precision, merging	FEAT_SVE || FEAT_SME
11	10	1	UCVTF (predicated) — 64-bit to single-precision, merging	FEAT_SVE || FEAT_SME
11	11	0	SCVTF (predicated) — 64-bit to double-precision, merging	FEAT_SVE || FEAT_SME
11	11	1	UCVTF (predicated) — 64-bit to double-precision, merging	FEAT_SVE || FEAT_SME

SVE floating-point convert to integer

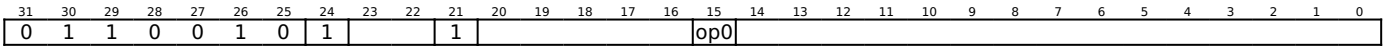
The encodings in this section are decoded from SVE floating-point unary operations - merging predicated.



Decode fields			Instruction Details	Condition
opc	opc2	U		
00		0	FLOGB	FEAT_SVE2 || FEAT_SME
00		1	UNALLOCATED	-
01	00		UNALLOCATED	-
01	01	0	FCVTZS — Half-precision to 16-bit, merging	FEAT_SVE || FEAT_SME
01	01	1	FCVTZU — Half-precision to 16-bit, merging	FEAT_SVE || FEAT_SME
01	10	0	FCVTZS — Half-precision to 32-bit, merging	FEAT_SVE || FEAT_SME
01	10	1	FCVTZU — Half-precision to 32-bit, merging	FEAT_SVE || FEAT_SME
01	11	0	FCVTZS — Half-precision to 64-bit, merging	FEAT_SVE || FEAT_SME
01	11	1	FCVTZU — Half-precision to 64-bit, merging	FEAT_SVE || FEAT_SME
10	10	0	FCVTZS — Single-precision to 32-bit, merging	FEAT_SVE || FEAT_SME
10	10	1	FCVTZU — Single-precision to 32-bit, merging	FEAT_SVE || FEAT_SME
10	!= 10		UNALLOCATED	-
11	00	0	FCVTZS — Double-precision to 32-bit, merging	FEAT_SVE || FEAT_SME
11	00	1	FCVTZU — Double-precision to 32-bit, merging	FEAT_SVE || FEAT_SME
11	01		UNALLOCATED	-
11	10	0	FCVTZS — Single-precision to 64-bit, merging	FEAT_SVE || FEAT_SME
11	10	1	FCVTZU — Single-precision to 64-bit, merging	FEAT_SVE || FEAT_SME
11	11	0	FCVTZS — Double-precision to 64-bit, merging	FEAT_SVE || FEAT_SME
11	11	1	FCVTZU — Double-precision to 64-bit, merging	FEAT_SVE || FEAT_SME

SVE floating-point multiply-add

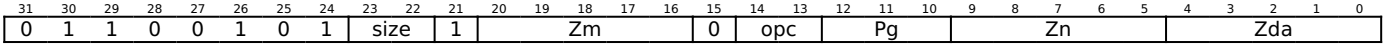
The encodings in this section are decoded from SVE encodings.



Decode fields	Instruction details
op0	
0	SVE floating-point multiply-accumulate writing addend
1	SVE floating-point multiply-accumulate writing multiplicand

SVE floating-point multiply-accumulate writing addend

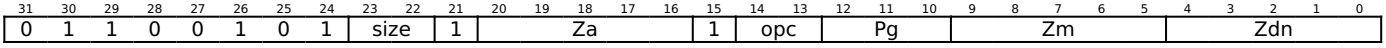
The encodings in this section are decoded from SVE floating-point multiply-add.



Decode fields		Instruction Details	Condition
size	opc		
00	00	BFMLA (vectors)	FEAT_SVE_B16B16
00	01	BFMLS (vectors)	FEAT_SVE_B16B16
00	1x	UNALLOCATED	-
!= 00	00	FMLA (vectors)	FEAT_SVE || FEAT_SME
!= 00	01	FMLS (vectors)	FEAT_SVE || FEAT_SME
!= 00	10	FNMLA	FEAT_SVE || FEAT_SME
!= 00	11	FNMLS	FEAT_SVE || FEAT_SME

SVE floating-point multiply-accumulate writing multiplicand

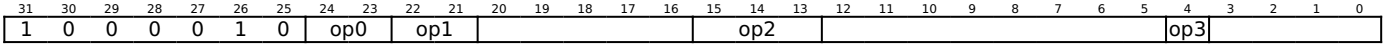
The encodings in this section are decoded from SVE floating-point multiply-add.



Decode fields opc	Instruction Details	Condition
00	FMAD	FEAT_SVE || FEAT_SME
01	FMSB	FEAT_SVE || FEAT_SME
10	FNMAD	FEAT_SVE || FEAT_SME
11	FNMSB	FEAT_SVE || FEAT_SME

SVE Memory - 32-bit Gather and Unsized Contiguous

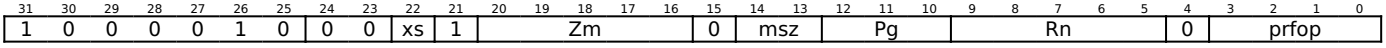
The encodings in this section are decoded from SVE encodings.



Decode fields op0	op1	op2	op3	Instruction details	Condition
00	x1	0xx	0	SVE 32-bit gather prefetch (scalar plus 32-bit scaled offsets)	-
00	x1	0xx	1	UNALLOCATED	-
01	x1	0xx		SVE 32-bit gather load halfwords (scalar plus 32-bit scaled offsets)	-
10	x1	0xx		SVE 32-bit gather load words (scalar plus 32-bit scaled offsets)	-
11	00	001	0	UNALLOCATED	-
11	00	011		LDAPRO	FEAT_SVE_LRCPC
11	01	011	1	UNALLOCATED	-
11	01	0x1	0	UNALLOCATED	-
11	0x	000	0	LDR (predicate)	FEAT_SVE || FEAT_SME
11	0x	00x~~000~~	1	UNALLOCATED	-
11	0x	010		LDR (vector)	FEAT_SVE || FEAT_SME
~~11~~	~~0x~~	~~0x1~~		~~UNALLOCATED~~	-
11	1x	0xx	0	SVE contiguous prefetch (scalar plus immediate)	-
11	1x	0xx	1	UNALLOCATED	-
	00	10x		SVE2 32-bit gather non-temporal load (vector plus scalar)	-
	00	110	0	SVE contiguous prefetch (scalar plus scalar)	-
	00	111	0	SVE 32-bit gather prefetch (vector plus immediate)	-
	00	11x	1	UNALLOCATED	-
	01	1xx		SVE 32-bit gather load (vector plus immediate)	-
	1x	1xx		SVE load and broadcast element	-
!= 11	x0	0xx		SVE 32-bit gather load (scalar plus 32-bit unscaled offsets)	-

SVE 32-bit gather prefetch (scalar plus 32-bit scaled offsets)

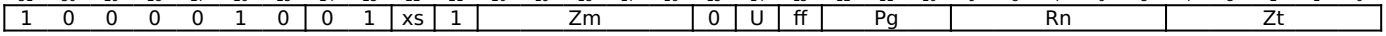
The encodings in this section are decoded from SVE Memory - 32-bit Gather and Unsized Contiguous.



Decode fields msz	Instruction Details	Condition
00	PRFB (scalar plus vector)	FEAT_SVE
01	PRFH (scalar plus vector)	FEAT_SVE
10	PRFW (scalar plus vector)	FEAT_SVE
11	PRFD (scalar plus vector)	FEAT_SVE

SVE 32-bit gather load halfwords (scalar plus 32-bit scaled offsets)

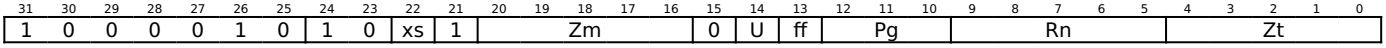
The encodings in this section are decoded from SVE Memory - 32-bit Gather and Unsized Contiguous.



Decode fields U	ff	Instruction Details	Condition
0	0	LD1SH (scalar plus vector)	FEAT_SVE
0	1	LDFF1SH (scalar plus vector)	FEAT_SVE
1	0	LD1H (scalar plus vector)	FEAT_SVE
1	1	LDFF1H (scalar plus vector)	FEAT_SVE

SVE 32-bit gather load words (scalar plus 32-bit scaled offsets)

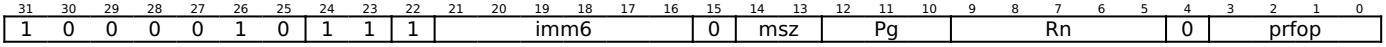
The encodings in this section are decoded from SVE Memory - 32-bit Gather and Unsized Contiguous.



Decode fields U	ff	Instruction Details	Condition
0		UNALLOCATED	-
1	0	LD1W (scalar plus vector)	FEAT_SVE
1	1	LDFF1W (scalar plus vector)	FEAT_SVE

SVE contiguous prefetch (scalar plus immediate)

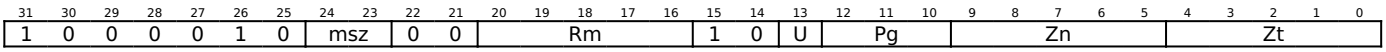
The encodings in this section are decoded from SVE Memory - 32-bit Gather and Unsized Contiguous.



Decode fields msz	Instruction Details	Condition
00	PRFB (scalar plus immediate)	FEAT_SVE || FEAT_SME
01	PRFH (scalar plus immediate)	FEAT_SVE || FEAT_SME
10	PRFW (scalar plus immediate)	FEAT_SVE || FEAT_SME
11	PRFD (scalar plus immediate)	FEAT_SVE || FEAT_SME

SVE2 32-bit gather non-temporal load (vector plus scalar)

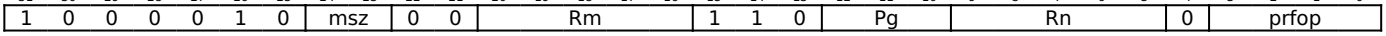
The encodings in this section are decoded from SVE Memory - 32-bit Gather and Unsized Contiguous.



Decode fields msz	U	Instruction Details	Condition
00	0	LDNT1SB	FEAT_SVE2
00	1	LDNT1B (vector plus scalar)	FEAT_SVE2
01	0	LDNT1SH	FEAT_SVE2
01	1	LDNT1H (vector plus scalar)	FEAT_SVE2
10	0	UNALLOCATED	-
10	1	LDNT1W (vector plus scalar)	FEAT_SVE2
11		UNALLOCATED	-

SVE contiguous prefetch (scalar plus scalar)

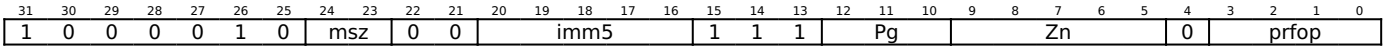
The encodings in this section are decoded from SVE Memory - 32-bit Gather and Unsized Contiguous.



Decode fields		Instruction Details	Condition
msz	Rm		
	11111	UNALLOCATED	-
00	!= 11111	PRFB (scalar plus scalar)	FEAT_SVE || FEAT_SME
01	!= 11111	PRFH (scalar plus scalar)	FEAT_SVE || FEAT_SME
10	!= 11111	PRFW (scalar plus scalar)	FEAT_SVE || FEAT_SME
11	!= 11111	PRFD (scalar plus scalar)	FEAT_SVE || FEAT_SME

SVE 32-bit gather prefetch (vector plus immediate)

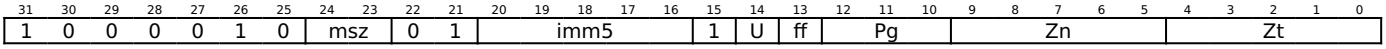
The encodings in this section are decoded from SVE Memory - 32-bit Gather and Unsized Contiguous.



Decode fields	Instruction Details	Condition
msz		
00	PRFB (vector plus immediate)	FEAT_SVE
01	PRFH (vector plus immediate)	FEAT_SVE
10	PRFW (vector plus immediate)	FEAT_SVE
11	PRFD (vector plus immediate)	FEAT_SVE

SVE 32-bit gather load (vector plus immediate)

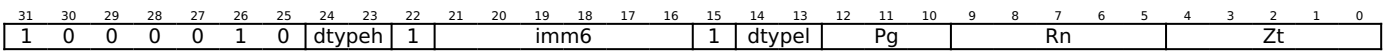
The encodings in this section are decoded from SVE Memory - 32-bit Gather and Unsized Contiguous.



Decode fields			Instruction Details	Condition
msz	U	ff		
00	0	0	LD1SB (vector plus immediate)	FEAT_SVE
00	0	1	LDFF1SB (vector plus immediate)	FEAT_SVE
00	1	0	LD1B (vector plus immediate)	FEAT_SVE
00	1	1	LDFF1B (vector plus immediate)	FEAT_SVE
01	0	0	LD1SH (vector plus immediate)	FEAT_SVE
01	0	1	LDFF1SH (vector plus immediate)	FEAT_SVE
01	1	0	LD1H (vector plus immediate)	FEAT_SVE
01	1	1	LDFF1H (vector plus immediate)	FEAT_SVE
10	0		UNALLOCATED	-
10	1	0	LD1W (vector plus immediate)	FEAT_SVE
10	1	1	LDFF1W (vector plus immediate)	FEAT_SVE
11			UNALLOCATED	-

SVE load and broadcast element

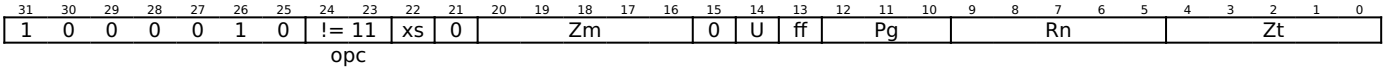
The encodings in this section are decoded from SVE Memory - 32-bit Gather and Unsized Contiguous.



Decode fields		Instruction Details	Condition
dtypeh	dtypel		
00	00	LD1RB — 8-bit element	FEAT_SVE || FEAT_SME
00	01	LD1RB — 16-bit element	FEAT_SVE || FEAT_SME
00	10	LD1RB — 32-bit element	FEAT_SVE || FEAT_SME
00	11	LD1RB — 64-bit element	FEAT_SVE || FEAT_SME
01	00	LD1RSW	FEAT_SVE || FEAT_SME
01	01	LD1RH — 16-bit element	FEAT_SVE || FEAT_SME
01	10	LD1RH — 32-bit element	FEAT_SVE || FEAT_SME
01	11	LD1RH — 64-bit element	FEAT_SVE || FEAT_SME
10	00	LD1RSH — 64-bit element	FEAT_SVE || FEAT_SME
10	01	LD1RSH — 32-bit element	FEAT_SVE || FEAT_SME
10	10	LD1RW — 32-bit element	FEAT_SVE || FEAT_SME
10	11	LD1RW — 64-bit element	FEAT_SVE || FEAT_SME
11	00	LD1RSB — 64-bit element	FEAT_SVE || FEAT_SME
11	01	LD1RSB — 32-bit element	FEAT_SVE || FEAT_SME
11	10	LD1RSB — 16-bit element	FEAT_SVE || FEAT_SME
11	11	LD1RD	FEAT_SVE || FEAT_SME

SVE 32-bit gather load (scalar plus 32-bit unscaled offsets)

The encodings in this section are decoded from SVE Memory - 32-bit Gather and Unsized Contiguous.

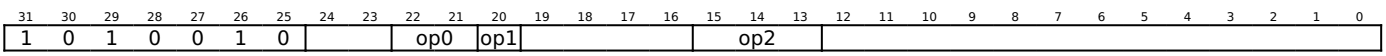


The following constraints also apply to this encoding: opc != '11'

Decode fields			Instruction Details	Condition
opc	U	ff		
00	0	0	LD1SB (scalar plus vector)	FEAT_SVE
00	0	1	LDFF1SB (scalar plus vector)	FEAT_SVE
00	1	0	LD1B (scalar plus vector)	FEAT_SVE
00	1	1	LDFF1B (scalar plus vector)	FEAT_SVE
01	0	0	LD1SH (scalar plus vector)	FEAT_SVE
01	0	1	LDFF1SH (scalar plus vector)	FEAT_SVE
01	1	0	LD1H (scalar plus vector)	FEAT_SVE
01	1	1	LDFF1H (scalar plus vector)	FEAT_SVE
10	0		UNALLOCATED	-
10	1	0	LD1W (scalar plus vector)	FEAT_SVE
10	1	1	LDFF1W (scalar plus vector)	FEAT_SVE

SVE Memory - Contiguous Load

The encodings in this section are decoded from SVE encodings.



Decode fields			Instruction details
op0	op1	op2	
00	0	111	SVE contiguous non-temporal load (scalar plus immediate)
00	1	001	SVE contiguous load (quadwords, scalar plus immediate)
00	1	111	SVE load multiple structures (quadwords, scalar plus immediate)
00		100	SVE contiguous load (quadwords, scalar plus scalar)
00		110	SVE contiguous non-temporal load (scalar plus scalar)
01		100	SVE load multiple structures (quadwords, scalar plus scalar)
1x		100	UNALLOCATED
	0	001	SVE load and broadcast quadword (scalar plus immediate)
	0	101	SVE contiguous load (scalar plus immediate)
	1	101	SVE contiguous non-fault load (scalar plus immediate)
		000	SVE load and broadcast quadword (scalar plus scalar)
		010	SVE contiguous load (scalar plus scalar)
		011	SVE contiguous first-fault load (scalar plus scalar)
!= 00	0	111	SVE load multiple structures (scalar plus immediate)
!= 00	1	001	UNALLOCATED
!= 00	1	111	UNALLOCATED
!= 00		110	SVE load multiple structures (scalar plus scalar)

SVE contiguous non-temporal load (scalar plus immediate)

The encodings in this section are decoded from SVE Memory - Contiguous Load.

31	30	29	28	27	26	25	24–23	22	21	20	19–16	15	14	13	12–10	9–5	4–0
1	0	1	0	0	1	0	msz	0	0	0	imm4	1	1	1	Pg	Rn	Zt

Decode fields	Instruction Details	Condition
msz		
00	LDNT1B (scalar plus immediate, single register)	FEAT_SVE || FEAT_SME
01	LDNT1H (scalar plus immediate, single register)	FEAT_SVE || FEAT_SME
10	LDNT1W (scalar plus immediate, single register)	FEAT_SVE || FEAT_SME
11	LDNT1D (scalar plus immediate, single register)	FEAT_SVE || FEAT_SME

SVE contiguous load (quadwords, scalar plus immediate)

The encodings in this section are decoded from SVE Memory - Contiguous Load.

31	30	29	28	27	26	25	24–23	22	21	20	19–16	15	14	13	12–10	9–5	4–0
1	0	1	0	0	1	0	dtype	0	0	1	imm4	0	0	1	Pg	Rn	Zt

Decode fields	Instruction Details	Condition
dtype		
0x	UNALLOCATED	-
10	LD1W (scalar plus immediate, single register)	FEAT_SVE2p1
11	LD1D (scalar plus immediate, single register)	FEAT_SVE2p1

SVE load multiple structures (quadwords, scalar plus immediate)

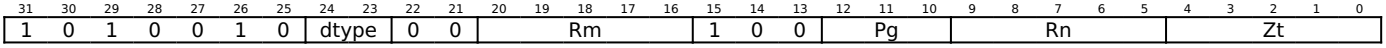
The encodings in this section are decoded from SVE Memory - Contiguous Load.

31	30	29	28	27	26	25	24–23	22	21	20	19–16	15	14	13	12–10	9–5	4–0
1	0	1	0	0	1	0	num	0	0	1	imm4	1	1	1	Pg	Rn	Zt

Decode fields num	Instruction Details	Condition
00	UNALLOCATED	-
01	LD2Q (scalar plus immediate)	FEAT_SVE2p1 || FEAT_SME2p1
10	LD3Q (scalar plus immediate)	FEAT_SVE2p1 || FEAT_SME2p1
11	LD4Q (scalar plus immediate)	FEAT_SVE2p1 || FEAT_SME2p1

SVE contiguous load (quadwords, scalar plus scalar)

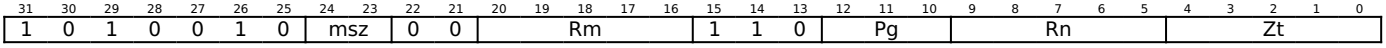
The encodings in this section are decoded from SVE Memory - Contiguous Load.



dtype	Decode fields Rm	Instruction Details	Condition
0x		UNALLOCATED	-
1x	11111	UNALLOCATED	-
10	!= 11111	LD1W (scalar plus scalar, single register)	FEAT_SVE2p1
11	!= 11111	LD1D (scalar plus scalar, single register)	FEAT_SVE2p1

SVE contiguous non-temporal load (scalar plus scalar)

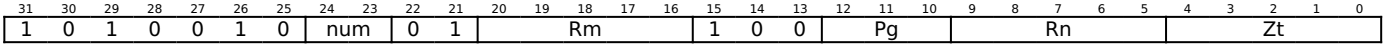
The encodings in this section are decoded from SVE Memory - Contiguous Load.



msz	Decode fields Rm	Instruction Details	Condition
	11111	UNALLOCATED	-
00	!= 11111	LDNT1B (scalar plus scalar, single register)	FEAT_SVE || FEAT_SME
01	!= 11111	LDNT1H (scalar plus scalar, single register)	FEAT_SVE || FEAT_SME
10	!= 11111	LDNT1W (scalar plus scalar, single register)	FEAT_SVE || FEAT_SME
11	!= 11111	LDNT1D (scalar plus scalar, single register)	FEAT_SVE || FEAT_SME

SVE load multiple structures (quadwords, scalar plus scalar)

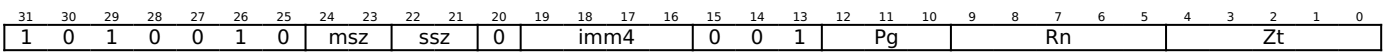
The encodings in this section are decoded from SVE Memory - Contiguous Load.



num	Decode fields Rm	Instruction Details	Condition
00		UNALLOCATED	-
01	!= 11111	LD2Q (scalar plus scalar)	FEAT_SVE2p1 || FEAT_SME2p1
10	!= 11111	LD3Q (scalar plus scalar)	FEAT_SVE2p1 || FEAT_SME2p1
11	!= 11111	LD4Q (scalar plus scalar)	FEAT_SVE2p1 || FEAT_SME2p1
!= 00	11111	UNALLOCATED	-

SVE load and broadcast quadword (scalar plus immediate)

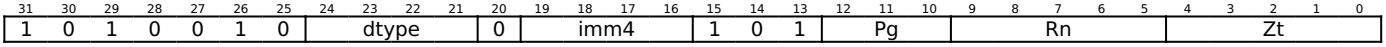
The encodings in this section are decoded from SVE Memory - Contiguous Load.



Decode fields msz	ssz	Instruction Details	Condition
	1x	UNALLOCATED	-
00	00	LD1RQB (scalar plus immediate)	FEAT_SVE || FEAT_SME
00	01	LD1ROB (scalar plus immediate)	FEAT_F64MM
01	00	LD1RQH (scalar plus immediate)	FEAT_SVE || FEAT_SME
01	01	LD1ROH (scalar plus immediate)	FEAT_F64MM
10	00	LD1RQW (scalar plus immediate)	FEAT_SVE || FEAT_SME
10	01	LD1ROW (scalar plus immediate)	FEAT_F64MM
11	00	LD1RQD (scalar plus immediate)	FEAT_SVE || FEAT_SME
11	01	LD1ROD (scalar plus immediate)	FEAT_F64MM

SVE contiguous load (scalar plus immediate)

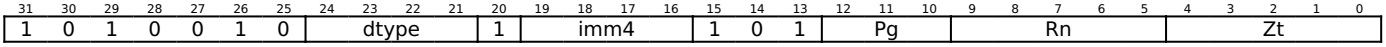
The encodings in this section are decoded from SVE Memory - Contiguous Load.



Decode fields dtype	Instruction Details	Condition
0000	LD1B (scalar plus immediate, single register) — 8-bit element	FEAT_SVE || FEAT_SME
0001	LD1B (scalar plus immediate, single register) — 16-bit element	FEAT_SVE || FEAT_SME
0010	LD1B (scalar plus immediate, single register) — 32-bit element	FEAT_SVE || FEAT_SME
0011	LD1B (scalar plus immediate, single register) — 64-bit element	FEAT_SVE || FEAT_SME
0100	LD1SW (scalar plus immediate)	FEAT_SVE || FEAT_SME
0101	LD1H (scalar plus immediate, single register) — 16-bit element	FEAT_SVE || FEAT_SME
0110	LD1H (scalar plus immediate, single register) — 32-bit element	FEAT_SVE || FEAT_SME
0111	LD1H (scalar plus immediate, single register) — 64-bit element	FEAT_SVE || FEAT_SME
1000	LD1SH (scalar plus immediate) — 64-bit element	FEAT_SVE || FEAT_SME
1001	LD1SH (scalar plus immediate) — 32-bit element	FEAT_SVE || FEAT_SME
1010	LD1W (scalar plus immediate, single register) — 32-bit element	FEAT_SVE || FEAT_SME
1011	LD1W (scalar plus immediate, single register) — 64-bit element	FEAT_SVE || FEAT_SME
1100	LD1SB (scalar plus immediate) — 64-bit element	FEAT_SVE || FEAT_SME
1101	LD1SB (scalar plus immediate) — 32-bit element	FEAT_SVE || FEAT_SME
1110	LD1SB (scalar plus immediate) — 16-bit element	FEAT_SVE || FEAT_SME
1111	LD1D (scalar plus immediate, single register)	FEAT_SVE || FEAT_SME

SVE contiguous non-fault load (scalar plus immediate)

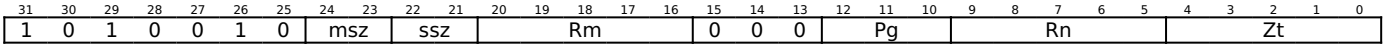
The encodings in this section are decoded from SVE Memory - Contiguous Load.



Decode fields dtype	Instruction Details	Condition
0000	LDNF1B — 8-bit element	FEAT_SVE
0001	LDNF1B — 16-bit element	FEAT_SVE
0010	LDNF1B — 32-bit element	FEAT_SVE
0011	LDNF1B — 64-bit element	FEAT_SVE
0100	LDNF1SW	FEAT_SVE
0101	LDNF1H — 16-bit element	FEAT_SVE
0110	LDNF1H — 32-bit element	FEAT_SVE
0111	LDNF1H — 64-bit element	FEAT_SVE
1000	LDNF1SH — 64-bit element	FEAT_SVE
1001	LDNF1SH — 32-bit element	FEAT_SVE
1010	LDNF1W — 32-bit element	FEAT_SVE
1011	LDNF1W — 64-bit element	FEAT_SVE
1100	LDNF1SB — 64-bit element	FEAT_SVE
1101	LDNF1SB — 32-bit element	FEAT_SVE
1110	LDNF1SB — 16-bit element	FEAT_SVE
1111	LDNF1D	FEAT_SVE

SVE load and broadcast quadword (scalar plus scalar)

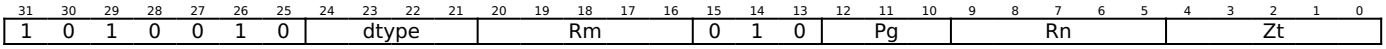
The encodings in this section are decoded from SVE Memory - Contiguous Load.



Decode fields msz	ssz	Rm	Instruction Details	Condition
	0x	11111	UNALLOCATED	-
	1x		UNALLOCATED	-
00	00	!= 11111	LD1RQB (scalar plus scalar)	FEAT_SVE || FEAT_SME
00	01	!= 11111	LD1ROB (scalar plus scalar)	FEAT_F64MM
01	00	!= 11111	LD1RQH (scalar plus scalar)	FEAT_SVE || FEAT_SME
01	01	!= 11111	LD1ROH (scalar plus scalar)	FEAT_F64MM
10	00	!= 11111	LD1RQW (scalar plus scalar)	FEAT_SVE || FEAT_SME
10	01	!= 11111	LD1ROW (scalar plus scalar)	FEAT_F64MM
11	00	!= 11111	LD1RQD (scalar plus scalar)	FEAT_SVE || FEAT_SME
11	01	!= 11111	LD1ROD (scalar plus scalar)	FEAT_F64MM

SVE contiguous load (scalar plus scalar)

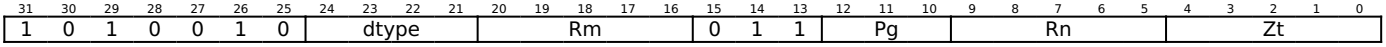
The encodings in this section are decoded from SVE Memory - Contiguous Load.



Decode fields dtype	Rm	Instruction Details	Condition
	11111	UNALLOCATED	-
0000	!= 11111	LD1B (scalar plus scalar, single register) — 8-bit element	FEAT_SVE || FEAT_SME
0001	!= 11111	LD1B (scalar plus scalar, single register) — 16-bit element	FEAT_SVE || FEAT_SME
0010	!= 11111	LD1B (scalar plus scalar, single register) — 32-bit element	FEAT_SVE || FEAT_SME
0011	!= 11111	LD1B (scalar plus scalar, single register) — 64-bit element	FEAT_SVE || FEAT_SME
0100	!= 11111	LD1SW (scalar plus scalar)	FEAT_SVE || FEAT_SME
0101	!= 11111	LD1H (scalar plus scalar, single register) — 16-bit element	FEAT_SVE || FEAT_SME
0110	!= 11111	LD1H (scalar plus scalar, single register) — 32-bit element	FEAT_SVE || FEAT_SME
0111	!= 11111	LD1H (scalar plus scalar, single register) — 64-bit element	FEAT_SVE || FEAT_SME
1000	!= 11111	LD1SH (scalar plus scalar) — 64-bit element	FEAT_SVE || FEAT_SME
1001	!= 11111	LD1SH (scalar plus scalar) — 32-bit element	FEAT_SVE || FEAT_SME
1010	!= 11111	LD1W (scalar plus scalar, single register) — 32-bit element	FEAT_SVE || FEAT_SME
1011	!= 11111	LD1W (scalar plus scalar, single register) — 64-bit element	FEAT_SVE || FEAT_SME
1100	!= 11111	LD1SB (scalar plus scalar) — 64-bit element	FEAT_SVE || FEAT_SME
1101	!= 11111	LD1SB (scalar plus scalar) — 32-bit element	FEAT_SVE || FEAT_SME
1110	!= 11111	LD1SB (scalar plus scalar) — 16-bit element	FEAT_SVE || FEAT_SME
1111	!= 11111	LD1D (scalar plus scalar, single register)	FEAT_SVE || FEAT_SME

SVE contiguous first-fault load (scalar plus scalar)

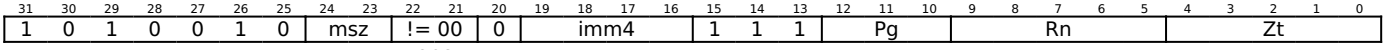
The encodings in this section are decoded from SVE Memory - Contiguous Load.



Decode fields dtype	Instruction Details	Condition
0000	LDFF1B (scalar plus scalar) — 8-bit element	FEAT_SVE
0001	LDFF1B (scalar plus scalar) — 16-bit element	FEAT_SVE
0010	LDFF1B (scalar plus scalar) — 32-bit element	FEAT_SVE
0011	LDFF1B (scalar plus scalar) — 64-bit element	FEAT_SVE
0100	LDFF1SW (scalar plus scalar)	FEAT_SVE
0101	LDFF1H (scalar plus scalar) — 16-bit element	FEAT_SVE
0110	LDFF1H (scalar plus scalar) — 32-bit element	FEAT_SVE
0111	LDFF1H (scalar plus scalar) — 64-bit element	FEAT_SVE
1000	LDFF1SH (scalar plus scalar) — 64-bit element	FEAT_SVE
1001	LDFF1SH (scalar plus scalar) — 32-bit element	FEAT_SVE
1010	LDFF1W (scalar plus scalar) — 32-bit element	FEAT_SVE
1011	LDFF1W (scalar plus scalar) — 64-bit element	FEAT_SVE
1100	LDFF1SB (scalar plus scalar) — 64-bit element	FEAT_SVE
1101	LDFF1SB (scalar plus scalar) — 32-bit element	FEAT_SVE
1110	LDFF1SB (scalar plus scalar) — 16-bit element	FEAT_SVE
1111	LDFF1D (scalar plus scalar)	FEAT_SVE

SVE load multiple structures (scalar plus immediate)

The encodings in this section are decoded from SVE Memory - Contiguous Load.

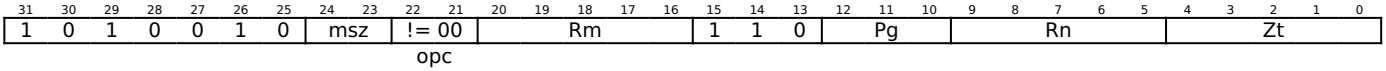


The following constraints also apply to this encoding: opc != '00'

Decode fields		Instruction Details	Condition
msz	opc		
00	01	LD2B (scalar plus immediate)	FEAT_SVE || FEAT_SME
00	10	LD3B (scalar plus immediate)	FEAT_SVE || FEAT_SME
00	11	LD4B (scalar plus immediate)	FEAT_SVE || FEAT_SME
01	01	LD2H (scalar plus immediate)	FEAT_SVE || FEAT_SME
01	10	LD3H (scalar plus immediate)	FEAT_SVE || FEAT_SME
01	11	LD4H (scalar plus immediate)	FEAT_SVE || FEAT_SME
10	01	LD2W (scalar plus immediate)	FEAT_SVE || FEAT_SME
10	10	LD3W (scalar plus immediate)	FEAT_SVE || FEAT_SME
10	11	LD4W (scalar plus immediate)	FEAT_SVE || FEAT_SME
11	01	LD2D (scalar plus immediate)	FEAT_SVE || FEAT_SME
11	10	LD3D (scalar plus immediate)	FEAT_SVE || FEAT_SME
11	11	LD4D (scalar plus immediate)	FEAT_SVE || FEAT_SME

SVE load multiple structures (scalar plus scalar)

The encodings in this section are decoded from SVE Memory - Contiguous Load.

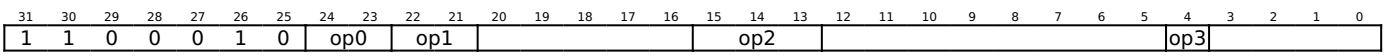


The following constraints also apply to this encoding: opc != '00'

Decode fields			Instruction Details	Condition
msz	opc	Rm		
	!= 00	11111	UNALLOCATED	-
00	01	!= 11111	LD2B (scalar plus scalar)	FEAT_SVE || FEAT_SME
00	10	!= 11111	LD3B (scalar plus scalar)	FEAT_SVE || FEAT_SME
00	11	!= 11111	LD4B (scalar plus scalar)	FEAT_SVE || FEAT_SME
01	01	!= 11111	LD2H (scalar plus scalar)	FEAT_SVE || FEAT_SME
01	10	!= 11111	LD3H (scalar plus scalar)	FEAT_SVE || FEAT_SME
01	11	!= 11111	LD4H (scalar plus scalar)	FEAT_SVE || FEAT_SME
10	01	!= 11111	LD2W (scalar plus scalar)	FEAT_SVE || FEAT_SME
10	10	!= 11111	LD3W (scalar plus scalar)	FEAT_SVE || FEAT_SME
10	11	!= 11111	LD4W (scalar plus scalar)	FEAT_SVE || FEAT_SME
11	01	!= 11111	LD2D (scalar plus scalar)	FEAT_SVE || FEAT_SME
11	10	!= 11111	LD3D (scalar plus scalar)	FEAT_SVE || FEAT_SME
11	11	!= 11111	LD4D (scalar plus scalar)	FEAT_SVE || FEAT_SME

SVE Memory - 64-bit Gather

The encodings in this section are decoded from SVE encodings.



Decode fields				Instruction details	Condition
op0	op1	op2	op3		
00	00	101		LD1Q	FEAT_SVE2p1
00	01	0xx	1	UNALLOCATED	-
00	11	1xx	0	SVE 64-bit gather prefetch (scalar plus 64-bit scaled offsets)	-
00	11		1	UNALLOCATED	-
00	x1	0xx	0	SVE 64-bit gather prefetch (scalar plus unpacked 32-bit scaled offsets)	-
	00	111	0	SVE 64-bit gather prefetch (vector plus immediate)	-
	00	111	1	UNALLOCATED	-
	00	1x0		SVE2 64-bit gather non-temporal load (vector plus scalar)	-
	01	1xx		SVE 64-bit gather load (vector plus immediate)	-
	10	1xx		SVE 64-bit gather load (scalar plus 64-bit unscaled offsets)	-
	x0	0xx		SVE 64-bit gather load (scalar plus unpacked 32-bit unscaled offsets)	-
!= 00	00	101		UNALLOCATED	-
!= 00	11	1xx		SVE 64-bit gather load (scalar plus 64-bit scaled offsets)	-
!= 00	x1	0xx		SVE 64-bit gather load (scalar plus 32-bit unpacked scaled offsets)	-

SVE 64-bit gather prefetch (scalar plus 64-bit scaled offsets)

The encodings in this section are decoded from SVE Memory - 64-bit Gather.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	1	0	0	0	1	1	Zm					1	msz		Pg			Rn					0	prfop			

Decode fields	Instruction Details	Condition
msz		
00	PRFB (scalar plus vector)	FEAT_SVE
01	PRFH (scalar plus vector)	FEAT_SVE
10	PRFW (scalar plus vector)	FEAT_SVE
11	PRFD (scalar plus vector)	FEAT_SVE

SVE 64-bit gather prefetch (scalar plus unpacked 32-bit scaled offsets)

The encodings in this section are decoded from SVE Memory - 64-bit Gather.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	1	0	0	0	xs	1	Zm					0	msz		Pg			Rn					0	prfop			

Decode fields	Instruction Details	Condition
msz		
00	PRFB (scalar plus vector)	FEAT_SVE
01	PRFH (scalar plus vector)	FEAT_SVE
10	PRFW (scalar plus vector)	FEAT_SVE
11	PRFD (scalar plus vector)	FEAT_SVE

SVE 64-bit gather prefetch (vector plus immediate)

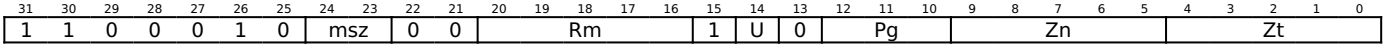
The encodings in this section are decoded from SVE Memory - 64-bit Gather.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	1	0	msz		0	0	imm5					1	1	1	Pg			Zn					0	prfop			

Decode fields msz	Instruction Details	Condition
00	PRFB (vector plus immediate)	FEAT_SVE
01	PRFH (vector plus immediate)	FEAT_SVE
10	PRFW (vector plus immediate)	FEAT_SVE
11	PRFD (vector plus immediate)	FEAT_SVE

SVE2 64-bit gather non-temporal load (vector plus scalar)

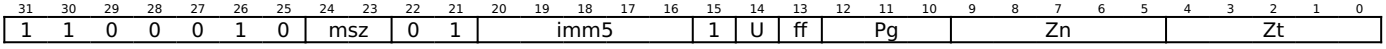
The encodings in this section are decoded from SVE Memory - 64-bit Gather.



Decode fields msz	U	Instruction Details	Condition
00	0	LDNT1SB	FEAT_SVE2
00	1	LDNT1B (vector plus scalar)	FEAT_SVE2
01	0	LDNT1SH	FEAT_SVE2
01	1	LDNT1H (vector plus scalar)	FEAT_SVE2
10	0	LDNT1SW	FEAT_SVE2
10	1	LDNT1W (vector plus scalar)	FEAT_SVE2
11	0	UNALLOCATED	-
11	1	LDNT1D (vector plus scalar)	FEAT_SVE2

SVE 64-bit gather load (vector plus immediate)

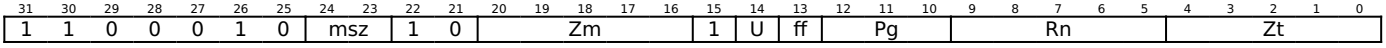
The encodings in this section are decoded from SVE Memory - 64-bit Gather.



Decode fields msz	U	ff	Instruction Details	Condition
00	0	0	LD1SB (vector plus immediate)	FEAT_SVE
00	0	1	LDFF1SB (vector plus immediate)	FEAT_SVE
00	1	0	LD1B (vector plus immediate)	FEAT_SVE
00	1	1	LDFF1B (vector plus immediate)	FEAT_SVE
01	0	0	LD1SH (vector plus immediate)	FEAT_SVE
01	0	1	LDFF1SH (vector plus immediate)	FEAT_SVE
01	1	0	LD1H (vector plus immediate)	FEAT_SVE
01	1	1	LDFF1H (vector plus immediate)	FEAT_SVE
10	0	0	LD1SW (vector plus immediate)	FEAT_SVE
10	0	1	LDFF1SW (vector plus immediate)	FEAT_SVE
10	1	0	LD1W (vector plus immediate)	FEAT_SVE
10	1	1	LDFF1W (vector plus immediate)	FEAT_SVE
11	0		UNALLOCATED	-
11	1	0	LD1D (vector plus immediate)	FEAT_SVE
11	1	1	LDFF1D (vector plus immediate)	FEAT_SVE

SVE 64-bit gather load (scalar plus 64-bit unscaled offsets)

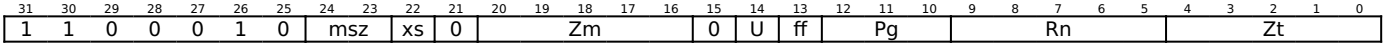
The encodings in this section are decoded from SVE Memory - 64-bit Gather.



Decode fields			Instruction Details	Condition
msz	U	ff		
00	0	0	LD1SB (scalar plus vector)	FEAT_SVE
00	0	1	LDFF1SB (scalar plus vector)	FEAT_SVE
00	1	0	LD1B (scalar plus vector)	FEAT_SVE
00	1	1	LDFF1B (scalar plus vector)	FEAT_SVE
01	0	0	LD1SH (scalar plus vector)	FEAT_SVE
01	0	1	LDFF1SH (scalar plus vector)	FEAT_SVE
01	1	0	LD1H (scalar plus vector)	FEAT_SVE
01	1	1	LDFF1H (scalar plus vector)	FEAT_SVE
10	0	0	LD1SW (scalar plus vector)	FEAT_SVE
10	0	1	LDFF1SW (scalar plus vector)	FEAT_SVE
10	1	0	LD1W (scalar plus vector)	FEAT_SVE
10	1	1	LDFF1W (scalar plus vector)	FEAT_SVE
11	0		UNALLOCATED	-
11	1	0	LD1D (scalar plus vector)	FEAT_SVE
11	1	1	LDFF1D (scalar plus vector)	FEAT_SVE

SVE 64-bit gather load (scalar plus unpacked 32-bit unscaled offsets)

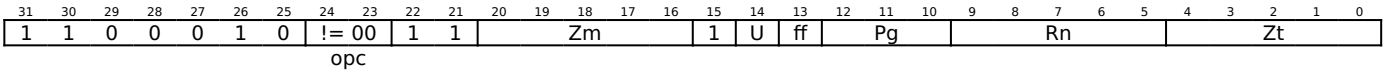
The encodings in this section are decoded from SVE Memory - 64-bit Gather.



Decode fields			Instruction Details	Condition
msz	U	ff		
00	0	0	LD1SB (scalar plus vector)	FEAT_SVE
00	0	1	LDFF1SB (scalar plus vector)	FEAT_SVE
00	1	0	LD1B (scalar plus vector)	FEAT_SVE
00	1	1	LDFF1B (scalar plus vector)	FEAT_SVE
01	0	0	LD1SH (scalar plus vector)	FEAT_SVE
01	0	1	LDFF1SH (scalar plus vector)	FEAT_SVE
01	1	0	LD1H (scalar plus vector)	FEAT_SVE
01	1	1	LDFF1H (scalar plus vector)	FEAT_SVE
10	0	0	LD1SW (scalar plus vector)	FEAT_SVE
10	0	1	LDFF1SW (scalar plus vector)	FEAT_SVE
10	1	0	LD1W (scalar plus vector)	FEAT_SVE
10	1	1	LDFF1W (scalar plus vector)	FEAT_SVE
11	0		UNALLOCATED	-
11	1	0	LD1D (scalar plus vector)	FEAT_SVE
11	1	1	LDFF1D (scalar plus vector)	FEAT_SVE

SVE 64-bit gather load (scalar plus 64-bit scaled offsets)

The encodings in this section are decoded from SVE Memory - 64-bit Gather.

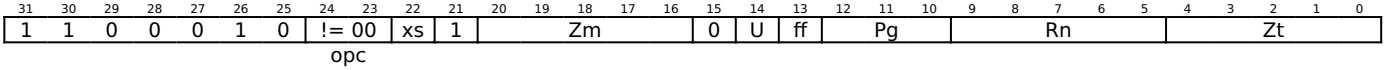


The following constraints also apply to this encoding: opc != '00'

Decode fields			Instruction Details	Condition
opc	U	ff		
01	0	0	LD1SH (scalar plus vector)	FEAT_SVE
01	0	1	LDFF1SH (scalar plus vector)	FEAT_SVE
01	1	0	LD1H (scalar plus vector)	FEAT_SVE
01	1	1	LDFF1H (scalar plus vector)	FEAT_SVE
10	0	0	LD1SW (scalar plus vector)	FEAT_SVE
10	0	1	LDFF1SW (scalar plus vector)	FEAT_SVE
10	1	0	LD1W (scalar plus vector)	FEAT_SVE
10	1	1	LDFF1W (scalar plus vector)	FEAT_SVE
11	0		UNALLOCATED	-
11	1	0	LD1D (scalar plus vector)	FEAT_SVE
11	1	1	LDFF1D (scalar plus vector)	FEAT_SVE

SVE 64-bit gather load (scalar plus 32-bit unpacked scaled offsets)

The encodings in this section are decoded from SVE Memory - 64-bit Gather.

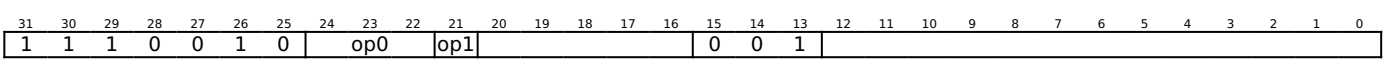


The following constraints also apply to this encoding: opc != '00'

Decode fields			Instruction Details	Condition
opc	U	ff		
01	0	0	LD1SH (scalar plus vector)	FEAT_SVE
01	0	1	LDFF1SH (scalar plus vector)	FEAT_SVE
01	1	0	LD1H (scalar plus vector)	FEAT_SVE
01	1	1	LDFF1H (scalar plus vector)	FEAT_SVE
10	0	0	LD1SW (scalar plus vector)	FEAT_SVE
10	0	1	LDFF1SW (scalar plus vector)	FEAT_SVE
10	1	0	LD1W (scalar plus vector)	FEAT_SVE
10	1	1	LDFF1W (scalar plus vector)	FEAT_SVE
11	0		UNALLOCATED	-
11	1	0	LD1D (scalar plus vector)	FEAT_SVE
11	1	1	LDFF1D (scalar plus vector)	FEAT_SVE

SVE Memory - Non-temporal and Quadword Scatter Store

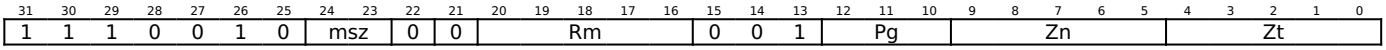
The encodings in this section are decoded from SVE encodings.



Decode fields		Instruction details	Condition
op0	op1		
000	1	ST1Q	FEAT_SVE2p1
xx0	0	SVE2 64-bit scatter non-temporal store (vector plus scalar)	-
xx1	0	SVE2 32-bit scatter non-temporal store (vector plus scalar)	-
!= 000	1	UNALLOCATED	-

SVE2 64-bit scatter non-temporal store (vector plus scalar)

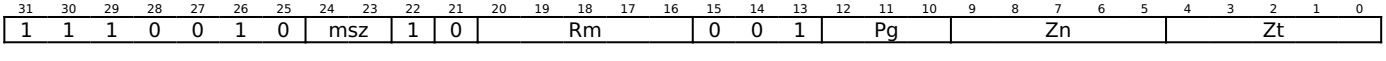
The encodings in this section are decoded from SVE Memory - Non-temporal and Quadword Scatter Store.



Decode fields msz	Instruction Details	Condition
00	STNT1B (vector plus scalar)	FEAT_SVE2
01	STNT1H (vector plus scalar)	FEAT_SVE2
10	STNT1W (vector plus scalar)	FEAT_SVE2
11	STNT1D (vector plus scalar)	FEAT_SVE2

SVE2 32-bit scatter non-temporal store (vector plus scalar)

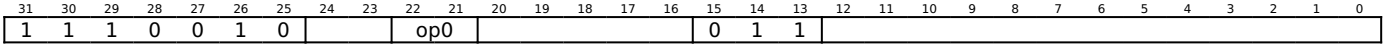
The encodings in this section are decoded from SVE Memory - Non-temporal and Quadword Scatter Store.



Decode fields msz	Instruction Details	Condition
00	STNT1B (vector plus scalar)	FEAT_SVE2
01	STNT1H (vector plus scalar)	FEAT_SVE2
10	STNT1W (vector plus scalar)	FEAT_SVE2
11	UNALLOCATED	-

SVE Memory - Non-temporal and Multi-register Contiguous Store

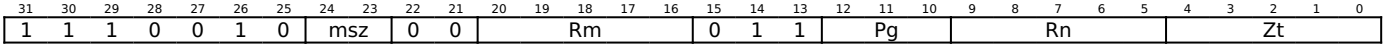
The encodings in this section are decoded from SVE encodings.



Decode fields op0	Instruction details
00	SVE contiguous non-temporal store (scalar plus scalar)
!= 00	SVE store multiple structures (scalar plus scalar)

SVE contiguous non-temporal store (scalar plus scalar)

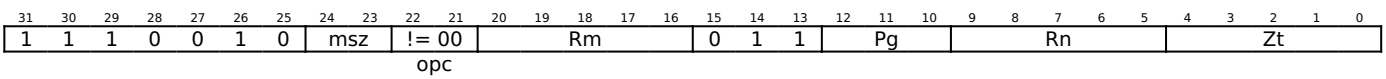
The encodings in this section are decoded from SVE Memory - Non-temporal and Multi-register Contiguous Store.



Decode fields msz	Decode fields Rm	Instruction Details	Condition
	11111	UNALLOCATED	-
00	!= 11111	STNT1B (scalar plus scalar, single register)	FEAT_SVE || FEAT_SME
01	!= 11111	STNT1H (scalar plus scalar, single register)	FEAT_SVE || FEAT_SME
10	!= 11111	STNT1W (scalar plus scalar, single register)	FEAT_SVE || FEAT_SME
11	!= 11111	STNT1D (scalar plus scalar, single register)	FEAT_SVE || FEAT_SME

SVE store multiple structures (scalar plus scalar)

The encodings in this section are decoded from SVE Memory - Non-temporal and Multi-register Contiguous Store.

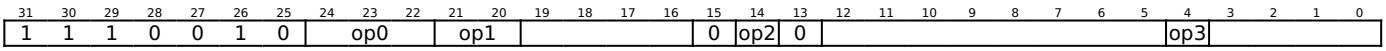


The following constraints also apply to this encoding: opc != '00'

Decode fields msz	opc	Rm	Instruction Details	Condition
	!= 00	11111	UNALLOCATED	-
00	01	!= 11111	ST2B (scalar plus scalar)	FEAT_SVE || FEAT_SME
00	10	!= 11111	ST3B (scalar plus scalar)	FEAT_SVE || FEAT_SME
00	11	!= 11111	ST4B (scalar plus scalar)	FEAT_SVE || FEAT_SME
01	01	!= 11111	ST2H (scalar plus scalar)	FEAT_SVE || FEAT_SME
01	10	!= 11111	ST3H (scalar plus scalar)	FEAT_SVE || FEAT_SME
01	11	!= 11111	ST4H (scalar plus scalar)	FEAT_SVE || FEAT_SME
10	01	!= 11111	ST2W (scalar plus scalar)	FEAT_SVE || FEAT_SME
10	10	!= 11111	ST3W (scalar plus scalar)	FEAT_SVE || FEAT_SME
10	11	!= 11111	ST4W (scalar plus scalar)	FEAT_SVE || FEAT_SME
11	01	!= 11111	ST2D (scalar plus scalar)	FEAT_SVE || FEAT_SME
11	10	!= 11111	ST3D (scalar plus scalar)	FEAT_SVE || FEAT_SME
11	11	!= 11111	ST4D (scalar plus scalar)	FEAT_SVE || FEAT_SME

SVE Memory - Contiguous Store and Unsized Contiguous

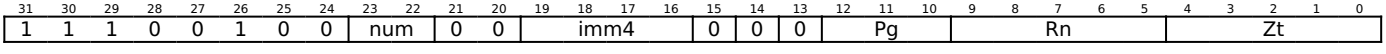
The encodings in this section are decoded from SVE encodings.



Decode fields op0	op1	op2	op3	Instruction details	Condition
0xx	00	0		SVE store multiple structures (quadwords, scalar plus immediate)	-
0xx	01	0		UNALLOCATED	-
0xx	1x	0		SVE store multiple structures (quadwords, scalar plus scalar)	-
10x		0		UNALLOCATED	-
110		0	0	STR (predicate)	FEAT_SVE || FEAT_SME
110		0	1	UNALLOCATED	-
110		1		STR (vector)	FEAT_SVE || FEAT_SME
111	0x	0		~~UNALLOCATED~~STLRO	FEAT_SVE_LRCPC-
111	1x	0		UNALLOCATED	-
!= 110		1		SVE contiguous store (scalar plus scalar)	-

SVE store multiple structures (quadwords, scalar plus immediate)

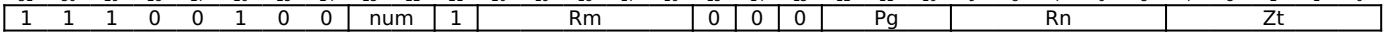
The encodings in this section are decoded from SVE Memory - Contiguous Store and Unsized Contiguous.



Decode fields num	Instruction Details	Condition
00	UNALLOCATED	-
01	ST2Q (scalar plus immediate)	FEAT_SVE2p1 || FEAT_SME2p1
10	ST3Q (scalar plus immediate)	FEAT_SVE2p1 || FEAT_SME2p1
11	ST4Q (scalar plus immediate)	FEAT_SVE2p1 || FEAT_SME2p1

SVE store multiple structures (quadwords, scalar plus scalar)

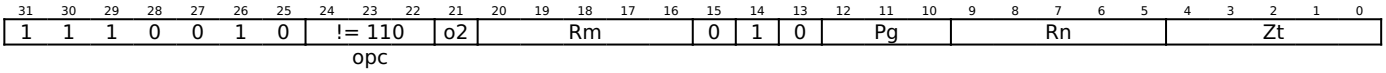
The encodings in this section are decoded from SVE Memory - Contiguous Store and Unsized Contiguous.



Decode fields num	Rm	Instruction Details	Condition
00		UNALLOCATED	-
01	!= 11111	ST2Q (scalar plus scalar)	FEAT_SVE2p1 || FEAT_SME2p1
10	!= 11111	ST3Q (scalar plus scalar)	FEAT_SVE2p1 || FEAT_SME2p1
11	!= 11111	ST4Q (scalar plus scalar)	FEAT_SVE2p1 || FEAT_SME2p1
!= 00	11111	UNALLOCATED	-

SVE contiguous store (scalar plus scalar)

The encodings in this section are decoded from SVE Memory - Contiguous Store and Unsized Contiguous.

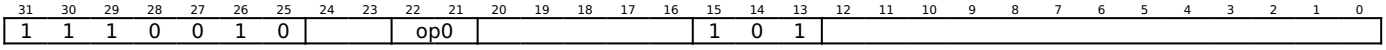


The following constraints also apply to this encoding: opc != '110'

Decode fields opc	o2	Rm	Instruction Details	Condition
0xx		11111	UNALLOCATED	-
00x		!= 11111	ST1B (scalar plus scalar, single register)	FEAT_SVE || FEAT_SME
01x		!= 11111	ST1H (scalar plus scalar, single register)	FEAT_SVE || FEAT_SME
10x	0	11111	UNALLOCATED	-
100	0	!= 11111	ST1W (scalar plus scalar, single register) — SVE2	FEAT_SVE2p1
100	1		UNALLOCATED	-
101		!= 11111	ST1W (scalar plus scalar, single register) — SVE	FEAT_SVE || FEAT_SME
101	1	11111	UNALLOCATED	-
111		11111	UNALLOCATED	-
111	0	!= 11111	ST1D (scalar plus scalar, single register) — SVE2	FEAT_SVE2p1
111	1	!= 11111	ST1D (scalar plus scalar, single register) — SVE	FEAT_SVE || FEAT_SME

SVE Memory - Scatter

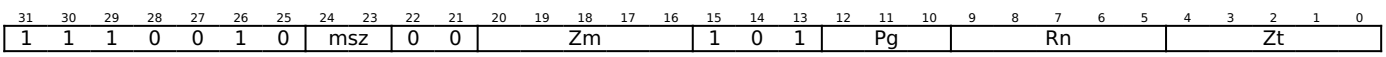
The encodings in this section are decoded from SVE encodings.



Decode fields op0	Instruction details
00	SVE 64-bit scatter store (scalar plus 64-bit unscaled offsets)
01	SVE 64-bit scatter store (scalar plus 64-bit scaled offsets)
10	SVE 64-bit scatter store (vector plus immediate)
11	SVE 32-bit scatter store (vector plus immediate)

SVE 64-bit scatter store (scalar plus 64-bit unscaled offsets)

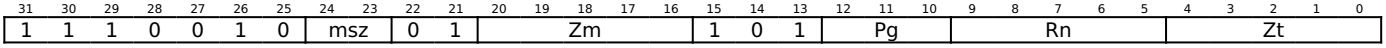
The encodings in this section are decoded from SVE Memory - Scatter.



Decode fields msz	Instruction Details	Condition
00	ST1B (scalar plus vector)	FEAT_SVE
01	ST1H (scalar plus vector)	FEAT_SVE
10	ST1W (scalar plus vector)	FEAT_SVE
11	ST1D (scalar plus vector)	FEAT_SVE

SVE 64-bit scatter store (scalar plus 64-bit scaled offsets)

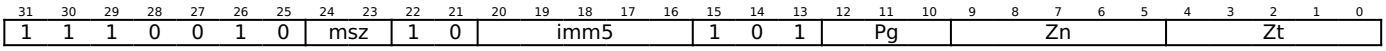
The encodings in this section are decoded from SVE Memory - Scatter.



Decode fields msz	Instruction Details	Condition
00	UNALLOCATED	-
01	ST1H (scalar plus vector)	FEAT_SVE
10	ST1W (scalar plus vector)	FEAT_SVE
11	ST1D (scalar plus vector)	FEAT_SVE

SVE 64-bit scatter store (vector plus immediate)

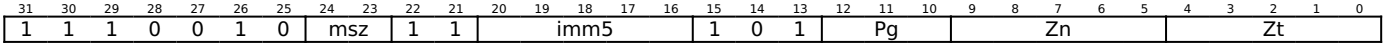
The encodings in this section are decoded from SVE Memory - Scatter.



Decode fields msz	Instruction Details	Condition
00	ST1B (vector plus immediate)	FEAT_SVE
01	ST1H (vector plus immediate)	FEAT_SVE
10	ST1W (vector plus immediate)	FEAT_SVE
11	ST1D (vector plus immediate)	FEAT_SVE

SVE 32-bit scatter store (vector plus immediate)

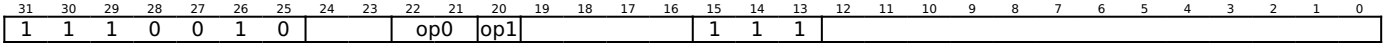
The encodings in this section are decoded from SVE Memory - Scatter.



Decode fields msz	Instruction Details	Condition
00	ST1B (vector plus immediate)	FEAT_SVE
01	ST1H (vector plus immediate)	FEAT_SVE
10	ST1W (vector plus immediate)	FEAT_SVE
11	UNALLOCATED	-

SVE Memory - Contiguous Store with Immediate Offset

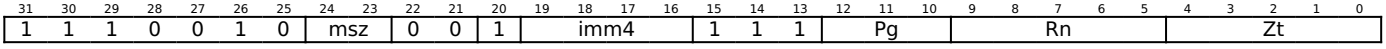
The encodings in this section are decoded from SVE encodings.



Decode fields op0	op1	Instruction details
00	1	SVE contiguous non-temporal store (scalar plus immediate)
	0	SVE contiguous store (scalar plus immediate)
!= 00	1	SVE store multiple structures (scalar plus immediate)

SVE contiguous non-temporal store (scalar plus immediate)

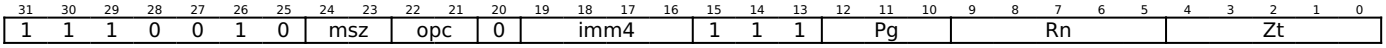
The encodings in this section are decoded from SVE Memory - Contiguous Store with Immediate Offset.



Decode fields msz	Instruction Details	Condition
00	STNT1B (scalar plus immediate, single register)	FEAT_SVE || FEAT_SME
01	STNT1H (scalar plus immediate, single register)	FEAT_SVE || FEAT_SME
10	STNT1W (scalar plus immediate, single register)	FEAT_SVE || FEAT_SME
11	STNT1D (scalar plus immediate, single register)	FEAT_SVE || FEAT_SME

SVE contiguous store (scalar plus immediate)

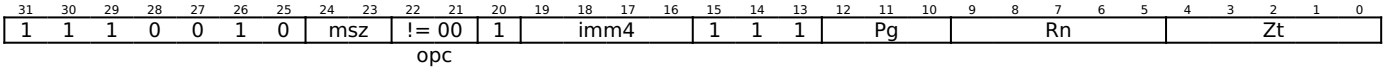
The encodings in this section are decoded from SVE Memory - Contiguous Store with Immediate Offset.



Decode fields msz	opc	Instruction Details	Condition
00		ST1B (scalar plus immediate, single register)	FEAT_SVE || FEAT_SME
01		ST1H (scalar plus immediate, single register)	FEAT_SVE || FEAT_SME
10	00	ST1W (scalar plus immediate, single register) — SVE2	FEAT_SVE2p1
10	01	UNALLOCATED	-
10	1x	ST1W (scalar plus immediate, single register) — SVE	FEAT_SVE || FEAT_SME
11	0x	UNALLOCATED	-
11	10	ST1D (scalar plus immediate, single register) — SVE2	FEAT_SVE2p1
11	11	ST1D (scalar plus immediate, single register) — SVE	FEAT_SVE || FEAT_SME

SVE store multiple structures (scalar plus immediate)

The encodings in this section are decoded from SVE Memory - Contiguous Store with Immediate Offset.

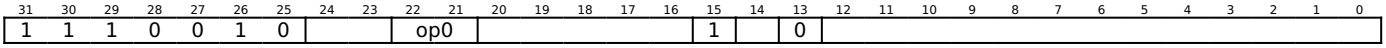


The following constraints also apply to this encoding: opc != '00'

Decode fields msz	opc	Instruction Details	Condition
00	01	ST2B (scalar plus immediate)	FEAT_SVE || FEAT_SME
00	10	ST3B (scalar plus immediate)	FEAT_SVE || FEAT_SME
00	11	ST4B (scalar plus immediate)	FEAT_SVE || FEAT_SME
01	01	ST2H (scalar plus immediate)	FEAT_SVE || FEAT_SME
01	10	ST3H (scalar plus immediate)	FEAT_SVE || FEAT_SME
01	11	ST4H (scalar plus immediate)	FEAT_SVE || FEAT_SME
10	01	ST2W (scalar plus immediate)	FEAT_SVE || FEAT_SME
10	10	ST3W (scalar plus immediate)	FEAT_SVE || FEAT_SME
10	11	ST4W (scalar plus immediate)	FEAT_SVE || FEAT_SME
11	01	ST2D (scalar plus immediate)	FEAT_SVE || FEAT_SME
11	10	ST3D (scalar plus immediate)	FEAT_SVE || FEAT_SME
11	11	ST4D (scalar plus immediate)	FEAT_SVE || FEAT_SME

SVE Memory - Scatter with Optional Sign Extend

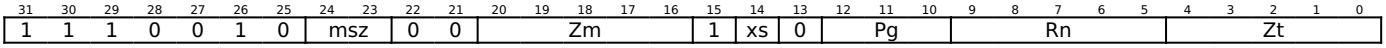
The encodings in this section are decoded from SVE encodings.



Decode fields op0	Instruction details
00	SVE 64-bit scatter store (scalar plus unpacked 32-bit unscaled offsets)
01	SVE 64-bit scatter store (scalar plus unpacked 32-bit scaled offsets)
10	SVE 32-bit scatter store (scalar plus 32-bit unscaled offsets)
11	SVE 32-bit scatter store (scalar plus 32-bit scaled offsets)

SVE 64-bit scatter store (scalar plus unpacked 32-bit unscaled offsets)

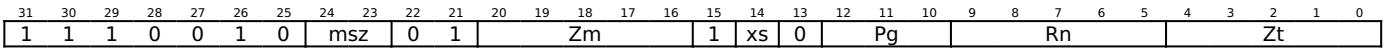
The encodings in this section are decoded from SVE Memory - Scatter with Optional Sign Extend.



Decode fields msz	Instruction Details	Condition
00	ST1B (scalar plus vector)	FEAT_SVE
01	ST1H (scalar plus vector)	FEAT_SVE
10	ST1W (scalar plus vector)	FEAT_SVE
11	ST1D (scalar plus vector)	FEAT_SVE

SVE 64-bit scatter store (scalar plus unpacked 32-bit scaled offsets)

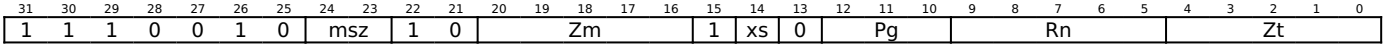
The encodings in this section are decoded from SVE Memory - Scatter with Optional Sign Extend.



Decode fields msz	Instruction Details	Condition
00	UNALLOCATED	-
01	ST1H (scalar plus vector)	FEAT_SVE
10	ST1W (scalar plus vector)	FEAT_SVE
11	ST1D (scalar plus vector)	FEAT_SVE

SVE 32-bit scatter store (scalar plus 32-bit unscaled offsets)

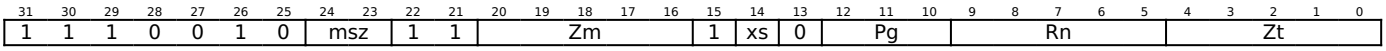
The encodings in this section are decoded from SVE Memory - Scatter with Optional Sign Extend.



Decode fields msz	Instruction Details	Condition
00	ST1B (scalar plus vector)	FEAT_SVE
01	ST1H (scalar plus vector)	FEAT_SVE
10	ST1W (scalar plus vector)	FEAT_SVE
11	UNALLOCATED	-

SVE 32-bit scatter store (scalar plus 32-bit scaled offsets)

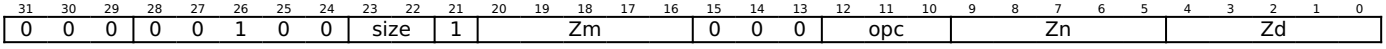
The encodings in this section are decoded from SVE Memory - Scatter with Optional Sign Extend.



Decode fields msz	Instruction Details	Condition
00	UNALLOCATED	-
01	ST1H (scalar plus vector)	FEAT_SVE
10	ST1W (scalar plus vector)	FEAT_SVE
11	UNALLOCATED	-

SVE integer add/subtract vectors (unpredicated)

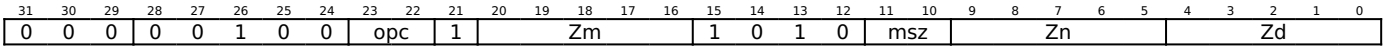
The encodings in this section are decoded from SVE encodings.



Decode fields size	Decode fields opc	Instruction Details	Condition
	000	ADD (vectors, unpredicated)	FEAT_SVE || FEAT_SME
	001	SUB (vectors, unpredicated)	FEAT_SVE || FEAT_SME
	100	SQADD (vectors, unpredicated)	FEAT_SVE || FEAT_SME
	101	UQADD (vectors, unpredicated)	FEAT_SVE || FEAT_SME
	110	SQSUB (vectors, unpredicated)	FEAT_SVE || FEAT_SME
	111	UQSUB (vectors, unpredicated)	FEAT_SVE || FEAT_SME
11	010	ADDPT (unpredicated)	FEAT_SVE && FEAT_CPA
11	011	SUBPT (unpredicated)	FEAT_SVE && FEAT_CPA
!= 11	01x	UNALLOCATED	-

SVE address generation

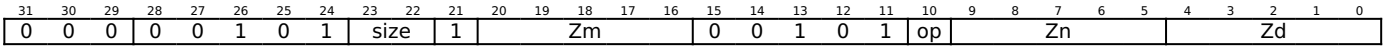
The encodings in this section are decoded from SVE encodings.



Decode fields opc	Instruction Details	Condition
00	ADR — Unpacked 32-bit signed offsets	FEAT_SVE
01	ADR — Unpacked 32-bit unsigned offsets	FEAT_SVE
1x	ADR — Packed offsets	FEAT_SVE

SVE table lookup (three sources)

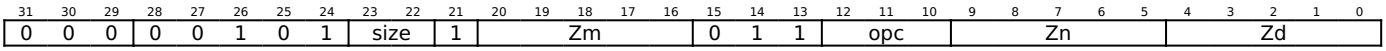
The encodings in this section are decoded from SVE encodings.



Decode fields op	Instruction Details	Condition
0	TBL	FEAT_SVE2 || FEAT_SME
1	TBX	FEAT_SVE2 || FEAT_SME

SVE permute vector elements

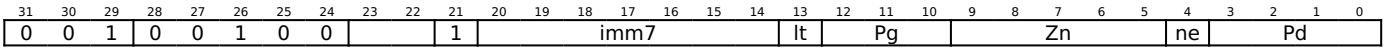
The encodings in this section are decoded from SVE encodings.



Decode fields opc	Instruction Details	Condition
000	ZIP1, ZIP2 (vectors) — Low halves	FEAT_SVE || FEAT_SME
001	ZIP1, ZIP2 (vectors) — High halves	FEAT_SVE || FEAT_SME
010	UZP1, UZP2 (vectors) — Even	FEAT_SVE || FEAT_SME
011	UZP1, UZP2 (vectors) — Odd	FEAT_SVE || FEAT_SME
100	TRN1, TRN2 (vectors) — Even	FEAT_SVE || FEAT_SME
101	TRN1, TRN2 (vectors) — Odd	FEAT_SVE || FEAT_SME
11x	UNALLOCATED	-

SVE integer compare with unsigned immediate

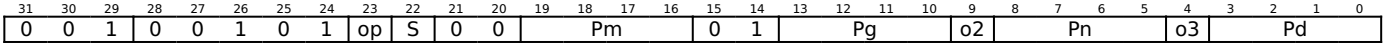
The encodings in this section are decoded from SVE encodings.



Decode fields lt	ne	Instruction Details	Condition
0	0	CMP<cc> (immediate) — Higher or same	FEAT_SVE || FEAT_SME
0	1	CMP<cc> (immediate) — Higher	FEAT_SVE || FEAT_SME
1	0	CMP<cc> (immediate) — Lower	FEAT_SVE || FEAT_SME
1	1	CMP<cc> (immediate) — Lower or same	FEAT_SVE || FEAT_SME

SVE predicate logical operations

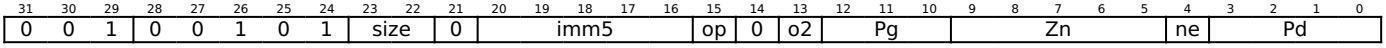
The encodings in this section are decoded from SVE encodings.



Decode fields op	S	o2	o3	Instruction Details	Condition
0	0	0	0	AND (predicates)	FEAT_SVE || FEAT_SME
0	0	0	1	BIC (predicates)	FEAT_SVE || FEAT_SME
0	0	1	0	EOR (predicates)	FEAT_SVE || FEAT_SME
0	0	1	1	SEL (predicates)	FEAT_SVE || FEAT_SME
0	1	0	0	ANDS	FEAT_SVE || FEAT_SME
0	1	0	1	BICS	FEAT_SVE || FEAT_SME
0	1	1	0	EORS	FEAT_SVE || FEAT_SME
0	1	1	1	UNALLOCATED	-
1	0	0	0	ORR (predicates)	FEAT_SVE || FEAT_SME
1	0	0	1	ORN (predicates)	FEAT_SVE || FEAT_SME
1	0	1	0	NOR	FEAT_SVE || FEAT_SME
1	0	1	1	NAND	FEAT_SVE || FEAT_SME
1	1	0	0	ORRS	FEAT_SVE || FEAT_SME
1	1	0	1	ORNS	FEAT_SVE || FEAT_SME
1	1	1	0	NORS	FEAT_SVE || FEAT_SME
1	1	1	1	NANDS	FEAT_SVE || FEAT_SME

SVE integer compare with signed immediate

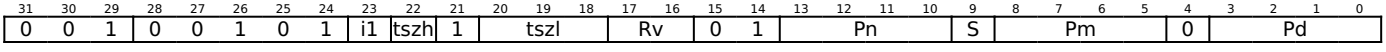
The encodings in this section are decoded from SVE encodings.



Decode fields op	o2	ne	Instruction Details	Condition
0	0	0	CMP<cc> (immediate) — Greater than or equal	FEAT_SVE || FEAT_SME
0	0	1	CMP<cc> (immediate) — Greater than	FEAT_SVE || FEAT_SME
0	1	0	CMP<cc> (immediate) — Less than	FEAT_SVE || FEAT_SME
0	1	1	CMP<cc> (immediate) — Less than or equal	FEAT_SVE || FEAT_SME
1	0	0	CMP<cc> (immediate) — Equal	FEAT_SVE || FEAT_SME
1	0	1	CMP<cc> (immediate) — Not equal	FEAT_SVE || FEAT_SME
1	1		UNALLOCATED	-

SVE broadcast predicate element

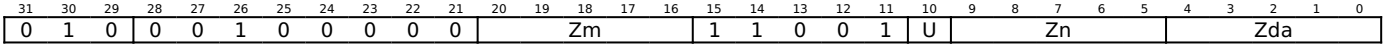
The encodings in this section are decoded from SVE encodings.



Decode fields S	Instruction Details	Condition
0	PSEL	FEAT_SME || FEAT_SVE2p1
1	UNALLOCATED	-

SVE two-way dot product

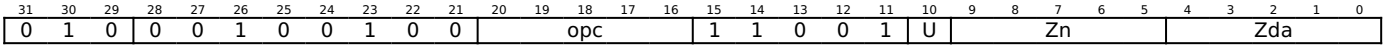
The encodings in this section are decoded from SVE encodings.



Decode fields U	Instruction Details	Condition
0	SDOT (2-way, vectors)	FEAT_SME2 || FEAT_SVE2p1
1	UDOT (2-way, vectors)	FEAT_SME2 || FEAT_SVE2p1

SVE two-way dot product (indexed)

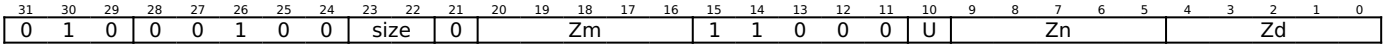
The encodings in this section are decoded from SVE encodings.



Decode fields U	Instruction Details	Condition
0	SDOT (2-way, indexed)	FEAT_SME2 || FEAT_SVE2p1
1	UDOT (2-way, indexed)	FEAT_SME2 || FEAT_SVE2p1

SVE integer clamp

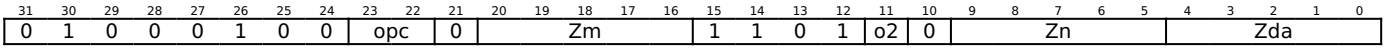
The encodings in this section are decoded from SVE encodings.



Decode fields U	Instruction Details	Condition
0	SCLAMP	FEAT_SME || FEAT_SVE2p1
1	UCLAMP	FEAT_SME || FEAT_SVE2p1

SVE2 multiply-add (checked pointer)

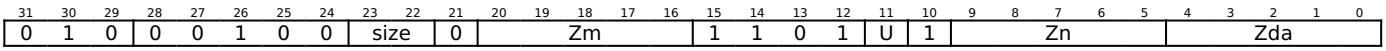
The encodings in this section are decoded from SVE encodings.



Decode fields opc	o2	Instruction Details	Condition
11	0	MLAPT	FEAT_SVE && FEAT_CPA
11	1	MADPT	FEAT_SVE && FEAT_CPA
!= 11		UNALLOCATED	-

SVE2 absolute difference and accumulate long

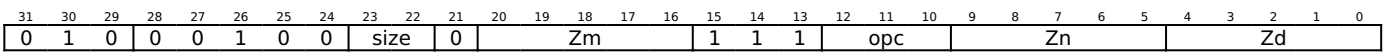
The encodings in this section are decoded from SVE encodings.



Decode fields U	Instruction Details	Condition
0	SABAL	FEAT_SVE2p3 || FEAT_SME2p3
1	UABAL	FEAT_SVE2p3 || FEAT_SME2p3

SVE permute vector elements (quadwords)

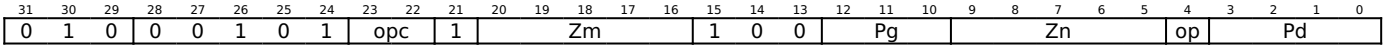
The encodings in this section are decoded from SVE encodings.



Decode fields opc	Instruction Details	Condition
000	ZIPQ1	FEAT_SVE2p1 || FEAT_SME2p1
001	ZIPQ2	FEAT_SVE2p1 || FEAT_SME2p1
010	UZPQ1	FEAT_SVE2p1 || FEAT_SME2p1
011	UZPQ2	FEAT_SVE2p1 || FEAT_SME2p1
10x	UNALLOCATED	-
110	TBLQ	FEAT_SVE2p1 || FEAT_SME2p1
111	UNALLOCATED	-

SVE2 character match

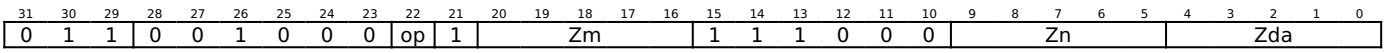
The encodings in this section are decoded from SVE encodings.



Decode fields opc	Decode fields op	Instruction Details	Condition
0x	0	MATCH	FEAT_SVE2
0x	1	NMATCH	FEAT_SVE2
1x		UNALLOCATED	-

SVE2 FP8 matrix multiply-accumulate

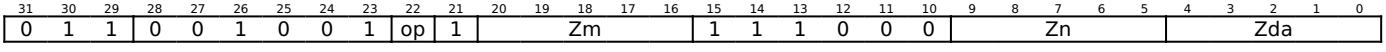
The encodings in this section are decoded from SVE encodings.



Decode fields op	Instruction Details	Condition
0	FMMLA (widening, FP8 to FP32)	FEAT_SVE2 && FEAT_F8F32MM
1	FMMLA (widening, FP8 to FP16)	FEAT_SVE2 && FEAT_F8F16MM

SVE2 floating-point non-widening matrix-multiply accumulate

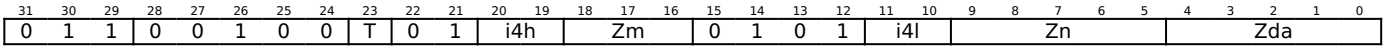
The encodings in this section are decoded from SVE encodings.



Decode fields op	Instruction Details	Condition
0	FMMLA (non-widening)	FEAT_SVE2p2 && FEAT_F16MM
1	BFMMLA (non-widening)	FEAT_SVE_B16MM

SVE2 FP8 multiply-add long (indexed)

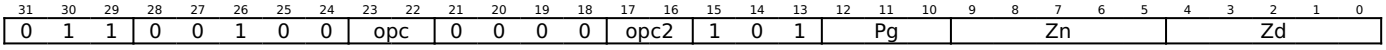
The encodings in this section are decoded from SVE encodings.



Decode fields T	Instruction Details	Condition
0	FMLALB (indexed, FP8 to FP16)	FEAT_SSVE_FP8FMA || (FEAT_SVE2 && FEAT_FP8FMA)
1	FMLALT (indexed, FP8 to FP16)	FEAT_SSVE_FP8FMA || (FEAT_SVE2 && FEAT_FP8FMA)

SVE floating-point convert (top, predicated)

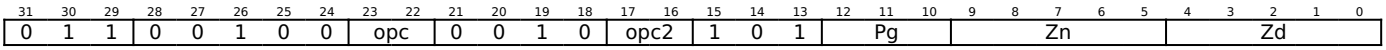
The encodings in this section are decoded from SVE encodings.



Decode fields opc	Decode fields opc2	Instruction Details	Condition
x0	11	UNALLOCATED	-
00	0x	UNALLOCATED	-
00	10	FCVTXNT	FEAT_SVE2p2 || FEAT_SME2p2
01		UNALLOCATED	-
10	00	FCVTNT (predicated) — Single-precision to half-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
10	01	FCVTLT — Half-precision to single-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
10	10	BFCVTNT	FEAT_SVE2p2 || FEAT_SME2p2
11	0x	UNALLOCATED	-
11	10	FCVTNT (predicated) — Double-precision to single-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
11	11	FCVTLT — Single-precision to double-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2

SVE floating-point convert precision odd elements

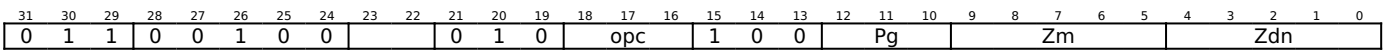
The encodings in this section are decoded from SVE encodings.



Decode fields opc	Decode fields opc2	Instruction Details	Condition
x0	11	UNALLOCATED	-
00	0x	UNALLOCATED	-
00	10	FCVTXNT	FEAT_SVE2 || FEAT_SME
01		UNALLOCATED	-
10	00	FCVTNT (predicated) — Single-precision to half-precision, merging	FEAT_SVE2 || FEAT_SME
10	01	FCVTLT — Half-precision to single-precision, merging	FEAT_SVE2 || FEAT_SME
10	10	BFCVTNT	(FEAT_SVE && FEAT_BF16) || (FEAT_SME && FEAT_BF16)
11	0x	UNALLOCATED	-
11	10	FCVTNT (predicated) — Double-precision to single-precision, merging	FEAT_SVE2 || FEAT_SME
11	11	FCVTLT — Single-precision to double-precision, merging	FEAT_SVE2 || FEAT_SME

SVE2 floating-point pairwise operations

The encodings in this section are decoded from SVE encodings.



Decode fields opc	Instruction Details	Condition
000	FADDP	FEAT_SVE2 || FEAT_SME
001	UNALLOCATED	-
01x	UNALLOCATED	-
100	FMAXNMP	FEAT_SVE2 || FEAT_SME
101	FMINNMP	FEAT_SVE2 || FEAT_SME
110	FMAXP	FEAT_SVE2 || FEAT_SME
111	FMINP	FEAT_SVE2 || FEAT_SME

SVE floating-point recursive reduction (quadwords)

The encodings in this section are decoded from SVE encodings.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	0	size		0	1	0	opc			1	0	1	Pg			Zn					Vd				

Decode fields opc	Instruction Details	Condition
000	FADDQV	FEAT_SVE2p1 || FEAT_SME2p1
001	UNALLOCATED	-
01x	UNALLOCATED	-
100	FMAXNMQV	FEAT_SVE2p1 || FEAT_SME2p1
101	FMINNMQV	FEAT_SVE2p1 || FEAT_SME2p1
110	FMAXQV	FEAT_SVE2p1 || FEAT_SME2p1
111	FMINQV	FEAT_SVE2p1 || FEAT_SME2p1

SVE floating-point multiply-add (indexed)

The encodings in this section are decoded from SVE encodings.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	0	size		1	opc					0	0	0	0	o2	op	Zn					Zda				

Decode fields size	o2	op	Instruction Details	Condition
0x	0	0	FMLA (indexed) — Half-precision	FEAT_SVE || FEAT_SME
0x	0	1	FMLS (indexed) — Half-precision	FEAT_SVE || FEAT_SME
0x	1	0	BFMLA (indexed)	FEAT_SVE_B16B16
0x	1	1	BFMLS (indexed)	FEAT_SVE_B16B16
1x	1		UNALLOCATED	-
10	0	0	FMLA (indexed) — Single-precision	FEAT_SVE || FEAT_SME
10	0	1	FMLS (indexed) — Single-precision	FEAT_SVE || FEAT_SME
11	0	0	FMLA (indexed) — Double-precision	FEAT_SVE || FEAT_SME
11	0	1	FMLS (indexed) — Double-precision	FEAT_SVE || FEAT_SME

SVE floating-point complex multiply-add (indexed)

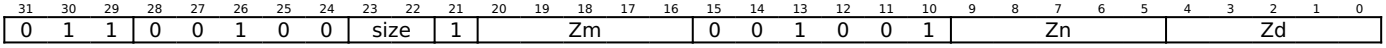
The encodings in this section are decoded from SVE encodings.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	0	size		1	opc					0	0	0	1	rot		Zn					Zda				

Decode fields size	Instruction Details	Condition
0x	UNALLOCATED	-
10	FCMLA (indexed) — Half-precision	FEAT_SVE || FEAT_SME
11	FCMLA (indexed) — Single-precision	FEAT_SVE || FEAT_SME

SVE FP clamp

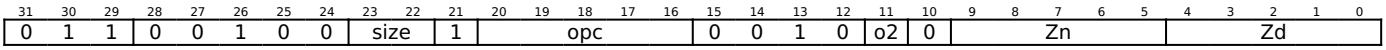
The encodings in this section are decoded from SVE encodings.



Decode fields size	Instruction Details	Condition
00	BFCLAMP	FEAT_SVE_B16B16
!= 00	FCLAMP	FEAT_SME2 || FEAT_SVE2p1

SVE floating-point multiply (indexed)

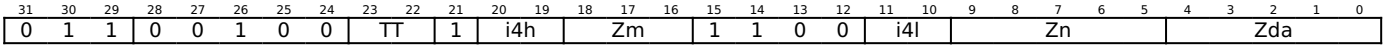
The encodings in this section are decoded from SVE encodings.



Decode fields size	o2	Instruction Details	Condition
0x	0	FMUL (indexed) — Half-precision	FEAT_SVE || FEAT_SME
0x	1	BFMUL (indexed)	FEAT_SVE_B16B16
1x	1	UNALLOCATED	-
10	0	FMUL (indexed) — Single-precision	FEAT_SVE || FEAT_SME
11	0	FMUL (indexed) — Double-precision	FEAT_SVE || FEAT_SME

SVE2 FP8 multiply-add long long (indexed)

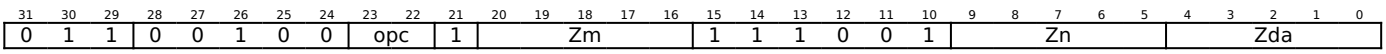
The encodings in this section are decoded from SVE encodings.



Decode fields TT	Instruction Details	Condition
00	FMLALLBB (indexed)	FEAT_SSVE_FP8FMA || (FEAT_SVE2 && FEAT_FP8FMA)
01	FMLALLBT (indexed)	FEAT_SSVE_FP8FMA || (FEAT_SVE2 && FEAT_FP8FMA)
10	FMLALLTB (indexed)	FEAT_SSVE_FP8FMA || (FEAT_SVE2 && FEAT_FP8FMA)
11	FMLALLTT (indexed)	FEAT_SSVE_FP8FMA || (FEAT_SVE2 && FEAT_FP8FMA)

SVE floating-point matrix multiply accumulate

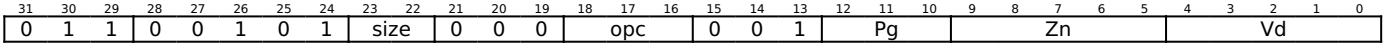
The encodings in this section are decoded from SVE encodings.



Decode fields opc	Instruction Details	Condition
00	FMMLA (widening, FP16 to FP32)	FEAT_SVE_F16F32MM
01	BFMMLA (widening)	FEAT_SVE && FEAT_BF16
10	FMMLA (non-widening) — 32-bit element	FEAT_F32MM
11	FMMLA (non-widening) — 64-bit element	FEAT_F64MM

SVE floating-point recursive reduction

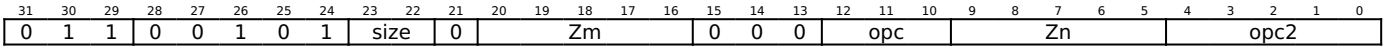
The encodings in this section are decoded from SVE encodings.



Decode fields opc	Instruction Details	Condition
000	FADDV	FEAT_SVE || FEAT_SME
001	UNALLOCATED	-
01x	UNALLOCATED	-
100	FMAXNMV	FEAT_SVE || FEAT_SME
101	FMINNMV	FEAT_SVE || FEAT_SME
110	FMAXV	FEAT_SVE || FEAT_SME
111	FMINV	FEAT_SVE || FEAT_SME

SVE floating-point arithmetic (unpredicated)

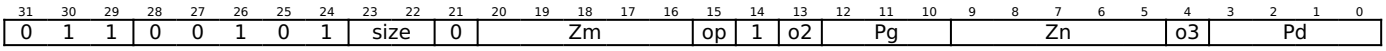
The encodings in this section are decoded from SVE encodings.



Decode fields size	opc	Instruction Details	Condition
	011	FTSMUL	FEAT_SVE
	10x	UNALLOCATED	-
	110	FRECPS	FEAT_SVE || FEAT_SME
	111	FRSQRTS	FEAT_SVE || FEAT_SME
00	000	BFADD (unpredicated)	FEAT_SVE_B16B16
00	001	BFSUB (unpredicated)	FEAT_SVE_B16B16
00	010	BFMUL (vectors, unpredicated)	FEAT_SVE_B16B16
!= 00	000	FADD (vectors, unpredicated)	FEAT_SVE || FEAT_SME
!= 00	001	FSUB (vectors, unpredicated)	FEAT_SVE || FEAT_SME
!= 00	010	FMUL (vectors, unpredicated)	FEAT_SVE || FEAT_SME

SVE floating-point compare vectors

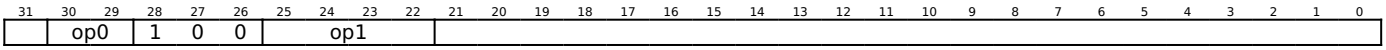
The encodings in this section are decoded from SVE encodings.



Decode fields			Instruction Details	Condition
op	o2	o3		
0	0	0	FCM<cc> (vectors) — Greater than or equal	FEAT_SVE || FEAT_SME
0	0	1	FCM<cc> (vectors) — Greater than	FEAT_SVE || FEAT_SME
0	1	0	FCM<cc> (vectors) — Equal	FEAT_SVE || FEAT_SME
0	1	1	FCM<cc> (vectors) — Not equal	FEAT_SVE || FEAT_SME
1	0	0	FCM<cc> (vectors) — Unordered	FEAT_SVE || FEAT_SME
1	0	1	FAC<cc> — Greater than or equal	FEAT_SVE || FEAT_SME
1	1	0	UNALLOCATED	-
1	1	1	FAC<cc> — Greater than	FEAT_SVE || FEAT_SME

Data Processing -- Immediate

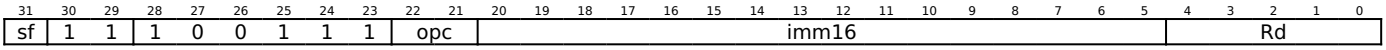
The encodings in this section are decoded from A64 instruction set encoding.



Decode fields		Instruction details
op0	op1	
11	111x	Data-processing (1 source immediate)
	00xx	PC-rel. addressing
	010x	Add/subtract (immediate)
	0110	Add/subtract (immediate, with tags)
	0111	Min/max (immediate)
	100x	Logical (immediate)
	101x	Move wide (immediate)
	110x	Bitfield
!= 11	111x	Extract

Data-processing (1 source immediate)

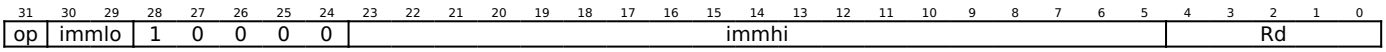
The encodings in this section are decoded from Data Processing -- Immediate.



Decode fields			Instruction Details	Condition
sf	opc	Rd		
0			UNALLOCATED	-
1	0x	!= 11111	UNALLOCATED	-
1	00	11111	AUTIASPPC	FEAT_PAuth_LR
1	01	11111	AUTIBSPPC	FEAT_PAuth_LR
1	1x		UNALLOCATED	-

PC-rel. addressing

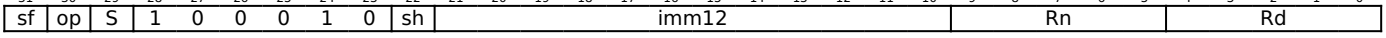
The encodings in this section are decoded from Data Processing -- Immediate.



Decode fields		Instruction Details
op		
0		ADR
1		ADRP

Add/subtract (immediate)

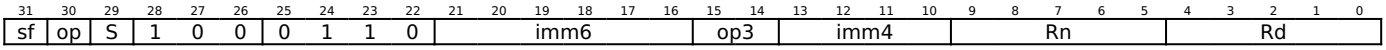
The encodings in this section are decoded from Data Processing -- Immediate.



Decode fields			Instruction Details
sf	op	S	
0	0	0	ADD (immediate) — 32-bit
0	0	1	ADDS (immediate) — 32-bit
0	1	0	SUB (immediate) — 32-bit
0	1	1	SUBS (immediate) — 32-bit
1	0	0	ADD (immediate) — 64-bit
1	0	1	ADDS (immediate) — 64-bit
1	1	0	SUB (immediate) — 64-bit
1	1	1	SUBS (immediate) — 64-bit

Add/subtract (immediate, with tags)

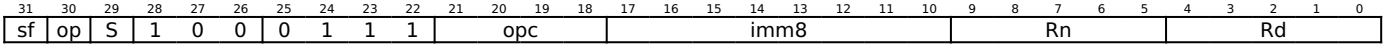
The encodings in this section are decoded from Data Processing -- Immediate.



Decode fields			Instruction Details	Condition
sf	op	S		
0			UNALLOCATED	-
1		1	UNALLOCATED	-
1	0	0	ADDG	FEAT_MTE
1	1	0	SUBG	FEAT_MTE

Min/max (immediate)

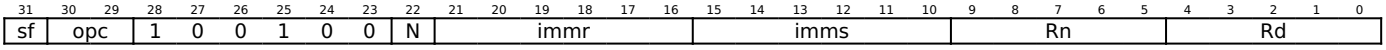
The encodings in this section are decoded from Data Processing -- Immediate.



Decode fields				Instruction Details	Condition
sf	op	S	opc		
	0	0	!= 00xx	UNALLOCATED	-
	0	1		UNALLOCATED	-
	1			UNALLOCATED	-
0	0	0	0000	SMAX (immediate) — 32-bit	FEAT_CSSC
0	0	0	0001	UMAX (immediate) — 32-bit	FEAT_CSSC
0	0	0	0010	SMIN (immediate) — 32-bit	FEAT_CSSC
0	0	0	0011	UMIN (immediate) — 32-bit	FEAT_CSSC
1	0	0	0000	SMAX (immediate) — 64-bit	FEAT_CSSC
1	0	0	0001	UMAX (immediate) — 64-bit	FEAT_CSSC
1	0	0	0010	SMIN (immediate) — 64-bit	FEAT_CSSC
1	0	0	0011	UMIN (immediate) — 64-bit	FEAT_CSSC

Logical (immediate)

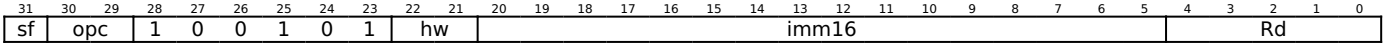
The encodings in this section are decoded from Data Processing -- Immediate.



Decode fields			Instruction Details
sf	opc	N	
0		1	UNALLOCATED
0	00	0	AND (immediate) — 32-bit
0	01	0	ORR (immediate) — 32-bit
0	10	0	EOR (immediate) — 32-bit
0	11	0	ANDS (immediate) — 32-bit
1	00		AND (immediate) — 64-bit
1	01		ORR (immediate) — 64-bit
1	10		EOR (immediate) — 64-bit
1	11		ANDS (immediate) — 64-bit

Move wide (immediate)

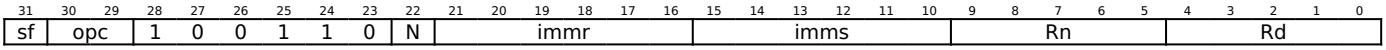
The encodings in this section are decoded from Data Processing -- Immediate.



Decode fields			Instruction Details
sf	opc	hw	
	01	0x	UNALLOCATED
0		1x	UNALLOCATED
0	00	0x	MOVN — 32-bit
0	10	0x	MOVZ — 32-bit
0	11	0x	MOVK — 32-bit
1	00		MOVN — 64-bit
1	01	1x	UNALLOCATED
1	10		MOVZ — 64-bit
1	11		MOVK — 64-bit

Bitfield

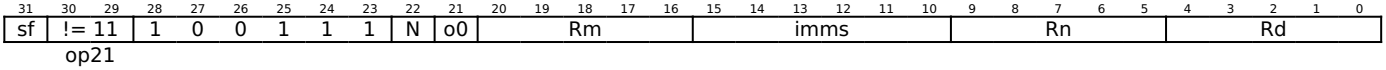
The encodings in this section are decoded from Data Processing -- Immediate.



Decode fields			Instruction Details
sf	opc	N	
0		1	UNALLOCATED
0	00	0	SBFM — 32-bit
0	01	0	BFM — 32-bit
0	10	0	UBFM — 32-bit
0	11	0	UNALLOCATED
1		0	UNALLOCATED
1	00	1	SBFM — 64-bit
1	01	1	BFM — 64-bit
1	10	1	UBFM — 64-bit
1	11	1	UNALLOCATED

Extract

The encodings in this section are decoded from Data Processing -- Immediate.

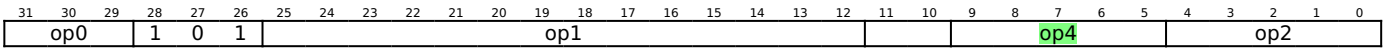


The following constraints also apply to this encoding: op21 != '11'

Decode fields					Instruction Details
sf	op21	N	o0	imms	
	00		1		UNALLOCATED
	01				UNALLOCATED
	10				UNALLOCATED
0	00	0	0	0xxxxx	EXTR — 32-bit
0	00	0	0	1xxxxx	UNALLOCATED
0	00	1	0		UNALLOCATED
1	00	0	0		UNALLOCATED
1	00	1	0		EXTR — 64-bit

Branches, Exception Generating and System instructions

The encodings in this section are decoded from A64 instruction set encoding.

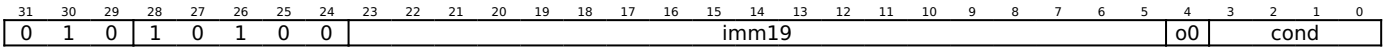


Decode fields				Instruction details	Condition
op0	op1	op4	op2		
010	00xxxxxxxxxxxx		Conditional branch (immediate)	Conditional branch (immediate)	-
010	01xxxxxxxxxxxx		Miscellaneous branch (immediate)	Miscellaneous branch (immediate)	-
010	1xxxxxxxxxxxxx			UNALLOCATED	-
011	00xxxxxxxx1xxx		Compare bytes/halfwords in registers and branch	Compare bytes/halfwords in registers and branch	-
01x	1xxxxxxxxxxxxx		UNALLOCATED	-	
110	00xxxxxxxxxxxx		Exception generation	Exception generation	-
110	010000000x0010		HINTE	HINTE	FEAT_HINTE
110	010000000x00x1			UNALLOCATED	-
110	01000000100010		HINTE	HINTE	FEAT_HINTE
110	010000001000x1			UNALLOCATED	-
110	01000000110001		System instructions with register argument	System instructions with register argument	-
110	01000000110010		11111	Hints	-
110	01000000110010		!= 11111	HINTE	FEAT_HINTE
110	01000000110011		Barriers	Barriers	-
110	01000001xx0010		HINTE	HINTE	FEAT_HINTE
110	01000001xx00x1			UNALLOCATED	-
110	0100000xxx0100		PSTATE	PSTATE	-
110	0100000xxx0101			UNALLOCATED	-
110	0100000xxx011x			UNALLOCATED	-
110	0100100xxx0010		HINTE	HINTE	FEAT_HINTE
110	0100100xxx00x1			UNALLOCATED	-
110	0100100xxx01xx			UNALLOCATED	-
110	0100x00xxx0000			UNALLOCATED	-
110	0100x00xxx1xxx			UNALLOCATED	-
110	0100x01xxxxxxx		System instructions	System instructions	-
110	0100x1xxxxxxxx		System register move	System register move	-
110	0101x00xxxxxxx			UNALLOCATED	-
110	0101x01xxxxxxx		System pair instructions	System pair instructions	-
110	0101x1xxxxxxxx		System register pair move	System register pair move	-
110	011xxxxxxxxxxx			UNALLOCATED	-
110	1xxxxxxxxxxxxx		Unconditional branch (register)	Unconditional branch (register)	-
111	00xxxxxxxx1xxx			UNALLOCATED	-
111	1xxxxxxxxxxxxx		UNALLOCATED	-	
x00			Unconditional branch (immediate)	Unconditional branch (immediate)	-
x01	0xxxxxxxxxxxxx		Compare and branch (immediate)	Compare and branch (immediate)	-
x01	1xxxxxxxxxxxxx		Test and branch (immediate)	Test and branch (immediate)	-
x11	00xxxxxxxx00xx		Compare registers and branch	Compare registers and branch	-

x11	00xxxxxxxx01xx			UNALLOCATED	-
x11	01xxxxxxxxx0xx		~~Compare register with immediate and branch~~	Compare register with immediate and branch	-
x11	01xxxxxxxxx1xx			UNALLOCATED	-
x11	10xxxxx0000000	1001x		Test bit and fault	-
x11	10xxxxx0010000	1001x		UNALLOCATED	-
x11	10xxxxx01x0000	1001x		UNALLOCATED	-
x11	10xxxxx1xx0000	1001x		UNALLOCATED	-
x11	10xxxxxxxx0000	0xxxx		Compare and fault (immediate)	-
x11	10xxxxxxxx0000	10000		Compare and fault (register)	-
x11	10xxxxxxxx0000	10001		UNALLOCATED	-
x11	10xxxxxxxx0000	101xx		UNALLOCATED	-
x11	10xxxxxxxx0000	11xxx		UNALLOCATED	-
x11	10xxxxxxxx0100			UNALLOCATED	-
x11	10xxxxxxxx0x01			UNALLOCATED	-
x11	10xxxxxxxx0x1x			UNALLOCATED	-
x11	10xxxxxxxx1xxx			UNALLOCATED	-
x11	11xxxxxxxxxxxx			UNALLOCATED	-

Conditional branch (immediate)

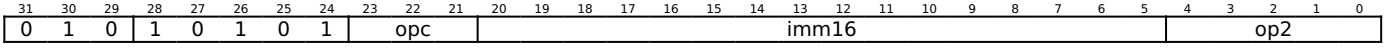
The encodings in this section are decoded from Branches, Exception Generating and System instructions.



Decode fields o0	Instruction Details	Condition
0	B.cond	-
1	BC.cond	FEAT_HBC

Miscellaneous branch (immediate)

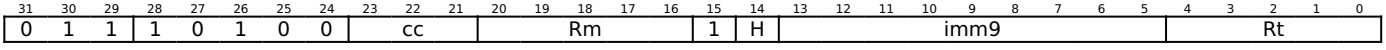
The encodings in this section are decoded from Branches, Exception Generating and System instructions.



Decode fields opc	Decode fields op2	Instruction Details	Condition
00x	!= 11111	UNALLOCATED	-
000	11111	RETAASPPC, RETABSPPC — RETAASPPC	FEAT_PAuth_LR
001	11111	RETAASPPC, RETABSPPC — RETABSPPC	FEAT_PAuth_LR
!= 00x		UNALLOCATED	-

Compare bytes/halfwords in registers and branch

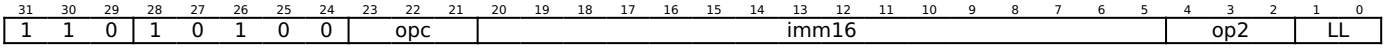
The encodings in this section are decoded from Branches, Exception Generating and System instructions.



Decode fields		Instruction Details	Condition
cc	H		
000	0	CBB<cc> — Greater than	FEAT_CMPBR
000	1	CBH<cc> — Greater than	FEAT_CMPBR
001	0	CBB<cc> — Greater than or equal	FEAT_CMPBR
001	1	CBH<cc> — Greater than or equal	FEAT_CMPBR
010	0	CBB<cc> — Higher	FEAT_CMPBR
010	1	CBH<cc> — Higher	FEAT_CMPBR
011	0	CBB<cc> — Higher or same	FEAT_CMPBR
011	1	CBH<cc> — Higher or same	FEAT_CMPBR
10x		UNALLOCATED	-
110	0	CBB<cc> — Equal	FEAT_CMPBR
110	1	CBH<cc> — Equal	FEAT_CMPBR
111	0	CBB<cc> — Not equal	FEAT_CMPBR
111	1	CBH<cc> — Not equal	FEAT_CMPBR

Exception generation

The encodings in this section are decoded from Branches, Exception Generating and System instructions.



Decode fields			Instruction Details	Condition
opc	op2	LL		
x01	001		UNALLOCATED	-
x01	01x		UNALLOCATED	-
x11	0xx		UNALLOCATED	-
000	000	00	UNALLOCATED	-
000	000	01	SVC	-
000	000	10	HVC	-
000	000	11	SMC	-
000	!= 000		UNALLOCATED	-
001	000	00	BRK	-
001	000	!= 00	UNALLOCATED	-
001	1xx		FLT.cond	FEAT_CFLT
010		!= 00	UNALLOCATED	-
010	000	00	HLT	-
010	!= 000	00	UNALLOCATED	-
011	1xx		UNALLOCATED	-
1x0			UNALLOCATED	-
1x1	1xx		UNALLOCATED	-
101	000	00	UNALLOCATED	-
101	000	01	DCPS1	-
101	000	10	DCPS2	-
101	000	11	DCPS3	-

Decode fields			Instruction Details
opc	op2	LL	
	!= 000		UNALLOCATED
x11	000		UNALLOCATED
000	000	00	UNALLOCATED
000	000	01	SVC
000	000	10	HVC
000	000	11	SMC
001	000	00	BRK
001	000	!= 00	UNALLOCATED
010	000	00	HLT
010	000	!= 00	UNALLOCATED
1x0	000		UNALLOCATED
101	000	00	UNALLOCATED
101	000	01	DCPS1
101	000	10	DCPS2
101	000	11	DCPS3

System instructions with register argument

The encodings in this section are decoded from Branches, Exception Generating and System instructions.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	1	0	1	0	1	0	0	0	0	0	0	1	1	0	0	0	1	CRm				op2			Rt				

Decode fields		Instruction Details	Condition
CRm	op2		
0000	000	WFET	FEAT_WFxT
0000	001	WFIT	FEAT_WFxT
0000	!= 00x	UNALLOCATED	-
!= 0000		UNALLOCATED	-

Hints

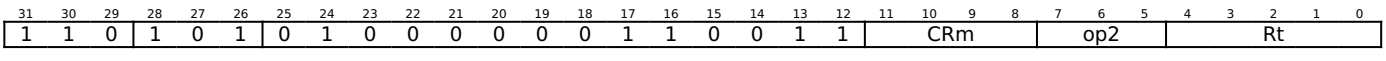
The encodings in this section are decoded from Branches, Exception Generating and System instructions.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	1	0	1	0	1	0	0	0	0	0	0	1	1	0	0	1	0	CRm				op2			1	1	1	1	1

Decode fields		Instruction Details	Condition
CRm	op2		
		HINT	-
0000	000	NOP	-
0000	001	YIELD	-
0000	010	WFE	-
0000	011	WFI	-
0000	100	SEV	-
0000	101	SEVL	-
0000	110	DGH	FEAT_DGH
0000	111	XPACD, XPACI, XPACLRI	FEAT_PAuth
0001	000	PACIA, PACIA1716, PACIASP, PACIAZ, PACIZA — PACIA1716	FEAT_PAuth
0001	010	PACIB, PACIB1716, PACIBSP, PACIBZ, PACIZB — PACIB1716	FEAT_PAuth
0001	100	AUTIA, AUTIA1716, AUTIASP, AUTIAZ, AUTIZA — AUTIA1716	FEAT_PAuth
0001	110	AUTIB, AUTIB1716, AUTIBSP, AUTIBZ, AUTIZB — AUTIB1716	FEAT_PAuth
0010	000	ESB	FEAT_RAS
0010	001	PSB	FEAT_SPE
0010	010	TSB	FEAT_TRF
0010	011	GCSB	FEAT_GCS
0010	100	CSDB	-
0010	110	CLRBHB	FEAT_CLRBHB
0011	000	PACIA, PACIA1716, PACIASP, PACIAZ, PACIZA — PACIAZ	FEAT_PAuth
0011	001	PACIA, PACIA1716, PACIASP, PACIAZ, PACIZA — PACIASP	FEAT_PAuth
0011	010	PACIB, PACIB1716, PACIBSP, PACIBZ, PACIZB — PACIBZ	FEAT_PAuth
0011	011	PACIB, PACIB1716, PACIBSP, PACIBZ, PACIZB — PACIBSP	FEAT_PAuth
0011	100	AUTIA, AUTIA1716, AUTIASP, AUTIAZ, AUTIZA — AUTIAZ	FEAT_PAuth
0011	101	AUTIA, AUTIA1716, AUTIASP, AUTIAZ, AUTIZA — AUTIASP	FEAT_PAuth
0011	110	AUTIB, AUTIB1716, AUTIBSP, AUTIBZ, AUTIZB — AUTIBZ	FEAT_PAuth
0011	111	AUTIB, AUTIB1716, AUTIBSP, AUTIBZ, AUTIZB — AUTIBSP	FEAT_PAuth
0100	xx0	BTI	FEAT_BTI
0100	111	PACM	FEAT_PAuth_LR
0101	000	CHKFEAT	FEAT_CHK
0110	00x	STSHH	FEAT_PCDPHINT
0110	01x	SHUH	FEAT_CMH
0110	100	STCPH	FEAT_CMH
0110	11x	SRLS	FEAT_RLCS
0111	000	SLBND	FEAT_RLCS

Barriers

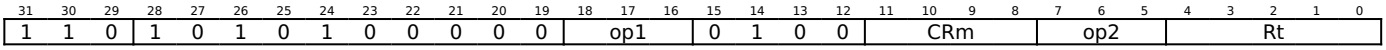
The encodings in this section are decoded from Branches, Exception Generating and System instructions.



Decode fields			Instruction Details	Condition
CRm	op2	Rt		
		!= 11111	UNALLOCATED	-
	000	11111	UNALLOCATED	-
	010	11111	CLREX	-
	100	11111	DSB — Memory barrier	-
	101	11111	DMB	-
	110	11111	ISB	-
	111	11111	SB	FEAT_SB
xx10	001	11111	DSB — Memory nXS barrier	FEAT_XS
xx10	011	11111	UNALLOCATED	-
!= xx10	0x1	11111	UNALLOCATED	-

PSTATE

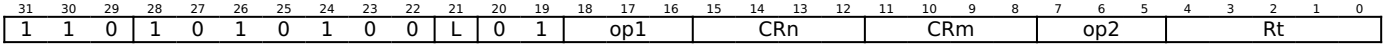
The encodings in this section are decoded from Branches, Exception Generating and System instructions.



Decode fields			Instruction Details	Condition
op1	op2	Rt		
		!= 11111	UNALLOCATED	-
000	000	11111	CFINV	FEAT_FlagM
000	001	11111	XAFLAG	FEAT_FlagM2
000	010	11111	AXFLAG	FEAT_FlagM2
000	011	11111	MSR (immediate)	-
000	1xx	11111	MSR (immediate)	-
!= 000		11111	MSR (immediate)	-

System instructions

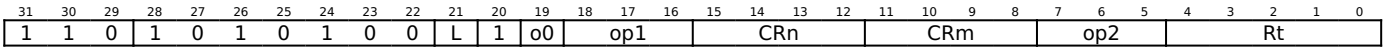
The encodings in this section are decoded from Branches, Exception Generating and System instructions.



Decode fields	Instruction Details
L	
0	SYS
1	SYSL

System register move

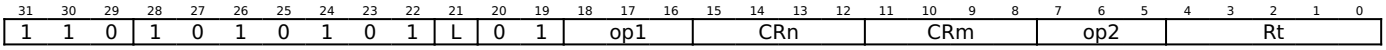
The encodings in this section are decoded from Branches, Exception Generating and System instructions.



Decode fields	Instruction Details
L	
0	MSR (register)
1	MRS

System pair instructions

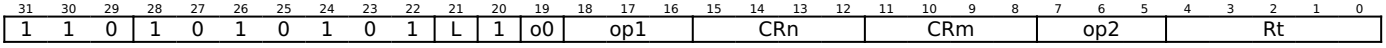
The encodings in this section are decoded from Branches, Exception Generating and System instructions.



Decode fields L	Instruction Details	Condition
0	SYSP	FEAT_SYSINSTR128
1	UNALLOCATED	-

System register pair move

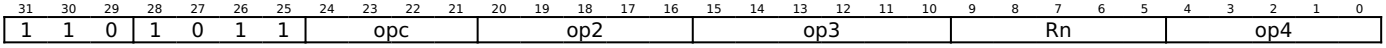
The encodings in this section are decoded from Branches, Exception Generating and System instructions.



Decode fields L	Instruction Details	Condition
0	MSRR	FEAT_SYSREG128
1	MRRS	FEAT_SYSREG128

Unconditional branch (register)

The encodings in this section are decoded from Branches, Exception Generating and System instructions.

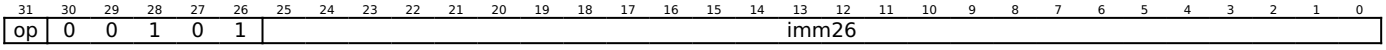


Decode fields					Instruction Details	Condition
opc	op2	op3	Rn	op4		
	11111	!= 0000xx			UNALLOCATED	-
	!= 11111				UNALLOCATED	-
00x0	11111	000000		!= 00000	UNALLOCATED	-
00x0	11111	000001			UNALLOCATED	-
000x	11111	00001x		!= 11111	UNALLOCATED	-
0000	11111	000000		00000	BR	-
0000	11111	000010		11111	BRAA, BRAAZ, BRAB, BRABZ — Key A, zero modifier	FEAT_PAuth
0000	11111	000011		11111	BRAA, BRAAZ, BRAB, BRABZ — Key B, zero modifier	FEAT_PAuth
0001	11111	000000		00000	BLR	-
0001	11111	000000		!= 00000	UNALLOCATED	-
0001	11111	000001			UNALLOCATED	-
0001	11111	000010		11111	BLRAA, BLRAAZ, BLRAB, BLRABZ — Key A, zero modifier	FEAT_PAuth
0001	11111	000011		11111	BLRAA, BLRAAZ, BLRAB, BLRABZ — Key B, zero modifier	FEAT_PAuth
0010	11111	000000		00000	RET	-
0010	11111	00001x	!= 11111		UNALLOCATED	-
0010	11111	000010	11111	11111	RETAA, RETAB — RETAA	FEAT_PAuth
0010	11111	000010	11111	!= 11111	RETAASPPCR, RETABSPPCR — RETAASPPCR	FEAT_PAuth_LR
0010	11111	000011	11111	11111	RETAA, RETAB — RETAB	FEAT_PAuth
0010	11111	000011	11111	!= 11111	RETAASPPCR, RETABSPPCR — RETABSPPCR	FEAT_PAuth_LR
0011	11111	0000xx			UNALLOCATED	-
010x	11111	0000xx	!= 11111		UNALLOCATED	-
010x	11111	000000	11111	!= 00000	UNALLOCATED	-
010x	11111	000001	11111		UNALLOCATED	-
010x	11111	00001x	11111	xxxx0	UNALLOCATED	-
0100	11111	000000	11111	00000	ERET	-
0100	11111	00001x	11111	0xxx1	UNALLOCATED	-
0100	11111	00001x	11111	10xx1	UNALLOCATED	-
0100	11111	00001x	11111	110x1	UNALLOCATED	-
0100	11111	00001x	11111	11101	UNALLOCATED	-
0100	11111	000010	11111	11111	ERETAA, ERETAB — ERETAA	FEAT_PAuth
0100	11111	000011	11111	11111	ERETAA, ERETAB — ERETAB	FEAT_PAuth
0101	11111	000000	11111	00000	DRPS	-
0101	11111	00001x	11111	xxxx1	UNALLOCATED	-
011x	11111	0000xx			UNALLOCATED	-

Decode fields					Instruction Details	Condition
opc	op2	op3	Rn	op4		
100x	11111	00000x			UNALLOCATED	-
1000	11111	000010			BRAA, BRAAZ, BRAB, BRABZ — Key A, register modifier	FEAT_PAuth
1000	11111	000011			BRAA, BRAAZ, BRAB, BRABZ — Key B, register modifier	FEAT_PAuth
1001	11111	000010			BLRAA, BLRAAZ, BLRAB, BLRABZ — Key A, register modifier	FEAT_PAuth
1001	11111	000011			BLRAA, BLRAAZ, BLRAB, BLRABZ — Key B, register modifier	FEAT_PAuth
101x	11111	0000xx			UNALLOCATED	-
11xx	11111	0000xx			UNALLOCATED	-

Unconditional branch (immediate)

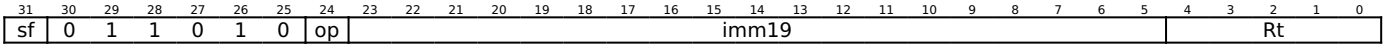
The encodings in this section are decoded from Branches, Exception Generating and System instructions.



Decode fields	Instruction Details
op	
0	B
1	BL

Compare and branch (immediate)

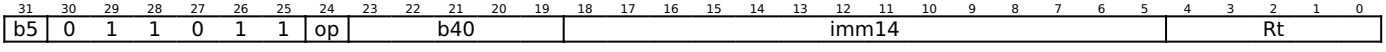
The encodings in this section are decoded from Branches, Exception Generating and System instructions.



Decode fields		Instruction Details
sf	op	
0	0	CBZ — 32-bit
0	1	CBNZ — 32-bit
1	0	CBZ — 64-bit
1	1	CBNZ — 64-bit

Test and branch (immediate)

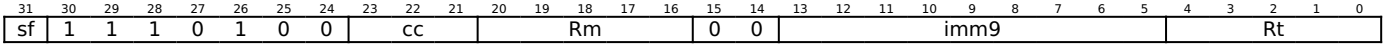
The encodings in this section are decoded from Branches, Exception Generating and System instructions.



Decode fields	Instruction Details
op	
0	TBZ
1	TBNZ

Compare registers and branch

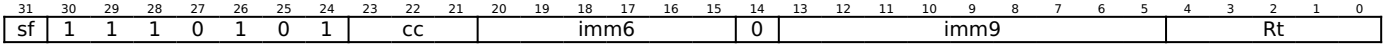
The encodings in this section are decoded from Branches, Exception Generating and System instructions.



Decode fields sf	cc	Instruction Details	Condition
	10x	UNALLOCATED	-
0	000	CB<cc> (register) — 32-bit greater than	FEAT_CMPBR
0	001	CB<cc> (register) — 32-bit greater than or equal	FEAT_CMPBR
0	010	CB<cc> (register) — 32-bit higher	FEAT_CMPBR
0	011	CB<cc> (register) — 32-bit higher or same	FEAT_CMPBR
0	110	CB<cc> (register) — 32-bit equal	FEAT_CMPBR
0	111	CB<cc> (register) — 32-bit not equal	FEAT_CMPBR
1	000	CB<cc> (register) — 64-bit greater than	FEAT_CMPBR
1	001	CB<cc> (register) — 64-bit greater than or equal	FEAT_CMPBR
1	010	CB<cc> (register) — 64-bit higher	FEAT_CMPBR
1	011	CB<cc> (register) — 64-bit higher or same	FEAT_CMPBR
1	110	CB<cc> (register) — 64-bit equal	FEAT_CMPBR
1	111	CB<cc> (register) — 64-bit not equal	FEAT_CMPBR

Compare register with immediate and branch

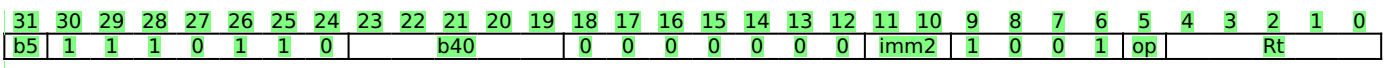
The encodings in this section are decoded from Branches, Exception Generating and System instructions.



Decode fields sf	cc	Instruction Details	Condition
	10x	UNALLOCATED	-
0	000	CB<cc> (immediate) — 32-bit greater than	FEAT_CMPBR
0	001	CB<cc> (immediate) — 32-bit less than	FEAT_CMPBR
0	010	CB<cc> (immediate) — 32-bit higher	FEAT_CMPBR
0	011	CB<cc> (immediate) — 32-bit lower	FEAT_CMPBR
0	110	CB<cc> (immediate) — 32-bit equal	FEAT_CMPBR
0	111	CB<cc> (immediate) — 32-bit not equal	FEAT_CMPBR
1	000	CB<cc> (immediate) — 64-bit greater than	FEAT_CMPBR
1	001	CB<cc> (immediate) — 64-bit less than	FEAT_CMPBR
1	010	CB<cc> (immediate) — 64-bit higher	FEAT_CMPBR
1	011	CB<cc> (immediate) — 64-bit lower	FEAT_CMPBR
1	110	CB<cc> (immediate) — 64-bit equal	FEAT_CMPBR
1	111	CB<cc> (immediate) — 64-bit not equal	FEAT_CMPBR

Test bit and fault

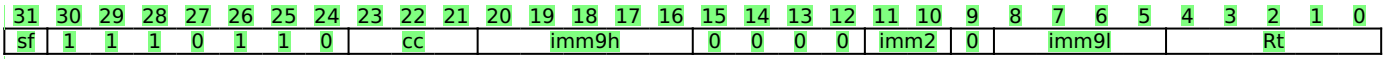
The encodings in this section are decoded from Branches, Exception Generating and System instructions.



Decode fields op	Instruction Details	Condition
0	TFLTZ	FEAT_CFLT
1	TFLTNZ	FEAT_CFLT

Compare and fault (immediate)

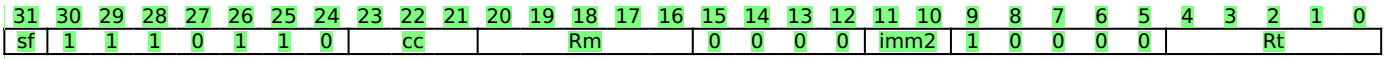
The encodings in this section are decoded from Branches, Exception Generating and System instructions.



Decode fields		Instruction Details	Condition
sf	cc		
	11x	UNALLOCATED	-
0	000	CFLT<cc> (immediate) — 32-bit greater than	FEAT_CFLT
0	001	CFLT<cc> (immediate) — 32-bit less than	FEAT_CFLT
0	010	CFLT<cc> (immediate) — 32-bit higher	FEAT_CFLT
0	011	CFLT<cc> (immediate) — 32-bit lower	FEAT_CFLT
0	100	CFLT<cc> (immediate) — 32-bit equal	FEAT_CFLT
0	101	CFLT<cc> (immediate) — 32-bit not equal	FEAT_CFLT
1	000	CFLT<cc> (immediate) — 64-bit greater than	FEAT_CFLT
1	001	CFLT<cc> (immediate) — 64-bit less than	FEAT_CFLT
1	010	CFLT<cc> (immediate) — 64-bit higher	FEAT_CFLT
1	011	CFLT<cc> (immediate) — 64-bit lower	FEAT_CFLT
1	100	CFLT<cc> (immediate) — 64-bit equal	FEAT_CFLT
1	101	CFLT<cc> (immediate) — 64-bit not equal	FEAT_CFLT

Compare and fault (register)

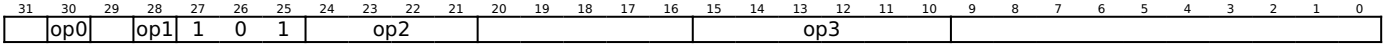
The encodings in this section are decoded from Branches, Exception Generating and System instructions.



Decode fields		Instruction Details	Condition
sf	cc		
	10x	UNALLOCATED	-
0	000	CFLT<cc> (register) — 32-bit greater than	FEAT_CFLT
0	001	CFLT<cc> (register) — 32-bit greater than or equal	FEAT_CFLT
0	010	CFLT<cc> (register) — 32-bit higher	FEAT_CFLT
0	011	CFLT<cc> (register) — 32-bit higher or same	FEAT_CFLT
0	110	CFLT<cc> (register) — 32-bit equal	FEAT_CFLT
0	111	CFLT<cc> (register) — 32-bit not equal	FEAT_CFLT
1	000	CFLT<cc> (register) — 64-bit greater than	FEAT_CFLT
1	001	CFLT<cc> (register) — 64-bit greater than or equal	FEAT_CFLT
1	010	CFLT<cc> (register) — 64-bit higher	FEAT_CFLT
1	011	CFLT<cc> (register) — 64-bit higher or same	FEAT_CFLT
1	110	CFLT<cc> (register) — 64-bit equal	FEAT_CFLT
1	111	CFLT<cc> (register) — 64-bit not equal	FEAT_CFLT

Data Processing -- Register

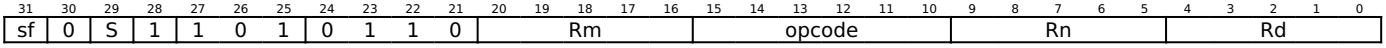
The encodings in this section are decoded from A64 instruction set encoding.



Decode fields				Instruction details
op0	op1	op2	op3	
0	1	0110		Data-processing (2 source)
1	1	0110		Data-processing (1 source)
	0	0xxx		Logical (shifted register)
	0	1xx0		Add/subtract (shifted register)
	0	1xx1		Add/subtract (extended register)
	1	0000	000000	Add/subtract (with carry)
	1	0000	001xxx	Add/subtract (checked pointer)
	1	0000	011xxx	UNALLOCATED
	1	0000	100000	UNALLOCATED
	1	0000	1x1xxx	UNALLOCATED
	1	0000	x00001	Rotate right into flags
	1	0000	x0010x	UNALLOCATED
	1	0000	x10x0x	UNALLOCATED
	1	0000	xx0010	Evaluate into flags
	1	0000	xx0011	UNALLOCATED
	1	0000	xx011x	UNALLOCATED
	1	0010	xxxx0x	Conditional compare (register)
	1	0010	xxxx1x	Conditional compare (immediate)
	1	0100		Conditional select
	1	0xx1		UNALLOCATED
	1	1xxx		Data-processing (3 source)

Data-processing (2 source)

The encodings in this section are decoded from Data Processing -- Register.



Decode fields			Instruction Details	Condition
sf	S	opcode		
		1xxxxx	UNALLOCATED	-
	0	00011x	UNALLOCATED	-
	1	00001x	UNALLOCATED	-
	1	0001xx	UNALLOCATED	-
	1	001xxx	UNALLOCATED	-
	1	01xxxx	UNALLOCATED	-
0		00000x	UNALLOCATED	-
0	0	0x11xx	UNALLOCATED	-
0	0	000010	UDIV (quotient) — 32-bit	-
0	0	000011	SDIV (quotient) — 32-bit	-
0	0	00010x	UNALLOCATED	-
0	0	001000	LSLV — 32-bit	-
0	0	001001	LSRV — 32-bit	-
0	0	001010	ASRV — 32-bit	-
0	0	001011	RORV — 32-bit	-
0	0	010x11	UNALLOCATED	-
0	0	010000	CRC32B, CRC32H, CRC32W, CRC32X — CRC32B	FEAT_CRC32
0	0	010001	CRC32B, CRC32H, CRC32W, CRC32X — CRC32H	FEAT_CRC32
0	0	010010	CRC32B, CRC32H, CRC32W, CRC32X — CRC32W	FEAT_CRC32
0	0	010100	CRC32CB, CRC32CH, CRC32CW, CRC32CX — CRC32CB	FEAT_CRC32
0	0	010101	CRC32CB, CRC32CH, CRC32CW, CRC32CX — CRC32CH	FEAT_CRC32
0	0	010110	CRC32CB, CRC32CH, CRC32CW, CRC32CX — CRC32CW	FEAT_CRC32
0	0	011000	SMAX (register) — 32-bit	FEAT_CSSC
0	0	011001	UMAX (register) — 32-bit	FEAT_CSSC
0	0	011010	SMIN (register) — 32-bit	FEAT_CSSC
0	0	011011	UMIN (register) — 32-bit	FEAT_CSSC
1		000001	UNALLOCATED	-
1	0	000000	SUBP	FEAT_MTE
1	0	000010	UDIV (quotient) — 64-bit	-
1	0	000011	SDIV (quotient) — 64-bit	-
1	0	000100	IRG	FEAT_MTE
1	0	000101	GMI	FEAT_MTE
1	0	001000	LSLV — 64-bit	-
1	0	001001	LSRV — 64-bit	-
1	0	001010	ASRV — 64-bit	-
1	0	001011	RORV — 64-bit	-
1	0	001100	PACGA	FEAT_PAuth
1	0	001101	UNALLOCATED	-
1	0	00111x	UNALLOCATED	-
1	0	010xx0	UNALLOCATED	-
1	0	010001	UNALLOCATED	-
1	0	010011	CRC32B, CRC32H, CRC32W, CRC32X — CRC32X	FEAT_CRC32
1	0	010101	UNALLOCATED	-
1	0	010111	CRC32CB, CRC32CH, CRC32CW, CRC32CX — CRC32CX	FEAT_CRC32
1	0	011000	SMAX (register) — 64-bit	FEAT_CSSC

Decode fields			Instruction Details	Condition
sf	S	opcode		
1	0	011001	UMAX (register) — 64-bit	FEAT_CSSC
1	0	011010	SMIN (register) — 64-bit	FEAT_CSSC
1	0	011011	UMIN (register) — 64-bit	FEAT_CSSC
1	0	0111xx	UNALLOCATED	-
1	1	000000	SUBPS	FEAT_MTE

Data-processing (1 source)

The encodings in this section are decoded from Data Processing -- Register.

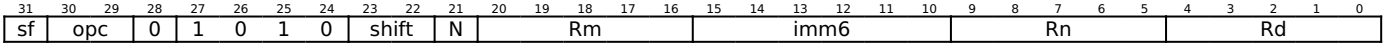
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
sf	1	S	1	1	0	1	0	1	1	0	opcode2					opcode						Rn					Rd				

			Decode fields			Instruction Details	Condition
sf	S	opcode2	opcode	Rn	Rd		
	0	00000	001001			UNALLOCATED	-
	0	00000	00101x			UNALLOCATED	-
	0	00000	0011xx			UNALLOCATED	-
	0	00000	01xxxx			UNALLOCATED	-
	0	00000	1xxxxx			UNALLOCATED	-
	0	!= 0000x				UNALLOCATED	-
	1					UNALLOCATED	-
0	0	00000	000000			RBIT — 32-bit	-
0	0	00000	000001			REV16 — 32-bit	-
0	0	00000	000010			REV — 32-bit	-
0	0	00000	000011			UNALLOCATED	-
0	0	00000	000100			CLZ — 32-bit	-
0	0	00000	000101			CLS — 32-bit	-
0	0	00000	000110			CTZ — 32-bit	FEAT_CSSC
0	0	00000	000111			CNT — 32-bit	FEAT_CSSC
0	0	00000	001000			ABS — 32-bit	FEAT_CSSC
0	0	00001				UNALLOCATED	-
1	0	00000	000000			RBIT — 64-bit	-
1	0	00000	000001			REV16 — 64-bit	-
1	0	00000	000010			REV32	-
1	0	00000	000011			REV — 64-bit	-
1	0	00000	000100			CLZ — 64-bit	-
1	0	00000	000101			CLS — 64-bit	-
1	0	00000	000110			CTZ — 64-bit	FEAT_CSSC
1	0	00000	000111			CNT — 64-bit	FEAT_CSSC
1	0	00000	001000			ABS — 64-bit	FEAT_CSSC
1	0	00001	000000			PACIA, PACIA1716, PACIASP, PACIAZ, PACIZA — PACIA	FEAT_PAuth
1	0	00001	000001			PACIB, PACIB1716, PACIBSP, PACIBZ, PACIZB — PACIB	FEAT_PAuth
1	0	00001	000010			PACDA, PACDZA — PACDA	FEAT_PAuth
1	0	00001	000011			PACDB, PACDZB — PACDB	FEAT_PAuth
1	0	00001	000100			AUTIA, AUTIA1716, AUTIASP, AUTIAZ, AUTIZA — AUTIA	FEAT_PAuth
1	0	00001	000101			AUTIB, AUTIB1716, AUTIBSP, AUTIBZ, AUTIZB — AUTIB	FEAT_PAuth
1	0	00001	000110			AUTDA, AUTDZA — AUTDA	FEAT_PAuth
1	0	00001	000111			AUTDB, AUTDZB — AUTDB	FEAT_PAuth
1	0	00001	001xxx	!= 11111		UNALLOCATED	-
1	0	00001	001000	11111		PACIA, PACIA1716, PACIASP, PACIAZ, PACIZA — PACIZA	FEAT_PAuth
1	0	00001	001001	11111		PACIB, PACIB1716, PACIBSP, PACIBZ, PACIZB — PACIZB	FEAT_PAuth
1	0	00001	001010	11111		PACDA, PACDZA — PACDZA	FEAT_PAuth
1	0	00001	001011	11111		PACDB, PACDZB — PACDZB	FEAT_PAuth
1	0	00001	001100	11111		AUTIA, AUTIA1716, AUTIASP, AUTIAZ, AUTIZA — AUTIZA	FEAT_PAuth

Decode fields						Instruction Details	Condition
sf	S	opcode2	opcode	Rn	Rd		
1	0	00001	001101	11111		AUTIB, AUTIB1716, AUTIBSP, AUTIBZ, AUTIZB — AUTIZB	FEAT_PAuth
1	0	00001	001110	11111		AUTDA, AUTDZA — AUTDZA	FEAT_PAuth
1	0	00001	001111	11111		AUTDB, AUTDZB — AUTDZB	FEAT_PAuth
1	0	00001	01000x	!= 11111		UNALLOCATED	-
1	0	00001	010000	11111		XPACD, XPACI, XPACLRI — XPACI	FEAT_PAuth
1	0	00001	010001	11111		XPACD, XPACI, XPACLRI — XPACD	FEAT_PAuth
1	0	00001	01001x			UNALLOCATED	-
1	0	00001	0101xx			UNALLOCATED	-
1	0	00001	011xxx			UNALLOCATED	-
1	0	00001	10xxxx		!= 11110	UNALLOCATED	-
1	0	00001	1000xx	!= 11111	11110	UNALLOCATED	-
1	0	00001	100000	11111	11110	PACNBIASPPC	FEAT_PAuth_LR
1	0	00001	100001	11111	11110	PACNBIBSPPC	FEAT_PAuth_LR
1	0	00001	100010	11111	11110	PACIA171615	FEAT_PAuth_LR
1	0	00001	100011	11111	11110	PACIB171615	FEAT_PAuth_LR
1	0	00001	100100		11110	AUTIASPPCR	FEAT_PAuth_LR
1	0	00001	100101		11110	AUTIBSPPCR	FEAT_PAuth_LR
1	0	00001	10011x		11110	UNALLOCATED	-
1	0	00001	101xxx	!= 11111	11110	UNALLOCATED	-
1	0	00001	101000	11111	11110	PACIASPPC	FEAT_PAuth_LR
1	0	00001	101001	11111	11110	PACIBSPPC	FEAT_PAuth_LR
1	0	00001	10101x	11111	11110	UNALLOCATED	-
1	0	00001	10110x	11111	11110	UNALLOCATED	-
1	0	00001	101110	11111	11110	AUTIA171615	FEAT_PAuth_LR
1	0	00001	101111	11111	11110	AUTIB171615	FEAT_PAuth_LR
1	0	00001	11xxxx			UNALLOCATED	-

Logical (shifted register)

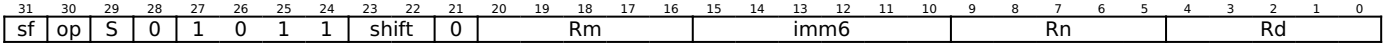
The encodings in this section are decoded from Data Processing -- Register.



Decode fields			Instruction Details
sf	opc	N	
0	00	0	AND (shifted register) — 32-bit
0	00	1	BIC (shifted register) — 32-bit
0	01	0	ORR (shifted register) — 32-bit
0	01	1	ORN (shifted register) — 32-bit
0	10	0	EOR (shifted register) — 32-bit
0	10	1	EON (shifted register) — 32-bit
0	11	0	ANDS (shifted register) — 32-bit
0	11	1	BICS (shifted register) — 32-bit
1	00	0	AND (shifted register) — 64-bit
1	00	1	BIC (shifted register) — 64-bit
1	01	0	ORR (shifted register) — 64-bit
1	01	1	ORN (shifted register) — 64-bit
1	10	0	EOR (shifted register) — 64-bit
1	10	1	EON (shifted register) — 64-bit
1	11	0	ANDS (shifted register) — 64-bit
1	11	1	BICS (shifted register) — 64-bit

Add/subtract (shifted register)

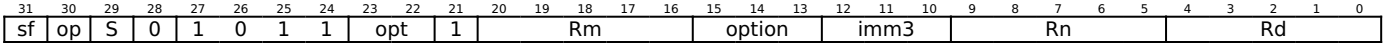
The encodings in this section are decoded from Data Processing -- Register.



Decode fields			Instruction Details
sf	op	S	
0	0	0	ADD (shifted register) — 32-bit
0	0	1	ADDS (shifted register) — 32-bit
0	1	0	SUB (shifted register) — 32-bit
0	1	1	SUBS (shifted register) — 32-bit
1	0	0	ADD (shifted register) — 64-bit
1	0	1	ADDS (shifted register) — 64-bit
1	1	0	SUB (shifted register) — 64-bit
1	1	1	SUBS (shifted register) — 64-bit

Add/subtract (extended register)

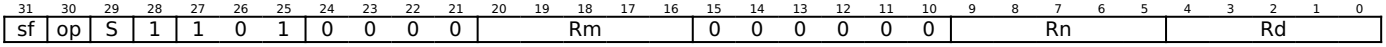
The encodings in this section are decoded from Data Processing -- Register.



Decode fields				Instruction Details
sf	op	S	opt	
			!= 00	UNALLOCATED
0	0	0	00	ADD (extended register) — 32-bit
0	0	1	00	ADDS (extended register) — 32-bit
0	1	0	00	SUB (extended register) — 32-bit
0	1	1	00	SUBS (extended register) — 32-bit
1	0	0	00	ADD (extended register) — 64-bit
1	0	1	00	ADDS (extended register) — 64-bit
1	1	0	00	SUB (extended register) — 64-bit
1	1	1	00	SUBS (extended register) — 64-bit

Add/subtract (with carry)

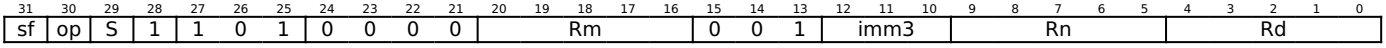
The encodings in this section are decoded from Data Processing -- Register.



Decode fields			Instruction Details
sf	op	S	
0	0	0	ADC — 32-bit
0	0	1	ADCS — 32-bit
0	1	0	SBC — 32-bit
0	1	1	SBCS — 32-bit
1	0	0	ADC — 64-bit
1	0	1	ADCS — 64-bit
1	1	0	SBC — 64-bit
1	1	1	SBCS — 64-bit

Add/subtract (checked pointer)

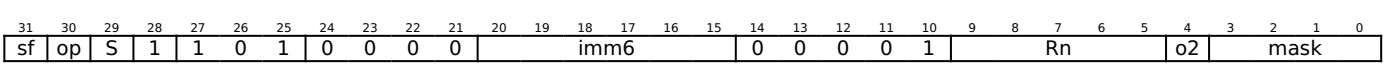
The encodings in this section are decoded from Data Processing -- Register.



Decode fields			Instruction Details	Condition
sf	op	S		
0			UNALLOCATED	-
1		1	UNALLOCATED	-
1	0	0	ADDPT	FEAT_CPA
1	1	0	SUBPT	FEAT_CPA

Rotate right into flags

The encodings in this section are decoded from Data Processing -- Register.



Decode fields				Instruction Details	Condition
sf	op	S	o2		
0				UNALLOCATED	-
1	0	0		UNALLOCATED	-
1	0	1	0	RMIF	FEAT_FlagM
1	0	1	1	UNALLOCATED	-
1	1			UNALLOCATED	-

Evaluate into flags

The encodings in this section are decoded from Data Processing -- Register.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
sf	op	S	1	1	0	1	0	0	0	0	opcode2						sz	0	0	1	0	Rn					o3	mask			

Decode fields							Instruction Details	Condition
sf	op	S	opcode2	sz	o3	mask		
0	0	0					UNALLOCATED	-
0	0	1	000000		0	!= 1101	UNALLOCATED	-
0	0	1	000000		1		UNALLOCATED	-
0	0	1	000000	0	0	1101	SETF8, SETF16 — SETF8	FEAT_FlagM
0	0	1	000000	1	0	1101	SETF8, SETF16 — SETF16	FEAT_FlagM
0	0	1	!= 000000				UNALLOCATED	-
0	1						UNALLOCATED	-
1							UNALLOCATED	-

Conditional compare (register)

The encodings in this section are decoded from Data Processing -- Register.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
sf	op	S	1	1	0	1	0	0	1	0	Rm					cond				0	o2	Rn					o3	nzcv			

Decode fields					Instruction Details
sf	op	S	o2	o3	
		0			UNALLOCATED
		1	0	1	UNALLOCATED
		1	1		UNALLOCATED
0	0	1	0	0	CCMN (register) — 32-bit
0	1	1	0	0	CCMP (register) — 32-bit
1	0	1	0	0	CCMN (register) — 64-bit
1	1	1	0	0	CCMP (register) — 64-bit

Conditional compare (immediate)

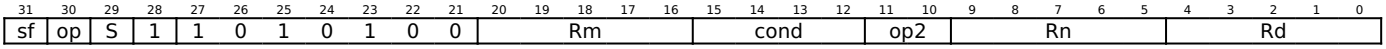
The encodings in this section are decoded from Data Processing -- Register.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
sf	op	S	1	1	0	1	0	0	1	0	imm5					cond				1	o2	Rn					o3	nzcv			

Decode fields					Instruction Details
sf	op	S	o2	o3	
		0			UNALLOCATED
		1	0	1	UNALLOCATED
		1	1		UNALLOCATED
0	0	1	0	0	CCMN (immediate) — 32-bit
0	1	1	0	0	CCMP (immediate) — 32-bit
1	0	1	0	0	CCMN (immediate) — 64-bit
1	1	1	0	0	CCMP (immediate) — 64-bit

Conditional select

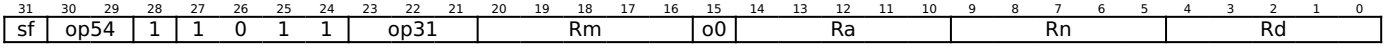
The encodings in this section are decoded from Data Processing -- Register.



Decode fields				Instruction Details
sf	op	S	op2	
		0	1x	UNALLOCATED
		1		UNALLOCATED
0	0	0	00	CSEL — 32-bit
0	0	0	01	CSINC — 32-bit
0	1	0	00	CSINV — 32-bit
0	1	0	01	CSNEG — 32-bit
1	0	0	00	CSEL — 64-bit
1	0	0	01	CSINC — 64-bit
1	1	0	00	CSINV — 64-bit
1	1	0	01	CSNEG — 64-bit

Data-processing (3 source)

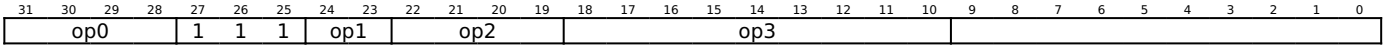
The encodings in this section are decoded from Data Processing -- Register.



Decode fields				Instruction Details	Condition
sf	op54	op31	o0		
	00	100		UNALLOCATED	-
	!= 00			UNALLOCATED	-
0	00	!= x00		UNALLOCATED	-
0	00	000	0	MADD — 32-bit	-
0	00	000	1	MSUB — 32-bit	-
1	00	x10	1	UNALLOCATED	-
1	00	000	0	MADD — 64-bit	-
1	00	000	1	MSUB — 64-bit	-
1	00	001	0	SMADDL	-
1	00	001	1	SMSUBL	-
1	00	010	0	SMULH	-
1	00	011	0	MADDPT	FEAT_CPA
1	00	011	1	MSUBPT	FEAT_CPA
1	00	101	0	UMADDL	-
1	00	101	1	UMSUBL	-
1	00	110	0	UMULH	-
1	00	111		UNALLOCATED	-

Data Processing -- Scalar Floating-Point and Advanced SIMD

The encodings in this section are decoded from A64 instruction set encoding.

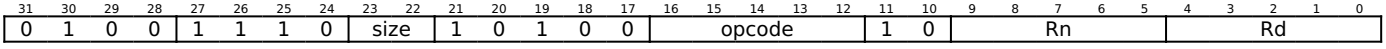


Decode fields				Instruction details	Condition
op0	op1	op2	op3		
00x0	0x	x101	00xxxxx10	UNALLOCATED	-
00x0	11		xxxxxxxx1	UNALLOCATED	-
0100	0x	x101	00xxxxx10	Cryptographic AES	-
0101	0x	x0xx	xxx0xxx00	Cryptographic three-register SHA	-
0101	0x	x0xx	xxx0xxx10	UNALLOCATED	-
0101	0x	x0xx	xxx1xxxx0	UNALLOCATED	-
0101	0x	x101	00xxxxx10	Cryptographic two-register SHA	-
0111	0x	x0xx	xxxxxxxx0	UNALLOCATED	-
011x	0x	x101	00xxxxx10	UNALLOCATED	-
01x1	00	00xx	xxx0xxxx1	Advanced SIMD scalar copy	-
01x1	01	00xx	xxx0xxxx1	UNALLOCATED	-
01x1	0x	0111	00xxxxx10	UNALLOCATED	-
01x1	0x	10xx	xxx00xxx1	Advanced SIMD scalar three same FP16	-
01x1	0x	10xx	xxx01xxx1	UNALLOCATED	-
01x1	0x	1111	00xxxxx10	Advanced SIMD scalar two-register miscellaneous FP16	-
01x1	0x	x0xx	xxx1xxxx1	Advanced SIMD scalar three same extra	-
01x1	0x	x100	00xxxxx10	Advanced SIMD scalar two-register miscellaneous	-
01x1	0x	x110	00xxxxx10	Advanced SIMD scalar pairwise	-
01x1	0x	x1xx	01xxxxx10	UNALLOCATED	-
01x1	0x	x1xx	1xxxxxx10	UNALLOCATED	-
01x1	0x	x1xx	xxxxxxx00	Advanced SIMD scalar three different	-
01x1	0x	x1xx	xxxxxxxx1	Advanced SIMD scalar three same	-
01x1	10		xxxxxxxx1	Advanced SIMD scalar shift by immediate	-
01x1	1x		xxxxxxxx0	Advanced SIMD scalar x indexed element	-
01xx	11		xxxxxxxx1	UNALLOCATED	-
0x00	0x	x0xx	xxx0xxx00	Advanced SIMD table lookup	-
0x00	0x	x0xx	xxx0xxx10	Advanced SIMD permute	-
0x10	0x	x0xx	xxx0xxxx0	Advanced SIMD extract	-
0xx0	00	00xx	xxx0xxxx1	Advanced SIMD copy	-
0xx0	01	00xx	xxx0xxxx1	UNALLOCATED	-
0xx0	0x	0111	00xxxxx10	UNALLOCATED	-
0xx0	0x	10xx	xxx00xxx1	Advanced SIMD three same (FP16)	-
0xx0	0x	10xx	xxx01xxx1	UNALLOCATED	-
0xx0	0x	1111	00xxxxx10	Advanced SIMD two-register miscellaneous (FP16)	-
0xx0	0x	x0xx	xxx1xxxx0	UNALLOCATED	-
0xx0	0x	x0xx	xxx1xxxx1	Advanced SIMD three-register extension	-
0xx0	0x	x100	00xxxxx10	Advanced SIMD two-register miscellaneous	-
0xx0	0x	x110	00xxxxx10	Advanced SIMD across lanes	-
0xx0	0x	x1xx	01xxxxx10	UNALLOCATED	-
0xx0	0x	x1xx	1xxxxxx10	UNALLOCATED	-
0xx0	0x	x1xx	xxxxxxx00	Advanced SIMD three different	-
0xx0	0x	x1xx	xxxxxxxx1	Advanced SIMD three same	-
0xx0	10	0000	xxxxxxxx1	Advanced SIMD modified immediate	-
0xx0	10	!= 0000	xxxxxxxx1	Advanced SIMD shift by immediate	-
0xx0	1x		xxxxxxxx0	Advanced SIMD vector x indexed element	-

10x0				UNALLOCATED	-
1100	00	0xxx	xxx1xxxxx	UNALLOCATED	-
1100	00	10xx	xxx10xxxx	Cryptographic three-register, imm2	-
1100	00	10xx	xxx11xxxx	UNALLOCATED	-
1100	00	11xx	xxx1x00xx	Cryptographic three-register SHA 512	-
1100	00	11xx	xxx1x01xx	UNALLOCATED	-
1100	00	11xx	xxx1x1xxx	UNALLOCATED	-
1100	00		xxx0xxxxx	Cryptographic four-register	-
1100	01	00xx		XAR	FEAT_SHA3
1100	01	1000	0001000xx	Cryptographic two-register SHA 512	-
1100	01	1000	!= 0001000xx	UNALLOCATED	-
1100	01	1001		UNALLOCATED	-
1100	01	101x		UNALLOCATED	-
1100	01	x1xx		UNALLOCATED	-
1100	1x			UNALLOCATED	-
1110				UNALLOCATED	-
11x1				UNALLOCATED	-
x0x1	0x	x0xx		Conversion between floating-point and fixed-point	-
x0x1	0x	x1xx	xxx000000	Conversion between floating-point and integer	-
x0x1	0x	x1xx	xxx100000	UNALLOCATED	-
x0x1	0x	x1xx	xxxx10000	Floating-point data-processing (1 source)	-
x0x1	0x	x1xx	xxxxx1000	Floating-point compare	-
x0x1	0x	x1xx	xxxxxx100	Floating-point immediate	-
x0x1	0x	x1xx	xxxxxxx01	Floating-point conditional compare	-
x0x1	0x	x1xx	xxxxxxx10	Floating-point data-processing (2 source)	-
x0x1	0x	x1xx	xxxxxxx11	Floating-point conditional select	-
x0x1	1x			Floating-point data-processing (3 source)	-

Cryptographic AES

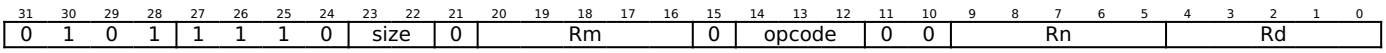
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields		Instruction Details	Condition
size	opcode		
00	00100	AESE	FEAT_AES
00	00101	AESD	FEAT_AES
00	00110	AESMC	FEAT_AES
00	00111	AESIMC	FEAT_AES
00	!= 001xx	UNALLOCATED	-
!= 00		UNALLOCATED	-

Cryptographic three-register SHA

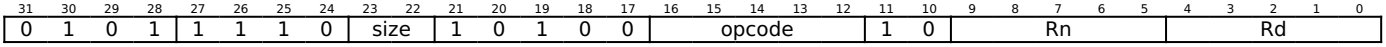
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields		Instruction Details	Condition
size	opcode		
00	000	SHA1C	FEAT_SHA1
00	001	SHA1P	FEAT_SHA1
00	010	SHA1M	FEAT_SHA1
00	011	SHA1SU0	FEAT_SHA1
00	100	SHA256H	FEAT_SHA256
00	101	SHA256H2	FEAT_SHA256
00	110	SHA256SU1	FEAT_SHA256
00	111	UNALLOCATED	-
!= 00		UNALLOCATED	-

Cryptographic two-register SHA

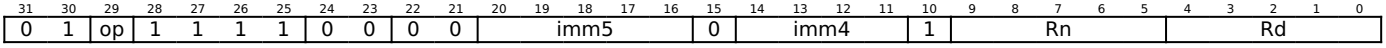
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields		Instruction Details	Condition
size	opcode		
00	00000	SHA1H	FEAT_SHA1
00	00001	SHA1SU1	FEAT_SHA1
00	00010	SHA256SU0	FEAT_SHA256
00	00011	UNALLOCATED	-
00	001xx	UNALLOCATED	-
00	01xxx	UNALLOCATED	-
00	1xxxx	UNALLOCATED	-
!= 00		UNALLOCATED	-

Advanced SIMD scalar copy

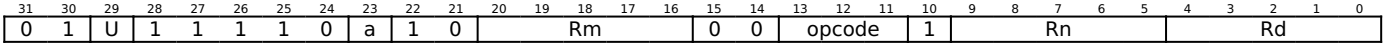
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields		Instruction Details	Condition
op	imm4		
0	0000	DUP (element)	FEAT_AdvSIMD
0	!= 0000	UNALLOCATED	-
1		UNALLOCATED	-

Advanced SIMD scalar three same FP16

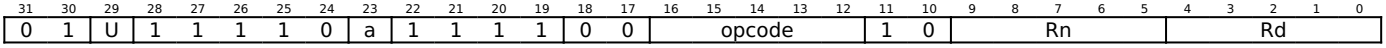
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields			Instruction Details	Condition
U	a	opcode		
		00x	UNALLOCATED	-
	0	010	UNALLOCATED	-
0		101	UNALLOCATED	-
0	0	011	FMULX	FEAT_AdvSIMD && FEAT_FP16
0	0	100	FCMEQ (register)	FEAT_AdvSIMD && FEAT_FP16
0	0	110	UNALLOCATED	-
0	0	111	FRECPS	FEAT_AdvSIMD && FEAT_FP16
0	1	x10	UNALLOCATED	-
0	1	011	UNALLOCATED	-
0	1	100	UNALLOCATED	-
0	1	111	FRSQRTS	FEAT_AdvSIMD && FEAT_FP16
1		011	UNALLOCATED	-
1		11x	UNALLOCATED	-
1	0	100	FCMGE (register)	FEAT_AdvSIMD && FEAT_FP16
1	0	101	FACGE	FEAT_AdvSIMD && FEAT_FP16
1	1	010	FABD	FEAT_AdvSIMD && FEAT_FP16
1	1	100	FCMGT (register)	FEAT_AdvSIMD && FEAT_FP16
1	1	101	FACGT	FEAT_AdvSIMD && FEAT_FP16

Advanced SIMD scalar two-register miscellaneous FP16

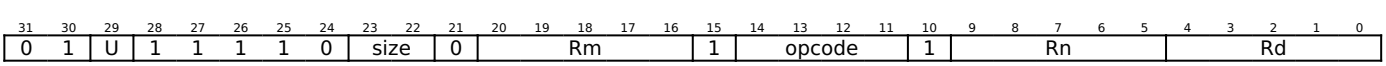
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields			Instruction Details	Condition
U	a	opcode		
		x0xxx	UNALLOCATED	-
		1100x	UNALLOCATED	-
	0	01xxx	UNALLOCATED	-
	1	010xx	UNALLOCATED	-
	1	11100	UNALLOCATED	-
0	0	11010	FCVTNS (vector)	FEAT_AdvSIMD && FEAT_FP16
0	0	11011	FCVTMS (vector)	FEAT_AdvSIMD && FEAT_FP16
0	0	11100	FCVTAS (vector)	FEAT_AdvSIMD && FEAT_FP16
0	0	11101	SCVTF (vector, integer)	FEAT_AdvSIMD && FEAT_FP16
0	0	1111x	UNALLOCATED	-
0	1	01100	FCMGT (zero)	FEAT_AdvSIMD && FEAT_FP16
0	1	01101	FCMEQ (zero)	FEAT_AdvSIMD && FEAT_FP16
0	1	01110	FCMLT (zero)	FEAT_AdvSIMD && FEAT_FP16
0	1	01111	UNALLOCATED	-
0	1	11010	FCVTPS (vector)	FEAT_AdvSIMD && FEAT_FP16
0	1	11011	FCVTZS (vector, integer)	FEAT_AdvSIMD && FEAT_FP16
0	1	11101	FRECPE	FEAT_AdvSIMD && FEAT_FP16
0	1	11110	UNALLOCATED	-
0	1	11111	FRECPX	FEAT_AdvSIMD && FEAT_FP16
1		1111x	UNALLOCATED	-
1	0	11010	FCVTNU (vector)	FEAT_AdvSIMD && FEAT_FP16
1	0	11011	FCVTMU (vector)	FEAT_AdvSIMD && FEAT_FP16
1	0	11100	FCVTAU (vector)	FEAT_AdvSIMD && FEAT_FP16
1	0	11101	UCVTF (vector, integer)	FEAT_AdvSIMD && FEAT_FP16
1	1	01100	FCMGE (zero)	FEAT_AdvSIMD && FEAT_FP16
1	1	01101	FCMLE (zero)	FEAT_AdvSIMD && FEAT_FP16
1	1	0111x	UNALLOCATED	-
1	1	11010	FCVTPU (vector)	FEAT_AdvSIMD && FEAT_FP16
1	1	11011	FCVTZU (vector, integer)	FEAT_AdvSIMD && FEAT_FP16
1	1	11101	FRSQRTE	FEAT_AdvSIMD && FEAT_FP16

Advanced SIMD scalar three same extra

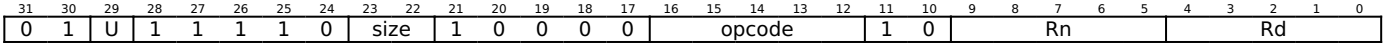
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields		Instruction Details	Condition
U	opcode		
0		UNALLOCATED	-
1	0000	SQRDMLAH (vector)	FEAT_RDM
1	0001	SQRDMLSH (vector)	FEAT_RDM
1	!= 000x	UNALLOCATED	-

Advanced SIMD scalar two-register miscellaneous

The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.

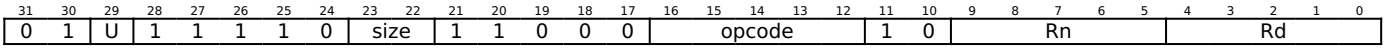

31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
0 1 U 1 1 1 1 0 size 1 0 0 0 0 opcode 1 0 Rn Rd

Decode fields			Instruction Details	Condition
U	size	opcode		
		00xx0	UNALLOCATED	-
		00x01	UNALLOCATED	-
		1x000	UNALLOCATED	-
		10xx1	UNALLOCATED	-
		11001	UNALLOCATED	-
	0x	01xxx	UNALLOCATED	-
	0x	1111x	UNALLOCATED	-
	1x	01111	UNALLOCATED	-
	1x	11100	UNALLOCATED	-
	10	010x1	UNALLOCATED	-
	10	01000	UNALLOCATED	-
0		00011	SUQADD	FEAT_AdvSIMD
0		00111	SQABS	FEAT_AdvSIMD
0		10010	UNALLOCATED	-
0		10100	SQXTN, SQXTN2	FEAT_AdvSIMD
0		10110	UNALLOCATED	-
0	0x	11010	FCVTNS (vector)	FEAT_AdvSIMD
0	0x	11011	FCVTMS (vector)	FEAT_AdvSIMD
0	0x	11100	FCVTAS (vector)	FEAT_AdvSIMD
0	0x	11101	SCVTF (vector, integer)	FEAT_AdvSIMD
0	1x	01100	FCMGT (zero)	FEAT_AdvSIMD
0	1x	01101	FCMEQ (zero)	FEAT_AdvSIMD
0	1x	01110	FCMLT (zero)	FEAT_AdvSIMD
0	1x	11010	FCVTPS (vector)	FEAT_AdvSIMD
0	1x	11011	FCVTZS (vector, integer)	FEAT_AdvSIMD
0	1x	11101	FRECPE	FEAT_AdvSIMD
0	1x	11110	UNALLOCATED	-
0	1x	11111	FRECPX	FEAT_AdvSIMD
0	10	01010	UNALLOCATED	-
0	11	01000	CMGT (zero)	FEAT_AdvSIMD
0	11	01001	CMEQ (zero)	FEAT_AdvSIMD
0	11	01010	CMLT (zero)	FEAT_AdvSIMD
0	11	01011	ABS	FEAT_AdvSIMD
1		00011	USQADD	FEAT_AdvSIMD
1		00111	SQNEG	FEAT_AdvSIMD
1		10010	SQXTUN, SQXTUN2	FEAT_AdvSIMD
1		10100	UQXTN, UQXTN2	FEAT_AdvSIMD
1	0x	11010	FCVTNU (vector)	FEAT_AdvSIMD
1	0x	11011	FCVTMU (vector)	FEAT_AdvSIMD
1	0x	11100	FCVTAU (vector)	FEAT_AdvSIMD
1	0x	11101	UCVTF (vector, integer)	FEAT_AdvSIMD
1	00	10110	UNALLOCATED	-
1	01	10110	FCVTXN, FCVTXN2	FEAT_AdvSIMD
1	1x	01x10	UNALLOCATED	-
1	1x	01100	FCMGE (zero)	FEAT_AdvSIMD

Decode fields			Instruction Details	Condition
U	size	opcode		
1	1x	01101	FCMLE (zero)	FEAT_AdvSIMD
1	1x	1x110	UNALLOCATED	-
1	1x	11010	FCVTPU (vector)	FEAT_AdvSIMD
1	1x	11011	FCVTZU (vector, integer)	FEAT_AdvSIMD
1	1x	11101	FRSQRTE	FEAT_AdvSIMD
1	1x	11111	UNALLOCATED	-
1	11	01000	CMGE (zero)	FEAT_AdvSIMD
1	11	01001	CMLE (zero)	FEAT_AdvSIMD
1	11	01011	NEG (vector)	FEAT_AdvSIMD

Advanced SIMD scalar pairwise

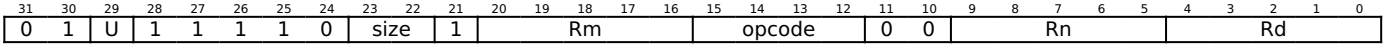
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields			Instruction Details	Condition
U	size	opcode		
		x0xxx	UNALLOCATED	-
		01110	UNALLOCATED	-
		111xx	UNALLOCATED	-
	10	01101	UNALLOCATED	-
0	x1	011x1	UNALLOCATED	-
0	x1	01100	UNALLOCATED	-
0	00	01100	FMAXNMP (scalar) — Half-precision	FEAT_AdvSIMD && FEAT_FP16
0	00	01101	FADDP (scalar) — Half-precision	FEAT_AdvSIMD && FEAT_FP16
0	00	01111	FMAXP (scalar) — Half-precision	FEAT_AdvSIMD && FEAT_FP16
0	10	01100	FMINNMP (scalar) — Half-precision	FEAT_AdvSIMD && FEAT_FP16
0	10	01111	FMINP (scalar) — Half-precision	FEAT_AdvSIMD && FEAT_FP16
0	11	x100x	UNALLOCATED	-
0	11	x1010	UNALLOCATED	-
0	11	01011	UNALLOCATED	-
0	11	11011	ADDP (scalar)	FEAT_AdvSIMD
0	!= 11	x10xx	UNALLOCATED	-
1		x10xx	UNALLOCATED	-
1	0x	01100	FMAXNMP (scalar) — Single-precision and double-precision	FEAT_AdvSIMD
1	0x	01101	FADDP (scalar) — Single-precision and double-precision	FEAT_AdvSIMD
1	0x	01111	FMAXP (scalar) — Single-precision and double-precision	FEAT_AdvSIMD
1	1x	01100	FMINNMP (scalar) — Single-precision and double-precision	FEAT_AdvSIMD
1	1x	01111	FMINP (scalar) — Single-precision and double-precision	FEAT_AdvSIMD
1	11	01101	UNALLOCATED	-

Advanced SIMD scalar three different

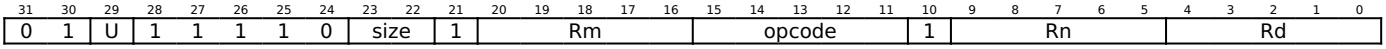
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields		Instruction Details	Condition
U	opcode		
0	0xxx	UNALLOCATED	-
0	1xx0	UNALLOCATED	-
0	1001	SQDMLAL, SQDMLAL2 (vector)	FEAT_AdvSIMD
0	1011	SQDMLSL, SQDMLSL2 (vector)	FEAT_AdvSIMD
0	1101	SQDMULL, SQDMULL2 (vector)	FEAT_AdvSIMD
0	1111	UNALLOCATED	-
1		UNALLOCATED	-

Advanced SIMD scalar three same

The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.

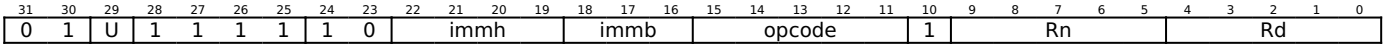


Decode fields			Instruction Details	Condition
U	size	opcode		
		x0011	UNALLOCATED	-
		011x1	UNALLOCATED	-
		11001	UNALLOCATED	-
		11110	UNALLOCATED	-
	x0	0x1x0	UNALLOCATED	-
	x1	00100	UNALLOCATED	-
	x1	011x0	UNALLOCATED	-
	0x	xx0x0	UNALLOCATED	-
	0x	10x01	UNALLOCATED	-
	01	00110	UNALLOCATED	-
	1x	10010	UNALLOCATED	-
	1x	11000	UNALLOCATED	-
	10	0x0x0	UNALLOCATED	-
	10	10x0x	UNALLOCATED	-
	11	00000	UNALLOCATED	-
	11	00010	UNALLOCATED	-
	11	10101	UNALLOCATED	-
	!= 10	10100	UNALLOCATED	-
	!= 11	00111	UNALLOCATED	-
0		00001	SQADD	FEAT_AdvSIMD
0		00101	SQSUB	FEAT_AdvSIMD
0		01001	SQSHL (register)	FEAT_AdvSIMD
0		01011	SQRSHL	FEAT_AdvSIMD
0		10110	SQDMULH (vector)	FEAT_AdvSIMD
0		10111	UNALLOCATED	-
0		11101	UNALLOCATED	-
0	0x	11011	FMULX	FEAT_AdvSIMD
0	0x	11100	FCMEQ (register)	FEAT_AdvSIMD
0	0x	11111	FRECPS	FEAT_AdvSIMD
0	1x	11010	UNALLOCATED	-
0	1x	11011	UNALLOCATED	-
0	1x	11100	UNALLOCATED	-
0	1x	11111	FRSQRTS	FEAT_AdvSIMD
0	11	00110	CMGT (register)	FEAT_AdvSIMD
0	11	00111	CMGE (register)	FEAT_AdvSIMD
0	11	01000	SSHL	FEAT_AdvSIMD
0	11	01010	SRSHL	FEAT_AdvSIMD
0	11	10000	ADD (vector)	FEAT_AdvSIMD
0	11	10001	CMTST	FEAT_AdvSIMD
1		00001	UQADD	FEAT_AdvSIMD
1		00101	UQSUB	FEAT_AdvSIMD
1		01001	UQSHL (register)	FEAT_AdvSIMD
1		01011	UQRSHL	FEAT_AdvSIMD
1		1x111	UNALLOCATED	-
1		10110	SQRDMULH (vector)	FEAT_AdvSIMD

Decode fields			Instruction Details	Condition
U	size	opcode		
1		11011	UNALLOCATED	-
1	0x	11100	FCMGE (register)	FEAT_AdvSIMD
1	0x	11101	FACGE	FEAT_AdvSIMD
1	1x	11010	FABD	FEAT_AdvSIMD
1	1x	11100	FCMGT (register)	FEAT_AdvSIMD
1	1x	11101	FACGT	FEAT_AdvSIMD
1	11	00110	CMHI (register)	FEAT_AdvSIMD
1	11	00111	CMHS (register)	FEAT_AdvSIMD
1	11	01000	USHL	FEAT_AdvSIMD
1	11	01010	URSHL	FEAT_AdvSIMD
1	11	10000	SUB (vector)	FEAT_AdvSIMD
1	11	10001	CMEQ (register)	FEAT_AdvSIMD

Advanced SIMD scalar shift by immediate

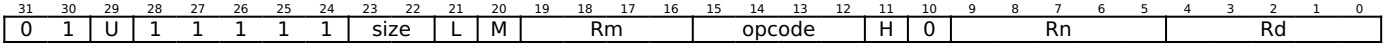
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields			Instruction Details	Condition
U	immh	opcode		
		0xxx1	UNALLOCATED	-
		101xx	UNALLOCATED	-
		11101	UNALLOCATED	-
	0xxx	x10x0	UNALLOCATED	-
	0xxx	00xx0	UNALLOCATED	-
	0xxx	110x1	UNALLOCATED	-
	0000	x1110	UNALLOCATED	-
	0000	1001x	UNALLOCATED	-
	0000	11100	UNALLOCATED	-
	0000	11111	UNALLOCATED	-
	1xxx	110xx	UNALLOCATED	-
	!= 0000	11110	UNALLOCATED	-
0		01100	UNALLOCATED	-
0		1000x	UNALLOCATED	-
0	1xxx	00000	SSHR	FEAT_AdvSIMD
0	1xxx	00010	SSRA	FEAT_AdvSIMD
0	1xxx	00100	SRSHR	FEAT_AdvSIMD
0	1xxx	00110	SRSRA	FEAT_AdvSIMD
0	1xxx	01000	UNALLOCATED	-
0	1xxx	01010	SHL	FEAT_AdvSIMD
0	!= 0000	01110	SQSHL (immediate)	FEAT_AdvSIMD
0	!= 0000	10010	SQSHRN, SQSHRN2	FEAT_AdvSIMD
0	!= 0000	10011	SQRSHRN, SQRSHRN2	FEAT_AdvSIMD
0	!= 0000	11100	SCVTF (vector, fixed-point)	FEAT_AdvSIMD
0	!= 0000	11111	FCVTZS (vector, fixed-point)	FEAT_AdvSIMD
1	0000	01100	UNALLOCATED	-
1	0000	1000x	UNALLOCATED	-
1	1xxx	00000	USHR	FEAT_AdvSIMD
1	1xxx	00010	USRA	FEAT_AdvSIMD
1	1xxx	00100	URSHR	FEAT_AdvSIMD
1	1xxx	00110	URSRA	FEAT_AdvSIMD
1	1xxx	01000	SRI	FEAT_AdvSIMD
1	1xxx	01010	SLI	FEAT_AdvSIMD
1	!= 0000	01100	SQSHLU	FEAT_AdvSIMD
1	!= 0000	01110	UQSHL (immediate)	FEAT_AdvSIMD
1	!= 0000	10000	SQSHRUN, SQSHRUN2	FEAT_AdvSIMD
1	!= 0000	10001	SQRSHRUN, SQRSHRUN2	FEAT_AdvSIMD
1	!= 0000	10010	UQSHRN, UQSHRN2	FEAT_AdvSIMD
1	!= 0000	10011	UQRSHRN, UQRSHRN2	FEAT_AdvSIMD
1	!= 0000	11100	UCVTF (vector, fixed-point)	FEAT_AdvSIMD
1	!= 0000	11111	FCVTZU (vector, fixed-point)	FEAT_AdvSIMD

Advanced SIMD scalar x indexed element

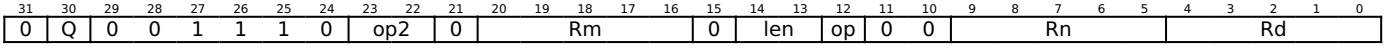
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields			Instruction Details	Condition
U	size	opcode		
	01	1001	UNALLOCATED	-
0		0xx0	UNALLOCATED	-
0		0011	SQDMLAL, SQDMLAL2 (by element)	FEAT_AdvSIMD
0		0111	SQDMLSL, SQDMLSL2 (by element)	FEAT_AdvSIMD
0		10x0	UNALLOCATED	-
0		1011	SQDMULL, SQDMULL2 (by element)	FEAT_AdvSIMD
0		1100	SQDMULH (by element)	FEAT_AdvSIMD
0		1101	SQRDMULH (by element)	FEAT_AdvSIMD
0		111x	UNALLOCATED	-
0	00	0001	FMLA (by element) — Scalar, half-precision	FEAT_AdvSIMD && FEAT_FP16
0	00	0101	FMLS (by element) — Scalar, half-precision	FEAT_AdvSIMD && FEAT_FP16
0	00	1001	FMUL (by element) — Scalar, half-precision	FEAT_AdvSIMD && FEAT_FP16
0	01	0x01	UNALLOCATED	-
0	1x	0001	FMLA (by element) — Scalar, single-precision and double-precision	FEAT_AdvSIMD
0	1x	0101	FMLS (by element) — Scalar, single-precision and double-precision	FEAT_AdvSIMD
0	1x	1001	FMUL (by element) — Scalar, single-precision and double-precision	FEAT_AdvSIMD
1		0xxx	UNALLOCATED	-
1		1xx0	UNALLOCATED	-
1		1011	UNALLOCATED	-
1		1101	SQRDMLAH (by element)	FEAT_RDM
1		1111	SQRDMLSH (by element)	FEAT_RDM
1	00	1001	FMULX (by element) — Scalar, half-precision	FEAT_AdvSIMD && FEAT_FP16
1	1x	1001	FMULX (by element) — Scalar, single-precision and double-precision	FEAT_AdvSIMD

Advanced SIMD table lookup

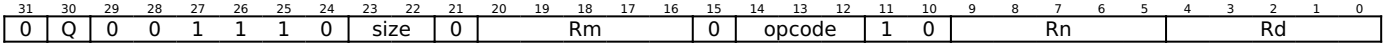
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields				Instruction Details	Condition
Q	op2	len	op		
	00	00	0	TBL — Single register table	FEAT_AdvSIMD
	00	00	1	TBX — Single register table	FEAT_AdvSIMD
	00	01	0	TBL — Two register table	FEAT_AdvSIMD
	00	01	1	TBX — Two register table	FEAT_AdvSIMD
	00	10	0	TBL — Three register table	FEAT_AdvSIMD
	00	10	1	TBX — Three register table	FEAT_AdvSIMD
	00	11	0	TBL — Four register table	FEAT_AdvSIMD
	00	11	1	TBX — Four register table	FEAT_AdvSIMD
0	!= 00			UNALLOCATED	-
1	01		1	LUTI4 — Halfword	FEAT_AdvSIMD && FEAT_LUT
1	01	x0	0	UNALLOCATED	-
1	01	x1	0	LUTI4 — Byte	FEAT_AdvSIMD && FEAT_LUT
1	10		0	UNALLOCATED	-
1	10		1	LUTI2 — Byte	FEAT_AdvSIMD && FEAT_LUT
1	11			LUTI2 — Halfword	FEAT_AdvSIMD && FEAT_LUT

Advanced SIMD permute

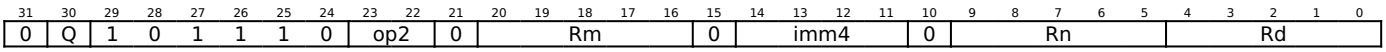
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields	Instruction Details	Condition
opcode		
x00	UNALLOCATED	-
001	UZP1	FEAT_AdvSIMD
010	TRN1	FEAT_AdvSIMD
011	ZIP1	FEAT_AdvSIMD
101	UZP2	FEAT_AdvSIMD
110	TRN2	FEAT_AdvSIMD
111	ZIP2	FEAT_AdvSIMD

Advanced SIMD extract

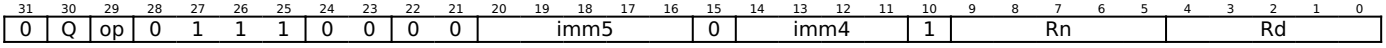
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields	Instruction Details	Condition
op2		
00	EXT	FEAT_AdvSIMD
!= 00	UNALLOCATED	-

Advanced SIMD copy

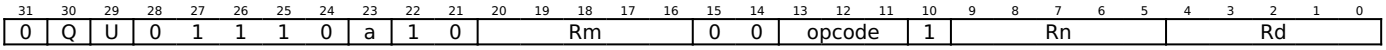
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Q	op	Decode fields imm5	imm4	Instruction Details	Condition
	0		0000	DUP (element)	FEAT_AdvSIMD
	0		0001	DUP (general)	FEAT_AdvSIMD
	0		01x0	UNALLOCATED	-
	0		1xxx	UNALLOCATED	-
0	0		001x	UNALLOCATED	-
0	0		0101	SMOV — 32-bit	FEAT_AdvSIMD
0	0		0111	UMOV — 32-bit	FEAT_AdvSIMD
0	1			UNALLOCATED	-
1	0		0010	UNALLOCATED	-
1	0		0011	INS (general)	FEAT_AdvSIMD
1	0		0101	SMOV — 64-bit	FEAT_AdvSIMD
1	0	x1000	0111	UMOV — 64-bit	FEAT_AdvSIMD
1	0	!= x1000	0111	UNALLOCATED	-
1	1			INS (element)	FEAT_AdvSIMD

Advanced SIMD three same (FP16)

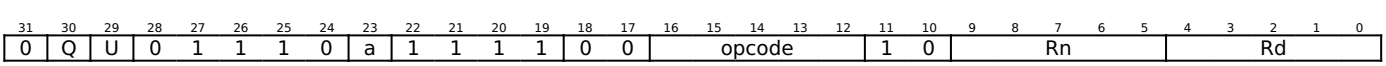
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields			Instruction Details	Condition
U	a	opcode		
0	0	000	FMAXNM (vector)	FEAT_AdvSIMD && FEAT_FP16
0	0	001	FMLA (vector)	FEAT_AdvSIMD && FEAT_FP16
0	0	010	FADD (vector)	FEAT_AdvSIMD && FEAT_FP16
0	0	011	FMULX	FEAT_AdvSIMD && FEAT_FP16
0	0	100	FCMEQ (register)	FEAT_AdvSIMD && FEAT_FP16
0	0	101	UNALLOCATED	-
0	0	110	FMAX (vector)	FEAT_AdvSIMD && FEAT_FP16
0	0	111	FRECPS	FEAT_AdvSIMD && FEAT_FP16
0	1	000	FMINNM (vector)	FEAT_AdvSIMD && FEAT_FP16
0	1	001	FMLS (vector)	FEAT_AdvSIMD && FEAT_FP16
0	1	010	FSUB (vector)	FEAT_AdvSIMD && FEAT_FP16
0	1	011	FAMAX	FEAT_AdvSIMD && FEAT_FAMINMAX
0	1	10x	UNALLOCATED	-
0	1	110	FMIN (vector)	FEAT_AdvSIMD && FEAT_FP16
0	1	111	FRSQRTS	FEAT_AdvSIMD && FEAT_FP16
1		001	UNALLOCATED	-
1	0	000	FMAXNMP (vector)	FEAT_AdvSIMD && FEAT_FP16
1	0	010	FADDP (vector)	FEAT_AdvSIMD && FEAT_FP16
1	0	011	FMUL (vector)	FEAT_AdvSIMD && FEAT_FP16
1	0	100	FCMGE (register)	FEAT_AdvSIMD && FEAT_FP16
1	0	101	FACGE	FEAT_AdvSIMD && FEAT_FP16
1	0	110	FMAXP (vector)	FEAT_AdvSIMD && FEAT_FP16
1	0	111	FDIV (vector)	FEAT_AdvSIMD && FEAT_FP16
1	1	000	FMINNMP (vector)	FEAT_AdvSIMD && FEAT_FP16
1	1	010	FABD	FEAT_AdvSIMD && FEAT_FP16
1	1	011	FAMIN	FEAT_AdvSIMD && FEAT_FAMINMAX
1	1	100	FCMGT (register)	FEAT_AdvSIMD && FEAT_FP16
1	1	101	FACGT	FEAT_AdvSIMD && FEAT_FP16
1	1	110	FMINP (vector)	FEAT_AdvSIMD && FEAT_FP16
1	1	111	FSCALE	FEAT_FP8

Advanced SIMD two-register miscellaneous (FP16)

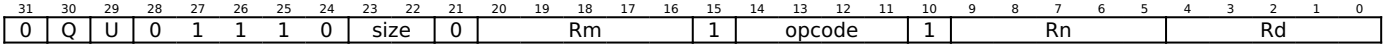
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields			Instruction Details	Condition
U	a	opcode		
		x0xxx	UNALLOCATED	-
	0	01xxx	UNALLOCATED	-
	0	1111x	UNALLOCATED	-
	1	010xx	UNALLOCATED	-
	1	11100	UNALLOCATED	-
0	0	11000	FRINTN (vector)	FEAT_AdvSIMD && FEAT_FP16
0	0	11001	FRINTM (vector)	FEAT_AdvSIMD && FEAT_FP16
0	0	11010	FCVTNS (vector)	FEAT_AdvSIMD && FEAT_FP16
0	0	11011	FCVTMS (vector)	FEAT_AdvSIMD && FEAT_FP16
0	0	11100	FCVTAS (vector)	FEAT_AdvSIMD && FEAT_FP16
0	0	11101	SCVTF (vector, integer)	FEAT_AdvSIMD && FEAT_FP16
0	1	01100	FCMGT (zero)	FEAT_AdvSIMD && FEAT_FP16
0	1	01101	FCMEQ (zero)	FEAT_AdvSIMD && FEAT_FP16
0	1	01110	FCMLT (zero)	FEAT_AdvSIMD && FEAT_FP16
0	1	01111	FABS (vector)	FEAT_AdvSIMD && FEAT_FP16
0	1	11000	FRINTP (vector)	FEAT_AdvSIMD && FEAT_FP16
0	1	11001	FRINTZ (vector)	FEAT_AdvSIMD && FEAT_FP16
0	1	11010	FCVTPS (vector)	FEAT_AdvSIMD && FEAT_FP16
0	1	11011	FCVTZS (vector, integer)	FEAT_AdvSIMD && FEAT_FP16
0	1	11101	FRECPE	FEAT_AdvSIMD && FEAT_FP16
0	1	1111x	UNALLOCATED	-
1	0	11000	FRINTA (vector)	FEAT_AdvSIMD && FEAT_FP16
1	0	11001	FRINTX (vector)	FEAT_AdvSIMD && FEAT_FP16
1	0	11010	FCVTNU (vector)	FEAT_AdvSIMD && FEAT_FP16
1	0	11011	FCVTMU (vector)	FEAT_AdvSIMD && FEAT_FP16
1	0	11100	FCVTAU (vector)	FEAT_AdvSIMD && FEAT_FP16
1	0	11101	UCVTF (vector, integer)	FEAT_AdvSIMD && FEAT_FP16
1	1	x1110	UNALLOCATED	-
1	1	01100	FCMGE (zero)	FEAT_AdvSIMD && FEAT_FP16
1	1	01101	FCMLE (zero)	FEAT_AdvSIMD && FEAT_FP16
1	1	01111	FNEG (vector)	FEAT_AdvSIMD && FEAT_FP16
1	1	11000	UNALLOCATED	-
1	1	11001	FRINTI (vector)	FEAT_AdvSIMD && FEAT_FP16
1	1	11010	FCVTPU (vector)	FEAT_AdvSIMD && FEAT_FP16
1	1	11011	FCVTZU (vector, integer)	FEAT_AdvSIMD && FEAT_FP16
1	1	11101	FRSQRTE	FEAT_AdvSIMD && FEAT_FP16
1	1	11111	FSQRT (vector)	FEAT_AdvSIMD && FEAT_FP16

Advanced SIMD three-register extension

The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.

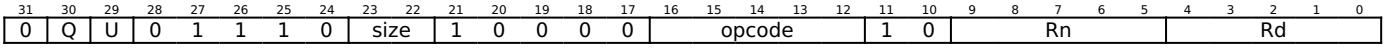


Decode fields				Instruction Details	Condition
Q	U	size	opcode		
	0		0000	UNALLOCATED	-
	0		0010	SDOT (vector)	FEAT_DotProd
	0	x0	0001	UNALLOCATED	-
	0	x0	1001	UNALLOCATED	-
	0	x0	1011	UNALLOCATED	-
	0	x1	x0x1	UNALLOCATED	-
	0	0x	1010	UNALLOCATED	-
	0	0x	1100	UNALLOCATED	-
	0	00	0011	UNALLOCATED	-
	0	00	1110	FCVTN, FCVTN2 (single-precision to 8-bit floating-point)	FEAT_FP8
	0	00	1111	FDOT (8-bit floating-point to single-precision, vector)	FEAT_FP8DOT4
	0	01	1110	FCVTN (half-precision to 8-bit floating-point)	FEAT_FP8
	0	01	1111	FDOT (8-bit floating-point to half-precision, vector)	FEAT_FP8DOT2
	0	1x	1xx0	UNALLOCATED	-
	0	10	0011	USDOT (vector)	FEAT_I8MM
	0	10	1111	FDOT (half-precision to single-precision, vector)	FEAT_F16F32DOT
	1		0000	SQRDMLAH (vector)	FEAT_RDM
	1		0001	SQRDMLSH (vector)	FEAT_RDM
	1		0010	UDOT (vector)	FEAT_DotProd
	1		0011	UNALLOCATED	-
	1		10xx	FCMLA	FEAT_FCMA
	1		11x0	FCADD	FEAT_FCMA
	1	x0	1111	UNALLOCATED	-
	1	01	1111	BFDOT (vector)	FEAT_BF16
	1	11	1111	BFMLALB, BFMLALT (vector)	FEAT_BF16
0			01xx	UNALLOCATED	-
0			1101	UNALLOCATED	-
0	0	00	1000	FMLALLBB, FMLALLBT, FMLALLTB, FMLALLTT (vector) — FMLALLBB	FEAT_FP8FMA
0	0	01	1000	FMLALLBB, FMLALLBT, FMLALLTB, FMLALLTT (vector) — FMLALLBT	FEAT_FP8FMA
0	0	11	1111	FMLALB, FMLALT (vector) — FMLALB	FEAT_FP8FMA
1		10	011x	UNALLOCATED	-
1		!= 10	01xx	UNALLOCATED	-
1	0	x0	1101	UNALLOCATED	-
1	0	00	1000	FMLALLBB, FMLALLBT, FMLALLTB, FMLALLTT (vector) — FMLALLTB	FEAT_FP8FMA
1	0	01	1000	FMLALLBB, FMLALLBT, FMLALLTB, FMLALLTT (vector) — FMLALLTT	FEAT_FP8FMA
1	0	01	1101	FMMLA (widening, half-precision to single-precision)	FEAT_F16F32MM
1	0	10	0100	SMMLA (vector)	FEAT_I8MM
1	0	10	0101	USMMLA (vector)	FEAT_I8MM
1	0	11	1101	FMMLA (non-widening)	FEAT_F16MM
1	0	11	1111	FMLALB, FMLALT (vector) — FMLALT	FEAT_FP8FMA
1	1	00	1101	FMMLA (widening, 8-bit floating-point to half-precision)	FEAT_F8F16MM
1	1	01	1101	BFMMLA (widening)	FEAT_BF16
1	1	10	0100	UMMLA (vector)	FEAT_I8MM
1	1	10	0101	UNALLOCATED	-
1	1	10	1101	FMMLA (widening, 8-bit floating-point to single-precision)	FEAT_F8F32MM

Decode fields				Instruction Details	Condition
Q	U	size	opcode		
1	1	11	1101	UNALLOCATED	-

Advanced SIMD two-register miscellaneous

The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.

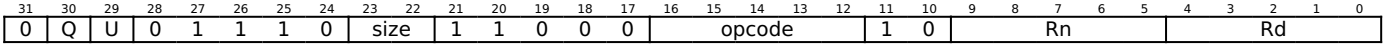


Decode fields			Instruction Details	Condition
U	size	opcode		
		1000x	UNALLOCATED	-
		10101	UNALLOCATED	-
	0x	011xx	UNALLOCATED	-
	10	11110	UNALLOCATED	-
	11	1x110	UNALLOCATED	-
0		00000	REV64	FEAT_AdvSIMD
0		00001	REV16 (vector)	FEAT_AdvSIMD
0		00010	SADDLP	FEAT_AdvSIMD
0		00011	SUQADD	FEAT_AdvSIMD
0		00100	CLS (vector)	FEAT_AdvSIMD
0		00101	CNT	FEAT_AdvSIMD
0		00110	SADALP	FEAT_AdvSIMD
0		00111	SQABS	FEAT_AdvSIMD
0		01000	CMGT (zero)	FEAT_AdvSIMD
0		01001	CMEQ (zero)	FEAT_AdvSIMD
0		01010	CMLT (zero)	FEAT_AdvSIMD
0		01011	ABS	FEAT_AdvSIMD
0		10010	XTN, XTN2	FEAT_AdvSIMD
0		10011	UNALLOCATED	-
0		10100	SQXTN, SQXTN2	FEAT_AdvSIMD
0	0x	10110	FCVTN, FCVTN2 (double to single-precision, single to half-precision)	FEAT_AdvSIMD
0	0x	10111	FCVTL, FCVTL2	FEAT_AdvSIMD
0	0x	11000	FRINTN (vector)	FEAT_AdvSIMD
0	0x	11001	FRINTM (vector)	FEAT_AdvSIMD
0	0x	11010	FCVTNS (vector)	FEAT_AdvSIMD
0	0x	11011	FCVTMS (vector)	FEAT_AdvSIMD
0	0x	11100	FCVTAS (vector)	FEAT_AdvSIMD
0	0x	11101	SCVTF (vector, integer)	FEAT_AdvSIMD
0	0x	11110	FRINT32Z (vector)	FEAT_FRINTTS
0	0x	11111	FRINT64Z (vector)	FEAT_FRINTTS
0	1x	01100	FCMGT (zero)	FEAT_AdvSIMD
0	1x	01101	FCMEQ (zero)	FEAT_AdvSIMD
0	1x	01110	FCMLT (zero)	FEAT_AdvSIMD
0	1x	01111	FABS (vector)	FEAT_AdvSIMD
0	1x	1x111	UNALLOCATED	-
0	1x	11000	FRINTP (vector)	FEAT_AdvSIMD
0	1x	11001	FRINTZ (vector)	FEAT_AdvSIMD
0	1x	11010	FCVTPS (vector)	FEAT_AdvSIMD
0	1x	11011	FCVTZS (vector, integer)	FEAT_AdvSIMD
0	1x	11100	URECPE	FEAT_AdvSIMD
0	1x	11101	FRECPE	FEAT_AdvSIMD
0	10	10110	BFCVTN, BFCVTN2	FEAT_BF16
1		00000	REV32 (vector)	FEAT_AdvSIMD
1		00010	UADDLP	FEAT_AdvSIMD
1		00011	USQADD	FEAT_AdvSIMD

Decode fields			Instruction Details	Condition
U	size	opcode		
1		00100	CLZ (vector)	FEAT_AdvSIMD
1		00110	UADALP	FEAT_AdvSIMD
1		00111	SQNEG	FEAT_AdvSIMD
1		01000	CMGE (zero)	FEAT_AdvSIMD
1		01001	CMLE (zero)	FEAT_AdvSIMD
1		01010	UNALLOCATED	-
1		01011	NEG (vector)	FEAT_AdvSIMD
1		10010	SQXTUN, SQXTUN2	FEAT_AdvSIMD
1		10011	SHLL, SHLL2	FEAT_AdvSIMD
1		10100	UQXTN, UQXTN2	FEAT_AdvSIMD
1	x0	10110	UNALLOCATED	-
1	0x	00001	UNALLOCATED	-
1	0x	11000	FRINTA (vector)	FEAT_AdvSIMD
1	0x	11001	FRINTX (vector)	FEAT_AdvSIMD
1	0x	11010	FCVTNU (vector)	FEAT_AdvSIMD
1	0x	11011	FCVTMU (vector)	FEAT_AdvSIMD
1	0x	11100	FCVTAU (vector)	FEAT_AdvSIMD
1	0x	11101	UCVTF (vector, integer)	FEAT_AdvSIMD
1	0x	11110	FRINT32X (vector)	FEAT_FRINTTS
1	0x	11111	FRINT64X (vector)	FEAT_FRINTTS
1	00	00101	NOT	FEAT_AdvSIMD
1	00	10111	F1CVTL, F1CVTL2, F2CVTL, F2CVTL2 — F1CVTL{2}	FEAT_FP8
1	01	00101	RBIT (vector)	FEAT_AdvSIMD
1	01	10110	FCVTXN, FCVTXN2	FEAT_AdvSIMD
1	01	10111	F1CVTL, F1CVTL2, F2CVTL, F2CVTL2 — F2CVTL{2}	FEAT_FP8
1	1x	00x01	UNALLOCATED	-
1	1x	01100	FCMGE (zero)	FEAT_AdvSIMD
1	1x	01101	FCMLE (zero)	FEAT_AdvSIMD
1	1x	01110	UNALLOCATED	-
1	1x	01111	FNEG (vector)	FEAT_AdvSIMD
1	1x	11000	UNALLOCATED	-
1	1x	11001	FRINTI (vector)	FEAT_AdvSIMD
1	1x	11010	FCVTPU (vector)	FEAT_AdvSIMD
1	1x	11011	FCVTZU (vector, integer)	FEAT_AdvSIMD
1	1x	11100	URSQRTE	FEAT_AdvSIMD
1	1x	11101	FRSQRTE	FEAT_AdvSIMD
1	1x	11111	FSQRT (vector)	FEAT_AdvSIMD
1	10	10111	BF1CVTL, BF1CVTL2, BF2CVTL, BF2CVTL2 — BF1CVTL{2}	FEAT_FP8
1	11	10111	BF1CVTL, BF1CVTL2, BF2CVTL, BF2CVTL2 — BF2CVTL{2}	FEAT_FP8

Advanced SIMD across lanes

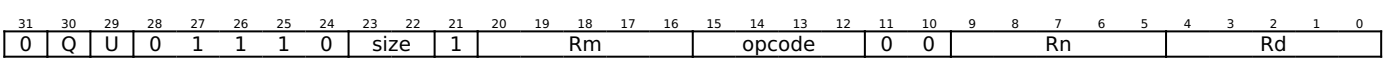
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields				Instruction Details	Condition
Q	U	size	opcode		
			x100x	UNALLOCATED	-
			000x0	UNALLOCATED	-
			00001	UNALLOCATED	-
			01011	UNALLOCATED	-
			100xx	UNALLOCATED	-
		x0	001xx	UNALLOCATED	-
		x0	01101	UNALLOCATED	-
		x0	101xx	UNALLOCATED	-
		x0	111xx	UNALLOCATED	-
		x1	xx1xx	UNALLOCATED	-
	0		00011	SADDLV	FEAT_AdvSIMD
	0		01010	SMAXV	FEAT_AdvSIMD
	0		11010	SMINV	FEAT_AdvSIMD
	0		11011	ADDV	FEAT_AdvSIMD
	0	x0	01110	UNALLOCATED	-
	0	00	01100	FMAXNMV — Half-precision	FEAT_AdvSIMD && FEAT_FP16
	0	00	01111	FMAXV — Half-precision	FEAT_AdvSIMD && FEAT_FP16
	0	10	01100	FMINNMV — Half-precision	FEAT_AdvSIMD && FEAT_FP16
	0	10	01111	FMINV — Half-precision	FEAT_AdvSIMD && FEAT_FP16
	1		00011	UADDLV	FEAT_AdvSIMD
	1		01010	UMAXV	FEAT_AdvSIMD
	1		11010	UMINV	FEAT_AdvSIMD
	1		11011	UNALLOCATED	-
0	1	x0	011x0	UNALLOCATED	-
0	1	x0	01111	UNALLOCATED	-
1	1	x0	01110	UNALLOCATED	-
1	1	00	01100	FMAXNMV — Single-precision	FEAT_AdvSIMD
1	1	00	01111	FMAXV — Single-precision	FEAT_AdvSIMD
1	1	10	01100	FMINNMV — Single-precision	FEAT_AdvSIMD
1	1	10	01111	FMINV — Single-precision	FEAT_AdvSIMD

Advanced SIMD three different

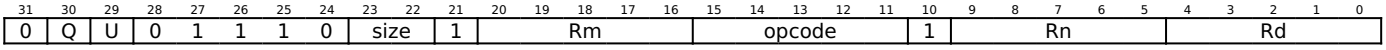
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields		Instruction Details	Condition
U	opcode		
0	0000	SADDL, SADDL2	FEAT_AdvSIMD
0	0001	SADDW, SADDW2	FEAT_AdvSIMD
0	0010	SSUBL, SSUBL2	FEAT_AdvSIMD
0	0011	SSUBW, SSUBW2	FEAT_AdvSIMD
0	0100	ADDHN, ADDHN2	FEAT_AdvSIMD
0	0101	SABAL, SABAL2	FEAT_AdvSIMD
0	0110	SUBHN, SUBHN2	FEAT_AdvSIMD
0	0111	SABDL, SABDL2	FEAT_AdvSIMD
0	1000	SMLAL, SMLAL2 (vector)	FEAT_AdvSIMD
0	1001	SQDMLAL, SQDMLAL2 (vector)	FEAT_AdvSIMD
0	1010	SMLSL, SMLSL2 (vector)	FEAT_AdvSIMD
0	1011	SQDMLSL, SQDMLSL2 (vector)	FEAT_AdvSIMD
0	1100	SMULL, SMULL2 (vector)	FEAT_AdvSIMD
0	1101	SQDMULL, SQDMULL2 (vector)	FEAT_AdvSIMD
0	1110	PMULL, PMULL2	FEAT_AdvSIMD
0	1111	UNALLOCATED	-
1	0000	UADDL, UADDL2	FEAT_AdvSIMD
1	0001	UADDW, UADDW2	FEAT_AdvSIMD
1	0010	USUBL, USUBL2	FEAT_AdvSIMD
1	0011	USUBW, USUBW2	FEAT_AdvSIMD
1	0100	RADDHN, RADDHN2	FEAT_AdvSIMD
1	0101	UABAL, UABAL2	FEAT_AdvSIMD
1	0110	RSUBHN, RSUBHN2	FEAT_AdvSIMD
1	0111	UABDL, UABDL2	FEAT_AdvSIMD
1	1xx1	UNALLOCATED	-
1	1000	UMLAL, UMLAL2 (vector)	FEAT_AdvSIMD
1	1010	UMLSL, UMLSL2 (vector)	FEAT_AdvSIMD
1	1100	UMULL, UMULL2 (vector)	FEAT_AdvSIMD
1	1110	UNALLOCATED	-

Advanced SIMD three same

The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.

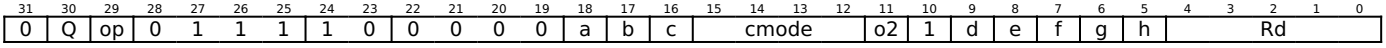


Decode fields			Instruction Details	Condition
U	size	opcode		
0		00000	SHADD	FEAT_AdvSIMD
0		00001	SQADD	FEAT_AdvSIMD
0		00010	SRHADD	FEAT_AdvSIMD
0		00100	SHSUB	FEAT_AdvSIMD
0		00101	SQSUB	FEAT_AdvSIMD
0		00110	CMGT (register)	FEAT_AdvSIMD
0		00111	CMGE (register)	FEAT_AdvSIMD
0		01000	SSHL	FEAT_AdvSIMD
0		01001	SQSHL (register)	FEAT_AdvSIMD
0		01010	SRSHL	FEAT_AdvSIMD
0		01011	SQRSHL	FEAT_AdvSIMD
0		01100	SMAX	FEAT_AdvSIMD
0		01101	SMIN	FEAT_AdvSIMD
0		01110	SABD	FEAT_AdvSIMD
0		01111	SABA	FEAT_AdvSIMD
0		10000	ADD (vector)	FEAT_AdvSIMD
0		10001	CMTST	FEAT_AdvSIMD
0		10010	MLA (vector)	FEAT_AdvSIMD
0		10011	MUL (vector)	FEAT_AdvSIMD
0		10100	SMAXP	FEAT_AdvSIMD
0		10101	SMINP	FEAT_AdvSIMD
0		10110	SQDMULH (vector)	FEAT_AdvSIMD
0		10111	ADDP (vector)	FEAT_AdvSIMD
0	x1	11101	UNALLOCATED	-
0	0x	11000	FMAXNM (vector)	FEAT_AdvSIMD
0	0x	11001	FMLA (vector)	FEAT_AdvSIMD
0	0x	11010	FADD (vector)	FEAT_AdvSIMD
0	0x	11011	FMULX	FEAT_AdvSIMD
0	0x	11100	FCMEQ (register)	FEAT_AdvSIMD
0	0x	11110	FMAX (vector)	FEAT_AdvSIMD
0	0x	11111	FRECPS	FEAT_AdvSIMD
0	00	00011	AND (vector)	FEAT_AdvSIMD
0	00	11101	FMLAL, FMLAL2 (vector) — FMLAL	FEAT_FHM
0	01	00011	BIC (vector, register)	FEAT_AdvSIMD
0	1x	11000	FMINNM (vector)	FEAT_AdvSIMD
0	1x	11001	FMLS (vector)	FEAT_AdvSIMD
0	1x	11010	FSUB (vector)	FEAT_AdvSIMD
0	1x	11011	FAMAX	FEAT_AdvSIMD && FEAT_FAMINMAX
0	1x	11100	UNALLOCATED	-
0	1x	11110	FMIN (vector)	FEAT_AdvSIMD
0	1x	11111	FRSQRTS	FEAT_AdvSIMD
0	10	00011	ORR (vector, register)	FEAT_AdvSIMD
0	10	11101	FMLSL, FMLSL2 (vector) — FMLSL	FEAT_FHM
0	11	00011	ORN (vector)	FEAT_AdvSIMD
1		00000	UHADD	FEAT_AdvSIMD

Decode fields			Instruction Details	Condition
U	size	opcode		
1		00001	UQADD	FEAT_AdvSIMD
1		00010	URHADD	FEAT_AdvSIMD
1		00100	UHSUB	FEAT_AdvSIMD
1		00101	UQSUB	FEAT_AdvSIMD
1		00110	CMHI (register)	FEAT_AdvSIMD
1		00111	CMHS (register)	FEAT_AdvSIMD
1		01000	USHL	FEAT_AdvSIMD
1		01001	UQSHL (register)	FEAT_AdvSIMD
1		01010	URSHL	FEAT_AdvSIMD
1		01011	UQRSHL	FEAT_AdvSIMD
1		01100	UMAX	FEAT_AdvSIMD
1		01101	UMIN	FEAT_AdvSIMD
1		01110	UABD	FEAT_AdvSIMD
1		01111	UABA	FEAT_AdvSIMD
1		10000	SUB (vector)	FEAT_AdvSIMD
1		10001	CMEQ (register)	FEAT_AdvSIMD
1		10010	MLS (vector)	FEAT_AdvSIMD
1		10011	PMUL	FEAT_AdvSIMD
1		10100	UMAXP	FEAT_AdvSIMD
1		10101	UMINP	FEAT_AdvSIMD
1		10110	SQRDMULH (vector)	FEAT_AdvSIMD
1		10111	UNALLOCATED	-
1	x1	11001	UNALLOCATED	-
1	0x	11000	FMAXNMP (vector)	FEAT_AdvSIMD
1	0x	11010	FADDP (vector)	FEAT_AdvSIMD
1	0x	11011	FMUL (vector)	FEAT_AdvSIMD
1	0x	11100	FCMGE (register)	FEAT_AdvSIMD
1	0x	11101	FACGE	FEAT_AdvSIMD
1	0x	11110	FMAXP (vector)	FEAT_AdvSIMD
1	0x	11111	FDIV (vector)	FEAT_AdvSIMD
1	00	00011	EOR (vector)	FEAT_AdvSIMD
1	00	11001	FMLAL, FMLAL2 (vector) — FMLAL2	FEAT_FHM
1	01	00011	BSL	FEAT_AdvSIMD
1	1x	11000	FMINNMP (vector)	FEAT_AdvSIMD
1	1x	11010	FABD	FEAT_AdvSIMD
1	1x	11011	FAMIN	FEAT_AdvSIMD && FEAT_FAMINMAX
1	1x	11100	FCMGT (register)	FEAT_AdvSIMD
1	1x	11101	FACGT	FEAT_AdvSIMD
1	1x	11110	FMINP (vector)	FEAT_AdvSIMD
1	1x	11111	FSCALE	FEAT_FP8
1	10	00011	BIT	FEAT_AdvSIMD
1	10	11001	FMLSL, FMLSL2 (vector) — FMLSL2	FEAT_FHM
1	11	00011	BIF	FEAT_AdvSIMD

Advanced SIMD modified immediate

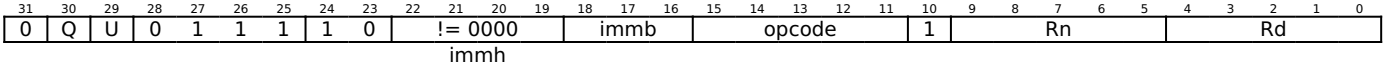
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Q	op	Decode fields cmode	o2	Instruction Details	Condition
	0	0xx0	0	MOVI — 32-bit shifted immediate	FEAT_AdvSIMD
	0	0xx1	0	ORR (vector, immediate) — 32-bit	FEAT_AdvSIMD
	0	10x0	0	MOVI — 16-bit shifted immediate	FEAT_AdvSIMD
	0	10x1	0	ORR (vector, immediate) — 16-bit	FEAT_AdvSIMD
	0	110x	0	MOVI — 32-bit shifting ones	FEAT_AdvSIMD
	0	1110	0	MOVI — 8-bit	FEAT_AdvSIMD
	0	1111	0	FMOV (vector, immediate) — Single-precision	FEAT_AdvSIMD
	0	1111	1	FMOV (vector, immediate) — Half-precision	FEAT_AdvSIMD && FEAT_FP16
	0	!= 1111	1	UNALLOCATED	-
	1		1	UNALLOCATED	-
	1	0xx0	0	MVNI — 32-bit shifted immediate	FEAT_AdvSIMD
	1	0xx1	0	BIC (vector, immediate) — 32-bit	FEAT_AdvSIMD
	1	10x0	0	MVNI — 16-bit shifted immediate	FEAT_AdvSIMD
	1	10x1	0	BIC (vector, immediate) — 16-bit	FEAT_AdvSIMD
	1	110x	0	MVNI — 32-bit shifting ones	FEAT_AdvSIMD
0	1	1110	0	MOVI — 64-bit scalar	FEAT_AdvSIMD
0	1	1111	0	UNALLOCATED	-
1	1	1110	0	MOVI — 64-bit vector	FEAT_AdvSIMD
1	1	1111	0	FMOV (vector, immediate) — Double-precision	FEAT_AdvSIMD

Advanced SIMD shift by immediate

The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.

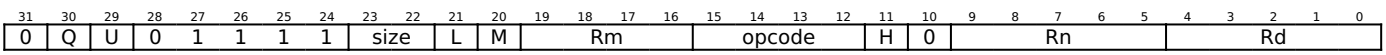


The following constraints also apply to this encoding: immh != '0000'

Decode fields			Instruction Details	Condition
U	immh	opcode		
	!= 0000	0xxx1	UNALLOCATED	-
	!= 0000	1x110	UNALLOCATED	-
	!= 0000	101x1	UNALLOCATED	-
	!= 0000	110xx	UNALLOCATED	-
	!= 0000	11101	UNALLOCATED	-
0	!= 0000	00000	SSHR	FEAT_AdvSIMD
0	!= 0000	00010	SSRA	FEAT_AdvSIMD
0	!= 0000	00100	SRSHR	FEAT_AdvSIMD
0	!= 0000	00110	SRSRA	FEAT_AdvSIMD
0	!= 0000	01x00	UNALLOCATED	-
0	!= 0000	01010	SHL	FEAT_AdvSIMD
0	!= 0000	01110	SQSHL (immediate)	FEAT_AdvSIMD
0	!= 0000	10000	SHRN, SHRN2	FEAT_AdvSIMD
0	!= 0000	10001	RSHRN, RSHRN2	FEAT_AdvSIMD
0	!= 0000	10010	SQSHRN, SQSHRN2	FEAT_AdvSIMD
0	!= 0000	10011	SQRSHRN, SQRSHRN2	FEAT_AdvSIMD
0	!= 0000	10100	SSHLL, SSHLL2	FEAT_AdvSIMD
0	!= 0000	11100	SCVTF (vector, fixed-point)	FEAT_AdvSIMD
0	!= 0000	11111	FCVTZS (vector, fixed-point)	FEAT_AdvSIMD
1	!= 0000	00000	USHR	FEAT_AdvSIMD
1	!= 0000	00010	USRA	FEAT_AdvSIMD
1	!= 0000	00100	URSHR	FEAT_AdvSIMD
1	!= 0000	00110	URSRA	FEAT_AdvSIMD
1	!= 0000	01000	SRI	FEAT_AdvSIMD
1	!= 0000	01010	SLI	FEAT_AdvSIMD
1	!= 0000	01100	SQSHLU	FEAT_AdvSIMD
1	!= 0000	01110	UQSHL (immediate)	FEAT_AdvSIMD
1	!= 0000	10000	SQSHRUN, SQSHRUN2	FEAT_AdvSIMD
1	!= 0000	10001	SQRSHRUN, SQRSHRUN2	FEAT_AdvSIMD
1	!= 0000	10010	UQSHRN, UQSHRN2	FEAT_AdvSIMD
1	!= 0000	10011	UQRSHRN, UQRSHRN2	FEAT_AdvSIMD
1	!= 0000	10100	USHLL, USHLL2	FEAT_AdvSIMD
1	!= 0000	11100	UCVTF (vector, fixed-point)	FEAT_AdvSIMD
1	!= 0000	11111	FCVTZU (vector, fixed-point)	FEAT_AdvSIMD

Advanced SIMD vector x indexed element

The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields				Instruction Details	Condition
Q	U	size	opcode		
	0		0010	SMLAL, SMLAL2 (by element)	FEAT_AdvSIMD
	0		0011	SQDMLAL, SQDMLAL2 (by element)	FEAT_AdvSIMD
	0		0110	SMLSL, SMLSL2 (by element)	FEAT_AdvSIMD
	0		0111	SQDMLSL, SQDMLSL2 (by element)	FEAT_AdvSIMD
	0		1000	MUL (by element)	FEAT_AdvSIMD
	0		1010	SMULL, SMULL2 (by element)	FEAT_AdvSIMD
	0		1011	SQDMULL, SQDMULL2 (by element)	FEAT_AdvSIMD
	0		1100	SQDMULH (by element)	FEAT_AdvSIMD
	0		1101	SQRDMULH (by element)	FEAT_AdvSIMD
	0		1110	SDOT (by element)	FEAT_DotProd
	0	00	0000	FDOT (8-bit floating-point to single-precision, by element)	FEAT_FP8DOT4
	0	00	0001	FMLA (by element) — Vector, half-precision	FEAT_AdvSIMD && FEAT_FP16
	0	00	0101	FMLS (by element) — Vector, half-precision	FEAT_AdvSIMD && FEAT_FP16
	0	00	1001	FMUL (by element) — Vector, half-precision	FEAT_AdvSIMD && FEAT_FP16
	0	00	1111	SUDOT (by element)	FEAT_I8MM
	0	01	0x01	UNALLOCATED	-
	0	01	0000	FDOT (8-bit floating-point to half-precision, by element)	FEAT_FP8DOT2
	0	01	1001	FDOT (half-precision to single-precision, by element)	FEAT_F16F32DOT
	0	01	1111	BFDOT (by element)	FEAT_BF16
	0	1x	0001	FMLA (by element) — Vector, single-precision and double-precision	FEAT_AdvSIMD
	0	1x	0101	FMLS (by element) — Vector, single-precision and double-precision	FEAT_AdvSIMD
	0	1x	1001	FMUL (by element) — Vector, single-precision and double-precision	FEAT_AdvSIMD
	0	10	0000	FMLAL, FMLAL2 (by element) — FMLAL	FEAT_FHM
	0	10	0100	FMLSL, FMLSL2 (by element) — FMLSL	FEAT_FHM
	0	10	1111	USDOT (by element)	FEAT_I8MM
	0	11	1111	BFMLALB, BFMLALT (by element)	FEAT_BF16
	0	!= 10	0100	UNALLOCATED	-
	1		0xx1	FCMLA (by element)	FEAT_FCMA
	1		0000	MLA (by element)	FEAT_AdvSIMD
	1		0010	UMLAL, UMLAL2 (by element)	FEAT_AdvSIMD
	1		0100	MLS (by element)	FEAT_AdvSIMD
	1		0110	UMLSL, UMLSL2 (by element)	FEAT_AdvSIMD
	1		1010	UMULL, UMULL2 (by element)	FEAT_AdvSIMD
	1		1011	UNALLOCATED	-
	1		1101	SQRDMLAH (by element)	FEAT_RDM
	1		1110	UDOT (by element)	FEAT_DotProd
	1		1111	SQRDMLSH (by element)	FEAT_RDM
	1	0x	1100	UNALLOCATED	-
	1	00	1001	FMULX (by element) — Vector, half-precision	FEAT_AdvSIMD && FEAT_FP16
	1	01	1001	UNALLOCATED	-
	1	1x	1001	FMULX (by element) — Vector, single-precision and double-precision	FEAT_AdvSIMD
	1	10	1000	FMLAL, FMLAL2 (by element) — FMLAL2	FEAT_FHM

Decode fields				Instruction Details	Condition
Q	U	size	opcode		
	1	10	1100	FMLSL, FMLSL2 (by element) — FMLSL2	FEAT_FHM
	1	11	1x00	UNALLOCATED	-
0	0	11	0000	FMLALB, FMLALT (by element) — FMLALB	FEAT_FP8FMA
0	1	00	1000	FMLALLBB, FMLALLBT, FMLALLTB, FMLALLTT (by element) — FMLALLBB	FEAT_FP8FMA
0	1	01	1000	FMLALLBB, FMLALLBT, FMLALLTB, FMLALLTT (by element) — FMLALLBT	FEAT_FP8FMA
1	0	11	0000	FMLALB, FMLALT (by element) — FMLALT	FEAT_FP8FMA
1	1	00	1000	FMLALLBB, FMLALLBT, FMLALLTB, FMLALLTT (by element) — FMLALLTB	FEAT_FP8FMA
1	1	01	1000	FMLALLBB, FMLALLBT, FMLALLTB, FMLALLTT (by element) — FMLALLTT	FEAT_FP8FMA

Cryptographic three-register, imm2

The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	1	1	1	0	0	1	0	Rm					1	0	imm2		opcode		Rn					Rd				

Decode fields	Instruction Details	Condition
opcode		
00	SM3TT1A	FEAT_SM3
01	SM3TT1B	FEAT_SM3
10	SM3TT2A	FEAT_SM3
11	SM3TT2B	FEAT_SM3

Cryptographic three-register SHA 512

The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	1	1	1	0	0	1	1	Rm					1	O	0	0	opcode		Rn					Rd				

Decode fields		Instruction Details	Condition
O	opcode		
0	00	SHA512H	FEAT_SHA512
0	01	SHA512H2	FEAT_SHA512
0	10	SHA512SU1	FEAT_SHA512
0	11	RAX1	FEAT_SHA3
1	00	SM3PARTW1	FEAT_SM3
1	01	SM3PARTW2	FEAT_SM3
1	10	SM4EKEY	FEAT_SM4
1	11	UNALLOCATED	-

Cryptographic four-register

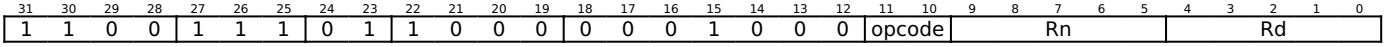
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	1	1	1	0	0	Op0		Rm					0	Ra					Rn					Rd				

Decode fields Op0	Instruction Details	Condition
00	EOR3	FEAT_SHA3
01	BCAX	FEAT_SHA3
10	SM3SS1	FEAT_SM3
11	UNALLOCATED	-

Cryptographic two-register SHA 512

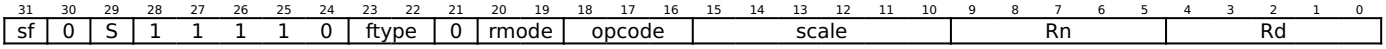
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields opcode	Instruction Details	Condition
00	SHA512SU0	FEAT_SHA512
01	SM4E	FEAT_SM4
1x	UNALLOCATED	-

Conversion between floating-point and fixed-point

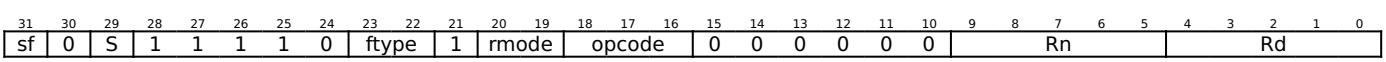
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields					Instruction Details	Condition
sf	S	ftype	rmode	opcode		
	0			1xx	UNALLOCATED	-
	0	10		0xx	UNALLOCATED	-
	0	!= 10	x1	01x	UNALLOCATED	-
	0	!= 10	0x	00x	UNALLOCATED	-
	0	!= 10	10	0xx	UNALLOCATED	-
	1				UNALLOCATED	-
0	0	00	00	010	SCVTF (scalar, fixed-point) — 32-bit to single-precision	FEAT_FP
0	0	00	00	011	UCVTF (scalar, fixed-point) — 32-bit to single-precision	FEAT_FP
0	0	00	11	000	FCVTZS (scalar, fixed-point) — Single-precision to 32-bit	FEAT_FP
0	0	00	11	001	FCVTZU (scalar, fixed-point) — Single-precision to 32-bit	FEAT_FP
0	0	01	00	010	SCVTF (scalar, fixed-point) — 32-bit to double-precision	FEAT_FP
0	0	01	00	011	UCVTF (scalar, fixed-point) — 32-bit to double-precision	FEAT_FP
0	0	01	11	000	FCVTZS (scalar, fixed-point) — Double-precision to 32-bit	FEAT_FP
0	0	01	11	001	FCVTZU (scalar, fixed-point) — Double-precision to 32-bit	FEAT_FP
0	0	11	00	010	SCVTF (scalar, fixed-point) — 32-bit to half-precision	FEAT_FP16
0	0	11	00	011	UCVTF (scalar, fixed-point) — 32-bit to half-precision	FEAT_FP16
0	0	11	11	000	FCVTZS (scalar, fixed-point) — Half-precision to 32-bit	FEAT_FP16
0	0	11	11	001	FCVTZU (scalar, fixed-point) — Half-precision to 32-bit	FEAT_FP16
1	0	00	00	010	SCVTF (scalar, fixed-point) — 64-bit to single-precision	FEAT_FP
1	0	00	00	011	UCVTF (scalar, fixed-point) — 64-bit to single-precision	FEAT_FP
1	0	00	11	000	FCVTZS (scalar, fixed-point) — Single-precision to 64-bit	FEAT_FP
1	0	00	11	001	FCVTZU (scalar, fixed-point) — Single-precision to 64-bit	FEAT_FP
1	0	01	00	010	SCVTF (scalar, fixed-point) — 64-bit to double-precision	FEAT_FP
1	0	01	00	011	UCVTF (scalar, fixed-point) — 64-bit to double-precision	FEAT_FP
1	0	01	11	000	FCVTZS (scalar, fixed-point) — Double-precision to 64-bit	FEAT_FP
1	0	01	11	001	FCVTZU (scalar, fixed-point) — Double-precision to 64-bit	FEAT_FP
1	0	11	00	010	SCVTF (scalar, fixed-point) — 64-bit to half-precision	FEAT_FP16
1	0	11	00	011	UCVTF (scalar, fixed-point) — 64-bit to half-precision	FEAT_FP16
1	0	11	11	000	FCVTZS (scalar, fixed-point) — Half-precision to 64-bit	FEAT_FP16
1	0	11	11	001	FCVTZU (scalar, fixed-point) — Half-precision to 64-bit	FEAT_FP16

Conversion between floating-point and integer

The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields					Instruction Details	Condition
sf	S	ftype	rmode	opcode		
	0	10	x0		UNALLOCATED	-
	0	10	x1	!= 11x	UNALLOCATED	-
	0	11	x1	11x	UNALLOCATED	-
	0	!= 10	01	10x	UNALLOCATED	-
	1				UNALLOCATED	-
0	0	x0	01	11x	UNALLOCATED	-
0	0	00	00	000	FCVTNS (scalar) — Single-precision to 32-bit	FEAT_FP
0	0	00	00	001	FCVTNU (scalar) — Single-precision to 32-bit	FEAT_FP
0	0	00	00	010	SCVTF (scalar, integer) — 32-bit to single-precision	FEAT_FP
0	0	00	00	011	UCVTF (scalar, integer) — 32-bit to single-precision	FEAT_FP
0	0	00	00	100	FCVTAS (scalar) — Single-precision to 32-bit	FEAT_FP
0	0	00	00	101	FCVTAU (scalar) — Single-precision to 32-bit	FEAT_FP
0	0	00	00	110	FMOV (general) — Single-precision to 32-bit	FEAT_FP
0	0	00	00	111	FMOV (general) — 32-bit to single-precision	FEAT_FP
0	0	00	01	000	FCVTPS (scalar) — Single-precision to 32-bit	FEAT_FP
0	0	00	01	001	FCVTPU (scalar) — Single-precision to 32-bit	FEAT_FP
0	0	00	1x	1xx	UNALLOCATED	-
0	0	00	10	000	FCVTMS (scalar) — Single-precision to 32-bit	FEAT_FP
0	0	00	10	001	FCVTMU (scalar) — Single-precision to 32-bit	FEAT_FP
0	0	00	11	000	FCVTZS (scalar, integer) — Single-precision to 32-bit	FEAT_FP
0	0	00	11	001	FCVTZU (scalar, integer) — Single-precision to 32-bit	FEAT_FP
0	0	00	!= 00	01x	UNALLOCATED	-
0	0	01	0x	11x	UNALLOCATED	-
0	0	01	00	000	FCVTNS (scalar) — Double-precision to 32-bit	FEAT_FP
0	0	01	00	001	FCVTNU (scalar) — Double-precision to 32-bit	FEAT_FP
0	0	01	00	010	SCVTF (scalar, integer) — 32-bit to double-precision	FEAT_FP
0	0	01	00	011	UCVTF (scalar, integer) — 32-bit to double-precision	FEAT_FP
0	0	01	00	100	FCVTAS (scalar) — Double-precision to 32-bit	FEAT_FP
0	0	01	00	101	FCVTAU (scalar) — Double-precision to 32-bit	FEAT_FP
0	0	01	01	000	FCVTPS (scalar) — Double-precision to 32-bit	FEAT_FP
0	0	01	01	001	FCVTPU (scalar) — Double-precision to 32-bit	FEAT_FP
0	0	01	01	010	FCVTNS (scalar SIMD&FP) — Double-precision to 32-bit	FEAT_FPRCVT
0	0	01	01	011	FCVTNU (scalar SIMD&FP) — Double-precision to 32-bit	FEAT_FPRCVT
0	0	01	10	000	FCVTMS (scalar) — Double-precision to 32-bit	FEAT_FP
0	0	01	10	001	FCVTMU (scalar) — Double-precision to 32-bit	FEAT_FP
0	0	01	10	010	FCVTPS (scalar SIMD&FP) — Double-precision to 32-bit	FEAT_FPRCVT
0	0	01	10	011	FCVTPU (scalar SIMD&FP) — Double-precision to 32-bit	FEAT_FPRCVT
0	0	01	10	100	FCVTMS (scalar SIMD&FP) — Double-precision to 32-bit	FEAT_FPRCVT
0	0	01	10	101	FCVTMU (scalar SIMD&FP) — Double-precision to 32-bit	FEAT_FPRCVT
0	0	01	10	110	FCVTZS (scalar SIMD&FP) — Double-precision to 32-bit	FEAT_FPRCVT
0	0	01	10	111	FCVTZU (scalar SIMD&FP) — Double-precision to 32-bit	FEAT_FPRCVT
0	0	01	11	000	FCVTZS (scalar, integer) — Double-precision to 32-bit	FEAT_FP
0	0	01	11	001	FCVTZU (scalar, integer) — Double-precision to 32-bit	FEAT_FP
0	0	01	11	010	FCVTAS (scalar SIMD&FP) — Double-precision to 32-bit	FEAT_FPRCVT
0	0	01	11	011	FCVTAU (scalar SIMD&FP) — Double-precision to 32-bit	FEAT_FPRCVT

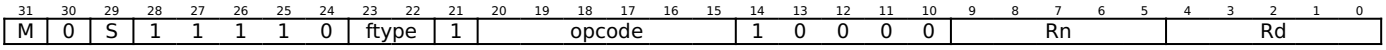
Decode fields					Instruction Details	Condition
sf	S	ftype	rmode	opcode		
0	0	01	11	100	SCVTF (scalar SIMD&FP) — 32-bit to double-precision	FEAT_FPRCVT
0	0	01	11	101	UCVTF (scalar SIMD&FP) — 32-bit to double-precision	FEAT_FPRCVT
0	0	01	11	110	FJCVTZS	FEAT_JSCVT
0	0	01	11	111	UNALLOCATED	-
0	0	10	11	11x	UNALLOCATED	-
0	0	11	00	000	FCVTNS (scalar) — Half-precision to 32-bit	FEAT_FP16
0	0	11	00	001	FCVTNU (scalar) — Half-precision to 32-bit	FEAT_FP16
0	0	11	00	010	SCVTF (scalar, integer) — 32-bit to half-precision	FEAT_FP16
0	0	11	00	011	UCVTF (scalar, integer) — 32-bit to half-precision	FEAT_FP16
0	0	11	00	100	FCVTAS (scalar) — Half-precision to 32-bit	FEAT_FP16
0	0	11	00	101	FCVTAU (scalar) — Half-precision to 32-bit	FEAT_FP16
0	0	11	00	110	FMOV (general) — Half-precision to 32-bit	FEAT_FP16
0	0	11	00	111	FMOV (general) — 32-bit to half-precision	FEAT_FP16
0	0	11	01	000	FCVTPS (scalar) — Half-precision to 32-bit	FEAT_FP16
0	0	11	01	001	FCVTPU (scalar) — Half-precision to 32-bit	FEAT_FP16
0	0	11	01	010	FCVTNS (scalar SIMD&FP) — Half-precision to 32-bit	FEAT_FPRCVT
0	0	11	01	011	FCVTNU (scalar SIMD&FP) — Half-precision to 32-bit	FEAT_FPRCVT
0	0	11	10	000	FCVTMS (scalar) — Half-precision to 32-bit	FEAT_FP16
0	0	11	10	001	FCVTMU (scalar) — Half-precision to 32-bit	FEAT_FP16
0	0	11	10	010	FCVTPS (scalar SIMD&FP) — Half-precision to 32-bit	FEAT_FPRCVT
0	0	11	10	011	FCVTPU (scalar SIMD&FP) — Half-precision to 32-bit	FEAT_FPRCVT
0	0	11	10	100	FCVTMS (scalar SIMD&FP) — Half-precision to 32-bit	FEAT_FPRCVT
0	0	11	10	101	FCVTMU (scalar SIMD&FP) — Half-precision to 32-bit	FEAT_FPRCVT
0	0	11	10	110	FCVTZS (scalar SIMD&FP) — Half-precision to 32-bit	FEAT_FPRCVT
0	0	11	10	111	FCVTZU (scalar SIMD&FP) — Half-precision to 32-bit	FEAT_FPRCVT
0	0	11	11	000	FCVTZS (scalar, integer) — Half-precision to 32-bit	FEAT_FP16
0	0	11	11	001	FCVTZU (scalar, integer) — Half-precision to 32-bit	FEAT_FP16
0	0	11	11	010	FCVTAS (scalar SIMD&FP) — Half-precision to 32-bit	FEAT_FPRCVT
0	0	11	11	011	FCVTAU (scalar SIMD&FP) — Half-precision to 32-bit	FEAT_FPRCVT
0	0	11	11	100	SCVTF (scalar SIMD&FP) — 32-bit to half-precision	FEAT_FPRCVT
0	0	11	11	101	UCVTF (scalar SIMD&FP) — 32-bit to half-precision	FEAT_FPRCVT
1	0	x0	11	11x	UNALLOCATED	-
1	0	00	0x	11x	UNALLOCATED	-
1	0	00	00	000	FCVTNS (scalar) — Single-precision to 64-bit	FEAT_FP
1	0	00	00	001	FCVTNU (scalar) — Single-precision to 64-bit	FEAT_FP
1	0	00	00	010	SCVTF (scalar, integer) — 64-bit to single-precision	FEAT_FP
1	0	00	00	011	UCVTF (scalar, integer) — 64-bit to single-precision	FEAT_FP
1	0	00	00	100	FCVTAS (scalar) — Single-precision to 64-bit	FEAT_FP
1	0	00	00	101	FCVTAU (scalar) — Single-precision to 64-bit	FEAT_FP
1	0	00	01	000	FCVTPS (scalar) — Single-precision to 64-bit	FEAT_FP
1	0	00	01	001	FCVTPU (scalar) — Single-precision to 64-bit	FEAT_FP
1	0	00	01	010	FCVTNS (scalar SIMD&FP) — Single-precision to 64-bit	FEAT_FPRCVT
1	0	00	01	011	FCVTNU (scalar SIMD&FP) — Single-precision to 64-bit	FEAT_FPRCVT
1	0	00	10	000	FCVTMS (scalar) — Single-precision to 64-bit	FEAT_FP
1	0	00	10	001	FCVTMU (scalar) — Single-precision to 64-bit	FEAT_FP
1	0	00	10	010	FCVTPS (scalar SIMD&FP) — Single-precision to 64-bit	FEAT_FPRCVT

Decode fields					Instruction Details	Condition
sf	S	ftype	rmode	opcode		
1	0	00	10	011	FCVTPU (scalar SIMD&FP) — Single-precision to 64-bit	FEAT_FPRCVT
1	0	00	10	100	FCVTMS (scalar SIMD&FP) — Single-precision to 64-bit	FEAT_FPRCVT
1	0	00	10	101	FCVTMU (scalar SIMD&FP) — Single-precision to 64-bit	FEAT_FPRCVT
1	0	00	10	110	FCVTZS (scalar SIMD&FP) — Single-precision to 64-bit	FEAT_FPRCVT
1	0	00	10	111	FCVTZU (scalar SIMD&FP) — Single-precision to 64-bit	FEAT_FPRCVT
1	0	00	11	000	FCVTZS (scalar, integer) — Single-precision to 64-bit	FEAT_FP
1	0	00	11	001	FCVTZU (scalar, integer) — Single-precision to 64-bit	FEAT_FP
1	0	00	11	010	FCVTAS (scalar SIMD&FP) — Single-precision to 64-bit	FEAT_FPRCVT
1	0	00	11	011	FCVTAU (scalar SIMD&FP) — Single-precision to 64-bit	FEAT_FPRCVT
1	0	00	11	100	SCVTF (scalar SIMD&FP) — 64-bit to single-precision	FEAT_FPRCVT
1	0	00	11	101	UCVTF (scalar SIMD&FP) — 64-bit to single-precision	FEAT_FPRCVT
1	0	01	00	000	FCVTNS (scalar) — Double-precision to 64-bit	FEAT_FP
1	0	01	00	001	FCVTNU (scalar) — Double-precision to 64-bit	FEAT_FP
1	0	01	00	010	SCVTF (scalar, integer) — 64-bit to double-precision	FEAT_FP
1	0	01	00	011	UCVTF (scalar, integer) — 64-bit to double-precision	FEAT_FP
1	0	01	00	100	FCVTAS (scalar) — Double-precision to 64-bit	FEAT_FP
1	0	01	00	101	FCVTAU (scalar) — Double-precision to 64-bit	FEAT_FP
1	0	01	00	110	FMOV (general) — Double-precision to 64-bit	FEAT_FP
1	0	01	00	111	FMOV (general) — 64-bit to double-precision	FEAT_FP
1	0	01	01	x1x	UNALLOCATED	-
1	0	01	01	000	FCVTPS (scalar) — Double-precision to 64-bit	FEAT_FP
1	0	01	01	001	FCVTPU (scalar) — Double-precision to 64-bit	FEAT_FP
1	0	01	1x	!= 00x	UNALLOCATED	-
1	0	01	10	000	FCVTMS (scalar) — Double-precision to 64-bit	FEAT_FP
1	0	01	10	001	FCVTMU (scalar) — Double-precision to 64-bit	FEAT_FP
1	0	01	11	000	FCVTZS (scalar, integer) — Double-precision to 64-bit	FEAT_FP
1	0	01	11	001	FCVTZU (scalar, integer) — Double-precision to 64-bit	FEAT_FP
1	0	10	01	110	FMOV (general) — Top half of 128-bit to 64-bit	FEAT_FP
1	0	10	01	111	FMOV (general) — 64-bit to top half of 128-bit	FEAT_FP
1	0	11	00	000	FCVTNS (scalar) — Half-precision to 64-bit	FEAT_FP16
1	0	11	00	001	FCVTNU (scalar) — Half-precision to 64-bit	FEAT_FP16
1	0	11	00	010	SCVTF (scalar, integer) — 64-bit to half-precision	FEAT_FP16
1	0	11	00	011	UCVTF (scalar, integer) — 64-bit to half-precision	FEAT_FP16
1	0	11	00	100	FCVTAS (scalar) — Half-precision to 64-bit	FEAT_FP16
1	0	11	00	101	FCVTAU (scalar) — Half-precision to 64-bit	FEAT_FP16
1	0	11	00	110	FMOV (general) — Half-precision to 64-bit	FEAT_FP16
1	0	11	00	111	FMOV (general) — 64-bit to half-precision	FEAT_FP16
1	0	11	01	000	FCVTPS (scalar) — Half-precision to 64-bit	FEAT_FP16
1	0	11	01	001	FCVTPU (scalar) — Half-precision to 64-bit	FEAT_FP16
1	0	11	01	010	FCVTNS (scalar SIMD&FP) — Half-precision to 64-bit	FEAT_FPRCVT
1	0	11	01	011	FCVTNU (scalar SIMD&FP) — Half-precision to 64-bit	FEAT_FPRCVT
1	0	11	10	000	FCVTMS (scalar) — Half-precision to 64-bit	FEAT_FP16
1	0	11	10	001	FCVTMU (scalar) — Half-precision to 64-bit	FEAT_FP16
1	0	11	10	010	FCVTPS (scalar SIMD&FP) — Half-precision to 64-bit	FEAT_FPRCVT
1	0	11	10	011	FCVTPU (scalar SIMD&FP) — Half-precision to 64-bit	FEAT_FPRCVT
1	0	11	10	100	FCVTMS (scalar SIMD&FP) — Half-precision to 64-bit	FEAT_FPRCVT

Decode fields					Instruction Details	Condition
sf	S	ftype	rmode	opcode		
1	0	11	10	101	FCVTMU (scalar SIMD&FP) — Half-precision to 64-bit	FEAT_FPRCVT
1	0	11	10	110	FCVTZS (scalar SIMD&FP) — Half-precision to 64-bit	FEAT_FPRCVT
1	0	11	10	111	FCVTZU (scalar SIMD&FP) — Half-precision to 64-bit	FEAT_FPRCVT
1	0	11	11	000	FCVTZS (scalar, integer) — Half-precision to 64-bit	FEAT_FP16
1	0	11	11	001	FCVTZU (scalar, integer) — Half-precision to 64-bit	FEAT_FP16
1	0	11	11	010	FCVTAS (scalar SIMD&FP) — Half-precision to 64-bit	FEAT_FPRCVT
1	0	11	11	011	FCVTAU (scalar SIMD&FP) — Half-precision to 64-bit	FEAT_FPRCVT
1	0	11	11	100	SCVTF (scalar SIMD&FP) — 64-bit to half-precision	FEAT_FPRCVT
1	0	11	11	101	UCVTF (scalar SIMD&FP) — 64-bit to half-precision	FEAT_FPRCVT

Floating-point data-processing (1 source)

The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.

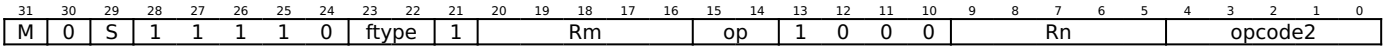


Decode fields				Instruction Details	Condition
M	S	ftype	opcode		
0	0		1xxxxx	UNALLOCATED	-
0	0	0x	0101xx	UNALLOCATED	-
0	0	0x	011xxx	UNALLOCATED	-
0	0	00	000000	FMOV (register) — Single-precision	FEAT_FP
0	0	00	000001	FABS (scalar) — Single-precision	FEAT_FP
0	0	00	000010	FNEG (scalar) — Single-precision	FEAT_FP
0	0	00	000011	FSQRT (scalar) — Single-precision	FEAT_FP
0	0	00	0001x0	UNALLOCATED	-
0	0	00	000101	FCVT — Single-precision to double-precision	FEAT_FP
0	0	00	000111	FCVT — Single-precision to half-precision	FEAT_FP
0	0	00	001000	FRINTN (scalar) — Single-precision	FEAT_FP
0	0	00	001001	FRINTP (scalar) — Single-precision	FEAT_FP
0	0	00	001010	FRINTM (scalar) — Single-precision	FEAT_FP
0	0	00	001011	FRINTZ (scalar) — Single-precision	FEAT_FP
0	0	00	001100	FRINTA (scalar) — Single-precision	FEAT_FP
0	0	00	001110	FRINTX (scalar) — Single-precision	FEAT_FP
0	0	00	001111	FRINTI (scalar) — Single-precision	FEAT_FP
0	0	00	010000	FRINT32Z (scalar) — Single-precision	FEAT_FRINTTS
0	0	00	010001	FRINT32X (scalar) — Single-precision	FEAT_FRINTTS
0	0	00	010010	FRINT64Z (scalar) — Single-precision	FEAT_FRINTTS
0	0	00	010011	FRINT64X (scalar) — Single-precision	FEAT_FRINTTS
0	0	01	000000	FMOV (register) — Double-precision	FEAT_FP
0	0	01	000001	FABS (scalar) — Double-precision	FEAT_FP
0	0	01	000010	FNEG (scalar) — Double-precision	FEAT_FP
0	0	01	000011	FSQRT (scalar) — Double-precision	FEAT_FP
0	0	01	000100	FCVT — Double-precision to single-precision	FEAT_FP
0	0	01	000101	UNALLOCATED	-
0	0	01	000110	BFCVT	FEAT_BF16
0	0	01	000111	FCVT — Double-precision to half-precision	FEAT_FP
0	0	01	001000	FRINTN (scalar) — Double-precision	FEAT_FP
0	0	01	001001	FRINTP (scalar) — Double-precision	FEAT_FP
0	0	01	001010	FRINTM (scalar) — Double-precision	FEAT_FP
0	0	01	001011	FRINTZ (scalar) — Double-precision	FEAT_FP
0	0	01	001100	FRINTA (scalar) — Double-precision	FEAT_FP
0	0	01	001110	FRINTX (scalar) — Double-precision	FEAT_FP
0	0	01	001111	FRINTI (scalar) — Double-precision	FEAT_FP
0	0	01	010000	FRINT32Z (scalar) — Double-precision	FEAT_FRINTTS
0	0	01	010001	FRINT32X (scalar) — Double-precision	FEAT_FRINTTS
0	0	01	010010	FRINT64Z (scalar) — Double-precision	FEAT_FRINTTS
0	0	01	010011	FRINT64X (scalar) — Double-precision	FEAT_FRINTTS
0	0	10	0xxxxx	UNALLOCATED	-
0	0	11	000000	FMOV (register) — Half-precision	FEAT_FP16
0	0	11	000001	FABS (scalar) — Half-precision	FEAT_FP16
0	0	11	000010	FNEG (scalar) — Half-precision	FEAT_FP16
0	0	11	000011	FSQRT (scalar) — Half-precision	FEAT_FP16

M	S	Decode fields ftype	opcode	Instruction Details	Condition
0	0	11	000100	FCVT — Half-precision to single-precision	FEAT_FP
0	0	11	000101	FCVT — Half-precision to double-precision	FEAT_FP
0	0	11	00011x	UNALLOCATED	-
0	0	11	001000	FRINTN (scalar) — Half-precision	FEAT_FP16
0	0	11	001001	FRINTP (scalar) — Half-precision	FEAT_FP16
0	0	11	001010	FRINTM (scalar) — Half-precision	FEAT_FP16
0	0	11	001011	FRINTZ (scalar) — Half-precision	FEAT_FP16
0	0	11	001100	FRINTA (scalar) — Half-precision	FEAT_FP16
0	0	11	001110	FRINTX (scalar) — Half-precision	FEAT_FP16
0	0	11	001111	FRINTI (scalar) — Half-precision	FEAT_FP16
0	0	11	01xxxx	UNALLOCATED	-
0	0	!= 10	001101	UNALLOCATED	-
0	1			UNALLOCATED	-
1				UNALLOCATED	-

Floating-point compare

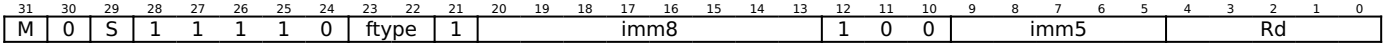
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



M	S	Decode fields ftype	op	opcode2	Instruction Details	Condition
0	0		00	!= xx000	UNALLOCATED	-
0	0		!= 00		UNALLOCATED	-
0	0	00	00	00000	FCMP — Single-precision	FEAT_FP
0	0	00	00	01000	FCMP — Single-precision, zero	FEAT_FP
0	0	00	00	10000	FCMPE — Single-precision	FEAT_FP
0	0	00	00	11000	FCMPE — Single-precision, zero	FEAT_FP
0	0	01	00	00000	FCMP — Double-precision	FEAT_FP
0	0	01	00	01000	FCMP — Double-precision, zero	FEAT_FP
0	0	01	00	10000	FCMPE — Double-precision	FEAT_FP
0	0	01	00	11000	FCMPE — Double-precision, zero	FEAT_FP
0	0	10	00	xx000	UNALLOCATED	-
0	0	11	00	00000	FCMP — Half-precision	FEAT_FP16
0	0	11	00	01000	FCMP — Half-precision, zero	FEAT_FP16
0	0	11	00	10000	FCMPE — Half-precision	FEAT_FP16
0	0	11	00	11000	FCMPE — Half-precision, zero	FEAT_FP16
0	1				UNALLOCATED	-
1					UNALLOCATED	-

Floating-point immediate

The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields				Instruction Details	Condition
M	S	ftype	imm5		
0	0		!= 00000	UNALLOCATED	-
0	0	00	00000	FMOV (scalar, immediate) — Single-precision	FEAT_FP
0	0	01	00000	FMOV (scalar, immediate) — Double-precision	FEAT_FP
0	0	10	00000	UNALLOCATED	-
0	0	11	00000	FMOV (scalar, immediate) — Half-precision	FEAT_FP16
0	1			UNALLOCATED	-
1				UNALLOCATED	-

Floating-point conditional compare

The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
M	0	S	1	1	1	1	0	ftype		1	Rm					cond				0	1	Rn					op	nzcv			

Decode fields				Instruction Details	Condition
M	S	ftype	op		
0	0	00	0	FCCMP — Single-precision	FEAT_FP
0	0	00	1	FCCMPE — Single-precision	FEAT_FP
0	0	01	0	FCCMP — Double-precision	FEAT_FP
0	0	01	1	FCCMPE — Double-precision	FEAT_FP
0	0	10		UNALLOCATED	-
0	0	11	0	FCCMP — Half-precision	FEAT_FP16
0	0	11	1	FCCMPE — Half-precision	FEAT_FP16
0	1			UNALLOCATED	-
1				UNALLOCATED	-

Floating-point data-processing (2 source)

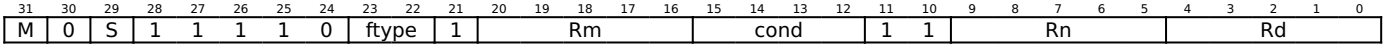
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
M	0	S	1	1	1	1	0	ftype		1	Rm					opcode				1	0	Rn					Rd				

Decode fields				Instruction Details	Condition
M	S	ftype	opcode		
0	0	00	0000	FMUL (scalar) — Single-precision	FEAT_FP
0	0	00	0001	FDIV (scalar) — Single-precision	FEAT_FP
0	0	00	0010	FADD (scalar) — Single-precision	FEAT_FP
0	0	00	0011	FSUB (scalar) — Single-precision	FEAT_FP
0	0	00	0100	FMAX (scalar) — Single-precision	FEAT_FP
0	0	00	0101	FMIN (scalar) — Single-precision	FEAT_FP
0	0	00	0110	FMAXNM (scalar) — Single-precision	FEAT_FP
0	0	00	0111	FMINNM (scalar) — Single-precision	FEAT_FP
0	0	00	1000	FNMUL (scalar) — Single-precision	FEAT_FP
0	0	01	0000	FMUL (scalar) — Double-precision	FEAT_FP
0	0	01	0001	FDIV (scalar) — Double-precision	FEAT_FP
0	0	01	0010	FADD (scalar) — Double-precision	FEAT_FP
0	0	01	0011	FSUB (scalar) — Double-precision	FEAT_FP
0	0	01	0100	FMAX (scalar) — Double-precision	FEAT_FP
0	0	01	0101	FMIN (scalar) — Double-precision	FEAT_FP
0	0	01	0110	FMAXNM (scalar) — Double-precision	FEAT_FP
0	0	01	0111	FMINNM (scalar) — Double-precision	FEAT_FP
0	0	01	1000	FNMUL (scalar) — Double-precision	FEAT_FP
0	0	10		UNALLOCATED	-
0	0	11	0000	FMUL (scalar) — Half-precision	FEAT_FP16
0	0	11	0001	FDIV (scalar) — Half-precision	FEAT_FP16
0	0	11	0010	FADD (scalar) — Half-precision	FEAT_FP16
0	0	11	0011	FSUB (scalar) — Half-precision	FEAT_FP16
0	0	11	0100	FMAX (scalar) — Half-precision	FEAT_FP16
0	0	11	0101	FMIN (scalar) — Half-precision	FEAT_FP16
0	0	11	0110	FMAXNM (scalar) — Half-precision	FEAT_FP16
0	0	11	0111	FMINNM (scalar) — Half-precision	FEAT_FP16
0	0	11	1000	FNMUL (scalar) — Half-precision	FEAT_FP16
0	0	!= 10	1001	UNALLOCATED	-
0	0	!= 10	101x	UNALLOCATED	-
0	0	!= 10	11xx	UNALLOCATED	-
0	1			UNALLOCATED	-
1				UNALLOCATED	-

Floating-point conditional select

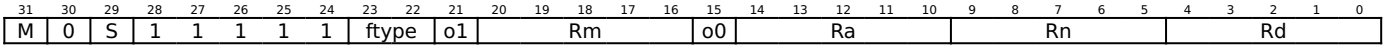
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields			Instruction Details	Condition
M	S	ftype		
0	0	00	FCSEL — Single-precision	FEAT_FP
0	0	01	FCSEL — Double-precision	FEAT_FP
0	0	10	UNALLOCATED	-
0	0	11	FCSEL — Half-precision	FEAT_FP16
0	1		UNALLOCATED	-
1			UNALLOCATED	-

Floating-point data-processing (3 source)

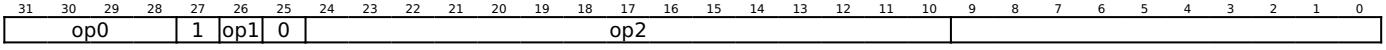
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields					Instruction Details	Condition
M	S	ftype	o1	o0		
0	0	00	0	0	FMADD — Single-precision	FEAT_FP
0	0	00	0	1	FMSUB — Single-precision	FEAT_FP
0	0	00	1	0	FNMADD — Single-precision	FEAT_FP
0	0	00	1	1	FNMSUB — Single-precision	FEAT_FP
0	0	01	0	0	FMADD — Double-precision	FEAT_FP
0	0	01	0	1	FMSUB — Double-precision	FEAT_FP
0	0	01	1	0	FNMADD — Double-precision	FEAT_FP
0	0	01	1	1	FNMSUB — Double-precision	FEAT_FP
0	0	10			UNALLOCATED	-
0	0	11	0	0	FMADD — Half-precision	FEAT_FP16
0	0	11	0	1	FMSUB — Half-precision	FEAT_FP16
0	0	11	1	0	FNMADD — Half-precision	FEAT_FP16
0	0	11	1	1	FNMSUB — Half-precision	FEAT_FP16
0	1				UNALLOCATED	-
1					UNALLOCATED	-

Loads and Stores

The encodings in this section are decoded from A64 instruction set encoding.

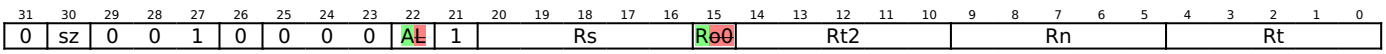


Decode fields			Instruction details
op0	op1	op2	
0000	0	1xx1xxxxxxxxxx	UNALLOCATED
0001	1	1xx1xxxxxxxxxx	UNALLOCATED
0x00	0	00x1xxxxxxxxxx	Compare and swap pair
0x00	0	10x0xxxxxxxxxx	UNALLOCATED
0x00	0	11x0xxxxxxxxxx	Compare and swap pair (unprivileged)
0x00	1	00x000000xxxxxx	Advanced SIMD load/store multiple structures
0x00	1	00x000001xxxxxx	UNALLOCATED
0x00	1	01x0xxxxxxxxxx	Advanced SIMD load/store multiple structures (post-indexed)
0x00	1	0xx1xxxxxxxxxx	UNALLOCATED
0x00	1	10x10001xxxxxxx	UNALLOCATED
0x00	1	10x1001xxxxxxxx	UNALLOCATED
0x00	1	10x101xxxxxxxxx	UNALLOCATED
0x00	1	10x11xxxxxxxxxx	UNALLOCATED
0x00	1	10xx0000xxxxxxx	Advanced SIMD load/store single structure
0x00	1	11xxxxxxxxxxxx	Advanced SIMD load/store single structure (post-indexed)
0x00	1	x0x00001xxxxxxx	UNALLOCATED
0x00	1	x0x0001xxxxxxxx	UNALLOCATED
0x00	1	x0x001xxxxxxxxx	UNALLOCATED
0x00	1	x0x01xxxxxxxxx	UNALLOCATED
0x01	0	1xx0xxxxxxxx11	UNALLOCATED
0x01	0	1xx1xxxxx000010	RCW compare and swap
0x01	0	1xx1xxxxx000011	RCW compare and swap pair
0x01	0	1xx1xxxxx00011x	UNALLOCATED
0x01	0	1xx1xxxxx001x1x	UNALLOCATED
0x01	0	1xx1xxxxx01xx1x	UNALLOCATED
0x01	0	1xx1xxxxx1xxx1x	UNALLOCATED
0x01	0	1xx1xxxxxxxxx00	128-bit atomic memory operations
0x01	0	1xx1xxxxxxxxx01	Atomic memory operations (unprivileged)
1001	0	1xx0xxxxxxxx11	UNALLOCATED
1001	1	1xx1xxxxxxxxxx	UNALLOCATED
100x	0	1xx1xxxxxxxxxx	UNALLOCATED
1101	0	1000111110xxx11	GCS load/store
1101	0	1100111110xxx11	UNALLOCATED
1101	0	1x000xxxxxxxx11	UNALLOCATED
1101	0	1x0010xxxxxxx11	UNALLOCATED
1101	0	1x00110xxxxxx11	UNALLOCATED
1101	0	1x001110xxxxx11	UNALLOCATED
1101	0	1x0011110xxxx11	UNALLOCATED
1101	0	1x00111111xxx11	UNALLOCATED
1101	0	1x10xxxxxxxx11	UNALLOCATED
1101	0	1xx1xxxxxxxxxx	Load/store memory tags
1x00	0	00x1xxxxxxxxxx	Load/store exclusive pair
1x00	0	10x0xxxxxxxxxx	Load/store exclusive register (unprivileged)
1x00	0	11x0xxxxxxxxxx	Compare and swap (unprivileged)
1x00	1		UNALLOCATED

x100	0	1xx1xxxxxxxxxx	UNALLOCATED
x101	1	1xx1xxxxxxxxxx	UNALLOCATED
xx00	0	00x0xxxxxxxxxx	Load/store exclusive register
xx00	0	01x0xxxxxxxxxx	Load/store ordered
xx00	0	01x1xxxxxxxxxx	Compare and swap
xx01	0	10x0xxxxxxxxx10	Load/store ordered register pair
xx01	0	11x000000000010	Load/store ordered (writeback)
xx01	0	11x000000000110	UNALLOCATED
xx01	0	11x000000001x10	UNALLOCATED
xx01	0	11x00000001xx10	UNALLOCATED
xx01	0	11x0000001xxx10	UNALLOCATED
xx01	0	11x000001xxxx10	UNALLOCATED
xx01	0	11x00001xxxxx10	UNALLOCATED
xx01	0	11x0001xxxxxx10	UNALLOCATED
xx01	0	11x001xxxxxx10	UNALLOCATED
xx01	0	11x01xxxxxxx10	UNALLOCATED
xx01	0	1xx0xxxxxxxx00	Load/store ordered (unscaled immediate)
xx01	1	1xx0xxxxxxxx00	UNALLOCATED
xx01	1	1xx0xxxxxxxx10	Load/store ordered (SIMD&FP)
xx01	1	1xx0xxxxxxxx11	UNALLOCATED
xx01		0xxxxxxxxxxxxx	Load register (literal)
xx01		1xx0xxxxxxxx01	Memory Copy and Memory Set
xx10		00xxxxxxxxxxxx	Load/store no-allocate pair (offset)
xx10		01xxxxxxxxxxxx	Load/store register pair (post-indexed)
xx10		10xxxxxxxxxxxx	Load/store register pair (offset)
xx10		11xxxxxxxxxxxx	Load/store register pair (pre-indexed)
xx11		0xx0xxxxxxxx00	Load/store register (unscaled immediate)
xx11		0xx0xxxxxxxx01	Load/store register (immediate post-indexed)
xx11		0xx0xxxxxxxx10	Load/store register (unprivileged)
xx11		0xx0xxxxxxxx11	Load/store register (immediate pre-indexed)
xx11		0xx1xxxxxxxx00	Atomic memory operations
xx11		0xx1xxxxxxxx10	Load/store register (register offset)
xx11		0xx1xxxxxxxxx1	Load/store register (pac)
xx11		1xxxxxxxxxxxxx	Load/store register (unsigned immediate)

Compare and swap pair

The encodings in this section are decoded from Loads and Stores.



Decode fields				Instruction Details	Condition
sz	AL	Ro0	Rt2		
			!= 11111	UNALLOCATED	-
0	0	0	11111	CASP, CASPA, CASPAL, CASPL — 32-bit CASP	FEAT_LSE
0	0	1	11111	CASP, CASPA, CASPAL, CASPL — 32-bit CASPL	FEAT_LSE
0	1	0	11111	CASP, CASPA, CASPAL, CASPL — 32-bit CASPA	FEAT_LSE
0	1	1	11111	CASP, CASPA, CASPAL, CASPL — 32-bit CASPAL	FEAT_LSE
1	0	0	11111	CASP, CASPA, CASPAL, CASPL — 64-bit CASP	FEAT_LSE
1	0	1	11111	CASP, CASPA, CASPAL, CASPL — 64-bit CASPL	FEAT_LSE
1	1	0	11111	CASP, CASPA, CASPAL, CASPL — 64-bit CASPA	FEAT_LSE
1	1	1	11111	CASP, CASPA, CASPAL, CASPL — 64-bit CASPAL	FEAT_LSE

Compare and swap pair (unprivileged)

The encodings in this section are decoded from Loads and Stores.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	sz	0	0	1	0	0	1	1	AL	0	Rs					Ro0	Rt2					Rn					Rt				

Decode fields				Instruction Details	Condition
sz	AL	Ro0	Rt2		
0				UNALLOCATED	-
1			!= 11111	UNALLOCATED	-
1	0	0	11111	CASPT, CASPAT, CASPALT, CASPLT — CASPT	FEAT_LSUI
1	0	1	11111	CASPT, CASPAT, CASPALT, CASPLT — CASPLT	FEAT_LSUI
1	1	0	11111	CASPT, CASPAT, CASPALT, CASPLT — CASPAT	FEAT_LSUI
1	1	1	11111	CASPT, CASPAT, CASPALT, CASPLT — CASPALT	FEAT_LSUI

Advanced SIMD load/store multiple structures

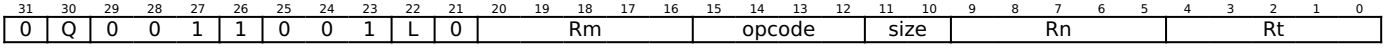
The encodings in this section are decoded from Loads and Stores.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	Q	0	0	1	1	0	0	0	L	0	0	0	0	0	0	opcode				size		Rn					Rt				

Decode fields L	Decode fields opcode	Instruction Details	Condition
	x0x1	UNALLOCATED	-
	0101	UNALLOCATED	-
	11xx	UNALLOCATED	-
0	0000	ST4 (multiple structures)	FEAT_AdvSIMD
0	0010	ST1 (multiple structures) — Four registers	FEAT_AdvSIMD
0	0100	ST3 (multiple structures)	FEAT_AdvSIMD
0	0110	ST1 (multiple structures) — Three registers	FEAT_AdvSIMD
0	0111	ST1 (multiple structures) — One register	FEAT_AdvSIMD
0	1000	ST2 (multiple structures)	FEAT_AdvSIMD
0	1010	ST1 (multiple structures) — Two registers	FEAT_AdvSIMD
1	0000	LD4 (multiple structures)	FEAT_AdvSIMD
1	0010	LD1 (multiple structures) — Four registers	FEAT_AdvSIMD
1	0100	LD3 (multiple structures)	FEAT_AdvSIMD
1	0110	LD1 (multiple structures) — Three registers	FEAT_AdvSIMD
1	0111	LD1 (multiple structures) — One register	FEAT_AdvSIMD
1	1000	LD2 (multiple structures)	FEAT_AdvSIMD
1	1010	LD1 (multiple structures) — Two registers	FEAT_AdvSIMD

Advanced SIMD load/store multiple structures (post-indexed)

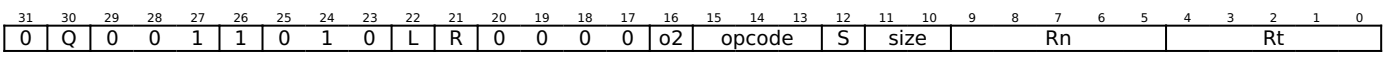
The encodings in this section are decoded from Loads and Stores.



Decode fields			Instruction Details	Condition
L	Rm	opcode		
		x0x1	UNALLOCATED	-
		0101	UNALLOCATED	-
		11xx	UNALLOCATED	-
0	11111	0000	ST4 (multiple structures) — Immediate offset	FEAT_AdvSIMD
0	11111	0010	ST1 (multiple structures) — Four registers, immediate offset	FEAT_AdvSIMD
0	11111	0100	ST3 (multiple structures) — Immediate offset	FEAT_AdvSIMD
0	11111	0110	ST1 (multiple structures) — Three registers, immediate offset	FEAT_AdvSIMD
0	11111	0111	ST1 (multiple structures) — One register, immediate offset	FEAT_AdvSIMD
0	11111	1000	ST2 (multiple structures) — Immediate offset	FEAT_AdvSIMD
0	11111	1010	ST1 (multiple structures) — Two registers, immediate offset	FEAT_AdvSIMD
0	!= 11111	0000	ST4 (multiple structures) — Register offset	FEAT_AdvSIMD
0	!= 11111	0010	ST1 (multiple structures) — Four registers, register offset	FEAT_AdvSIMD
0	!= 11111	0100	ST3 (multiple structures) — Register offset	FEAT_AdvSIMD
0	!= 11111	0110	ST1 (multiple structures) — Three registers, register offset	FEAT_AdvSIMD
0	!= 11111	0111	ST1 (multiple structures) — One register, register offset	FEAT_AdvSIMD
0	!= 11111	1000	ST2 (multiple structures) — Register offset	FEAT_AdvSIMD
0	!= 11111	1010	ST1 (multiple structures) — Two registers, register offset	FEAT_AdvSIMD
1	11111	0000	LD4 (multiple structures) — Immediate offset	FEAT_AdvSIMD
1	11111	0010	LD1 (multiple structures) — Four registers, immediate offset	FEAT_AdvSIMD
1	11111	0100	LD3 (multiple structures) — Immediate offset	FEAT_AdvSIMD
1	11111	0110	LD1 (multiple structures) — Three registers, immediate offset	FEAT_AdvSIMD
1	11111	0111	LD1 (multiple structures) — One register, immediate offset	FEAT_AdvSIMD
1	11111	1000	LD2 (multiple structures) — Immediate offset	FEAT_AdvSIMD
1	11111	1010	LD1 (multiple structures) — Two registers, immediate offset	FEAT_AdvSIMD
1	!= 11111	0000	LD4 (multiple structures) — Register offset	FEAT_AdvSIMD
1	!= 11111	0010	LD1 (multiple structures) — Four registers, register offset	FEAT_AdvSIMD
1	!= 11111	0100	LD3 (multiple structures) — Register offset	FEAT_AdvSIMD
1	!= 11111	0110	LD1 (multiple structures) — Three registers, register offset	FEAT_AdvSIMD
1	!= 11111	0111	LD1 (multiple structures) — One register, register offset	FEAT_AdvSIMD
1	!= 11111	1000	LD2 (multiple structures) — Register offset	FEAT_AdvSIMD
1	!= 11111	1010	LD1 (multiple structures) — Two registers, register offset	FEAT_AdvSIMD

Advanced SIMD load/store single structure

The encodings in this section are decoded from Loads and Stores.



Decode fields						Instruction Details	Condition
L	R	o2	opcode	S	size		
		0	01x		x1	UNALLOCATED	-
		0	10x		1x	UNALLOCATED	-
		0	10x	1	01	UNALLOCATED	-
	0	1	100		!= 01	UNALLOCATED	-
	0	1	100	1	01	UNALLOCATED	-
	0	1	!= 100			UNALLOCATED	-
	1	1				UNALLOCATED	-
0		0	11x			UNALLOCATED	-
0	0	0	000			ST1 (single structure) — 8-bit	FEAT_AdvSIMD
0	0	0	001			ST3 (single structure) — 8-bit	FEAT_AdvSIMD
0	0	0	010		x0	ST1 (single structure) — 16-bit	FEAT_AdvSIMD
0	0	0	011		x0	ST3 (single structure) — 16-bit	FEAT_AdvSIMD
0	0	0	100		00	ST1 (single structure) — 32-bit	FEAT_AdvSIMD
0	0	0	100	0	01	ST1 (single structure) — 64-bit	FEAT_AdvSIMD
0	0	0	101		00	ST3 (single structure) — 32-bit	FEAT_AdvSIMD
0	0	0	101	0	01	ST3 (single structure) — 64-bit	FEAT_AdvSIMD
0	0	1	100	0	01	STL1 (SIMD&FP)	FEAT_AdvSIMD && FEAT_LRCPC3
0	1	0	000			ST2 (single structure) — 8-bit	FEAT_AdvSIMD
0	1	0	001			ST4 (single structure) — 8-bit	FEAT_AdvSIMD
0	1	0	010		x0	ST2 (single structure) — 16-bit	FEAT_AdvSIMD
0	1	0	011		x0	ST4 (single structure) — 16-bit	FEAT_AdvSIMD
0	1	0	100		00	ST2 (single structure) — 32-bit	FEAT_AdvSIMD
0	1	0	100	0	01	ST2 (single structure) — 64-bit	FEAT_AdvSIMD
0	1	0	101		00	ST4 (single structure) — 32-bit	FEAT_AdvSIMD
0	1	0	101	0	01	ST4 (single structure) — 64-bit	FEAT_AdvSIMD
1		0	11x	1		UNALLOCATED	-
1	0	0	000			LD1 (single structure) — 8-bit	FEAT_AdvSIMD
1	0	0	001			LD3 (single structure) — 8-bit	FEAT_AdvSIMD
1	0	0	010		x0	LD1 (single structure) — 16-bit	FEAT_AdvSIMD
1	0	0	011		x0	LD3 (single structure) — 16-bit	FEAT_AdvSIMD
1	0	0	100		00	LD1 (single structure) — 32-bit	FEAT_AdvSIMD
1	0	0	100	0	01	LD1 (single structure) — 64-bit	FEAT_AdvSIMD
1	0	0	101		00	LD3 (single structure) — 32-bit	FEAT_AdvSIMD
1	0	0	101	0	01	LD3 (single structure) — 64-bit	FEAT_AdvSIMD
1	0	0	110	0		LD1R	FEAT_AdvSIMD
1	0	0	111	0		LD3R	FEAT_AdvSIMD
1	0	1	100	0	01	LDAP1 (SIMD&FP)	FEAT_AdvSIMD && FEAT_LRCPC3
1	1	0	000			LD2 (single structure) — 8-bit	FEAT_AdvSIMD
1	1	0	001			LD4 (single structure) — 8-bit	FEAT_AdvSIMD
1	1	0	010		x0	LD2 (single structure) — 16-bit	FEAT_AdvSIMD
1	1	0	011		x0	LD4 (single structure) — 16-bit	FEAT_AdvSIMD
1	1	0	100		00	LD2 (single structure) — 32-bit	FEAT_AdvSIMD
1	1	0	100	0	01	LD2 (single structure) — 64-bit	FEAT_AdvSIMD
1	1	0	101		00	LD4 (single structure) — 32-bit	FEAT_AdvSIMD
1	1	0	101	0	01	LD4 (single structure) — 64-bit	FEAT_AdvSIMD

Decode fields						Instruction Details	Condition
L	R	o2	opcode	S	size		
1	1	0	110	0		LD2R	FEAT_AdvSIMD
1	1	0	111	0		LD4R	FEAT_AdvSIMD

Advanced SIMD load/store single structure (post-indexed)

The encodings in this section are decoded from Loads and Stores.

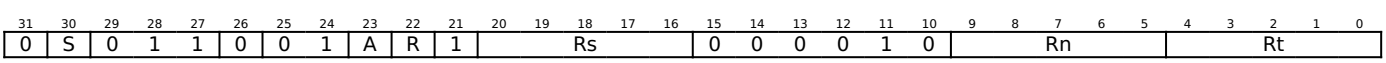
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	Q	0	0	1	1	0	1	1	L	R	Rm					opcode			S	size		Rn					Rt				

Decode fields						Instruction Details	Condition
L	R	Rm	opcode	S	size		
			01x		x1	UNALLOCATED	-
			10x		1x	UNALLOCATED	-
			10x	1	01	UNALLOCATED	-
0			11x			UNALLOCATED	-
0	0	11111	000			ST1 (single structure) — 8-bit, immediate offset	FEAT_AdvSIMD
0	0	11111	001			ST3 (single structure) — 8-bit, immediate offset	FEAT_AdvSIMD
0	0	11111	010		x0	ST1 (single structure) — 16-bit, immediate offset	FEAT_AdvSIMD
0	0	11111	011		x0	ST3 (single structure) — 16-bit, immediate offset	FEAT_AdvSIMD
0	0	11111	100		00	ST1 (single structure) — 32-bit, immediate offset	FEAT_AdvSIMD
0	0	11111	100	0	01	ST1 (single structure) — 64-bit, immediate offset	FEAT_AdvSIMD
0	0	11111	101		00	ST3 (single structure) — 32-bit, immediate offset	FEAT_AdvSIMD
0	0	11111	101	0	01	ST3 (single structure) — 64-bit, immediate offset	FEAT_AdvSIMD
0	0	!= 11111	000			ST1 (single structure) — 8-bit, register offset	FEAT_AdvSIMD
0	0	!= 11111	001			ST3 (single structure) — 8-bit, register offset	FEAT_AdvSIMD
0	0	!= 11111	010		x0	ST1 (single structure) — 16-bit, register offset	FEAT_AdvSIMD
0	0	!= 11111	011		x0	ST3 (single structure) — 16-bit, register offset	FEAT_AdvSIMD
0	0	!= 11111	100		00	ST1 (single structure) — 32-bit, register offset	FEAT_AdvSIMD
0	0	!= 11111	100	0	01	ST1 (single structure) — 64-bit, register offset	FEAT_AdvSIMD
0	0	!= 11111	101		00	ST3 (single structure) — 32-bit, register offset	FEAT_AdvSIMD
0	0	!= 11111	101	0	01	ST3 (single structure) — 64-bit, register offset	FEAT_AdvSIMD
0	1	11111	000			ST2 (single structure) — 8-bit, immediate offset	FEAT_AdvSIMD
0	1	11111	001			ST4 (single structure) — 8-bit, immediate offset	FEAT_AdvSIMD
0	1	11111	010		x0	ST2 (single structure) — 16-bit, immediate offset	FEAT_AdvSIMD
0	1	11111	011		x0	ST4 (single structure) — 16-bit, immediate offset	FEAT_AdvSIMD
0	1	11111	100		00	ST2 (single structure) — 32-bit, immediate offset	FEAT_AdvSIMD
0	1	11111	100	0	01	ST2 (single structure) — 64-bit, immediate offset	FEAT_AdvSIMD
0	1	11111	101		00	ST4 (single structure) — 32-bit, immediate offset	FEAT_AdvSIMD
0	1	11111	101	0	01	ST4 (single structure) — 64-bit, immediate offset	FEAT_AdvSIMD
0	1	!= 11111	000			ST2 (single structure) — 8-bit, register offset	FEAT_AdvSIMD
0	1	!= 11111	001			ST4 (single structure) — 8-bit, register offset	FEAT_AdvSIMD
0	1	!= 11111	010		x0	ST2 (single structure) — 16-bit, register offset	FEAT_AdvSIMD
0	1	!= 11111	011		x0	ST4 (single structure) — 16-bit, register offset	FEAT_AdvSIMD
0	1	!= 11111	100		00	ST2 (single structure) — 32-bit, register offset	FEAT_AdvSIMD
0	1	!= 11111	100	0	01	ST2 (single structure) — 64-bit, register offset	FEAT_AdvSIMD
0	1	!= 11111	101		00	ST4 (single structure) — 32-bit, register offset	FEAT_AdvSIMD
0	1	!= 11111	101	0	01	ST4 (single structure) — 64-bit, register offset	FEAT_AdvSIMD
1			11x	1		UNALLOCATED	-
1	0	11111	000			LD1 (single structure) — 8-bit, immediate offset	FEAT_AdvSIMD
1	0	11111	001			LD3 (single structure) — 8-bit, immediate offset	FEAT_AdvSIMD
1	0	11111	010		x0	LD1 (single structure) — 16-bit, immediate offset	FEAT_AdvSIMD
1	0	11111	011		x0	LD3 (single structure) — 16-bit, immediate offset	FEAT_AdvSIMD
1	0	11111	100		00	LD1 (single structure) — 32-bit, immediate offset	FEAT_AdvSIMD
1	0	11111	100	0	01	LD1 (single structure) — 64-bit, immediate offset	FEAT_AdvSIMD
1	0	11111	101		00	LD3 (single structure) — 32-bit, immediate offset	FEAT_AdvSIMD
1	0	11111	101	0	01	LD3 (single structure) — 64-bit, immediate offset	FEAT_AdvSIMD

Decode fields						Instruction Details	Condition
L	R	Rm	opcode	S	size		
1	0	11111	110	0		LD1R — Immediate offset	FEAT_AdvSIMD
1	0	11111	111	0		LD3R — Immediate offset	FEAT_AdvSIMD
1	0	!= 11111	000			LD1 (single structure) — 8-bit, register offset	FEAT_AdvSIMD
1	0	!= 11111	001			LD3 (single structure) — 8-bit, register offset	FEAT_AdvSIMD
1	0	!= 11111	010		x0	LD1 (single structure) — 16-bit, register offset	FEAT_AdvSIMD
1	0	!= 11111	011		x0	LD3 (single structure) — 16-bit, register offset	FEAT_AdvSIMD
1	0	!= 11111	100		00	LD1 (single structure) — 32-bit, register offset	FEAT_AdvSIMD
1	0	!= 11111	100	0	01	LD1 (single structure) — 64-bit, register offset	FEAT_AdvSIMD
1	0	!= 11111	101		00	LD3 (single structure) — 32-bit, register offset	FEAT_AdvSIMD
1	0	!= 11111	101	0	01	LD3 (single structure) — 64-bit, register offset	FEAT_AdvSIMD
1	0	!= 11111	110	0		LD1R — Register offset	FEAT_AdvSIMD
1	0	!= 11111	111	0		LD3R — Register offset	FEAT_AdvSIMD
1	1	11111	000			LD2 (single structure) — 8-bit, immediate offset	FEAT_AdvSIMD
1	1	11111	001			LD4 (single structure) — 8-bit, immediate offset	FEAT_AdvSIMD
1	1	11111	010		x0	LD2 (single structure) — 16-bit, immediate offset	FEAT_AdvSIMD
1	1	11111	011		x0	LD4 (single structure) — 16-bit, immediate offset	FEAT_AdvSIMD
1	1	11111	100		00	LD2 (single structure) — 32-bit, immediate offset	FEAT_AdvSIMD
1	1	11111	100	0	01	LD2 (single structure) — 64-bit, immediate offset	FEAT_AdvSIMD
1	1	11111	101		00	LD4 (single structure) — 32-bit, immediate offset	FEAT_AdvSIMD
1	1	11111	101	0	01	LD4 (single structure) — 64-bit, immediate offset	FEAT_AdvSIMD
1	1	11111	110	0		LD2R — Immediate offset	FEAT_AdvSIMD
1	1	11111	111	0		LD4R — Immediate offset	FEAT_AdvSIMD
1	1	!= 11111	000			LD2 (single structure) — 8-bit, register offset	FEAT_AdvSIMD
1	1	!= 11111	001			LD4 (single structure) — 8-bit, register offset	FEAT_AdvSIMD
1	1	!= 11111	010		x0	LD2 (single structure) — 16-bit, register offset	FEAT_AdvSIMD
1	1	!= 11111	011		x0	LD4 (single structure) — 16-bit, register offset	FEAT_AdvSIMD
1	1	!= 11111	100		00	LD2 (single structure) — 32-bit, register offset	FEAT_AdvSIMD
1	1	!= 11111	100	0	01	LD2 (single structure) — 64-bit, register offset	FEAT_AdvSIMD
1	1	!= 11111	101		00	LD4 (single structure) — 32-bit, register offset	FEAT_AdvSIMD
1	1	!= 11111	101	0	01	LD4 (single structure) — 64-bit, register offset	FEAT_AdvSIMD
1	1	!= 11111	110	0		LD2R — Register offset	FEAT_AdvSIMD
1	1	!= 11111	111	0		LD4R — Register offset	FEAT_AdvSIMD

RCW compare and swap

The encodings in this section are decoded from Loads and Stores.



Decode fields S	A	R	Instruction Details	Condition
0	0	0	RCWCAS, RCWCASA, RCWCASAL, RCWCASL — RCWCAS	FEAT_THE
0	0	1	RCWCAS, RCWCASA, RCWCASAL, RCWCASL — RCWCASL	FEAT_THE
0	1	0	RCWCAS, RCWCASA, RCWCASAL, RCWCASL — RCWCASA	FEAT_THE
0	1	1	RCWCAS, RCWCASA, RCWCASAL, RCWCASL — RCWCASAL	FEAT_THE
1	0	0	RCWSCAS, RCWSCASA, RCWSCASAL, RCWSCASL — RCWSCAS	FEAT_THE
1	0	1	RCWSCAS, RCWSCASA, RCWSCASAL, RCWSCASL — RCWSCASL	FEAT_THE
1	1	0	RCWSCAS, RCWSCASA, RCWSCASAL, RCWSCASL — RCWSCASA	FEAT_THE
1	1	1	RCWSCAS, RCWSCASA, RCWSCASAL, RCWSCASL — RCWSCASAL	FEAT_THE

RCW compare and swap pair

The encodings in this section are decoded from Loads and Stores.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	S	0	1	1	0	0	1	A	R	1	Rs					0	0	0	0	1	1	Rn					Rt				

Decode fields S	A	R	Instruction Details	Condition
0	0	0	RCWCASP, RCWCASPA, RCWCASPAL, RCWCASPL — RCWCASP	FEAT_D128 && FEAT_THE
0	0	1	RCWCASP, RCWCASPA, RCWCASPAL, RCWCASPL — RCWCASPL	FEAT_D128 && FEAT_THE
0	1	0	RCWCASP, RCWCASPA, RCWCASPAL, RCWCASPL — RCWCASPA	FEAT_D128 && FEAT_THE
0	1	1	RCWCASP, RCWCASPA, RCWCASPAL, RCWCASPL — RCWCASPAL	FEAT_D128 && FEAT_THE
1	0	0	RCWSCASP, RCWSCASPA, RCWSCASPAL, RCWSCASPL — RCWSCASP	FEAT_D128 && FEAT_THE
1	0	1	RCWSCASP, RCWSCASPA, RCWSCASPAL, RCWSCASPL — RCWSCASPL	FEAT_D128 && FEAT_THE
1	1	0	RCWSCASP, RCWSCASPA, RCWSCASPAL, RCWSCASPL — RCWSCASPA	FEAT_D128 && FEAT_THE
1	1	1	RCWSCASP, RCWSCASPA, RCWSCASPAL, RCWSCASPL — RCWSCASPAL	FEAT_D128 && FEAT_THE

128-bit atomic memory operations

The encodings in this section are decoded from Loads and Stores.

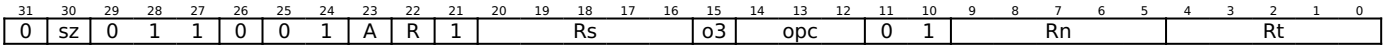
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	S	0	1	1	0	0	1	A	R	1	Rt2					o3	opc			0	0	Rn					Rt				

Decode fields					Instruction Details	Condition
S	A	R	o3	opc		
				1xx	UNALLOCATED	-
0			0	0x0	UNALLOCATED	-
0	0	0	0	001	LDCLRP, LDCLRPA, LDCLRPAL, LDCLRPL — LDCLRP	FEAT_LSE128
0	0	0	0	011	LDSETP, LDSETPA, LDSETPAL, LDSETPL — LDSETP	FEAT_LSE128
0	0	0	1	000	SWPP, SWPPA, SWPPAL, SWPPL — SWPP	FEAT_LSE128
0	0	0	1	001	RCWCLRP, RCWCLRPA, RCWCLRPAL, RCWCLRPL — RCWCLRP	FEAT_D128 && FEAT_THE
0	0	0	1	010	RCWSWPP, RCWSWPPA, RCWSWPPAL, RCWSWPPL — RCWSWPP	FEAT_D128 && FEAT_THE
0	0	0	1	011	RCWSETP, RCWSETPA, RCWSETPAL, RCWSETPL — RCWSETP	FEAT_D128 && FEAT_THE
0	0	1	0	001	LDCLRP, LDCLRPA, LDCLRPAL, LDCLRPL — LDCLRPL	FEAT_LSE128
0	0	1	0	011	LDSETP, LDSETPA, LDSETPAL, LDSETPL — LDSETPL	FEAT_LSE128
0	0	1	1	000	SWPP, SWPPA, SWPPAL, SWPPL — SWPPL	FEAT_LSE128
0	0	1	1	001	RCWCLRP, RCWCLRPA, RCWCLRPAL, RCWCLRPL — RCWCLRPL	FEAT_D128 && FEAT_THE
0	0	1	1	010	RCWSWPP, RCWSWPPA, RCWSWPPAL, RCWSWPPL — RCWSWPPL	FEAT_D128 && FEAT_THE
0	0	1	1	011	RCWSETP, RCWSETPA, RCWSETPAL, RCWSETPL — RCWSETPL	FEAT_D128 && FEAT_THE
0	1	0	0	001	LDCLRP, LDCLRPA, LDCLRPAL, LDCLRPL — LDCLRPA	FEAT_LSE128
0	1	0	0	011	LDSETP, LDSETPA, LDSETPAL, LDSETPL — LDSETPA	FEAT_LSE128
0	1	0	1	000	SWPP, SWPPA, SWPPAL, SWPPL — SWPPA	FEAT_LSE128
0	1	0	1	001	RCWCLRP, RCWCLRPA, RCWCLRPAL, RCWCLRPL — RCWCLRPA	FEAT_D128 && FEAT_THE
0	1	0	1	010	RCWSWPP, RCWSWPPA, RCWSWPPAL, RCWSWPPL — RCWSWPPA	FEAT_D128 && FEAT_THE
0	1	0	1	011	RCWSETP, RCWSETPA, RCWSETPAL, RCWSETPL — RCWSETPA	FEAT_D128 && FEAT_THE
0	1	1	0	001	LDCLRP, LDCLRPA, LDCLRPAL, LDCLRPL — LDCLRPAL	FEAT_LSE128
0	1	1	0	011	LDSETP, LDSETPA, LDSETPAL, LDSETPL — LDSETPAL	FEAT_LSE128
0	1	1	1	000	SWPP, SWPPA, SWPPAL, SWPPL — SWPPAL	FEAT_LSE128
0	1	1	1	001	RCWCLRP, RCWCLRPA, RCWCLRPAL, RCWCLRPL — RCWCLRPAL	FEAT_D128 && FEAT_THE
0	1	1	1	010	RCWSWPP, RCWSWPPA, RCWSWPPAL, RCWSWPPL — RCWSWPPAL	FEAT_D128 && FEAT_THE
0	1	1	1	011	RCWSETP, RCWSETPA, RCWSETPAL, RCWSETPL — RCWSETPAL	FEAT_D128 && FEAT_THE
1			0	0xx	UNALLOCATED	-
1			1	000	UNALLOCATED	-
1	0	0	1	001	RCWSCLRP, RCWSCLRPA, RCWSCLRPAL, RCWSCLRPL — RCWSCLRP	FEAT_D128 && FEAT_THE
1	0	0	1	010	RCWSSWPP, RCWSSWPPA, RCWSSWPPAL, RCWSSWPPL — RCWSSWPP	FEAT_D128 && FEAT_THE
1	0	0	1	011	RCWSSETP, RCWSSETPA, RCWSSETPAL, RCWSSETPL — RCWSSETP	FEAT_D128 && FEAT_THE
1	0	1	1	001	RCWSCLRP, RCWSCLRPA, RCWSCLRPAL, RCWSCLRPL — RCWSCLRPL	FEAT_D128 && FEAT_THE
1	0	1	1	010	RCWSSWPP, RCWSSWPPA, RCWSSWPPAL, RCWSSWPPL — RCWSSWPPL	FEAT_D128 && FEAT_THE
1	0	1	1	011	RCWSSETP, RCWSSETPA, RCWSSETPAL, RCWSSETPL — RCWSSETPL	FEAT_D128 && FEAT_THE
1	1	0	1	001	RCWSCLRP, RCWSCLRPA, RCWSCLRPAL, RCWSCLRPL — RCWSCLRPA	FEAT_D128 && FEAT_THE

Decode fields					Instruction Details	Condition
S	A	R	o3	opc		
1	1	0	1	010	RCWSSWPP, RCWSSWPPA, RCWSSWPPAL, RCWSSWPPL — RCWSSWPPA	FEAT_D128 && FEAT_THE
1	1	0	1	011	RCWSSETP, RCWSSETPA, RCWSSETPAL, RCWSSETPL — RCWSSETPA	FEAT_D128 && FEAT_THE
1	1	1	1	001	RCWSCLRP, RCWSCLRPA, RCWSCLRPAL, RCWSCLRPL — RCWSCLRPAL	FEAT_D128 && FEAT_THE
1	1	1	1	010	RCWSSWPP, RCWSSWPPA, RCWSSWPPAL, RCWSSWPPL — RCWSSWPPAL	FEAT_D128 && FEAT_THE
1	1	1	1	011	RCWSSETP, RCWSSETPA, RCWSSETPAL, RCWSSETPL — RCWSSETPAL	FEAT_D128 && FEAT_THE

Atomic memory operations (unprivileged)

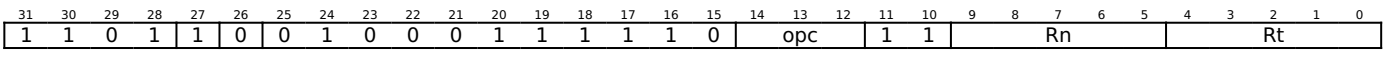
The encodings in this section are decoded from Loads and Stores.



Decode fields					Instruction Details	Condition
sz	A	R	o3	opc		
				010	UNALLOCATED	-
				1xx	UNALLOCATED	-
			1	0x1	UNALLOCATED	-
0	0	0	0	000	LDTADD, LDTADDA, LDTADDAL, LDTADDL — 32-bit no memory ordering	FEAT_LSUI
0	0	0	0	001	LDTCLR, LDTCLRA, LDTCLRAL, LDTCLRL — 32-bit no memory ordering	FEAT_LSUI
0	0	0	0	011	LDTSET, LDTSETA, LDTSETAL, LDTSETL — 32-bit no memory ordering	FEAT_LSUI
0	0	0	1	000	SWPT, SWPTA, SWPTAL, SWPTL — 32-bit SWPT	FEAT_LSUI
0	0	1	0	000	LDTADD, LDTADDA, LDTADDAL, LDTADDL — 32-bit release	FEAT_LSUI
0	0	1	0	001	LDTCLR, LDTCLRA, LDTCLRAL, LDTCLRL — 32-bit release	FEAT_LSUI
0	0	1	0	011	LDTSET, LDTSETA, LDTSETAL, LDTSETL — 32-bit release	FEAT_LSUI
0	0	1	1	000	SWPT, SWPTA, SWPTAL, SWPTL — 32-bit SWPTL	FEAT_LSUI
0	1	0	0	000	LDTADD, LDTADDA, LDTADDAL, LDTADDL — 32-bit acquire	FEAT_LSUI
0	1	0	0	001	LDTCLR, LDTCLRA, LDTCLRAL, LDTCLRL — 32-bit acquire	FEAT_LSUI
0	1	0	0	011	LDTSET, LDTSETA, LDTSETAL, LDTSETL — 32-bit acquire	FEAT_LSUI
0	1	0	1	000	SWPT, SWPTA, SWPTAL, SWPTL — 32-bit SWPTA	FEAT_LSUI
0	1	1	0	000	LDTADD, LDTADDA, LDTADDAL, LDTADDL — 32-bit acquire-release	FEAT_LSUI
0	1	1	0	001	LDTCLR, LDTCLRA, LDTCLRAL, LDTCLRL — 32-bit acquire-release	FEAT_LSUI
0	1	1	0	011	LDTSET, LDTSETA, LDTSETAL, LDTSETL — 32-bit acquire-release	FEAT_LSUI
0	1	1	1	000	SWPT, SWPTA, SWPTAL, SWPTL — 32-bit SWPTAL	FEAT_LSUI
1	0	0	0	000	LDTADD, LDTADDA, LDTADDAL, LDTADDL — 64-bit no memory ordering	FEAT_LSUI
1	0	0	0	001	LDTCLR, LDTCLRA, LDTCLRAL, LDTCLRL — 64-bit no memory ordering	FEAT_LSUI
1	0	0	0	011	LDTSET, LDTSETA, LDTSETAL, LDTSETL — 64-bit no memory ordering	FEAT_LSUI
1	0	0	1	000	SWPT, SWPTA, SWPTAL, SWPTL — 64-bit SWPT	FEAT_LSUI
1	0	1	0	000	LDTADD, LDTADDA, LDTADDAL, LDTADDL — 64-bit release	FEAT_LSUI
1	0	1	0	001	LDTCLR, LDTCLRA, LDTCLRAL, LDTCLRL — 64-bit release	FEAT_LSUI
1	0	1	0	011	LDTSET, LDTSETA, LDTSETAL, LDTSETL — 64-bit release	FEAT_LSUI
1	0	1	1	000	SWPT, SWPTA, SWPTAL, SWPTL — 64-bit SWPTL	FEAT_LSUI
1	1	0	0	000	LDTADD, LDTADDA, LDTADDAL, LDTADDL — 64-bit acquire	FEAT_LSUI
1	1	0	0	001	LDTCLR, LDTCLRA, LDTCLRAL, LDTCLRL — 64-bit acquire	FEAT_LSUI
1	1	0	0	011	LDTSET, LDTSETA, LDTSETAL, LDTSETL — 64-bit acquire	FEAT_LSUI
1	1	0	1	000	SWPT, SWPTA, SWPTAL, SWPTL — 64-bit SWPTA	FEAT_LSUI
1	1	1	0	000	LDTADD, LDTADDA, LDTADDAL, LDTADDL — 64-bit acquire-release	FEAT_LSUI
1	1	1	0	001	LDTCLR, LDTCLRA, LDTCLRAL, LDTCLRL — 64-bit acquire-release	FEAT_LSUI
1	1	1	0	011	LDTSET, LDTSETA, LDTSETAL, LDTSETL — 64-bit acquire-release	FEAT_LSUI
1	1	1	1	000	SWPT, SWPTA, SWPTAL, SWPTL — 64-bit SWPTAL	FEAT_LSUI

GCS load/store

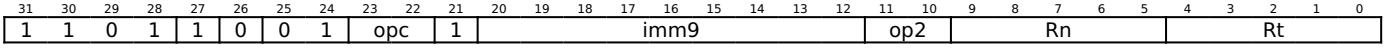
The encodings in this section are decoded from Loads and Stores.



Decode fields	Instruction Details	Condition
opc		
000	GCSSTR	FEAT_GCS
001	GCSSTTR	FEAT_GCS
!= 00x	UNALLOCATED	-

Load/store memory tags

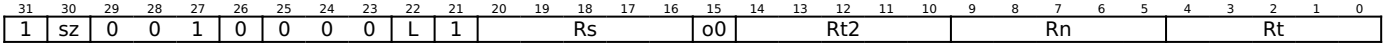
The encodings in this section are decoded from Loads and Stores.



Decode fields			Instruction Details	Condition
opc	imm9	op2		
00		01	STG — Post-index	FEAT_MTE
00		10	STG — Signed offset	FEAT_MTE
00		11	STG — Pre-index	FEAT_MTE
00	000000000	00	STZGM	FEAT_MTE2
01		00	LDG	FEAT_MTE
01		01	STZG — Post-index	FEAT_MTE
01		10	STZG — Signed offset	FEAT_MTE
01		11	STZG — Pre-index	FEAT_MTE
10		01	ST2G — Post-index	FEAT_MTE
10		10	ST2G — Signed offset	FEAT_MTE
10		11	ST2G — Pre-index	FEAT_MTE
10	000000000	00	STGM	FEAT_MTE2
11		01	STZ2G — Post-index	FEAT_MTE
11		10	STZ2G — Signed offset	FEAT_MTE
11		11	STZ2G — Pre-index	FEAT_MTE
11	000000000	00	LDGM	FEAT_MTE2
!= 01	!= 000000000	00	UNALLOCATED	-

Load/store exclusive pair

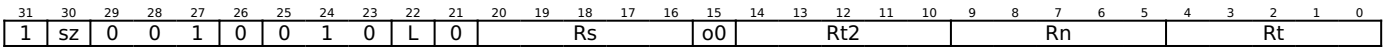
The encodings in this section are decoded from Loads and Stores.



Decode fields			Instruction Details
sz	L	o0	
0	0	0	STXP — 32-bit
0	0	1	STLXP — 32-bit
0	1	0	LDXP — 32-bit
0	1	1	LDAXP — 32-bit
1	0	0	STXP — 64-bit
1	0	1	STLXP — 64-bit
1	1	0	LDXP — 64-bit
1	1	1	LDAXP — 64-bit

Load/store exclusive register (unprivileged)

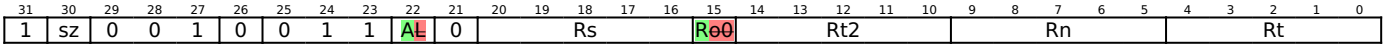
The encodings in this section are decoded from Loads and Stores.



Decode fields			Instruction Details	Condition
sz	L	o0		
0	0	0	STTXR — 32-bit	FEAT_LSUI
0	0	1	STLTXR — 32-bit	FEAT_LSUI
0	1	0	LDTXR — 32-bit	FEAT_LSUI
0	1	1	LDATXR — 32-bit	FEAT_LSUI
1	0	0	STTXR — 64-bit	FEAT_LSUI
1	0	1	STLTXR — 64-bit	FEAT_LSUI
1	1	0	LDTXR — 64-bit	FEAT_LSUI
1	1	1	LDATXR — 64-bit	FEAT_LSUI

Compare and swap (unprivileged)

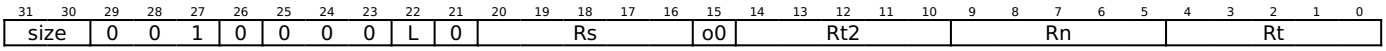
The encodings in this section are decoded from Loads and Stores.



Decode fields				Instruction Details	Condition
sz	A~~L~~	R~~o~~0	Rt2		
0				UNALLOCATED	-
1			!= 11111	UNALLOCATED	-
1	0	0	11111	CAST, CASAT, CASALT, CASLT — CAST	FEAT_LSUI
1	0	1	11111	CAST, CASAT, CASALT, CASLT — CASLT	FEAT_LSUI
1	1	0	11111	CAST, CASAT, CASALT, CASLT — CASAT	FEAT_LSUI
1	1	1	11111	CAST, CASAT, CASALT, CASLT — CASALT	FEAT_LSUI

Load/store exclusive register

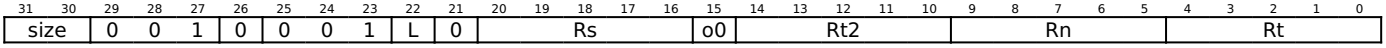
The encodings in this section are decoded from Loads and Stores.



Decode fields			Instruction Details
size	L	o0	
00	0	0	STXRB
00	0	1	STLXRB
00	1	0	LDXRB
00	1	1	LDAXRB
01	0	0	STXRH
01	0	1	STLXRH
01	1	0	LDXRH
01	1	1	LDAXRH
10	0	0	STXR — 32-bit
10	0	1	STLXR — 32-bit
10	1	0	LDXR — 32-bit
10	1	1	LDAXR — 32-bit
11	0	0	STXR — 64-bit
11	0	1	STLXR — 64-bit
11	1	0	LDXR — 64-bit
11	1	1	LDAXR — 64-bit

Load/store ordered

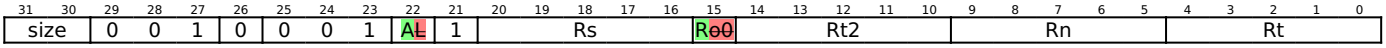
The encodings in this section are decoded from Loads and Stores.



Decode fields			Instruction Details	Condition
size	L	o0		
00	0	0	STLLRB	FEAT_LOR
00	0	1	STLRB	-
00	1	0	LDLARB	FEAT_LOR
00	1	1	LDARB	-
01	0	0	STLLRH	FEAT_LOR
01	0	1	STLRH	-
01	1	0	LDLARH	FEAT_LOR
01	1	1	LDARH	-
10	0	0	STLLR — 32-bit	FEAT_LOR
10	0	1	STLR — 32-bit	-
10	1	0	LDLAR — 32-bit	FEAT_LOR
10	1	1	LDAR — 32-bit	-
11	0	0	STLLR — 64-bit	FEAT_LOR
11	0	1	STLR — 64-bit	-
11	1	0	LDLAR — 64-bit	FEAT_LOR
11	1	1	LDAR — 64-bit	-

Compare and swap

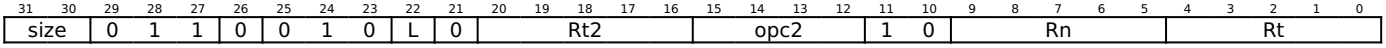
The encodings in this section are decoded from Loads and Stores.



Decode fields				Instruction Details	Condition
size	AL	Rs0	Rt2		
			!= 11111	UNALLOCATED	-
00	0	0	11111	CASB, CASAB, CASALB, CASLB — CASB	FEAT_LSE
00	0	1	11111	CASB, CASAB, CASALB, CASLB — CASLB	FEAT_LSE
00	1	0	11111	CASB, CASAB, CASALB, CASLB — CASAB	FEAT_LSE
00	1	1	11111	CASB, CASAB, CASALB, CASLB — CASALB	FEAT_LSE
01	0	0	11111	CASH, CASAH, CASALH, CASLH — CASH	FEAT_LSE
01	0	1	11111	CASH, CASAH, CASALH, CASLH — CASLH	FEAT_LSE
01	1	0	11111	CASH, CASAH, CASALH, CASLH — CASAH	FEAT_LSE
01	1	1	11111	CASH, CASAH, CASALH, CASLH — CASALH	FEAT_LSE
10	0	0	11111	CAS, CASA, CASAL, CASL — 32-bit CAS	FEAT_LSE
10	0	1	11111	CAS, CASA, CASAL, CASL — 32-bit CASL	FEAT_LSE
10	1	0	11111	CAS, CASA, CASAL, CASL — 32-bit CASA	FEAT_LSE
10	1	1	11111	CAS, CASA, CASAL, CASL — 32-bit CASAL	FEAT_LSE
11	0	0	11111	CAS, CASA, CASAL, CASL — 64-bit CAS	FEAT_LSE
11	0	1	11111	CAS, CASA, CASAL, CASL — 64-bit CASL	FEAT_LSE
11	1	0	11111	CAS, CASA, CASAL, CASL — 64-bit CASA	FEAT_LSE
11	1	1	11111	CAS, CASA, CASAL, CASL — 64-bit CASAL	FEAT_LSE

Load/store ordered register pair

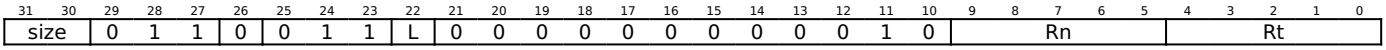
The encodings in this section are decoded from Loads and Stores.



Decode fields			Instruction Details	Condition
size	L	opc2		
0x			UNALLOCATED	-
1x		1xxx	UNALLOCATED	-
1x	0	001x	UNALLOCATED	-
1x	1	001x	UNALLOCATED	-
10		01xx	UNALLOCATED	-
10	0	0000	STILP — 32-bit pre-index	FEAT_LRCPC3
10	0	0001	STILP — 32-bit	FEAT_LRCPC3
10	1	0000	LDIAPP — 32-bit post-index	FEAT_LRCPC3
10	1	0001	LDIAPP — 32-bit	FEAT_LRCPC3
11		0100	UNALLOCATED	-
11	0	0000	STILP — 64-bit pre-index	FEAT_LRCPC3
11	0	0001	STILP — 64-bit	FEAT_LRCPC3
11	0	0101	STLP	FEAT_LSCP
11	0	011x	UNALLOCATED	-
11	1	0000	LDIAPP — 64-bit post-index	FEAT_LRCPC3
11	1	0001	LDIAPP — 64-bit	FEAT_LRCPC3
11	1	0101	LDAP	FEAT_LSCP
11	1	0110	UNALLOCATED	-
11	1	0111	LDAPP	FEAT_LSCP

Load/store ordered (writeback)

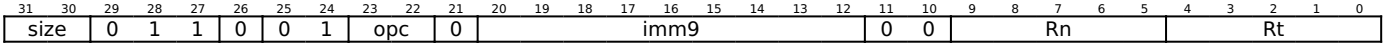
The encodings in this section are decoded from Loads and Stores.



Decode fields		Instruction Details	Condition
size	L		
0x		UNALLOCATED	-
10	0	STLR — 32-bit	FEAT_LRCPC3
10	1	LDAPR — 32-bit	FEAT_LRCPC3
11	0	STLR — 64-bit	FEAT_LRCPC3
11	1	LDAPR — 64-bit	FEAT_LRCPC3

Load/store ordered (unscaled immediate)

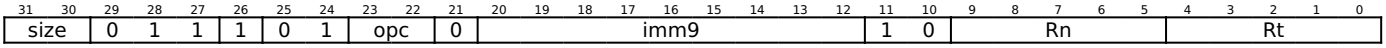
The encodings in this section are decoded from Loads and Stores.



Decode fields		Instruction Details	Condition
size	opc		
00	00	STLURB	FEAT_LRCPC2
00	01	LDAPURB	FEAT_LRCPC2
00	10	LDAPURSB — 64-bit	FEAT_LRCPC2
00	11	LDAPURSB — 32-bit	FEAT_LRCPC2
01	00	STLURH	FEAT_LRCPC2
01	01	LDAPURH	FEAT_LRCPC2
01	10	LDAPURSH — 64-bit	FEAT_LRCPC2
01	11	LDAPURSH — 32-bit	FEAT_LRCPC2
10	00	STLUR — 32-bit	FEAT_LRCPC2
10	01	LDAPUR — 32-bit	FEAT_LRCPC2
10	10	LDAPURSW	FEAT_LRCPC2
10	11	UNALLOCATED	-
11	00	STLUR — 64-bit	FEAT_LRCPC2
11	01	LDAPUR — 64-bit	FEAT_LRCPC2
11	1x	UNALLOCATED	-

Load/store ordered (SIMD&FP)

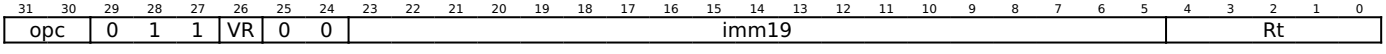
The encodings in this section are decoded from Loads and Stores.



Decode fields		Instruction Details	Condition
size	opc		
00	00	STLUR (SIMD&FP) — 8-bit	FEAT_FP && FEAT_LRCPC3
00	01	LDAPUR (SIMD&FP) — 8-bit	FEAT_FP && FEAT_LRCPC3
00	10	STLUR (SIMD&FP) — 128-bit	FEAT_FP && FEAT_LRCPC3
00	11	LDAPUR (SIMD&FP) — 128-bit	FEAT_FP && FEAT_LRCPC3
01	00	STLUR (SIMD&FP) — 16-bit	FEAT_FP && FEAT_LRCPC3
01	01	LDAPUR (SIMD&FP) — 16-bit	FEAT_FP && FEAT_LRCPC3
10	00	STLUR (SIMD&FP) — 32-bit	FEAT_FP && FEAT_LRCPC3
10	01	LDAPUR (SIMD&FP) — 32-bit	FEAT_FP && FEAT_LRCPC3
11	00	STLUR (SIMD&FP) — 64-bit	FEAT_FP && FEAT_LRCPC3
11	01	LDAPUR (SIMD&FP) — 64-bit	FEAT_FP && FEAT_LRCPC3
!= 00	1x	UNALLOCATED	-

Load register (literal)

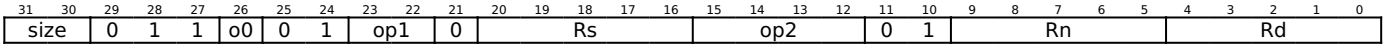
The encodings in this section are decoded from Loads and Stores.



Decode fields		Instruction Details	Condition
opc	VR		
00	0	LDR (literal) — 32-bit	-
00	1	LDR (literal, SIMD&FP) — 32-bit	FEAT_FP
01	0	LDR (literal) — 64-bit	-
01	1	LDR (literal, SIMD&FP) — 64-bit	FEAT_FP
10	0	LDRSW (literal)	-
10	1	LDR (literal, SIMD&FP) — 128-bit	FEAT_FP
11	0	PRFM (literal)	-
11	1	UNALLOCATED	-

Memory Copy and Memory Set

The encodings in this section are decoded from Loads and Stores.



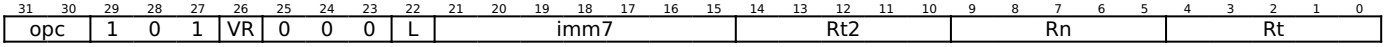
Decode fields			Instruction Details	Condition
o0	op1	op2		
	11	11xx	UNALLOCATED	-
0	00	0000	CPYFP, CPYFM, CPYFE — Prologue	FEAT_MOPS
0	00	0001	CPYFPWT, CPYFMWT, CPYFEWT — Prologue	FEAT_MOPS
0	00	0010	CPYFPRT, CPYFMRT, CPYFERT — Prologue	FEAT_MOPS
0	00	0011	CPYFPT, CPYFMT, CPYFET — Prologue	FEAT_MOPS
0	00	0100	CPYFPWN, CPYFMWN, CPYFEWN — Prologue	FEAT_MOPS
0	00	0101	CPYFPWTWN, CPYFMWTWN, CPYFEWTWN — Prologue	FEAT_MOPS
0	00	0110	CPYFPRTWN, CPYFMRTWN, CPYFERTWN — Prologue	FEAT_MOPS
0	00	0111	CPYFPTWN, CPYFMTWN, CPYFETWN — Prologue	FEAT_MOPS
0	00	1000	CPYFPRN, CPYFMRN, CPYFERN — Prologue	FEAT_MOPS
0	00	1001	CPYFPWTRN, CPYFMWTRN, CPYFEWTRN — Prologue	FEAT_MOPS
0	00	1010	CPYFPRTRN, CPYFMRTRN, CPYFERTRN — Prologue	FEAT_MOPS
0	00	1011	CPYFPTRN, CPYFMTRN, CPYFETRN — Prologue	FEAT_MOPS
0	00	1100	CPYFPN, CPYFMN, CPYFEN — Prologue	FEAT_MOPS
0	00	1101	CPYFPWTN, CPYFMWTN, CPYFEWTN — Prologue	FEAT_MOPS
0	00	1110	CPYFPRTN, CPYFMRTN, CPYFERTN — Prologue	FEAT_MOPS
0	00	1111	CPYFPTN, CPYFMTN, CPYFETN — Prologue	FEAT_MOPS
0	01	0000	CPYFP, CPYFM, CPYFE — Main	FEAT_MOPS
0	01	0001	CPYFPWT, CPYFMWT, CPYFEWT — Main	FEAT_MOPS
0	01	0010	CPYFPRT, CPYFMRT, CPYFERT — Main	FEAT_MOPS
0	01	0011	CPYFPT, CPYFMT, CPYFET — Main	FEAT_MOPS
0	01	0100	CPYFPWN, CPYFMWN, CPYFEWN — Main	FEAT_MOPS
0	01	0101	CPYFPWTWN, CPYFMWTWN, CPYFEWTWN — Main	FEAT_MOPS
0	01	0110	CPYFPRTWN, CPYFMRTWN, CPYFERTWN — Main	FEAT_MOPS
0	01	0111	CPYFPTWN, CPYFMTWN, CPYFETWN — Main	FEAT_MOPS
0	01	1000	CPYFPRN, CPYFMRN, CPYFERN — Main	FEAT_MOPS
0	01	1001	CPYFPWTRN, CPYFMWTRN, CPYFEWTRN — Main	FEAT_MOPS
0	01	1010	CPYFPRTRN, CPYFMRTRN, CPYFERTRN — Main	FEAT_MOPS
0	01	1011	CPYFPTRN, CPYFMTRN, CPYFETRN — Main	FEAT_MOPS
0	01	1100	CPYFPN, CPYFMN, CPYFEN — Main	FEAT_MOPS
0	01	1101	CPYFPWTN, CPYFMWTN, CPYFEWTN — Main	FEAT_MOPS
0	01	1110	CPYFPRTN, CPYFMRTN, CPYFERTN — Main	FEAT_MOPS
0	01	1111	CPYFPTN, CPYFMTN, CPYFETN — Main	FEAT_MOPS
0	10	0000	CPYFP, CPYFM, CPYFE — Epilogue	FEAT_MOPS
0	10	0001	CPYFPWT, CPYFMWT, CPYFEWT — Epilogue	FEAT_MOPS
0	10	0010	CPYFPRT, CPYFMRT, CPYFERT — Epilogue	FEAT_MOPS
0	10	0011	CPYFPT, CPYFMT, CPYFET — Epilogue	FEAT_MOPS
0	10	0100	CPYFPWN, CPYFMWN, CPYFEWN — Epilogue	FEAT_MOPS
0	10	0101	CPYFPWTWN, CPYFMWTWN, CPYFEWTWN — Epilogue	FEAT_MOPS
0	10	0110	CPYFPRTWN, CPYFMRTWN, CPYFERTWN — Epilogue	FEAT_MOPS
0	10	0111	CPYFPTWN, CPYFMTWN, CPYFETWN — Epilogue	FEAT_MOPS
0	10	1000	CPYFPRN, CPYFMRN, CPYFERN — Epilogue	FEAT_MOPS
0	10	1001	CPYFPWTRN, CPYFMWTRN, CPYFEWTRN — Epilogue	FEAT_MOPS
0	10	1010	CPYFPRTRN, CPYFMRTRN, CPYFERTRN — Epilogue	FEAT_MOPS
0	10	1011	CPYFPTRN, CPYFMTRN, CPYFETRN — Epilogue	FEAT_MOPS

Decode fields			Instruction Details	Condition
o0	op1	op2		
0	10	1100	CPYFPN, CPYFMN, CPYFEN — Epilogue	FEAT_MOPS
0	10	1101	CPYFPWTN, CPYFMWTN, CPYFEWTN — Epilogue	FEAT_MOPS
0	10	1110	CPYFPRTN, CPYFMRTN, CPYFERTN — Epilogue	FEAT_MOPS
0	10	1111	CPYFPTN, CPYFMTN, CPYFETN — Epilogue	FEAT_MOPS
0	11	0000	SETP, SETM, SETE — Prologue	FEAT_MOPS
0	11	0001	SETPT, SETMT, SETET — Prologue	FEAT_MOPS
0	11	0010	SETPN, SETMN, SETEN — Prologue	FEAT_MOPS
0	11	0011	SETPTN, SETMTN, SETETN — Prologue	FEAT_MOPS
0	11	0100	SETP, SETM, SETE — Main	FEAT_MOPS
0	11	0101	SETPT, SETMT, SETET — Main	FEAT_MOPS
0	11	0110	SETPN, SETMN, SETEN — Main	FEAT_MOPS
0	11	0111	SETPTN, SETMTN, SETETN — Main	FEAT_MOPS
0	11	1000	SETP, SETM, SETE — Epilogue	FEAT_MOPS
0	11	1001	SETPT, SETMT, SETET — Epilogue	FEAT_MOPS
0	11	1010	SETPN, SETMN, SETEN — Epilogue	FEAT_MOPS
0	11	1011	SETPTN, SETMTN, SETETN — Epilogue	FEAT_MOPS
1	00	0000	CPYP, CPYM, CPYE — Prologue	FEAT_MOPS
1	00	0001	CPYPWT, CPYMWT, CPYEWT — Prologue	FEAT_MOPS
1	00	0010	CPYPRT, CPYMRT, CPYERT — Prologue	FEAT_MOPS
1	00	0011	CPYPT, CPYMT, CPYET — Prologue	FEAT_MOPS
1	00	0100	CPYPWN, CPYMWN, CPYEWN — Prologue	FEAT_MOPS
1	00	0101	CPYPWTWN, CPYMWTWN, CPYEWTWN — Prologue	FEAT_MOPS
1	00	0110	CPYPRTWN, CPYMRTWN, CPYERTWN — Prologue	FEAT_MOPS
1	00	0111	CPYPTWN, CPYMTWN, CPYETWN — Prologue	FEAT_MOPS
1	00	1000	CPYPRN, CPYMRN, CPYERN — Prologue	FEAT_MOPS
1	00	1001	CPYPWTRN, CPYMWTRN, CPYEWTRN — Prologue	FEAT_MOPS
1	00	1010	CPYPRTRN, CPYMRTRN, CPYERTRN — Prologue	FEAT_MOPS
1	00	1011	CPYPTRN, CPYMTRN, CPYETRN — Prologue	FEAT_MOPS
1	00	1100	CPYPN, CPYMN, CPYEN — Prologue	FEAT_MOPS
1	00	1101	CPYPWTN, CPYMWTN, CPYEWTN — Prologue	FEAT_MOPS
1	00	1110	CPYPRTN, CPYMRTN, CPYERTN — Prologue	FEAT_MOPS
1	00	1111	CPYPTN, CPYMTN, CPYETN — Prologue	FEAT_MOPS
1	01	0000	CPYP, CPYM, CPYE — Main	FEAT_MOPS
1	01	0001	CPYPWT, CPYMWT, CPYEWT — Main	FEAT_MOPS
1	01	0010	CPYPRT, CPYMRT, CPYERT — Main	FEAT_MOPS
1	01	0011	CPYPT, CPYMT, CPYET — Main	FEAT_MOPS
1	01	0100	CPYPWN, CPYMWN, CPYEWN — Main	FEAT_MOPS
1	01	0101	CPYPWTWN, CPYMWTWN, CPYEWTWN — Main	FEAT_MOPS
1	01	0110	CPYPRTWN, CPYMRTWN, CPYERTWN — Main	FEAT_MOPS
1	01	0111	CPYPTWN, CPYMTWN, CPYETWN — Main	FEAT_MOPS
1	01	1000	CPYPRN, CPYMRN, CPYERN — Main	FEAT_MOPS
1	01	1001	CPYPWTRN, CPYMWTRN, CPYEWTRN — Main	FEAT_MOPS
1	01	1010	CPYPRTRN, CPYMRTRN, CPYERTRN — Main	FEAT_MOPS
1	01	1011	CPYPTRN, CPYMTRN, CPYETRN — Main	FEAT_MOPS
1	01	1100	CPYPN, CPYMN, CPYEN — Main	FEAT_MOPS
1	01	1101	CPYPWTN, CPYMWTN, CPYEWTN — Main	FEAT_MOPS

Decode fields			Instruction Details	Condition
o0	op1	op2		
1	01	1110	CPYPRTN, CPYMRTN, CPYERTN — Main	FEAT_MOPS
1	01	1111	CPYPTN, CPYMTN, CPYETN — Main	FEAT_MOPS
1	10	0000	CPYP, CPYM, CPYE — Epilogue	FEAT_MOPS
1	10	0001	CPYPWT, CPYMWT, CPYEWT — Epilogue	FEAT_MOPS
1	10	0010	CPYPRT, CPYMRT, CPYERT — Epilogue	FEAT_MOPS
1	10	0011	CPYPT, CPYMT, CPYET — Epilogue	FEAT_MOPS
1	10	0100	CPYPWN, CPYMWN, CPYEWN — Epilogue	FEAT_MOPS
1	10	0101	CPYPWTWN, CPYMWTWN, CPYEWTWN — Epilogue	FEAT_MOPS
1	10	0110	CPYPRTWN, CPYMRTWN, CPYERTWN — Epilogue	FEAT_MOPS
1	10	0111	CPYPTWN, CPYMTWN, CPYETWN — Epilogue	FEAT_MOPS
1	10	1000	CPYPRN, CPYMRN, CPYERN — Epilogue	FEAT_MOPS
1	10	1001	CPYPWTRN, CPYMWTRN, CPYEWTRN — Epilogue	FEAT_MOPS
1	10	1010	CPYPRTRN, CPYMRTRN, CPYERTRN — Epilogue	FEAT_MOPS
1	10	1011	CPYPTRN, CPYMTRN, CPYETRN — Epilogue	FEAT_MOPS
1	10	1100	CPYPN, CPYMN, CPYEN — Epilogue	FEAT_MOPS
1	10	1101	CPYPWTN, CPYMWTN, CPYEWTN — Epilogue	FEAT_MOPS
1	10	1110	CPYPRTN, CPYMRTN, CPYERTN — Epilogue	FEAT_MOPS
1	10	1111	CPYPTN, CPYMTN, CPYETN — Epilogue	FEAT_MOPS
1	11	0000	SETGP, SETGM, SETGE — Prologue	FEAT_MOPS && FEAT_MTE
1	11	0001	SETGPT, SETGMT, SETGET — Prologue	FEAT_MOPS && FEAT_MTE
1	11	0010	SETGPN, SETGMN, SETGEN — Prologue	FEAT_MOPS && FEAT_MTE
1	11	0011	SETGPTN, SETGMTN, SETGETN — Prologue	FEAT_MOPS && FEAT_MTE
1	11	0100	SETGP, SETGM, SETGE — Main	FEAT_MOPS && FEAT_MTE
1	11	0101	SETGPT, SETGMT, SETGET — Main	FEAT_MOPS && FEAT_MTE
1	11	0110	SETGPN, SETGMN, SETGEN — Main	FEAT_MOPS && FEAT_MTE
1	11	0111	SETGPTN, SETGMTN, SETGETN — Main	FEAT_MOPS && FEAT_MTE
1	11	1000	SETGP, SETGM, SETGE — Epilogue	FEAT_MOPS && FEAT_MTE
1	11	1001	SETGPT, SETGMT, SETGET — Epilogue	FEAT_MOPS && FEAT_MTE
1	11	1010	SETGPN, SETGMN, SETGEN — Epilogue	FEAT_MOPS && FEAT_MTE
1	11	1011	SETGPTN, SETGMTN, SETGETN — Epilogue	FEAT_MOPS && FEAT_MTE

Load/store no-allocate pair (offset)

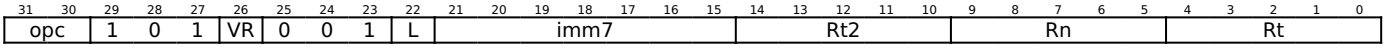
The encodings in this section are decoded from Loads and Stores.



Decode fields			Instruction Details	Condition
opc	VR	L		
00	0	0	STNP — 32-bit	-
00	0	1	LDNP — 32-bit	-
00	1	0	STNP (SIMD&FP) — 32-bit	FEAT_FP
00	1	1	LDNP (SIMD&FP) — 32-bit	FEAT_FP
01	0		UNALLOCATED	-
01	1	0	STNP (SIMD&FP) — 64-bit	FEAT_FP
01	1	1	LDNP (SIMD&FP) — 64-bit	FEAT_FP
10	0	0	STNP — 64-bit	-
10	0	1	LDNP — 64-bit	-
10	1	0	STNP (SIMD&FP) — 128-bit	FEAT_FP
10	1	1	LDNP (SIMD&FP) — 128-bit	FEAT_FP
11	0	0	STTNP	FEAT_LSUI
11	0	1	LDTNP	FEAT_LSUI
11	1	0	STTNP (SIMD&FP)	FEAT_FP && FEAT_LSUI
11	1	1	LDTNP (SIMD&FP)	FEAT_FP && FEAT_LSUI

Load/store register pair (post-indexed)

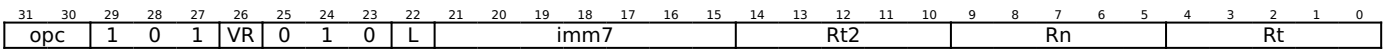
The encodings in this section are decoded from Loads and Stores.



Decode fields			Instruction Details	Condition
opc	VR	L		
00	0	0	STP — 32-bit	-
00	0	1	LDP — 32-bit	-
00	1	0	STP (SIMD&FP) — 32-bit	FEAT_FP
00	1	1	LDP (SIMD&FP) — 32-bit	FEAT_FP
01	0	0	STGP	FEAT_MTE
01	0	1	LDPSW	-
01	1	0	STP (SIMD&FP) — 64-bit	FEAT_FP
01	1	1	LDP (SIMD&FP) — 64-bit	FEAT_FP
10	0	0	STP — 64-bit	-
10	0	1	LDP — 64-bit	-
10	1	0	STP (SIMD&FP) — 128-bit	FEAT_FP
10	1	1	LDP (SIMD&FP) — 128-bit	FEAT_FP
11	0	0	STTP	FEAT_LSUI
11	0	1	LDTP	FEAT_LSUI
11	1	0	STTP (SIMD&FP)	FEAT_FP && FEAT_LSUI
11	1	1	LDTP (SIMD&FP)	FEAT_FP && FEAT_LSUI

Load/store register pair (offset)

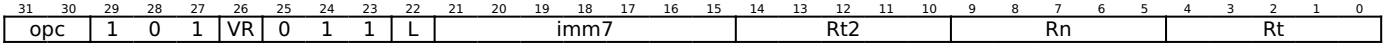
The encodings in this section are decoded from Loads and Stores.



Decode fields			Instruction Details	Condition
opc	VR	L		
00	0	0	STP — 32-bit	-
00	0	1	LDP — 32-bit	-
00	1	0	STP (SIMD&FP) — 32-bit	FEAT_FP
00	1	1	LDP (SIMD&FP) — 32-bit	FEAT_FP
01	0	0	STGP	FEAT_MTE
01	0	1	LDPSW	-
01	1	0	STP (SIMD&FP) — 64-bit	FEAT_FP
01	1	1	LDP (SIMD&FP) — 64-bit	FEAT_FP
10	0	0	STP — 64-bit	-
10	0	1	LDP — 64-bit	-
10	1	0	STP (SIMD&FP) — 128-bit	FEAT_FP
10	1	1	LDP (SIMD&FP) — 128-bit	FEAT_FP
11	0	0	STTP	FEAT_LSUI
11	0	1	LDTP	FEAT_LSUI
11	1	0	STTP (SIMD&FP)	FEAT_FP && FEAT_LSUI
11	1	1	LDTP (SIMD&FP)	FEAT_FP && FEAT_LSUI

Load/store register pair (pre-indexed)

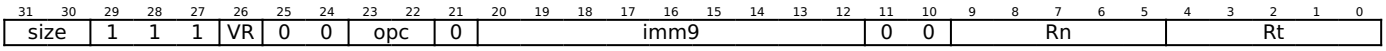
The encodings in this section are decoded from Loads and Stores.



Decode fields			Instruction Details	Condition
opc	VR	L		
00	0	0	STP — 32-bit	-
00	0	1	LDP — 32-bit	-
00	1	0	STP (SIMD&FP) — 32-bit	FEAT_FP
00	1	1	LDP (SIMD&FP) — 32-bit	FEAT_FP
01	0	0	STGP	FEAT_MTE
01	0	1	LDPSW	-
01	1	0	STP (SIMD&FP) — 64-bit	FEAT_FP
01	1	1	LDP (SIMD&FP) — 64-bit	FEAT_FP
10	0	0	STP — 64-bit	-
10	0	1	LDP — 64-bit	-
10	1	0	STP (SIMD&FP) — 128-bit	FEAT_FP
10	1	1	LDP (SIMD&FP) — 128-bit	FEAT_FP
11	0	0	STTP	FEAT_LSUI
11	0	1	LDTP	FEAT_LSUI
11	1	0	STTP (SIMD&FP)	FEAT_FP && FEAT_LSUI
11	1	1	LDTP (SIMD&FP)	FEAT_FP && FEAT_LSUI

Load/store register (unscaled immediate)

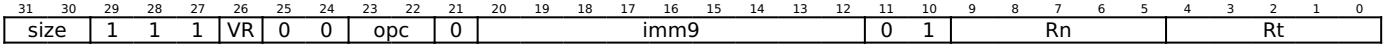
The encodings in this section are decoded from Loads and Stores.



Decode fields			Instruction Details	Condition
size	VR	opc		
00	0	00	STURB	-
00	0	01	LDURB	-
00	0	10	LDURSB — 64-bit	-
00	0	11	LDURSB — 32-bit	-
00	1	00	STUR (SIMD&FP) — 8-bit	FEAT_FP
00	1	01	LDUR (SIMD&FP) — 8-bit	FEAT_FP
00	1	10	STUR (SIMD&FP) — 128-bit	FEAT_FP
00	1	11	LDUR (SIMD&FP) — 128-bit	FEAT_FP
01	0	00	STURH	-
01	0	01	LDURH	-
01	0	10	LDURSH — 64-bit	-
01	0	11	LDURSH — 32-bit	-
01	1	00	STUR (SIMD&FP) — 16-bit	FEAT_FP
01	1	01	LDUR (SIMD&FP) — 16-bit	FEAT_FP
1x	0	11	UNALLOCATED	-
10	0	00	STUR — 32-bit	-
10	0	01	LDUR — 32-bit	-
10	0	10	LDURSW	-
10	1	00	STUR (SIMD&FP) — 32-bit	FEAT_FP
10	1	01	LDUR (SIMD&FP) — 32-bit	FEAT_FP
11	0	00	STUR — 64-bit	-
11	0	01	LDUR — 64-bit	-
11	0	10	PRFUM	-
11	1	00	STUR (SIMD&FP) — 64-bit	FEAT_FP
11	1	01	LDUR (SIMD&FP) — 64-bit	FEAT_FP
!= 00	1	1x	UNALLOCATED	-

Load/store register (immediate post-indexed)

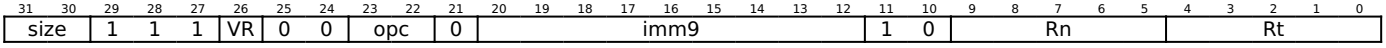
The encodings in this section are decoded from Loads and Stores.



Decode fields			Instruction Details	Condition
size	VR	opc		
00	0	00	STRB (immediate)	-
00	0	01	LDRB (immediate)	-
00	0	10	LDRSB (immediate) — 64-bit	-
00	0	11	LDRSB (immediate) — 32-bit	-
00	1	00	STR (immediate, SIMD&FP) — 8-bit	FEAT_FP
00	1	01	LDR (immediate, SIMD&FP) — 8-bit	FEAT_FP
00	1	10	STR (immediate, SIMD&FP) — 128-bit	FEAT_FP
00	1	11	LDR (immediate, SIMD&FP) — 128-bit	FEAT_FP
01	0	00	STRH (immediate)	-
01	0	01	LDRH (immediate)	-
01	0	10	LDRSH (immediate) — 64-bit	-
01	0	11	LDRSH (immediate) — 32-bit	-
01	1	00	STR (immediate, SIMD&FP) — 16-bit	FEAT_FP
01	1	01	LDR (immediate, SIMD&FP) — 16-bit	FEAT_FP
01	1	1x	UNALLOCATED	-
10	0	00	STR (immediate) — 32-bit	-
10	0	01	LDR (immediate) — 32-bit	-
10	0	10	LDRSW (immediate)	-
10	0	11	UNALLOCATED	-
10	1	00	STR (immediate, SIMD&FP) — 32-bit	FEAT_FP
10	1	01	LDR (immediate, SIMD&FP) — 32-bit	FEAT_FP
10	1	1x	UNALLOCATED	-
11		1x	UNALLOCATED	-
11	0	00	STR (immediate) — 64-bit	-
11	0	01	LDR (immediate) — 64-bit	-
11	1	00	STR (immediate, SIMD&FP) — 64-bit	FEAT_FP
11	1	01	LDR (immediate, SIMD&FP) — 64-bit	FEAT_FP

Load/store register (unprivileged)

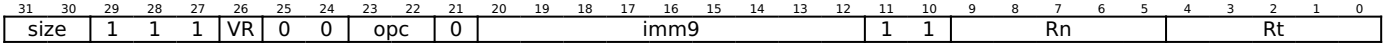
The encodings in this section are decoded from Loads and Stores.



Decode fields			Instruction Details
size	VR	opc	
	1		UNALLOCATED
00	0	00	STTRB
00	0	01	LDTRB
00	0	10	LDTRSB — 64-bit
00	0	11	LDTRSB — 32-bit
01	0	00	STTRH
01	0	01	LDTRH
01	0	10	LDTRSH — 64-bit
01	0	11	LDTRSH — 32-bit
10	0	00	STTR — 32-bit
10	0	01	LDTR — 32-bit
10	0	10	LDTRSW
10	0	11	UNALLOCATED
11	0	00	STTR — 64-bit
11	0	01	LDTR — 64-bit
11	0	1x	UNALLOCATED

Load/store register (immediate pre-indexed)

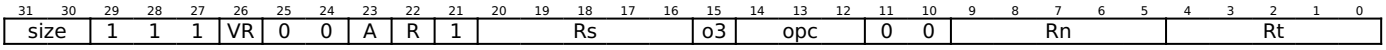
The encodings in this section are decoded from Loads and Stores.



Decode fields			Instruction Details	Condition
size	VR	opc		
00	0	00	STRB (immediate)	-
00	0	01	LDRB (immediate)	-
00	0	10	LDRSB (immediate) — 64-bit	-
00	0	11	LDRSB (immediate) — 32-bit	-
00	1	00	STR (immediate, SIMD&FP) — 8-bit	FEAT_FP
00	1	01	LDR (immediate, SIMD&FP) — 8-bit	FEAT_FP
00	1	10	STR (immediate, SIMD&FP) — 128-bit	FEAT_FP
00	1	11	LDR (immediate, SIMD&FP) — 128-bit	FEAT_FP
01	0	00	STRH (immediate)	-
01	0	01	LDRH (immediate)	-
01	0	10	LDRSH (immediate) — 64-bit	-
01	0	11	LDRSH (immediate) — 32-bit	-
01	1	00	STR (immediate, SIMD&FP) — 16-bit	FEAT_FP
01	1	01	LDR (immediate, SIMD&FP) — 16-bit	FEAT_FP
01	1	1x	UNALLOCATED	-
10	0	00	STR (immediate) — 32-bit	-
10	0	01	LDR (immediate) — 32-bit	-
10	0	10	LDRSW (immediate)	-
10	0	11	UNALLOCATED	-
10	1	00	STR (immediate, SIMD&FP) — 32-bit	FEAT_FP
10	1	01	LDR (immediate, SIMD&FP) — 32-bit	FEAT_FP
10	1	1x	UNALLOCATED	-
11		1x	UNALLOCATED	-
11	0	00	STR (immediate) — 64-bit	-
11	0	01	LDR (immediate) — 64-bit	-
11	1	00	STR (immediate, SIMD&FP) — 64-bit	FEAT_FP
11	1	01	LDR (immediate, SIMD&FP) — 64-bit	FEAT_FP

Atomic memory operations

The encodings in this section are decoded from Loads and Stores.



Decode fields								Instruction Details	Condition
size	VR	A	R	Rs	o3	opc	Rt		
	0		1		1	10x		UNALLOCATED	-
	~~0~~	~~0~~			~~1~~	~~100~~		~~UNALLOCATED~~	~~-~~
	0	0	0		1	100~~11x~~		UNALLOCATED	-
	0	0~~1~~	0~~1~~		1	11x~~100~~		UNALLOCATED	-
	0	0	1		1	11x		UNALLOCATED	-
	1				0	001		UNALLOCATED	-
	1				0	01x		UNALLOCATED	-
	1	0			1		!= 11111	UNALLOCATED	-
	1	0			1	001	11111	UNALLOCATED	-
	1	0			1	01x	11111	UNALLOCATED	-
	1	1			1			UNALLOCATED	-
0x	0	1	0		1	11x~~101~~		UNALLOCATED	-
0x	0	1~~0~~	1		1	11x~~101~~		UNALLOCATED	-
~~0x~~	~~0~~	~~1~~			~~1~~	~~11x~~		~~UNALLOCATED~~	~~-~~
~~0x~~	~~0~~	~~1~~	~~1~~		~~1~~	~~101~~		~~UNALLOCATED~~	~~-~~
00	0	0	0		0	000		LDADDB, LDADDAB, LDADDALB, LDADDLB — No memory ordering	FEAT_LSE
00	0	0	0		0	001		LDCLRB, LDCLRAB, LDCLRALB, LDCLRLB — No memory ordering	FEAT_LSE
00	0	0	0		0	010		LDEORB, LDEORAB, LDEORALB, LDEORLB — No memory ordering	FEAT_LSE
00	0	0	0		0	011		LDSETB, LDSETAB, LDSETALB, LDSETLB — No memory ordering	FEAT_LSE
00	0	0	0		0	100		LDSMAXB, LDSMAXAB, LDSMAXALB, LDSMAXLB — No memory ordering	FEAT_LSE
00	0	0	0		0	101		LDSMINB, LDSMINAB, LDSMINALB, LDSMINLB — No memory ordering	FEAT_LSE
00	0	0	0		0	110		LDUMAXB, LDUMAXAB, LDUMAXALB, LDUMAXLB — No memory ordering	FEAT_LSE
00	0	0	0		0	111		LDUMINB, LDUMINAB, LDUMINALB, LDUMINLB — No memory ordering	FEAT_LSE
00	0	0	0		1	000		SWPB, SWPAB, SWPALB, SWPLB — SWPB	FEAT_LSE
00	0	0	0		1	001		RCWCLR, RCWCLRA, RCWCLRAL, RCWCLRL — RCWCLR	FEAT_THE
00	0	0	0		1	010		RCWSWP, RCWSWPA, RCWSWPAL, RCWSWPL — RCWSWP	FEAT_THE
00	0	0	0		1	011		RCWSET, RCWSETA, RCWSETAL, RCWSETL — RCWSET	FEAT_THE
00	0	0	1		0	000		LDADDB, LDADDAB, LDADDALB, LDADDLB — Release	FEAT_LSE
00	0	0	1		0	001		LDCLRB, LDCLRAB, LDCLRALB, LDCLRLB — Release	FEAT_LSE
00	0	0	1		0	010		LDEORB, LDEORAB, LDEORALB, LDEORLB — Release	FEAT_LSE

Decode fields								Instruction Details	Condition
size	VR	A	R	Rs	o3	opc	Rt		
00	0	0	1		0	011		LDSETB, LDSETAB, LDSETALB, LDSETLB — Release	FEAT_LSE
00	0	0	1		0	100		LDSMAXB, LDSMAXAB, LDSMAXALB, LDSMAXLB — Release	FEAT_LSE
00	0	0	1		0	101		LDSMINB, LDSMINAB, LDSMINALB, LDSMINLB — Release	FEAT_LSE
00	0	0	1		0	110		LDUMAXB, LDUMAXAB, LDUMAXALB, LDUMAXLB — Release	FEAT_LSE
00	0	0	1		0	111		LDUMINB, LDUMINAB, LDUMINALB, LDUMINLB — Release	FEAT_LSE
00	0	0	1		1	000		SWPB, SWPAB, SWPALB, SWPLB — SWPLB	FEAT_LSE
00	0	0	1		1	001		RCWCLR, RCWCLRA, RCWCLRAL, RCWCLRL — RCWCLRL	FEAT_THE
00	0	0	1		1	010		RCWSWP, RCWSWPA, RCWSWPAL, RCWSWPL — RCWSWPL	FEAT_THE
00	0	0	1		1	011		RCWSET, RCWSETA, RCWSETAL, RCWSETL — RCWSETL	FEAT_THE
00	0	1	0		0	000		LDADDB, LDADDAB, LDADDALB, LDADDLB — Acquire	FEAT_LSE
00	0	1	0		0	001		LDCLRB, LDCLRAB, LDCLRALB, LDCLRLB — Acquire	FEAT_LSE
00	0	1	0		0	010		LDEORB, LDEORAB, LDEORALB, LDEORLB — Acquire	FEAT_LSE
00	0	1	0		0	011		LDSETB, LDSETAB, LDSETALB, LDSETLB — Acquire	FEAT_LSE
00	0	1	0		0	100		LDSMAXB, LDSMAXAB, LDSMAXALB, LDSMAXLB — Acquire	FEAT_LSE
00	0	1	0		0	101		LDSMINB, LDSMINAB, LDSMINALB, LDSMINLB — Acquire	FEAT_LSE
00	0	1	0		0	110		LDUMAXB, LDUMAXAB, LDUMAXALB, LDUMAXLB — Acquire	FEAT_LSE
00	0	1	0		0	111		LDUMINB, LDUMINAB, LDUMINALB, LDUMINLB — Acquire	FEAT_LSE
00	0	1	0		1	000		SWPB, SWPAB, SWPALB, SWPLB — SWPAB	FEAT_LSE
00	0	1	0		1	001		RCWCLR, RCWCLRA, RCWCLRAL, RCWCLRL — RCWCLRA	FEAT_THE
00	0	1	0		1	010		RCWSWP, RCWSWPA, RCWSWPAL, RCWSWPL — RCWSWPA	FEAT_THE
00	0	1	0		1	011		RCWSET, RCWSETA, RCWSETAL, RCWSETL — RCWSETA	FEAT_THE
00	0	1	0		1	100		LDAPRB	FEAT_LRCPC
00	0	1	1		0	000		LDADDB, LDADDAB, LDADDALB, LDADDLB — Acquire-release	FEAT_LSE

Decode fields								Instruction Details	Condition
size	VR	A	R	Rs	o3	opc	Rt		
00	0	1	1		0	001		LDCLRB, LDCLRAB, LDCLRALB, LDCLRLB — Acquire-release	FEAT_LSE
00	0	1	1		0	010		LDEORB, LDEORAB, LDEORALB, LDEORLB — Acquire-release	FEAT_LSE
00	0	1	1		0	011		LDSETB, LDSETAB, LDSETALB, LDSETLB — Acquire-release	FEAT_LSE
00	0	1	1		0	100		LDSMAXB, LDSMAXAB, LDSMAXALB, LDSMAXLB — Acquire-release	FEAT_LSE
00	0	1	1		0	101		LDSMINB, LDSMINAB, LDSMINALB, LDSMINLB — Acquire-release	FEAT_LSE
00	0	1	1		0	110		LDUMAXB, LDUMAXAB, LDUMAXALB, LDUMAXLB — Acquire-release	FEAT_LSE
00	0	1	1		0	111		LDUMINB, LDUMINAB, LDUMINALB, LDUMINLB — Acquire-release	FEAT_LSE
00	0	1	1		1	000		SWPB, SWPAB, SWPALB, SWPLB — SWPALB	FEAT_LSE
00	0	1	1		1	001		RCWCLR, RCWCLRA, RCWCLRAL, RCWCLRL — RCWCLRAL	FEAT_THE
00	0	1	1		1	010		RCWSWP, RCWSWPA, RCWSWPAL, RCWSWPL — RCWSWPAL	FEAT_THE
00	0	1	1		1	011		RCWSET, RCWSETA, RCWSETAL, RCWSETL — RCWSETAL	FEAT_THE
00	1	0	0		0	000		LDBFADD, LDBFADDA, LDBFADDAL, LDBFADDL — No memory ordering	FEAT_LSFE
00	1	0	0		0	100		LDBFMAX, LDBFMAXA, LDBFMAXAL, LDBFMAXL — No memory ordering	FEAT_LSFE
00	1	0	0		0	101		LDBFMIN, LDBFMINA, LDBFMINAL, LDBFMINL — No memory ordering	FEAT_LSFE
00	1	0	0		0	110		LDBFMAXNM, LDBFMAXNMA, LDBFMAXNMAL, LDBFMAXNML — No memory ordering	FEAT_LSFE
00	1	0	0		0	111		LDBFMINNM, LDBFMINNMA, LDBFMINNMAL, LDBFMINNML — No memory ordering	FEAT_LSFE
00	1	0	0		1	000	11111	STBFADD, STBFADDL — No memory ordering	FEAT_LSFE
00	1	0	0		1	100	11111	STBFMAX, STBFMAXL — No memory ordering	FEAT_LSFE
00	1	0	0		1	101	11111	STBFMIN, STBFMINL — No memory ordering	FEAT_LSFE
00	1	0	0		1	110	11111	STBFMAXNM, STBFMAXNML — No memory ordering	FEAT_LSFE
00	1	0	0		1	111	11111	STBFMINNM, STBFMINNML — No memory ordering	FEAT_LSFE
00	1	0	1		0	000		LDBFADD, LDBFADDA, LDBFADDAL, LDBFADDL — Release	FEAT_LSFE
00	1	0	1		0	100		LDBFMAX, LDBFMAXA, LDBFMAXAL, LDBFMAXL — Release	FEAT_LSFE

Decode fields								Instruction Details	Condition
size	VR	A	R	Rs	o3	opc	Rt		
00	1	0	1		0	101		LDBFMIN, LDBFMINA, LDBFMINAL, LDBFMINL — Release	FEAT_LSFE
00	1	0	1		0	110		LDBFMAXNM, LDBFMAXNMA, LDBFMAXNMAL, LDBFMAXNML — Release	FEAT_LSFE
00	1	0	1		0	111		LDBFMINNM, LDBFMINNMA, LDBFMINNMAL, LDBFMINNML — Release	FEAT_LSFE
00	1	0	1		1	000	11111	STBFADD, STBFADDL — Release	FEAT_LSFE
00	1	0	1		1	100	11111	STBFMAX, STBFMAXL — Release	FEAT_LSFE
00	1	0	1		1	101	11111	STBFMIN, STBFMINL — Release	FEAT_LSFE
00	1	0	1		1	110	11111	STBFMAXNM, STBFMAXNML — Release	FEAT_LSFE
00	1	0	1		1	111	11111	STBFMINNM, STBFMINNML — Release	FEAT_LSFE
00	1	1	0		0	000		LDBFADD, LDBFADDA, LDBFADDAL, LDBFADDL — Acquire	FEAT_LSFE
00	1	1	0		0	100		LDBFMAX, LDBFMAXA, LDBFMAXAL, LDBFMAXL — Acquire	FEAT_LSFE
00	1	1	0		0	101		LDBFMIN, LDBFMINA, LDBFMINAL, LDBFMINL — Acquire	FEAT_LSFE
00	1	1	0		0	110		LDBFMAXNM, LDBFMAXNMA, LDBFMAXNMAL, LDBFMAXNML — Acquire	FEAT_LSFE
00	1	1	0		0	111		LDBFMINNM, LDBFMINNMA, LDBFMINNMAL, LDBFMINNML — Acquire	FEAT_LSFE
00	1	1	1		0	000		LDBFADD, LDBFADDA, LDBFADDAL, LDBFADDL — Acquire-release	FEAT_LSFE
00	1	1	1		0	100		LDBFMAX, LDBFMAXA, LDBFMAXAL, LDBFMAXL — Acquire-release	FEAT_LSFE
00	1	1	1		0	101		LDBFMIN, LDBFMINA, LDBFMINAL, LDBFMINL — Acquire-release	FEAT_LSFE
00	1	1	1		0	110		LDBFMAXNM, LDBFMAXNMA, LDBFMAXNMAL, LDBFMAXNML — Acquire-release	FEAT_LSFE
00	1	1	1		0	111		LDBFMINNM, LDBFMINNMA, LDBFMINNMAL, LDBFMINNML — Acquire-release	FEAT_LSFE
01	0	0	0		0	000		LDADDH, LDADDAH, LDADDALH, LDADDLH — No memory ordering	FEAT_LSE
01	0	0	0		0	001		LDCLRH, LDCLRAH, LDCLRALH, LDCLRLH — No memory ordering	FEAT_LSE
01	0	0	0		0	010		LDEORH, LDEORAH, LDEORALH, LDEORLH — No memory ordering	FEAT_LSE
01	0	0	0		0	011		LDSETH, LDSETAH, LDSETALH, LDSETLH — No memory ordering	FEAT_LSE
01	0	0	0		0	100		LDSMAXH, LDSMAXAH, LDSMAXALH, LDSMAXLH — No memory ordering	FEAT_LSE

Decode fields								Instruction Details	Condition
size	VR	A	R	Rs	o3	opc	Rt		
01	0	0	0		0	101		LDSMINH, LDSMINAH, LDSMINALH, LDSMINLH — No memory ordering	FEAT_LSE
01	0	0	0		0	110		LDUMAXH, LDUMAXAH, LDUMAXALH, LDUMAXLH — No memory ordering	FEAT_LSE
01	0	0	0		0	111		LDUMINH, LDUMINAH, LDUMINALH, LDUMINLH — No memory ordering	FEAT_LSE
01	0	0	0		1	000		SWPH, SWPAH, SWPALH, SWPLH — SWPH	FEAT_LSE
01	0	0	0		1	001		RCWSCLR, RCWSCLRA, RCWSCLRAL, RCWSCLRL — RCWSCLR	FEAT_THE
01	0	0	0		1	010		RCWSSWP, RCWSSWPA, RCWSSWPAL, RCWSSWPL — RCWSSWP	FEAT_THE
01	0	0	0		1	011		RCWSSET, RCWSSETA, RCWSSETAL, RCWSSETL — RCWSSET	FEAT_THE
01	0	0	1		0	000		LDADDH, LDADDAH, LDADDALH, LDADDLH — Release	FEAT_LSE
01	0	0	1		0	001		LDCLRH, LDCLRAH, LDCLRALH, LDCLRLH — Release	FEAT_LSE
01	0	0	1		0	010		LDEORH, LDEORAH, LDEORALH, LDEORLH — Release	FEAT_LSE
01	0	0	1		0	011		LDSETH, LDSETAH, LDSETALH, LDSETLH — Release	FEAT_LSE
01	0	0	1		0	100		LDSMAXH, LDSMAXAH, LDSMAXALH, LDSMAXLH — Release	FEAT_LSE
01	0	0	1		0	101		LDSMINH, LDSMINAH, LDSMINALH, LDSMINLH — Release	FEAT_LSE
01	0	0	1		0	110		LDUMAXH, LDUMAXAH, LDUMAXALH, LDUMAXLH — Release	FEAT_LSE
01	0	0	1		0	111		LDUMINH, LDUMINAH, LDUMINALH, LDUMINLH — Release	FEAT_LSE
01	0	0	1		1	000		SWPH, SWPAH, SWPALH, SWPLH — SWPLH	FEAT_LSE
01	0	0	1		1	001		RCWSCLR, RCWSCLRA, RCWSCLRAL, RCWSCLRL — RCWSCLRL	FEAT_THE
01	0	0	1		1	010		RCWSSWP, RCWSSWPA, RCWSSWPAL, RCWSSWPL — RCWSSWPL	FEAT_THE
01	0	0	1		1	011		RCWSSET, RCWSSETA, RCWSSETAL, RCWSSETL — RCWSSETL	FEAT_THE
01	0	1	0		0	000		LDADDH, LDADDAH, LDADDALH, LDADDLH — Acquire	FEAT_LSE
01	0	1	0		0	001		LDCLRH, LDCLRAH, LDCLRALH, LDCLRLH — Acquire	FEAT_LSE
01	0	1	0		0	010		LDEORH, LDEORAH, LDEORALH, LDEORLH — Acquire	FEAT_LSE
01	0	1	0		0	011		LDSETH, LDSETAH, LDSETALH, LDSETLH — Acquire	FEAT_LSE

Decode fields								Instruction Details	Condition
size	VR	A	R	Rs	o3	opc	Rt		
01	0	1	0		0	100		LDSMAXH, LDSMAXAH, LDSMAXALH, LDSMAXLH — Acquire	FEAT_LSE
01	0	1	0		0	101		LDSMINH, LDSMINAH, LDSMINALH, LDSMINLH — Acquire	FEAT_LSE
01	0	1	0		0	110		LDUMAXH, LDUMAXAH, LDUMAXALH, LDUMAXLH — Acquire	FEAT_LSE
01	0	1	0		0	111		LDUMINH, LDUMINAH, LDUMINALH, LDUMINLH — Acquire	FEAT_LSE
01	0	1	0		1	000		SWPH, SWPAH, SWPALH, SWPLH — SWPAH	FEAT_LSE
01	0	1	0		1	001		RCWSCLR, RCWSCLRA, RCWSCLRAL, RCWSCLRL — RCWSCLRA	FEAT_THE
01	0	1	0		1	010		RCWSSWP, RCWSSWPA, RCWSSWPAL, RCWSSWPL — RCWSSWPA	FEAT_THE
01	0	1	0		1	011		RCWSSET, RCWSSETA, RCWSSETAL, RCWSSETL — RCWSSETA	FEAT_THE
01	0	1	0		1	100		LDAPRH	FEAT_LRCPC
01	0	1	1		0	000		LDADDH, LDADDAH, LDADDALH, LDADDLH — Acquire-release	FEAT_LSE
01	0	1	1		0	001		LDCLRH, LDCLRAH, LDCLRALH, LDCLRLH — Acquire-release	FEAT_LSE
01	0	1	1		0	010		LDEORH, LDEORAH, LDEORALH, LDEORLH — Acquire-release	FEAT_LSE
01	0	1	1		0	011		LDSETH, LDSETAH, LDSETALH, LDSETLH — Acquire-release	FEAT_LSE
01	0	1	1		0	100		LDSMAXH, LDSMAXAH, LDSMAXALH, LDSMAXLH — Acquire-release	FEAT_LSE
01	0	1	1		0	101		LDSMINH, LDSMINAH, LDSMINALH, LDSMINLH — Acquire-release	FEAT_LSE
01	0	1	1		0	110		LDUMAXH, LDUMAXAH, LDUMAXALH, LDUMAXLH — Acquire-release	FEAT_LSE
01	0	1	1		0	111		LDUMINH, LDUMINAH, LDUMINALH, LDUMINLH — Acquire-release	FEAT_LSE
01	0	1	1		1	000		SWPH, SWPAH, SWPALH, SWPLH — SWPALH	FEAT_LSE
01	0	1	1		1	001		RCWSCLR, RCWSCLRA, RCWSCLRAL, RCWSCLRL — RCWSCLRAL	FEAT_THE
01	0	1	1		1	010		RCWSSWP, RCWSSWPA, RCWSSWPAL, RCWSSWPL — RCWSSWPAL	FEAT_THE
01	0	1	1		1	011		RCWSSET, RCWSSETA, RCWSSETAL, RCWSSETL — RCWSSETAL	FEAT_THE
01	1	0	0		0	000		LDFADD, LDFADDA, LDFADDAL, LDFADDL — Half-precision no memory ordering	FEAT_LSFE
01	1	0	0		0	100		LDFMAX, LDFMAXA, LDFMAXAL, LDFMAXL — Half-precision no memory ordering	FEAT_LSFE

Decode fields								Instruction Details	Condition
size	VR	A	R	Rs	o3	opc	Rt		
01	1	0	0		0	101		LDFMIN, LDFMINA, LDFMINAL, LDFMINL — Half-precision no memory ordering	FEAT_LSFE
01	1	0	0		0	110		LDFMAXNM, LDFMAXNMA, LDFMAXNMAL, LDFMAXNML — Half-precision no memory ordering	FEAT_LSFE
01	1	0	0		0	111		LDFMINNM, LDFMINNMA, LDFMINNMAL, LDFMINNML — Half-precision no memory ordering	FEAT_LSFE
01	1	0	0		1	000	11111	STFADD, STFADDL — Half-precision no memory ordering	FEAT_LSFE
01	1	0	0		1	100	11111	STFMAX, STFMAXL — Half-precision no memory ordering	FEAT_LSFE
01	1	0	0		1	101	11111	STFMIN, STFMINL — Half-precision no memory ordering	FEAT_LSFE
01	1	0	0		1	110	11111	STFMAXNM, STFMAXNML — Half-precision no memory ordering	FEAT_LSFE
01	1	0	0		1	111	11111	STFMINNM, STFMINNML — Half-precision no memory ordering	FEAT_LSFE
01	1	0	1		0	000		LDFADD, LDFADDA, LDFADDAL, LDFADDL — Half-precision release	FEAT_LSFE
01	1	0	1		0	100		LDFMAX, LDFMAXA, LDFMAXAL, LDFMAXL — Half-precision release	FEAT_LSFE
01	1	0	1		0	101		LDFMIN, LDFMINA, LDFMINAL, LDFMINL — Half-precision release	FEAT_LSFE
01	1	0	1		0	110		LDFMAXNM, LDFMAXNMA, LDFMAXNMAL, LDFMAXNML — Half-precision release	FEAT_LSFE
01	1	0	1		0	111		LDFMINNM, LDFMINNMA, LDFMINNMAL, LDFMINNML — Half-precision release	FEAT_LSFE
01	1	0	1		1	000	11111	STFADD, STFADDL — Half-precision release	FEAT_LSFE
01	1	0	1		1	100	11111	STFMAX, STFMAXL — Half-precision release	FEAT_LSFE
01	1	0	1		1	101	11111	STFMIN, STFMINL — Half-precision release	FEAT_LSFE
01	1	0	1		1	110	11111	STFMAXNM, STFMAXNML — Half-precision release	FEAT_LSFE
01	1	0	1		1	111	11111	STFMINNM, STFMINNML — Half-precision release	FEAT_LSFE
01	1	1	0		0	000		LDFADD, LDFADDA, LDFADDAL, LDFADDL — Half-precision acquire	FEAT_LSFE
01	1	1	0		0	100		LDFMAX, LDFMAXA, LDFMAXAL, LDFMAXL — Half-precision acquire	FEAT_LSFE
01	1	1	0		0	101		LDFMIN, LDFMINA, LDFMINAL, LDFMINL — Half-precision acquire	FEAT_LSFE
01	1	1	0		0	110		LDFMAXNM, LDFMAXNMA, LDFMAXNMAL, LDFMAXNML — Half-precision acquire	FEAT_LSFE
01	1	1	0		0	111		LDFMINNM, LDFMINNMA, LDFMINNMAL, LDFMINNML — Half-precision acquire	FEAT_LSFE
01	1	1	1		0	000		LDFADD, LDFADDA, LDFADDAL, LDFADDL — Half-precision acquire-release	FEAT_LSFE

Decode fields								Instruction Details	Condition
size	VR	A	R	Rs	o3	opc	Rt		
01	1	1	1		0	100		LDFMAX, LDFMAXA, LDFMAXAL, LDFMAXL — Half-precision acquire-release	FEAT_LSFE
01	1	1	1		0	101		LDFMIN, LDFMINA, LDFMINAL, LDFMINL — Half-precision acquire-release	FEAT_LSFE
01	1	1	1		0	110		LDFMAXNM, LDFMAXNMA, LDFMAXNMAL, LDFMAXNML — Half-precision acquire-release	FEAT_LSFE
01	1	1	1		0	111		LDFMINNM, LDFMINNMA, LDFMINNMAL, LDFMINNML — Half-precision acquire-release	FEAT_LSFE
1x	0	1			1	x1x~~!=~~ ~~x00~~		UNALLOCATED	-
10	0				1	001~~x01~~		UNALLOCATED	-
10	0	0			1	010		UNALLOCATED	-
10	0	0			1	011		UNALLOCATED	-
10	0	0	0		0	000		LDADD, LDADDA, LDADDAL, LDADDL — 32-bit no memory ordering	FEAT_LSE
10	0	0	0		0	001		LDCLR, LDCLRA, LDCLRAL, LDCLRL — 32-bit no memory ordering	FEAT_LSE
10	0	0	0		0	010		LDEOR, LDEORA, LDEORAL, LDEORL — 32-bit no memory ordering	FEAT_LSE
10	0	0	0		0	011		LDSET, LDSETA, LDSETAL, LDSETL — 32-bit no memory ordering	FEAT_LSE
10	0	0	0		0	100		LDSMAX, LDSMAXA, LDSMAXAL, LDSMAXL — 32-bit no memory ordering	FEAT_LSE
10	0	0	0		0	101		LDSMIN, LDSMINA, LDSMINAL, LDSMINL — 32-bit no memory ordering	FEAT_LSE
10	0	0	0		0	110		LDUMAX, LDUMAXA, LDUMAXAL, LDUMAXL — 32-bit no memory ordering	FEAT_LSE
10	0	0	0		0	111		LDUMIN, LDUMINA, LDUMINAL, LDUMINL — 32-bit no memory ordering	FEAT_LSE
10	0	0	0		1	000		SWP, SWPA, SWPAL, SWPL — 32-bit SWP	FEAT_LSE
10	0	0	1		0	000		LDADD, LDADDA, LDADDAL, LDADDL — 32-bit release	FEAT_LSE
10	0	0	1		0	001		LDCLR, LDCLRA, LDCLRAL, LDCLRL — 32-bit release	FEAT_LSE
10	0	0	1		0	010		LDEOR, LDEORA, LDEORAL, LDEORL — 32-bit release	FEAT_LSE
10	0	0	1		0	011		LDSET, LDSETA, LDSETAL, LDSETL — 32-bit release	FEAT_LSE
10	0	0	1		0	100		LDSMAX, LDSMAXA, LDSMAXAL, LDSMAXL — 32-bit release	FEAT_LSE
10	0	0	1		0	101		LDSMIN, LDSMINA, LDSMINAL, LDSMINL — 32-bit release	FEAT_LSE
10	0	0	1		0	110		LDUMAX, LDUMAXA, LDUMAXAL, LDUMAXL — 32-bit release	FEAT_LSE
10	0	0	1		0	111		LDUMIN, LDUMINA, LDUMINAL, LDUMINL — 32-bit release	FEAT_LSE

Decode fields								Instruction Details	Condition
size	VR	A	R	Rs	o3	opc	Rt		
10	0	0	1		1	000		SWP, SWPA, SWPAL, SWPL — 32-bit SWPL	FEAT_LSE
10	0	1	0		0	000		LDADD, LDADDA, LDADDAL, LDADDL — 32-bit acquire	FEAT_LSE
10	0	1	0		0	001		LDCLR, LDCLRA, LDCLRAL, LDCLRL — 32-bit acquire	FEAT_LSE
10	0	1	0		0	010		LDEOR, LDEORA, LDEORAL, LDEORL — 32-bit acquire	FEAT_LSE
10	0	1	0		0	011		LDSET, LDSETA, LDSETAL, LDSETL — 32-bit acquire	FEAT_LSE
10	0	1	0		0	100		LDSMAX, LDSMAXA, LDSMAXAL, LDSMAXL — 32-bit acquire	FEAT_LSE
10	0	1	0		0	101		LDSMIN, LDSMINA, LDSMINAL, LDSMINL — 32-bit acquire	FEAT_LSE
10	0	1	0		0	110		LDUMAX, LDUMAXA, LDUMAXAL, LDUMAXL — 32-bit acquire	FEAT_LSE
10	0	1	0		0	111		LDUMIN, LDUMINA, LDUMINAL, LDUMINL — 32-bit acquire	FEAT_LSE
10	0	1	0		1	000		SWP, SWPA, SWPAL, SWPL — 32-bit SWPA	FEAT_LSE
10	0	1	0		1	100		LDAPR — 32-bit	FEAT_LRCPC
10	0	1	1		0	000		LDADD, LDADDA, LDADDAL, LDADDL — 32-bit acquire-release	FEAT_LSE
10	0	1	1		0	001		LDCLR, LDCLRA, LDCLRAL, LDCLRL — 32-bit acquire-release	FEAT_LSE
10	0	1	1		0	010		LDEOR, LDEORA, LDEORAL, LDEORL — 32-bit acquire-release	FEAT_LSE
10	0	1	1		0	011		LDSET, LDSETA, LDSETAL, LDSETL — 32-bit acquire-release	FEAT_LSE
10	0	1	1		0	100		LDSMAX, LDSMAXA, LDSMAXAL, LDSMAXL — 32-bit acquire-release	FEAT_LSE
10	0	1	1		0	101		LDSMIN, LDSMINA, LDSMINAL, LDSMINL — 32-bit acquire-release	FEAT_LSE
10	0	1	1		0	110		LDUMAX, LDUMAXA, LDUMAXAL, LDUMAXL — 32-bit acquire-release	FEAT_LSE
10	0	1	1		0	111		LDUMIN, LDUMINA, LDUMINAL, LDUMINL — 32-bit acquire-release	FEAT_LSE
10	0	1	1		1	000		SWP, SWPA, SWPAL, SWPL — 32-bit SWPAL	FEAT_LSE
10	1	0	0		0	000		LDFADD, LDFADDA, LDFADDAL, LDFADDL — Single-precision no memory ordering	FEAT_LSFE
10	1	0	0		0	100		LDFMAX, LDFMAXA, LDFMAXAL, LDFMAXL — Single-precision no memory ordering	FEAT_LSFE
10	1	0	0		0	101		LDFMIN, LDFMINA, LDFMINAL, LDFMINL — Single-precision no memory ordering	FEAT_LSFE
10	1	0	0		0	110		LDFMAXNM, LDFMAXNMA, LDFMAXNMAL, LDFMAXNML — Single-precision no memory ordering	FEAT_LSFE
10	1	0	0		0	111		LDFMINNM, LDFMINNMA, LDFMINNMAL, LDFMINNML — Single-precision no memory ordering	FEAT_LSFE

Decode fields								Instruction Details	Condition
size	VR	A	R	Rs	o3	opc	Rt		
10	1	0	0		1	000	11111	STFADD, STFADDL — Single-precision no memory ordering	FEAT_LSFE
10	1	0	0		1	100	11111	STFMAX, STFMAXL — Single-precision no memory ordering	FEAT_LSFE
10	1	0	0		1	101	11111	STFMIN, STFMINL — Single-precision no memory ordering	FEAT_LSFE
10	1	0	0		1	110	11111	STFMAXNM, STFMAXNML — Single-precision no memory ordering	FEAT_LSFE
10	1	0	0		1	111	11111	STFMINNM, STFMINNML — Single-precision no memory ordering	FEAT_LSFE
10	1	0	1		0	000		LDFADD, LDFADDA, LDFADDAL, LDFADDL — Single-precision release	FEAT_LSFE
10	1	0	1		0	100		LDFMAX, LDFMAXA, LDFMAXAL, LDFMAXL — Single-precision release	FEAT_LSFE
10	1	0	1		0	101		LDFMIN, LDFMINA, LDFMINAL, LDFMINL — Single-precision release	FEAT_LSFE
10	1	0	1		0	110		LDFMAXNM, LDFMAXNMA, LDFMAXNMAL, LDFMAXNML — Single-precision release	FEAT_LSFE
10	1	0	1		0	111		LDFMINNM, LDFMINNMA, LDFMINNMAL, LDFMINNML — Single-precision release	FEAT_LSFE
10	1	0	1		1	000	11111	STFADD, STFADDL — Single-precision release	FEAT_LSFE
10	1	0	1		1	100	11111	STFMAX, STFMAXL — Single-precision release	FEAT_LSFE
10	1	0	1		1	101	11111	STFMIN, STFMINL — Single-precision release	FEAT_LSFE
10	1	0	1		1	110	11111	STFMAXNM, STFMAXNML — Single-precision release	FEAT_LSFE
10	1	0	1		1	111	11111	STFMINNM, STFMINNML — Single-precision release	FEAT_LSFE
10	1	1	0		0	000		LDFADD, LDFADDA, LDFADDAL, LDFADDL — Single-precision acquire	FEAT_LSFE
10	1	1	0		0	100		LDFMAX, LDFMAXA, LDFMAXAL, LDFMAXL — Single-precision acquire	FEAT_LSFE
10	1	1	0		0	101		LDFMIN, LDFMINA, LDFMINAL, LDFMINL — Single-precision acquire	FEAT_LSFE
10	1	1	0		0	110		LDFMAXNM, LDFMAXNMA, LDFMAXNMAL, LDFMAXNML — Single-precision acquire	FEAT_LSFE
10	1	1	0		0	111		LDFMINNM, LDFMINNMA, LDFMINNMAL, LDFMINNML — Single-precision acquire	FEAT_LSFE
10	1	1	1		0	000		LDFADD, LDFADDA, LDFADDAL, LDFADDL — Single-precision acquire-release	FEAT_LSFE
10	1	1	1		0	100		LDFMAX, LDFMAXA, LDFMAXAL, LDFMAXL — Single-precision acquire-release	FEAT_LSFE
10	1	1	1		0	101		LDFMIN, LDFMINA, LDFMINAL, LDFMINL — Single-precision acquire-release	FEAT_LSFE
10	1	1	1		0	110		LDFMAXNM, LDFMAXNMA, LDFMAXNMAL, LDFMAXNML — Single-precision acquire-release	FEAT_LSFE

Decode fields								Instruction Details	Condition
size	VR	A	R	Rs	o3	opc	Rt		
10	1	1	1		0	111		LDFMINNM, LDFMINNMA, LDFMINNMAL, LDFMINNML — Single-precision acquire-release	FEAT_LSFE
11	0		0	!= 11111	1	101		UNALLOCATED	-
11	0	0		!= 11111	1	001		UNALLOCATED	-
11	0	0	0		0	000		LDADD, LDADDA, LDADDAL, LDADDL — 64-bit no memory ordering	FEAT_LSE
11	0	0	0		0	001		LDCLR, LDCLRA, LDCLRAL, LDCLRL — 64-bit no memory ordering	FEAT_LSE
11	0	0	0		0	010		LDEOR, LDEORA, LDEORAL, LDEORL — 64-bit no memory ordering	FEAT_LSE
11	0	0	0		0	011		LDSET, LDSETA, LDSETAL, LDSETL — 64-bit no memory ordering	FEAT_LSE
11	0	0	0		0	100		LDSMAX, LDSMAXA, LDSMAXAL, LDSMAXL — 64-bit no memory ordering	FEAT_LSE
11	0	0	0		0	101		LDSMIN, LDSMINA, LDSMINAL, LDSMINL — 64-bit no memory ordering	FEAT_LSE
11	0	0	0		0	110		LDUMAX, LDUMAXA, LDUMAXAL, LDUMAXL — 64-bit no memory ordering	FEAT_LSE
11	0	0	0		0	111		LDUMIN, LDUMINA, LDUMINAL, LDUMINL — 64-bit no memory ordering	FEAT_LSE
11	0	0	0		1	000		SWP, SWPA, SWPAL, SWPL — 64-bit SWP	FEAT_LSE
11	0	0	0		1	010		ST64BV0	FEAT_LS64_ACCDATA
11	0	0	0		1	011		ST64BV	FEAT_LS64_V
11	0	0	0	11111	1	001		ST64B	FEAT_LS64
11	0	0	0	11111	1	101		LD64B	FEAT_LS64
~~11~~	~~0~~	~~0~~	~~0~~	~~!= 11111~~	~~1~~	~~x01~~		~~UNALLOCATED~~	~~-~~
11	0	0	1		0	000		LDADD, LDADDA, LDADDAL, LDADDL — 64-bit release	FEAT_LSE
11	0	0	1		0	001		LDCLR, LDCLRA, LDCLRAL, LDCLRL — 64-bit release	FEAT_LSE
11	0	0	1		0	010		LDEOR, LDEORA, LDEORAL, LDEORL — 64-bit release	FEAT_LSE
11	0	0	1		0	011		LDSET, LDSETA, LDSETAL, LDSETL — 64-bit release	FEAT_LSE
11	0	0	1		0	100		LDSMAX, LDSMAXA, LDSMAXAL, LDSMAXL — 64-bit release	FEAT_LSE
11	0	0	1		0	101		LDSMIN, LDSMINA, LDSMINAL, LDSMINL — 64-bit release	FEAT_LSE
11	0	0	1		0	110		LDUMAX, LDUMAXA, LDUMAXAL, LDUMAXL — 64-bit release	FEAT_LSE
11	0	0	1		0	111		LDUMIN, LDUMINA, LDUMINAL, LDUMINL — 64-bit release	FEAT_LSE
~~11~~	~~0~~	~~0~~	~~1~~		~~1~~	~~x01~~		~~UNALLOCATED~~	~~-~~

Decode fields								Instruction Details	Condition
size	VR	A	R	Rs	o3	opc	Rt		
11	0	0	1		1	000		SWP, SWPA, SWPAL, SWPL — 64-bit SWPL	FEAT_LSE
11	0	0	1		1	010		~~UNALLOCATED~~STL64BV0	FEAT_LS64_ACCDATA && FEAT_LSC64B-
11	0	0	1		1	011		~~UNALLOCATED~~STL64BV	FEAT_LS64_V && FEAT_LSC64B-
11	0	0	1	11111	1	001		STL64B	FEAT_LS64 && FEAT_LSC64B
11	0	1			1	001		UNALLOCATED	-
11	0	1	0		0	000		LDADD, LDADDA, LDADDAL, LDADDL — 64-bit acquire	FEAT_LSE
11	0	1	0		0	001		LDCLR, LDCLRA, LDCLRAL, LDCLRL — 64-bit acquire	FEAT_LSE
11	0	1	0		0	010		LDEOR, LDEORA, LDEORAL, LDEORL — 64-bit acquire	FEAT_LSE
11	0	1	0		0	011		LDSET, LDSETA, LDSETAL, LDSETL — 64-bit acquire	FEAT_LSE
11	0	1	0		0	100		LDSMAX, LDSMAXA, LDSMAXAL, LDSMAXL — 64-bit acquire	FEAT_LSE
11	0	1	0		0	101		LDSMIN, LDSMINA, LDSMINAL, LDSMINL — 64-bit acquire	FEAT_LSE
11	0	1	0		0	110		LDUMAX, LDUMAXA, LDUMAXAL, LDUMAXL — 64-bit acquire	FEAT_LSE
11	0	1	0		0	111		LDUMIN, LDUMINA, LDUMINAL, LDUMINL — 64-bit acquire	FEAT_LSE
11	0	1	0		1	000		SWP, SWPA, SWPAL, SWPL — 64-bit SWPA	FEAT_LSE
11	0	1	0		1	100		LDAPR — 64-bit	FEAT_LRCPC
11	0	1	0	11111	1	101		LDA64B	FEAT_LS64 && FEAT_LSC64B
11	0	1	1		0	000		LDADD, LDADDA, LDADDAL, LDADDL — 64-bit acquire-release	FEAT_LSE
11	0	1	1		0	001		LDCLR, LDCLRA, LDCLRAL, LDCLRL — 64-bit acquire-release	FEAT_LSE
11	0	1	1		0	010		LDEOR, LDEORA, LDEORAL, LDEORL — 64-bit acquire-release	FEAT_LSE
11	0	1	1		0	011		LDSET, LDSETA, LDSETAL, LDSETL — 64-bit acquire-release	FEAT_LSE
11	0	1	1		0	100		LDSMAX, LDSMAXA, LDSMAXAL, LDSMAXL — 64-bit acquire-release	FEAT_LSE
11	0	1	1		0	101		LDSMIN, LDSMINA, LDSMINAL, LDSMINL — 64-bit acquire-release	FEAT_LSE
11	0	1	1		0	110		LDUMAX, LDUMAXA, LDUMAXAL, LDUMAXL — 64-bit acquire-release	FEAT_LSE
11	0	1	1		0	111		LDUMIN, LDUMINA, LDUMINAL, LDUMINL — 64-bit acquire-release	FEAT_LSE
11	0	1	1		1	000		SWP, SWPA, SWPAL, SWPL — 64-bit SWPAL	FEAT_LSE
11	1	0	0		0	000		LDFADD, LDFADDA, LDFADDAL, LDFADDL — Double-precision no memory ordering	FEAT_LSFE
11	1	0	0		0	100		LDFMAX, LDFMAXA, LDFMAXAL, LDFMAXL — Double-precision no memory ordering	FEAT_LSFE

Decode fields								Instruction Details	Condition
size	VR	A	R	Rs	o3	opc	Rt		
11	1	0	0		0	101		LDFMIN, LDFMINA, LDFMINAL, LDFMINL — Double-precision no memory ordering	FEAT_LSFE
11	1	0	0		0	110		LDFMAXNM, LDFMAXNMA, LDFMAXNMAL, LDFMAXNML — Double-precision no memory ordering	FEAT_LSFE
11	1	0	0		0	111		LDFMINNM, LDFMINNMA, LDFMINNMAL, LDFMINNML — Double-precision no memory ordering	FEAT_LSFE
11	1	0	0		1	000	11111	STFADD, STFADDL — Double-precision no memory ordering	FEAT_LSFE
11	1	0	0		1	100	11111	STFMAX, STFMAXL — Double-precision no memory ordering	FEAT_LSFE
11	1	0	0		1	101	11111	STFMIN, STFMINL — Double-precision no memory ordering	FEAT_LSFE
11	1	0	0		1	110	11111	STFMAXNM, STFMAXNML — Double-precision no memory ordering	FEAT_LSFE
11	1	0	0		1	111	11111	STFMINNM, STFMINNML — Double-precision no memory ordering	FEAT_LSFE
11	1	0	1		0	000		LDFADD, LDFADDA, LDFADDAL, LDFADDL — Double-precision release	FEAT_LSFE
11	1	0	1		0	100		LDFMAX, LDFMAXA, LDFMAXAL, LDFMAXL — Double-precision release	FEAT_LSFE
11	1	0	1		0	101		LDFMIN, LDFMINA, LDFMINAL, LDFMINL — Double-precision release	FEAT_LSFE
11	1	0	1		0	110		LDFMAXNM, LDFMAXNMA, LDFMAXNMAL, LDFMAXNML — Double-precision release	FEAT_LSFE
11	1	0	1		0	111		LDFMINNM, LDFMINNMA, LDFMINNMAL, LDFMINNML — Double-precision release	FEAT_LSFE
11	1	0	1		1	000	11111	STFADD, STFADDL — Double-precision release	FEAT_LSFE
11	1	0	1		1	100	11111	STFMAX, STFMAXL — Double-precision release	FEAT_LSFE
11	1	0	1		1	101	11111	STFMIN, STFMINL — Double-precision release	FEAT_LSFE
11	1	0	1		1	110	11111	STFMAXNM, STFMAXNML — Double-precision release	FEAT_LSFE
11	1	0	1		1	111	11111	STFMINNM, STFMINNML — Double-precision release	FEAT_LSFE
11	1	1	0		0	000		LDFADD, LDFADDA, LDFADDAL, LDFADDL — Double-precision acquire	FEAT_LSFE
11	1	1	0		0	100		LDFMAX, LDFMAXA, LDFMAXAL, LDFMAXL — Double-precision acquire	FEAT_LSFE
11	1	1	0		0	101		LDFMIN, LDFMINA, LDFMINAL, LDFMINL — Double-precision acquire	FEAT_LSFE
11	1	1	0		0	110		LDFMAXNM, LDFMAXNMA, LDFMAXNMAL, LDFMAXNML — Double-precision acquire	FEAT_LSFE
11	1	1	0		0	111		LDFMINNM, LDFMINNMA, LDFMINNMAL, LDFMINNML — Double-precision acquire	FEAT_LSFE

Decode fields								Instruction Details	Condition
size	VR	A	R	Rs	o3	opc	Rt		
11	1	1	1		0	000		LDFADD, LDFADDA, LDFADDAL, LDFADDL — Double-precision acquire-release	FEAT_LSFE
11	1	1	1		0	100		LDFMAX, LDFMAXA, LDFMAXAL, LDFMAXL — Double-precision acquire-release	FEAT_LSFE
11	1	1	1		0	101		LDFMIN, LDFMINA, LDFMINAL, LDFMINL — Double-precision acquire-release	FEAT_LSFE
11	1	1	1		0	110		LDFMAXNM, LDFMAXNMA, LDFMAXNMAL, LDFMAXNML — Double-precision acquire-release	FEAT_LSFE
11	1	1	1		0	111		LDFMINNM, LDFMINNMA, LDFMINNMAL, LDFMINNML — Double-precision acquire-release	FEAT_LSFE
!= 11	0		0		1	101		UNALLOCATED	-

Load/store register (register offset)

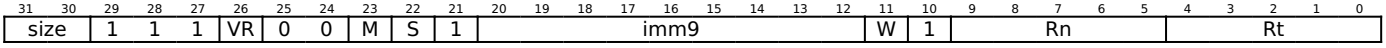
The encodings in this section are decoded from Loads and Stores.

31 30	29 28 27	26	25 24	23 22	21	20 19 18 17 16	15 14 13	12	11 10	9 8 7 6 5	4 3 2 1 0
size	1 1 1	VR	0 0	opc	1	Rm	option	S	1 0	Rn	Rt

Decode fields					Instruction Details	Condition
size	VR	opc	option	Rt		
00	0	00	011		STRB (register) — Shifted register	-
00	0	00	!= 011		STRB (register) — Extended register	-
00	0	01	011		LDRB (register) — Shifted register	-
00	0	01	!= 011		LDRB (register) — Extended register	-
00	0	10	011		LDRSB (register) — 64-bit with shifted register offset	-
00	0	10	!= 011		LDRSB (register) — 64-bit with extended register offset	-
00	0	11	011		LDRSB (register) — 32-bit with shifted register offset	-
00	0	11	!= 011		LDRSB (register) — 32-bit with extended register offset	-
00	1	00	011		STR (register, SIMD&FP) — 8-bit with shifted register offset	FEAT_FP
00	1	00	!= 011		STR (register, SIMD&FP) — 8-bit with extended register offset	FEAT_FP
00	1	01	011		LDR (register, SIMD&FP) — 8-bit with shifted register offset	FEAT_FP
00	1	01	!= 011		LDR (register, SIMD&FP) — 8-bit with extended register offset	FEAT_FP
00	1	10			STR (register, SIMD&FP) — 128-bit	FEAT_FP
00	1	11			LDR (register, SIMD&FP) — 128-bit	FEAT_FP
01	0	00			STRH (register)	-
01	0	01			LDRH (register)	-
01	0	10			LDRSH (register) — 64-bit	-
01	0	11			LDRSH (register) — 32-bit	-
01	1	00			STR (register, SIMD&FP) — 16-bit	FEAT_FP
01	1	01			LDR (register, SIMD&FP) — 16-bit	FEAT_FP
1x	0	11			UNALLOCATED	-
10	0	00			STR (register) — 32-bit	-
10	0	01			LDR (register) — 32-bit	-
10	0	10			LDRSW (register)	-
10	1	00			STR (register, SIMD&FP) — 32-bit	FEAT_FP
10	1	01			LDR (register, SIMD&FP) — 32-bit	FEAT_FP
11	0	00			STR (register) — 64-bit	-
11	0	01			LDR (register) — 64-bit	-
11	0	10	x0x		UNALLOCATED	-
11	0	10	x1x	11xxx	RPRFM	FEAT_RPRFM
11	0	10	x1x	!= 11xxx	PRFM (register)	-
11	1	00			STR (register, SIMD&FP) — 64-bit	FEAT_FP
11	1	01			LDR (register, SIMD&FP) — 64-bit	FEAT_FP
!= 00	1	1x			UNALLOCATED	-

Load/store register (pac)

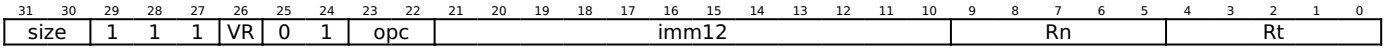
The encodings in this section are decoded from Loads and Stores.



Decode fields size	VR	M	W	Instruction Details	Condition
11	0	0	0	LDRAA, LDRAB — Key A, offset	FEAT_PAuth
11	0	0	1	LDRAA, LDRAB — Key A, pre-indexed	FEAT_PAuth
11	0	1	0	LDRAA, LDRAB — Key B, offset	FEAT_PAuth
11	0	1	1	LDRAA, LDRAB — Key B, pre-indexed	FEAT_PAuth
11	1			UNALLOCATED	-
!= 11				UNALLOCATED	-

Load/store register (unsigned immediate)

The encodings in this section are decoded from Loads and Stores.



Decode fields size	VR	opc	Instruction Details	Condition
00	0	00	STRB (immediate)	-
00	0	01	LDRB (immediate)	-
00	0	10	LDRSB (immediate) — 64-bit	-
00	0	11	LDRSB (immediate) — 32-bit	-
00	1	00	STR (immediate, SIMD&FP) — 8-bit	FEAT_FP
00	1	01	LDR (immediate, SIMD&FP) — 8-bit	FEAT_FP
00	1	10	STR (immediate, SIMD&FP) — 128-bit	FEAT_FP
00	1	11	LDR (immediate, SIMD&FP) — 128-bit	FEAT_FP
01	0	00	STRH (immediate)	-
01	0	01	LDRH (immediate)	-
01	0	10	LDRSH (immediate) — 64-bit	-
01	0	11	LDRSH (immediate) — 32-bit	-
01	1	00	STR (immediate, SIMD&FP) — 16-bit	FEAT_FP
01	1	01	LDR (immediate, SIMD&FP) — 16-bit	FEAT_FP
1x	0	11	UNALLOCATED	-
10	0	00	STR (immediate) — 32-bit	-
10	0	01	LDR (immediate) — 32-bit	-
10	0	10	LDRSW (immediate)	-
10	1	00	STR (immediate, SIMD&FP) — 32-bit	FEAT_FP
10	1	01	LDR (immediate, SIMD&FP) — 32-bit	FEAT_FP
11	0	00	STR (immediate) — 64-bit	-
11	0	01	LDR (immediate) — 64-bit	-
11	0	10	PRFM (immediate)	-
11	1	00	STR (immediate, SIMD&FP) — 64-bit	FEAT_FP
11	1	01	LDR (immediate, SIMD&FP) — 64-bit	FEAT_FP
!= 00	1	1x	UNALLOCATED	-

2026-09_rcl~~2026-06_diff_tag~~2026-09-21~~2026-09-08~~ 14~~09~~:22~~44~~:38~~01~~

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved.

(old) htmldiff from- (new)

Shared Pseudocode Functions

This page displays common pseudocode functions shared by many pages

Pseudocode

Library pseudocode for aarch32/at/AArch32_EncodePARLD

```
// AArch32_EncodePARLD()
// =====================
// Returns 64-bit format PAR on address translation instruction.

func AArch32_EncodePARLD(addrdesc : AddressDescriptor, ss : SecurityState)
begin
    PAR()[63:0] = Zeros{64};
    if !IsFault(addrdesc) then
        var ns : bit;
        if ss == SS_NonSecure then
            ns = ARBITRARY : bit;
        elsif addrdesc.paddress.paspace == PAS_Secure then
            ns = '0';
        else
            ns = '1';
        end;
        PAR().F      = '0';
        PAR().F0_64.SH     = PAR().SH      = ReportedPARShareability(PAREncodeShareability(addrdesc.memattrs));
        PAR().NS     = ns;
        PAR()[10]    = ImpDefBit("Non-Faulting PAR");               // IMPDEF
        PAR().LPAE   = '1';
        PAR().F0_64.PA     = addrdesc.paddress.address[39:12];
        PAR().PA     = addrdesc.paddress.address[39:12];
        PAR().ATTR   = ReportedPARAttrs(EncodePARAttrs(addrdesc.memattrs));
    else
        PAR().F      = '1';
        PAR().FST    = AArch32_PARFaultStatusLD(addrdesc.fault);
        PAR().S2WLK  = if addrdesc.fault.s2fs1walk then '1' else '0';
        PAR().FSTAGE = if addrdesc.fault.secondstage then '1' else '0';
        PAR().LPAE   = '1';
        PAR()[63:48] = ImpDefBits{16}("Faulting PAR");             // IMPDEF
    end;
    return;
end;
```

Library pseudocode for aarch32/at/AArch32_EncodePARSD

```
// AArch32_EncodePARSD()
// =====================
// Returns 32-bit format PAR on address translation instruction.

func AArch32_EncodePARSD(addrdesc_in : AddressDescriptor, supersection : bit, ss : SecurityState)
begin
    PAR()[31:0] = Zeros{32};
    var addrdesc : AddressDescriptor = addrdesc_in;
    if !IsFault(addrdesc) then
        if (addrdesc.mematttrs.memtype == MemType_Device ||
                (addrdesc.mematttrs.inner.attrs == MemAttr_NC &&
                 addrdesc.mematttrs.outer.attrs == MemAttr_NC)) then
            addrdesc.mematttrs.shareability = Shareability_OSH;
        end;
        var ns : bit;
        if ss == SS_NonSecure then
            ns = ARBITRARY : bit;
        elsif addrdesc.paddress.paspace == PAS_Secure then
            ns = '0';
        else
            ns = '1';
        end;
        let sh : bits(2) =       let sh : bits(2) = (if addrdesc.mematttrs.shareability !=
ReportedPARShareabilityShareability_NSH(then '01'
                                 else '00');
        PAR().F      = '0';
        PAR().SS     = supersection;
        PAR().Outer  =PAREncodeShareability(addrdesc.mematttrs));
        PAR().F      = '0';
        PAR().SS     = supersection;
        PAR().Outer  = AArch32_ReportedOuterAttrs(AArch32_PAROuterAttrs(addrdesc.mematttrs));
        PAR().Inner  = AArch32_ReportedInnerAttrs(AArch32_PARInnerAttrs(addrdesc.mematttrs));
        PAR().SH     = ReportedPARShareability(sh);
        PAR()[8]     = ImpDefBit("Non-Faulting PAR");               // IMPDEF
        PAR().NS     = ns;
        PAR().NOS    = if addrdesc.mematttrs.shareability == Shareability_OSH(addrdesc.mematttrs));
        PAR().F0_32.SH     = sh[1];
        PAR()[8]     = ImpDefBit("Non-Faulting PAR");               // IMPDEF
        PAR().NS     = ns;
        PAR().NOS    = sh[0];
then '0' else '1';
        PAR().LPAE   = '0';
        PAR().F0_32.PA     = addrdesc.paddress.address[31:12];
        PAR().PA     = addrdesc.paddress.address[39:12];
    else
        PAR().F      = '1';
        PAR().FST    = AArch32_PARFaultStatusSD(addrdesc.fault);
        PAR().LPAE   = '0';
        PAR()[31:16] = ImpDefBits{16}("Faulting PAR");             // IMPDEF
    end;
    return;
end;
```

Library pseudocode for aarch32/at/GetPAR_F

```
// GetPAR_F()
// ==========
// Query the PAR.F field.

func GetPAR_F() => bit
begin
    return PAR().F;
end;
```

Library pseudocode for aarch32/at/GetPAR_LPAE

```
// GetPAR_LPAE()
// =============
// Query the PAR.LPAE field.

func GetPAR_LPAE() => bit
begin
    return PAR().LPAE;
end;
```

Library pseudocode for aarch32/functions/system/BPIALL

```
// BPIALL()
// ========
// Invalidate all entries from branch predictors.

impdef func BPIALL()
begin
    return;
end;
```

Library pseudocode for aarch32/functions/system/BPIALLIS

```
// BPIALLIS()
// ==========
// Invalidate all entries from branch predictors, Inner Shareable.

impdef func BPIALLIS()
begin
    return;
end;
```

Library pseudocode for aarch32/functions/system/BPIMVA

```
// BPIMVA()
// ========
// Invalidate the virtual address from the branch predictors.

impdef func BPIMVA(val : bits(32))
begin
    return;
end;
```

Library pseudocode for aarch32/functions/system/CP15DMB

```
// CP15DMB()
// =========
// Legacy AArch32 system instruction form of DMB SY.

func CP15DMB()
begin
    DataMemoryBarrier(MBReqTypes_All);
    return;
end;
```

Library pseudocode for aarch32/functions/system/CP15DSB

```
// CP15DSB()
// =========
// Legacy AArch32 system instruction form of DSB SY.

func CP15DSB()
begin
    DataSynchronizationBarrier(MBMaintenanceScope_OuterShareable, MBReqTypes_All, FALSE);
    return;
end;
```

Library pseudocode for aarch32/functions/system/CP15ISB

```
// CP15ISB()
// =========
// Legacy AArch32 system instruction form of ISB.

func CP15ISB()
begin
    InstructionSynchronizationBarrier();
    return;
end;
```

Library pseudocode for aarch32/translation/attrs/ImpDefMemoryAttributes

```
// ImpDefMemoryAttributes()
// ========================
// Return an IMPLEMENTATION DEFINED set of memory attributes identified by the string parameter.

impdef func ImpDefMemoryAttributes(x : string) => MemoryAttributes
begin
    return ARBITRARY : MemoryAttributes;
end;
```

Library pseudocode for aarch32/translation/translation/AArch32_EL2Enabled

```
// AArch32_EL2Enabled()
// ====================
// When using VMSAv8-32, returns whether EL2 is enabled for the given Security State.
// Returns whether EL2 is enabled for the given Security State

func AArch32_EL2Enabled(ss : SecurityState) => boolean
begin
    assert     if ss == ELUsingAArch32(EL1);
    if ss == SS_Secure then
        if !(HaveEL(EL2) && IsFeatureImplemented(FEAT_SEL2)) then
            return FALSE;
        elsif HaveEL(EL3) then
            return SCR_EL3().EEL2 == '1';
        else
            return SecureOnlyImplementation();
) then
            return SCR_EL3().EEL2 == '1';
        else
            return ImpDefBool("Secure-only implementation");
        end;
    else
        return HaveEL(EL2);
    end;
end;
```

Library pseudocode for aarch32/translation/translation/AArch32_S1DisabledOutput

```
// AArch32_S1DisabledOutput()
// ==========================
// Flat map the VA to IPA/PA, depending on the regime, assigning default memory attributes

func AArch32_S1DisabledOutput(fault_in : FaultRecord, regime : Regime,
                              va : bits(32), aligned : boolean,
                              accdesc : AccessDescriptor) => (FaultRecord, AddressDescriptor)
begin

    var fault : FaultRecord = fault_in;
    // No memory page is guarded when stage 1 address translation is disabled
    SetInGuardedPage(FALSE);

    var memattrs : MemoryAttributes;
    var default_cacheable : bit;
    if regime == Regime_EL10 && AArch32_EL2Enabled(accdesc.ss) then
        if ELStateUsingAArch32(EL2, accdesc.ss == SS_Secure) then
            default_cacheable = HCR().DC;
        else
            default_cacheable = if IsFeatureImplemented(FEAT_HCRMAC) then HCR_EL2().DC else '0';
            default_cacheable = HCR_EL2().DC;
        end;
    else
        default_cacheable = '0';
    end;

    if default_cacheable == '1' then
        // Use default cacheable settings
        memattrs.memtype      = MemType_Normal;
        memattrs.inner.attrs  = MemAttr_WB;
        memattrs.inner.hints  = MemHint_RWA;
        memattrs.outer.attrs  = MemAttr_WB;
        memattrs.outer.hints  = MemHint_RWA;
        memattrs.shareability = Shareability_NSH;
        memattrs.tags     = MemTag_Untagged;
        memattrs.xs           = '0';
    elsif accdesc.acctype == AccessType_IFETCH then
        memattrs.memtype      = MemType_Normal;
        memattrs.shareability = Shareability_OSH;
        memattrs.tags         = MemTag_Untagged;
        if AArch32_S1ICacheEnabled(regime) then
            memattrs.inner.attrs = MemAttr_WT;
            memattrs.inner.hints = MemHint_RA;
            memattrs.outer.attrs = MemAttr_WT;
            memattrs.outer.hints = MemHint_RA;
        else
            memattrs.inner.attrs = MemAttr_NC;
            memattrs.outer.attrs = MemAttr_NC;
        end;
        memattrs.xs           = '1';
    else
        // Treat memory region as Device
        memattrs.memtype      = MemType_Device;
        memattrs.device       = DeviceType_nGnRnE;
        memattrs.shareability = Shareability_OSH;
        memattrs.tags         = MemTag_Untagged;
        memattrs.xs           = '1';
    end;

    var ntlsmd : bit;
    if IsFeatureImplemented(FEAT_LSMAOC) then
        case regime of
            when Regime_EL30 => ntlsmd = SCTLR_S().nTLSMD;
            when Regime_EL2  => ntlsmd = HSCTLR().nTLSMD;
            when Regime_EL10 =>
                if HaveEL(EL3) && ELUsingAArch32(EL3) then
                    ntlsmd = SCTLR_NS().nTLSMD;
                else
                    ntlsmd = SCTLR().nTLSMD;
                end;
        end;
    else
        ntlsmd = '1';
    end;

    if AArch32_S1HasAlignmentFaultDueToMemType(accdesc, aligned, ntlsmd, memattrs) then
        fault.statuscode  = Fault_Alignment;
        return (fault, ARBITRARY : AddressDescriptor);
    elsif (accdesc.exclusive &&
            !(regime == Regime_EL10 && EL2Enabled() &&
              ((!ELUsingAArch32(EL2) && IsFeatureImplemented(FEAT_HCRMAC) && HCR_EL2().DC == '1') ||
) && HCR_EL2().DC == '1') ||
                  (ELUsingAArch32(EL2) && HCR().DC == '1'))) &&
            ConstrainUnpredictableBool(Unpredictable_Atomic_MMU_IMPDEF_FAULT)) then
        fault.statuscode = Fault_Exclusive;
    end;

    var oa : FullAddress;
    oa.address = ZeroExtend{NUM_PABITS}(va);
```

```
    oa.paspace = if accdesc.ss == SS_Secure then PAS_Secure else PAS_NonSecure;
    let ipa : AddressDescriptor = CreateAddressDescriptor(ZeroExtend{64}(va),
                                                          oa,
                                                          memattrs,
                                                          accdesc);

    return (fault, ipa);
end;
```

Library pseudocode for aarch64/debug/brbe/BRBEException

```
// BRBEException()
// ===============
// Called to write exception branch record when BRBE is active.

func BRBEException(erec : ExceptionRecord, source_valid : boolean,
                   source_address_in : bits(64),
                   target_address_in : bits(64), target_el : bits(2),
                   trappedsyscallinst : boolean)
begin
    var target_address : bits(64) = target_address_in;
    let except : Exception = erec.exceptype;
    let iss : bits(25)     = erec.syndrome.iss;
    case target_el of
        when EL3 =>
            if !IsFeatureImplemented(FEAT_BRBEv1p1) || (MDCR_EL3().E3BREC == MDCR_EL3().E3BREW) then
                return;
            end;
        when EL2 => if BRBCR_EL2().EXCEPTION == '0' then return; end;
        when EL1 => if BRBCR_EL1().EXCEPTION == '0' then return; end;
    end;

    let target_valid : boolean = BranchRecordAllowed(target_el);

    if source_valid || target_valid then
        var branch_type : bits(6);
        case except of
            when Exception_Uncategorized         => branch_type = '100011'; // Trap
            when Exception_WFxTrap               => branch_type = '100011'; // Trap
            when Exception_CP15RTTrap            => branch_type = '100011'; // Trap
            when Exception_CP15RRTTrap           => branch_type = '100011'; // Trap
            when Exception_CP14RTTrap            => branch_type = '100011'; // Trap
            when Exception_CP14DTTrap            => branch_type = '100011'; // Trap
            when Exception_AdvSIMDFPAccessTrap   => branch_type = '100011'; // Trap
            when Exception_FPIDTrap              => branch_type = '100011'; // Trap
            when Exception_PACTrap               => branch_type = '100011'; // Trap
            when Exception_CP14RRTTrap           => branch_type = '100011'; // Trap
            when Exception_OtherTrap             => branch_type = '100011'; // Trap
            when Exception_BranchTarget          => branch_type = '101011'; // Inst Fault
            when Exception_IllegalState          => branch_type = '100011'; // Trap
            when Exception_SupervisorCall        =>
                if !trappedsyscallinst then      branch_type = '100010';    // Call
                else                             branch_type = '100011';    // Trap
                end;
            when Exception_HypervisorCall        => branch_type = '100010'; // Call
            when Exception_MonitorCall           =>
                if !trappedsyscallinst then      branch_type = '100010';    // Call
                else                             branch_type = '100011';    // Trap
                end;
            when Exception_SystemRegisterTrap    => branch_type = '100011'; // Trap
            when Exception_SystemRegister128Trap => branch_type = '100011'; // Trap
            when Exception_SVEAccessTrap         => branch_type = '100011'; // Trap
            when Exception_SMEAccessTrap         => branch_type = '100011'; // Trap
            when Exception_ERetTrap              => branch_type = '100011'; // Trap
            when Exception_PACFail               => branch_type = '101100'; // Data Fault
            when Exception_InstructionAbort      => branch_type = '101011'; // Inst Fault
            when Exception_PCAlignment           => branch_type = '101010'; // Alignment
            when Exception_DataAbort             => branch_type = '101100'; // Data Fault
            when Exception_NV2DataAbort          => branch_type = '101100'; // Data Fault
            when Exception_SPAlignment           => branch_type = '101010'; // Alignment
            when Exception_FPTrappedException    => branch_type = '100011'; // Trap
            when Exception_SError                => branch_type = '100100'; // System Error
            when Exception_Breakpoint            => branch_type = '100110'; // Inst debug
            when Exception_SoftwareStep          => branch_type = '100110'; // Inst debug
            when Exception_Watchpoint            => branch_type = '100111'; // Data debug
            when Exception_NV2Watchpoint         => branch_type = '100111'; // Data debug
            when Exception_SoftwareBreakpoint    => branch_type = '100110'; // Inst debug
            when Exception_IRQ                   => branch_type = '101110'; // IRQ
            when Exception_FIQ                   => branch_type = '101111'; // FIQ
            when Exception_MemCpyMemSet          => branch_type = '100011'; // Trap
            when Exception_GCSFail               =>
                if iss[23:20] == '0000' then        branch_type = '101100'; // Data Fault
                elsif iss[23:20] == '0001' then     branch_type = '101011'; // Inst Fault
                elsif iss[23:20] == '0010' then     branch_type = '100011'; // Trap
                else                                unreachable;
                end;
            when Exception_Profiling             => branch_type = '100110'; // Inst debug
            when Exception_GPC                   =>
                if iss[20] == '1' then              branch_type = '101011'; // Inst fault
                else                                branch_type = '101100'; // Data fault
                end;
            when otherwise                       => unreachable;
        end;

        var ccu : bit;
        var cc : bits(14);
        (ccu, cc) = Exception_ConditionalFault       => branch_type = '100110'; // Inst debug
            otherwise                                => unreachable;
        end;
```

```
        var ccu : bit;
        var cc : bits(14);
        (ccu, cc) = BranchEncCycleCount();
        let el : bits(2) = if target_valid then target_el else '00';
        let mispredict : bit = '0';
        let sv : bit = if source_valid then '1' else '0';
        let tv : bit = if target_valid then '1' else '0';
        let source_address : bits(64) = if source_valid then source_address_in else Zeros{64};

        if !target_valid then
            target_address = Zeros{64};
        else
            target_address = AArch64_BranchAddr(target_address, target_el);
        end;

        UpdateBranchRecordBuffer(ccu, cc, branch_type, el, mispredict,
                                 sv::tv, source_address, target_address);

        PMUEvent(PMU_EVENT_BRB_FILTRATE);
    end;

    return;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPEAddressEnabled

```
// SPEAddressEnabled()
// ===================
// Checks if an address packet is enabled and its data can be recorded.

func SPEAddressEnabled(address_index : integer) => boolean
begin
    var address_packet_enabled: boolean;
    let have_pbtfilt : boolean = IsFeatureImplemented(FEAT_SPE_PBTFILT);

    case address_index of
        when SPEAddrPosPrevBranchTarget =>
            address_packet_enabled = !have_pbtfilt || PMSFCR_EL1().nPBT == '0';
        otherwise =>
            address_packet_enabled = TRUE;
    end;

    return address_packet_enabled;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPEConstructRecord

```
// SPEConstructRecord()
// ====================
// Create new record and populate it with packets using sample storage data.
// This is an example implementation, packets may appear in
// any order as long as the record ends with an End or Timestamp packet.

func SPEConstructRecord()
begin
    // Empty the record.
    SPEEmptyRecord();

    var op_class : bits(2);
    var op_subclass : bits(8);
    // Add operation details
    (op_class, op_subclass) = SPEConstructClass();
    SPEAddPacketToRecord{8}('01', '10'::op_class, op_subclass);

    // Add contextEL1 if available
    if SPESampleContextEL1Valid then
        SPEAddPacketToRecord{32}('01', '0100', SPESampleContextEL1);
    end;

     // Add contextEL2 if available
    if SPESampleContextEL2Valid then
        SPEAddPacketToRecord{32}('01', '0101', SPESampleContextEL2);
    end;

    // Add valid counters
    for counter_index = 0 to (SPEMaxCounters - 1) do
        if SPESampleCounterValid[[counter_index]] then
            if counter_index >= 8 then
                // Need extended format
                SPEAddByteToRecord('001000'::counter_index[4:3]);
            end;
            // Check for overflow
            let large_counters : boolean = ImpDefBool("SPE 16bit counters");
            if large_counters then
                if SPESampleCounter[[counter_index]] > 0xFFFF then
                    SPESampleCounter[[counter_index]] = 0xFFFF;
                end;
            else
                if SPESampleCounter[[counter_index]] > 0xFFF then
                    SPESampleCounter[[counter_index]] = 0xFFF;
                end;
            end;

            // Add byte0 for short format (byte1 for extended format)
            SPEAddPacketToRecord{16}('10', '1'::counter_index[2:0],
                                     SPESampleCounter[[counter_index]][15:0]);
        end;
    end;

    // Add valid addresses
    if IsFeatureImplemented(FEAT_SPE_PBT) then
        // Under some conditions, it is IMPLEMENTATION_DEFINED whether the
        // previous branch packet is present.
        let include_prev_br : boolean = ImpDefBool("SPE get prev br if not br");
        if SPESampleOpAttr.op_type != SPEOpType_Branch && !include_prev_br then
            SPESampleAddressValid[[SPEAddrPosPrevBranchTarget]] = FALSE;
        end;
    end;

    // Data Virtual address should not be collected if this was an NV2 access and Statistical
    // Profiling is disabled at EL2.
    if !StatisticalProfilingEnabled(EL2) && SPESampleOpAttr.ldst_type == SPELDSTType_NV2 then
        SPESampleAddressValid[[SPEAddrPosDataVirtual]] = FALSE;
    end;

    for address_index = 0 to (SPEMaxAddrs - 1) do
        if SPESampleAddressValid[[address_index]] &&[[address_index]] then
            if address_index >= 8 then
                // Need extended format SPEAddressEnabled(address_index) then
            if address_index >= 8 then
                // Need extended format
                SPEAddByteToRecord('001000'::address_index[4:3]);
            end;
            SPEAddPacketToRecord{64}('10', '0'::address_index[2:0],
                                     SPESampleAddress[[address_index]]);
        end;
    end;

    // Add Data Source
    if SPESampleDataSourceValid then
        let ds_payload_size : integer{} = SPEGetDataSourcePayloadSize();
        SPEAddPacketToRecord{8 * ds_payload_size}('01', '0011',
                                                  SPESampleDataSource[8*ds_payload_size-1:0]);
    end;

    // Add events
```

```
    // Get size of payload in bytes.
    let payload_size : integer{} = SPEGetEventsPayloadSize();
    SPEAddPacketToRecord{8 * payload_size}('01', '0010', SPESampleEvents[8*payload_size-1:0]);

    // Add Timestamp to end the record if one is available.
    // Otherwise end with an End packet.
    if SPESampleTimestampValid then
        SPEAddPacketToRecord{64}('01', '0001', SPESampleTimestamp);
    else
        SPEAddByteToRecord('00000001');
    end;

    // Add padding
    while SPERecordSize MOD (1<<UInt(PMBIDR_EL1().Align)) != 0 looplimit 2048 do
        SPEAddByteToRecord(Zeros{8});
    end;
    SPEWriteToBuffer();
    CTI_SignalEvent(CrossTriggerIn_SPESample);
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPEMultiAccessSample

```
// SPEMultiAccessSample()
// ======================
// Called by instructions which make at least one store and one load access, where the configuration
// of the operation type filter affects which access is sampled.

func SPEMultiAccessSample() => boolean
begin
    // If loads or stores are filtered out, the other should be recorded.
    // If neither or both are filtered out, pick one in an unbiased way.

    // Bit positions in the PMSFCR_EL1().[TYPE, TYPEm] fields.
    let LD : integer = 1;
    let ST : integer = 2;

    // Are loads allowed by filter?
    let loads_pass_filter : boolean  = PMSFCR_EL1().FT == '1' && PMSFCR_EL1().TYPE.LD == '1';
    let loads_pass_filter : boolean  = PMSFCR_EL1().FT == '1' && PMSFCR_EL1().TYPE[LD] == '1';
    // Are stores allowed by filter?
    let stores_pass_filter : boolean = PMSFCR_EL1().FT == '1' && PMSFCR_EL1().TYPE.ST == '1';
    let stores_pass_filter : boolean = PMSFCR_EL1().FT == '1' && PMSFCR_EL1().TYPE[ST] == '1';

    var record_load : boolean;
    if loads_pass_filter && !stores_pass_filter then
        // Only loads pass filter
        record_load = TRUE;
    elsif !loads_pass_filter && stores_pass_filter then
        // Only stores pass filter
        record_load = FALSE;
    else
        // Both loads and stores pass the filter or neither pass the filter.
        // Pick between the load or the store access (pseudo-)randomly.
        record_load = SPEGetRandomBoolean();
    end;
    return record_load;
end;
```

Library pseudocode for aarch64/exceptions/aborts/AArch64_AbortSyndrome

```
// AArch64_AbortSyndrome()
// =======================
// Creates an exception syndrome record for Abort and Watchpoint exceptions
// from an AArch64 translation regime.

func AArch64_AbortSyndrome(exceptype : Exception, fault : FaultRecord,
                           target_el : bits(2)) => ExceptionRecord
begin
    var except : ExceptionRecord = ExceptionSyndrome(exceptype);

    except.syndrome = AArch64_FaultSyndrome(exceptype, fault, target_el);
    if exceptype IN {Exception_NV2Watchpoint, Exception_Watchpoint} then
        except.vaddress = fault.watchptinfo.vaddress;
    elsif fault.statuscode == Fault_TagCheck then
        if IsFeatureImplemented(FEAT_MTE_TAGGED_FAR) then
            except.vaddress = ZeroExtend{64}(fault.vaddress);
        else
            except.vaddress = ARBITRARY : bits(4) :: fault.vaddress[59:0];
        end;
    else
        except.vaddress = ZeroExtend{64}(fault.vaddress);
    end;

    if IPAValid(fault) then
        except.ipavalid = TRUE;
        except.NS = if fault.ipaddress.paspace == PAS_NonSecure then '1' else '0';
        except.ipaddress = fault.ipaddress.address;
    else
        except.ipavalid = FALSE;
    end;

    if except.syndrome.iss[24,14] == '01' then
    if except.syndrome.iss[14] == '1' then
        except.pavalid  = TRUE;
        except.paddress = fault.paddress;
    else
        except.pavalid  = FALSE;
        except.paddress = ARBITRARY : FullAddress;
    end;

    return except;
end;
```

Library pseudocode for aarch64/exceptions/aborts/BranchTargetException

```
// BranchTargetException()
// =======================
// Generate branch target exception.
// Raise branch target exception.

func AArch64_BranchTargetException(vaddress : bits(52))
begin
    let preferred_exception_return : bits(64) = ThisInstrAddr{}();
    let vect_offset : integer = 0x0;

    var except : ExceptionRecord = ExceptionSyndrome(Exception_BranchTarget);
    except.syndrome.iss[1:0]   = PSTATE.BTYPE;
    except.syndrome.iss[24:2]  = Zeros{23};         // RES0

    var target_el : bits(2) = EL1;
    if UInt(PSTATE.EL) > UInt(EL1) then
        target_el = PSTATE.EL;
    elsif PSTATE.EL == EL0 && EL2Enabled() && HCR_EL2().TGE == '1' then
        target_el = EL2;
    end;
    AArch64_TakeException(target_el, except, preferred_exception_return, vect_offset);
end;
```

Library pseudocode for aarch64/exceptions/exceptions/AArch64_ExceptionClass

```
// AArch64_ExceptionClass()
// ========================
// Returns the Exception Class and Instruction Length fields to be reported in ESR

func AArch64_ExceptionClass(exceptype : Exception, target_el : bits(2)) => (integer,bit)
begin

    var il_is_valid : boolean = TRUE;
    let from_32 : boolean = UsingAArch32();
    var ec : integer;
    case exceptype of
        when Exception_Uncategorized         =>  ec = 0x00; il_is_valid = FALSE;
        when Exception_WFxTrap               =>  ec = 0x01;
        when Exception_CP15RTTrap            =>  ec = 0x03; assert from_32;
        when Exception_CP15RRTTrap           =>  ec = 0x04; assert from_32;
        when Exception_CP14RTTrap            =>  ec = 0x05; assert from_32;
        when Exception_CP14DTTrap            =>  ec = 0x06; assert from_32;
        when Exception_AdvSIMDFPAccessTrap   =>  ec = 0x07;
        when Exception_FPIDTrap              =>  ec = 0x08;
        when Exception_PACTrap               =>  ec = 0x09;
        when Exception_OtherTrap             =>  ec = 0x0A;
        when Exception_GPC                   =>  ec = 0x1E;
        when Exception_ConditionalFault      =>  ec = 0x0B;
        when Exception_CP14RRTTrap           =>  ec = 0x0C; assert from_32;
        when Exception_BranchTarget          =>  ec = 0x0D;
        when Exception_IllegalState          =>  ec = 0x0E; il_is_valid = FALSE;
        when Exception_SupervisorCall        =>  ec = if from_32 then 0x11 else 0x15;
        when Exception_HypervisorCall        =>  ec = if from_32 then 0x12 else 0x16;
        when Exception_MonitorCall           =>  ec = if from_32 then 0x13 else 0x17;
        when Exception_SystemRegisterTrap    =>  ec = 0x18; assert !from_32;
        when Exception_SystemRegister128Trap =>  ec = 0x14; assert !from_32;
        when Exception_SVEAccessTrap         =>  ec = 0x19; assert !from_32;
        when Exception_ERetTrap              =>  ec = 0x1A; assert !from_32;
        when Exception_PACFail               =>  ec = 0x1C; assert !from_32;
        when Exception_SMEAccessTrap         =>  ec = 0x1D; assert !from_32;
        when Exception_InstructionAbort      =>
            ec = if target_el == PSTATE.EL then 0x21 else 0x20;
            il_is_valid = FALSE;
        when Exception_PCAlignment           =>  ec = 0x22; il_is_valid = FALSE;
        when Exception_DataAbort             =>  ec = if target_el == PSTATE.EL then 0x25 else 0x24;
        when Exception_NV2DataAbort          =>  ec = 0x25;
        when Exception_SPAlignment           =>  ec = 0x26; il_is_valid = FALSE; assert !from_32;
        when Exception_MemCpyMemSet          =>  ec = 0x27;
        when Exception_GCSFail               =>  ec = 0x2D; assert !from_32;
        when Exception_FPTrappedException    =>  ec = if from_32 then 0x28 else 0x2C;
        when Exception_SError                =>  ec = 0x2F; il_is_valid = FALSE;
        when Exception_Breakpoint =>
            ec = if target_el == PSTATE.EL then 0x31 else 0x30;
            il_is_valid = FALSE;
        when Exception_SoftwareStep =>
            ec = if target_el == PSTATE.EL then 0x33 else 0x32;
            il_is_valid = FALSE;
        when Exception_Watchpoint =>
            ec = if target_el == PSTATE.EL then 0x35 else 0x34;
            il_is_valid = FALSE;
        when Exception_NV2Watchpoint         =>  ec = 0x35; il_is_valid = FALSE;
        when Exception_SoftwareBreakpoint    =>  ec = if from_32 then 0x38 else 0x3C;
        when Exception_VectorCatch           =>  ec = 0x3A; il_is_valid = FALSE; assert from_32;
        when Exception_Profiling             =>  ec = 0x3D;
        otherwise                            =>  unreachable;
    end;

    var il : bit;
    if il_is_valid then
        il = if ThisInstrLength() == 32 then '1' else '0';
    else
        il = '1';
    end;
    assert from_32 || il == '1';            // AArch64 instructions always 32-bit

    return (ec,il);
end;
```

Library pseudocode for aarch64/exceptions/takeexception/AArch64_TakeException

```
// AArch64_TakeException()
// =======================
// Take an exception to an Exception level using AArch64.

noreturn func AArch64_TakeException(target_el : bits(2), exception_in : ExceptionRecord,
                                    preferred_exception_return : bits(64), vect_offset_in : integer)
begin
    assert HaveEL(target_el) && !ELUsingAArch32(target_el) && UInt(target_el) >= UInt(PSTATE.EL);
    if Halted() then
        AArch64_TakeExceptionInDebugState(target_el, exception_in);
    end;
    var except : ExceptionRecord = exception_in;
    var sync_errors : boolean;
    var iesb_req : boolean;
    if IsFeatureImplemented(FEAT_IESB) then
        sync_errors = EffectiveIESB(target_el) == '1' && except.exceptype != Exception_SError;
        if IsFeatureImplemented(FEAT_DoubleFault) then
            sync_errors = sync_errors || (SCR_EL3().[EA,NMEA] == '11' && target_el == EL3);
        end;
        if sync_errors && InsertIESBBeforeException(target_el) then
            SynchronizeErrors();
            if except.exceptype != Exception_SError(except.exceptype, target_el) then
            SynchronizeErrors();
            iesb_req = FALSE;
            sync_errors = FALSE;
            TakeUnmaskedPhysicalSErrorInterrupts(iesb_req);
then
                iesb_req = FALSE;
                sync_errors = FALSE;
                TakeUnmaskedPhysicalSErrorInterrupts(iesb_req);
            end;
        end;
    else
        sync_errors = FALSE;
    end;

    var brbe_source_allowed : boolean = FALSE;
    var brbe_source_address : bits(64) = Zeros{};
    if IsFeatureImplemented(FEAT_BRBE) then
        brbe_source_allowed = BranchRecordAllowed(PSTATE.EL);
        brbe_source_address = preferred_exception_return;
    end;

    if !IsFeatureImplemented(FEAT_ExS) || SCTLR_EL(target_el).EIS == '1' then
        // Synchronize the context, including Instruction Fetch Barrier effect
        SynchronizeContext();
    elsif !(except.exceptype == Exception_SoftwareBreakpoint ||
            (except.exceptype IN {Exception_SupervisorCall,
                                  Exception_HypervisorCall,
                                  Exception_MonitorCall} &&
             !except.trappedsyscallinst)) then
        InstructionFetchBarrier();
    end;

    // If coming from AArch32 state, the top parts of the X() registers might be set to zero
    let from_32 : boolean = UsingAArch32();
    if from_32 then AArch64_MaybeZeroRegisterUppers(); end;
    if from_32 && IsFeatureImplemented(FEAT_SME) && PSTATE.SM == '1' then
        ResetSVEState();
    else
        MaybeZeroSVEUppers(target_el);
    end;

    var vect_offset : integer = vect_offset_in;
    if UInt(target_el) > UInt(PSTATE.EL) then
        var lower_32 : boolean;
        if target_el == EL3 then
            if EL2Enabled() then
                lower_32 = ELUsingAArch32(EL2);
            else
                lower_32 = ELUsingAArch32(EL1);
            end;
        elsif IsInHost() && PSTATE.EL == EL0 && target_el == EL2 then
            lower_32 = ELUsingAArch32(EL0);
        else
            lower_32 = ELUsingAArch32(target_el - 1);
        end;
        vect_offset = vect_offset + (if lower_32 then 0x600 else 0x400);

    elsif PSTATE.SP == '1' then
        vect_offset = vect_offset + 0x200;
    end;

    assert vect_offset < 2048;
    let vect_base : bits(64) = VBAR_EL(target_el)[63:11]::Zeros{11};
    let target_vector : bits(64) = vect_base + vect_offset;

    var spsr : bits(64) = GetPSRFromPSTATE{}(AArch64_NonDebugState);
```

```
if PSTATE.EL == EL1 && target_el == EL1 && EL2Enabled() then
    if EffectiveHCR_EL2_NVx() IN {'x01', '111'} then
        spsr[3:2] = '10';
    end;
end;

if IsFeatureImplemented(FEAT_BTI) && !UsingAArch32() then
    var zero_btype : boolean;
    // SPSR_ELx[].BTYPE is only guaranteed valid for these exception types
    if except.exceptype IN {Exception_SError, Exception_IRQ, Exception_FIQ,
                            Exception_SoftwareStep, Exception_PCAlignment,
                            Exception_InstructionAbort, Exception_Breakpoint,
                            Exception_VectorCatch, Exception_SoftwareBreakpoint,
                            Exception_IllegalState, Exception_BranchTarget} then
        zero_btype = FALSE;
    else
        zero_btype = ConstrainUnpredictableBool(Unpredictable_ZEROBTYPE);
    end;
    if zero_btype then spsr[11:10] = '00'; end;
end;

if (IsFeatureImplemented(FEAT_NV2) &&
      except.exceptype == Exception_NV2DataAbort && target_el == EL3) then
    // External aborts are configured to be taken to EL3
    except.exceptype = Exception_DataAbort;
end;
if ! except.exceptype IN {Exception_IRQ, Exception_FIQ} then
    AArch64_ReportException(except, target_el);
end;

if IsFeatureImplemented(FEAT_BRBE) then
    BRBEException(except, brbe_source_allowed, brbe_source_address,
                  target_vector, target_el, except.trappedsyscallinst);
end;

if IsFeatureImplemented(FEAT_GCS) then
    if PSTATE.EL == target_el then
        if GetCurrentEXLOCKEN() then
            PSTATE.EXLOCK = '1';
        else
            PSTATE.EXLOCK = '0';
        end;
    else
        PSTATE.EXLOCK = '0';
    end;
end;

PSTATE.EL = target_el;
PSTATE.nRW = '0';
PSTATE.SP = '1';

SPSR_ELx() = spsr;
ELR_ELx() = preferred_exception_return;
PSTATE.SS = '0';
if IsFeatureImplemented(FEAT_SSBS) then PSTATE.SSBS = SCTLR_ELx().DSSBS; end;
if (IsFeatureImplemented(FEAT_EBEP) || IsFeatureImplemented(FEAT_SPE_EXC) ||
      IsFeatureImplemented(FEAT_TRBE_EXC)) then
    PSTATE.PM = '1';
end;
if IsFeatureImplemented(FEAT_NMI) then
    PSTATE.ALLINT = NOT SCTLR_ELx().SPINTMASK;
end;
PSTATE.[D,A,I,F] = '1111';
if IsFeatureImplemented(FEAT_MTE) then PSTATE.TCO = '1'; end;
PSTATE.IL = '0';
if IsFeatureImplemented(FEAT_UAO) then PSTATE.UAO = '0'; end;
if IsFeatureImplemented(FEAT_UINJ) then PSTATE.UINJ = '0'; end;
if IsFeatureImplemented(FEAT_PAuth_LR) then PSTATE.PACM = '0'; end;
if IsFeatureImplemented(FEAT_PAN) && SCTLR_ELx().SPAN == '0' then
    if PSTATE.EL == EL1 then
        PSTATE.PAN = '1';
    elsif PSTATE.EL == EL2 && ELIsInHost(EL0) then
        PSTATE.PAN = '1';
    elsif (PSTATE.EL == EL2 && ELIsInHost(EL2) &&
            (IsFeatureImplemented(FEAT_NV3) ||
             ImpDefBool("Set PAN as 1 when E2H,TGE is '10'"))) then
        PSTATE.PAN = '1';
    end;
end;
if from_32 then                             // Coming from AArch32
    PSTATE.IT = '00000000';
    PSTATE.T = '0';                         // PSTATE.J is RES0
end;
if IsFeatureImplemented(FEAT_BTI) then PSTATE.BTYPE = '00'; end;
let branch_conditional : boolean = FALSE;
BranchTo{64}(target_vector, BranchType_EXCEPTION, branch_conditional);
CheckExceptionCatch(TRUE);                  // Check for debug event on exception entry
```

```
    if sync_errors then
        SynchronizeErrors();
        iesb_req = TRUE;
        TakeUnmaskedPhysicalSErrorInterrupts(iesb_req);
    end;

    EndOfInstruction(EndOfInstructionReason_Exception);
end;
```

Library pseudocode for aarch64/exceptions/traps/CheckLDST64BEnabled

```
// CheckLDST64BEnabled()
// =====================
// Checks for trap on ST64B, LD64B, STL64B, and LDA64B instructions
// Checks for trap on ST64B and LD64B instructions

func CheckLDST64BEnabled(acqrel : boolean)
CheckLDST64BEnabled()
begin
    var trap : boolean = FALSE;
    let iss : bits(25) = if acqrel then 0x8[24:0] else 0x2[24:0];
    let iss : bits(25) = 0x2[24:0];
    var target_el : bits(2);

    if PSTATE.EL == EL0 then
        if !IsInHost() then
            trap = SCTLR_EL1().EnALS == '0';
            target_el = if EL2Enabled() && HCR_EL2().TGE == '1' then EL2 else EL1;
        else
            trap = SCTLR_EL2().EnALS == '0';
            target_el = EL2;
        end;
    else
        target_el = EL1;
    end;

    if !trap &&    if (!trap && EL2Enabled() && ((PSTATE.EL == EL0 && !IsInHost()) || PSTATE.EL == EL1) then
)) then
        trap = !IsHCRXEL2Enabled() || HCRX_EL2().EnALS == '0';
        target_el = EL2;
    end;

    if trap then AArch64_OtherInstrTrap(target_el, iss); end;
end;
```

Library pseudocode for aarch64/exceptions/traps/CheckST64BV0Enabled

```
// CheckST64BV0Enabled()
// ====================
// Checks for trap on ST64BV0 and STL64BV0 instructions
// Checks for trap on ST64BV0 instruction

func CheckST64BV0Enabled(acqrel : boolean)
CheckST64BV0Enabled()
begin
    var trap : boolean = FALSE;
    let iss : bits(25) = if acqrel then 0x7[24:0] else 0x1[24:0];
    let iss : bits(25) = 0x1[24:0];
    var target_el : bits(2);

    if    if ( PSTATE.EL != EL3 && HaveEL(EL3) && SCR_EL3().EnAS0 == '0' && EL3SDDUndefPriority() then
()) then
        Undefined();
    end;

    if PSTATE.EL == EL0 then
        if !IsInHost() then
            trap = SCTLR_EL1().EnAS0 == '0';
            target_el = if EL2Enabled() && HCR_EL2().TGE == '1' then EL2 else EL1;
        else
            trap = SCTLR_EL2().EnAS0 == '0';
            target_el = EL2;
        end;
    end;

    if !trap &&    if (!trap && EL2Enabled() && ((PSTATE.EL == EL0 && !IsInHost()) || PSTATE.EL == EL1) then
)) then
        trap = !IsHCRXEL2Enabled() || HCRX_EL2().EnAS0 == '0';
        target_el = EL2;
    end;

    if !trap && PSTATE.EL != EL3 then
        trap = HaveEL(EL3) && SCR_EL3().EnAS0 == '0';
        target_el = EL3;
    end;

    if trap then
        if target_el == EL3 && EL3SDDUndef() then
            Undefined();
        else
            AArch64_OtherInstrTrap(target_el, iss);
        end;
    end;
end;
```

Library pseudocode for aarch64/exceptions/traps/CheckST64BVEnabled

```
// CheckST64BVEnabled()
// ====================
// Checks for trap on ST64BV and STL64BV instructions
// Checks for trap on ST64BV instruction

func CheckST64BVEnabled(acqrel : boolean)
CheckST64BVEnabled()
begin
    var trap : boolean = FALSE;
    let iss : bits(25) = if acqrel then 0x6[24:0] else 0x0[24:0];
    let iss : bits(25) = 0x0[24:0];
    var target_el : bits(2);

    if PSTATE.EL == EL0 then
        if !IsInHost() then
            trap = SCTLR_EL1().EnASR == '0';
            target_el = if EL2Enabled() && HCR_EL2().TGE == '1' then EL2 else EL1;
        else
            trap = SCTLR_EL2().EnASR == '0';
            target_el = EL2;
        end;
    end;

    if !trap &&    if (!trap && EL2Enabled() && ((PSTATE.EL == EL0 && !IsInHost()) || PSTATE.EL == EL1) then
)) then
        trap = !IsHCRXEL2Enabled() || HCRX_EL2().EnASR == '0';
        target_el = EL2;
    end;

    if trap then AArch64_OtherInstrTrap(target_el, iss); end;
end;
```

Library pseudocode for aarch64/functions/aborts/AArch64_FaultSyndrome

```
// AArch64_FaultSyndrome()
// =======================
// Creates an exception syndrome value and updates the virtual address for Abort and Watchpoint
// exceptions taken to an Exception level using AArch64.

func AArch64_FaultSyndrome(exceptype : Exception, fault : FaultRecord,
                           target_el : bits(2)) => IssType
begin
    assert fault.statuscode != Fault_None;

    var isstype : IssType;
    isstype.iss  = Zeros{25};
    isstype.iss2 = Zeros{24};

    let d_side : boolean = exceptype IN {Exception_DataAbort, Exception_NV2DataAbort,
                                         Exception_Watchpoint, Exception_NV2Watchpoint};
    if IsFeatureImplemented(FEAT_RAS) && fault.statuscode == Fault_SyncExternal then
        let errstate : ErrorState = PEErrorState(fault);
        isstype.iss[12:11] = AArch64_EncodeSyncErrorSyndrome(errstate);  // SET
    end;

    if d_side then
        if (IsFeatureImplemented(FEAT_EAESR) && exceptype == Exception_DataAbort &&
              !fault.s2fs1walk && ! fault.accessdesc.acctype IN {AccessType_DC,
                                                                 AccessType_IC,
                                                                 AccessType_AT,
                                                                 AccessType_NV2}) then
            // LST2 field
            if fault.accessdesc.atomicop then
                isstype.iss2[15:13] = '001';    // Data Abort is from an Atomic instruction
            elsif (fault.accessdesc.acctype == AccessType_DCZero &&
                     fault.accessdesc.cachetype IN {CacheType_Data, CacheType_Data_Tag}) then
                isstype.iss2[15:13] = '010';    // Data Abort is from a DC ZVA or DC GZVA
            end;

            // Unpriv field
            if (fault.accessdesc.el == EL0 && PSTATE.EL != EL0 &&
                  fault.accessdesc.acctype IN {AccessType_GPR,
                                               AccessType_GCS,
                                               AccessType_ASIMD}) then
                isstype.iss2[12] = '1';         // Data Abort is from an unprivileged load or store
            end;
        end;

        if fault.accessdesc.acctype == AccessType_GCS then
            isstype.iss2[8] = '1';
        end;
        if exceptype IN {Exception_Watchpoint, Exception_NV2Watchpoint} then
            isstype.iss[23:0] = WatchpointRelatedSyndrome(fault);
        end;

        if AArch64_InstructionSyndromeValid(fault) then
            if fault.accessdesc.ls64 then
                (isstype.iss2, isstype.iss[24:14]) = LS64InstructionSyndrome();
            else
                isstype.iss[24:14] = LSInstructionSyndrome();
            end;
        end;

        if IsFeatureImplemented(FEAT_NV2) && fault.accessdesc.acctype == AccessType_NV2 then
            isstype.iss[13] = '1';   // Fault is generated by use of VNCR_EL2
        end;

        if (IsFeatureImplemented(FEAT_LS64) &&
              fault.statuscode IN {Fault_AccessFlag, Fault_Translation, Fault_Permission}) then
            isstype.iss[12:11] = GetLoadStoreType();
        end;

        if fault.accessdesc.acctype IN {AccessType_DC, AccessType_IC, AccessType_AT} then
            isstype.iss[8] = '1';
        end;

        if fault.accessdesc.acctype IN {AccessType_DC, AccessType_IC, AccessType_AT} then
            isstype.iss[6] = '1';
        elsif fault.statuscode IN {Fault_UnsupportedAtomicHWUpdate, Fault_Exclusive} then
            isstype.iss[6] = ARBITRARY : bit;
        elsif fault.accessdesc.atomicop && IsExternalAbort(fault) then
            isstype.iss[6] = ARBITRARY : bit;
        else
            isstype.iss[6] = if fault.write then '1' else '0';
        end;
        isstype.iss2[9] = if fault.tagaccess then '1' else '0';
        if exceptype IN {        if fault.statuscode ==Exception_DataAbortFault_Permission,then
            isstype.iss2[5] = if fault.dirtybit then '1' else '0';
            isstype.iss2[6] = if fault.overlay then '1' else '0';
            if isstype.iss[24] == '0' then
                isstype.iss[21] = if fault.toplevel then '1' else '0';
            end;
            isstype.iss2[7] = if fault.assuredonly then '1' else '0';
```

```
            isstype.iss2[10] = if fault.s1tagnotdata then '1' else '0';
        end;

    else
        if (fault.accessdesc.acctype == Exception_NV2DataAbortAccessType_IFETCH} then
            isstype.iss2[5] = if fault.dirtybit then '1' else '0';
            isstype.iss2[6] = if fault.overlay then '1' else '0';
            if isstype.iss[24] == '0' then
                isstype.iss[21] = if fault.toplevel then '1' else '0';
            end;
            isstype.iss2[7] = if fault.assuredonly then '1' else '0';
            isstype.iss2[10] = if fault.s1tagnotdata then '1' else '0';
        end;

    else
        if fault.accessdesc.acctype ==&&
            fault.statuscode == AccessType_IFETCHFault_Permission then
) then
            isstype.iss2[5] = if fault.dirtybit then '1' else '0';
            isstype.iss[21] = if fault.toplevel then '1' else '0';
            isstype.iss2[7] = if fault.assuredonly then '1' else '0';
            isstype.iss2[6] = if fault.overlay then '1' else '0';
        end;
    end;
    if fault.hdbssf then
        isstype.iss2[11] = '1';
    end;

    if (IsFeatureImplemented(FEAT_PFAR) && IsExternalSyncAbort(fault) &&
          !(EL2Enabled() &&
            (HCR_EL2().VM == '1' || (IsFeatureImplemented(FEAT_HCRMAC) && HCR_EL2().DC == '1')) &&
            target_el ==() && (HCR_EL2().VM == '1' || HCR_EL2().DC == '1') && target_el == EL1)) then
        isstype.iss[14] = if IsPFARValid(fault) then '1' else '0';
    end;

    if IsExternalAbort(fault) then isstype.iss[9] = fault.extflag; end;
    isstype.iss[7] = if fault.s2fs1walk then '1' else '0';
    isstype.iss[5:0] = EncodeLDFSC(fault.statuscode, fault.level);

    return isstype;
end;
```

Library pseudocode for aarch64/functions/cflt/AArch64_ConditionalFault

```
// AArch64_ConditionalFault()
// ==========================
// Generate a Conditional fault.

noreturn func AArch64_ConditionalFault(isflt : boolean, immediate : bits(16))
begin
    assert !UsingAArch32();
    var target_el : bits(2);
    if PSTATE.EL == EL0 then
        target_el = if EffectiveTGE() == '1' then EL2 else EL1;
    else
        target_el = PSTATE.EL;
    end;
    let preferred_exception_return : bits(64) = ThisInstrAddr{}();
    let vect_offset : integer{} = 0x0;

    var except : ExceptionRecord = ExceptionSyndrome(Exception_ConditionalFault);
    except.syndrome.iss[24]      = if isflt then '1' else '0';
    except.syndrome.iss[15:0]    = immediate;

    AArch64_TakeException(target_el, except, preferred_exception_return, vect_offset);
end;
```

Library pseudocode for aarch64/functions/cflt/CheckCFLTEnabled

```
// CheckCFLTEnabled()
// ==================
// Check for EL0 access to a conditional fault instruction

func CheckCFLTEnabled()
begin
    if PSTATE.EL == EL0 && !IsInHost() && !(IsSCTLR2EL1Enabled() && SCTLR2_EL1().EnCF0 == '1') then
        Undefined();
    end;

    if PSTATE.EL == EL0 && IsInHost() && !(IsSCTLR2EL2Enabled() && SCTLR2_EL2().EnCF0 == '1') then
        Undefined();
    end;
end;
```

Library pseudocode for aarch64/functions/eret/AArch64_ExceptionReturn

```
// AArch64_ExceptionReturn()
// =========================

func AArch64_ExceptionReturn(new_pc_in : bits(64), spsr : bits(64))
begin
    var new_pc : bits(64) = new_pc_in;

    if IsFeatureImplemented(FEAT_IESB) then
        var sync_errors : boolean = SCTLR_ELx().IESB == '1';
        if IsFeatureImplemented(FEAT_DoubleFault) then
            sync_errors = sync_errors || (SCR_EL3().[EA,NMEA] == '11' && PSTATE.EL == EL3);
        end;
        if sync_errors then
            SynchronizeErrors();
            // If FEAT_IESBvlp1 is implemented and an implicit error synchronization barrier
            // on exception return generates an SError exception, the resulting ESR_ELx.IESB is
            // set to 0. Otherwise, the value is an IMPLEMENTATION DEFINED choice of 0 or 1.
            let iesb_req : boolean = (!IsFeatureImplemented(FEAT_IESBvlp1) && ImpDefBool(
                      "ESR_ELx.IESB is 1 after Implicit Error Synchronization on exception return"));
            // If an implicit error synchronization barrier on exception return generates an
            // SError exception, the resulting ESR_ELx.IESB is an IMPLEMENTATION DEFINED
            // choice of 0 or 1.
            let iesb_req : boolean = ImpDefBool(
                      "ESR_ELx.IESB is 1 after Implicit Error Synchronization on exception return");
            TakeUnmaskedPhysicalSErrorInterrupts(iesb_req);
        end;
    end;

    var brbe_source_allowed : boolean = FALSE;
    var brbe_source_address : bits(64) = Zeros{};
    if IsFeatureImplemented(FEAT_BRBE) then
        brbe_source_allowed = BranchRecordAllowed(PSTATE.EL);
        brbe_source_address = PC64();
    end;


    if !IsFeatureImplemented(FEAT_ExS) || SCTLR_ELx().EOS == '1' then
        SynchronizeContext();
    end;

    // Attempts to change to an illegal state will invoke the Illegal Execution state mechanism
    let source_el : bits(2) = PSTATE.EL;
    let illegal_psr_state : boolean = IllegalExceptionReturn{64}(spsr);
    SetPSTATEFromPSR{64}(spsr, illegal_psr_state);
    ClearExclusiveLocal(ProcessorID());
    SendEventLocal();

    if illegal_psr_state && spsr[4] == '1' then
        // If the exception return is illegal, PC[63:32,1:0] are UNKNOWN
        new_pc[63:32] = ARBITRARY : bits(32);
        new_pc[1:0] = ARBITRARY : bits(2);
    elsif UsingAArch32() then                // Return to AArch32
        // ELR_ELx[1:0] or ELR_ELx[0] are treated as being 0, depending on the
        // target instruction set state
        if PSTATE.T == '1' then
            new_pc[0] = '0';                 // T32
        else
            new_pc[1:0] = '00';              // A32
        end;
    else                                     // Return to AArch64
        // ELR_ELx[63:56] might include a tag
        new_pc = AArch64_BranchAddr(new_pc, PSTATE.EL);
    end;

    if IsFeatureImplemented(FEAT_BRBE) then
        BRBEExceptionReturn(new_pc, source_el,
                            brbe_source_allowed, brbe_source_address);
    end;


    if UsingAArch32() then
        if IsFeatureImplemented(FEAT_SME) && PSTATE.SM == '1' then ResetSVEState(); end;

        // 32 most significant bits are ignored.
        let branch_conditional : boolean = FALSE;
        BranchTo{32}(new_pc[31:0], BranchType_ERET, branch_conditional);
    else
        BranchToAddr{64}(new_pc, BranchType_ERET);
    end;

    CheckExceptionCatch(FALSE);              // Check for debug event on exception return
end;
```

Library pseudocode for aarch64/functions/iesb/EIESB~~Library pseudocode for aarch64/functions/iesb/InsertIESBBeforeException~~

```
// EIESB
// =====
// Implementation options for the ID_AA64MMFR4_EL1.EIESB exception-entry behavior.
// InsertIESBBeforeException()
// ===========================
// Returns an implementation defined choice whether to insert an implicit error synchronization
// barrier before exception.
// If SCTLR_ELx.IESB is 1 when an exception is generated to ELx, any pending Unrecoverable
// SError interrupt must be taken before executing any instructions in the exception handler.
// However, this can be before the branch to the exception handler is made.

typefunc EIESB of enumeration {InsertIESBBeforeException(el : bits(2)) => boolean
begin
    return (IsFeatureImplemented(FEAT_IESB) && ImpDefBool(
                "Has Implicit Error Synchronization Barrier before Exception"));
end;
EIESB_None,                             // 0b0000
     EIESB_Before_EL3_Conditional,      // 0b0001
     EIESB_Before_All_Conditional,      // 0b0010
     EIESB_Before_All_NonSError,        // 0b0011
     EIESB_After_EL3_Conditional        // 0b1111
 };
```

Library pseudocode for aarch64/functions/iesb/GetEIESB

```
// GetEIESB()
// ==========
// Returns the EIESB implementation level.

impdef func GetEIESB() => EIESB
begin
    return EIESB_Before_All_NonSError;
end;
```

Library pseudocode for aarch64/functions/iesb/InsertIESBBeforeException

```
// InsertIESBBeforeException()
// ===========================
// Returns whether an implicit Error synchronization event is inserted before
// taking an exception, according to the EIESB implementation behavior.
//
// If the Effective value of SCTLR_ELx.IESB is 1 when an exception is generated
// to ELx, any pending Unrecoverable SError interrupt must be taken before
// executing any instructions in the exception handler. Depending on the EIESB
// implementation behavior, this can be before the branch to the exception
// handler is made.

func InsertIESBBeforeException(exceptype : Exception, target_el : bits(2)) => boolean
begin
    // Exclude SError exceptions.
    if exceptype == Exception_SError then
        return FALSE;
    end;

    case GetEIESB() of
        when {EIESB_After_EL3_Conditional,
              EIESB_None} =>
            // Might insert after taking an exception to EL3, unless the exception is also an SError
            // exception.
            // Or behavior is not described.
            return ImpDefBool("Has Implicit Error Synchronization Barrier before Exception");

        when EIESB_Before_EL3_Conditional =>
            // Insert before taking an exception to EL3,
            // unless the exception is also an SError exception.
            if target_el == EL3 && EffectiveEA() == '1' && EffectiveNMEA(EL3) == '1' then
                return TRUE;
            end;

        when EIESB_Before_All_Conditional =>
            // As EIESB_Before_EL3_Conditional. Also insert before taking exceptions to EL1/EL2
            // when the architectural conditions apply.
            if target_el == EL3 && EffectiveEA() == '1' && EffectiveNMEA(EL3) == '1' then
                return TRUE;

            elsif target_el IN {EL1, EL2} then
                if EffectiveNMEA(target_el) == '1' then
                    return TRUE;
                end;
            end;

        when EIESB_Before_All_NonSError =>
            // Insert before taking all exceptions other than SError exceptions,
            // regardless of the architectural conditions.
            return TRUE;
    end;
    return ImpDefBool("Has Implicit Error Synchronization Barrier before Exception");
end;
```

Library pseudocode for aarch64/functions/memory/AArch64_MemSingleRead

```
// AArch64_MemSingleRead()
// =======================
// Perform an atomic, little-endian read of 'size' bits.

func AArch64_MemSingleRead{size : integer{8, 16, 32, 64, 128, 256}}(address : bits(64),
                                                                    accdesc_in : AccessDescriptor,
                                                                    aligned : boolean
                                                                   ) => (bits(size),
                                                                         AddressDescriptor,
                                                                         PhysMemRetStatus)
begin
    var value : bits(size) = ARBITRARY : bits(size);
    var memstatus : PhysMemRetStatus = ARBITRARY : PhysMemRetStatus;
    let bytes : integer{} = size DIV 8;
    memstatus.statuscode = Fault_None;
    var accdesc : AccessDescriptor = accdesc_in;
    if IsFeatureImplemented(FEAT_LSE2) then
        assert ((IsFeatureImplemented(FEAT_LS64WB) &&
                 bytes == 32 && accdesc.acctype == AccessType_ASIMD) ||
                 (IsFeatureImplemented(FEAT_SVE_LRCPC) && bytes == 32) ||) ++
AllInSCAGranule(address, bytes));
    else
        assert IsAlignedSize(address, bytes);
    end;

    // If the instruction encoding permits tag checking, confer with system register configuration
    // which may override this.
    if accdesc.tagchecked then
        accdesc.tagchecked = AArch64_AccessIsTagChecked(address, accdesc);
    end;

    var memaddrdesc : AddressDescriptor;
    memaddrdesc = AArch64_TranslateAddress(address, accdesc, aligned, bytes);

    // Check for aborts or debug exceptions
    if IsFault(memaddrdesc) then
        return (value, memaddrdesc, memstatus);
    end;

    // Memory array access
    if accdesc.tagchecked then
        let ltag : bits(4) = AArch64_LogicalAddressTag(address);
        let readcheck : boolean = TRUE;
        let fault : FaultRecord = AArch64_CheckTag(memaddrdesc, accdesc, readcheck, bytes, ltag);
        if fault.statuscode != Fault_None then
            memaddrdesc.fault = fault;
            return (value, memaddrdesc, memstatus);
        end;
    end;

    if accdesc.acctype != AccessType_IFETCH && SPESampleInFlight then
        let is_load : boolean = TRUE;
        SPESampleLoadStore(is_load, accdesc, memaddrdesc);
    end;
    var atomic : boolean;
    if IsWBShareable(memaddrdesc.memattrs) then
        atomic = TRUE;
    elsif accdesc.exclusive then
        atomic = TRUE;
    elsif (accdesc.acctype == AccessType_SVE && accdesc.predicated &&
           bytes == 8 && IsAlignedSize(address, 8)) then
        // An SVE predicated load of a 128-bit element that is 64-bit aligned
        // is treated as a pair of 64-bit single-copy atomic accesses.
        // This is one of the 64-bit single-copy atomic access.
        atomic = TRUE;
    elsif aligned then
        atomic = !accdesc.ispair;
    else
        // Misaligned accesses within a single-copy atomic granule but
        // not Normal Cacheable Writeback are Atomic
        atomic = ImpDefBool("FEAT_LSE2: access is atomic");
    end;

    if atomic then
        (memstatus, value) = PhysMemRead{size}(memaddrdesc, accdesc);

    elsif accdesc.acctype == AccessType_ASIMD && bytes == 32 && accdesc.ispair then
        // A 32 byte LDP (SIMD&FP) that does not target Normal Inner Write-Back, Outer
        // Write-Back cacheable, Shareable memory is treated as four 8 byte atomic accesses.
        // As this access was not split in Mem(), it must be aligned to 32 bytes.
        assert IsAlignedSize(address, 32);
        accdesc.ispair = FALSE;
        for i = 0 to 3 do
            if !IsFault(memstatus) then     // Do not continue past a fault
                (memstatus, value[i*64+:64]) = PhysMemRead{64}(memaddrdesc, accdesc);
            end;

            if !IsFault(memstatus) then
                memaddrdesc.paddress.address = memaddrdesc.paddress.address + 8;
```

```
                memaddrdesc.vaddress         = memaddrdesc.vaddress + 8;
            end;
        end;

    elsif aligned && accdesc.ispair then
        let half : integer{} = (size DIV 2) as integer{32, 64, 128};
        (memstatus, value[0*:half]) = PhysMemRead{half}(memaddrdesc, accdesc);

        if !IsFault(memstatus) then
            memaddrdesc.paddress.address = memaddrdesc.paddress.address + (half DIV 8);
            memaddrdesc.vaddress         = memaddrdesc.vaddress + (half DIV 8);
            (memstatus, value[1*:half]) = PhysMemRead{half}(memaddrdesc, accdesc);
        end;

    else
        for i = 0 to bytes-1 do
            if !IsFault(memstatus) then     // Do not continue past a fault
                (memstatus, value[i*:8]) = PhysMemRead{8}(memaddrdesc, accdesc);
            end;

            if !IsFault(memstatus) then
                memaddrdesc.paddress.address = memaddrdesc.paddress.address + 1;
                memaddrdesc.vaddress         = memaddrdesc.vaddress + 1;
            end;

        end;
    end;

    if IsFault(memstatus) then
        return (value, memaddrdesc, memstatus);
    end;

    if ((accdesc.acctype != AccessType_IFETCH &&
           ImpDefBool(
               "GPF, GPC, Data SyncExternal Aborts, TagChecks prioritized over Watchpoints")) ||
          (accdesc.acctype == AccessType_IFETCH &&
           ImpDefBool("Instruction SyncExternal Aborts prioritized over Breakpoints"))) then
        memaddrdesc.fault = AArch64_CheckDebug(address, accdesc, bytes);
    end;

    return (value, memaddrdesc, memstatus);
end;
```

Library pseudocode for aarch64/functions/memory/AArch64_MemSingleWrite

```
// AArch64_MemSingleWrite()
// ========================
// Perform an atomic, little-endian write of 'size' bits.

func AArch64_MemSingleWrite{size : integer{8, 16, 32, 64, 128, 256}}(address : bits(64),
                                                                     accdesc_in : AccessDescriptor,
                                                                     aligned : boolean,
                                                                     value : bits(size)
                                                                   ) => (AddressDescriptor, PhysMemRetStatus)
begin
    var accdesc : AccessDescriptor = accdesc_in;
    let bytes : integer{} = size DIV 8;
    if IsFeatureImplemented(FEAT_LSE2) then
        assert ((IsFeatureImplemented(FEAT_LS64WB) &&
                 bytes == 32 && accdesc.acctype == AccessType_ASIMD) ||
                (IsFeatureImplemented(FEAT_SVE_LRCPC) && bytes == 32) ||) ||
AllInSCAGranule(address, bytes));
    else
        assert IsAlignedSize(address, bytes);
    end;

    // If the instruction encoding permits tag checking, confer with system register configuration
    // which may override this.
    if accdesc.tagchecked then
        accdesc.tagchecked = AArch64_AccessIsTagChecked(address, accdesc);
    end;

    var memaddrdesc : AddressDescriptor;
    var memstatus : PhysMemRetStatus = ARBITRARY : PhysMemRetStatus;
    memaddrdesc = AArch64_TranslateAddress(address, accdesc, aligned, bytes);

    // Check for aborts or debug exceptions
    if IsFault(memaddrdesc) then
        return (memaddrdesc, memstatus);
    end;

    // Effect on exclusives
    if memaddrdesc.mematttrs.shareability != Shareability_NSH then
        ClearExclusiveByAddress(memaddrdesc.paddress, ProcessorID(), bytes);
    end;

    if accdesc.tagchecked then
        let ltag : bits(4) = AArch64_LogicalAddressTag(address);
        let readcheck : boolean = FALSE;
        let fault : FaultRecord = AArch64_CheckTag(memaddrdesc, accdesc, readcheck, bytes, ltag);
        if fault.statuscode != Fault_None then
            memaddrdesc.fault = fault;
            return (memaddrdesc, memstatus);
        end;
    end;

    if SPESampleInFlight then
        let is_load : boolean = FALSE;
        SPESampleLoadStore(is_load, accdesc, memaddrdesc);
    end;

    var atomic : boolean;
    if IsWBShareable(memaddrdesc.mematttrs) then
        atomic = TRUE;
    elsif accdesc.exclusive then
        atomic = TRUE;
    elsif (accdesc.acctype == AccessType_SVE && accdesc.predicated &&
           bytes == 8 && IsAlignedSize(address, 8)) then
        // An SVE predicated load of a 128-bit element that is 64-bit aligned
        // is treated as a pair of 64-bit single-copy atomic accesses.
        // This is one of the 64-bit single-copy atomic access.
        atomic = TRUE;
    elsif aligned then
        atomic = !accdesc.ispair;
    else
        // Misaligned accesses within a single-copy atomic granule but
        // not Normal Cacheable Writeback are Atomic
        atomic = ImpDefBool("FEAT_LSE2: access is atomic");
    end;
    if atomic then
        memstatus = PhysMemWrite{size}(memaddrdesc, accdesc, value);
        if IsFault(memstatus) then
            return (memaddrdesc, memstatus);
        end;

    elsif accdesc.acctype == AccessType_ASIMD && bytes == 32 && accdesc.ispair then
        // A 32 byte STP (SIMD&FP) that does not target Normal Inner Write-Back, Outer
        // Write-Back cacheable, Shareable memory is treated as four 8 byte atomic accesses.
        // As this access was not split in Mem(), it must be aligned to 32 bytes.
        assert IsAlignedSize(address, 32);
        accdesc.ispair = FALSE;
        for i = 0 to 3 do
            memstatus = PhysMemWrite{64}(memaddrdesc, accdesc, value[64*i+:64]);
            if IsFault(memstatus) then
```

```
                return (memaddrdesc, memstatus);
            end;
            memaddrdesc.paddress.address = memaddrdesc.paddress.address + 8;
            memaddrdesc.vaddress         = memaddrdesc.vaddress + 8;
        end;

    elsif aligned && accdesc.ispair then
        let half : integer{} = (size DIV 2) as integer{32, 64};
        memstatus = PhysMemWrite{half}(memaddrdesc, accdesc, value[0*:half]);
        if IsFault(memstatus) then
            return (memaddrdesc, memstatus);
        end;

        memaddrdesc.paddress.address = memaddrdesc.paddress.address + (half DIV 8);
        memaddrdesc.vaddress         = memaddrdesc.vaddress + (half DIV 8);
        memstatus = PhysMemWrite{half}(memaddrdesc, accdesc, value[1*:half]);
        if IsFault(memstatus) then
            return (memaddrdesc, memstatus);
        end;

    else
        for i = 0 to bytes-1 do
            memstatus = PhysMemWrite{8}(memaddrdesc, accdesc, value[i*:8]);
            if IsFault(memstatus) then
                return (memaddrdesc, memstatus);
            end;
            memaddrdesc.paddress.address = memaddrdesc.paddress.address + 1;
            memaddrdesc.vaddress         = memaddrdesc.vaddress + 1;
        end;
    end;

    if (accdesc.acctype != AccessType_IFETCH &&
          ImpDefBool(
              "GPF, GPC, Data SyncExternal Aborts, TagChecks prioritized over Watchpoints")) then
        memaddrdesc.fault = AArch64_CheckDebug(address, accdesc, bytes);
    end;

    return (memaddrdesc, memstatus);
end;
```

Library pseudocode for aarch64/functions/memory/AArch64_UnalignedAccessFaults

```
// AArch64_UnalignedAccessFaults()
// ===============================
// Determine whether the unaligned access generates an Alignment fault

func AArch64_UnalignedAccessFaults(accdesc : AccessDescriptor,
                                   address : bits(64),
                                   size : integer) => boolean
begin
    if AlignmentEnforced() then
        return TRUE;
    elsif accdesc.acctype == AccessType_GCS then
        return TRUE;
    elsif accdesc.rcw then
        return TRUE;
    elsif accdesc.ls64 then
        return TRUE;
    elsif (accdesc.exclusive || accdesc.atomicop)  then
        return (!IsFeatureImplemented(FEAT_LSE2) ||
                  !AllInSCAGranule(address, size));
    elsif (accdesc.acqsc || accdesc.acqpc || accdesc.relsc) then
        // If nAA is 0, the address accessed by each register in the pair
        // must lie within a single 16-byte aligned block
        if accdesc.ispair then
            return  (SCTLR_ELx().nAA == '0' &&
                       (!AllInAlignedQuantity{64}(address, size DIV 2, 16) ||
                        !AllInAlignedQuantity{64}(address + size DIV 2, size DIV 2, 16)));
        // Octaword load/store are 32 bytes aligned
        elsif IsFeatureImplemented(FEAT_SVE_LRCPC) && (size == 32) then
        else
            return  (SCTLR_ELx().nAA == '0' &&
                       !AllInAlignedQuantity{64}(address, size, 32));
        else
            return  (~~{64}(address, size, 16)~~);
        ~~end;~~
    ~~else~~
        ~~return FALSE;~~
    ~~end;~~
~~end;~~SCTLR_ELx().nAA == '0' &&
                       !AllInAlignedQuantity{64}(address, size, 16));
        end;
    else
        return FALSE;
    end;
end;
```

Library pseudocode for aarch64/functions/memory/AllInSCAGranule

```
// AllInSCAGranule()
// =================
// When FEAT_LSE2 is implemented, for some memory accesses if all bytes of the access lie within
// the single-copy atomicity granule corresponding to CTR_EL0.LSE2Size and satisfy additional
// requirements, then the access is guaranteed to be single-copy atomic.
// When FEAT_LSE2 is implemented, for some memory accesses if all bytes
// of the accesses are within 16-byte quantity aligned to 16-bytes and
// satisfy additional requirements - then the access is guaranteed to
// be single copy atomic.
// However, when the accesses do not all lie within such a boundary, it
// is CONSTRAINED UNPREDICTABLE if the access is single copy atomic.
// In the pseudocode, this CONSTRAINED UNPREDICTABLE aspect is modeled via
// an IMPLEMENTATION DEFINED aligned quantity that is at least 16 bytes and
// at most 4096 bytes.
// This is a limitation of the pseudocode.

readonly func AllInSCAGranule(address : bits(64), size : integer) => boolean
begin
    if NUM_SCA_GRANULE_BYTES == 0 then
        // The single-copy atomicity granule size is unbounded.
        return TRUE;
    end;
    let quantity : integer = ImpDefInt("Aligned quantity for atomic access");
    // access is assumed to be within 4096 byte aligned quantity to
    // avoid multiple translations for a single copy atomic access.
    assert (quantity >= 16) && (quantity <= 4096);
    return AllInAlignedQuantity{64}(address, size, NUM_SCA_GRANULE_BYTES);
{64}(address, size, quantity);
end;
```

Library pseudocode for aarch64/functions/memory/MemAtomic

```
// MemAtomic()
// ===========
// Performs load and store memory operations for a given virtual address.

func MemAtomic{size : integer{8, 16, 32, 64, 128}}(address : bits(64),
                                                   cmpoperand : bits(size),
                                                   operand : bits(size),
                                                   accdesc_in : AccessDescriptor
                                                  ) => bits(size)
begin
    assert accdesc_in.atomicop;

    let bytes : integer{} = (size DIV 8) as integer{1, 2, 4, 8, 16};

    var newvalue : bits(size);
    var oldvalue : bits(size);
    var accdesc : AccessDescriptor = accdesc_in;
    let aligned : boolean = IsAlignedSize(address, bytes);

    // If the instruction encoding permits tag checking, confer with system register configuration
    // which may override this.
    if accdesc.tagchecked then
        accdesc.tagchecked = AArch64_AccessIsTagChecked(address, accdesc);
    end;

    if !aligned && AArch64_UnalignedAccessFaults(accdesc, address, bytes) then
        let fault : FaultRecord = AlignmentFault(accdesc, address);
        AArch64_Abort(fault);
    end;

    // MMU or MPU lookup
    let memaddrdesc : AddressDescriptor = AArch64_TranslateAddress(address, accdesc,
                                                                   aligned, bytes);

    // Check for aborts or debug exceptions
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    end;

    if (!IsWBShareable(memaddrdesc.mematttrs) &&
            ConstrainUnpredictableBool(Unpredictable_Atomic_NOP)) then
        return ARBITRARY : bits(size);
    end;

    // Effect on exclusives
    if memaddrdesc.mematttrs.shareability != Shareability_NSH then
        ClearExclusiveByAddress(memaddrdesc.paddress, ProcessorID(), bytes);
    end;

    // For Store-only Tag checking, the tag check is performed on the store.
    if (accdesc.tagchecked &&
          (!IsFeatureImplemented(FEAT_MTE_STORE_ONLY) ||
           !StoreOnlyTagCheckingEnabled(accdesc.el))) then
        let ltag : bits(4) = AArch64_LogicalAddressTag(address);
        let readcheck : boolean = TRUE;
        let fault : FaultRecord = AArch64_CheckTag(memaddrdesc, accdesc, readcheck, bytes, ltag);
        if fault.statuscode != Fault_None then
            AArch64_Abort(fault);
        end;
    end;

    // All observers in the shareability domain observe the following load and store atomically.
    var memstatus : PhysMemRetStatus;
    (memstatus, oldvalue) = PhysMemRead{size}(memaddrdesc, accdesc);
    // Depending on the memory type of the physical address, the access might generate
    // either a synchronous External abort or an SError exception
    // among other CONSTRAINED UNPREDICTABLE choices.

    if IsFault(memstatus) then
        HandleExternalReadAbort(memstatus, memaddrdesc, bytes, accdesc);
    end;
    if BigEndian(accdesc.acctype) then
        oldvalue = BigEndianReverse{size}(oldvalue);
    end;

    var cmpfail : boolean = FALSE;
    var retvalue : bits(size) = oldvalue;
    if accdesc.acctype == AccessType_FP then
        newvalue = MemAtomicFP{size}(accdesc.modop, oldvalue, operand);
    else
        (newvalue, cmpfail, retvalue) = MemAtomicInt{size}(accdesc.modop, oldvalue,
                                                           operand, cmpoperand);
    end;

    let requirewrite : boolean = (!cmpfail ||
                                  ConstrainUnpredictableBool(Unpredictable_WRITEFAILEDCAS));

    if IsFeatureImplemented(FEAT_MTE_STORE_ONLY) && StoreOnlyTagCheckingEnabled(accdesc.el) then
        // If the compare on a Tag checked CAS fails and no write is performed, then it is
```

```
        // CONSTRAINED UNPREDICTABLE whether the Tag check is performed.
        // If the compare on a CAS fails, then it is CONSTRAINED UNPREDICTABLE whether the
        // Tag check is performed.
        if accdesc.tagchecked && !requirewrite then
            accdesc.tagchecked = ConstrainUnpredictableBool(Unpredictable_STRONLYTAGCHECKEDCAS);
        end;

        if accdesc.tagchecked then
            let ltag : bits(4) = AArch64_LogicalAddressTag(address);
            let readcheck : boolean = FALSE;
            var fault : FaultRecord = AArch64_CheckTag(memaddrdesc, accdesc, readcheck,
                                                       bytes, ltag);
            if fault.statuscode != Fault_None then
                // For a synchronous Tag Check Fault due to FEAT_MTE_STORE_ONLY, set WnR.
                fault.write = TRUE;
                AArch64_Abort(fault);
            end;
        end;
    end;

    if requirewrite then
        if BigEndian(accdesc.acctype) then
            newvalue = BigEndianReverse{size}(newvalue);
        end;
        memstatus = PhysMemWrite{size}(memaddrdesc, accdesc, newvalue);
        if IsFault(memstatus) then
            HandleExternalWriteAbort(memstatus, memaddrdesc, bytes, accdesc);
        end;
    end;

    if SPESampleInFlight then
        let is_load : boolean = FALSE;
        SPESampleLoadStore(is_load, accdesc, memaddrdesc);
    end;

    // Load operations return the old (pre-operation) value.
    // Compare and Swap operations return the old (pre-operation) value. For a successful CAS,
    // this might be the value from the compare operand or from memory.
    return retvalue;
end;
```

Library pseudocode for aarch64/functions/memory/MemAtomicRCW

```
// MemAtomicRCW()
// ==============
// Perform a single-copy-atomic access with Read-Check-Write operation

func MemAtomicRCW{size : integer{64, 128}}(address : bits(64), cmpoperand : bits(size),
                                           operand : bits(size), accdesc_in : AccessDescriptor
                                    ) => (bits(4), bits(size))
begin
    assert accdesc_in.atomicop;
    assert accdesc_in.rcw;

    let bytes : integer{} = (size DIV 8) as integer{8, 16};

    var nzcv : bits(4);
    var oldvalue : bits(size);
    var newvalue : bits(size);
    var accdesc : AccessDescriptor = accdesc_in;
    let aligned : boolean = IsAlignedSize(address, bytes);

    // If the instruction encoding permits tag checking, confer with system register configuration
    // which may override this.
    if accdesc.tagchecked then
        accdesc.tagchecked = AArch64_AccessIsTagChecked(address, accdesc);
    end;

    if !aligned && AArch64_UnalignedAccessFaults(accdesc, address, bytes) then
        let fault : FaultRecord = AlignmentFault(accdesc, address);
        AArch64_Abort(fault);
    end;

    // MMU or MPU lookup
    let memaddrdesc : AddressDescriptor = AArch64_TranslateAddress(address, accdesc,
                                                                   aligned, bytes);

    // Check for aborts or debug exceptions
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    end;

    if (!IsWBShareable(memaddrdesc.memattrs) &&
            ConstrainUnpredictableBool(Unpredictable_Atomic_NOP)) then
        return (ARBITRARY : bits(4), ARBITRARY : bits(size));
    end;

    // Effect on exclusives
    if memaddrdesc.memattrs.shareability != Shareability_NSH then
        ClearExclusiveByAddress(memaddrdesc.paddress, ProcessorID(), bytes);
    end;

    // For Store-only Tag checking, the tag check is performed on the store.
    if (accdesc.tagchecked &&
          (!IsFeatureImplemented(FEAT_MTE_STORE_ONLY) ||
           !StoreOnlyTagCheckingEnabled(accdesc.el))) then
        let ltag : bits(4) = AArch64_LogicalAddressTag(address);
        let readcheck : boolean = TRUE;
        let fault : FaultRecord = AArch64_CheckTag(memaddrdesc, accdesc, readcheck, bytes, ltag);
        if fault.statuscode != Fault_None then
            AArch64_Abort(fault);
        end;
    end;

    // All observers in the shareability domain observe the following load and store atomically.
    var memstatus : PhysMemRetStatus;
    (memstatus, oldvalue) = PhysMemRead{size}(memaddrdesc, accdesc);
    // Depending on the memory type of the physical address, the access might generate
    // either a synchronous External abort or an SError exception
    // among other CONSTRAINED UNPREDICTABLE choices.

    if IsFault(memstatus) then
        HandleExternalReadAbort(memstatus, memaddrdesc, bytes, accdesc);
    end;
    if BigEndian(accdesc.acctype) then
        oldvalue = BigEndianReverse{size}(oldvalue);
    end;

    var cmpfail : boolean = FALSE;
    var retvalue : bits(size) = oldvalue;
    case accdesc.modop of
        when MemAtomicOp_BIC => newvalue = oldvalue AND NOT(operand);
        when MemAtomicOp_ORR => newvalue = oldvalue OR operand;
        when MemAtomicOp_SWP => newvalue = operand;
        when MemAtomicOp_CAS =>
            (newvalue, cmpfail, retvalue) = CASCompare{size}(oldvalue, cmpoperand, operand);
    end;

    if cmpfail then
        nzcv = '1010'; // N = 1 indicates compare failure
    else
        nzcv = RCWCheck{size}(retvalue, newvalue, accdesc.rcws);
```

```
    end;

    // If RCWCheck() passes, it returns nzcv == '0010'
    let requirewrite : boolean = (nzcv == '0010' ||
                                  ConstrainUnpredictableBool(Unpredictable_WRITEFAILEDRCWCAS));

    if IsFeatureImplemented(FEAT_MTE_STORE_ONLY) && StoreOnlyTagCheckingEnabled(accdesc.el) then
        // If the RCW(S) checks, or if the compare on a Tag checked RCW(S)CAS fails and no write is
        // performed, then it is CONSTRAINED UNPREDICTABLE whether the Tag check is performed.
        // If the RCW(S) check fails, or if the compare on a RCW(S)CAS fails,
        // then it is CONSTRAINED UNPREDICTABLE whether the Tag check is performed.
        if accdesc.tagchecked && !requirewrite then
            accdesc.tagchecked = ConstrainUnpredictableBool(Unpredictable_STRONLYTAGCHECKEDRCWSCAS);
        end;

        if accdesc.tagchecked then
            let ltag : bits(4) = AArch64_LogicalAddressTag(address);
            let readcheck : boolean = FALSE;
            var fault : FaultRecord = AArch64_CheckTag(memaddrdesc, accdesc, readcheck,
                                                       bytes, ltag);
            if fault.statuscode != Fault_None then
                // For a synchronous Tag Check Fault due to FEAT_MTE_STORE_ONLY, set WnR.
                fault.write = TRUE;
                AArch64_Abort(fault);
            end;
        end;
    end;

    if requirewrite then
        if BigEndian(accdesc.acctype) then
            newvalue = BigEndianReverse{size}(newvalue);
        end;

        memstatus = PhysMemWrite{size}(memaddrdesc, accdesc, newvalue);

        if IsFault(memstatus) then
            HandleExternalWriteAbort(memstatus, memaddrdesc, bytes, accdesc);
        end;
    end;

    if SPESampleInFlight then
        let is_load : boolean = TRUE;
        SPESampleLoadStore(is_load, accdesc, memaddrdesc);
    end;

    // Load operations return the old (pre-operation) value.
    // Compare and Swap operations return the old (pre-operation) value. For a successful CAS,
    // this might be the value from the compare operand or from memory.
    return (nzcv, retvalue);
end;
```

Library pseudocode for aarch64/functions/mops/MaxBlockSizeCopiedBytes

```
// MaxBlockSizeCopiedBytes()
// =========================
// Returns the maximum number of bytes that can used in a single block of the copy.

func MaxBlockSizeCopiedBytes() => integer
begin
    return ImpDefInt("Maximum bytes used in a single block of a copy");
end;
```

Library pseudocode for aarch64/functions/ras/AArch64_PhysicalSErrorSyndrome

```
// AArch64_PhysicalSErrorSyndrome()
// ================================
// Generate SError syndrome.

func AArch64_PhysicalSErrorSyndrome(is_esb : boolean, implicit_esb : boolean) => bits(25)
begin
    var syndrome : bits(25) = Zeros{};
    var target_el : bits(2);
    (-, target_el) = PhysicalSErrorTarget();

    if ReportErrorAsUncategorized() then
        syndrome = Zeros{25};
    elsif ReportErrorAsIMPDEF() then
        syndrome[24]   = '1';                                          // IDS
        syndrome[23:0] = ImpDefBits{24}("IMPDEF ErrorState");
    else
        let fault : FaultRecord = GetPendingPhysicalSError();
        let errorstate : ErrorState = PEErrorState(fault);
        syndrome[24]    = '0';                                         // IDS
        if (!is_esb && IsFeatureImplemented(FEAT_PFAR) &&
              !(EL2Enabled() &&
              (HCR_EL2().VM == '1' || (IsFeatureImplemented(FEAT_HCRMAC) && HCR_EL2().DC == '1')) &&
              target_el == (HCR_EL2().VM == '1' || HCR_EL2().DC == '1') && target_el == EL1)) then
            syndrome[14] = if IsPFARValid(fault) then '1' else '0';
        end;
        syndrome[13]    = if implicit_esb then '1' else '0';           // IESB
        syndrome[12:10] = AArch64_EncodeAsyncErrorSyndrome(errorstate);   // AET
        syndrome[9]     = fault.extflag;                               // EA
        syndrome[5:0]   = '010001';                                    // DFSC
    end;

    return syndrome;
end;
```

Library pseudocode for aarch64/functions/ras/IsDeviceMemoryReadErrorSynchronous

```
// IsDeviceMemoryReadErrorSynchronous()
// ====================================
// Returns TRUE when the Device memory read is synchronous in the current Exception level.

func IsDeviceMemoryReadErrorSynchronous(accdesc : AccessDescriptor, desc : AddressDescriptor)
                                        => boolean
begin
    if !IsFeatureImplemented(FEAT_ADERR) || !    if IsFeatureImplemented(FEAT_ADERR) &&CheckEnADERRApplies(accdesc)
then
        return ImpDefBool("Device read error sync when EnADERR not implemented or does not apply");
    end;
    case        case S1TranslationRegime(accdesc.el) of
        when EL3 =>
            if ((!IsFeatureImplemented(FEAT_ANERR) ||                if
HaveEL(EL3) && SCR_EL3().SCTLR2En == '0' then
                    return TRUE;
                end;
                if ((!IsFeatureImplemented(FEAT_ANERR) ||
                    GetADERRErrorMode() != ErrSignalMode_TiedToPeer ||
                 SCTLR2_EL3().EnANERR == '0') && SCTLR2_EL3().EnADERR == '0') then
                return TRUE;
            end;

        when EL2 =>
            if !                if (!IsSCTLR2EL2Enabled() then
                if ImpDefBool("SCTLR2_ELx.EnADERR resets to 0 at highest ELx") then
                    return TRUE;
                end;
            elsif ((!IsFeatureImplemented(FEAT_ANERR) ||() ||
                    ((!IsFeatureImplemented(FEAT_ANERR) ||
GetADERRErrorMode() != ErrSignalMode_TiedToPeer ||
                    SCTLR2_EL2().EnANERR == '0') && SCTLR2_EL2().EnADERR == '0') then
                return TRUE;
            end;
                      SCTLR2_EL2().EnANERR == '0') && SCTLR2_EL2().EnADERR == '0')) then
                return TRUE;
            end;

        when            when EL1 =>
            if IsHCRXEL2Enabled =>
            if() && EffectiveHCRX_EL2_EnSDERR() == '1' then
                return TRUE;
            end;
            if !                    return TRUE;
            end;
            if (!IsSCTLR2EL1Enabled() then
            if ImpDefBool("SCTLR2_ELx.EnADERR resets to 0 at highest ELx") then
                return TRUE;
            end;
        elsif ((!IsFeatureImplemented(FEAT_ANERR) ||() ||
                    ((!IsFeatureImplemented(FEAT_ANERR) ||
GetADERRErrorMode() != ErrSignalMode_TiedToPeer ||
                    SCTLR2_EL1().EnANERR == '0') &&
                  SCTLR2_EL1().EnADERR == '0') then
            return TRUE;
        end;
        otherwise => unreachable;
                      SCTLR2_EL1().EnANERR == '0') &&
                    SCTLR2_EL1().EnADERR == '0')) then
                return TRUE;
            end;
            otherwise => unreachable;
        end;
    end;
    return ImpDefBool("Device read error sync when the Effective value of EnADERR is 1");
    return ImpDefBool("Device read error sync when EnADERR not implemented or enabled");
end;
```

Library pseudocode for aarch64/functions/ras/IsNormalMemoryReadErrorSynchronous

```
// IsNormalMemoryReadErrorSynchronous()
// ====================================
// Returns TRUE when error signaled on a load from Normal memory is taken synchronously.

func IsNormalMemoryReadErrorSynchronous(accdesc : AccessDescriptor, desc : AddressDescriptor)
                                        => boolean
begin
    if !IsFeatureImplemented(FEAT_ANERR) || !    if IsFeatureImplemented(FEAT_ANERR) &&CheckEnANERRApplies(accdesc,
desc) then
        return ImpDefBool("Normal read error sync when EnANERR not implemented or does not apply");
    end;
    case       case S1TranslationRegime(accdesc.el) of
        when EL3 =>
            if ((!IsFeatureImplemented(FEAT_ADERR) ||
                 GetANERRErrorMode() != ErrSignalMode_TiedToPeer ||
                 SCTLR2_EL3().EnADERR == '0') && SCTLR2_EL3().EnANERR == '0') then
                return TRUE;
            end;

        when EL2 =>
            if !               if (!IsSCTLR2EL2Enabled() then
                if ImpDefBool("SCTLR2_ELx.EnANERR resets to 0 at highest ELx") then
                    return TRUE;
                end;
            elsif ((!IsFeatureImplemented(FEAT_ADERR) ||() ||
                     ((!IsFeatureImplemented(FEAT_ADERR) ||
GetANERRErrorMode() != ErrSignalMode_TiedToPeer ||
                    SCTLR2_EL2().EnADERR == '0') && SCTLR2_EL2().EnANERR == '0') then
                return TRUE;
            end;
                        SCTLR2_EL2().EnADERR == '0') && SCTLR2_EL2().EnANERR == '0')) then
                    return TRUE;
                end;

        when           when EL1 =>
            if EffectiveHCRX_EL2_EnSNERR() == '1' then
                return TRUE;
            end;
            if !                    return TRUE;
                end;
                if (!IsSCTLR2EL1Enabled() then
                if ImpDefBool("SCTLR2_ELx.EnANERR resets to 0 at highest ELx") then
                    return TRUE;
                end;
            elsif ((!IsFeatureImplemented(FEAT_ADERR) ||() ||
                     ((!IsFeatureImplemented(FEAT_ADERR) ||
GetANERRErrorMode() != ErrSignalMode_TiedToPeer ||
                    SCTLR2_EL1().EnADERR == '0') &&
                   SCTLR2_EL1().EnANERR == '0') then
                return TRUE;
            end;
        otherwise => unreachable;
                        SCTLR2_EL1().EnADERR == '0') &&
                      SCTLR2_EL1().EnANERR == '0')) then
                    return TRUE;
                end;
            otherwise => unreachable;
        end;
    end;
    return ImpDefBool("Normal read error sync when the Effective value of EnANERR is 1");
    return ImpDefBool("Normal read error sync when EnANERR not implemented or enabled");
end;
```

Library pseudocode for aarch64/functions/sve/CheckSMEAccess

```
// CheckSMEAccess()
// ================
// Check that access to SME System registers is enabled.

func CheckSMEAccess()
begin
    var disabled : boolean;

    if HaveEL(EL3) && CPTR_EL3().ESM == '0' && EL3SDDUndefPriority() then
        Undefined();
    end;

    // Check if access disabled in CPACR_EL1
    if PSTATE.EL IN {EL0, EL1} && !IsInHost() then
        // Check SME at EL0/EL1
        case CPACR_EL1().SMEN of
            when 'x0' => disabled = TRUE;
            when '01' => disabled = EffectivelyAtEL0NotInHostPSTATE();
            when '11' => disabled = FALSE;
        end;
        if disabled then.EL == EL0;
            when '11' => disabled = FALSE;
        end;
        if disabled then SMEAccessTrap(SMEExceptionType_AccessTrap, EL1); end;
    end;

    if PSTATE.EL IN {EL0, EL1, EL2} && EL2Enabled() then
        if ELIsInHost(EL2) then
            // Check SME at EL2
            case CPTR_EL2().SMEN of
                when 'x0' => disabled = TRUE;
                when '01' => disabled = .EL == EL0EffectivelyAtEL0InHostPSTATE();
&& HCR_EL2().TGE == '1';
                when '11' => disabled = FALSE;
            end;
            if disabled then SMEAccessTrap(SMEExceptionType_AccessTrap, EL2); end;
        else
            if CPTR_EL2().TSM == '1' then SMEAccessTrap(SMEExceptionType_AccessTrap, EL2); end;
        end;
    end;

    // Check if access disabled in CPTR_EL3
    if HaveEL(EL3) then
        if CPTR_EL3().ESM == '0' then
            if EL3SDDUndef() then Undefined(); end;
            SMEAccessTrap(SMEExceptionType_AccessTrap, EL3);
        end;
    end;

end;
```

Library pseudocode for aarch64/functions/sve/FPMinNormal

```
// FPMinNormal()
// =============

func FPMinNormal{N}(sign : bit) => bits(N)
begin
    assert N IN {16,32,64};
    let E : integer{} = (if N == 16 then 5 else (if N == 32 then 8 else 11));
    let F : integer{} = N - (E + 1);
    let exp : bits(E) = Zeros{E-1}::'1';
    let frac : bits(F) = Zeros{};
    return sign :: exp :: frac;
end;
```

Library pseudocode for aarch64/functions/sve/IntReducePredicated

```
// IntReducePredicated()
// =====================
func IntReducePredicated{esize, N, M}(op : ReduceOp, input : bits(N),
                                      mask : bits(M), identity : bits(esize)) => bits(esize)
begin
    assert(N == M * 8);
    assert IsPow2(N);
    var operand : bits(N);
    let elements : integer = N DIV esize;

    for e = 0 to elements-1 do
        if e * esize < N && ActivePredicateElement{M}(mask, e, esize) then
            operand[e*:esize] = input[e*:esize];
        else
            operand[e*:esize] = identity;
        end;
    end;

    return IntReduce{esize, N}(op, operand);
end;
```

Library pseudocode for aarch64/functions/sve/MaxImplementedVL

```
// MaxImplementedVL()
// ==================

readonly func MaxImplementedVL() => integer
begin
    let max_vl : integer = ImpDefInt("Max implemented VL");
    if IsFeatureImplemented(FEAT_SME_FA64) then
        let max_svl : integer = ImpDefInt("Max implemented SVL");
        return Max(max_vl, max_svl);
    end;

    return max_vl;
    return ImpDefInt("Max implemented VL");
end;
```

Library pseudocode for aarch64/functions/sve/MemNF

```
// MemNF
// =====

func MemNF{size : integer{8, 16, 32, 64, 128}}(address : bits(64),
                                               accdesc : AccessDescriptor
                                              ) => (bits(size), boolean)
begin
    let bytes : integer{} = size DIV 8;
    var value : bits(size);
    var bad : boolean;

    var aligned : boolean = IsAlignedSize{64}(address, bytes);

    if !aligned && AArch64_UnalignedAccessFaults~~AlignmentEnforced~~(accdesc, address, bytes) then
        return (ARBITRARY : bits(size), TRUE);
    end;

    let atomic : boolean = (aligned ||
                            (IsFeatureImplemented(FEAT_LSE2) && AllInSCAGranule(address, bytes)));
~~() then~~
~~        return (ARBITRARY : bits(size), TRUE);~~
~~    end;~~

~~    let atomic : boolean = aligned || bytes == 1;~~

    if !atomic then
        (value[7:0], bad) = MemSingleNF{8}(address, accdesc, aligned);

        if bad then
            return (ARBITRARY : bits(size), TRUE);
        end;

        // For subsequent bytes, if they cross to a new translation page which assigns
        // Device memory type, it is CONSTRAINED UNPREDICTABLE whether an unaligned access
        // will generate an Alignment Fault.
        if !aligned then
            let c : Constraint = ConstrainUnpredictable(Unpredictable_DEVPAGE2);
            assert c IN {Constraint_FAULT, Constraint_NONE};
            if c == Constraint_NONE then aligned = TRUE; end;
        end;

        for i = 1 to bytes-1 do
            (value[i*:8], bad) = MemSingleNF{8}(address+i, accdesc, aligned);

            if bad then
                return (ARBITRARY : bits(size), TRUE);
            end;
        end;
    else
        (value, bad) = MemSingleNF{size}(address, accdesc, aligned);
        if bad then
            return (ARBITRARY : bits(size), TRUE);
        end;
    end;

    if BigEndian(accdesc.acctype) then
        value = BigEndianReverse{size}(value);
    end;

    return (value, FALSE);
end;
```

Library pseudocode for aarch64/functions/sve/MemSingleNF

```
// MemSingleNF
// ===========

func MemSingleNF{size : integer{8, 16, 32, 64, 128}}(address : bits(64),
                                                     accdesc_in : AccessDescriptor,
                                                     aligned : boolean) => (bits(size), boolean)
begin
    assert accdesc_in.acctype == AccessType_SVE;
    assert accdesc_in.nonfault || (accdesc_in.firstfault && !accdesc_in.first);

    let bytes : integer{} = size DIV 8;
    var value : bits(size);
    var memaddrdesc : AddressDescriptor;
    var memstatus : PhysMemRetStatus;
    var accdesc : AccessDescriptor = accdesc_in;
    var fault : FaultRecord = NoFault(accdesc, address);

    // Implementation may suppress NF load for any reason
    if ConstrainUnpredictableBool(Unpredictable_NONFAULT) then
        return (ARBITRARY : bits(size), TRUE);
    end;

    // If the instruction encoding permits tag checking, confer with system register configuration
    // which may override this.
    if accdesc.tagchecked then
        accdesc.tagchecked = AArch64_AccessIsTagChecked(address, accdesc);
    end;

    // MMU or MPU
    memaddrdesc = AArch64_TranslateAddress(address, accdesc, aligned, bytes);

    // Non-fault and first-fault loads from non-Conventional Device memory must not be performed
    // externally. For Conventional memory, it is IMPLEMENTATION DEFINED.
    // Non-fault load from Device memory must not be performed externally
    if memaddrdesc.memattrs.memtype == MemType_Device then
        if (!       return (ARBITRARY : bits(size), TRUE);
    end;

    // Check for aborts or debug exceptions
    if IsFault(memaddrdesc) then
        return (ARBITRARY : bits(size), TRUE);
    end;

    if accdesc.tagchecked then
        let ltag : bits(4) =IsConventionalMemory(memaddrdesc) ||
              ImpDefBool("SVE NF/FF load from Device Conventional memory is suppressed")) then
            return (ARBITRARY : bits(size), TRUE);
        end;
    end;

    // Check for aborts or debug exceptions
    if IsFault(memaddrdesc) then
        return (ARBITRARY : bits(size), TRUE);
    end;

    if accdesc.tagchecked then
        let ltag : bits(4) = AArch64_LogicalAddressTag(address);
        let readcheck : boolean = TRUE;
        fault = AArch64_CheckTag(memaddrdesc, accdesc, readcheck, bytes, ltag);
        if fault.statuscode != Fault_None then
            return (ARBITRARY : bits(size), TRUE);
        end;
    end;

    (memstatus, value) = PhysMemRead{size}(memaddrdesc, accdesc);
    if IsFault(memstatus) then
        let iswrite : boolean = FALSE;
        if IsExternalAbortTakenSynchronously(memstatus, iswrite, memaddrdesc, bytes, accdesc) then
            return (ARBITRARY : bits(size), TRUE);
        end;
        fault.merrorstate = memstatus.merrorstate;
        fault.extflag     = memstatus.extflag;
        fault.statuscode = memstatus.statuscode;
        PendSErrorInterrupt(fault);
    end;

    return (value, FALSE);
end;
```

Library pseudocode for aarch64/functions/syshintop/SystemHintOp

```
// SystemHintOp
// ============
// System Hint instruction types.

type SystemHintOp of enumeration {
    SystemHintOp_NOP,
    SystemHintOp_YIELD,
    SystemHintOp_WFE,
    SystemHintOp_WFI,
    SystemHintOp_SEV,
    SystemHintOp_SEVL,
    SystemHintOp_DGH,
    SystemHintOp_ESB,
    SystemHintOp_PSB,
    SystemHintOp_TSB,
    SystemHintOp_BTI,
    SystemHintOp_WFET,
    SystemHintOp_WFIT,
    SystemHintOp_CLRBHB,
    SystemHintOp_GCSB,
    SystemHintOp_CHKFEAT,
    SystemHintOp_STSHH,
    SystemHintOp_SHUH,
    SystemHintOp_STCPH,
    SystemHintOp_SLBND,SystemHintOp_CSDB
};
SystemHintOp_SRLS,
    SystemHintOp_CSDB
};
```

Library pseudocode for aarch64/functions/sysregisters/GetSPSR_EL1_M4

```
// GetSPSR_EL1_M4()
// ================
// Query the SPSR_EL1.M[4] field.

func GetSPSR_EL1_M4() => bit
begin
    return SPSR_EL1().M[4];
end;
```

Library pseudocode for aarch64/functions/sysregisters/GetSPSR_EL2_M4

```
// GetSPSR_EL2_M4()
// ================
// Query the SPSR_EL2.M[4] field.

func GetSPSR_EL2_M4() => bit
begin
    return SPSR_EL2().M[4];
end;
```

Library pseudocode for aarch64/functions/sysregisters/GetSPSR_EL3_M4

```
// GetSPSR_EL3_M4()
// ================
// Query the SPSR_EL3.M[4] field.

func GetSPSR_EL3_M4() => bit
begin
    return SPSR_EL3().M[4];
end;
```

Library pseudocode for aarch64/functions/system/AArch64_CheckDAIFAccess

```
// AArch64_CheckDAIFAccess()
// =========================
// Check that an AArch64 MSR/MRS access to the DAIF flags is permitted.

func AArch64_CheckDAIFAccess(field : PSTATEField)
begin
    if field IN {    ifPSTATE.EL == EL0 && field IN {PSTATEField_DAIFSet, PSTATEField_DAIFClr} then
        if (        ifEffectivelyAtEL0InHostIsInHost() ||
            (() || SCTLR_EL1().UMA == '0' then
            ifEffectivelyAtEL0NotInHostEL2Enabled() && SCTLR_EL1().UMA == '0')) then() && HCR_EL2().TGE == '1' then
(EL2, 0x18);
            else
                AArch64_SystemAccessTrap(EL1AArch64_SystemAccessTraptoEL1orEL2AArch64_SystemAccessTrap(0x18);
, 0x18);
            end;
        end;
    end;
end;
```

Library pseudocode for aarch64/functions/system/AArch64_ChooseEIRGNonExcludedTag

```
// AArch64_ChooseEIRGNonExcludedTag()
// ==================================
// Return a tag derived from SEED and EIRGSTATE, excluding any tags in the
// given mask.

func AArch64_ChooseEIRGNonExcludedTag(exclude : bits(16)) => bits(4)
begin
    assert GCR_EL1().RRND == '0';

    if IsOnes(exclude) then
        return '0000';
    end;

    var bins : array[[16]] of bits(4);

    var N : integer = 0;
    for i = 0 to 15 do
        if exclude[i] == '0' then
            bins[[N]] = i[3:0];
            N = N + 1;
        end;
    end;

    let seed : bits(16) = RGSR_EL1().RRND0.SEED[15:0];
    let seed : bits(16) = RGSR_EL1().SEED[15:0];
    var eirgstate : integer = UInt(RGSR_EL1().TAG);

    eirgstate = eirgstate + 1;
    if eirgstate >= N then
        eirgstate = 0;
    end;

    let lbin : integer = (eirgstate + (N * UInt(seed)) >> 16) MOD N;

    RGSR_EL1().TAG = eirgstate[3:0];

    let top : bit = seed[5] XOR seed[3] XOR seed[2] XOR seed[0];
    RGSR_EL1().RRND0.SEED[15:0] = top::seed[15:1];
    RGSR_EL1().SEED[15:0] = top::seed[15:1];

    return bins[[lbin]];
end;
```

Library pseudocode for aarch64/functions/system/AArch64_ChooseNonExcludedTagOrZeroLibrary pseudocode for aarch64/functions/system/AArch64_ChooseNonExludedTagOrZero

```
// AArch64_ChooseNonExcludedTagOrZero()
// ====================================
// AArch64_ChooseNonExludedTagOrZero()
// ===================================
// Return a tag derived from the start and the offset values, excluding any
// tags in the given mask, or zero if Allocation Tag access is not enabled.

func AArch64_ChooseNonExcludedTagOrZero(tag : bits(4), offset : bits(4),
                                        exclude : bits(16)) => bits(4)
begin
    if IsMTEEnabled(PSTATE.EL) then
        return AArch64_ChooseNonExcludedTag(tag, offset, exclude);
    else
        return '0000';
    end;
end;
```

Library pseudocode for aarch64/functions/system/AArch64_NextRandomTagBit

```
// AArch64_NextRandomTagBit()
// ==========================
// Generate a random bit suitable for generating a random Allocation Tag.

func AArch64_NextRandomTagBit() => bit
begin
    assert GCR_EL1().RRND == '0';
    let lfsr : bits(16) = RGSR_EL1().RRND0.SEED[15:0];
    let lfsr : bits(16) = RGSR_EL1().SEED[15:0];
    let top : bit = lfsr[5] XOR lfsr[3] XOR lfsr[2] XOR lfsr[0];
    RGSR_EL1().RRND0.SEED[15:0] = top::lfsr[15:1];
    RGSR_EL1().SEED[15:0] = top::lfsr[15:1];
    return top;
end;
```

Library pseudocode for aarch64/functions/system/IsGMEnabledAtEL0

```
// IsGMEnabledAtEL0()
// ==================

func IsGMEnabledAtEL0() => boolean
begin
    if IsFeatureImplemented(FEAT_MTE_GM0) then
        if IsInHost() then
            return IsSCTLR2EL2Enabled() && SCTLR2_EL2().GM0 == '1';
        else
            return IsSCTLR2EL1Enabled() && SCTLR2_EL1().GM0 == '1';
        end;
    else
        return FALSE;
    end;
end;
```

Library pseudocode for aarch64/functions/tlbi/AArch64_TLBI_PAALL

```
// AArch64_TLBI_PAALL()
// ====================
// TLB Invalidate ALL GPT Information.
// Invalidates cached copies of GPT entries from TLBs in the indicated
// Shareability domain.
// Shareabilty domain.
// The invalidation applies to all TLB entries containing GPT information.

func AArch64_TLBI_PAALL(broadcast : Broadcast)
begin
    assert IsFeatureImplemented(FEAT_RME) && PSTATE.EL == EL3;

    var r : TLBIRecord;

    // r.security and r.regime do not apply for TLBI by PA operations
    r.op    = TLBIOp_PAALL;
    r.level = TLBILevel_Any;
    r.attr  = TLBI_AllAttr;

    TLBI(r);
    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r); end;
    return;
end;
```

Library pseudocode for aarch64/functions/tlbi/AArch64_TLBI_RPA

```
// AArch64_TLBI_RPA()
// ==================
// TLB Range Invalidate GPT Information by PA.
// Invalidates cached copies of GPT entries from TLBs in the indicated
// Shareability domain.
// Shareabilty domain.
// The invalidation applies to TLB entries containing GPT information relating
// to the indicated physical address range.
// When the indicated level is
//     TLBILevel_Any  : this applies to TLB entries containing GPT information
//                      from all levels of the GPT walk
//     TLBILevel_Last : this applies to TLB entries containing GPT information
//                      from the last level of the GPT walk

func AArch64_TLBI_RPA(level : TLBILevel, Xt : bits(64), broadcast : Broadcast)
begin
    assert IsFeatureImplemented(FEAT_RME) && PSTATE.EL == EL3;

    var r : TLBIRecord;
    var range_bits : AddressSize;
    var p : AddressSize;

    // r.security and r.regime do not apply for TLBI by PA operations
    r.op    = TLBIOp_RPA;
    r.level = level;
    r.attr  = TLBI_AllAttr;

    // SIZE field
    case Xt[47:44] of
        when '0000' => range_bits = 12; // 4KB
        when '0001' => range_bits = 14; // 16KB
        when '0010' => range_bits = 16; // 64KB
        when '0011' => range_bits = 21; // 2MB
        when '0100' => range_bits = 25; // 32MB
        when '0101' => range_bits = 29; // 512MB
        when '0110' => range_bits = 30; // 1GB
        when '0111' => range_bits = 34; // 16GB
        when '1000' => range_bits = 36; // 64GB
        when '1001' => range_bits = 39; // 512GB
        otherwise =>   return;  // Reserved encoding, no TLB entries are required to be invalidated
    end;

    // If SIZE selects a range smaller than PGS, then PGS is used instead
    case DecodePGS(GPCCR_EL3().PGS) of
        when PGS_4KB =>  p = 12;
        when PGS_16KB => p = 14;
        when PGS_64KB => p = 16;
        otherwise => return;    // Reserved encoding, no TLB entries are required to be invalidated
    end;

    if range_bits < p then
        range_bits = p;
    end;

    var BaseADDR : bits(NUM_PABITS) = Zeros{};
    case GPCCR_EL3().PGS of
        when '00' => BaseADDR[51:12] = Xt[39:0];   // 4KB
        when '10' => BaseADDR[51:14] = Xt[39:2];   // 16KB
        when '01' => BaseADDR[51:16] = Xt[39:4];   // 64KB
    end;

    if IsFeatureImplemented(FEAT_D128) && AArch64_PAMax() == 56 then
        BaseADDR[55:52] = Xt[43:40]; // Extension to Address
    end;

    // The calculation here automatically aligns BaseADDR to the size of
    // the region specified in SIZE. However, the architecture does not
    // require this alignment and if BaseADDR is not aligned to the region
    // specified by SIZE then no entries are required to be invalidated.
    let range_pbits : integer{} = range_bits;
    let start_addr : bits(NUM_PABITS) = BaseADDR AND NOT ZeroExtend{NUM_PABITS}(Ones{range_pbits});
    let end_addr : bits(NUM_PABITS)   = start_addr + ZeroExtend{NUM_PABITS}(Ones{range_pbits});

    // PASpace is not considered in TLBI by PA operations
    r.address     = ZeroExtend{64}(start_addr);
    r.end_address = ZeroExtend{64}(end_addr);

    TLBI(r);
    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r); end;
    return;
end;
```

Library pseudocode for aarch64/functions/tlbi/HasLargeAddress

```
// HasLargeAddress()
// =================
// Returns TRUE if the regime is configured for 52 bit addresses, FALSE otherwise.

func HasLargeAddress(regime : Regime) => boolean
begin
    if !IsFeatureImplemented(FEAT_LPA2) then
        return FALSE;
    end;
    case regime of
        when Regime_EL3 =>
            return TCR_EL3().DS == '1';
        when Regime_EL2 =>
            return TCR_EL2().E2H0.DS == '1';
            return TCR_EL2().DS == '1';
        when Regime_EL20 =>
            return TCR_EL2().E2H1.DS == '1';
            return TCR_EL2().DS == '1';
        when Regime_EL10 =>
            return TCR_EL1().DS == '1';
        otherwise =>
            unreachable;
    end;
end;
```

Library pseudocode for aarch64/translation/vmsa_addrcalc/AArch64_S1SLTTEntryAddress

```
// AArch64_S1SLTTEntryAddress()
// ============================
// Compute the first stage 1 translation table descriptor address within the
// table pointed to by the base at the start level

func AArch64_S1SLTTEntryAddress(level : integer, walkparams : S1TTWParams,
                                ia : bits(64), tablebase : FullAddress) => FullAddress
begin
    let descsizelog2 : integer{} = if walkparams.d128 == '1' then 4 else 3;
    let nstride : integer = 1;
    let start_level_lookup : boolean = TRUE;
    let (lsb, msb) =     // Input Address size
    let iasize : AArch64_TTEntryAddressBoundsAddressSize(level, walkparams.txsz, walkparams.tgx,
                                                         descsizelog2, start_level_lookup, nstride);
    let index : bits(=AArch64_IASize(walkparams.txsz);
    let granulebits : AddressSize   = TGxGranuleBits(walkparams.tgx);
    let descsizelog2 : integer{} = if walkparams.d128 == '1' then 4 else 3;
    let stride : integer{}         = granulebits - descsizelog2;
    let levels : integer           = FINAL_LEVEL - level;

    var index : bits(NUM_PABITS);
    let lsb : AddressSize = (levels*stride + granulebits) as AddressSize;
    let msb : AddressSize = (iasize - 1) as AddressSize) = ZeroExtend{;
    index = ZeroExtend{NUM_PABITS}(ia[msb:lsb]::Zeros{descsizelog2});

    var descaddress : FullAddress;
    descaddress.address = tablebase.address OR index;
    descaddress.paspace = tablebase.paspace;

    return descaddress;
end;
```

Library pseudocode for aarch64/translation/vmsa_addrcalc/AArch64_S1TTEntryAddress

```
// AArch64_S1TTEntryAddress()
// ==========================
// Compute translation table descriptor address within the table pointed to by
// the table base

func AArch64_S1TTEntryAddress{N}(level : integer, walkparams : S1TTWParams, skl : bits(2),
                                 ia : bits(64), tablebase : FullAddress,
                                 descriptor : bits(N)) => FullAddress
begin
    let nstride : integer = if walkparams.d128 == '1' then UInt(skl) + 1 else 1;
    let descsizelog2 : integer{} = if walkparams.d128 == '1' then 4 else 3;
    let start_level_lookup : boolean = FALSE;
    let (lsb, msb) =     // Input Address size
    let iasize : AArch64_TTEntryAddressBoundsAddressSize(level, walkparams.txsz, walkparams.tgx,
                                                         descsizelog2, start_level_lookup, nstride);
    let index : bits(=AArch64_IASize(walkparams.txsz);
    let granulebits : AddressSize = TGxGranuleBits(walkparams.tgx);
    let descsizelog2 : integer{} = if walkparams.d128 == '1' then 4 else 3;
    let stride : integer{}       = granulebits - descsizelog2;
    let levels : integer         = FINAL_LEVEL - level;

    var index : bits(NUM_PABITS);

    let lsb : AddressSize = (levels*stride + granulebits) as AddressSize;
    let nstride : integer = if walkparams.d128 == '1' then UInt(skl) + 1 else 1;
    let msb : AddressSize = ((lsb + (stride * nstride)) - 1) as AddressSize) = ZeroExtend{;
    index = ZeroExtend{NUM_PABITS}(ia[msb:lsb]::Zeros{descsizelog2});

    var descaddress : FullAddress;
    descaddress.address = tablebase.address OR index;
    descaddress.paspace = tablebase.paspace;

    return descaddress;
end;
```

Library pseudocode for aarch64/translation/vmsa_addrcalc/AArch64_S2SLTTEntryAddress

```
// AArch64_S2SLTTEntryAddress()
// ============================
// Compute the first stage 2 translation table descriptor address within the
// table pointed to by the base at the start level

func AArch64_S2SLTTEntryAddress(walkparams : S2TTWParams, ipa : bits(NUM_PABITS),
                                tablebase : FullAddress) => FullAddress
begin
    let startlevel   = AArch64_S2StartLevel(walkparams);
    let descsizelog2 : integer{} = if walkparams.d128 == '1' then 4 else 3;
    let nstride : integer = 1;
    let start_level_lookup : boolean = TRUE;
    let (lsb, msb) =    let iasize       = AArch64_TTEntryAddressBoundsAArch64_IASize(startlevel, walkparams.txsz,
walkparams.tgx,
                                                         descsizelog2, start_level_lookup, nstride);
    let index : bits(-(walkparams.txsz);
    let granulebits  = TGxGranuleBits(walkparams.tgx);
    let descsizelog2 : integer{} = if walkparams.d128 == '1' then 4 else 3;
    let stride : integer{}       = granulebits - descsizelog2;
    let levels : integer         =FINAL_LEVEL - startlevel;

    var index : bits(NUM_PABITS);
    let lsb : AddressSize = (levels*stride + granulebits) as AddressSize;
    let msb : AddressSize = (iasize - 1) as AddressSize) = ZeroExtend{;
    index = ZeroExtend{NUM_PABITS}(ipa[msb:lsb]::Zeros{descsizelog2});

    var descaddress : FullAddress;
    descaddress.address = tablebase.address OR index;
    descaddress.paspace = tablebase.paspace;

    return descaddress;
end;
```

Library pseudocode for aarch64/translation/vmsa_addrcalc/AArch64_S2TTEntryAddress

```
// AArch64_S2TTEntryAddress()
// ==========================
// Compute translation table descriptor address within the table pointed to by
// the table base

func AArch64_S2TTEntryAddress(level : integer, walkparams : S2TTWParams, skl :  bits(2),
                              ipa :  bits(NUM_PABITS), tablebase : FullAddress) => FullAddress
begin
    let nstride : integer = if walkparams.d128 == '1' then UInt(skl) + 1 else 1;
    let descsizelog2 : integer{} = if walkparams.d128 == '1' then 4 else 3;
    let start_level_lookup : boolean = FALSE;
    let (lsb, msb) =     let ipasize : AArch64_TTEntryAddressBoundsAddressSize(level, walkparams.txsz, walkparams.tgx,
                                                descsizelog2, start_level_lookup, nstride);
    let index : bits(=AArch64_IASize(walkparams.txsz);
    let granulebits : AddressSize = TGxGranuleBits(walkparams.tgx);
    let descsizelog2 : integer{} = if walkparams.d128 == '1' then 4 else 3;
    let stride : integer{}       = granulebits - descsizelog2;
    let levels : integer         = FINAL_LEVEL - level;

    var index : bits(NUM_PABITS);

    let lsb : AddressSize = (levels*stride + granulebits) as AddressSize;
    let nstride : integer = if walkparams.d128 == '1' then UInt(skl) + 1 else 1;
    let msb : AddressSize = ((lsb + (stride * nstride)) - 1) as AddressSize) = ZeroExtend(;
    index = ZeroExtend{NUM_PABITS}(ipa[msb:lsb]::Zeros{descsizelog2});

    var descaddress : FullAddress;
    descaddress.address = tablebase.address OR index;
    descaddress.paspace = tablebase.paspace;

    return descaddress;
end;
```

Library pseudocode for aarch64/translation/vmsa_addrcalc/AArch64_TTEntryAddressBounds

```
// AArch64_TTEntryAddressBounds()
// ==============================
// Calculate the input-address bounds used to index a translation table entry

func AArch64_TTEntryAddressBounds(level : integer, txsz : bits(6), tgx : TGx,
                                  descsizelog2 : integer{3,4}, start_level_lookup : boolean,
                                  nstride : integer) => (AddressSize, AddressSize)
begin
    let iasize : AddressSize      = AArch64_IASize(txsz);
    let granulebits : AddressSize = TGxGranuleBits(tgx);
    let stride : integer{}        = granulebits - descsizelog2;
    let levels : integer          = FINAL_LEVEL - level;

    let lsb : AddressSize = (levels * stride + granulebits) as AddressSize;
    var msb : AddressSize;
    if start_level_lookup then
        msb = (iasize - 1) as AddressSize;
    else
        msb = ((lsb + (stride * nstride)) - 1) as AddressSize;
    end;

    return(lsb, msb);
end;
```

Library pseudocode for aarch64/translation/vmsa_faults/AArch64_PermissionsOverlaysAppliedLibrary pseudocode for aarch64/translation/vmsa_faults/AArch64_PermissionOverlaysApplied

```
// AArch64_PermissionsOverlaysApplied()
// ====================================
// AArch64_PermissionOverlaysApplied()
// ===================================
// Returns TRUE if Permission overlays are applied for the given access type.

func AArch64_PermissionsOverlaysApplied(acctype : AccessType) => boolean
begin
    case acctype of
        when AccessType_SPE        => return FALSE;
        when AccessType_TRBE       => return FALSE;
        otherwise                  => return TRUE;
    end;
end;
```

Library pseudocode for aarch64/translation/vmsa_faults/AArch64_S1DirectBasePermissions

```
// AArch64_S1DirectBasePermissions()
// =================================
// Computes the stage 1 direct base permissions

func AArch64_S1DirectBasePermissions(regime : Regime, walkstate : TTWState,
                                     walkparams : S1TTWParams,
                                     accdesc : AccessDescriptor) => S1AccessControls
begin
    var r,  w,  x,  wxn  : bit;
    var pr, pw, px, pwxn : bit;
    var ur, uw, ux, uwxn : bit;
    var permissions : Permissions = walkstate.permissions;
    var s1perms : S1AccessControls;
    // Descriptors marked with DBM set have the effective value of AP[2] cleared.
    // This implies no Permission faults caused by lack of write permissions are
    // reported, and the Dirty bit can be set.
    if permissions.dbm == '1' && walkparams.hd == '1' then
        permissions.ap[2] = '0';
    end;

    if HasUnprivileged(regime) then
        // Apply leaf permissions
        case permissions.ap[2:1] of
            when '00' => (pr,pw,ur,uw) = ('1','1','0','0'); // Privileged access
            when '01' => (pr,pw,ur,uw) = ('1','1','1','1'); // No effect
            when '10' => (pr,pw,ur,uw) = ('1','0','0','0'); // Read-only, privileged access
            when '11' => (pr,pw,ur,uw) = ('1','0','1','0'); // Read-only
        end;

        // Apply hierarchical permissions
        case permissions.ap_table of
            when '00' => (pr,pw,ur,uw) = (pr, pw, ur, uw);  // No effect
            when '01' => (pr,pw,ur,uw) = (pr, pw,'0','0');  // Privileged access
            when '10' => (pr,pw,ur,uw) = (pr,'0', ur,'0');  // Read-only
            when '11' => (pr,pw,ur,uw) = (pr,'0','0','0');  // Read-only, privileged access
        end;

        // Locations writable by unprivileged cannot be executed by privileged
        px = NOT(permissions.pxn OR permissions.pxn_table OR uw);
        ux = NOT(permissions.uxn OR permissions.uxn_table);
        pwxn = walkparams.wxn AND pw AND px;
        uwxn = walkparams.wxn AND uw AND ux;

        if (IsFeatureImplemented(FEAT_PAN) && accdesc.pan && !(regime == Regime_EL10 &&
              walkparams.nv1 == '1')) then
            if (ImpDefBool("SCR_EL3.SIF affects EPAN") && accdesc.ss == SS_Secure &&
                  walkstate.baseaddress.paspace == PAS_NonSecure && walkparams.sif == '1') then
                ux = '0';
            end;

            if (ImpDefBool("Realm EL2&0 regime affects EPAN") && accdesc.ss == SS_Realm &&
                  regime == Regime_EL20 && walkstate.baseaddress.paspace != PAS_Realm) then
                ux = '0';
            end;

            let pan : bit = PSTATE.PAN AND (ur OR uw OR (walkparams.epan AND ux));
            pr = pr AND NOT(pan);
            pw = pw AND NOT(pan);
        end;

    else
        // Apply leaf permissions
        case permissions.ap[2] of
            when '0' => (pr,pw) = ('1','1'); // No effect
            when '1' => (pr,pw) = ('1','0'); // Read-only
        end;

        // Apply hierarchical permissions
        case permissions.ap_table[1] of
            when '0' => (pr,pw) = (pr, pw);  // No effect
            when '1' => (pr,pw) = (pr,'0');  // Read-only
        end;

        px   = NOT(permissions.xn OR permissions.xn_table);
        pwxn = walkparams.wxn AND pw AND px;
    end;

    (r,w,x,wxn) = if accdesc.el == EL0 then (ur,uw,ux,uwxn) else (pr,pw,px,pwxn);

    // Prevent execution from Non-secure space by PE in secure state if SIF is set
    if accdesc.ss == SS_Secure && walkstate.baseaddress.paspace == PAS_NonSecure then
        x = x AND NOT(walkparams.sif);
    end;
    // Prevent execution from non-Root space by Root
    if accdesc.ss == SS_Root && walkstate.baseaddress.paspace != PAS_Root then
        x = '0';
    end;
    // Prevent execution from non-Realm space by Realm EL2 and Realm EL2&0
    if (accdesc.ss == SS_Realm && regime IN {Regime_EL2, Regime_EL20} &&
```

```
            walkstate.baseaddress.paspace != PAS_Realm) then
        x = '0';
    end;


    s1perms.r   = r;
    s1perms.w   = w;
    s1perms.x   = x;
    s1perms.gcs = '0';
    s1perms.wxn = wxn;
    s1perms.overlay =     s1perms.overlay = TRUE;

    return s1perms;
end; AArch64_PermissionsOverlaysApplied(accdesc.acctype);

    return s1perms;
end;
```

Library pseudocode for aarch64/translation/vmsa_faults/AArch64_S1IndirectBasePermissions

```
// AArch64_S1IndirectBasePermissions()
// ===================================
// Computes the stage 1 indirect base permissions

func AArch64_S1IndirectBasePermissions(regime : Regime, walkstate : TTWState,
                                       walkparams : S1TTWParams,
                                       accdesc : AccessDescriptor) => S1AccessControls
begin

    var r,  w,  x,  gcs,  wxn,   overlay : bit;
    var pr, pw, px, pgcs, pwxn, p_overlay : bit;
    var ur, uw, ux, ugcs, uwxn, u_overlay : bit;
    let permissions : Permissions = walkstate.permissions;
    var s1perms : S1AccessControls;

    // Apply privileged indirect permissions
    case permissions.ppi of
        when '0000' => (pr,pw,px,pgcs) = ('0','0','0','0'); // No access
        when '0001' => (pr,pw,px,pgcs) = ('1','0','0','0'); // Privileged read
        when '0010' => (pr,pw,px,pgcs) = ('0','0','1','0'); // Privileged execute
        when '0011' => (pr,pw,px,pgcs) = ('1','0','1','0'); // Privileged read and execute
        when '0100' => (pr,pw,px,pgcs) = ('0','0','0','0'); // Reserved
        when '0101' => (pr,pw,px,pgcs) = ('1','1','0','0'); // Privileged read and write
        when '0110' => (pr,pw,px,pgcs) = ('1','1','1','0'); // Privileged read, write and execute
        when '0111' => (pr,pw,px,pgcs) = ('1','1','1','0'); // Privileged read, write and execute
        when '1000' => (pr,pw,px,pgcs) = ('1','0','0','0'); // Privileged read
        when '1001' =>
            if IsFeatureImplemented(FEAT_GCS) then
                (pr,pw,px,pgcs) = ('1','0','0','1');     // Privileged read and gcs
            else
                (pr,pw,px,pgcs) = ('0','0','0','0');     // Reserved
            end;
        when '1010' => (pr,pw,px,pgcs) = ('1','0','1','0'); // Privileged read and execute
        when '1011' => (pr,pw,px,pgcs) = ('0','0','0','0'); // Reserved
        when '1100' => (pr,pw,px,pgcs) = ('1','1','0','0'); // Privileged read and write
        when '1101' => (pr,pw,px,pgcs) = ('0','0','0','0'); // Reserved
        when '1110' => (pr,pw,px,pgcs) = ('1','1','1','0'); // Privileged read, write and execute
        when '1111' => (pr,pw,px,pgcs) = ('0','0','0','0'); // Reserved
    end;

    p_overlay = NOT(permissions.ppi[3]);
    pwxn = if permissions.ppi == '0110' then '1' else '0';
    var reserved_upi : boolean = FALSE;

    if HasUnprivileged(regime) then
        // Apply unprivileged indirect permissions
        case permissions.upi of
            when '0000' => (ur,uw,ux,ugcs) = ('0','0','0','0'); // No access
            when '0001' => (ur,uw,ux,ugcs) = ('1','0','0','0'); // Unprivileged read
            when '0010' => (ur,uw,ux,ugcs) = ('0','0','1','0'); // Unprivileged execute
            when '0011' => (ur,uw,ux,ugcs) = ('1','0','1','0'); // Unprivileged read and execute
            when '0100' =>
                (ur,uw,ux,ugcs) = ('0','0','0','0');         // Reserved
                reserved_upi    = TRUE;
            when '0101' => (ur,uw,ux,ugcs)=('1','1','0','0');// Unprivileged read and write
            when '0110' => (ur,uw,ux,ugcs)=('1','1','1','0');// Unprivileged read, write and execute
            when '0111' => (ur,uw,ux,ugcs)=('1','1','1','0');// Unprivileged read, write and execute
            when '1000' => (ur,uw,ux,ugcs)=('1','0','0','0');// Unprivileged read
            when '1001' =>
                if IsFeatureImplemented(FEAT_GCS) then
                    (ur,uw,ux,ugcs) = ('1','0','0','1');     // Unprivileged read and gcs
                else
                    (ur,uw,ux,ugcs) = ('0','0','0','0');     // Reserved
                    reserved_upi    = TRUE;
                end;
            when '1010' => (ur,uw,ux,ugcs)=('1','0','1','0');// Unprivileged read and execute
            when '1011' =>
                (ur,uw,ux,ugcs) = ('0','0','0','0');          // Reserved
                reserved_upi    = TRUE;
            when '1100' => (ur,uw,ux,ugcs) =('1','1','0','0');// Unprivileged read and write
            when '1101' =>
                (ur,uw,ux,ugcs) =('0','0','0','0');           // Reserved
                reserved_upi    = TRUE;
            when '1110' => (ur,uw,ux,ugcs) =('1','1','1','0');// Unprivileged read, write
                                                               // and execute
            when '1111' =>
                (ur,uw,ux,ugcs) = ('0','0','0','0');          // Reserved
                reserved_upi    = TRUE;

        end;

        u_overlay = NOT(permissions.upi[3]);
        uwxn = if permissions.upi == '0110' then '1' else '0';

        // If the decoded permissions has either px or pgcs along with either uw or ugcs,
        // then all effective Stage 1 Base Permissions are set to 0
        if ((px == '1' || pgcs == '1') && (uw == '1' || ugcs == '1')) then
            (pr,pw,px,pgcs) = ('0','0','0','0');
            (ur,uw,ux,ugcs) = ('0','0','0','0');
```

```
        end;
    end;

    // Prevent execution from Non-secure space by PE in secure state
    if accdesc.ss == SS_Secure && walkstate.baseaddress.paspace == PAS_NonSecure then
        pgcs = '0';
        if HasUnprivileged(regime) then ugcs = '0'; end;

        if walkparams.sif == '1' then
            px = '0';
            if HasUnprivileged(regime) then ux = '0'; end;
        end;
    end;

    // Prevent execution from non-Root space by Root
    if accdesc.ss == SS_Root && walkstate.baseaddress.paspace != PAS_Root then
        (px, pgcs)   = ('0','0');
        if HasUnprivileged(regime) then
            (ux, ugcs)   = ('0','0');
        end;
    end;

    // Prevent execution from non-Realm space by Realm EL2 and Realm EL2&0
    if (accdesc.ss == SS_Realm && regime IN {Regime_EL2, Regime_EL20} &&
          walkstate.baseaddress.paspace != PAS_Realm) then
        (px, pgcs)   = ('0','0');
        if HasUnprivileged(regime) then
            (ux, ugcs)   = ('0','0');
        end;
    end;

    if (HasUnprivileged(regime) && IsFeatureImplemented(FEAT_PAN) && accdesc.pan &&
          !(regime == Regime_EL10 && walkparams.nv1 == '1') &&
          PSTATE.PAN == '1' && !IsZero(permissions.upi)) then
        if reserved_upi then
            if ImpDefBool("PAN applies when S1UnprivBasePerm is reserved") then
                (pr,pw) = ('0','0');
            end;
        elsif (!IsZero(ur::uw::ux::ugcs) ||
                 ImpDefBool("PAN applies when S1UnprivBasePerms are removed")) then
            (pr,pw) = ('0','0');
        end;
    end;

    if accdesc.el == EL0 then
        (r,w,x,gcs,wxn,overlay) = (ur,uw,ux,ugcs,uwxn,u_overlay);
    else
        (r,w,x,gcs,wxn,overlay) = (pr,pw,px,pgcs,pwxn,p_overlay);
    end;

    s1perms.r       = r;
    s1perms.w       = w;
    s1perms.x       = x;
    s1perms.gcs     = gcs;
    s1perms.wxn     = wxn;
    s1perms.overlay =     s1perms.overlay = overlay == '1';

    return s1perms;
end; AArch64_PermissionsOverlaysApplied(accdesc.acctype) && overlay == '1';

    return s1perms;
end;
```

Library pseudocode for aarch64/translation/vmsa_faults/AArch64_S2CheckPermissions

```
// AArch64_S2CheckPermissions()
// ============================
// Verifies memory access with available permissions.

func AArch64_S2CheckPermissions(fault_in : FaultRecord, walkstate : TTWState,
                                walkparams : S2TTWParams, ipa : AddressDescriptor,
                                accdesc : AccessDescriptor) => (FaultRecord, boolean)
begin
    let memtype : MemType = walkstate.memattrs.memtype;
    let permissions : Permissions = walkstate.permissions;
    var fault : FaultRecord = fault_in;
    let s2perms : S2AccessControls = AArch64_S2ComputePermissions(permissions, walkparams,
                                                                  accdesc);

    var r,  w : bit;
    var or, ow : bit;

    if accdesc.acctype == AccessType_TTW then
        r = s2perms.r_mmu;
        w = s2perms.w_mmu;
        or = s2perms.or_mmu;
        ow = s2perms.ow_mmu;
    elsif accdesc.rcw then
        r = s2perms.r_rcw;
        w = s2perms.w_rcw;
        or = s2perms.or_rcw;
        ow = s2perms.ow_rcw;
    else
        r = s2perms.r;
        w = s2perms.w;
        or = s2perms.or;
        ow = s2perms.ow;
    end;

    if accdesc.acctype == AccessType_TTW then
        if (accdesc.toplevel && accdesc.varange == VARange_LOWER &&
              ((walkparams.tl0 == '1' && s2perms.toplevel0 == '0') ||
              (walkparams.tl1 == '1' && s2perms.[toplevel1,toplevel0] == '10'))) then
            fault.statuscode = Fault_Permission;
            fault.toplevel   = TRUE;
        elsif (accdesc.toplevel && accdesc.varange == VARange_UPPER &&
                ((walkparams.tl1 == '1' && s2perms.toplevel1 == '0') ||
                (walkparams.tl0 == '1' && s2perms.[toplevel1,toplevel0] == '01'))) then
            fault.statuscode = Fault_Permission;
            fault.toplevel   = TRUE;
        // Stage 2 Permission fault due to AssuredOnly check
        elsif (walkstate.s2assuredonly == '1' && !ipa.s1assured) then
                fault.statuscode  = Fault_Permission;
                fault.assuredonly = TRUE;

        elsif s2perms.overlay && or == '0' then
            fault.statuscode = Fault_Permission;
            fault.overlay    = TRUE;
        elsif accdesc.write && s2perms.overlay && ow == '0' then
            fault.statuscode = Fault_Permission;
            fault.overlay    = TRUE;

        elsif walkparams.ptw == '1' && memtype == MemType_Device then
            fault.statuscode = Fault_Permission;
        // Prevent translation table walks in Non-secure space by Realm state
        elsif accdesc.ss == SS_Realm && walkstate.baseaddress.paspace != PAS_Realm then
            fault.statuscode = Fault_Permission;
        elsif r == '0' then
            fault.statuscode = Fault_Permission;
        elsif accdesc.write && w == '0' then
            fault.statuscode = Fault_Permission;
            fault.hdbssf = walkparams.hdbss == '1' && !CanAppendToHDBSS() && permissions.dbm == '1';
        elsif (accdesc.write && walkparams.s2pie == '1' && permissions.s2dirty == '0' &&
                 (walkparams.hd != '1' ||
                 (accdesc.acctype != AccessType_AT &&
                  walkparams.hdbss == '1' && !CanAppendToHDBSS()))) then
            fault.statuscode = Fault_Permission;
            fault.dirtybit   = TRUE;
            fault.hdbssf = walkparams.hdbss == '1' && !CanAppendToHDBSS();

        end;

    // Stage 2 Permission fault due to AssuredOnly check
    elsif ((walkstate.s2assuredonly == '1' && !ipa.s1assured) ||
            (walkstate.s2assuredonly != '1' && IsFeatureImplemented(FEAT_GCS) &&
             VTCR_EL2().GCSH == '1' && accdesc.acctype == AccessType_GCS && accdesc.el != EL0)) then
        fault.statuscode  = Fault_Permission;
        fault.assuredonly = TRUE;

    elsif accdesc.acctype == AccessType_IFETCH then
        if s2perms.overlay && s2perms.ox == '0' then
            fault.statuscode = Fault_Permission;
            fault.overlay    = TRUE;
        elsif (memtype == MemType_Device &&
```

```
            ConstrainUnpredictable(Unpredictable_INSTRDEVICE) == Constraint_FAULT) then
            fault.statuscode = Fault_Permission;

        // Prevent execution from Non-secure space by Realm state
        elsif accdesc.ss == SS_Realm && walkstate.baseaddress.paspace != PAS_Realm then
            fault.statuscode = Fault_Permission;
        elsif s2perms.x == '0' then
            fault.statuscode = Fault_Permission;
        end;

    elsif accdesc.acctype == AccessType_DC then
        if accdesc.cacheop == CacheOp_Invalidate then
            if !ELUsingAArch32(EL1) && s2perms.overlay && ow == '0' then
                fault.statuscode = Fault_Permission;
                fault.overlay    = TRUE;
            elsif !ELUsingAArch32(EL1) && w == '0' then
                fault.statuscode = Fault_Permission;
            elsif (walkparams.hd != '1' && walkparams.s2pie == '1' &&
                     permissions.s2dirty == '0') then
                fault.statuscode = Fault_Permission;
                fault.dirtybit   = TRUE;
            end;
        elsif !ELUsingAArch32(EL1) && accdesc.el == EL0 && s2perms.overlay && or == '0' then
            fault.statuscode = Fault_Permission;
            fault.overlay    = TRUE;
        elsif (walkparams.cmow == '1' && accdesc.cacheop == CacheOp_CleanInvalidate &&
                s2perms.overlay && ow == '0') then
            fault.statuscode = Fault_Permission;
            fault.overlay    = TRUE;
        elsif !ELUsingAArch32(EL1) && accdesc.el == EL0 && r == '0' then
            fault.statuscode = Fault_Permission;
        elsif (walkparams.cmow == '1' && accdesc.cacheop == CacheOp_CleanInvalidate &&
                w == '0') then
            fault.statuscode = Fault_Permission;
        elsif (walkparams.cmow == '1' && accdesc.cacheop == CacheOp_CleanInvalidate &&
                 walkparams.hd != '1' && walkparams.s2pie == '1' && permissions.s2dirty == '0') then
            fault.statuscode = Fault_Permission;
            fault.dirtybit   = TRUE;
        end;

    elsif accdesc.acctype == AccessType_IC then
        if (!ELUsingAArch32(EL1) && accdesc.el == EL0 && s2perms.overlay && or == '0' &&
                ImpDefBool("Permission fault on EL0 IC_IVAU execution")) then
            fault.statuscode = Fault_Permission;
            fault.overlay    = TRUE;
        elsif walkparams.cmow == '1' && s2perms.overlay && ow == '0' then
            fault.statuscode = Fault_Permission;
            fault.overlay    = TRUE;
        elsif (!ELUsingAArch32(EL1) && accdesc.el == EL0 &&  r == '0' &&
              ImpDefBool("Permission fault on EL0 IC_IVAU execution")) then
            fault.statuscode = Fault_Permission;
        elsif walkparams.cmow == '1' && w == '0' then
            fault.statuscode = Fault_Permission;
        elsif (walkparams.cmow == '1' && walkparams.hd != '1' && walkparams.s2pie == '1' &&
                 permissions.s2dirty == '0') then
            fault.statuscode = Fault_Permission;
            fault.dirtybit   = TRUE;
        end;

    elsif accdesc.read && s2perms.overlay && or == '0' then
        fault.statuscode = Fault_Permission;
        fault.overlay    = TRUE;
        fault.write      = FALSE;
    elsif accdesc.write && s2perms.overlay && ow == '0' then
        fault.statuscode = Fault_Permission;
        fault.overlay    = TRUE;
        fault.write      = TRUE;
    elsif accdesc.read && r == '0' then
        fault.statuscode = Fault_Permission;
        fault.write      = FALSE;
    elsif accdesc.write && w == '0' then
        fault.statuscode = Fault_Permission;
        fault.write      = TRUE;
        fault.hdbssf     = walkparams.hdbss == '1' && !CanAppendToHDBSS() && permissions.dbm == '1';
    elsif (IsFeatureImplemented(FEAT_MTE_PERM) &&
             ((accdesc.tagchecked &&
               AArch64_EffectiveTCF(accdesc.el, accdesc.read) != TCFType_Ignore) ||
              accdesc.tagaccess) &&
             ipa.memattrs.tags == MemTag_AllocationTagged &&
             permissions.s2tag_na == '1' && S2DCacheEnabled()) then
        fault.statuscode = Fault_Permission;
        fault.tagaccess  = TRUE;
        fault.write      = accdesc.tagaccess && accdesc.write;
    elsif (accdesc.write &&
            (walkparams.hd != '1' ||
            (walkparams.hdbss == '1' && accdesc.acctype != ~~(walkparams.hd != '1' || (walkparams.hdbss == '1' && !~~ AccessType_AT &&
                !CanAppendToHDBSS())) &&
             walkparams.s2pie == '1' && permissions.s2dirty == '0') then
```

```
        fault.statuscode = Fault_Permission;
        fault.dirtybit   = TRUE;
        fault.write      = TRUE;
        fault.hdbssf     = (walkparams.hdbss == '1' && accdesc.acctype != AccessType_AT &&
                            !;
        fault.dirtybit   = TRUE;
        fault.write      = TRUE;
        fault.hdbssf     = walkparams.hdbss == '1' && !CanAppendToHDBSS());
();
    end;
    // MRO* allows only RCW and MMU writes
    var mro : boolean;
    if s2perms.overlay then
        mro = (s2perms.[w,w_rcw,w_mmu] AND s2perms.[ow,ow_rcw,ow_mmu]) == '011';
    else
        mro = s2perms.[w,w_rcw,w_mmu] == '011';
    end;

    return (fault, mro);
end;
```

Library pseudocode for aarch64/translation/vmsa_faults/AArch64_S2DirectBasePermissions

```
// AArch64_S2DirectBasePermissions()
// =================================
// Computes the stage 2 direct base permissions.

func AArch64_S2DirectBasePermissions(permissions : Permissions,
                                     accdesc : AccessDescriptor,
                                     walkparams : S2TTWParams) => S2AccessControls
begin
    var s2perms : S2AccessControls;
    var w : bit;
    let r : bit = permissions.s2ap[0];
    if permissions.s2ap[1] == '1' then
        w = '1';
    // Descriptors marked with DBM set have the effective value of S2AP[1] set.
    // This implies no Permission faults caused by lack of write permissions are
    // reported, and the Dirty bit can be set.
    elsif permissions.dbm == '1' && walkparams.hd == '1' then
        // An update occurs here, conditional to being able to append to HDBSS
        if (walkparams.hdbss == '1' &&
              accdesc.acctype IN !{        if walkparams.hdbss == '1' then
            w = ifAccessType_DC, AccessType_IC, AccessType_AT}) then
            w = if CanAppendToHDBSS() then '1' else '0';
        else
            w = '1';
        end;
    else
        w = '0';
    end;

    var px, ux : bit;
    case (permissions.s2xn::permissions.s2xnx) of
        when '00' => (px,ux) = ('1','1');
        when '01' => (px,ux) = ('0','1');
        when '10' => (px,ux) = ('0','0');
        when '11' => (px,ux) = ('1','0');
    end;

    let x : bit = if accdesc.el == EL0 then ux else px;
    s2perms.r = r;
    s2perms.w = w;
    s2perms.x = x;
    s2perms.r_rcw = r;
    s2perms.w_rcw = w;
    s2perms.r_mmu = r;
    s2perms.w_mmu = w;
    s2perms.toplevel0 = '0';
    s2perms.toplevel1 = '0';
    s2perms.overlay = FALSE;

    return s2perms;
end;
```

Library pseudocode for aarch64/translation/vmsa_translation/AArch64_S1DisabledOutput

```
// AArch64_S1DisabledOutput()
// ==========================
// Map the VA to IPA/PA and assign default memory attributes

func AArch64_S1DisabledOutput(fault_in : FaultRecord, regime : Regime,
                              va_in :  bits(64), accdesc : AccessDescriptor,
                              aligned : boolean) => (FaultRecord, AddressDescriptor)
begin
    var va : bits(64) = va_in;
    let walkparams : S1TTWParams = AArch64_GetS1TTWParams(regime, accdesc.el, accdesc.ss, va);
    var fault : FaultRecord = fault_in;

    if accdesc.acctype ==     // No memory page is guarded when stage 1 address translation is disabled
AccessType_IFETCH then
        // No memory page is guarded when stage 1 address translation is disabled
        SetInGuardedPage(FALSE);
    end;

    // Output Address
    var oa : FullAddress;
    oa.address = va[NUM_PABITS-1:0];
    case accdesc.ss of
        when SS_Secure =>    oa.paspace = PAS_Secure;
        when SS_NonSecure => oa.paspace = PAS_NonSecure;
        when SS_Root =>      oa.paspace = PAS_Root;
        when SS_Realm =>     oa.paspace = PAS_Realm;
    end;

    var memattrs : MemoryAttributes;
    if regime == Regime_EL10 && EL2Enabled() && walkparams.dc == '1' then
        var default_cacheability : MemAttrHints;
        default_cacheability.attrs      = MemAttr_WB;
        default_cacheability.hints      = MemHint_RWA;
        default_cacheability.transient = FALSE;

        memattrs.memtype      = MemType_Normal;
        memattrs.outer        = default_cacheability;
        memattrs.inner        = default_cacheability;
        memattrs.shareability = Shareability_NSH;
        if walkparams.dct == '1' then
            memattrs.tags     = MemTag_AllocationTagged;
        elsif IsFeatureImplemented(FEAT_MTE_CANONICAL_TAGS) && walkparams.mtx == '1' then
            memattrs.tags     = MemTag_CanonicallyTagged;
        else
            memattrs.tags     = MemTag_Untagged;
        end;
        memattrs.xs           = '0';
    elsif accdesc.acctype == AccessType_IFETCH then
        var i_cache_attr : MemAttrHints;
        if AArch64_S1ICacheEnabled(regime) then
            i_cache_attr.attrs     = MemAttr_WT;
            i_cache_attr.hints     = MemHint_RA;
            i_cache_attr.transient = FALSE;
        else
            i_cache_attr.attrs     = MemAttr_NC;
        end;

        memattrs.memtype      = MemType_Normal;
        memattrs.outer        = i_cache_attr;
        memattrs.inner        = i_cache_attr;
        memattrs.shareability = Shareability_OSH;
        memattrs.tags         = MemTag_Untagged;
        memattrs.xs           = '1';
    elsif accdesc.acctype == AccessType_SPE && EffectivePMBLIMITR_EL1_nVM() == '1' then
        memattrs = S1DecodeMemAttrs(PMBMAR_EL1().Attr, PMBMAR_EL1().SH, TRUE,
                                    walkparams, accdesc.acctype);
    elsif accdesc.acctype == AccessType_TRBE && EffectiveTRBLIMITR_EL1_nVM() == '1' then
        memattrs = S1DecodeMemAttrs(TRBMAR_EL1().Attr, TRBMAR_EL1().SH, TRUE,
                                    walkparams, accdesc.acctype);
    else
        memattrs.memtype      = MemType_Device;
        memattrs.device       = DeviceType_nGnRnE;
        memattrs.shareability = Shareability_OSH;
        if IsFeatureImplemented(FEAT_MTE_CANONICAL_TAGS) && walkparams.mtx == '1' then
            memattrs.tags = MemTag_CanonicallyTagged;
        else
            memattrs.tags = MemTag_Untagged;
        end;
        memattrs.xs           = '1';
    end;
    memattrs.notagaccess = FALSE;

    if walkparams.mtx == '1' && walkparams.tbi == '0' && accdesc.acctype != AccessType_IFETCH then
        // For the purpose of the checks in this function, the MTE tag bits are ignored.
        va[59:56] = if HasUnprivileged(regime) then Replicate{4}(va[55]) else '0000';
    end;

    fault.level = 0;
    let addrtop : AddressSize = AArch64_AddrTop(walkparams.tbid, accdesc.acctype,
```

```
                                                walkparams.tbi);
    let pamax : AddressSize = AArch64_PAMax();

    let oamax : AddressSize = AArch64_OAMax();
    var addrchecked : boolean = FALSE;

    if accdesc.acctype == AccessType_SPE && EffectivePMBLIMITR_EL1_nVM() == '1' then
        addrchecked = TRUE;
        if oamax > pamax && !IsZero(va[(oamax-1):pamax])then
            fault.statuscode = Fault_AddressSize;
        elsif !IsZero(va[63:oamax]) then
            if ConstrainUnpredictableBool(Unpredictable_PMBPTR_Fault_AddressSize)then
                fault.statuscode = Fault_AddressSize;
            else
                oa.address = ZeroExtend{NUM_PABITS}(oa.address[oamax-1:0]);
            end;
        end;
    end;

    if accdesc.acctype == AccessType_TRBE && EffectiveTRBLIMITR_EL1_nVM() == '1' then
        addrchecked = TRUE;
        if oamax > pamax && !IsZero(va[(oamax-1):pamax])then
            fault.statuscode = Fault_AddressSize;
        elsif !IsZero(va[63:oamax]) then
            if ConstrainUnpredictableBool(Unpredictable_TRBPTR_Fault_AddressSize)then
                fault.statuscode = Fault_AddressSize;
            else
                oa.address = ZeroExtend{NUM_PABITS}(oa.address[oamax-1:0]);
            end;
        end;
    end;

    if fault.statuscode == Fault_None then
        if !addrchecked && addrtop > pamax && !IsZero(va[addrtop:pamax]) then
            fault.statuscode = Fault_AddressSize;
        elsif AArch64_S1HasAlignmentFaultDueToMemType(regime, accdesc, aligned, walkparams.ntlsmd,
                                                      memattrs) then
            fault.statuscode = Fault_Alignment;
        elsif ((accdesc.exclusive || accdesc.atomicop) &&
                !(regime == Regime_EL10 && EL2Enabled() &&
                  IsFeatureImplemented(FEAT_HCRMAC) && HCR_EL2().DC == '1') &&
() && HCR_EL2().DC == '1') &&
                ConstrainUnpredictableBool(Unpredictable_Atomic_MMU_IMPDEF_FAULT)) then
            fault.statuscode = Fault_Exclusive;
        end;
    end;

    if fault.statuscode != Fault_None then
        return (fault, ARBITRARY : AddressDescriptor);
    else
        var ipa : AddressDescriptor = CreateAddressDescriptor(va_in, oa, memattrs, accdesc);
        ipa.mecid = AArch64_S1DisabledOutputMECID(walkparams, regime, ipa.paddress.paspace);
        return (fault, ipa);
    end;
end;
```

Library pseudocode for aarch64/translation/vmsa_walkparams/AArch64_NSS2TTWParams

```
// AArch64_NSS2TTWParams()
// =======================
// Gather walk parameters specific for Non-secure stage 2 translation

func AArch64_NSS2TTWParams(s1aarch64 : boolean) => S2TTWParams
begin
    var walkparams : S2TTWParams;

    if IsFeatureImplemented(FEAT_HCRMAC) then
        walkparams.vm = HCR_EL2().VM OR HCR_EL2().DC;
    else
        walkparams.vm = HCR_EL2().VM;
    end;
    walkparams.vm   = HCR_EL2().VM OR HCR_EL2().DC;
    walkparams.tgx  = AArch64_S2DecodeTG0(VTCR_EL2().TG0);
    walkparams.txsz = VTCR_EL2().T0SZ;
    walkparams.ps   = VTCR_EL2().PS;
    walkparams.irgn = VTCR_EL2().IRGN0;
    walkparams.orgn = VTCR_EL2().ORGN0;
    walkparams.sh   = VTCR_EL2().SH0;
    walkparams.ee   = SCTLR_EL2().EE;
    walkparams.d128 = if IsFeatureImplemented(FEAT_D128)  then VTCR_EL2().D128 else '0';
    if walkparams.d128 == '1' then
        walkparams.skl = VTTBR_EL2().SKL;
    else
        walkparams.sl0 = VTCR_EL2().SL0;
    end;

    if HCR_EL2().TGE == '0' then
        walkparams.ptw = if IsFeatureImplemented(FEAT_HCRMAC) then HCR_EL2().PTW else '1';
    else
        walkparams.ptw = '0';
    end;
    if IsFeatureImplemented(FEAT_S2FWB) then
        walkparams.fwb = if IsFeatureImplemented(FEAT_HCRMAC) then HCR_EL2().FWB else '1';
    else
        walkparams.fwb = '0';
    end;
    walkparams.ptw = if HCR_EL2().TGE == '0' then HCR_EL2().PTW else '0';
    walkparams.fwb = if IsFeatureImplemented(FEAT_S2FWB) then HCR_EL2().FWB else '0';
    walkparams.ha = if IsFeatureImplemented(FEAT_HAF) then VTCR_EL2().HA else '0';
    if IsFeatureImplemented(FEAT_HAFDBS) && walkparams.ha == '1' then
        walkparams.hd = VTCR_EL2().HD;
    else
        walkparams.hd = '0';
    end;
    if walkparams.tgx IN {TGx_4KB, TGx_16KB} && IsFeatureImplemented(FEAT_LPA2) then
        walkparams.ds = VTCR_EL2().DS;
    else
        walkparams.ds = '0';
    end;
    if walkparams.tgx == TGx_4KB && IsFeatureImplemented(FEAT_LPA2) then
        walkparams.sl2 = VTCR_EL2().SL2 AND VTCR_EL2().DS;
    else
        walkparams.sl2 = '0';
    end;
    walkparams.cmow = (if IsFeatureImplemented(FEAT_CMOW) && IsHCRXEL2Enabled() then HCRX_EL2().CMOW
                       else '0');
    if walkparams.d128 == '1' then
        walkparams.s2pie = '1';
    else
        walkparams.s2pie = if IsFeatureImplemented(FEAT_S2PIE) then VTCR_EL2().S2PIE else '0';
    end;
    if IsFeatureImplemented(FEAT_S2PIE) then
        if !(HaveEL(EL3) && SCR_EL3().PIEn == '0' &&
             ImpDefBool("SCR_EL3.PIEn forces PIR_ELx or POR_ELx to zero")) then
            walkparams.s2pir = S2PIR_EL2() as S2PIRType;
        else
            walkparams.s2pir = Zeros{64} as S2PIRType;
        end;
    end;
    if IsFeatureImplemented(FEAT_THE) && walkparams.d128 != '1' then
        walkparams.assuredonly = VTCR_EL2().AssuredOnly;
    else
        walkparams.assuredonly = '0';
    end;
    walkparams.tl0   = if IsFeatureImplemented(FEAT_THE) then VTCR_EL2().TL0 else '0';
    walkparams.tl1   = if IsFeatureImplemented(FEAT_THE) then VTCR_EL2().TL1 else '0';
    if IsFeatureImplemented(FEAT_HAFT) && walkparams.ha == '1' then
        walkparams.haft = VTCR_EL2().HAFT;
    else
        walkparams.haft = '0';
    end;
    if (IsFeatureImplemented(FEAT_HDBSS) && walkparams.hd == '1' &&
        (!HaveEL(EL3) || SCR_EL3().HDBSSEn == '1')) then
        walkparams.hdbss = VTCR_EL2().HDBSS;
    else
        walkparams.hdbss = '0';
    end;
```

```
    return walkparams;
end;
```

Library pseudocode for aarch64/translation/vmsa_walkparams/AArch64_S1TTWParamsEL10

```
// AArch64_S1TTWParamsEL10()
// =========================
// Gather stage 1 translation table walk parameters for EL1&0 regime
// (with EL2 enabled or disabled)

readonly func AArch64_S1TTWParamsEL10(el : bits(2), varange : VARange) => S1TTWParams
begin
    var walkparams : S1TTWParams;

    if IsFeatureImplemented(FEAT_D128) && IsTCR2EL1Enabled() then
        walkparams.d128 = TCR2_EL1().D128;
    else
        walkparams.d128 = '0';
    end;
    let nvs : bits(3) = EffectiveHCR_EL2_NVx();
    walkparams.nv1 = nvs[1] AND nvs[0];
    walkparams.nv1 = nvs[1];

    if IsFeatureImplemented(FEAT_AIE) then
        walkparams.mair2 = MAIR2_EL1() as MAIRType;
    end;
    walkparams.aie  = (if IsFeatureImplemented(FEAT_AIE) && IsTCR2EL1Enabled() then TCR2_EL1().AIE
                       else '0');
    if walkparams.d128 == '1' then
        walkparams.pie = '1';
    else
        walkparams.pie = (if IsFeatureImplemented(FEAT_S1PIE) &&
                          IsTCR2EL1Enabled() then TCR2_EL1().PIE else '0');
    end;
    if IsFeatureImplemented(FEAT_S1PIE) then
        if !(HaveEL(EL3) && SCR_EL3().PIEn == '0' &&
               ImpDefBool("SCR_EL3.PIEn forces PIR_ELx or POR_ELx to zero")) then
            walkparams.pir = PIR_EL1() as S1PIRType;
            if walkparams.nv1 == '1' then
                walkparams.pire0 = Zeros{64} as S1PIRType;
            else
                walkparams.pire0 = PIRE0_EL1() as S1PIRType;
            end;
        else
            walkparams.pir   = Zeros{64} as S1PIRType;
            walkparams.pire0 = Zeros{64} as S1PIRType;
        end;
    end;

    if varange == VARange_LOWER then
        walkparams.tgx   = AArch64_S1DecodeTG0(TCR_EL1().TG0);
        walkparams.txsz  = TCR_EL1().T0SZ;
        walkparams.irgn  = TCR_EL1().IRGN0;
        walkparams.orgn  = TCR_EL1().ORGN0;
        walkparams.sh    = TCR_EL1().SH0;
        walkparams.tbi   = TCR_EL1().TBI0;

        walkparams.nfd   = if IsFeatureImplemented(FEAT_SVE)   then TCR_EL1().NFD0  else '0';
        walkparams.tbid  = if IsFeatureImplemented(FEAT_PAuth) then TCR_EL1().TBID0 else '0';
        walkparams.e0pd  = if IsFeatureImplemented(FEAT_E0PD)  then TCR_EL1().E0PD0 else '0';
        walkparams.hpd   = if IsFeatureImplemented(FEAT_HPDS)  then TCR_EL1().HPD0  else '0';
        if walkparams.hpd == '0' then
            if walkparams.aie == '1' then walkparams.hpd = '1'; end;
            if walkparams.pie == '1' then walkparams.hpd = '1'; end;
            if (AArch64_S1POEnabled(Regime_EL10) ||
                  AArch64_S1E0POEnabled(Regime_EL10, walkparams.nv1)) then walkparams.hpd = '1';
            end;
        end;
        if (IsFeatureImplemented(FEAT_MTE_NO_ADDRESS_TAGS) ||
              IsFeatureImplemented(FEAT_MTE_CANONICAL_TAGS)) then
            walkparams.mtx  = TCR_EL1().MTX0;
        else
            walkparams.mtx  = '0';
        end;
        walkparams.skl   = if walkparams.d128 == '1' then TTBR0_EL1().SKL else '00';
        walkparams.disch = if walkparams.d128 == '1' then TCR2_EL1().DisCH0 else '0';
        if IsFeatureImplemented(FEAT_ASID2) && IsTCR2EL1Enabled() then
            walkparams.fng = TCR2_EL1().FNG0;
        else
            walkparams.fng = '0';
        end;
        if IsFeatureImplemented(FEAT_THE) && IsTCR2EL1Enabled() then
            walkparams.fngna = TCR2_EL1().FNGNA0;
        else
            walkparams.fngna = '0';
        end;
    else
        walkparams.tgx   = AArch64_S1DecodeTG1(TCR_EL1().TG1);
        walkparams.txsz  = TCR_EL1().T1SZ;
        walkparams.irgn  = TCR_EL1().IRGN1;
        walkparams.orgn  = TCR_EL1().ORGN1;
        walkparams.sh    = TCR_EL1().SH1;
        walkparams.tbi   = TCR_EL1().TBI1;
```

```
        walkparams.nfd   = if IsFeatureImplemented(FEAT_SVE)   then TCR_EL1().NFD1  else '0';
        walkparams.tbid  = if IsFeatureImplemented(FEAT_PAuth) then TCR_EL1().TBID1 else '0';
        walkparams.e0pd  = if IsFeatureImplemented(FEAT_E0PD)  then TCR_EL1().E0PD1 else '0';
        walkparams.hpd   = if IsFeatureImplemented(FEAT_HPDS)  then TCR_EL1().HPD1  else '0';
        if walkparams.hpd == '0' then
            if walkparams.aie == '1' then walkparams.hpd = '1'; end;
            if walkparams.pie == '1' then walkparams.hpd = '1'; end;
            if (AArch64_S1POEnabled(Regime_EL10) ||
                  AArch64_S1E0POEnabled(Regime_EL10, walkparams.nv1)) then walkparams.hpd = '1';
            end;
        end;
        if (IsFeatureImplemented(FEAT_MTE_NO_ADDRESS_TAGS) ||
              IsFeatureImplemented(FEAT_MTE_CANONICAL_TAGS)) then
            walkparams.mtx  = TCR_EL1().MTX1;
        else
            walkparams.mtx  = '0';
        end;
        walkparams.skl   = if walkparams.d128 == '1' then TTBR1_EL1().SKL else '00';
        walkparams.disch = if walkparams.d128 == '1' then TCR2_EL1().DisCH1 else '0';
        if IsFeatureImplemented(FEAT_ASID2) && IsTCR2EL1Enabled() then
            walkparams.fng = TCR2_EL1().FNG1;
        else
            walkparams.fng = '0';
        end;
        if IsFeatureImplemented(FEAT_THE) && IsTCR2EL1Enabled() then
            walkparams.fngna = TCR2_EL1().FNGNA1;
        else
            walkparams.fngna = '0';
        end;
    end;

    walkparams.mair = MAIR_EL1() as MAIRType;
    walkparams.wxn  = SCTLR_EL1().WXN;
    walkparams.ps   = TCR_EL1().IPS;
    walkparams.ee   = SCTLR_EL1().EE;
    if (HaveEL(EL3) && (!IsFeatureImplemented(FEAT_RME) || IsFeatureImplemented(FEAT_SEL2))) then
        walkparams.sif = SCR_EL3().SIF;
    else
        walkparams.sif = '0';
    end;

    if EL2Enabled() then
        walkparams.dc  = if IsFeatureImplemented(FEAT_HCRMAC) then HCR_EL2().DC else '0';
        if IsFeatureImplemented(FEAT_MTE2) && IsFeatureImplemented(FEAT_HCRMAC) then
            walkparams.dct = HCR_EL2().DCT;
        else
            walkparams.dct = '0';
        end;
        walkparams.dc  = HCR_EL2().DC;
        walkparams.dct = if IsFeatureImplemented(FEAT_MTE2) then HCR_EL2().DCT else '0';
    end;

    if IsFeatureImplemented(FEAT_LSMAOC) then
        walkparams.ntlsmd = SCTLR_EL1().nTLSMD;
    else
        walkparams.ntlsmd = '1';
    end;

    walkparams.cmow = if IsFeatureImplemented(FEAT_CMOW)   then SCTLR_EL1().CMOW else '0';
    walkparams.ha = if IsFeatureImplemented(FEAT_HAF) then TCR_EL1().HA else '0';
    if IsFeatureImplemented(FEAT_HAFDBS) && walkparams.ha == '1' then
        walkparams.hd = TCR_EL1().HD;
    else
        walkparams.hd = '0';
    end;
    if (walkparams.tgx IN {TGx_4KB, TGx_16KB} && IsFeatureImplemented(FEAT_LPA2) &&
          walkparams.d128 == '0') then
        walkparams.ds = TCR_EL1().DS;
    else
        walkparams.ds = '0';
    end;
    if IsFeatureImplemented(FEAT_PAN3) then
        walkparams.epan = if walkparams.pie == '0' then SCTLR_EL1().EPAN else '1';
    else
        walkparams.epan = '0';
    end;
    if IsFeatureImplemented(FEAT_THE) && walkparams.d128 == '0' && IsTCR2EL1Enabled() then
        walkparams.pnch = TCR2_EL1().PnCH;
    else
        walkparams.pnch = '0';
    end;
    if IsFeatureImplemented(FEAT_HAFT) && walkparams.ha == '1' && IsTCR2EL1Enabled() then
        walkparams.haft = TCR2_EL1().HAFT;
    else
        walkparams.haft = '0';
    end;
    walkparams.emec = (if IsFeatureImplemented(FEAT_MEC) &&
                       IsSCTLR2EL2Enabled() then SCTLR2_EL2().EMEC else '0');
```

```
    return walkparams;
end;
```

Library pseudocode for aarch64/translation/vmsa_walkparams/AArch64_S1TTWParamsEL2

```
// AArch64_S1TTWParamsEL2()
// ========================
// Gather stage 1 translation table walk parameters for EL2 regime

readonly func AArch64_S1TTWParamsEL2(ss : SecurityState) => S1TTWParams
begin
    var walkparams : S1TTWParams;

    walkparams.tgx  = AArch64_S1DecodeTG0(TCR_EL2().TG0);
    walkparams.txsz = TCR_EL2().T0SZ;
    walkparams.ps   = TCR_EL2().PS;
    walkparams.irgn = TCR_EL2().IRGN0;
    walkparams.orgn = TCR_EL2().ORGN0;
    walkparams.sh   = TCR_EL2().SH0;
    walkparams.tbi  = TCR_EL2().TBI;
    walkparams.mair = MAIR_EL2() as MAIRType;

    walkparams.pie = (if IsFeatureImplemented(FEAT_S1PIE) && IsTCR2EL2Enabled() then TCR2_EL2().PIE
                      else '0');

    if IsFeatureImplemented(FEAT_S1PIE) then
        if !(HaveEL(EL3) && SCR_EL3().PIEn == '0' &&
               ImpDefBool("SCR_EL3.PIEn forces PIR_ELx or POR_ELx to zero")) then
            walkparams.pir = PIR_EL2() as S1PIRType;
        else
            walkparams.pir = Zeros{64} as S1PIRType;
        end;
    end;
    if IsFeatureImplemented(FEAT_AIE) then
        walkparams.mair2 = MAIR2_EL2() as MAIRType;
    end;
    walkparams.aie  = (if IsFeatureImplemented(FEAT_AIE) && IsTCR2EL2Enabled() then TCR2_EL2().AIE
                       else '0');
    walkparams.wxn  = SCTLR_EL2().WXN;
    walkparams.ee   = SCTLR_EL2().EE;
    if (HaveEL(EL3) && (!IsFeatureImplemented(FEAT_RME) || IsFeatureImplemented(FEAT_SEL2))) then
        walkparams.sif = SCR_EL3().SIF;
    else
        walkparams.sif = '0';
    end;

    walkparams.tbid = if IsFeatureImplemented(FEAT_PAuth)  then TCR_EL2().TBID else '0';
    walkparams.hpd  = if IsFeatureImplemented(FEAT_HPDS)   then TCR_EL2().HPD  else '0';
    if walkparams.hpd == '0' then
        if walkparams.aie == '1' then walkparams.hpd = '1'; end;
        if walkparams.pie == '1' then walkparams.hpd = '1'; end;
        if AArch64_S1POEnabled(Regime_EL2) then walkparams.hpd = '1'; end;
    end;
    walkparams.ha = if IsFeatureImplemented(FEAT_HAF) then TCR_EL2().E2H0.HA else '0';
    walkparams.ha = if IsFeatureImplemented(FEAT_HAF) then TCR_EL2().HA else '0';
    if IsFeatureImplemented(FEAT_HAFDBS) && walkparams.ha == '1' then
        walkparams.hd = TCR_EL2().E2H0.HD;
        walkparams.hd = TCR_EL2().HD;
    else
        walkparams.hd = '0';
    end;
    if walkparams.tgx IN {TGx_4KB, TGx_16KB} && IsFeatureImplemented(FEAT_LPA2) then
        walkparams.ds = TCR_EL2().E2H0.DS;
        walkparams.ds = TCR_EL2().DS;
    else
        walkparams.ds = '0';
    end;
    if (IsFeatureImplemented(FEAT_MTE_NO_ADDRESS_TAGS) ||
          IsFeatureImplemented(FEAT_MTE_CANONICAL_TAGS)) then
        walkparams.mtx  = TCR_EL2().MTX;
    else
        walkparams.mtx  = '0';
    end;
    walkparams.pnch = (if IsFeatureImplemented(FEAT_THE) && IsTCR2EL2Enabled() then TCR2_EL2().PnCH
                       else '0');
    if IsFeatureImplemented(FEAT_HAFT) && walkparams.ha == '1' && IsTCR2EL2Enabled() then
        walkparams.haft = TCR2_EL2().HAFT;
    else
        walkparams.haft = '0';
    end;
    walkparams.emec = (if IsFeatureImplemented(FEAT_MEC) &&
                       IsSCTLR2EL2Enabled() then SCTLR2_EL2().EMEC else '0');
    if IsFeatureImplemented(FEAT_MEC) && ss == SS_Realm && IsTCR2EL2Enabled() then
        walkparams.amec = TCR2_EL2().AMEC0;
    else
        walkparams.amec = '0';
    end;

    return walkparams;
end;
```

Library pseudocode for aarch64/translation/vmsa_walkparams/AArch64_S1TTWParamsEL20

```
// AArch64_S1TTWParamsEL20()
// =========================
// Gather stage 1 translation table walk parameters for EL2&0 regime

readonly func AArch64_S1TTWParamsEL20(el : bits(2), ss : SecurityState,
                                      varange : VARange) => S1TTWParams
begin
    var walkparams : S1TTWParams;

    if IsFeatureImplemented(FEAT_D128) && IsTCR2EL2Enabled() then
        walkparams.d128 = TCR2_EL2().D128;
    else
        walkparams.d128 = '0';
    end;

    if walkparams.d128 == '1' then
        walkparams.pie = '1';
    else
        walkparams.pie = (if IsFeatureImplemented(FEAT_S1PIE) &&
                          IsTCR2EL2Enabled() then TCR2_EL2().PIE else '0');
    end;
    if IsFeatureImplemented(FEAT_S1PIE) then
        if !(HaveEL(EL3) && SCR_EL3().PIEn == '0' &&
               ImpDefBool("SCR_EL3.PIEn forces PIR_ELx or POR_ELx to zero")) then
            walkparams.pir   = PIR_EL2() as S1PIRType;
            walkparams.pire0 = PIRE0_EL2() as S1PIRType;
        else
            walkparams.pir   = Zeros{64} as S1PIRType;
            walkparams.pire0 = Zeros{64} as S1PIRType;
        end;
    end;

    if IsFeatureImplemented(FEAT_AIE) then
        walkparams.mair2 = MAIR2_EL2() as MAIRType;
    end;
    walkparams.aie  = (if IsFeatureImplemented(FEAT_AIE) && IsTCR2EL2Enabled() then TCR2_EL2().AIE
                       else '0');
    if varange == VARange_LOWER then
        walkparams.tgx   = AArch64_S1DecodeTG0(TCR_EL2().TG0);
        walkparams.txsz  = TCR_EL2().T0SZ;
        walkparams.irgn  = TCR_EL2().IRGN0;
        walkparams.orgn  = TCR_EL2().ORGN0;
        walkparams.sh    = TCR_EL2().SH0;
        walkparams.tbi   = TCR_EL2().TBI0;

        walkparams.nfd   = if IsFeatureImplemented(FEAT_SVE)   then TCR_EL2().NFD0  else '0';
        walkparams.tbid  = if IsFeatureImplemented(FEAT_PAuth) then TCR_EL2().TBID0 else '0';
        walkparams.e0pd  = if IsFeatureImplemented(FEAT_E0PD)  then TCR_EL2().E0PD0 else '0';
        walkparams.hpd   = if IsFeatureImplemented(FEAT_HPDS)  then TCR_EL2().HPD0  else '0';
        if walkparams.hpd == '0' then
            if walkparams.aie == '1' then walkparams.hpd = '1'; end;
            if walkparams.pie == '1' then walkparams.hpd = '1'; end;
            if AArch64_S1POEnabled(Regime_EL20) || AArch64_S1E0POEnabled(Regime_EL20, '0') then
                walkparams.hpd = '1';
            end;
        end;
        if (IsFeatureImplemented(FEAT_MTE_NO_ADDRESS_TAGS) ||
              IsFeatureImplemented(FEAT_MTE_CANONICAL_TAGS)) then
            walkparams.mtx  = TCR_EL2().MTX0;
        else
            walkparams.mtx  = '0';
        end;
        walkparams.skl   = if walkparams.d128 == '1' then TTBR0_EL2().SKL else '00';
        walkparams.disch = if walkparams.d128 == '1' then TCR2_EL2().DisCH0 else '0';
        if IsFeatureImplemented(FEAT_ASID2) && IsTCR2EL2Enabled() then
            walkparams.fng = TCR2_EL2().FNG0;
        else
            walkparams.fng = '0';
        end;
    else
        walkparams.tgx   = AArch64_S1DecodeTG1(TCR_EL2().TG1);
        walkparams.txsz  = TCR_EL2().T1SZ;
        walkparams.irgn  = TCR_EL2().IRGN1;
        walkparams.orgn  = TCR_EL2().ORGN1;
        walkparams.sh    = TCR_EL2().SH1;
        walkparams.tbi   = TCR_EL2().TBI1;

        walkparams.nfd   = if IsFeatureImplemented(FEAT_SVE)   then TCR_EL2().NFD1  else '0';
        walkparams.tbid  = if IsFeatureImplemented(FEAT_PAuth) then TCR_EL2().TBID1 else '0';
        walkparams.e0pd  = if IsFeatureImplemented(FEAT_E0PD)  then TCR_EL2().E0PD1 else '0';
        walkparams.hpd   = if IsFeatureImplemented(FEAT_HPDS)  then TCR_EL2().HPD1  else '0';
        if walkparams.hpd == '0' then
            if walkparams.aie == '1' then walkparams.hpd = '1'; end;
            if walkparams.pie == '1' then walkparams.hpd = '1'; end;
            if AArch64_S1POEnabled(Regime_EL20) || AArch64_S1E0POEnabled(Regime_EL20, '0') then
                walkparams.hpd = '1';
            end;
        end;
        if (IsFeatureImplemented(FEAT_MTE_NO_ADDRESS_TAGS) ||
```

```
              IsFeatureImplemented(FEAT_MTE_CANONICAL_TAGS)) then
            walkparams.mtx  = TCR_EL2().MTX1;
        else
            walkparams.mtx  = '0';
        end;
        walkparams.skl   = if walkparams.d128 == '1' then TTBR1_EL2().SKL else '00';
        walkparams.disch = if walkparams.d128 == '1' then TCR2_EL2().DisCH1 else '0';
        if IsFeatureImplemented(FEAT_ASID2) && IsTCR2EL2Enabled() then
            walkparams.fng = TCR2_EL2().FNG1;
        else
            walkparams.fng = '0';
        end;
    end;

    walkparams.mair = MAIR_EL2() as MAIRType;
    walkparams.wxn  = SCTLR_EL2().WXN;
    walkparams.ps   = TCR_EL2().IPS;
    walkparams.ee   = SCTLR_EL2().EE;
    if (HaveEL(EL3) && (!IsFeatureImplemented(FEAT_RME) || IsFeatureImplemented(FEAT_SEL2))) then
        walkparams.sif = SCR_EL3().SIF;
    else
        walkparams.sif = '0';
    end;

    if IsFeatureImplemented(FEAT_LSMAOC) then
        walkparams.ntlsmd = SCTLR_EL2().nTLSMD;
    else
        walkparams.ntlsmd = '1';
    end;

    walkparams.cmow = if IsFeatureImplemented(FEAT_CMOW)   then SCTLR_EL2().CMOW else '0';
    walkparams.ha = if IsFeatureImplemented(FEAT_HAF) then TCR_EL2().E2H1.HA else '0';
    walkparams.ha = if IsFeatureImplemented(FEAT_HAF) then TCR_EL2().HA else '0';
    if IsFeatureImplemented(FEAT_HAFDBS) && walkparams.ha == '1' then
        walkparams.hd = TCR_EL2().E2H1.HD;
        walkparams.hd = TCR_EL2().HD;
    else
        walkparams.hd = '0';
    end;
    if (walkparams.tgx IN {TGx_4KB, TGx_16KB} && IsFeatureImplemented(FEAT_LPA2) &&
          walkparams.d128 == '0') then
        walkparams.ds = TCR_EL2().E2H1.DS;
        walkparams.ds = TCR_EL2().DS;
    else
        walkparams.ds = '0';
    end;
    if IsFeatureImplemented(FEAT_PAN3) then
        walkparams.epan = if walkparams.pie == '0' then SCTLR_EL2().EPAN else '1';
    else
        walkparams.epan = '0';
    end;
    if IsFeatureImplemented(FEAT_THE) && walkparams.d128 == '0' && IsTCR2EL2Enabled() then
        walkparams.pnch = TCR2_EL2().PnCH;
    else
        walkparams.pnch = '0';
    end;
    if IsFeatureImplemented(FEAT_HAFT) && walkparams.ha == '1' && IsTCR2EL2Enabled() then
        walkparams.haft = TCR2_EL2().HAFT;
    else
        walkparams.haft = '0';
    end;
    walkparams.emec = (if IsFeatureImplemented(FEAT_MEC) && IsSCTLR2EL2Enabled()
                       then SCTLR2_EL2().EMEC else '0');
    if IsFeatureImplemented(FEAT_MEC) && ss == SS_Realm && IsTCR2EL2Enabled() then
        walkparams.amec = if varange == VARange_LOWER then TCR2_EL2().AMEC0 else TCR2_EL2().AMEC1;
    else
        walkparams.amec = '0';
    end;

    return walkparams;
end;
```

Library pseudocode for aarch64/translation/vmsa_walkparams/AArch64_SS2TTWParams

```
// AArch64_SS2TTWParams()
// ======================
// Gather walk parameters specific for secure stage 2 translation

func AArch64_SS2TTWParams(ipaspace : PASpace, s1aarch64 : boolean) => S2TTWParams
begin
    var walkparams : S2TTWParams;

    walkparams.d128 = if IsFeatureImplemented(FEAT_D128) then VTCR_EL2().D128 else '0';
    if ipaspace == PAS_Secure then
        walkparams.tgx  = AArch64_S2DecodeTG0(VSTCR_EL2().TG0);
        walkparams.txsz = VSTCR_EL2().T0SZ;
        if walkparams.d128 == '1' then
            walkparams.skl = VSTTBR_EL2().SKL;
        else
            walkparams.sl0 = VSTCR_EL2().SL0;
        end;
        if walkparams.tgx == TGx_4KB && IsFeatureImplemented(FEAT_LPA2) then
            walkparams.sl2 = VSTCR_EL2().SL2 AND VTCR_EL2().DS;
        else
            walkparams.sl2 = '0';
        end;
    elsif ipaspace == PAS_NonSecure then
        walkparams.tgx  = AArch64_S2DecodeTG0(VTCR_EL2().TG0);
        walkparams.txsz = VTCR_EL2().T0SZ;
        if walkparams.d128 == '1' then
            walkparams.skl = VTTBR_EL2().SKL;
        else
            walkparams.sl0 = VTCR_EL2().SL0;
        end;
        if walkparams.tgx == TGx_4KB && IsFeatureImplemented(FEAT_LPA2) then
            walkparams.sl2 = VTCR_EL2().SL2 AND VTCR_EL2().DS;
        else
            walkparams.sl2 = '0';
        end;
    else
        unreachable;
    end;

    walkparams.sw   = VSTCR_EL2().SW;
    walkparams.nsw  = VTCR_EL2().NSW;
    walkparams.sa   = VSTCR_EL2().SA;
    walkparams.nsa  = VTCR_EL2().NSA;
    if IsFeatureImplemented(FEAT_HCRMAC) then
        walkparams.vm  = HCR_EL2().VM OR HCR_EL2().DC;
    else
        walkparams.vm  = HCR_EL2().VM;
    end;
    walkparams.vm   = HCR_EL2().VM OR HCR_EL2().DC;
    walkparams.ps   = VTCR_EL2().PS;
    walkparams.irgn = VTCR_EL2().IRGN0;
    walkparams.orgn = VTCR_EL2().ORGN0;
    walkparams.sh   = VTCR_EL2().SH0;
    walkparams.ee   = SCTLR_EL2().EE;

    if HCR_EL2().TGE == '0' then
        walkparams.ptw = if IsFeatureImplemented(FEAT_HCRMAC) then HCR_EL2().PTW else '1';
    else
        walkparams.ptw = '0';
    end;
    if IsFeatureImplemented(FEAT_S2FWB) then
        walkparams.fwb = if IsFeatureImplemented(FEAT_HCRMAC) then HCR_EL2().FWB else '1';
    else
        walkparams.fwb = '0';
    end;
    walkparams.ptw = if HCR_EL2().TGE == '0' then HCR_EL2().PTW else '0';
    walkparams.fwb = if IsFeatureImplemented(FEAT_S2FWB) then HCR_EL2().FWB else '0';
    walkparams.ha = if IsFeatureImplemented(FEAT_HAF) then VTCR_EL2().HA else '0';
    if IsFeatureImplemented(FEAT_HAFDBS) && walkparams.ha == '1' then
        walkparams.hd = VTCR_EL2().HD;
    else
        walkparams.hd = '0';
    end;
    if walkparams.tgx IN {TGx_4KB, TGx_16KB} && IsFeatureImplemented(FEAT_LPA2) then
        walkparams.ds = VTCR_EL2().DS;
    else
        walkparams.ds = '0';
    end;
    walkparams.cmow = (if IsFeatureImplemented(FEAT_CMOW) && IsHCRXEL2Enabled() then HCRX_EL2().CMOW
                       else '0');
    if walkparams.d128 == '1' then
        walkparams.s2pie = '1';
    else
        walkparams.s2pie = if IsFeatureImplemented(FEAT_S2PIE) then VTCR_EL2().S2PIE else '0';
    end;
    if IsFeatureImplemented(FEAT_S2PIE) then
        if !(HaveEL(EL3) && SCR_EL3().PIEn == '0' &&
                ImpDefBool("SCR_EL3.PIEn forces PIR_ELx or POR_ELx to zero")) then
            walkparams.s2pir = S2PIR_EL2() as S2PIRType;
```

```
        else
            walkparams.s2pir = Zeros{64} as S2PIRType;
        end;
    end;
    if IsFeatureImplemented(FEAT_THE) && walkparams.d128 != '1' then
        walkparams.assuredonly = VTCR_EL2().AssuredOnly;
    else
        walkparams.assuredonly = '0';
    end;
    walkparams.tl0   = if IsFeatureImplemented(FEAT_THE) then VTCR_EL2().TL0 else '0';
    walkparams.tl1   = if IsFeatureImplemented(FEAT_THE) then VTCR_EL2().TL1 else '0';
    if IsFeatureImplemented(FEAT_HAFT) && walkparams.ha == '1' then
        walkparams.haft = VTCR_EL2().HAFT;
    else
        walkparams.haft = '0';
    end;
    walkparams.emec = '0';
    if (IsFeatureImplemented(FEAT_HDBSS) && walkparams.hd == '1' &&
          (!HaveEL(EL3) || SCR_EL3().HDBSSEn == '1')) then
        walkparams.hdbss = VTCR_EL2().HDBSS;
    else
        walkparams.hdbss = '0';
    end;

    return walkparams;
end;
```

Library pseudocode for aarch64/translation/vmsa_walkparams/S2DCacheEnabled

```
// S2DCacheEnabled()
// =================
// Returns TRUE if Stage 2 Data access cacheability is enabled

readonly func S2DCacheEnabled() => boolean
begin
    return !IsFeatureImplemented(FEAT_HCRMAC) || HCR_EL2().CD == '0';
    return HCR_EL2().CD == '0';
end;
```

Library pseudocode for aarch64/translation/vmsa_walkparams/S2ICacheEnabled

```
// S2ICacheEnabled()
// =================
// Returns TRUE if Stage 2 Instruction access cacheability is enabled

readonly func S2ICacheEnabled() => boolean
begin
    return !IsFeatureImplemented(FEAT_HCRMAC) || HCR_EL2().ID == '0';
    return HCR_EL2().ID == '0';
end;
```

Library pseudocode for shared/debug/amu/G1ActivityMonitorImplementedMask

```
// G1ActivityMonitorImplementedMask()
// ==================================
// Return the implementation-defined G1 activity monitor mask.

func G1ActivityMonitorImplementedMask() => bits(16)
begin
    assert NUM_AMU_CG1_MONITORS <= 16;
    var mask : bits(16) = Zeros{16};

    for i = 0 to (NUM_AMU_CG1_MONITORS-1) do
        mask[i] = if IsG1ActivityMonitorImplemented(i) then '1' else '0';
    end;
    return mask;
end;
```

Library pseudocode for shared/debug/amu/Write_AMCNTENCLR0

```
// Write_AMCNTENCLR0()
// ===================
// System register write to AMCNTENCLR0.

func Write_AMCNTENCLR0(value_in : bits(32))
begin
    AMCNTENCLR0().P = AMCNTENCLR0().P AND NOT value_in[0 +: NUM_AMU_CG0_MONITORS];
end;
```

Library pseudocode for shared/debug/amu/Write_AMCNTENCLR0_EL0

```
// Write_AMCNTENCLR0_EL0()
// =======================
// System register write to AMCNTENCLR0_EL0.

func Write_AMCNTENCLR0_EL0(value_in : bits(64))
begin
    AMCNTENCLR0_EL0().P = AMCNTENCLR0_EL0().P AND NOT value_in[0 +: NUM_AMU_CG0_MONITORS];
end;
```

Library pseudocode for shared/debug/amu/Write_AMCNTENCLR1

```
// Write_AMCNTENCLR1()
// ===================
// System register write to AMCNTENCLR1.

func Write_AMCNTENCLR1(value_in : bits(32))
begin
    assert NUM_AMU_CG1_MONITORS <= 16;
    let implemented : bits(16) = G1ActivityMonitorImplementedMask();

    AMCNTENCLR1().P = (AMCNTENCLR1().P AND NOT
                       (ZeroExtend{16}(value_in[0 +: NUM_AMU_CG1_MONITORS]) AND implemented));
end;
```

Library pseudocode for shared/debug/amu/Write_AMCNTENCLR1_EL0

```
// Write_AMCNTENCLR1_EL0()
// =======================
// System register write to AMCNTENCLR1_EL0.

func Write_AMCNTENCLR1_EL0(value_in : bits(64))
begin
    assert NUM_AMU_CG1_MONITORS <= 16;
    let implemented : bits(16) = G1ActivityMonitorImplementedMask();

    AMCNTENCLR1_EL0().P = (AMCNTENCLR1_EL0().P AND NOT
                           (ZeroExtend{16}(value_in[0 +: NUM_AMU_CG1_MONITORS]) AND implemented));
end;
```

Library pseudocode for shared/debug/amu/Write_AMCNTENSET0

```
// Write_AMCNTENSET0()
// ===================
// System register write to AMCNTENSET0.

func Write_AMCNTENSET0(value_in : bits(32))
begin
    AMCNTENSET0().P = AMCNTENSET0().P OR value_in[0 +: NUM_AMU_CG0_MONITORS];
end;
```

Library pseudocode for shared/debug/amu/Write_AMCNTENSET0_EL0

```
// Write_AMCNTENSET0_EL0()
// =======================
// System register write to AMCNTENSET0_EL0.

func Write_AMCNTENSET0_EL0(value_in : bits(64))
begin
    AMCNTENSET0_EL0().P = AMCNTENSET0_EL0().P OR value_in[0 +: NUM_AMU_CG0_MONITORS];
end;
```

Library pseudocode for shared/debug/amu/Write_AMCNTENSET1

```
// Write_AMCNTENSET1()
// ===================
// System register write to AMCNTENSET1.

func Write_AMCNTENSET1(value_in : bits(32))
begin
    assert NUM_AMU_CG1_MONITORS <= 16;
    let implemented : bits(16) = G1ActivityMonitorImplementedMask();

    AMCNTENSET1().P = (AMCNTENSET1().P OR
                       (ZeroExtend{16}(value_in[0 +: NUM_AMU_CG1_MONITORS]) AND implemented));
end;
```

Library pseudocode for shared/debug/amu/Write_AMCNTENSET1_EL0

```
// Write_AMCNTENSET1_EL0()
// =======================
// System register write to AMCNTENSET1_EL0.

func Write_AMCNTENSET1_EL0(value_in : bits(64))
begin
    assert NUM_AMU_CG1_MONITORS <= 16;
    let implemented : bits(16) = G1ActivityMonitorImplementedMask();

    AMCNTENSET1_EL0().P = (AMCNTENSET1_EL0().P OR
                           (ZeroExtend{16}(value_in[0 +: NUM_AMU_CG1_MONITORS]) AND implemented));
end;
```

Library pseudocode for shared/debug/claim

```
var DBGClaimTags : bits(8);
```

Library pseudocode for shared/debug/claim/Write_DBGCLAIMCLR

```
// Write_DBGCLAIMCLR()
// ===================
// System register write to DBGCLAIMCLR.

func Write_DBGCLAIMCLR(value_in : bits(32))
begin
    DBGClaimTags = DBGClaimTags AND NOT value_in[7:0];
end;
```

Library pseudocode for shared/debug/claim/Write_DBGCLAIMCLR_EL1

```
// Write_DBGCLAIMCLR_EL1()
// =======================
// System register write to DBGCLAIMCLR_EL1.

func Write_DBGCLAIMCLR_EL1(value_in : bits(64))
begin
    DBGClaimTags = DBGClaimTags AND NOT value_in[7:0];
end;
```

Library pseudocode for shared/debug/claim/Write_DBGCLAIMSET

```
// Write_DBGCLAIMSET()
// ===================
// System register write to DBGCLAIMSET.

func Write_DBGCLAIMSET(value_in : bits(32))
begin
    DBGClaimTags = DBGClaimTags OR value_in[7:0];
end;
```

Library pseudocode for shared/debug/claim/Write_DBGCLAIMSET_EL1

```
// Write_DBGCLAIMSET_EL1()
// =======================
// System register write to DBGCLAIMSET_EL1.

func Write_DBGCLAIMSET_EL1(value_in : bits(64))
begin
    DBGClaimTags = DBGClaimTags OR value_in[7:0];
end;
```

Library pseudocode for shared/debug/cti/CrossTrigger

```
// CrossTrigger
// ============

type CrossTriggerOut of enumeration {CrossTriggerOut_DebugRequest, CrossTriggerOut_RestartRequest,
                                     CrossTriggerOut_IRQ,          CrossTriggerOut_RSVD3,
                                     CrossTriggerOut_TraceExtIn0,  CrossTriggerOut_TraceExtIn1,
                                     CrossTriggerOut_TraceExtIn2,  CrossTriggerOut_TraceExtIn3};

type CrossTriggerIn of enumeration {CrossTriggerIn_CrossHalt,     CrossTriggerIn_PMUOverflow,
                                    CrossTriggerIn_SPESample,     CrossTriggerIn_RSVD3,
                                    CrossTriggerIn_TraceExtOut0,  CrossTriggerIn_TraceExtOut1,
                                    CrossTriggerIn_TraceExtOut2,  CrossTriggerIn_TraceExtOut3,
                                    CrossTriggerIn_TRBEStop,      CrossTriggerIn_TRBEMgmt,
                                    CrossTriggerIn_TRBEWrap};

varCrossTriggerIn_TRBEWrap}; CTIClaimTags : bits(32);
```

Library pseudocode for shared/debug/pmu/Write_PMCNTENCLR

```
// Write_PMCNTENCLR()
// ==================
// System register write to PMCNTENCLR.

func Write_PMCNTENCLR(value_in : bits(32))
begin
    let write_mask : bits(32) = GetPMUWriteMask(FALSE)[31:0];

    PMCNTENCLR() = PMCNTENCLR() AND NOT (value_in AND write_mask);
end;
```

Library pseudocode for shared/debug/pmu/Write_PMCNTENCLR_EL0

```
// Write_PMCNTENCLR_EL0()
// ======================
// System register write to PMCNTENCLR_EL0.

func Write_PMCNTENCLR_EL0(value_in : bits(64))
begin
    let write_mask : bits(64) = GetPMUWriteMask(FALSE);

    PMCNTENCLR_EL0() = PMCNTENCLR_EL0() AND NOT (value_in AND write_mask);
end;
```

Library pseudocode for shared/debug/pmu/Write_PMCNTENSET

```
// Write_PMCNTENSET()
// ==================
// System register write to PMCNTENSET.

func Write_PMCNTENSET(value_in : bits(32))
begin
    let write_mask : bits(32) = GetPMUWriteMask(FALSE)[31:0];

    PMCNTENSET() = PMCNTENSET() OR (value_in AND write_mask);
end;
```

Library pseudocode for shared/debug/pmu/Write_PMCNTENSET_EL0

```
// Write_PMCNTENSET_EL0()
// ======================
// System register write to PMCNTENSET_EL0.

func Write_PMCNTENSET_EL0(value_in : bits(64))
begin
    let write_mask : bits(64) = GetPMUWriteMask(FALSE);

    PMCNTENSET_EL0() = PMCNTENSET_EL0() OR (value_in AND write_mask);
end;
```

Library pseudocode for shared/debug/pmu/Write_PMINTENCLR

```
// Write_PMINTENCLR()
// ==================
// System register write to PMINTENCLR.

func Write_PMINTENCLR(value_in : bits(32))
begin
    let write_mask : bits(32) = GetPMUWriteMask(FALSE)[31:0];

    PMINTENCLR() = PMINTENCLR() AND NOT (value_in AND write_mask);
end;
```

Library pseudocode for shared/debug/pmu/Write_PMINTENCLR_EL1

```
// Write_PMINTENCLR_EL1()
// ======================
// System register write to PMINTENCLR_EL1.

func Write_PMINTENCLR_EL1(value_in : bits(64))
begin
    let write_mask : bits(64) = GetPMUWriteMask(FALSE);

    PMINTENCLR_EL1() = PMINTENCLR_EL1() AND NOT (value_in AND write_mask);
end;
```

Library pseudocode for shared/debug/pmu/Write_PMINTENSET

```
// Write_PMINTENSET()
// ==================
// System register write to PMINTENSET.

func Write_PMINTENSET(value_in : bits(32))
begin
    let write_mask : bits(32) = GetPMUWriteMask(FALSE)[31:0];

    PMINTENSET() = PMINTENSET() OR (value_in AND write_mask);
end;
```

Library pseudocode for shared/debug/pmu/Write_PMINTENSET_EL1

```
// Write_PMINTENSET_EL1()
// ======================
// System register write to PMINTENSET_EL1.

func Write_PMINTENSET_EL1(value_in : bits(64))
begin
    let write_mask : bits(64) = GetPMUWriteMask(FALSE);

    PMINTENSET_EL1() = PMINTENSET_EL1() OR (value_in AND write_mask);
end;
```

Library pseudocode for shared/debug/pmu/Write_PMOVSCLR_EL0

```
// Write_PMOVSCLR_EL0()
// ====================
// System register write to PMOVSCLR_EL0.

func Write_PMOVSCLR_EL0(value_in : bits(64))
begin
    let write_mask : bits(64) = GetPMUWriteMask(FALSE);

    PMOVSCLR_EL0() = PMOVSCLR_EL0() AND NOT (value_in AND write_mask);
end;
```

Library pseudocode for shared/debug/pmu/Write_PMOVSR

```
// Write_PMOVSR()
// ==============
// System register write to PMOVSR.

func Write_PMOVSR(value_in : bits(32))
begin
    let write_mask : bits(32) = GetPMUWriteMask(FALSE)[31:0];

    PMOVSR() = PMOVSR() AND NOT (value_in AND write_mask);
end;
```

Library pseudocode for shared/debug/pmu/Write_PMOVSSET

```
// Write_PMOVSSET()
// ================
// System register write to PMOVSSET.

func Write_PMOVSSET(value_in : bits(32))
begin
    let write_mask : bits(32) = GetPMUWriteMask(FALSE)[31:0];

    PMOVSSET() = PMOVSSET() OR (value_in AND write_mask);
end;
```

Library pseudocode for shared/debug/pmu/Write_PMOVSSET_EL0

```
// Write_PMOVSSET_EL0()
// ====================
// System register write to PMOVSSET_EL0.

func Write_PMOVSSET_EL0(value_in : bits(64))
begin
    let write_mask : bits(64) = GetPMUWriteMask(FALSE);

    PMOVSSET_EL0() = PMOVSSET_EL0() OR (value_in AND write_mask);
end;
```

Library pseudocode for shared/debug/samplebasedprofiling/Read_EDPCSRlo

```
// Read_EDPCSRlo()
// ===============

func Read_EDPCSRlo(memory_mapped : boolean) => bits(32)
begin
    // The Software lock is OPTIONAL.
    let update : boolean = !memory_mapped || EDLSR().SLK == '0';// Software locked: no side-effects
    var sample : bits(32);
    if pc_sample.valid then
        sample = pc_sample.pc[31:0];
        if update then
            if IsFeatureImplemented(FEAT_VHE) && EDSCR().SC2 == '1' then
                EDPCSR().SC2_1.EDPCSRhi  = (if EDPCSRhi.PC = (if pc_sample.rw == '0' then Zeros{24}
                                            else pc_sample.pc[55:32]);
                EDPCSR().EL              = EDPCSRhi.EL = pc_sample.el;
                EDPCSR().NS              = (if EDPCSRhi.NS = (if pc_sample.ss == SS_Secure then '0' else
'1');
            else
                EDPCSR().SC2_0.EDPCSRhi  = (if EDPCSRhi = (if pc_sample.rw == '0' then Zeros{32}
                                            else pc_sample.pc[63:32]);
            end;
            EDCIDSR() = pc_sample.contextidr;
            if ((IsFeatureImplemented(FEAT_VHE) || IsFeatureImplemented(FEAT_Debugv8p2)) &&
                  EDSCR().SC2 == '1') then
                EDVIDSR() = (if pc_sample.has_el2 then pc_sample.contextidr_el2
                             else ARBITRARY : bits(32));
            else
                EDVIDSR().VMID = (if pc_sample.has_el2 && pc_sample.el IN {EL1,EL0}
                                  then pc_sample.vmid else Zeros{NUM_VMIDBITS});
                EDVIDSR().NS = (if pc_sample.ss == SS_Secure then '0' else '1');
                EDVIDSR().E2 = (if pc_sample.el == EL2 then '1' else '0');
                EDVIDSR().E3 = (if pc_sample.el == EL3 then '1' else '0') AND pc_sample.rw;
                // The conditions for setting HV are not specified if PCSRhi is zero.
                // An example implementation may be "pc_sample.rw".
                EDVIDSR().HV = (if !IsZero(EDPCSR().SC2_0.EDPCSRhi) then '1'
                EDVIDSR().HV = (if !IsZero(EDPCSRhi) then '1'
                                else ImpDefBit("0 or 1"));
            end;
        end;
    else
        sample = Ones{32};
        if update then
            EDPCSR()[63:32] = ARBITRARY : bits(32);
            EDPCSRhi  = ARBITRARY : bits(32);
            EDCIDSR() = ARBITRARY : bits(32);
            EDVIDSR() = ARBITRARY : bits(32);
        end;
    end;

    return sample;
end;
```

Library pseudocode for shared/exceptions/aborts/EffectiveNMEA

```
// EffectiveNMEA()
// ===============
// Returns the Effective value of SCR_EL3.NMEA or SCTLR2_ELx.NMEA.
// Return the Effective value of SCR_EL3.NMEA or SCTLR2_ELx.NMEA.

func EffectiveNMEA(el : bits(2)) => bit
EffectiveNMEA() => bit
begin
    if IsFeatureImplemented(FEAT_DoubleFault2) then
        if el ==          if PSTATE.EL == EL3 && !ELUsingAArch32UsingAArch32(() then
            return SCR_EL3().NMEA;
        elsif (EL3PSTATE) then
            return.EL == EffectiveSCR_EL3_NMEA();
        elsif (el == EL2 || (el ==|| EL0IsInHost &&()) && ! ELIsInHost(el))) && !ELUsingAArch32(EL2) then
            return (if IsSCTLR2EL2Enabled() then SCTLR2_EL2().NMEA else '0');
        elsif (el ==          elsif ! EL1 || (el == EL0 && !ELIsInHost(el))) && !ELUsingAArch32(EL1) then
            return (if IsSCTLR2EL1Enabled() then SCTLR2_EL1().NMEA else '0');
        else
            return '0';
        end;
    end;
    if IsFeatureImplemented(FEAT_DoubleFault) && el ==     elsif IsFeatureImplemented(FEAT_DoubleFault) && PSTATE.EL ==
EL3 && !ELUsingAArch32UsingAArch32(() then
        return SCR_EL3().NMEA ANDEL3EffectiveEA) then
        return EffectiveSCR_EL3_NMEA();
    else
        return '0';
    end;
end;
```

Library pseudocode for shared/exceptions/aborts/PhysicalSErrorTarget

```
// PhysicalSErrorTarget()
// ======================
// Returns a tuple of whether SError exception can be taken and, if so, the
// target Exception level.
// If EL3 is implemented and using AArch32, then a target Exception level of
// EL1 means Abort mode, and EL3 means Monitor mode, including in Secure
// state when Abort mode is part of EL3.

func PhysicalSErrorTarget() => (boolean, bits(2))
begin
    if Halted() then
        return (TRUE, ARBITRARY : bits(2));
    end;

    let effective_ea : bit = EffectiveEA();
    let effective_amo : bit = EffectiveHCR_AMO();
    let effective_tge : bit = EffectiveTGE();
    let effective_nmea : bit = EffectiveNMEA();

    // When EL3 is implemented and using AArch32, the SCR.AW bit can allow PSTATE.A
    // to mask SError exceptions in Non-secure state when SCR.EA is 1 and the Effective
    // value of HCR.AMO is 0.
    var effective_aw : bit;
    if (PSTATE.EL);

    // When EL3 is implemented and using AArch32, the SCR.AW bit can allow PSTATE.A
    // to mask SError exceptions in Non-secure state when SCR.EA is 1 and the Effective
    // value of HCR.AMO is 0.
    var effective_aw : bit;
    if (ELUsingAArch32(EL3) && effective_ea == '1' &&
          CurrentSecurityState() == SS_NonSecure && effective_amo == '0') then
        effective_aw = SCR().AW;
    else
        effective_aw = '0';
    end;

    // The exception is masked by software.
    var masked : boolean;
    case PSTATE.EL of
        when EL3 =>
            masked = (!UsingAArch32() && effective_ea == '0') || PSTATE.A == '1';
        when EL2 =>
            masked = ((effective_ea == '0' || effective_aw == '1') &&
                      ((!UsingAArch32() && effective_tge == '0' && effective_amo == '0') ||
                       PSTATE.A == '1'));
        when EL1, EL0 =>
            masked = ((effective_ea == '0' || effective_aw == '1') &&
                      effective_amo == '0' && PSTATE.A == '1');
    end;

    // When FEAT_DoubleFault or FEAT_DoubleFault2 is implemented, the mask might be overridden.
    masked = (masked && effective_nmea == '0');

    // External debug might disable the exception in the current Security state.
    // This is not relevant at EL3.
    let intdis : boolean = PSTATE.EL != EL3 && ExternalDebugInterruptsDisabled(EL1);

    var target_el : bits(2) = ARBITRARY : bits(2);
    if effective_ea == '1' || (PSTATE.EL == EL3 && !ELUsingAArch32(EL3)) then
        if !masked then target_el = EL3; end;

    elsif EL2Enabled() && effective_amo == '1' && !intdis && PSTATE.EL IN {EL0, EL1} then
        target_el = EL2;
        masked = FALSE;

    elsif (EffectiveHCRX_EL2_TMEA() == '1' && !intdis &&
             ((PSTATE.EL == EL1 && PSTATE.A == '1') ||
              (PSTATE.EL == EL0 && masked && !IsInHost()))) then
        target_el = EL2;
        masked = FALSE;

    elsif (EffectiveSCR_EL3_TMEA() == '1' &&
             ((PSTATE.EL IN {EL2, EL1} && PSTATE.A == '1') ||
             (PSTATE.EL IN {EL2, EL0} && masked) || intdis)) then
        target_el = EL3;
        masked = FALSE;

    elsif PSTATE.EL == EL2 || IsInHost() then
        if !masked then target_el = EL2; end;

    else
        assert (PSTATE.EL == EL1 ||
                  (PSTATE.EL == EL3 && ELUsingAArch32(EL3)) ||
                  (PSTATE.EL == EL0 && !IsInHost()));
        if !masked then target_el = EL1; end;
    end;

    // External debug might disable the exception for the target Exception level.
    if !masked && ExternalDebugInterruptsDisabled(target_el) then
```

```
        masked = TRUE;
        target_el = ARBITRARY : bits(2);
    end;

    return (masked, target_el);
end;
```

Library pseudocode for shared/exceptions/exceptions/Exception

```
// Exception
// =========
// Classes of exception.

type Exception of enumeration {
        Exception_Uncategorized,        // Uncategorized or unknown reason
        Exception_WFxTrap,              // Trapped WFI or WFE instruction
        Exception_CP15RTTrap,           // Trapped AArch32 MCR or MRC access, coproc=0b111
        Exception_CP15RRTTrap,          // Trapped AArch32 MCRR or MRRC access, coproc=0b1111
        Exception_CP14RTTrap,           // Trapped AArch32 MCR or MRC access, coproc=0b1110
        Exception_CP14DTTrap,           // Trapped AArch32 LDC or STC access, coproc=0b1110
        Exception_CP14RRTTrap,          // Trapped AArch32 MRRC access, coproc=0b1110
        Exception_AdvSIMDFPAccessTrap,  // HCPTR-trapped access to SIMD or FP
        Exception_FPIDTrap,             // Trapped access to SIMD or FP ID register
        Exception_OtherTrap,            // Trapped access to other instructions
        // Trapped BXJ instruction not supported in Armv8
        Exception_PACTrap,               // Trapped invalid PAC use
        Exception_IllegalState,          // Illegal Execution state
        Exception_SupervisorCall,        // Supervisor Call
        Exception_HypervisorCall,        // Hypervisor Call
        Exception_MonitorCall,           // Monitor Call or Trapped SMC instruction
        Exception_SystemRegisterTrap,    // Trapped MRS or MSR System register access
        Exception_ERetTrap,              // Trapped invalid ERET use
        Exception_InstructionAbort,      // Instruction Abort or Prefetch Abort
        Exception_PCAlignment,           // PC alignment fault
        Exception_DataAbort,             // Data Abort
        Exception_NV2DataAbort,          // Data abort at EL1 reported as being from EL2
        Exception_PACFail,               // PAC Authentication failure
        Exception_SPAlignment,           // SP alignment fault
        Exception_FPTrappedException,    // IEEE trapped FP exception
        Exception_SError,                // SError interrupt
        Exception_Breakpoint,            // (Hardware) Breakpoint
        Exception_SoftwareStep,          // Software Step
        Exception_Watchpoint,            // Watchpoint
        Exception_NV2Watchpoint,         // Watchpoint at EL1 reported as being from EL2
        Exception_SoftwareBreakpoint,    // Software Breakpoint Instruction
        Exception_VectorCatch,           // AArch32 Vector Catch
        Exception_IRQ,                   // IRQ interrupt
        Exception_SVEAccessTrap,         // HCPTR trapped access to SVE
        Exception_SMEAccessTrap,         // HCPTR trapped access to SME
        Exception_GPC,                   // Granule protection check
        Exception_BranchTarget,          // Branch Target Identification
        Exception_MemCpyMemSet,          // Exception from a CPY* or SET* instruction
        Exception_GCSFail,               // GCS Exceptions
        Exception_Profiling,             // Profiling exception
        Exception_SystemRegister128Trap, // Trapped MRRS or MSRR System register or SYSP access
        Exception_ConditionalFault,      // Exception due to a CFLT instructionException_FIQ};                  // FIQ interrupt
Exception_FIQ};                  // FIQ interrupt
```

Library pseudocode for shared/functions/common/IsZeroBit

```
// IsZeroBit()
// ===========

pure func IsZeroBit{N}(x : bits(N)) => bit
begin
    return if IsZero(x) then '1' else '0';
end;funcIsZeroBit{N}(x : bits(N)) => bit
begin
    return if IsZero(x) then '1' else '0';
end;
```

Library pseudocode for shared/functions/common/NormalizeReal

```
// NormalizeReal
// =============
// Normalizes x to the form 1.xxx... x 2^y and returns (mantissa, exponent)

pure func NormalizeReal(x : real) => (real, integer)
begin
    let exponent : integer = ILog2(x);
    let mantissa : real = x / (2.0 ^ exponent);
    return (mantissa, exponent);
end;func NormalizeReal(x : real) => (real, integer)
begin
    let exponent : integer = ILog2(x);
    let mantissa : real = x / (2.0 ^ exponent);
    return (mantissa, exponent);
end;
```

Library pseudocode for shared/functions/common/RShr

```
// RShr()
// ======
// Shift integer value right with rounding

pure func RShr(value : integer, shift : integer, round : boolean) => integer
begin
    assert shift > 0;
    if round then
        return (value + (1 << (shift - 1))) >> shift;
    else
        return value >> shift;
    end;
end;func RShr(value : integer, shift : integer, round : boolean) => integer
begin
    assert shift > 0;
    if round then
        return (value + (1 << (shift - 1))) >> shift;
    else
        return value >> shift;
    end;
end;
```

Library pseudocode for shared/functions/float/fpexceptrapsenabled/FPExcepTrapsEnabled

```
// FPExcepTrapsEnabled()
// =====================

func FPExcepTrapsEnabled() => boolean
begin
    return (ImpDefBool("IMPLEMENTED_trapping of Input Denormal floating-point exceptions") ||
            ImpDefBool("IMPLEMENTED_trapping of Inexact floating-point exceptions") ||
            ImpDefBool("IMPLEMENTED_trapping of Underflow floating-point exceptions") ||
            ImpDefBool("IMPLEMENTED_trapping of Overflow floating-point exceptions") ||
            ImpDefBool("IMPLEMENTED_trapping of Divide by Zero floating-point exceptions") ||
            ImpDefBool("IMPLEMENTED_trapping of Invalid Operation floating-point exceptions"));
end;
```

Library pseudocode for shared/functions/float/fpprocessexception/FPProcessException~~Library pseudocode for shared/functions/float/fpprocessdenorms/FPProcessDenorms4~~

```
// FPProcessException()
// ====================
//
// The 'fpcr' argument supplies FPCR control bits. Status information is
// updated directly in the FPSR where appropriate.
// FPProcessDenorms4()
// ===================
// Handles denormal input in case of single-precision or double-precision
// when using alternative floating-point mode.

func FPProcessException(except :FPProcessDenorms4(type1 : FPExcFPType, fpcr : FPCR_Type)
begin
    var cumul : integer;
    // Determine the cumulative exception bit number
    case except of
        when, type2 : FPExc_InvalidOpFPType =>      cumul = 0;
        when, type3 : FPExc_DivideByZeroFPType =>   cumul = 1;
        when, type4 : FPExc_OverflowFPType =>       cumul = 2;
        when,
                      N : integer, fpcr : FPCR_Type)
begin
    let altfp : boolean = IsFeatureImplemented(FEAT_AFP) && ! FPExc_UnderflowUsingAArch32 =>      cumul = 3;
        when() && fpcr.AH == '1';
    if altfp && N != 16 && (type1 == FPExc_InexactFPType_Denormal =>        cumul = 4;
        when|| type2 == FPExc_InputDenormFPType_Denormal =>    cumul = 7;
    end;
    // When FEAT_TRAPFPE is implemented, the control bits FPCR.{IDE, IXE, UFE, OFE, DZE, IOE}
    // can be independently set to 1.
    // Otherwise when FEAT_TRAPFPE is not implemented, bits FPCR.{IDE, IXE, UFE, OFE, DZE, IOE}
    // are RAZ/WI.
    let enable : integer = cumul + 8;
    if (fpcr[enable] == '1' && (!IsFeatureImplemented(FEAT_SME) ||||
        type3 == PSTATEFPType_Denormal.SM == '0' |||| type4 ==
IsFullA64EnabledFPType_Denormal())) then
        // Trapping of the exception enabled.
        // It is IMPLEMENTATION DEFINED how exceptions and in what order that they may be
        // accumulated before calling FPTrappedException().
        var accumulated_exceptions : bits(8) =) then GetAccumulatedFPExceptionsFPProcessException();
        accumulated_exceptions[cumul] = '1';
        if !ImpDefBool("Accumulate floating-point exceptions before trapping") then
            if( UsingAArch32FPExc_InputDenorm() then
                AArch32_FPTrappedException(accumulated_exceptions);
            else
                let is_ase : boolean = IsASEInstruction();
                AArch64_FPTrappedException(is_ase, accumulated_exceptions);
            end;
        else
            // The exceptions generated by this instruction are accumulated by the PE and
            // FPTrappedException is called later during its execution, before the next
            // instruction is executed. This field is cleared at the start of each FP instruction.
            SetAccumulatedFPExceptions(accumulated_exceptions);
        end;
    elsif UsingAArch32() then
        // Set the cumulative exception bit
        FPSCR()[cumul] = '1';
    else
        // Set the cumulative exception bit
        FPSR()[cumul] = '1';
, fpcr);
    end;
    return;
end;
```

Library pseudocode for shared/functions/float/fpprocessexception/FPProcessException

```
// FPProcessException()
// ====================
//
// The 'fpcr' argument supplies FPCR control bits. Status information is
// updated directly in the FPSR where appropriate.

func FPProcessException(except : FPExc, fpcr : FPCR_Type)
begin
    var cumul : integer;
    // Determine the cumulative exception bit number
    case except of
        when FPExc_InvalidOp =>      cumul = 0;
        when FPExc_DivideByZero =>   cumul = 1;
        when FPExc_Overflow =>       cumul = 2;
        when FPExc_Underflow =>      cumul = 3;
        when FPExc_Inexact =>        cumul = 4;
        when FPExc_InputDenorm =>    cumul = 7;
    end;
    let enable : integer = cumul + 8;
    if (fpcr[enable] == '1' && (!IsFeatureImplemented(FEAT_SME) || PSTATE.SM == '0' ||
          IsFullA64Enabled())) then
        // Trapping of the exception enabled.
        // It is IMPLEMENTATION DEFINED whether the enable bit may be set at all,
        // and if so then how exceptions and in what order that they may be
        // accumulated before calling FPTrappedException().
        var accumulated_exceptions : bits(8) = GetAccumulatedFPExceptions();
        accumulated_exceptions[cumul] = '1';
        if ImpDefBool("Support trapping of floating-point exceptions") then
            if UsingAArch32() then
                AArch32_FPTrappedException(accumulated_exceptions);
            else
                let is_ase : boolean = IsASEInstruction();
                AArch64_FPTrappedException(is_ase, accumulated_exceptions);
            end;
        else
            // The exceptions generated by this instruction are accumulated by the PE and
            // FPTrappedException is called later during its execution, before the next
            // instruction is executed. This field is cleared at the start of each FP instruction.
            SetAccumulatedFPExceptions(accumulated_exceptions);
        end;
    elsif UsingAArch32() then
        // Set the cumulative exception bit
        FPSCR()[cumul] = '1';
    else
        // Set the cumulative exception bit
        FPSR()[cumul] = '1';
    end;
    return;
end;
```

Library pseudocode for shared/functions/float/fpunpack/FPUnpackBase

```
// FPUnpackBase()
// ==============

func FPUnpackBase{N}(fpval : bits(N), fpcr : FPCR_Type, fpexc : boolean) => (FPType, bit, real)
begin
    let isbfloat16 : boolean = FALSE;
    let (fp_type, sign, value) = FPUnpackBase{N}(fpval, fpcr, fpexc, isbfloat16);
    return (fp_type, sign, value);
end;

// FPUnpackBase()
// ==============
//
// Unpack a floating-point number into its type, sign bit and the real number
// that it represents. The real number result has the correct sign for numbers
// and infinities, is very large in magnitude for infinities, and is 0.0 for
// NaNs. (These values are chosen to simplify the description of comparisons
// and conversions.)
//
// The 'fpcr_in' argument supplies FPCR control bits, 'fpexc' controls the
// generation of floating-point exceptions and 'isbfloat16' determines whether
// N=16 signifies BFloat16 or half-precision type. Status information is updated
// directly in the FPSR where appropriate.

func FPUnpackBase{N}(fpval : bits(N), fpcr_in : FPCR_Type, fpexc : boolean,
                     isbfloat16 : boolean) => (FPType, bit, real)
begin
    assert N IN {16,32,64};

    let fpcr : FPCR_Type = fpcr_in;

    let altfp : boolean = IsFeatureImplemented(FEAT_AFP) && !UsingAArch32();
    let fiz : boolean   = altfp && fpcr.FIZ == '1';
    let fz : boolean    = fpcr.FZ == '1' && !(altfp && fpcr.AH == '1');
    var value : real;
    var sign : bit;
    var fptype : FPType;

    if N == 16 && !isbfloat16 then
        sign   = fpval[15];
        let exp16 : bits(5)   = fpval[14:10];
        let frac16 : bits(10) = fpval[9:0];
        if IsZero(exp16) then
            if IsZero(frac16) || fpcr.FZ16 == '1' then
                fptype = FPType_Zero;  value = 0.0;
            else
                fptype = FPType_Denormal;  value = 2.0^-14 * (Real(UInt(frac16)) * 2.0^-10);
            end;
        elsif IsOnes(exp16) && fpcr.AHP == '0' then  // Infinity or NaN in IEEE format
            if IsZero(frac16) then
                fptype = FPType_Infinity;  value = 2.0^1000000;
            else
                fptype = if frac16[9] == '1' then FPType_QNaN else FPType_SNaN;
                value = 0.0;
            end;
        else
            fptype = FPType_Nonzero;
            value = 2.0^(UInt(exp16)-15) * (1.0 + Real(UInt(frac16)) * 2.0^-10);
        end;

    elsif N == 32 || isbfloat16 then
        var exp32 : bits(8);
        var frac32 : bits(23);
        if isbfloat16 then
            sign   = fpval[15];
            exp32  = fpval[14:7];
            frac32 = fpval[6:0] :: Zeros{16};
        else
            sign   = fpval[31];
            exp32  = fpval[30:23];
            frac32 = fpval[22:0];
        end;

        if IsZero(exp32) then
            if IsZero(frac32) then
                // Produce zero if value is zero.
                fptype = FPType_Zero;  value = 0.0;
            elsif fz || fiz then        // Flush-to-zero if FIZ==1 or AH,FZ==01
                fptype = FPType_Zero;  value = 0.0;
                // Check whether to generate Input Denormal floating-point exception.
                // fpcr.FIZ==1 does not generate Input Denormal exception.
                // Check whether to raise Input Denormal floating-point exception.
                // fpcr.FIZ==1 does not raise Input Denormal exception.
                if fz then
                    // Denormalized input flushed to zero
                    if fpexc then FPProcessException(FPExc_InputDenorm, fpcr); end;
                end;
            else
                fptype = FPType_Denormal;  value = 2.0^-126 * (Real(UInt(frac32)) * 2.0^-23);
```

```
            end;
        elsif IsOnes(exp32) then
            if IsZero(frac32) then
                fptype = FPType_Infinity;  value = 2.0^1000000;
            else
                fptype = if frac32[22] == '1' then FPType_QNaN else FPType_SNaN;
                value = 0.0;
            end;
        else
            fptype = FPType_Nonzero;
            value = 2.0^(UInt(exp32)-127) * (1.0 + Real(UInt(frac32)) * 2.0^-23);
        end;

    else // N == 64
        sign   = fpval[63];
        let exp64 : bits(11)  = fpval[62:52];
        let frac64 : bits(52) = fpval[51:0];

        if IsZero(exp64) then
            if IsZero(frac64) then
                // Produce zero if value is zero.
                fptype = FPType_Zero;  value = 0.0;
            elsif fz || fiz then        // Flush-to-zero if FIZ==1 or AH,FZ==01
                fptype = FPType_Zero;  value = 0.0;
                // Check whether to generate Input Denormal floating-point exception.
                // fpcr.FIZ==1 does not generate Input Denormal exception.
                // Check whether to raise Input Denormal floating-point exception.
                // fpcr.FIZ==1 does not raise Input Denormal exception.
                if fz then
                    // Denormalized input flushed to zero
                    if fpexc then FPProcessException(FPExc_InputDenorm, fpcr); end;
                end;
            else
                fptype = FPType_Denormal;  value = 2.0^-1022 * (Real(UInt(frac64)) * 2.0^-52);
            end;
        elsif IsOnes(exp64) then
            if IsZero(frac64) then
                fptype = FPType_Infinity;  value = 2.0^1000000;
            else
                fptype = if frac64[51] == '1' then FPType_QNaN else FPType_SNaN;
                value  = 0.0;
            end;
        else
            fptype = FPType_Nonzero;
            value  = 2.0^(UInt(exp64)-1023) * (1.0 + Real(UInt(frac64)) * 2.0^-52);
        end;
    end;

    if sign == '1' then value = -value; end;

    return (fptype, sign, value);
end;
```

Library pseudocode for shared/functions/impdef/ImpDef

```
// ImpDef()
// ========
// IMPLEMENTATION DEFINED behavior identified by the string parameter.

impdef func ImpDef(x : string)
begin
    return;
end;
```

Library pseudocode for shared/functions/impdef/ImpDefBit

```
// ImpDefBit()
// ===========
// Return an IMPLEMENTATION DEFINED bit value identified by the string parameter.

readonly impdef func ImpDefBit(x : string) => bit
begin
    return ARBITRARY : bit;
end;
```

Library pseudocode for shared/functions/impdef/ImpDefBits

```
// ImpDefBits()
// ============
// Return an IMPLEMENTATION DEFINED value of bits identified by the string parameter.

readonly impdef func ImpDefBits{N}(x : string) => bits(N)
begin
    return ARBITRARY : bits(N);
end;
```

Library pseudocode for shared/functions/impdef/ImpDefBool

```
// ImpDefBool()
// ============
// Return an IMPLEMENTATION DEFINED boolean value identified by the string parameter.

readonly impdef func ImpDefBool(x : string) => boolean
begin
    return ARBITRARY : boolean;
end;
```

Library pseudocode for shared/functions/impdef/ImpDefInt

```
// ImpDefInt()
// ===========
// Return an IMPLEMENTATION DEFINED integer value identified by the string parameter.

readonly impdef func ImpDefInt(x : string) => integer
begin
    return ARBITRARY : integer;
end;

type UnimplementedException of exception{-};
```

Library pseudocode for shared/functions/impdef/Unimplemented

```
// Unimplemented()
// ===============
// Behavior that is not implemented by this pseudocode.

noreturn impdef func Unimplemented()
begin
    throw UnimplementedException{-};
end;
```

Library pseudocode for shared/functions/memory/CreateAccDescLS64

```
// CreateAccDescLS64()
// ===================
// Access descriptor for accelerator-supporting memory accesses

func CreateAccDescLS64(memop : MemOp, withstatus : boolean, acqrel : boolean,
, withstatus : boolean,
                       tagchecked : boolean) => AccessDescriptor
begin
    var accdesc : AccessDescriptor = NewAccDesc(AccessType_GPR);

    accdesc.read            = memop == MemOp_LOAD;
    accdesc.write           = memop == MemOp_STORE;
    accdesc.acqsc           = memop ==     accdesc.pan             = TRUE;
    accdesc.ls64            = TRUE;
    accdesc.withstatus      = withstatus;
    accdesc.tagchecked      = tagchecked;

    return accdesc;
end; MemOp_LOAD && acqrel;
    accdesc.relsc           = memop == MemOp_STORE && acqrel;
    accdesc.pan             = TRUE;
    accdesc.ls64            = TRUE;
    accdesc.withstatus      = withstatus;
    accdesc.tagchecked      = tagchecked;

    return accdesc;
end;
```

Library pseudocode for shared/functions/mpam/GenMPAMTRBE

```
// GenMPAMTRBE()
// =============
// Returns MPAMinfo for Trace Buffer Unit in external mode.

func GenMPAMTRBE() => MPAMinfo
begin
    assert !SelfHostedTraceEnabled();
    var pspace : PARTIDSpaceType;
    if IsFeatureImplemented(FEAT_MPAMv2) then
        if !IsFeatureImplemented(FEAT_TRBE_MPAM) || TRBMPAM_EL1().EN == '0' then
            return DefaultMPAMInfo();
        end;
    else
        case TRBMAR_EL1().PAS of
            when '00' => pspace = PIDSpace_Secure;
            when '01' => pspace = PIDSpace_NonSecure;
            when '10' => pspace = PIDSpace_Root;
            when '11' => pspace = PIDSpace_Realm;
        end;

        if !IsFeatureImplemented(FEAT_TRBE_MPAM) || TRBMPAM_EL1().EN == '0' then
            return DefaultMPAMInfo(pspace);
        end;
    end;

    assert IsFeatureImplemented(FEAT_TRBE_MPAM);

    var mpaminfo : MPAMinfo;
    if !IsFeatureImplemented(FEAT_MPAMv2) then mpaminfo.mpam_sp = pspace; end;
    mpaminfo.partid = (if IsFeatureImplemented(FEAT_MPAMv2) then
                           TRBMPAM_EL1().MPAMv2.PARTID
                       else
                           TRBMPAM_EL1().MPAMv0or1px.PARTID);
    mpaminfo.partid = TRBMPAM_EL1().PARTID;
    var partid_out_of_range : boolean = FALSE;

    // When the PARTID is out-of-range, the behavior is a CONSTRAINED UNPREDICTABLE choice of:
    //     - Replace the out-of-range PARTID with any in-range PARTID
    //     - Replace the out-of-range PARTID with the default PARTID
    let partid_max : integer =    if UInt(mpaminfo.partid) > UInt(MPAMIDR_EL1().PARTID_MAX) then
        partid_out_of_range = TRUE;
        var c : GetMPAM_PARTID_MAX();
    if UInt(mpaminfo.partid) > partid_max then
        partid_out_of_range = TRUE;
        var c : Constraint;
        var partid : integer;
        (c, partid) = ConstrainUnpredictableInteger(0, partid_max,(0, UInt(MPAMIDR_EL1().PARTID_MAX),
Unpredictable_OUT_OF_RANGE_ID);
        assert c IN {Constraint_DEFAULT_ID, Constraint_ANY_ID};
        case c of
            when Constraint_DEFAULT_ID => mpaminfo.partid = DEFAULT_PARTID;
            when Constraint_ANY_ID     => mpaminfo.partid = partid[15:0];
        end;
    end;

    if partid_out_of_range && ConstrainUnpredictableBool(Unpredictable_USE_DEFAULT_PMG) then
        mpaminfo.pmg = DEFAULT_PMG;
    else
        mpaminfo.pmg = (if IsFeatureImplemented(FEAT_MPAMv2) then
                            TRBMPAM_EL1().MPAMv2.PMG
                        else
                            ZeroExtend{16}(TRBMPAM_EL1().MPAMv0or1px.PMG));
    end;

    let pmg_max : integer = GetMPAM_PMG_MAX();
    if UInt(mpaminfo.pmg) > pmg_max then
;
    else
        mpaminfo.pmg = TRBMPAM_EL1().PMG;
    end;

    if UInt(mpaminfo.pmg) > UInt(MPAMIDR_EL1().PMG_MAX) then
        var c : Constraint;
        var pmg : integer;
        (c, pmg) = ConstrainUnpredictableInteger(0, pmg_max,(0, UInt(MPAMIDR_EL1().PMG_MAX),
Unpredictable_OUT_OF_RANGE_ID);
        assert c IN {Constraint_DEFAULT_ID, Constraint_ANY_ID};
        case c of
            when Constraint_DEFAULT_ID => mpaminfo.pmg = DEFAULT_PMG;
            when Constraint_ANY_ID     => mpaminfo.pmg = pmg[15:0];
        end;
    end;

    return mpaminfo;
end;
```

Library pseudocode for shared/functions/mpam/GenPARTID

```
// GenPARTID()
// ===========
// Returns physical PARTID and error boolean for Exception level el.
// If mpamdata.sm is TRUE then PARTID is from MPAMSM_EL1.
// If mpamdata.ind is TRUE and alternative ID is enabled then PARTID is from MPAMn_ELx.altPARTID.
// Otherwise, the PARTID is from MPAMn_ELx.

func GenPARTID(el : bits(2), mpamdata : MPAMdata) => (PARTIDType, boolean)
begin
    var partidel : PARTIDType = GetMPAM_PARTID(el, mpamdata);
    var partid_err : boolean  = FALSE;

    // When the PARTID is out-of-range, the behavior is a CONSTRAINED UNPREDICTABLE choice of:
    // When the PARTID is out-of-range, the behaviour is a CONSTRAINED UNPREDICTABLE choice of:
    //     - Replace the out-of-range PARTID with any in-range PARTID
    //     - Replace the out-of-range PARTID with the default PARTID
    if UInt(partidel) >    if UInt(partidel) > UInt(MPAMIDR_EL1().PARTID_MAX) then
        var c : GetMPAM_PARTID_MAX() then
        var c : Constraint;
        var partid : integer;
        partid_err = TRUE;
        (c, partid) = ConstrainUnpredictableInteger(0, GetMPAM_PARTID_MAX(),(0, UInt(MPAMIDR_EL1().PARTID_MAX),
Unpredictable_OUT_OF_RANGE_ID);
        assert c IN {Constraint_DEFAULT_ID, Constraint_ANY_ID};
        case c of
            when Constraint_DEFAULT_ID => partidel = DEFAULT_PARTID;
            when Constraint_ANY_ID     => partidel = partid[15:0];
        end;
    end;

    if MPAMIsVirtual(el, mpamdata) then
        return MAP_vPARTID(partidel, partid_err);
    else
        return (partidel, partid_err);
    end;
end;
```

Library pseudocode for shared/functions/mpam/GenPMG

```
// GenPMG()
// ========
// Returns PMG for Exception level el.
// if mpamdata.sm is TRUE then PMG is from MPAMSM_EL1.
// If mpamdata.ind is TRUE and alternative id is enabled, then the PMG is from MPAMn_ELx.altPMG.
// Otherwise, PMG is from MPAMn_ELx.
// If PARTID generation (GenPARTID) encountered an error, GenPMG() should be
// called with partid_err as TRUE.

func GenPMG(el : bits(2), mpamdata : MPAMdata, partid_err : boolean) => PMGType
begin
    if (!IsFeatureImplemented(FEAT_MPAMv2) && partid_err &&
          ConstrainUnpredictableBool(Unpredictable_USE_DEFAULT_PMG)) then
        return DEFAULT_PMG;
    end;

    let groupel : PMGType = GetMPAM_PMG(el, mpamdata);
    let pmg_max : integer =    if UInt(groupel) <= UInt(MPAMIDR_EL1().PMG_MAX) then
        return groupel;
    end;
    return GetMPAM_PMG_MAX();
    if UInt(groupel) <= pmg_max then
        return groupel;
    end;
    return DEFAULT_PMG;
end;
```

Library pseudocode for shared/functions/mpam/GetMPAM_PARTID_MAX

```
// GetMPAM_PARTID_MAX()
// ====================
// Returns the maximum supported PARTID value for the implemented MPAM version.

func GetMPAM_PARTID_MAX() => integer
begin
    if IsFeatureImplemented(FEAT_MPAMv2) then
        return UInt(MPAMIDR_EL1().MPAMv2.PARTID_MAX);
    end;
    return UInt(MPAMIDR_EL1().MPAMv0or1px.PARTID_MAX);
end;
```

Library pseudocode for shared/functions/mpam/GetMPAM_PMG_MAX~~**Library pseudocode for shared/functions/mpam/MAP_vPARTID**~~

```
// GetMPAM_PMG_MAX()
// =================
// Returns the maximum supported PMG value for the implemented MPAM version.
// MAP_vPARTID()
// =============
// Performs conversion of virtual PARTID into physical PARTID
// Contains all of the error checking and implementation
// choices for the conversion.

func GetMPAM_PMG_MAX() => integer
begin
    if IsFeatureImplemented(FEAT_MPAMv2) then
        return UInt(MPAMIDR_EL1().MPAMv2.PMG_MAX);
    end;
    return UInt(MPAMIDR_EL1().MPAMv0or1px.PMG_MAX);
end;MAP_vPARTID(vpartid :PARTIDType, partid_err : boolean) => (PARTIDType, boolean)
begin
    // should not ever be called if EL2 is not implemented
    // or is implemented but not enabled in the current
    // security state.
    var ret : PARTIDType;
    var err : boolean  = partid_err;
    var virt : integer = UInt(vpartid);

    // The largest vpartid supported
    let vpartid_max : integer = (UInt(MPAMIDR_EL1().VPMR_MAX) << 2) + 3;

    if virt > vpartid_max then
        let (c, vid) = ConstrainUnpredictableInteger(0, vpartid_max,
                                                     Unpredictable_OUT_OF_RANGE_VID);
        assert c IN {Constraint_DEFAULT_ID, Constraint_ANY_ID};
        case c of
            when Constraint_DEFAULT_ID => virt = UInt(DEFAULT_PARTID);
            when Constraint_ANY_ID     => virt = vid;
        end;
    end;

    // Check for valid mapping entry.
    if MPAMVPMV_EL2()[virt] == '1' then
        // vpartid has a valid mapping, access the map.
        ret = mapvpmw(virt);

    // Check for default virtual PARTID valid
    elsif MPAMVPMV_EL2()[0] == '1' then
        // Use default mapping for vpartid == 0.
        ret = MPAMVPM0_EL2()[0 +: 16];

    // Neither is valid, use default physical PARTID.
    else
        ret = DEFAULT_PARTID;
        err = TRUE;
    end;

    // Check that the physical PARTID is in range.
    // This physical PARTID came from a virtual mapping entry.
    let partid_max : integer = UInt(MPAMIDR_EL1().PARTID_MAX);
    if UInt(ret) > partid_max then
        // Out of range, so return default physical PARTID
        ret = DEFAULT_PARTID;
        err = TRUE;
    end;
    return (ret, err);
end;
```

Library pseudocode for shared/functions/mpam/MAP_vPARTID

```
// MAP_vPARTID()
// =============
// Performs conversion of virtual PARTID into physical PARTID
// Contains all of the error checking and implementation
// choices for the conversion.

func MAP_vPARTID(vpartid : PARTIDType, partid_err : boolean) => (PARTIDType, boolean)
begin
    // should not ever be called if EL2 is not implemented
    // or is implemented but not enabled in the current
    // security state.
    var ret : PARTIDType;
    var err : boolean  = partid_err;
    var virt : integer = UInt(vpartid);

    // The largest vpartid supported
    let vpartid_max : integer = (UInt(MPAMIDR_EL1().VPMR_MAX) << 2) + 3;

    if virt > vpartid_max then
        let (c, vid) = ConstrainUnpredictableInteger(0, vpartid_max,
                                                     Unpredictable_OUT_OF_RANGE_VID);
        assert c IN {Constraint_DEFAULT_ID, Constraint_ANY_ID};
        case c of
            when Constraint_DEFAULT_ID => virt = UInt(DEFAULT_PARTID);
            when Constraint_ANY_ID     => virt = vid;
        end;
    end;

    // Check for valid mapping entry.
    if MPAMVPMV_EL2()[virt] == '1' then
        // vpartid has a valid mapping, access the map.
        ret = mapvpmw(virt);

    // Check for default virtual PARTID valid
    elsif MPAMVPMV_EL2()[0] == '1' then
        // Use default mapping for vpartid == 0.
        ret = MPAMVPM0_EL2()[0 +: 16];

    // Neither is valid, use default physical PARTID.
    else
        ret = DEFAULT_PARTID;
        err = TRUE;
    end;

    // Check that the physical PARTID is in-range.
    // This physical PARTID came from a virtual mapping entry.
    let partid_max : integer = GetMPAM_PARTID_MAX();
    if UInt(ret) > partid_max then
        // Out of range, so return default physical PARTID
        ret = DEFAULT_PARTID;
        err = TRUE;
    end;
    return (ret, err);
end;
```

Library pseudocode for shared/functions/predictionrestrict/ASID

```
// ASID
// ====
// Effective ASID.

func ASID() => bits(NUM_ASIDBITS)
begin
    if ELIsInHost(EL0) then
        var asid : bits(NUM_ASIDBITS);
        if TCR_EL2().A1 == '1' then
            asid = TTBR1_EL2().ASID;
            return TTBR1_EL2().ASID;
        else
            asid = TTBR0_EL2().ASID;
        end;
        if IsFeatureImplemented(FEAT_ASID16) && TCR_EL2().AS == '1' then
            return asid;
        else
            return ZeroExtend{NUM_ASIDBITS}(asid[7:0]);
            return TTBR0_EL2().ASID;
        end;
    end;
    if !ELUsingAArch32(EL1) then
        var asid : bits(NUM_ASIDBITS);
        if TCR_EL1().A1 == '1' then
            asid = TTBR1_EL1().ASID;
            return TTBR1_EL1().ASID;
        else
            asid = TTBR0_EL1().ASID;
        end;
        if IsFeatureImplemented(FEAT_ASID16) && TCR_EL1().AS == '1' then
            return asid;
        else
            return ZeroExtend{NUM_ASIDBITS}(asid[7:0]);
            return TTBR0_EL1().ASID;
        end;
    else
        if TTBCR().EAE == '0' then
            return ZeroExtend{NUM_ASIDBITS}(CONTEXTIDR().ASID);
        else
            if TTBCR().A1 == '1' then
                return ZeroExtend{NUM_ASIDBITS}(TTBR1().ASID);
            else
                return ZeroExtend{NUM_ASIDBITS}(TTBR0().ASID);
            end;
        end;
    end;
end;
```

Library pseudocode for shared/functions/registers/UnimplementedIDRegister

```
// UnimplementedIDRegister()
// =========================
// Trap access to unimplemented encodings in the feature ID register space.

func UnimplementedIDRegister()
begin
    if IsFeatureImplemented(FEAT_IDST) then
        if        var target_el : bits(2) = EffectivelyAtEL0InHostPSTATE() ||.EL;
        if EffectivelyAtEL0NotInHostPSTATE() then.EL ==
AArch64_SystemAccessTraptoELlorEL2EL0(0x18);
        elsethen
            target_el = if
EL2Enabled() && HCR_EL2().TGE == '1' then EL2 else EL1;
        end;
        AArch64_SystemAccessTrap(PSTATE.EL, 0x18);
        end;
(target_el, 0x18);
    end;
    Undefined();
end;
```

Library pseudocode for shared/functions/system/EffectiveHCR_EL2_TERR~~Library pseudocode for shared/functions/system/EffectiveSCR_EL3_ADEn~~

```
// EffectiveHCR_EL2_TERR()
// EffectiveSCR_EL3_ADEn()
// =======================
// Returns the Effective value of HCR_EL2().TERR.
// Returns the Effective value of SCR_EL3().ADEn.

readonly func EffectiveHCR_EL2_TERR() => bit
EffectiveSCR_EL3_ADEn() => bit
begin
    assert IsFeatureImplemented(FEAT_RAS);
    if !    assert !EL2Enabled() then return '0'; end;

    assert !ELUsingAArch32();
    assert IsFeatureImplemented(FEAT_LS64_ACCDATA);
    if !HaveEL(EL3EL2EL3);
) then return '1'; end;

    if (NUM_ERROR_RECORDS == 0 &&
            ImpDefBool("HCR_EL2.TERR,SCR_EL3.TERR/TWERR=0 when ERRIDR_EL1.NUM==0")) then
        return '0';
    end;

    return HCR_EL2().TERR;
    return SCR_EL3().ADEn;
end;
```

Library pseudocode for shared/functions/system/EffectiveIESB~~Library pseudocode for shared/functions/system/EffectiveSCR_EL3_AIEn~~

```
// EffectiveIESB()
// ===============
// Returns the Effective value of SCTLR_ELx().IESB.
// EffectiveSCR_EL3_AIEn()
// =======================
// Returns the Effective value of SCR_EL3().AIEn.

funcreadonly func EffectiveIESB(target_el : bits(2)) => bit
EffectiveSCR_EL3_AIEn() => bit
begin
    if    assert ! EffectiveNMEAELUsingAArch32(target_el) == '1' then return '1'; end;

    return( );
    assert IsFeatureImplemented(FEAT_AIE);
    if !HaveEL(EL3SCTLR_ELEL3(target_el).IESB;
) then return '1'; end;

    return SCR_EL3().AIEn;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_ADEn~~Library pseudocode for shared/functions/system/EffectiveSCR_EL3_AMVOFFEN~~

```
// EffectiveSCR_EL3_ADEn()
// =======================
// Returns the Effective value of SCR_EL3().ADEn.

// EffectiveSCR_EL3_AMVOFFEN()
// ===========================
// Returns the Effective value of SCR_EL3().AMVOFFEN.
readonly func EffectiveSCR_EL3_ADEn() => bit
EffectiveSCR_EL3_AMVOFFEN() => bit
begin
    assert IsFeatureImplemented(FEAT_LS64_ACCDATA);
    if !    assert !HaveELELUsingAArch32(EL3) then return '1'; end;

    assert !);
    assert IsFeatureImplemented(FEAT_AMUv1p1);
    if !ELUsingAArch32HaveEL(EL3);
) then return '1'; end;

    return SCR_EL3().ADEn;
    return SCR_EL3().AMVOFFEN;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_AIEn~~Library pseudocode for shared/functions/system/EffectiveSCR_EL3_API~~

```
// EffectiveSCR_EL3_AIEn()
// =======================
// Returns the Effective value of SCR_EL3().AIEn.
// EffectiveSCR_EL3_API()
// ======================
// Returns the Effective value of SCR_EL3().API.

readonly func EffectiveSCR_EL3_AIEn() => bit
EffectiveSCR_EL3_API() => bit
begin
    assert IsFeatureImplemented(FEAT_AIE);
    if !    assert !HaveEL(EL3) then return '1'; end;

    assert !);
    assert IsFeatureImplemented(FEAT_PAuth);
    if !ELUsingAArch32HaveEL(EL3);
) then return '1'; end;

    return SCR_EL3().AIEn;
    return SCR_EL3().API;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_AMVOFFEN~~Library pseudocode for shared/functions/system/EffectiveSCR_EL3_APK~~

```
// EffectiveSCR_EL3_AMVOFFEN()
// ===========================
// Returns the Effective value of SCR_EL3().AMVOFFEN.
// EffectiveSCR_EL3_APK()
// ======================
// Returns the Effective value of SCR_EL3().APK.

readonly func EffectiveSCR_EL3_AMVOFFEN() => bit
EffectiveSCR_EL3_APK() => bit
begin
    assert IsFeatureImplemented(FEAT_AMUv1p1);
    if !    assert !HaveEL(EL3) then return '1'; end;

    assert !);
    assert IsFeatureImplemented(FEAT_PAuth);
    if !ELUsingAArch32HaveEL(EL3);
) then return '1'; end;

    return SCR_EL3().AMVOFFEN;
    return SCR_EL3().APK;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_API~~Library pseudocode for shared/functions/system/EffectiveSCR_EL3_ATA~~

```
// EffectiveSCR_EL3_API()
// EffectiveSCR_EL3_ATA()
// ======================
// Returns the Effective value of SCR_EL3().API.
// Returns the Effective value of SCR_EL3().ATA.

readonly func EffectiveSCR_EL3_API() => bit
EffectiveSCR_EL3_ATA() => bit
begin
    assert IsFeatureImplemented(FEAT_PAuth);
    if !    assert !HaveEL(EL3) then return '1'; end;

    assert !);
    assert IsFeatureImplemented(FEAT_MTE2);
    if !ELUsingAArch32HaveEL(EL3);
) then return '1'; end;

    return SCR_EL3().API;
    return SCR_EL3().ATA;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_APK

```
// EffectiveSCR_EL3_APK()
// ======================
// Returns the Effective value of SCR_EL3().APK.

readonly func EffectiveSCR_EL3_APK() => bit
begin
    assert IsFeatureImplemented(FEAT_PAuth);
    if !HaveEL(EL3) then return '1'; end;

    assert !ELUsingAArch32(EL3);

    return SCR_EL3().APK;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_ATA

```
// EffectiveSCR_EL3_ATA()
// ======================
// Returns the Effective value of SCR_EL3().ATA.

readonly func EffectiveSCR_EL3_ATA() => bit
begin
    assert IsFeatureImplemented(FEAT_MTE2);
    if !HaveEL(EL3) then return '1'; end;

    assert !ELUsingAArch32(EL3);

    return SCR_EL3().ATA;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_DSE

```
// EffectiveSCR_EL3_DSE()
// ======================
// Returns the Effective value of SCR_EL3().DSE.

readonly func EffectiveSCR_EL3_DSE() => bit
begin
    assert IsFeatureImplemented(FEAT_E3DSE);
    if !    assert !HaveELELUsingAArch32(EL3) then return '0'; end;

    assert !);
    assert IsFeatureImplemented(FEAT_E3DSE);
    if !ELUsingAArch32HaveEL(EL3);

) then return '0'; end;
    if EffectiveSCR_EL3_EnDSE() == '0' then return '0'; end;

    return SCR_EL3().DSE;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_EA

```
// EffectiveSCR_EL3_EA()
// =====================
// Returns the Effective value of SCR_EL3().EA.

readonly func EffectiveSCR_EL3_EA() => bit
begin
    if !    assert !HaveELELUsingAArch32(EL3) then return '0'; end;

    assert !);
    if !ELUsingAArch32HaveEL(EL3);
) then return '0'; end;

    return SCR_EL3().EA;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_EASE

```
// EffectiveSCR_EL3_EASE()
// =======================
// Returns the Effective value of SCR_EL3().EASE.

readonly func EffectiveSCR_EL3_EASE() => bit
begin
    assert IsFeatureImplemented(FEAT_DoubleFault);
    assert    assert ! HaveELELUsingAArch32(EL3);
    assert !    assert IsFeatureImplemented(FEAT_DoubleFault);
    assertELUsingAArch32HaveEL(EL3);

    return SCR_EL3().EASE;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_ECVEn

```
// EffectiveSCR_EL3_ECVEn()
// ========================
// Returns the Effective value of SCR_EL3().ECVEn.

readonly func EffectiveSCR_EL3_ECVEn() => bit
begin
    assert IsFeatureImplemented(FEAT_ECV_POFF);
    if !    assert !HaveELELUsingAArch32(EL3) then return '1'; end;

    assert !);
    assert IsFeatureImplemented(FEAT_ECV_POFF);
    if !ELUsingAArch32HaveEL(EL3);
) then return '1'; end;

    return SCR_EL3().ECVEn;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_EEL2

```
// EffectiveSCR_EL3_EEL2()
// =======================
// Returns the Effective value of SCR_EL3().EEL2.

readonly func EffectiveSCR_EL3_EEL2() => bit
begin
    assert IsFeatureImplemented(FEAT_SEL2);
    if !    assert !HaveELELUsingAArch32(EL3) then return '1'; end;

    assert !);
    assert IsFeatureImplemented(FEAT_SEL2);
    if !ELUsingAArch32HaveEL(EL3);
) then return '1'; end;

    return SCR_EL3().EEL2;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_EnAS0

```
// EffectiveSCR_EL3_EnAS0()
// ========================
// Returns the Effective value of SCR_EL3().EnAS0.

readonly func EffectiveSCR_EL3_EnAS0() => bit
begin
    assert IsFeatureImplemented(FEAT_LS64_ACCDATA);
    if !    assert !HaveELELUsingAArch32(EL3) then return '1'; end;

    assert !);
    assert IsFeatureImplemented(FEAT_LS64_ACCDATA);
    if !ELUsingAArch32HaveEL(EL3);
) then return '1'; end;

    return SCR_EL3().EnAS0;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_EnDSE

```
// EffectiveSCR_EL3_EnDSE()
// ========================
// Returns the Effective value of SCR_EL3().EnDSE.

readonly func EffectiveSCR_EL3_EnDSE() => bit
begin
    assert IsFeatureImplemented(FEAT_E3DSE);
    assert !ELUsingAArch32(EL3);

    assert IsFeatureImplemented(FEAT_E3DSE);
    if SCR_EL3().EA == '0' && EffectiveSCR_EL3_TMEA() == '0' then return '0'; end;

    return SCR_EL3().EnDSE;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_EnSCXT

```
// EffectiveSCR_EL3_EnSCXT()
// =========================
// Returns the Effective value of SCR_EL3().EnSCXT.

readonly func EffectiveSCR_EL3_EnSCXT() => bit
begin
    assert (IsFeatureImplemented(FEAT_CSV2_2) || IsFeatureImplemented(FEAT_CSV2_1p2));
    if !    assert !HaveELELUsingAArch32(EL3) then return '1'; end;

    assert !);
    assert (IsFeatureImplemented(FEAT_CSV2_2) || IsFeatureImplemented(FEAT_CSV2_1p2));
    if !ELUsingAArch32HaveEL(EL3);
) then return '1'; end;

    return SCR_EL3().EnSCXT;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_FGTEn

```
// EffectiveSCR_EL3_FGTEn()
// ========================
// Returns the Effective value of SCR_EL3().FGTEn.

readonly func EffectiveSCR_EL3_FGTEn() => bit
begin
    assert IsFeatureImplemented(FEAT_FGT);
    if !    assert !HaveELELUsingAArch32(EL3) then return '1'; end;

    assert !);
    assert IsFeatureImplemented(FEAT_FGT);
    if !ELUsingAArch32HaveEL(EL3);
) then return '1'; end;

    return SCR_EL3().FGTEn;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_FGTEn2

```
// EffectiveSCR_EL3_FGTEn2()
// =========================
// Returns the Effective value of SCR_EL3().FGTEn2.

readonly func EffectiveSCR_EL3_FGTEn2() => bit
begin
    assert IsFeatureImplemented(FEAT_FGT2);
    if !    assert !HaveELELUsingAArch32(EL3) then return '1'; end;

    assert !);
    assert IsFeatureImplemented(FEAT_FGT2);
    if !ELUsingAArch32HaveEL(EL3);
) then return '1'; end;

    return SCR_EL3().FGTEn2;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_FIEN

```
// EffectiveSCR_EL3_FIEN()
// =======================
// Returns the Effective value of SCR_EL3().FIEN.

readonly func EffectiveSCR_EL3_FIEN() => bit
begin
    assert IsFeatureImplemented(FEAT_RASv1p1);
    if !    assert !HaveELELUsingAArch32(EL3) then return '1'; end;

    assert !);
    assert IsFeatureImplemented(FEAT_RASv1p1);
    if !ELUsingAArch32HaveEL(EL3);
) then return '1'; end;

    return SCR_EL3().FIEN;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_HCE

```
// EffectiveSCR_EL3_HCE()
// ======================
// Returns the Effective value of SCR_EL3().HCE.

readonly func EffectiveSCR_EL3_HCE() => bit
begin
    if !HaveEL(EL2) then return '0'; end;
    if !HaveEL(EL3) then return '1'; end;

    assert !ELUsingAArch32(EL3);

    return SCR_EL3().HCE;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_HXEn

```
// EffectiveSCR_EL3_HXEn()
// =======================
// Returns the Effective value of SCR_EL3().HXEn.

readonly func EffectiveSCR_EL3_HXEn() => bit
begin
    assert IsFeatureImplemented(FEAT_HCX);
    if !HaveEL(EL3) then return '1'; end;

    return SCR_EL3().HXEn;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_NMEA

```
// EffectiveSCR_EL3_NMEA()
// =======================
// Returns the Effective value of SCR_EL3().NMEA.

readonly func EffectiveSCR_EL3_NMEA() => bit
begin
    assert IsFeatureImplemented(FEAT_DoubleFault);
    if !HaveEL(EL3) then return '0'; end;

    if !IsFeatureImplemented(FEAT_DoubleFault2) && EffectiveSCR_EL3_EA() == '0' then
        return '0';
    end;

    return SCR_EL3().NMEA;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_PFAREn

```
// EffectiveSCR_EL3_PFAREn()
// =========================
// Returns the Effective value of SCR_EL3().PFAREn.

readonly func EffectiveSCR_EL3_PFAREn() => bit
begin
    assert IsFeatureImplemented(FEAT_PFAR);
    if !HaveEL(EL3) then return '1'; end;

    return SCR_EL3().PFAREn;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_PIEn

```
// EffectiveSCR_EL3_PIEn()
// =======================
// Returns the Effective value of SCR_EL3().PIEn.

readonly func EffectiveSCR_EL3_PIEn() => bit
begin
    assert (IsFeatureImplemented(FEAT_S1PIE) || IsFeatureImplemented(FEAT_S2PIE) ||
            IsFeatureImplemented(FEAT_S1POE) || IsFeatureImplemented(FEAT_S2POE));
    if !    assert !HaveELELUsingAArch32(EL3) then return '1'; end;

    assert !);
    assert (IsFeatureImplemented(FEAT_S1PIE) || IsFeatureImplemented(FEAT_S2PIE) ||
            IsFeatureImplemented(FEAT_S1POE) || IsFeatureImplemented(FEAT_S2POE));
    if !ELUsingAArch32HaveEL(EL3);
) then return '1'; end;

    return SCR_EL3().PIEn;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_RCWMASKEn

```
// EffectiveSCR_EL3_RCWMASKEn()
// ============================
// Returns the Effective value of SCR_EL3().RCWMASKEn.

readonly func EffectiveSCR_EL3_RCWMASKEn() => bit
begin
    assert IsFeatureImplemented(FEAT_THE);
    if !    assert !HaveELELUsingAArch32(EL3) then return '1'; end;

    assert !);
    assert IsFeatureImplemented(FEAT_THE);
    if !ELUsingAArch32HaveEL(EL3);
) then return '1'; end;

    return SCR_EL3().RCWMASKEn;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_SCTLR2En

```
// EffectiveSCR_EL3_SCTLR2En()
// ===========================
// Returns the Effective value of SCR_EL3().SCTLR2En.

readonly func EffectiveSCR_EL3_SCTLR2En() => bit
begin
    assert IsFeatureImplemented(FEAT_SCTLR2);
    if !    assert !HaveELELUsingAArch32(EL3) then return '1'; end;

    assert !);
    assert IsFeatureImplemented(FEAT_SCTLR2);
    if !ELUsingAArch32HaveEL(EL3);
) then return '1'; end;

    return SCR_EL3().SCTLR2En;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_SIF

```
// EffectiveSCR_EL3_SIF()
// ======================
// Returns the Effective value of SCR_EL3().SIF.

readonly func EffectiveSCR_EL3_SIF() => bit
begin
    if !    assert !HaveELELUsingAArch32(EL3) then return '0'; end;

    assert !);
    if !ELUsingAArch32HaveEL(EL3);

) then return '0'; end;
    if IsFeatureImplemented(FEAT_RME) && !IsFeatureImplemented(FEAT_SEL2) then
        return '0';
    end;

    return SCR_EL3().SIF;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_SMD

```
// EffectiveSCR_EL3_SMD()
// ======================
// Returns the Effective value of SCR_EL3().SMD.

readonly func EffectiveSCR_EL3_SMD() => bit
begin
    if !    assert !HaveELELUsingAArch32(EL3) then return '1'; end;

    assert !);
    if !ELUsingAArch32HaveEL(EL3);
) then return '1'; end;

    return SCR_EL3().SMD;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_SRMASKEn

```
// EffectiveSCR_EL3_SRMASKEn()
// ===========================
// Returns the Effective value of SCR_EL3().SRMASKEn.

readonly func EffectiveSCR_EL3_SRMASKEn() => bit
begin
    assert IsFeatureImplemented(FEAT_SRMASK);
    if !    assert !HaveELELUsingAArch32(EL3) then return '1'; end;

    assert !);
    assert IsFeatureImplemented(FEAT_SRMASK);
    if !ELUsingAArch32HaveEL(EL3);
) then return '1'; end;

    return SCR_EL3().SRMASKEn;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_ST

```
// EffectiveSCR_EL3_ST()
// =====================
// Returns the Effective value of SCR_EL3().ST.

readonly func EffectiveSCR_EL3_ST() => bit
begin
    if !    assert !HaveELELUsingAArch32(EL3) then return '1'; end;

    assert !);
    if !ELUsingAArch32HaveEL(EL3);
) then return '1'; end;

    return SCR_EL3().ST;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_TCR2En

```
// EffectiveSCR_EL3_TCR2En()
// =========================
// Returns the Effective value of SCR_EL3().TCR2En.

readonly func EffectiveSCR_EL3_TCR2En() => bit
begin
    assert IsFeatureImplemented(FEAT_TCR2);
    if !    assert !HaveELELUsingAArch32(EL3) then return '1'; end;

    assert !);
    assert IsFeatureImplemented(FEAT_TCR2);
    if !ELUsingAArch32HaveEL(EL3);
) then return '1'; end;

    return SCR_EL3().TCR2En;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_TERR

```
// EffectiveSCR_EL3_TERR()
// =======================
// Returns the Effective value of SCR_EL3().TERR.

readonly func EffectiveSCR_EL3_TERR() => bit
begin
    assert IsFeatureImplemented(FEAT_RAS);
    if !    assert !HaveELELUsingAArch32(EL3) then return '0'; end;

    assert !);
    assert IsFeatureImplemented(FEAT_RAS);
    if !ELUsingAArch32HaveEL(EL3);

    if (NUM_ERROR_RECORDS == 0 &&
            ImpDefBool("HCR_EL2.TERR,SCR_EL3.TERR/TWERR=0 when ERRIDR_EL1.NUM==0")) then
        return '0';
    end;
) then return '0'; end;

    return SCR_EL3().TERR;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_TID3

```
// EffectiveSCR_EL3_TID3()
// =======================
// Returns the Effective value of SCR_EL3().TID3.

readonly func EffectiveSCR_EL3_TID3() => bit
begin
    assert IsFeatureImplemented(FEAT_IDTE3);
    if !    assert !HaveELELUsingAArch32(EL3) then return '0'; end;

    assert !);
    assert IsFeatureImplemented(FEAT_IDTE3);
    if !ELUsingAArch32HaveEL(EL3);
) then return '0'; end;

    return SCR_EL3().TID3;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_TID5

```
// EffectiveSCR_EL3_TID5()
// =======================
// Returns the Effective value of SCR_EL3().TID5.

readonly func EffectiveSCR_EL3_TID5() => bit
begin
    assert IsFeatureImplemented(FEAT_IDTE3);
    assert IsFeatureImplemented(FEAT_MTE2);
    if !    assert !HaveELELUsingAArch32(EL3) then return '0'; end;

    assert !);
    assert IsFeatureImplemented(FEAT_IDTE3);
    assert IsFeatureImplemented(FEAT_MTE2);
    if !ELUsingAArch32HaveEL(EL3);
) then return '0'; end;

    return SCR_EL3().TID5;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_TLOR

```
// EffectiveSCR_EL3_TLOR()
// =======================
// Returns the Effective value of SCR_EL3().TLOR.

readonly func EffectiveSCR_EL3_TLOR() => bit
begin
    assert IsFeatureImplemented(FEAT_LOR);
    if !    assert !HaveELELUsingAArch32(EL3) then return '0'; end;

    assert !);
    assert IsFeatureImplemented(FEAT_LOR);
    if !ELUsingAArch32HaveEL(EL3);
) then return '0'; end;

    return SCR_EL3().TLOR;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_TRNDR

```
// EffectiveSCR_EL3_TRNDR()
// ========================
// Returns the Effective value of SCR_EL3().TRNDR.

readonly func EffectiveSCR_EL3_TRNDR() => bit
begin
    assert IsFeatureImplemented(FEAT_RNG_TRAP);
    if !    assert !HaveELELUsingAArch32(EL3) then return '0'; end;

    assert !);
    assert IsFeatureImplemented(FEAT_RNG_TRAP);
    if !ELUsingAArch32HaveEL(EL3);
) then return '0'; end;

    return SCR_EL3().TRNDR;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_TWE

```
// EffectiveSCR_EL3_TWE()
// ======================
// Returns the Effective value of SCR_EL3().TWE.

readonly func EffectiveSCR_EL3_TWE() => bit
begin
    if !    assert !HaveELELUsingAArch32(EL3) then return '0'; end;

    assert !);
    if !ELUsingAArch32HaveEL(EL3);
) then return '0'; end;

    return SCR_EL3().TWE;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_TWEDEL

```
// EffectiveSCR_EL3_TWEDEL()
// =========================
// Returns the Effective value of SCR_EL3().TWEDEL.

readonly func EffectiveSCR_EL3_TWEDEL() => bits(4)
begin
    assert IsFeatureImplemented(FEAT_TWED);
    if !    assert !HaveELELUsingAArch32(EL3) then return '0000'; end;

    assert !);
    assert IsFeatureImplemented(FEAT_TWED);
    if !ELUsingAArch32HaveEL(EL3);
) then return '0000'; end;

    return SCR_EL3().TWEDEL;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_TWEDEn

```
// EffectiveSCR_EL3_TWEDEn()
// =========================
// Returns the Effective value of SCR_EL3().TWEDEn.

readonly func EffectiveSCR_EL3_TWEDEn() => bit
begin
    assert IsFeatureImplemented(FEAT_TWED);
    if !    assert !HaveELELUsingAArch32(EL3) then return '0'; end;

    assert !);
    assert IsFeatureImplemented(FEAT_TWED);
    if !ELUsingAArch32HaveEL(EL3);
) then return '0'; end;

    return SCR_EL3().TWEDEn;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_TWERR

```
// EffectiveSCR_EL3_TWERR()
// ========================
// Returns the Effective value of SCR_EL3().TWERR.

readonly func EffectiveSCR_EL3_TWERR() => bit
begin
    assert IsFeatureImplemented(FEAT_RASv2);
    if !    assert !HaveELELUsingAArch32(EL3) then return '0'; end;

    assert !);
    assert IsFeatureImplemented(FEAT_RASv2);
    if !ELUsingAArch32HaveEL(EL3);

    if (NUM_ERROR_RECORDS == 0 &&
            ImpDefBool("HCR_EL2.TERR,SCR_EL3.TERR/TWERR=0 when ERRIDR_EL1.NUM==0")) then
        return '0';
    end;
) then return '0'; end;

    return SCR_EL3().TWERR;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_TWI

```
// EffectiveSCR_EL3_TWI()
// ======================
// Returns the Effective value of SCR_EL3().TWI.

readonly func EffectiveSCR_EL3_TWI() => bit
begin
    if !    assert !HaveELELUsingAArch32(EL3) then return '0'; end;

    assert !);
    if !ELUsingAArch32HaveEL(EL3);
) then return '0'; end;

    return SCR_EL3().TWI;
end;
```

Library pseudocode for shared/functions/system/EffectivelyAtEL0InHost

```
// EffectivelyAtEL0InHost()
// ========================
// ====================
// Accesses from EL0 when the Effective value of HCR_EL2().[E2H, TGE] is {1, 1} will be trapped
// or Undefined.

func EffectivelyAtEL0InHost() => boolean
begin
    return (PSTATE.EL == EL0 && ELIsInHost(EL0));
end;
```

Library pseudocode for shared/functions/system/EffectivelyAtEL0NotInHost

```
// EffectivelyAtEL0NotInHost()
// ===========================
// ======================
// Accesses from EL0 when the Effective value of HCR_EL2().[E2H, TGE] is not {1, 1} will be
// trapped or Undefined.

func EffectivelyAtEL0NotInHost() => boolean
begin
    return (PSTATE.EL == EL0 && !ELIsInHost(EL0));
end;
```

Library pseudocode for shared/functions/system/HaveELUsingSecurityState

```
// HaveELUsingSecurityState()
// ==========================
// Returns TRUE if Exception level 'el' with Security state 'ss' is supported, FALSE otherwise.
// Returns TRUE if Exception level 'el' with Security state 'secure' is supported,
// FALSE otherwise.

readonly func HaveELUsingSecurityState(el : bits(2), ss :HaveELUsingSecurityState(el : bits(2), secure : boolean) =>
boolean
begin

    case el of
        when SecurityStateEL3) => boolean
begin
    if !=>
            assert secure;
            returnHaveEL(el) then
        return FALSE;
    end;

    case el of
        when{ EL3 =>
            if IsFeatureImplemented(FEAT_RME) then
                return ss ==);
        when SS_RootEL2;
            end;

            return ss ==>
            if secure then
                return SS_SecureHaveEL;
        when{ EL2 =>
            case ss of
                when) && IsFeatureImplemented(FEAT_SEL2);
            else
                return SS_RealmHaveEL => return IsFeatureImplemented(FEAT_RME);
                when{ SS_NonSecureEL2 => return !);
            end;
        otherwise =>
            return {SecureOnlyImplementationHaveEL();
                when{ SS_SecureEL3    => return IsFeatureImplemented(FEAT_SEL2);
                otherwise         => return FALSE;
            end;
        when EL1 =>
            case ss of
                when SS_Realm     => return IsFeatureImplemented(FEAT_RME);
                when SS_NonSecure => return !SecureOnlyImplementation();
                when SS_Secure    => return IsFeatureImplemented(FEAT_Secure) && HaveAArch64();
                otherwise         => return FALSE;
            end;
        when EL0 =>
            case ss of
                when SS_Realm     => return IsFeatureImplemented(FEAT_RME);
                when SS_NonSecure => return !SecureOnlyImplementation();
                when SS_Secure => return IsFeatureImplemented(FEAT_Secure);
                otherwise         => return FALSE;
            end;
        otherwise =>
            unreachable;
) ||
                    (secure == ImpDefBool("Secure-only implementation")));
    end;
end;
```

Library pseudocode for shared/functions/system/HaveSecureState

```
// HaveSecureState()
// =================
// Return TRUE if Secure State is supported.

readonly func HaveSecureState() => boolean
begin
    return IsFeatureImplemented(FEAT_Secure);
end;    if !HaveEL(EL3) then
        return SecureOnlyImplementation();
    end;
    if IsFeatureImplemented(FEAT_RME) && !IsFeatureImplemented(FEAT_SEL2) then
        return FALSE;
    end;
    return TRUE;
end;
```

Library pseudocode for shared/functions/system/Hint_ScopeRelease

```
// Hint_ScopeRelease()
// ===================
// Signals to the memory system that if the next instruction in program order has Release semantics
// or is a Data Memory Barrier that defines a Barrier scope with a preceding Scoped Release
// Boundary, then these instructions do not create a Barrier-ordered-before relationship between
// any Effect in program order before the Scoped Release Boundary and the Effect with Release
// semantics or any Effect in program order after the DMB Effect.

impdef func Hint_ScopeRelease(stshstrm : boolean)
begin
    return;
end;
```

Library pseudocode for shared/functions/system/Hint_ScopedReleaseBoundary

```
// Hint_ScopedReleaseBoundary()
// ============================
// Signals to the memory system that a corresponding instruction with Release semantics or Data
// Memory Barrier will subsequently occur in program order, and if that corresponding instruction
// defines a Barrier scope with the Scoped Release Boundary, then these instructions do not create
// a Barrier-ordered-before relationship between any Effect in program order before the Scoped
// Release Boundary and the Effect with Release semantics or any Effect in program order after the
// DMB Effect.

impdef func Hint_ScopedReleaseBoundary()
begin
    return;
end;
```

Library pseudocode for shared/functions/system/IsSecureBelowEL3

```
// IsSecureBelowEL3()
// ==================
// Return TRUE if an Exception level below EL3 is in Secure state
// or would be following an exception return to that level.
//
// That is, if at AArch64 EL3 or in AArch32 Monitor mode, whether an
// exception return would pass to Secure or Non-secure state.

readonly func IsSecureBelowEL3() => boolean
begin
    if HaveEL(EL3) then
        return if !HaveAArch64() then SCR().NS == '0' else SCR_EL3().NS == '0';
    elsif HaveEL(EL2) && (!IsFeatureImplemented(FEAT_SEL2) || !HaveAArch64()) then
        // If Secure EL2 is not an architecture option then we must be Non-secure.
        return FALSE;
    else
        // TRUE if PE is Secure or FALSE if Non-secure.
        return~~ return ImpDefBool("Secure-only implementation");~~
    ~~end;~~
~~end;~~ SecureOnlyImplementation();
    end;
end;
```

Library pseudocode for shared/functions/system/NonSecureOnlyImplementation

```
// NonSecureOnlyImplementation()
// =============================
// Returns TRUE if the security state is always Non-secure for this implementation.

readonly func~~func~~ NonSecureOnlyImplementation() => boolean
begin
    return !IsFeatureImplemented(FEAT_Secure) && !~~ return ImpDefBool("Non-secure only implementation");~~
~~end;~~HaveEL(EL3);
end;
```

Library pseudocode for shared/functions/system/SecureOnlyImplementation

```
// SecureOnlyImplementation()
// ==========================
// Returns TRUE if the security state is always Secure for this implementation.

readonly func SecureOnlyImplementation() => boolean
begin
    return IsFeatureImplemented(FEAT_Secure) && !~~ return ImpDefBool("Secure-only implementation");~~
~~end;~~HaveEL(EL3);
end;
```

~~Library pseudocode for shared/functions/system/SmallestTranslationGranule~~

```
// SmallestTranslationGranule()
// ============================
// Smallest implemented translation granule.

func SmallestTranslationGranule() => integer
begin
    if IsFeatureImplemented(FEAT_TGran4K) then return 12; end;
    if IsFeatureImplemented(FEAT_TGran16K) then return 14; end;
    if IsFeatureImplemented(FEAT_TGran64K)  then return 16; end;
    unreachable;
end;
```

Library pseudocode for shared/functions/system/WatchpointRelatedSyndrome

```
// WatchpointRelatedSyndrome()
// ===========================
// Update common Watchpoint related fields.

func WatchpointRelatedSyndrome(fault : FaultRecord) => bits(24)
begin
    var syndrome : bits(24) = Zeros{};

    if fault.watchptinfo.maybe_false_match then
        syndrome[16] = '1';                                         // WPF
    elsif IsFeatureImplemented(FEAT_Debugv8p2) then
        syndrome[16] = ImpDefBit("WPF value on TRUE Watchpoint match");
    end;

    if IsRelaxedWatchpointAccess(fault.accessdesc) then
        if HaltOnBreakpointOrWatchpoint() then
            if ImpDefBool("EDWAR is not valid on watchpoint debug event") then
                syndrome[10] = '1';                                 // FnV
            end;
        else
            if ImpDefBool("FAR is not valid on watchpoint exception") then
                syndrome[10] = '1';                                 // FnV
            end;
        end;
    else
        if WatchpointFARNotPrecise(fault) then
            syndrome[15] = '1';                                     // FnP
        end;
    end;

    // Watchpoint number is valid if FEAT_Debugv8p9 is implemented or
    // if Feat_Debugv8p2 is implemented and below set of conditions are satisfied:
    // - Any of FnV = 1 or FnP = 1 or WPF = 1.
    // - Either FnV = 1 or FnP = 1.
    // - If the address recorded in FAR is not within a naturally-aligned block of memory.
    // Otherwise, it is IMPLEMENTATION DEFINED if watchpoint number is valid.
    if IsFeatureImplemented(FEAT_Debugv8p9) then
        syndrome[17]    = '1';                                      // WPTV
        syndrome[23:18] = fault.watchptinfo.watchpt_num[5:0];       // WPT
    elsif IsFeatureImplemented(FEAT_Debugv8p2) then
        if syndrome[15] == '1' || syndrome[10] == '1' || syndrome[16] == '1' then
        if syndrome[15] == '1' || syndrome[10] == '1' then          // Either of FnP or FnV is 1
            syndrome[17] = '1';                                     // WPTV
        elsif !AddressInNaturallyAlignedBlock(fault.vaddress, fault.watchptinfo.vaddress) then
            syndrome[17] = '1';                                     // WPTV
        elsif ImpDefBool("WPTV field is valid") then
            syndrome[17] = '1';
        end;
        if syndrome[17] == '1' then
            syndrome[23:18] = fault.watchptinfo.watchpt_num[5:0];   // WPT
        else
            syndrome[23:18] = ARBITRARY : bits(6);
        end;
    end;

    return syndrome;
end;
```

Library pseudocode for shared/functions/unpredictable/Unpredictable

```
// Unpredictable
// =============
// List of Constrained Unpredictable situations.

type Unpredictable of enumeration {
                            // VMSR on MVFR
                            Unpredictable_VMSR,
                            // Writeback/transfer register overlap (load)
                            Unpredictable_WBOVERLAPLD,
                            // Writeback/transfer register overlap (store)
                            Unpredictable_WBOVERLAPST,
                            // Load Pair transfer register overlap
                            Unpredictable_LDPOVERLAP,
                            // Store-exclusive base/status register overlap
                            Unpredictable_BASEOVERLAP,
                            // Store-exclusive data/status register overlap
                            Unpredictable_DATAOVERLAP,
                            // Load-store alignment checks
                            Unpredictable_DEVPAGE2,
                            // Instruction fetch from Device memory
                            Unpredictable_INSTRDEVICE,
                            // Reserved CPACR value
                            Unpredictable_RESCPACR,
                            // Reserved MAIR value
                            Unpredictable_RESMAIR,
                            // Effect of SCTLR_ELx.C on Tagged attribute
                            Unpredictable_S1CTAGGED,
                            // Reserved Stage 2 MemAttr value
                            Unpredictable_S2RESMEMATTR,
                            // Reserved TEX::C::B value
                            Unpredictable_RESTEXCB,
                            // Reserved PRRR value
                            Unpredictable_RESPRRR,
                            // Reserved DACR field
                            Unpredictable_RESDACR,
                            // Reserved VTCR.S value
                            Unpredictable_RESVTCRS,
                            // Reserved or out-of-range TCR.TnSZ value
                            Unpredictable_RESTnSZ,
                            // Reserved SCTLR_ELx.TCF value
                            Unpredictable_RESTCF,
                            // Tag stored to Device memory
                            Unpredictable_DEVICETAGSTORE,
                            // Syndrome for a known-passing conditional A32 instruction
                            Unpredictable_ESRCONDPASS,
                            // Illegal State exception: zero PSTATE.IT
                            Unpredictable_ILZEROIT,
                            // Illegal State exception: zero PSTATE.T
                            Unpredictable_ILZEROT,
                            // Debug: prioritization of Vector Catch
                            Unpredictable_BPVECTORCATCHPRI,
                            // Debug Vector Catch: match on 2nd halfword
                            Unpredictable_VCMATCHHALF,
                            // Debug Vector Catch: match on Data Abort
                            // or Prefetch abort
                            Unpredictable_VCMATCHDAPA,
                            // Debug watchpoints: nonzero MASK and non-ones BAS
                            Unpredictable_WPMASKANDBAS,
                            // Debug watchpoints: non-contiguous BAS
                            Unpredictable_WPBASCONTIGUOUS,
                            // Debug watchpoints: reserved MASK
                            Unpredictable_RESWPMASK,
                            // Debug watchpoints: nonzero MASKed bits of address
                            Unpredictable_WPMASKEDBITS,
                            // Debug breakpoints and watchpoints: reserved control bits
                            Unpredictable_RESBPWPCTRL,
                            // Debug breakpoints: not implemented
                            Unpredictable_BPNOTIMPL,
                            // Debug breakpoints: reserved type
                            Unpredictable_RESBPTYPE,
                            // Debug breakpoints and watchpoints: reserved MDSELR_EL1.BANK
                            Unpredictable_RESMDSELR,
                            // Debug breakpoints: not-context-aware breakpoint
                            Unpredictable_BPNOTCTXCMP,
                            // Debug breakpoints: match on 2nd halfword of instruction
                            Unpredictable_BPMATCHHALF,
                            // Debug breakpoints: mismatch on 2nd halfword of instruction
                            Unpredictable_BPMISMATCHHALF,
                            // Debug breakpoints: a breakpoint is linked to that is not
                            // programmed with linking enabled
                            Unpredictable_BPLINKINGDISABLED,
                            // Debug breakpoints: reserved MASK
                            Unpredictable_RESBPMASK,
                            // Debug breakpoints: MASK is set for a Context matching
                            // breakpoint or when DBGBCR_EL1(n).BAS != '1111'
                            Unpredictable_BPMASK,
                            // Debug breakpoints: nonzero MASKed bits of address
                            Unpredictable_BPMASKEDBITS,
                            // Debug breakpoints: A linked breakpoint is
```

```
                        // linked to an address matching breakpoint
                        Unpredictable_BPLINKEDADDRMATCH,
                        // Debug: restart to a misaligned AArch32 PC value
                        Unpredictable_RESTARTALIGNPC,
                        // Debug: restart to a not-zero-extended AArch32 PC value
                        Unpredictable_RESTARTZEROUPPERPC,
                        // Zero top 32 bits of X registers in AArch32 state
                        Unpredictable_ZEROUPPER,
                        // Zero top 32 bits of PC on illegal return to
                        // AArch32 state
                        Unpredictable_ERETZEROUPPERPC,
                        // Force address to be aligned when interworking
                        // branch to A32 state
                        Unpredictable_A32FORCEALIGNPC,
                        // SMC disabled
                        Unpredictable_SMD,
                        // FF speculation
                        Unpredictable_NONFAULT,
                        // Zero top bits of Z registers in EL change
                        Unpredictable_SVEZEROUPPER,
                        // Load mem data in NF loads
                        Unpredictable_SVELDNFDATA,
                        // Write zeros in NF loads
                        Unpredictable_SVELDNFZERO,
                        // SP alignment fault when predicate is all zero
                        Unpredictable_CHECKSPNONEACTIVE,
                        // Zero top bits of ZA registers in EL change
                        Unpredictable_SMEZEROUPPER,
                        // Watchpoint match of last rounded up memory access in case of
                        // 16 byte rounding
                        Unpredictable_16BYTEROUNDEDUPACCESS,
                        // Watchpoint match of first rounded down memory access in case of
                        // 16 byte rounding
                        Unpredictable_16BYTEROUNDEDDOWNACCESS,
                        // HCR_EL2.[NV,NV1] == '01'
                        Unpredictable_NVNV1,
                        // Upper bits of a BADDR are not RESS
                        Unpredictable_BADDR_RESS,
                        // Reserved shareability encoding
                        Unpredictable_Shareability,
                        // Access Flag Update by HW
                        Unpredictable_AFUPDATE,
                        // Dirty Bit State Update by HW
                        Unpredictable_DBUPDATE,
                        // Unsupported memory attributes for HW update
                        Unpredictable_Unsupported_Atomic_HW_Update,
                        // Consider SCTLR_ELx().IESB in Debug state
                        Unpredictable_IESBinDebug,
                        // Bad settings for PMSFCR_EL1/PMSEVFR_EL1/PMSLATFR_EL1// The
HCR_EL2.TERR / HCR2.TERR and SCR_EL3.TERR / SCR.TERR
                        // trap control can be treated as RES0.
Unpredictable_BADPMSFCR,
                        // Zero saved BType value in SPSR_ELx/DPSR_EL0Unpredictable_MINIMALRASv2,
                        // Bad settings for PMSFCR_EL1/PMSEVFR_EL1/PMSLATFR_EL1
Unpredictable_ZEROBTYPE,
                        // Timestamp constrained to virtual or physicalUnpredictable_BADPMSFCR,
                        // Zero saved BType value in SPSR_ELx/DPSR_EL0
Unpredictable_EL2TIMESTAMP,Unpredictable_ZEROBTYPE,
                        // Timestamp constrained to virtual or physical
Unpredictable_EL1TIMESTAMP,
                         // Reserved MDCR_EL3.[NSTBE,NSTB] or MDCR_EL3.[NSPBE,NSPB] valueUnpredictable_EL2TIMESTAMP,
Unpredictable_RESERVEDNSxB,Unpredictable_EL1TIMESTAMP,
                         // Reserved MDCR_EL3.[NSTBE,NSTB] or MDCR_EL3.[NSPBE,NSPB] value
Unpredictable_RESERVEDNSxB_Trap,
                        // WFET or WFIT instruction in Debug stateUnpredictable_RESERVEDNSxB,
Unpredictable_WFxTDEBUG,
                        // Address does not support LS64 instructionsUnpredictable_RESERVEDNSxB_Trap,
                        // WFET or WFIT instruction in Debug state
Unpredictable_LS64UNSUPPORTED,
                        // Unaligned exclusives, atomics, acquire/release
                        // to a region that is not to Normal inner write-back
                        // outer write-back shareable generate an Alignment fault.Unpredictable_WFxTDEBUG,
                        // Address does not support LS64 instructions
Unpredictable_LSE2_ALIGNMENT_FAULT,
                        // 128-bit Atomic or 128-bit RCW{S} transfer register overlapUnpredictable_LS64UNSUPPORTED,
                        // Unaligned exclusives, atomics, acquire/release
                        // to a region that is not to Normal inner write-back
                        // outer write-back shareable generate an Alignment fault.
Unpredictable_LSE128OVERLAP,
                        // Clearing DCC/ITR sticky flags when instruction is in
flightUnpredictable_LSE2_ALIGNMENT_FAULT,
                        // 128-bit Atomic or 128-bit RCW{S} transfer register overlap
Unpredictable_CLEARERRITEZERO,
                        // ALUEXCEPTIONRETURN when in user/system mode in
                        // A32 instructionsUnpredictable_LSE128OVERLAP,
                        // Clearing DCC/ITR sticky flags when instruction is in flight
Unpredictable_ALUEXCEPTIONRETURN,
                        // Trap to register in debug state are ignoredUnpredictable_CLEARERRITEZERO,
                        // ALUEXCEPTIONRETURN when in user/system mode in
```

```
                            // A32 instructions
Unpredictable_IGNORETRAPINDEBUG,
                            // Compare DBGBVR.RESS for BP/WPUnpredictable_ALUEXCEPTIONRETURN,
                            // Trap to register in debug state are ignored
Unpredictable_DBGxVR_RESS,
                            // Inaccessible event counterUnpredictable_IGNORETRAPINDEBUG,
                            // Compare DBGBVR.RESS for BP/WP
Unpredictable_PMUEVENTCOUNTER,
                            // Reserved PMSCR.PCT behaviorUnpredictable_DBGxVR_RESS,
                            // Inaccessible event counter
Unpredictable_PMSCR_PCT,
                            // MDCR_EL2.HPMN or HDCR.HPMN is larger than PMCR.N or
                            // FEAT_HPMN0 is not implemented and HPMN is 0.Unpredictable_PMUEVENTCOUNTER,
                            // Reserved PMSCR.PCT behavior
Unpredictable_RES_HPMN,
                            // Chained PMU counters idx and idx+1 are not in same rangeUnpredictable_PMSCR_PCT,
                            // MDCR_EL2.HPMN or HDCR.HPMN is larger than PMCR.N or
                            // FEAT_HPMN0 is not implemented and HPMN is 0.
Unpredictable_COUNT_CHAIN,
                            // PMCCR.EPMN is larger than PMCR.NUnpredictable_RES_HPMN,
                            // Chained PMU counters idx and idx+1 are not in same range
Unpredictable_RES_EPMN,
                            // Generate BRB_FILTRATE event on BRB injectionUnpredictable_COUNT_CHAIN,
                            // PMCCR.EPMN is larger than PMCR.N
Unpredictable_BRBFILTRATE,
                            // Generate PMU_SNAPSHOT event in Debug stateUnpredictable_RES_EPMN,
                            // Generate BRB_FILTRATE event on BRB injection
Unpredictable_PMUSNAPSHOTEVENT,
                            // Reserved MDCR_EL3.EPMSSAD valueUnpredictable_BRBFILTRATE,
                            // Generate PMU_SNAPSHOT event in Debug state
Unpredictable_RESEPMSSAD,
                            // Reserved PMECR_EL1.SSE valueUnpredictable_PMUSNAPSHOTEVENT,
                            // Reserved MDCR_EL3.EPMSSAD value
Unpredictable_RESPMSSE,
                            // Enable for PMU Profiling exception and PMUIRQUnpredictable_RESEPMSSAD,
                            // Reserved PMECR_EL1.SSE value
Unpredictable_RESPMEE,
                            // Enables for SPE Profiling exceptions and PMSIRQUnpredictable_RESPMSSE,
                            // Enable for PMU Profiling exception and PMUIRQ
Unpredictable_RESPMSEE,
                            // Enables for TRBE Profiling exceptions and PMSIRQUnpredictable_RESPMEE,
                            // Enables for SPE Profiling exceptions and PMSIRQ
Unpredictable_RESTRFEE,
                            // NonZero TRBPTR_EL1[63:(OAMax)]Unpredictable_RESPMSEE,
                            // Enables for TRBE Profiling exceptions and PMSIRQ
Unpredictable_TRBPTR_Fault_AddressSize,
                            // Operands for CPY*/SET* instructions overlapUnpredictable_RESTRFEE,
                            // NonZero TRBPTR_EL1[63:(OAMax)]
Unpredictable_MOPSOVERLAP,
                            // Operands for CPY*/SET* instructions use 0b11111
                            // as a register specifierUnpredictable_TRBPTR_Fault_AddressSize,
                            // Operands for CPY*/SET* instructions overlap
Unpredictable_MOPS_R31,
                            // PMBPTR_EL1[63:(OAMax)] NonZero causes Address Size faultUnpredictable_MOPSOVERLAP,
                            // Operands for CPY*/SET* instructions use 0b11111
                            // as a register specifier
Unpredictable_PMBPTR_Fault_AddressSize,
                            // Chooses which value to return in a non failed Atomic Compare and
SwapUnpredictable_MOPS_R31,
                            // PMBPTR_EL1[63:(OAMax)] NonZero causes Address Size fault
Unpredictable_CASRETURNOLDVALUE,
                            // Enables write of the newvalue in a failed Atomic Compare and
SwapUnpredictable_PMBPTR_Fault_AddressSize,
                            // Chooses which value to return in a non failed Atomic Compare and Swap
Unpredictable_WRITEFAILEDCAS,
                            // Enables write of the newvalue in a failed Atomic Compare and Swap
                            // or failed RCW{S}Unpredictable_CASRETURNOLDVALUE,
                            // Enables write of the newvalue in a failed Atomic Compare and Swap
Unpredictable_WRITEFAILEDRCWCAS,
                            // Store-only Tag checking on a failed Atomic Compare and SwapUnpredictable_WRITEFAILEDCAS,
                            // Enables write of the newvalue in a failed Atomic Compare and Swap
                            // or failed RCW{S}
Unpredictable_STRONLYTAGCHECKEDCAS,
                            // Store-only Tag checking on a failed RCW(S) checks
                            // or RCW(S) Atomic Compare and SwapUnpredictable_WRITEFAILEDRCWCAS,
                            // Store-only Tag checking on a failed Atomic Compare and Swap
Unpredictable_STRONLYTAGCHECKEDRCWSCAS,
                            // Reserved MDCR_EL3.ETBAD valueUnpredictable_STRONLYTAGCHECKEDCAS,
                            // Store-only Tag checking on a failed RCW(S) checks
                            // or RCW(S) Atomic Compare and Swap
Unpredictable_RES_ETBAD,
                            // Invalid Streaming Mode filter bitsUnpredictable_STRONLYTAGCHECKEDRCWSCAS,
                            // Reserved MDCR_EL3.ETBAD value
Unpredictable_RES_PMU_VS,
                            // Apply Checked Pointer Arithmetic on a sequential access to bytes
                            // that cross the 0xXXFF_FFFF_FFFF_FFFF boundary.Unpredictable_RES_ETBAD,
                            // Invalid Streaming Mode filter bits
Unpredictable_CPACHECK,
                            // Reserved PMEVTYPER_EL0(n).[TC,TE,TLC] valuesUnpredictable_RES_PMU_VS,
```

```
                          // Apply Checked Pointer Arithmetic on a sequential access to bytes
                          // that cross the 0xXXFF_FFFF_FFFF_FFFF boundary.
Unpredictable_RESTC,
                          // When FEAT_MTE is implemented, if Memory access mode is enabled
                          // and PSTATE.TCO is 0,  Reads and writes to the external debug
                          // interface DTR registers are CONSTRAINED UNPREDICTABLE for tagcheckUnpredictable_CPACHECK,
                          // Reserved PMEVTYPER_EL0(n).[TC,TE,TLC] values
Unpredictable_NODTRTAGCHK,
                          // Use the default PMG when the default PARTID is generated
                          // due to MPAM errorUnpredictable_RESTC,
                          // When FEAT_MTE is implemented, if Memory access mode is enabled
                          // and PSTATE.TCO is 0,  Reads and writes to the external debug
                          // interface DTR registers are CONSTRAINED UNPREDICTABLE for tagcheck
Unpredictable_USE_DEFAULT_PMG,
                          // MPAM PARTID/PMG exceeding PARTID_MAX/PMG_MAXUnpredictable_NODTRTAGCHK,
                          // Use the default PMG when the default PARTID is generated
                          // due to MPAM error
Unpredictable_OUT_OF_RANGE_ID,
                          // MPAM vPARTID exceeding (VPMR_MAX << 2) + 3Unpredictable_USE_DEFAULT_PMG,
                          // MPAM PARTID/PMG exceeding PARTID_MAX/PMG_MAX
Unpredictable_OUT_OF_RANGE_VID,
                          // If the atomic instructions are not atomic in regard to other
                          // agents that access memory, then the instruction can have one or
                          // more of the following effectsUnpredictable_OUT_OF_RANGE_ID,
                          // MPAM vPARTID exceeding (VPMR_MAX << 2) + 3
Unpredictable_Atomic_SYNC_ABORT,Unpredictable_OUT_OF_RANGE_VID,
                          // If the atomic instructions are not atomic in regard to other
                          // agents that access memory, then the instruction can have one or
                          // more of the following effects
Unpredictable_Atomic_SERROR,Unpredictable_Atomic_SYNC_ABORT,
Unpredictable_Atomic_NOP,Unpredictable_Atomic_SERROR,
Unpredictable_Atomic_MMU_IMPDEF_FAULT,
                          // Truncate virtual identifier to bottom 12 bits if it exceeds 2^12Unpredictable_Atomic_NOP,
Unpredictable_MPAM_TRUNCATE_VID,
                          // Accessing DBGDSCRint via MRC in debug stateUnpredictable_Atomic_MMU_IMPDEF_FAULT,
                          // Truncate virtual identifier to bottom 12 bits if it exceeds 2^12
Unpredictable_MRC_APSR_TARGET,
                          // Accessing Banked register not accessible from the PE modeUnpredictable_MPAM_TRUNCATE_VID,
                          // Accessing DBGDSCRint via MRC in debug state
Unpredictable_BankedRegister,
                          // Accessing unimplemented Banked registerUnpredictable_MRC_APSR_TARGET,
                          // Accessing Banked register not accessible from the PE mode
Unpredictable_UnimplementedRegister,
                          // Reserved ERRACR.{RLRA,SRA,NSRA} valuesUnpredictable_BankedRegister,
                          // Accessing unimplemented Banked register
Unpredictable_RES_RA,
                          // Linked target does not support linkingUnpredictable_UnimplementedRegister,
                          // Reserved ERRACR.{RLRA,SRA,NSRA} values
Unpredictable_BPNOTLINKING
};Unpredictable_RES_RA,
                          // Linked target does not support linkingUnpredictable_BPNOTLINKING
};
```

Library pseudocode for shared/trace/Common/DebugWriteFault

```
// DebugWriteFault()
// =================
// Populate the syndrome register for a fault caused by a call of DebugMemWrite().

func DebugWriteFault(vaddress : bits(64), fault : FaultRecord)
begin
    var ec : bits(6);
    if (IsFeatureImplemented(FEAT_RME) && fault.gpcf.gpf != GPCF_None &&
          fault.gpcf.gpf != GPCF_Fail) then
        ec = '011110';
    else
        ec = if fault.secondstage then '100101' else '100100';
    end;

    var mss2 : bits(24) = Zeros{};
    var mss : bits(16) = Zeros{};

    if fault.gpcf.gpf IN {    if fault.statuscode ==Fault_Permission then
        mss2[8] = if fault.toplevel then '1' else '0';       // TopLevel bit
        mss2[7] = if fault.assuredonly then '1' else '0';    // AssuredOnly bit
        mss2[6] = if fault.overlay then '1' else '0';        // Overlay bit
        mss2[5] = if fault.dirtybit then '1' else '0';       // DirtyBit
    end;


    var mss : bits(16) = Zeros{};
    if !(IsFeatureImplemented(FEAT_RME) && fault.gpcf.gpf != GPCF_None,&&
          fault.gpcf.gpf != GPCF_Fail} then
        mss2[11] = if fault.hdbssf then '1' else '0';             // HDBSSF
        mss2[8]  = if fault.toplevel then '1' else '0';           // TopLevel bit
        mss2[7]  = if fault.assuredonly then '1' else '0';        // AssuredOnly bit
        mss2[6]  = if fault.overlay then '1' else '0';            // Overlay bit
        mss2[5]  = if fault.dirtybit then '1' else '0';           // DirtyBit

) then
        mss[5:0] = EncodeLDFSC(fault.statuscode, fault.level);  // FSC bits
        if fault.s2fs1walk then
            if (IsFeatureImplemented(FEAT_SPE_EXC) &&
                fault.accessdesc.acctype == AccessType_SPE) then
                mss[7] = '1';                                    // S1PTW bit
            end;
            if (IsFeatureImplemented(FEAT_TRBE_EXC) &&
                fault.accessdesc.acctype == AccessType_TRBE) then
                mss[7] = '1';                                    // S1PTW bit
            end;
        end;
(fault.statuscode, fault.level);  // FSC bits
    end;

    // Generate a buffer management event, modifying the existing syndrome.
    var target_el : bits(2);
    var syndrome : bits(64);
    case fault.accessdesc.acctype of
        when AccessType_SPE =>
            target_el = ReportSPEEvent(ec, mss[5:0]);
            syndrome  = PMBSR_EL(target_el);
        when AccessType_TRBE =>
            target_el = ReportTRBEEvent(ec, mss[5:0]);
            syndrome  = TRBSR_EL(target_el);
        otherwise =>
            unreachable;
    end;

    // The following values are not modified if a previous buffer management event
    // has not been handled. Note that in this simple sequential model, this test
    // will never fail.
    if syndrome[17] == '0' then          // Check previous 'S' bit.
        syndrome[55:32] = mss2;          // MSS2 bits
        syndrome[31:26] = ec;            // EC bits
        if fault.accessdesc.acctype == AccessType_TRBE then
            syndrome[22] = '1';          // IRQ bit (TRBE only)
        end;
        syndrome[17] = '1';              // S bit
        syndrome[15:0] = mss;            // MSS bits
    end;

    // For SPE, PMBPTR_EL1 points to the address that generated the fault, and writing
    // to memory never started. Therefore, there isno data loss and DL is unchanged.

    case fault.accessdesc.acctype of
        when AccessType_SPE =>
            PMBSR_EL(target_el) = syndrome;

        when AccessType_TRBE =>
            TRBSR_EL(target_el) = syndrome;

        otherwise =>
            unreachable;
    end;
```

```
    return;
end;
```

Library pseudocode for shared/trace/claim

```
var TRCClaimTags : bits(32);
```

Library pseudocode for shared/trace/claim/Write_TRCCLAIMCLR

```
// Write_TRCCLAIMCLR()
// ===================
// System register write to TRCCLAIMCLR.

func Write_TRCCLAIMCLR(value_in : bits(64))
begin
    assert NUM_TRC_CLAIM_TAGS <= 32;

    TRCClaimTags = TRCClaimTags AND NOT ZeroExtend{32}(value_in[0 +: NUM_TRC_CLAIM_TAGS]);
end;
```

Library pseudocode for shared/trace/claim/Write_TRCCLAIMSET

```
// Write_TRCCLAIMSET()
// ===================
// System register write to TRCCLAIMSET.

func Write_TRCCLAIMSET(value_in : bits(64))
begin
    assert NUM_TRC_CLAIM_TAGS <= 32;

    TRCClaimTags = TRCClaimTags OR ZeroExtend{32}(value_in[0 +: NUM_TRC_CLAIM_TAGS]);
end;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

(old) htmldiff from- (new)